

目录

第 1 章 绪论	8
1.1 什么是编译	8
1.2 编译系统的结构	10
1.2.1 词法分析概述	14
1.2.2 语法分析概述	16
1.2.3 语义分析概述	17
1.2.4 中间代码生成概述	20
1.2.5 目标代码生成和代码优化概述	25
1.3 编译程序的生成	25
1.4 编译技术的应用	30
1.5 本章小结	32
第 2 章 程序设计语言及其文法	34
2.1 语言概述	34
2.2 基本概念	34
2.2.1 串	34
2.2.2 字母表	35
2.3 文法的定义	36
2.4 语言的定义	40
2.5 文法的分类	42
2.6 CFG 文法的分析树	44
2.7 CFG 的二义性	48
2.8 本章小结	50
第 3 章 词法分析	52
3.1 词法分析器的任务	52
3.2 单词的描述	52
3.2.1 正则文法	53
3.2.2 正则表达式	53
3.2.3 正则定义	55
3.3 单词的识别	56
3.3.1 有穷自动机	56

3.3.2 从正则表达式到有穷自动机.....	62
3.4 词法分析阶段的错误处理.....	69
3.5 词法分析器的自动生成.....	70
3.5.1 Lex 的使用.....	71
3.5.2 Lex 程序的结构.....	72
3.5.3 Lex 中的冲突解决.....	76
3.6 本章小结.....	76
第 4 章 语法分析	78
4.1 自顶向下的语法分析概述.....	79
4.1.1 自顶向下分析的工作方式.....	79
4.1.2 文法的改造.....	84
4.2 预测分析法.....	89
4.2.1 LL(1)文法.....	89
4.2.2 递归的预测分析法.....	100
4.2.3 非递归的预测分析法.....	105
4.2.4 预测分析中的错误恢复.....	111
4.3 自底向上的语法分析概述.....	114
4.3.1 自底向上分析的工作方式.....	114
4.3.2 移入-归约分析中的冲突.....	115
4.4 LR 分析法.....	123
4.4.1 LR(0)分析法.....	131
4.4.2 SLR(1)分析法.....	144
4.4.3 LR(1)分析法.....	147
4.4.4 LALR(1)分析法.....	156
4.4.5 二义性文法的 LR 分析.....	162
4.4.6 LR 分析中的错误处理.....	165
4.5 语法分析器自动生成工具.....	168
4.5.1 Yacc 的使用.....	168
4.5.2 Yacc 程序的结构.....	169
4.6 本章小结.....	173
第 5 章 语法制导翻译	174
5.1 语法制导定义 (SDD).....	175
5.1.1 非终结符的语义属性.....	176

5.1.2 终结符的语义属性	180
5.1.3 属性文法	181
5.1.4 SDD 的求值顺序	182
5.2 S-属性定义与 L-属性定义	185
5.2.1 S-属性定义	185
5.2.2 L-属性定义	185
5.3 语法制导翻译方案 SDT	191
5.3.1 S-属性定义的 SDT	192
5.3.2 S-属性的定义的自底向上翻译	193
5.3.3 L-属性定义的 SDT	199
5.4 L-属性定义的自顶向下翻译	200
5.4.1 在 LL(1)语法分析过程中进行语义翻译	204
5.4.2 在递归的预测分析过程中进行语义翻译	215
5.5 L-属性定义的自底向上翻译	219
5.6 本章小结	227
第 6 章 中间代码生成	229
6.1 声明语句的翻译	229
6.1.1 类型表达式	230
6.1.2 变量声明语句的翻译	230
6.2 赋值语句的翻译	238
6.2.1 赋值语句的 SDD	238
6.2.2 增量翻译	240
6.2.3 数组引用的翻译	246
6.3 控制语句的翻译	259
6.3.1 控制流语句的翻译	259
6.3.2 布尔表达式的翻译	266
6.3.3 避免生成冗余的 goto 指令	276
6.4 回填	283
6.4.1 布尔表达式的回填	284
6.4.2 控制流语句的回填	292
6.5 switch 语句的翻译	305
6.6 过程调用语句的翻译	308
6.7 本章小结	310

第 7 章 运行存储分配	311
7.1 存储组织	311
7.2 静态分配策略	313
7.2.1 静态存储分配	313
7.2.2 常用的静态存储分配方法	314
7.3 栈式分配策略	317
7.3.1 活动树	317
7.3.2 活动记录	319
7.3.3 调用序列和返回序列	323
7.3.4 栈中的变长数据	325
7.4 非局部数据的访问	326
7.4.1 静态作用域与动态作用域	327
7.4.2 无嵌套过程定义中的数据访问	329
7.4.3 有过程嵌套定义中的数据访问	330
7.4.4 访问链	330
7.4.5 嵌套层次显示表	333
7.5 堆式分配策略	336
7.5.1 堆式分配的主要任务	336
7.5.2 显式堆式分配方法	336
7.5.3 垃圾回收	338
7.6 符号表	339
7.6.1 符号表的信息	339
7.6.2 符号表的作用	340
7.6.3 单作用域符号表的组织	342
7.6.4 嵌套作用域符号表的组织	343
7.6.5 嵌套作用域符号表的建立	345
7.7 本章小节	349
第 8 章 代码优化	350
8.1 流图	351
8.1.1 流图的基本概念	351
8.1.2 流图的构建	351
8.2 优化的分类	354
8.2.1 删除公共子表达式	355

8.2.2 基本块中的强度削弱.....	357
8.2.3 删除无用代码.....	359
8.2.4 循环不变表达式外提.....	360
8.2.5 循环中的强度削弱.....	362
8.2.6 删除归纳变量.....	363
8.3 基本块的局部优化.....	364
8.3.1 基本块的 DAG 表示.....	365
8.3.2 基本块 DAG 的优化.....	367
8.3.3 从 DAG 到基本块的重组.....	369
8.4 数据流分析.....	371
8.4.1 数据流基本概念.....	371
8.4.2 数据流分析模式.....	373
8.4.3 到达-定值数据流方程.....	376
8.4.4 活跃变量数据流方程.....	382
8.4.5 可用表达式数据流方程.....	385
8.5 控制流分析.....	389
8.5.1 支配节点.....	390
8.5.2 回边.....	392
8.5.3 自然循环 (natural loop)	392
8.6 全局优化.....	394
8.6.1 全局公共子表达式消除.....	395
8.6.2 复制语句的删除.....	396
8.6.3 代码外提.....	397
8.6.4 强度削弱.....	401
8.6.5 归纳变量的删除.....	404
8.7 本章小节.....	406
第 9 章 目标代码生成.....	407
9.1 代码生成器的主要任务.....	407
9.1.1 指令选择.....	407
9.1.2 寄存器分配.....	409
9.1.3 指令排序.....	410
9.2 一个简单的目标机模型.....	410
9.3 指令选择.....	414

9.3.1 简单三地址语句的代码框架.....	414
9.3.2 过程调用和返回的目标代码.....	416
9.4 寄存器选择.....	421
9.4.1 基本块代码生成算法.....	421
9.4.2 寄存器描述符和地址描述符.....	423
9.4.3 描述符的维护.....	424
9.4.4 寄存器选择函数.....	426
9.5 窥孔优化.....	429
9.5.1 窥孔与窥孔优化.....	429
9.5.2 典型的窥孔优化技术.....	430
9.5.3 窥孔优化的实现方式.....	432
9.6 本章小节.....	433
参考文献.....	434

第1章 绪论

如果说高级语言是人类意图的优雅表达，那么编译器便是将其转化为机器指令的“翻译官”与“优化大师”。它不仅是程序得以运行的基石，其设计思想更是计算机科学中自动化、抽象化与工程化的典范体现。本章将系统性地揭示编译的全貌。首先阐明编译的本质，继而剖析一个现代编译器的经典结构与协同流程，探讨编译器本身的生成方法，并领略编译技术超越传统翻译之外的广阔应用天地。理解这一切，是后续深入词法、语法、语义乃至代码优化等精妙世界的必备蓝图。

1.1 什么是编译

提到编译，我们要从“计算机程序设计语言”讲起。计算机程序设计语言可以分为 3 个层次：机器语言（machine language）、汇编语言（assembly language）和高级语言（high level language）。

机器语言是可以被计算机直接理解的语言。由于计算机只认识二进制数 0 和 1，因此机器语言编写的程序都是由数字 0 和 1 组成的序列。例如：C706 0000 0002 表示在 IBM PC 上使用的 Intel 8x86 处理器将数字 2 移至地址 0000 的指令。这条指令是采用 16 进制形式书写的，其中，C706 是操作码，表示加载操作，0000 和 0002 是两个操作数。

从这个例子我们可以看出，机器语言的表达习惯与我们人类的表达习惯相去甚远（人类习惯于使用十进制数，而机器语言采用二进制以及十六进制），而且，程序员还要记住每一个操作码代表什么操作，这些特点都导致用机器语言编写和阅读程序都十分不方便，而且在编写过程中容易出错。于是很快出现了汇编语言。

在汇编语言中，引入了助记符，因此比较直观。例如，汇编语言指令“MOV X, 2”就与前面的机器指令等价（假设符号存储地址 X 是 0000）。其中 MOV 是助记符，它来自英文单词 move，表示移入、加载操作。虽然有了一定进步，但是汇编语言依然依赖于特定的机器，程序员需要了解机器，因此对于非计算机专业人员来说，使用上很受限制。而且，汇编语言的编写效率较低，即使一个比较简单的数学表达式也需要好几条指令。于是接下来出现了高

级语言。

高级语言以一种更类似于数学定义或自然语言的简洁形式来编写程序，而且与任何机器都无关。例如，高级语言语句 $x=2$ 表示与上面两条指令等价的功能，但是更接近人类的表达习惯，而且不管多长的表达式，只要一条语句就可以简洁地表达。

用高级语言和汇编语言编写的程序，最终都要“翻译”成 0、1 构成的机器代码方可在计算机上执行。将汇编语言翻译成机器语言的过程称为汇编，对应的软件程序称为汇编器（**assembler**）。将高级语言翻译成汇编语言或直接翻译成机器语言的过程称为编译，对应的软件程序称为编译器（**compiler**）。可以看出，编译的本质是一个翻译的过程，特指将高级语言翻译成汇编语言或机器语言的过程。这里，被翻译的语言（高级语言）称为源语言（**source language**），翻译成的语言（汇编语言或机器语言）称为目标语言（**object language**）。

如图 1-1 所示，为了建立可执行的目标程序，除了编译器以外，还需要一些其它程序。



图 1-1 编译器在语言处理系统中的位置

预处理器 (preprocessor)：源程序可能被分割成模块存储在不同的文件中，预处理器把存储在不同文件中的源程序聚合在一起，还负责把被称为宏的缩写语句扩展为原始语句。

加载器 (loader)：经过预处理的源程序在经过编译器和汇编器的处理后，生成可重定位的机器代码。所谓“可重定位”，是指汇编器生成的机器代码在内存中存放的起始位置 L 不是固定的。代码中的所有地址都是相对于这个起始地址的相对地址，起始地址加上相对地址得到的才是内存中的绝对地址。因此，将可重定位的机器代码加载到内存时需要修改可重定位地址，并且将修改后的

指令和数据放到内存中适当的位置。

链接器（linker）：大型程序经常被分成多个部分进行编译，因此，可重定位的机器代码可能要和其他可重定位的目标文件及库文件链接到一起，形成可执行代码。这一工作由链接器来完成。另外，链接器还负责解决外部内存地址问题。所谓外部内存地址是指一个文件中的代码可能使用另一个文件中的数据或过程，这些数据或过程地址对于这个程序来说就是外部地址。

1.2 编译系统的结构

前文提到，编译的本质是一个翻译的过程。编译器的输入是高级语言程序，例如一个 C 语言程序，输出是汇编语言程序或机器语言程序。那么，机器是如何将 C 语言程序自动翻译成汇编语言程序的呢？这里，我们可以参考一下人工翻译的过程。

假如说，我们要将一个英文句子 “In the room, he broke a window with a hammer” 翻译成汉语。这里，英语就是源语言，汉语是目标语言。那么我们是如何进行翻译的呢？

我们说，翻译的过程大体上可以分为两步：首先，从源语言（即英语）方面来分析理解这个句子要表达的含义（即句子的语义）。接下来，根据这个语义，再用目标语言（即汉语）的方式说一遍，即完成了翻译的过程。其中通过分析源语言来获得句子语义的这一过程称为语义分析（semantic analysis）。

那么我们是如何分析理解一个句子的含义（语义）的呢？通常是从划分句子成分入手。首先要抓住核心谓语动词（“break”）。因为谓语动词知道了，句子的一半意思就知道了。例如，对于上面这个英语句子，我们很容易找到它的核心谓语动词是“break”，表示“打”的意思。提到“打”这个动作，我们就想知道谁实施了“打”这个动作，谁是被打的对象，用什么打的？为什么打？打的结果怎么样等等。这些都可以通过分析“break”的上下文来获得。由于“broke”在这里是主动语态，所以主语“he”是“打”这个动作的施事者。宾语“window”是“打”这个动作的受事者。反过来，如果谓语动词使用的是被动语态“be broken”，那么主语就是动作的受事者。“with a hammer”是补语，表示完成这个动作所使用的工具。“in the room”是状语，表示动作发生的地点。由此，可以分析出“break”前后出现的这些名词性成分跟“break”之间的语义关系，并且用图的形式来表示，如图 1-2 所示。

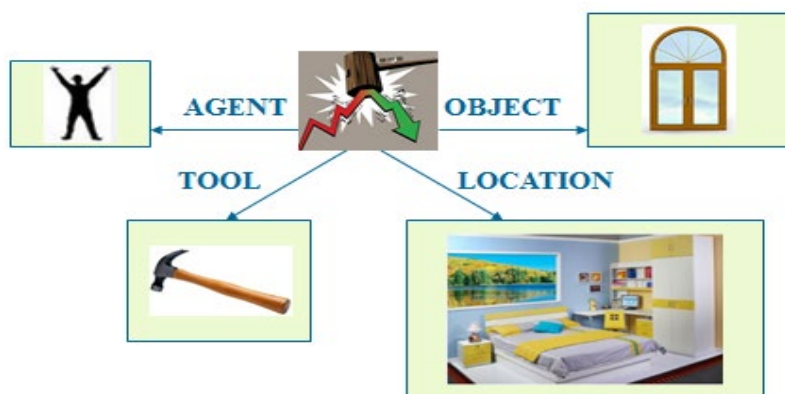


图 1-2 英文句子 “In the room, he broke a window with a hammer” 语义的表示

图中央的核心结点表示句子描述了“打”这样一个事件，其它四个节点对应句子中提到的 4 个实体，分别是“he”、“window”、“hammer”和“room”。从核心节点到其它四个节点分别引出四条边，这四条有向边上的信息表示了这四个实体与核心谓语动词之间的语义关系。其中“he”是动作的施事者（AGENT），“window”是动作的受事者（OBJECT），“hammer”是完成这个动作所使用的工具（TOOL），“room”是动作发生的地点（LOCATION）。这就是这个句子的语义。根据这个图所表达的意思，我们再用汉语的方式说一遍（也就是“在房间里，他用锤子砸了一扇窗户”），这样就完成了翻译的过程。该图是一种中间表示，独立于具体的语言，也就是说英语中可以用它来表示，汉语中也可以用它来表示，日语、法语、德语、意大利语等等都可以用它来表示。有了这张图，不管目标语言是什么，都可以进行翻译。因此，中间表示是非常重要的，它起到一个桥梁的作用。

由此可见，对于一个复杂的句子，要想分析其句子的含义，首先要划分句子成分，即识别出句子中的主、谓、宾、定、状、补等成分。那么如何识别这些成分？我们知道，谓语通常是由动词短语构成的，主语和宾语通常是由名词短语构成的，补语和状语通常是由介词短语构成的。因此，要想识别句子成分，关键是识别出句子中的各类短语，这一过程称为语法分析（syntax analysis）。

那么，我们又是如何识别各类短语的呢？显然可以通过词性。例如，冠词加上名词构成一个名词短语（如“the room”），一个代词本身也可以构成一个名词短语（如“he”），介词加上名词构成一个介词短语（如“in time”）。因此，要想识别各类短语，关键是确定各个单词的词性（或者说是词类），这一过程称为词法分析（lexical analysis）。

例如，对于英语句子 “In the room, he broke a window with a hammer.”，通过词法分析，我们可以得到与这个句子对应的词性序列，如图1-3所示。

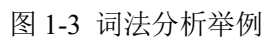
[illegible]

图 1-4 语法分析举例

- XII -



- XIII -

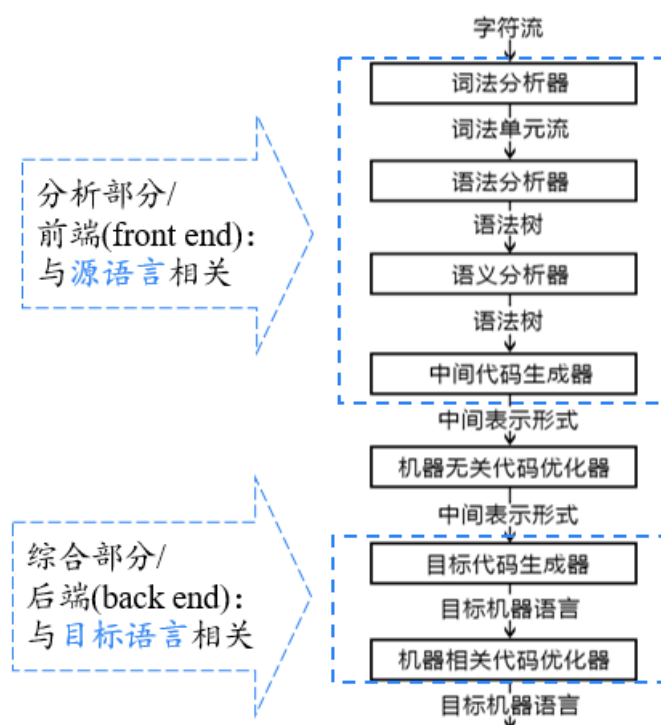


图 1-6 编译器的结构

值得注意的是，这里提到的阶段是编译器的逻辑组织方式。在实现过程中，多个阶段可能被组合在一起。例如，语义分析的结果通常直接表示成中间代码的形式，因此，语义分析与中间代码生成两个阶段通常是放在一起实现的。另外，可以在语法分析分析句子结构的同时结合语义规则直接进行语义分析，这一技术称为语法制导翻译。在这种情况下，语法分析、语义分析和中间代码生成三个阶段可以放在一起实现。

1.2.1 词法分析概述

词法分析器也叫扫描器（scanner）。我们在前面已经提到过，词法分析的主要任务就是确定单词的类型。具体来说就是从左向右逐行扫描源程序的字符，识别出各个单词，确定单词的类型。将识别出的单词转换成统一的机内表示——词法单元(token)形式：

<种别码, 属性值>

其中，种别码用来表示单词的种别。说到单词的种别，我们知道，自然语言句子中的每个单词都有一个词性（或者叫词类）。那么程序设计语言中的单

词又分为哪些种类呢？我们说，大体上可以分为 5 类：

(1) 关键字，如 “program”、“if”、“else”、“then” 等等。给定一个程序设计语言，其关键字集合是可以事先确定的，因此，我们给每一个关键字分配一个种别码，也就是一词一码。

(2) 标识符。标识符是程序员给数据或过程起的一些名字。由于这是一个开放集合，我们事先不能枚举所有的标识符，因此，我们将所有的标识符统一作为一种单词，为它们分配同一个种别码，也就是多词一码。为了区分不同的标识符，我们用 `token` 的第二个分量（属性值）来存放不同标识符的具体字面值。

(3) 常量，包括整型常量、浮点型常量、字符型常量、布尔型常量等等。常量跟标识符一样，也是一个开放集合，但是，不同类型常量的构成方式是不同的。因此，我们为每种类型的常量分配一个种别码，也就是一型一码。为了区分同一类型中的不同的常量，我们也是用 `token` 的第二个分量（属性值）来存放每一个常量对应的数值。

(4) 运算符，包括算术运算符（如 “+”、“-”、“*”、“/”、“++”、“—”）、关系运算符（如 “>”、“<”、“==”、“!=”、“>=”、“<=”）和逻辑运算符（如 “&”、“|”、“~”）。

(5) 界限符，如 “(”、“)”、“=”、“{”、“}” 等。运算符和界限符与关键字类似，是事先可以确定下来的，因此，我们给每一个运算符和界限符分别分配一个种别码，也就是一词一码。当然，我们也可以为每一类运算符分配一个种别码。为了区分同一类型中的不同运算符，我们也是用 `token` 的第二个分量（属性值）来存放具体的运算符。

根据以上的种别码分配原则，我们来看一个词法分析结果的例子。假如说输入的程序片段为 “while(value!=100){num++;}”，那么词法分析后得到的 `token` 序列如图 1-7 所示。

序号	单词	token		
1	while	< WHILE	, -	>
2	(	< SLP	, -	>
3	value	< IDN	, value	>
4	!=	< NE	, -	>
5	100	< CONST	, 100	>
6	)	< SRP	, -	>
7	{	< LP	, -	>
8	num	< IDN	, num	>
9	++	< INC	, -	>

10	;	<	SEMI	,	-	>
11	}	<	RP	,	-	>

图 1-7 “while(value!=100){num++;}”的词法分析结果

图 1-7 中的每一个 token 对应一个单词。种别码本身应该是一个整数。这里为了直观起见，采用了宏定义的形式。比如说用 WHILE 来表示关键字 while 的种别码，用 SLP 来表示 “(” 的种别码。

该程序片段中的第一个单词为 “while”。它是一个关键字，因此采用一词一码的编码方式。也就是说，根据它的种别码就可以将它与其它单词区分开。因此，其 token 的第二个分量（属性值）是空的。同理，“(”、“!=”、“)”、“{”、“++”、“;”和“}”都是一词一码的单词，因此，它们的 token 的第二个分量（属性值）都是空的。“value”和“num”是两个标识符，因此，它们的种别码都是“IDN”，表示标识符（identifier）。同时用 token 的第二个分量（属性值）来存放这两个标识符的字面值（分别是“value”和“num”），用来对不同的标识符进行区分。“100”是一个常量。其 token 的第二个分量（属性值）就是 100 本身。

那么词法分析器是如何将输入的字符序列自动地转换成 token 序列的呢？我们将在第 3 章（词法分析）中详细介绍。

1.2.2 语法分析概述

前面我们已经讲过，语法分析器的主要任务是从词法分析器输出的 token 序列中识别出各类短语，并构造语法分析树（parse tree）。语法分析树描述了句子的语法结构。

先来看一个例子——赋值语句的分析树。假如输入的程序片断是一条赋值语句 “position = initial + rate * 60 ;”，经过词法分析后得到一个 token 序列：

< id , position > < = > < id , initial > < + > < id , rate > < * > < num , 60 > < ; >

这里 “position”、“initial”和“rate”都是标识符，“60”是常数。该程序片断对应的分析树如图 1-8 所示。

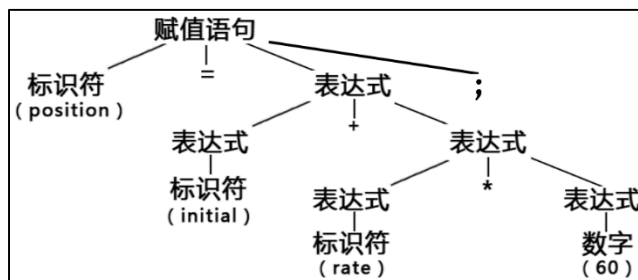


图 1-8 程序片断 “`position = initial + rate * 60 ;`” 对应的分析树

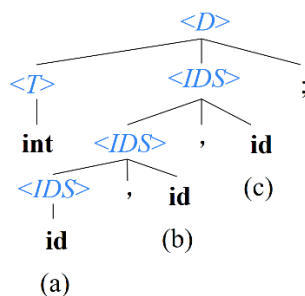
从图 1-8 所示的分析树可以看出，标识符和常数本身可以构成一个表达式。一个表达式加上另一个表达式或者乘以另一个表达式可以构成一个更大的表达式。一个标识符连接上一个赋值号再连接上一个表达式和一个分号可以构成一条赋值语句。

再来看一个变量声明语句的分析树的例子。假设有如下变量声明语句的文法：

$$\begin{aligned} \langle D \rangle &\rightarrow \langle T \rangle \langle IDS \rangle ; \\ \langle T \rangle &\rightarrow \text{int} \mid \text{real} \mid \text{char} \mid \text{bool} \\ \langle IDS \rangle &\rightarrow \text{id} \mid \langle IDS \rangle, \text{id} \end{aligned} \quad (1.1)$$

文法是由一系列规则构成的。该文法中， $\langle D \rangle$ 表示声明语句（declaration）， $\langle T \rangle$ 表示类型（type）， $\langle IDS \rangle$ 表示标识符序列（ID sequence）。第一条规则表示：一条声明语句 $\langle D \rangle$ 是由类型 $\langle T \rangle$ 连接上一个标识符序列 $\langle IDS \rangle$ 和一个分号构成的。根据第二条规则，这里的 $\langle T \rangle$ 可以是 **int**、**real**、**char** 或 **bool**。竖线表示“或”的关系。根据第三条规则，一个 **id**（标识符）本身可以构成一个标识符序列 $\langle IDS \rangle$ ，一个 $\langle IDS \rangle$ 连接上一个逗号和一个标识符 **id** 可以构成一个新的 $\langle IDS \rangle$ ，这是一个递归定义的规则。

根据以上规则，假如输入一条声明语句 “`int a, b, c ;`”，可以得到如图 1-9 所示的分析树。

图 1-9 程序片断 “`int a, b, c ;`” 对应的分析树

那么，语法分析器是如何自动地根据语法规则为输入句子构造分析树的呢？我们将在第 4 章（语法分析）中详细介绍。

1.2.3 语义分析概述

高级语言程序中的语句分为两大类：一类是声明语句，另一类是可执行语

句。其中的声明语句用来声明一些数据对象或过程，并为其分别起一个名字，即标识符。对于声明语句，语义分析的主要任务就是收集标识符的属性信息。那么标识符都有哪些属性呢？主要包括以下几个方面：

(1) 种属（kind）。也就是说，该标识符对应的是一个简单变量、复合变量（比如说数组、记录等等），还是过程。

(2) 类型（type）。也就是说，标识符对应的变量或者过程是整型、实型、还是字符型、布尔型、指针型等等。

(3) 存储位置、长度。因为每一个数据对象和过程在内存中都要为它分配一块存储空间，因此存储位置和占用空间的大小（即长度）就是标识符的一个很重要的属性。例如输入如下这样一个程序片断：

```
begin
    real    x[8];
    integer i, j;
    ...
end
```

(1.2)

其中声明了一些数据对象。这些数据对象的存储位置（相对地址）如图 1-10 所示。对于声明的第一个数据，实型数组 x ，它的相对地址为 0。假设一个实型变量的长度为 8 个字节，那么由 8 个元素构成的实型数组一共占用 64 个字节的空間。因此，接下来声明的整型变量 i 的相对地址就是 64。假设一个整型变量的长度为 4 个字节，那么接下来声明的整型变量 j 的相对地址就是 68。以此类推。

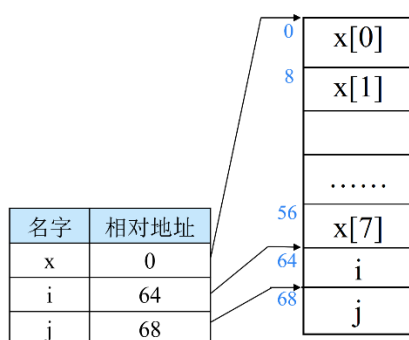


图 1-10 标识符的位置属性举例

(4) (变量的) 值。

(5) 作用域（scope）。程序中的每一个对象都有它的作用范围，即作用域。

(6) 参数和返回值信息。对于过程的名字，标识符的属性还包括参数和返

回值信息，如参数个数、参数类型、参数传递方式、返回值类型等等。

语义分析阶段获取的这些标识符属性信息被存放在一个称为符号表的数据结构中，如图 1-11 所示。每个标识符在符号表中都对应一条记录，记录的每个字段对应于该标识符的一个属性，比如说类型（TYPE）、种属（KIND）、值（VAL）、地址（ADDR）等等。其中，名字（NAME）一栏（对应于标识符 token 的第二个分量）采用的最简单的数据结构就是定长字符数组。很显然，采用定长字符数组来存储名字存在着明显的空间浪费。假设某语言规定标识符的最大长度为 128 个字符。这样的话，如果采用定长字符数组的方法，字符数组的长度就要设为 128 个字符。而事实上，大多数情况下，程序员不会起这么长的名字。因此，每个名字会浪费很多空间。



图 1-11 典型的符号表结构

一种解决上述问题的方法是：构造一个字符串表，专门用于存放程序中使用的标识符和字符常数。这样，名字（NAME）一栏就包括两个部分，一个用于存放该标识符在字符串表中的起始位置，另一个存放该标识符的字符串长度。例如，在这个例子中，第 1 个标识符“SIMPLE”的长度为 6 个字符，第 2 个标识符“SYMBLE”的长度也是 6 个字符，第 3 个标识符“TABLE”的长度为 5 个字符。

语义分析的另一个重要任务就是语义检查。常见的语义错误包括以下一些方面：

- (1) 变量或过程未经声明就使用。这类错误通过查询符号表就可以很容易检测出来。如果在符号表里没有查到相关的标识符，就表明其未经声明。
- (2) 变量或过程名重复声明。这类错误通过查询符号表也可以很容易检测出来。如果该标识符在符号表里已经登记，就表明其重复声明。

(3) 操作数类型不匹配。参与运算的两个操作数，原则上类型是应该是一致的。比如说，将一个浮点型的操作数去跟一个字符型或者布尔型的操作数做加法运算是有问题的。另外，当发现操作数的类型不一致时，可能还要进行自动类型转换：比如，一个二元算术运算符可以应用于一对整数或者一对浮点数。如果这个运算符应用于一个浮点数和一个整数，那么编译器可以把该整数（自动类型）转换成为一个浮点数。

(4) 操作符与操作数之间的类型不匹配。例如：赋值号左边出现一个只有右值的表达式（例如“2=i”），对非数组变量使用数组访问操作符“[···]”，数组访问操作符“[···]”中出现非整数（例如 a[1.5]），对非结构体类型变量使用“.”操作符，对非过程名使用过程调用操作符“(···)”，过程调用时实参与形参的数目或类型不匹配，return 语句的返回类型与函数定义的返回类型不匹配等。

1.2.4 中间代码生成概述

源程序的中间表示有多种形式，例如三地址码（three-address code）、语法结构树（syntax tree）等等。这里需要注意，语法结构树简称语法树，与前面提到的语法分析树（parse tree）不是一回事。我们将会在第 8 章中介绍语法树的变体——无环有向图（directed acyclic graph, DAG）。这里只介绍三地址码。

三地址码由类似于汇编语言的指令序列组成，每个指令最多有三个操作数（operand），因此称为三地址码。图 1-12 列出了一些常用的三地址指令。

序号	指令类型	指令形式
1	赋值指令	$x = y \text{ op } z$ $x = \text{op } y$
2	复制指令	$x = y$
3	条件跳转	if $x \text{ relop } y$ goto L
4	非条件跳转	goto L
5	参数传递	param x

6	过程调用 函数调用	$\text{call } p, n$ $y = \text{call } p, n$
7	过程返回	$\text{return } x$
8	数组引用	$x = y[i]$
9	数组赋值	$x[i] = y$
10	地址及指针操作	$x = \& y$ $x = * y$ $*x = y$

图 1-12 常用的三地址指令

(1) 形如 $x = y \text{ op } z$ 的赋值指令表示将地址 y 和地址 z 处的数据对象进行 op 运算，将运算结果存放到地址 x 。指令的操作符是 op （二目运算符）， x 、 y 和 z 是三个操作数。为方便起见，可以用源程序中的名字（也就是标识符）作为三地址指令中的地址，因为每个标识符对应的地址都存放在符号表中，因此，通过这些名字就可以确定它们的地址。另外，地址还可以是常量的形式，也可以是编译器生成的临时变量。形如 $x = \text{op } y$ 的赋值指令中， op 是单目运算符，因此这里只涉及到两个地址（运算分量的地址 y 和运算结果的存放地址 x ）。

(2) 形如 $x = y$ 的复制指令表示将地址 y 中存放的值复制到地址 x 中，指令的操作符是赋值号 $=$ ，只涉及两个操作数（ x 和 y ）。

(3) 形如 $\text{if } x \text{ relop } y \text{ goto } L$ 的条件跳转指令表示当地址 x 和地址 y 中存放的数据对象满足关系 relop 时，则执行标号为 L 的指令，否则将继续执行指令序列中紧跟在该指令之后的指令。指令的操作符是 relop （关系运算符）， x 、 y 和 L 是三个操作数。

(4) 形如 $\text{goto } L$ 的无条件跳转指令表示接下来直接执行标号为 L 的指令。

(5) 形如 $\text{param } x$ 的参数传递指令将 x 设置为参数。

(6) 形如 $\text{call } p, n$ 的过程调用指令和形如 $y = \text{call } p, n$ 的函数调用指令表示调用地址 p 处存放的过程/函数（这里用过程/函数的名字表示过程/函数的地址）。这里，函数是特指带有返回值的过程。 n 是过程/函数的参数个数。指令的操作符是 call ，操作数是 p 和 n 。

(7) 形如 $\text{return } x$ 的过程返回指令中， x 表示返回值的存放地址。

(8) 形如 $x = y[i]$ 的数组引用指令， y 是数组的基地址（这里用数组的名字表

示)。 i 是数组元素的偏移地址，（注意不是数组下标）。这条指令表示将一个数组元素（其地址为数组基地址 y 加上元素偏移地址 i ）的值赋给一个变量， x 是这个变量的地址（这里用变量的名字表示）。指令的操作符是 “ $=[]$ ”，表示赋值号右边是一个数组元素。三个操作数分别是 x 、 y 和 i 。

(9) 形如 $x[i] = y$ 的数组赋值指令表示将地址 y 中存放的值赋给一个数组元素。 x 是数组的基地址（这里用数组的名字表示）， i 是元素的偏移地址。指令的操作符是 “ $[] =$ ”，表示赋值号左边是一个数组元素。三个操作数分别是 x 、 i 和 y 。

(10) 形如 $x = \&y$ 、 $x = *y$ 或 $*x = y$ 的是地址及指针赋值指令。指令 $x = \&y$ 将 x 的右值设为 y 的地址(左值)。这个 y 通常是一个名字，也可能是一个临时变量。它表示一个诸如 $A[i][j]$ 这样具有左值的表达式。 x 是一个指针名字或临时变量。在指令 $x = *y$ 中，假定 y 是一个指针，或是一个其右值表示内存位置的临时变量。这个指令使得 x 的右值等于存储在这个位置中的值。最后，指令 $*x = y$ 把 y 的右值赋给由 x 指向的目标的右值。

三地址指令通常可以用四元式（quadruples）、三元式（triples）和间接三元式（indirect triples）等数据结构表示。这里我们以四元式为例。四元式的形式为 (op, y, z, x) 。其中第 1 个分量 op 表示操作符，后面三个分量 y 、 z 、 x 分别对应三个操作数的地址。图 1-13 列出了图 1-12 中的三地址指令的四元式表示。当某一指令涉及到的操作数不足 3 个时，四元式中的某些分量设置为空（用下划线 “ $_$ ” 表示）。

序号	指令类型	指令形式	四元式
1	赋值指令	$x = y \text{ op } z$ $x = \text{op } y$	(op, y, z, x) $(\text{op}, y, _, x)$
2	复制指令	$x = y$	$(=, y, _, x)$
3	条件跳转	$\text{if } x \text{ relop } y \text{ goto } L$	(relop, x, y, L)
4	非条件跳转	$\text{goto } L$	$(\text{goto}, _, _, L)$
5	参数传递	$\text{param } x$	$(\text{param}, _, _, x)$
6	过程调用 函数调用	$\text{call } p, n$ $y = \text{call } p, n$	$(\text{call}, p, n, _)$ (call, p, n, y)

7	过程返回	return x	(return, _, _, x)
8	数组引用	x = y[i]	(=[, y, i, x)
9	数组赋值	x[i] = y	([=, y, x, i)
10	地址及指针操作	x = & y x = * y *x = y	(&, y, _, x) (=*, y, _, x) (* =, y, _, x)

图 1-13 常用三地址指令的四元式表示

(1) 赋值指令“ $x = y \text{ op } z$ ”的四元式形式为“ (op, y, z, x) ”。其第一个分量是指令中的操作符——双目运算符 op ，第 2 个和第 3 个分量对应着指令的两个源操作数（ y 和 z ），第 4 个分量对应着指令的目标操作数 x 。

(2) 赋值指令“ $x = \text{op } y$ ”的四元式形式为“ $(\text{op}, y, _, x)$ ”。其第一个分量是指令中的操作符——单目运算符 op 。该指令只有一个源操作数（ y ），用第 2 个分量表示，第 3 个分量为空。第 4 个分量对应着指令的目标操作数 x 。

(3) 复制指令“ $x = y$ ”的四元式形式为“ $(=, y, _, x)$ ”。其第一个分量是指令中的操作符——赋值号 $=$ ，第 2 个和第 4 个分量分别对应源操作数 y 和目标操作数（ x ）。

(4) 条件跳转指令“ $\text{if } x \text{ relop } y \text{ goto } n$ ”的四元式形式为“ (relop, x, y, n) ”。其第一个分量是指令中的操作符—— relop ，第 2 个和第 3 个分量分别对应两个操作数（ x 和 y ），第 4 个分量对应着目标跳转地址 n 。

(5) 无条件跳转指令“ $\text{goto } n$ ”的四元式形式为“ (relop, x, y, n) ”。其第一个分量是指令中的操作符—— relop ，第 2 个和第 3 个分量分别对应两个操作数（ x 和 y ），第 4 个分量对应着目标跳转地址 n 。

其它指令的处理方式类似，这里不再赘述。

事实上，四元式形式与我们前面举的自然语言处理的中间表示有相似之处。例如，在自然语言中，给定核心谓语动词“打”，就要涉及到施事者、受事者、工具、地点等语义角色。而三地址指令的操作符就相当于句子的核心谓语动词，操作数就相当于各个语义角色，只不过这里每个操作符涉及的操作数最多为 3 个。

除赋值以外，每个三地址指令最多只有一个操作符（operator），也就是只完成一个操作。因此，这些指令确定了运算完成的顺序。

下面我们来看中间代码生成的一个例子。给定如下一个输入程序片段，

```

while a<b do
    if c<5 then
        while x>y do
            z=x+1;
        else x=y;
    (1.3)

```

该程序片段在一个 while 循环语句中嵌套一个 if-else 分支语句，if-else 分支语句中又嵌套一个 while 循环语句。图 1-14 给出了该程序片段对应的分析树，图 1-15 给出了该程序片段对应的中间代码——四元式序列（假设指令从 100 号开始）。

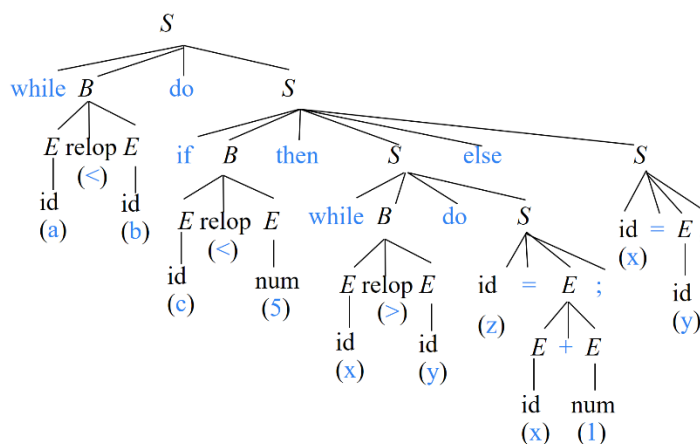


图 1-14 语句 “while a<b do if c<5 then while x>y do z=x+1; else x=y;” 的分析树

100:	(j<, a, b, 102)
101:	(j, -, -, 112)
102:	(j<, c, 5, 104)
103:	(j, -, -, 110)
104:	(j>, x, y, 106)
105:	(j, -, -, 100)
106:	(+, x, 1, t ₁)
107:	(=, t ₁ , -, z)
108:	(j, -, -, 104)
109:	(j, -, -, 100)
110:	(=, y, -, x)
111:	(j, -, -, 100)
112:	

图 1-15 语句 “while a<b do if c<5 then while x>y do z=x+1; else x=y;” 的中间代码

图 1-15 中冒号前面是各条指令的编号，这里从 100 开始，到 112 结束。第 100 号指令是一条条件跳转指令，其中的 “j” 是英语单词 “jump” 的首字母。这条指令表示，当 a<b 时，跳转到 102 号指令。否则继续执行 101 号指令。第

101 号指令是一条无条件跳转指令，跳转的目标地址是 112 号指令，也就是跳出整个 while 循环语句。下面来看一下这段三地址码完成了什么操作。

首先执行第 100 号指令，判断 a 是否小于 b 。如果 a 小于 b ，就跳转到 102 号指令去进一步判断 c 是否小于 5。如果 a 不小于 b ，则继续执行 101 号指令，无条件跳转到 112 号指令，也就是跳出整个 while 循环语句。

对于第 102 号指令，当 c 小于 5 的时候，就跳转到 104 号指令去进一步判断 x 是否大于 y 。如果 c 不小于 5，则继续执行 103 号指令，无条件跳转到 110 号指令，将 y 的值赋给 x 。执行完 110 号指令，继续执行 111 号指令，即跳转到 100 号指令，也就是回到 while 语句的起始处，重新判断 a 是否小于 b 。

对于第 104 号指令，当 x 大于 y 的时候，就跳转到 106 号指令，将 $x+1$ 的值赋给 t_1 ，然后执行 107 号指令，将 t_1 的值赋给 z ，。也就是说 106 号和 107 号两条指令完成了源程序片段中的“ $z=x+1;$ ”对应的操作。执行完这两条语句后，继续执行 108 号指令，即跳转到 104 号指令，重新判断 x 是否大于 y 。如果 x 不大于 y ，则继续执行 105 号指令，无条件跳转到 100 号指令，也就是回到 while 语句的起始处。事实上，根据源程序片段，如果 x 不大于 y ，那么内层 while 语句就执行完毕，从而 if 分支也执行完毕，因此就要回到外层 while 语句的起始处。

那么中间代码生成器是如何根据分析树来自动地生成中间代码的呢？我们将在第 6 章（中间代码生成）中详细介绍。

1.2.5 目标代码生成和代码优化概述

目标代码生成以源程序的中间表示形式作为输入，并把它映射到目标语言。目标代码生成的一个重要任务是为程序中使用的变量合理分配寄存器。

代码优化是为改进代码所进行的等价程序变换，使其运行得更快一些、占用空间更少一些，或者二者兼顾。代码优化分为机器无关优化（中间代码层面）和机器相关优化（目标代码层面）。优化的内容包括很多方面，比如说自动发现并删除代码中一些重复的运算和冗余的运算、把代价比较高的运算替换为代价较低的等价运算。详细内容将在第 8 章和第 9 章中介绍。

1.3 编译程序的生成

编译器作为一个运行在计算机上的程序，其自身的构建方式也经历了深刻的演变。从最初使用机器语言或汇编语言手工编写的艰难起步，到用更高级的

语言来重写编译器带来的可读性与可维护性飞跃，再到为了实现跨平台而进行的编译器移植，再到编译器的（半）自动生成，其发展史本身就是一部软件工程思想的浓缩。

（1）用机器语言编写编译程序

1970 年以前，几乎所有的编译程序都是用机器语言编写的。假如要在一台机器上实现某高级语言的编译程序，由于这个编译程序将在这台机器上运行，所以，最直接的方法就是用该机器的机器语言或者汇编语言来编写这个编译程序。优点是：由于机器语言直接控制硬件，因此可以更好地发挥硬件系统的效率。缺点是：可读性、可靠性、可维护性、编制效率差。尤其是高级语言往是比较复杂的，它的编译程序具有相当的规模。直接用机器语言一步到位的开发一个功能强大的编译器难度很大。

（2）用高级语言编写编译程序

1980 年以后，通常用高级语言来编写编译程序。这将涉及到一种称为自展的技术。所谓自展，就是假设已经有了用机器语言编写的某高级语言的编译器，那么就可以用这个高级语言来编写程序，当然也包括用它来编写一个编译程序，从而构造出一个能够编译更多语言成分的更复杂一点的编译器。为方便讨论这一技术，下面先介绍 T 形图。

当我们提到一个编译器 P ，通常会涉及 3 种语言。首先从功能的角度，编译器的本质是一个翻译工作，所以它必定会涉及到两种语言，就是源语言和目标语言，也就是从哪种语言往哪种语言翻译。这里我们用 S 来表示给编译器输入的源语言程序，用 T 来表示编译器输出的可执行的目标语言的程序。另外，编译器作为一个程序，它本身又是用某种语言开发的，称之为编译程序的“实现语言”，用 I 表示。这样，我们可以用一个 T 形图的三个端点来分别表示这三种语言，如图 1-16 所示。用 T 型的左上端表示输入的源语言程序，右上端表示输出的可执行的目标程序，用 T 型的下端来表示编译程序的实现语言。

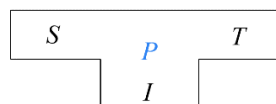


图 1-16 表示语言翻译的 T 形图

这里我们要注意以下几点：

- 1) I 表示的是语言，而 S 和 T 表示的是程序
- 2) 上端这两个元素体现了编译器的功能，即从哪种语言到哪种语言的翻译。直观上这就像是源语言程序和目标程序放在了天平的两端，天平本身就

有表示等价、相等的含义。

3) T 对应于某机器语言 (因为 T 是可在机器上执行的目标程序, “可在机器上执行” 决定 T 必须是一个机器语言)

有了 T 形图这一表现形式, 接下来我们讨论如何用高级语言编写编译程序。这个过程可以用下图表示。假如说已经有了一个可以在 A 机器上运行的高级语言 L_1 的编译器。既然它是一个编译器, 我们就可以用 T 形图来描述它 (如图 1-17(a) 所示)。首先我们先从功能的角度来看, 这个编译器要实现从高级语言 L_1 到机器语言 A 的翻译, 所以它的 T 形图的左上角就是 L_1 语言编写的程序, 右上角就是 A 语言编写的程序 (即 A 代码)。从实现语言的角度, 因为编译器要在 A 机器上运行, 因此需要用 A 机器语言来编写。我们把这个编译器称为 P_1 。

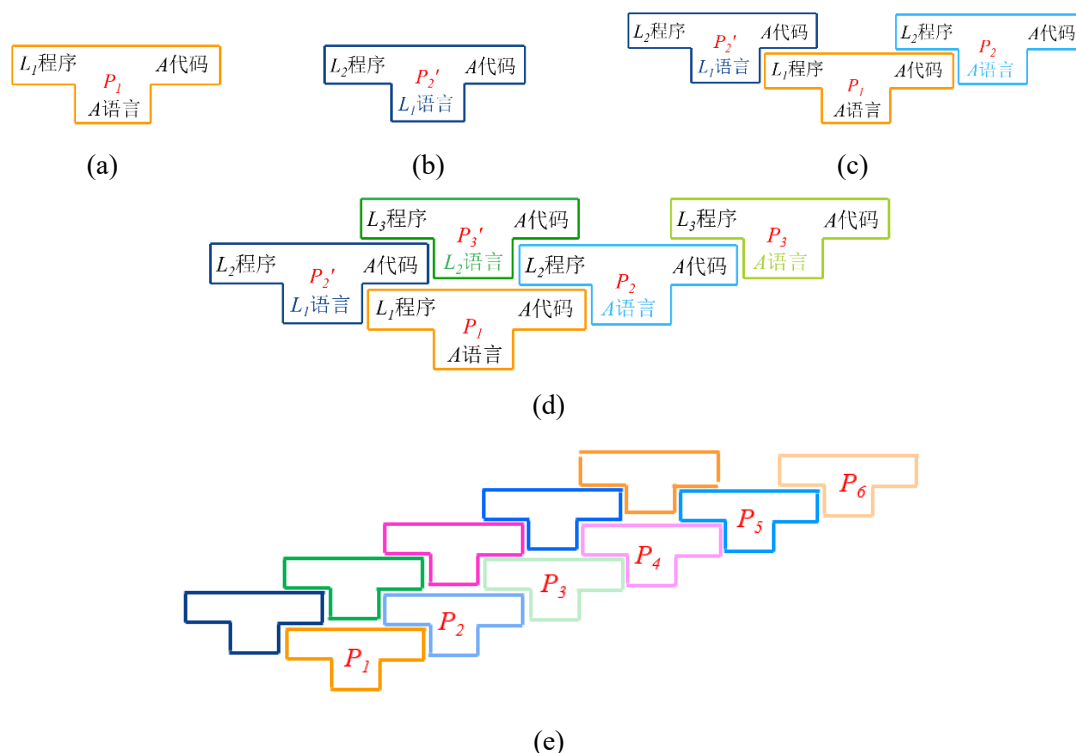


图 1-17 编译器的自展过程示意图

接下来, 在给定 P_1 的基础上, 我们想在 A 机器上构造一个功能更强大的新的语言 L_2 的编译器 P_2 。既然要构造一个编译器, 我们不妨用还是 T 形图来描述它。首先, 从实现语言的角度, 用什么语言来编写呢? 既然现在有了高级语言 L_1 的编译器 P_1 , 我们就可以绕过机器语言, 用高级语言 L_1 来编程了。从编译器功能的角度, 它的输入是一个 L_2 语言的源程序, 输出的是功能等价的 A 语言代码。这样就得到了一个编译器 P_2' (如图 1-17(b) 所示)。注意, 这里我们

并未把这个编译器命名为 P_2 ，这说明它跟 P_2 是有差别的。事实上， P_2' 是用高级语言 L_1 编写的程序，并不能直接在 A 机器上运行（我们可以把 L_1 想象成 C 语言。假设我们有一个扩展名为 “.c” 的 C 语言程序。这个.c 文件如果直接拿到一个没有安装 C 编译器的机器上是无法运行的），必须要通过 L_1 语言的编译器 P_1 （相当于 C 语言的编译器）来编译一下，将它翻译成等价的 A 语言代码才可以在机器 A 上运行。由于 P_2' 是一个用 L_1 语言编写的程序，因此可以作为 P_1 的输入。于是，我们把 P_2' 放置在图 1-17(c) 中所示位置，表示将 P_2' 作为给编译器 P_1 输入的 L_1 程序。

P_2 经 P_1 编译后得到的是什么呢？如图 1-17(c) 所示，在 P_1 右上方与 P_2' 对应的位置上可以得到一个功能上与 P_2' 等价，但是是由 A 语言编写的编译器（即 P_2 ）。功能等价体现在 P_2 与 P_2' 的上端是相同的，都是将 L_2 语言翻译成 A 语言。但是它们实现的语言不同。 P_2' 是用 L_1 语言实现的，而 P_2 是用 A 语言实现的（因为 P_1 的任务就是将 L_1 语言编写的程序翻译成 A 语言的版本）。这样就得到了一个 A 机器上运行的 L_2 语言的编译器 P_2 。

总结一下，利用已有的编译器 P_1 ，我们可以用高级语言 L_1 编写编译器，而无需用直接用机器语言编写。 P_1 会帮我们把我们用高级语言编写的编译器 P_2' 转化成机器代码版本 P_2 。

进一步，如果我们还想在 A 机器上构造一个新的语言 L_3 的编译器，既然 L_2 比 L_1 功能更强大，我们可以用 L_2 语言来编程，编写一个将 L_3 语言翻译成 A 语言的编译器 P_3' ，然后再经过 L_2 语言编译器 P_2 的编译，得到一个用 A 语言编写的 L_3 语言的编译器 P_3 （如图 1-17(d) 所示）。

可见，这种方法使得我们可以在同一台机器上实现不同语言的编译器。这个图继续画下去就如图 1-17(e) 所示。这种利用已有的高级语言来开发新的高级语言的编译程序的方法，使得我们可以非常高效的开发出越来越多的程序设计语言。

（3）编译器的移植

上面这一部分讨论了如何在同一台机器上实现不同语言的编译器。接下来将讨论如何构造同一个语言在不同机器上的编译程序。

硬件在不断发展，机器也在更新换代。新的机器制造出来以后，涉及到如何将之前已经开发的某高级语言的编译器移植到新的机器上。用新的机器语言编写这个高级语言的编译器肯定不是一个好的选择。如何利用 A 机器上已有的编译器高效构造出 B 机器上的编译器？

假设我们已经有了 A 机器上的 L 语言编译程序 P_1 ，我们希望把它移植到 B

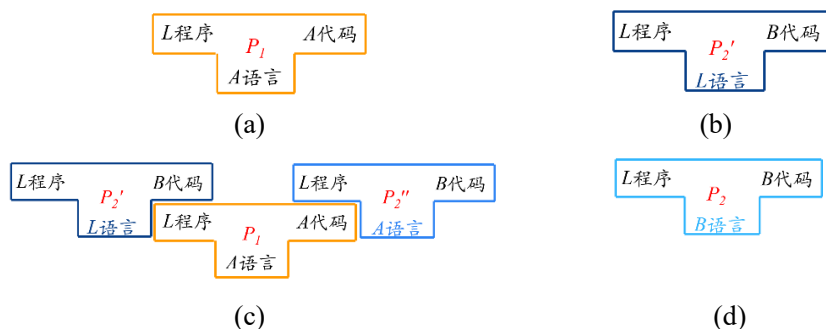
机器上，也就是构造一个在 B 机器上可以运行的 L 语言的编译程序 P_2 。实现过程如下图所示。

首先将 P_1 用 T 形图表示，如图 1-18(a)所示，上端表示功能，将 L 语言翻译成 A 代码，实现语言假设为 A 语言。接下来尝试构造一个新的编译器，如图 1-18(b)所示，从功能的角度来看，它的输入是一个 L 语言的源程序，输出的是功能等价的 B 机器代码。从实现的角度来看，现在有了 L 语言的编译器 P_1 ，就可以用 L 语言来编程了。这样就得到了编译器 P_2' 。将 P_2' 作为编译器 P_1 的输入，编译后得到的是什么呢？如图 1-18(c)所示，是一个功能上与 P_2' 等价但是是由 A 语言编写的编译器 P_2'' 。

到目前为止， P_2'' 功能上虽然符合我们的要求（将 L 语言翻译成 B 语言）但是它的实现语言是 A 语言，依然不满足我们的要求。我们想要的是一个在 B 机器上运行的编译器，也就是说我们希望得到图 1-18(d)所示的 B 语言版本的编译器，那怎么办？既然要生成的是一个 B 机器语言版本的程序，而图 1-18(c)中的 P_2'' 恰好能完成将一个输入程序翻译成 B 语言版本的功能。因此，我们尝试将 P_2 作为 P_2'' 的输出，来反推对应的输入应该是什么样子的一个程序 P ，如图 1-18(e)所示。

这个 P 从功能的角度来说应该与 P_2 是等价的，即，将 L 语言翻译成 B 语言。从实现的角度来讲， P_2'' 的输入是 L 语言的程序，因此， P 是一个用 L 语言编写的程序。由此得到 P 的 T 形图，如图 1-18(f)所示。可以看出， P 实际上就是我们之前已经构造出来的 P_2' 。

这样就得到了一个完整的移植方案：首先，既然已经有了 L 语言的编译器，那么就用 L 语言编写一个从 L 语言到 B 语言的编译器 P_2' 。经过 P_1 编译后得到用 A 语言编写的从 L 语言到 B 语言的编译器 P_2'' 。用新生成的编译器 P_2'' 再次编译刚刚用高级语言编写的编译程序 P_2' 就得到了 B 机器上运行的高级语言 L 的编译器，这样就将 A 机器上的编译器移植到了 B 机器上。我们需要做的就是用 L 语言编写一个从 L 语言到 B 语言的编译器 P_2' 。



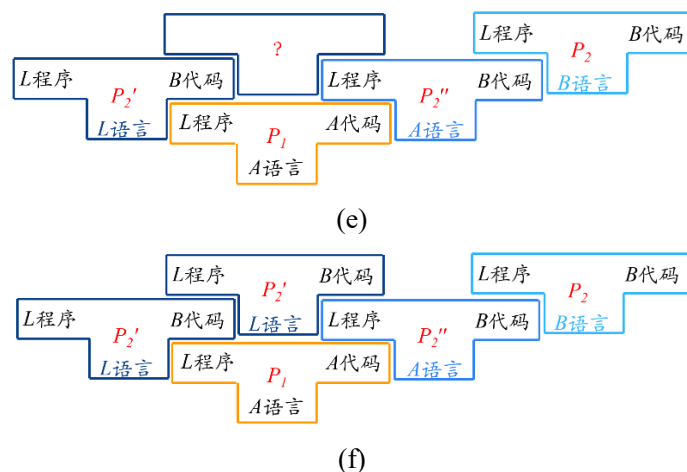


图 1-18 编译器的移植过程示意图

(4) 编译器的自动生成

在计算机理论科学的支持下，已经出现了许多自动生成部分编译器、甚至整个编译器的有效工具，有些工具能自动产生词法分析程序（如 LEX），有些工具能自动产生语法分析程序（如 YACC、ANTLR）。这些构造编译程序的工具，都有相似的工作模式：接受对源语言或目标语言的形式化描述（如 L 语言的语法描述或语义描述，目标语言或机器描述），这些描述都是形式化的编译程序，根据这些描述就会自动的产生一个 L 语言的编译程序。我们将在后续的词法分析，语法分析等章节学习这些自动生成工具的原理，这些工具的设计体现了很多计算机科学的思想精华。

1.4 编译技术的应用

编译技术不仅是实现程序设计语言的核心工具，更是现代计算机科学与工程领域的重要基础设施。尽管直接从事主流语言编译器开发的专业人员相对有限，但编译技术所蕴含的思想和方法论已广泛渗透至诸多领域。理解编译原理与技术，对于进行程序分析、软件安全、开发工具链建设、系统优化等方面具有深远意义，其应用价值早已超越编译器本身，成为计算机专业人员必备的基础能力。

(1) 复杂数据的分析处理

在信息时代，各种结构化和半结构化数据的处理成为常态。编译技术提供了一套系统化的方法论，可应用于多种复杂数据处理场景。例如，基于词法分

析器生成器（如 Lex、Flex）可以高效实现对日志文件、配置文件、数据交换格式（如 JSON、XML）等文本格式的解析；利用语法分析技术可以识别具有层次结构的数据模式；借助语法制导翻译方法，则可在解析过程中直接嵌入语义处理逻辑，实现数据清洗、转换和校验，形成完整的数据处理流水线。此类技术在网络协议分析、数据格式转换、大数据预处理等领域发挥着重要作用。

（2）数据库查询解释器

数据库管理系统（DBMS）的核心组件之一是查询处理子系统，其本质是一个高级语言翻译系统。SQL 等查询语言包含丰富的表达式和谓词逻辑，查询解释器或编译器首先对查询语句进行词法分析和语法分析，构建出查询的抽象语法树（AST）。随后通过语义分析检查类型与约束，并基于关系代数等中间表示进行查询优化，最终生成可在存储引擎上执行的低级访问计划。该过程与编译器前端至代码生成的过程高度相似，是编译技术在实际系统中成功应用的典范。

（3）软件工程中的应用

程序分析技术极大地推动了软件工程自动化与智能化水平的提升，催生了一系列高效的开发辅助工具。

1) 结构化编辑器

结构化编辑器是集成开发环境（IDE）的核心功能之一，它基于语法制导翻译和增量分析技术，在用户输入过程中实时进行语法检查与引导。例如，它可以自动完成关键字、高亮语法元素、实时提示括号匹配、根据语言语法提供代码模板，并在用户输入错误时给予即时反馈。这类工具显著降低了语法错误的可能性，加速了编码与调试过程，提升了软件开发的整体效率与代码质量。

2) 代码格式化工具

代码格式化工具（又称“美化打印器”）基于对程序结构的精确分析，自动调整代码的排版布局，以增强其可读性。它会根据语言语法确定代码块的嵌套层次，并施加一致的缩进规则；对长表达式进行智能换行；对注释采用特殊格式高亮；并可统一代码风格（如花括号位置、空格使用等）。这既保证了团队协作时代码风格的一致性，也使程序结构一目了然。

3) 静态分析工具

静态分析工具在不运行程序的前提下，通过对源代码或中间表示的深度分析，发现潜在的错误、漏洞及代码质量问题。它利用数据流分析、控制流分析、别名分析、约束求解等编译优化中的经典技术，可以检测出空指针解引用、数组越界、资源泄漏、数据竞争等运行时缺陷，也能识别死代码、未初始化变量、

重复逻辑等质量问题。此类工具已成为保障软件安全性与可靠性的关键环节。

4) 高级语言翻译与迁移工具

在计算机系统持续演进的过程中，常常面临将既有软件系统迁移到新硬件平台或新语言环境的需求。高级语言翻译工具（通常称为源码到源码编译器或转换器）旨在自动化这一迁移过程。它们对遗留代码进行完整的词法、语法及语义分析，理解其逻辑，然后生成语义等价的目标平台高级语言代码。此类技术不仅应用于语言升级（如 COBOL 到 Java）、平台迁移（如 x86 到 ARM），也广泛应用于代码重构、领域特定语言（DSL）的实现以及并行化自动转换等场景，极大地节约了移植成本并降低了风险。

通过以上实例可以看出，编译技术已深深融入现代软件生命周期的各个阶段，成为连接理论、语言与系统实现的桥梁。掌握编译原理，意味着获得了分析和处理一切形式化语言与结构化系统的强大能力。

1.5 本章小结

本章系统阐述了编译原理的基本概念与整体框架。首先界定了“编译”的本质——将高级语言程序转换为可执行机器代码的系统过程。随后剖析了现代编译器的典型逻辑结构，将其划分为前端、中端与后端，并详细说明了各阶段的核心任务与协作流程。接着探讨了编译程序自身的构建方法，介绍了从手工实现到利用自动生成工具的技术演进。最后，通过丰富实例展示了编译技术在软件工程、程序分析、系统优化等领域的广泛应用，揭示了其作为计算机科学基础支撑技术的深远影响。本章为后续深入各编译阶段的具体技术与算法奠定了坚实的知识基础。

本章要点如图 1-19 所示。

- 什么是编译
 - 将高级语言翻译成汇编语言或机器语言的过程
- 编译器的结构及各部分的主要任务
 - 词法分析 识别单词，确定单词的类型
 - 语法分析 识别短语/句子，构造语法分析树
 - 语义分析
 - （声明语句）收集标识符的属性信息
 - 语义检查
 - 变量或过程名重复声明
 - 变量或过程未经声明就使用
 - 操作数之间类型不匹配
 - 操作符与操作数之间的类型不匹配
 - 中间代码生成
 - 赋值号左边出现一个只有右值的表达式
 - 对非数组变量使用数组访问操作符 $a[x]$
 - 数组下标不是整数
 - 对非结构体类型变量使用“.”操作符 $a.b$
 - 对非过程名使用过程调用操作符
 - 过程调用的参数类型或数目不匹配 $a(...)$
 - 目标代码生成
 - 代码优化

图 1-19 第 1 章要点

第2章 程序设计语言及其文法

语言是编译的起点与核心。编译器之所以能“理解”并处理我们用高级语言编写的程序，其根本在于我们为它装备了形式化的语言学知识——文法。本章将引领我们进入程序设计语言的数学与逻辑世界，我们将系统阐述如何精确定义语言、如何用文法这一强大工具来描述语言的构成规则。从“串”、“字母表”等基本概念出发，到文法的形式化定义、乔姆斯基文法分类体系，再到上下文无关文法的核心概念——分析树与二义性，本章将奠定后续词法分析与语法分析的理论基石，揭示语言、文法与编译器之间的内在逻辑关联。

2.1 语言概述

为了有效地描述语言，首先必须弄清楚什么是语言。简单来讲，语言可以看作是句子的集合。句子可以看作是由词构成的串，而词可以看作是由字符构成的串。但是句子本身并不是由随机抽取的一些单词简单组合而成的串。例如有这样一个词串“I’m I do, sorry that afraid Dave I’m can’t.”面对这样一个词串，我们不知道它在说什么。同样还是这些单词，如果我们换一个顺序，将其变成“I’m sorry Dave, I’m afraid I can’t do that.”面对这样一个顺序的词串，我们就知道它在说什么了。因此，构成句子的各个单词之间需要满足一定的组合规律。同样的，构成单词的各个字符之间也要满足一定的规则，也就是所谓的构词法。汉语有汉语的构词法，英语有英语的构词法。比如说英语里面复数，什么情况下在名词的后面加s，什么时候加es，什么时候加ies都是有一定规律的。

综上所述，语言是字符及其组合规则的统一体。这里的规则指的就是文法。

2.2 基本概念

为便于进行相关讨论，首先定义以下基本概念。

2.2.1 串

串(string)是一个有穷符号(symbol)序列(“符号”是一个抽象的实体，我们将不去形式地定义它，就像在几何学中对“点”和“线”不加定义一样。符号的典型例子包括字母、数字和标点符号等)。例如，a、b和c是符号，

abcb 是串。如果我们把单词看作是一个串的话，那么串中的每个符号就是一个字符。如果我们把句子看作是一个串的话，那么串中的每个符号就是一个单词。因此，串的定义中这个符号并不是狭义的指一个字符。它具体的含义还要依据它的应用场景来确定。

串 s 的长度（通常记作 $|s|$ ）是指组成串 s 的符号的个数。例如， $|abcb|=4$ 。空串（empty string）是指长度为 0 的串，用 ε 表示。即： $|\varepsilon|=0$ 。

如果 x 和 y 是串，那么 x 和 y 的连接（concatenation）是把 y 附加到 x 后面而形成的串，记作 xy 。例如，如果 $x=\text{book}$ 且 $y=\text{store}$ ，那么 $xy=\text{bookstore}$ 。空串是连接运算的单位元（identity），即，对于任何串 s 都有， $\varepsilon s = s\varepsilon = s$ 。

如果将串的连接运算看作是串 s 的乘积，则可以定义串的“指数（exponentiation）”运算。串 s 的 n 次幂（power）定义为：

$$\begin{cases} s^0 = \varepsilon \\ s^n = s^{n-1}s, n \geq 1 \end{cases} \quad (2.1)$$

根据该公式，

$$s^1 = s^0 s = \varepsilon s = s$$

$$s^2 = s^1 s = ss$$

$$s^3 = s^2 s = sss$$

...

这样， s 的 n 次幂就是 n 个 s 连接在一起。例如：如果 $s=ba$ ，那么 $s^0=\varepsilon$ ， $s^1=ba$ ， $s^2=baba$ ， $s^3=bababa$ ，...

设 x 、 y 、 z 是三个串，如果 $x=yz$ ，则称 y 是 x 的前缀（prefix）， z 是 x 的后缀（suffix）。如果 $z \neq \varepsilon$ ，则称 y 是 x 的真前缀（true prefix）；如果 $y \neq \varepsilon$ ，则称 z 是 x 的真后缀（true suffix）。

2.2.2 字母表

字母表（alphabet） Σ 是一个有穷符号集合。符号的典型例子包括字母、数字和标点符号。由 0 和 1 两个字符构成的集合就是一个二进制字母表。另外，ASCII 字符集、Unicode 字符集都是字母表。

既然字母表是一个集合，那么就可以对它进行一些集合运算。

（1）字母表 Σ_1 和 Σ_2 的乘积（product）

假设 Σ_1 和 Σ_2 的是两个字母表，则 Σ_1 和 Σ_2 的乘积定义为：

$$\Sigma_1 \Sigma_2 = \{ab | a \in \Sigma_1, b \in \Sigma_2\} \quad (2.2)$$

例如: $\{0, 1\}^* \{a, b\} = \{0a, 0b, 1a, 1b\}$ 。

(2) 字母表 Σ 的 n 次幂 (power)

设 Σ 是一个字母表, 则 Σ 的 n 次幂的递归定义为:

$$\begin{cases} \Sigma^0 = \{\varepsilon\} \\ \Sigma^n = \Sigma \Sigma^{n-1}, n \geq 1 \end{cases} \quad (2.3)$$

其中, ε 是 2.1.1 节定义的空串。例如, $\{0, 1\}^3 = \{0, 1\} \{0, 1\} \{0, 1\} = \{000, 001, 010, 011, 100, 101, 110, 111\}$ 。由此可见, 字母表的 n 次幂实际上就是长度为 n 的串构成的集合。

(3) 字母表 Σ 的正闭包 (positive closure)

设 Σ 是一个字母表, 则 Σ 的正闭包定义为:

$$\Sigma^+ = \Sigma \cup \Sigma^2 \cup \Sigma^3 \cup \dots \quad (2.4)$$

也就是说, 字母表 Σ 的正闭包等于 Σ 的各个正数次幂构成的并集。例如,

$$\begin{aligned} \{a, b, c, d\}^+ = \{ & a, b, c, d, \\ & aa, ab, ac, ad, ba, bb, bc, bd, \dots, \\ & aaa, aab, aac, aad, aba, abb, abc, \dots \} \end{aligned}$$

由此可见, 字母表的正闭包实际上就是长度为正数 (也就是大于等于 1) 的符号串构成的集合

(4) 字母表 Σ 的克林闭包 (Kleene closure)

设 Σ 是一个字母表, 则 Σ 的克林闭包定义为:

$$\Sigma^* = \Sigma^0 \cup \Sigma^+ = \Sigma^0 \cup \Sigma \cup \Sigma^2 \cup \Sigma^3 \cup \dots \quad (2.5)$$

可见, 字母表 Σ 的克林闭包是在 Σ 的正闭包的基础上再并上 Σ^0 , 也就是在 Σ 的正闭包中再添加一个 ε 。例如,

$$\begin{aligned} \{a, b, c, d\}^* = \{ & \varepsilon, a, b, c, d, \\ & aa, ab, ac, ad, ba, bb, bc, bd, \dots, \\ & aaa, aab, aac, aad, aba, abb, abc, \dots \} \end{aligned}$$

由此可见, 字母表的克林闭包实际上就是一个长度可以为零的符号串集合。

有了上述基本定义, 就可以讨论文法的定义了。

2.3 文法的定义

什么是文法? 我们先来看一个自然语言的例子。

例 2.1 以下是一个简化版本的用来描述英语句子构成规则的文法。

(1) $\langle \text{句子} \rangle \rightarrow \langle \text{名词短语} \rangle \langle \text{动词短语} \rangle$

- (2) $\langle \text{名词短语} \rangle \rightarrow \langle \text{形容词} \rangle \langle \text{名词短语} \rangle$
- (3) $\langle \text{名词短语} \rangle \rightarrow \langle \text{名词} \rangle$
- (4) $\langle \text{动词短语} \rangle \rightarrow \langle \text{动词} \rangle \langle \text{名词短语} \rangle$
- (5) $\langle \text{形容词} \rangle \rightarrow \text{little}$
- (6) $\langle \text{名词} \rangle \rightarrow \text{boy}$
- (7) $\langle \text{名词} \rangle \rightarrow \text{apple}$
- (8) $\langle \text{动词} \rangle \rightarrow \text{eat}$ (2.6)

第(1)条规则表示，一个句子由一个名词短语和一个动词短语构成；第(2)和第(3)条规则表示，一个名词短语可以由一个形容词短语和一个名词短语构成，也可以由一个名词构成；后续几条规则表示，一个动词短语可以由一个动词和一个名词短语构成，形容词可以是 little，名词可以是 boy 或 apple，动词可以是 eat。□

从例 2.1 中可以提取一些文法的组成要素。该文法中出现的符号可以分为两类，一类用尖括号括起来，一类未用尖括号括起来。其中未用尖括号括起来部分表示的是一个单词，它们是最终可以出现在该文法所描述的语言中的符号，称为语言的基本符号。由于这个文法是用来描述句子的构成规则的，句子是由单词构成的串，所以它的基本符号指的就是单词。如果一个文法是用来描述单词的构成规则的，那么它的基本符号指的就是字符。用尖括号括起来符号并不能出现在最终的句子中，它们表示的是语法成分。

由此，给出文法的形式化定义。文法 (grammar) G 是一个四元组

$$G = (V_T, V_N, P, S) \quad (2.7)$$

(1) V_T : 终结符 (terminal symbol) 集合。终结符是文法所定义的语言的基本符号，有时也称为 token。例如，例 2.1 中的文法是用来描述句子的构成规则，而句子的基本符号是单词，因此，这些单词构成了该文法的终结符集合，即

$$V_T = \{ \text{apple, boy, eat, little} \}$$

如果 G 表示句法 (用来描述句子构成规则的文法)，因为句子是由单词构成的，则每个终结符是一个单词 (token)。如果 G 表示词法 (用来描述单词构成规则的文法)，则每个终结符是一个字符。

(2) V_N : 非终结符 (nonterminal) 集合。非终结符是用来表示语法成分的符号 (例如文法 2.1 中用尖括号括起来那些符号)。由于可以从它们进一步推出其它的句子成分，所以将它们称为非终结符号，有时也称为语法变量。例如，在例 1 中，

$$V_N = \{ \langle \text{句子} \rangle, \langle \text{名词短语} \rangle, \langle \text{动词短语} \rangle, \langle \text{名词} \rangle, \dots \}$$

V_T 和 V_N 是不相交的, 即 $V_T \cap V_N = \Phi$ 。终结符与非终结符统称为文法符号, 因此, $V_T \cup V_N$ 构成了文法符号集。

(3) P : 产生式 (production) 集合。产生式描述了将终结符和非终结符组合成串的方法。通俗地将, 产生式就是用来产生句子的规则。产生式的一般形式为:

$$\alpha \rightarrow \beta \quad (2.8)$$

读作: α 定义为 β 。 $\alpha \in (V_T \cup V_N)^+$, 且 α 中至少包含一个 V_N 中的元素, 称为产生式的头 (head) 或左部 (left side)。 $\beta \in (V_T \cup V_N)^*$, 称为产生式的体 (body) 或右部 (right side)。 $V_T \cup V_N$ 是文法符号集合, 其本质上是一个字母表。根据 2.2.2 节关于字母表上的运算的论述, $(V_T \cup V_N)^+$ 和 $(V_T \cup V_N)^*$ 都是文法符号串的集合。因此, α 和 β 都是文法符号串, 且 α 中至少包含一个非终结符。

在例 2.1 中, 每条规则都是一个产生式, 即

$$P = \{ \begin{aligned} &\langle \text{句子} \rangle \rightarrow \langle \text{名词短语} \rangle \langle \text{动词短} \rangle, \\ &\langle \text{名词短语} \rangle \rightarrow \langle \text{形容词} \rangle \langle \text{名词短语} \rangle, \\ &\dots \end{aligned} \}$$

(4) S : 文法的开始符号 (start symbol)。它是一个特殊的非终结符号, $S \in V_N$ 。开始符号表示的是该文法中最大的语法成分。例如, 例 1 中的文法是用来描述句子构成规则的, $\langle \text{句子} \rangle$ 是该文法中最大的语法成分, 因此, $\langle \text{句子} \rangle$ 是该文法开始符号。

除非特别说明, 文法的第一个产生式的左部就是开始符号。

根据以上定义, 我们来看一个文法的例子。

例 2.2 以下是一个简化版本的算术表达式文法:

$$G = (\{ \text{id}, +, *, (,) \}, \{E\}, P, E)$$

其中, $P = \{$

$$E \rightarrow E + E,$$

$$E \rightarrow E * E,$$

$$E \rightarrow (E),$$

$$E \rightarrow \text{id} \}$$

这里我们约定: 不引起歧义的前提下, 可以只写产生式, 即

$$G: \quad E \rightarrow E + E$$

$$E \rightarrow E * E$$

$$E \rightarrow (E)$$

$$E \rightarrow \mathbf{id} \quad \square$$

对一组有相同左部的 α 产生式

$$\begin{aligned} \alpha &\rightarrow \beta_1 \\ \alpha &\rightarrow \beta_2 \\ &\dots \\ \alpha &\rightarrow \beta_n \end{aligned} \quad (2.9)$$

可以简记为:

$$\alpha \rightarrow \beta_1 \mid \beta_2 \mid \dots \mid \beta_n \quad (2.10)$$

读作: α 定义为 β_1 , 或者 β_2 , ..., 或者 β_n 。 $\beta_1, \beta_2, \dots, \beta_n$ 称为 α 的候选式 (candidate)。例如,

$$\begin{aligned} E &\rightarrow E + E \\ E &\rightarrow E * E \\ E &\rightarrow (E) \\ E &\rightarrow \mathbf{id} \end{aligned}$$

可以简记为 $E \rightarrow E + E \mid E * E \mid (E) \mid \mathbf{id}$ 。

为了避免总是声明哪些是终结符号, 哪些是非终结符号, 哪些是单个符号, 哪些是串, 等等, 我们对文法符号的使用进行以下约定 (参见图 2-1)。

(1) 下述符号是终结符:

- 字母表中排在前面的小写字母, 如 a, b, c 。
- 运算符, 如 $+, *$ 等。
- 标点符号, 如括号、逗号等。
- 数字 $0, 1, \dots, 9$ 。
- 黑体字符串, 如 $\mathbf{id}, \mathbf{if}$ 等。

(2) 下述符号是非终结符:

- 字母表中排在前面的大写字母, 如 A, B, C 。
- 字母 S 。它出现时通常表示开始符号。
- 小写、斜体的名字, 如 $expr, stmt$ 等。
- 代表程序构造的大写字母。如 E (表达式)、 T (项) 和 F (因子)。

(3) 用排在字母表后面的大写字母表示文法结符。

(4) 用排在字母表后面的小写字母表示终结符号串。

(5) 用小写的希腊字母表示 (可能为空的) 文法符号串。

终结符	a, b, c	终结符号串	$u, v, \dots, z$
非终结符	A, B, C		

文法符号	X, Y, Z	文法符号串	α, β, γ
------	-----------	-------	-------------------------

图 2-1 终结符号/非终结符号（串）的使用约定

可以使用联想记忆法：终结符表示文法中的基本符号，非终结符表示语法成分，语法成分的粒度比基本符号大，因此，非终结符用大写字母表示，终结符用小写字母表示。文法符号既可以是终结符，也可以是非终结符，具有不确定性，类似于一个变量，因此用 X 、 Y 、 Z 来表示。

上图左边这一列对应单个符号，右边一列对应符号串。这里我们给终结符号串和文法符号串约定了符号，但是没有为非终结符号串约定符号，这是为什么呢？事实上，非终结符号串没有对应的应用场景。终结符号串的应用场景是句子，文法符号串的应用场景是句型。如果句型中的所有符号都是终结符，那么这样的句型称为句子。而要求句型中的所有符号都是非终结符，这是什么应用场景呢？这种情况太少见了，不具备普遍性，因此没有约定相应的符号。

2.4 语言的定义

有了文法的定义后，我们来讨论一下如何根据文法来定义语言。我们说语言是由满足文法规则的终结符号串（即句子）组成的。如何判定一个终结符号串是否满足文法规则？通过什么途径判定？有两种方式，一是通过推导，一是通过归约。我们首先给出这两个概念。

给定文法 $G=(V_T, V_N, P, S)$ ，如果 $\alpha \rightarrow \beta \in P$ ，那么可以将符号串 $\gamma\alpha\delta$ 中的 α 替换为 β ，也就是说，将 $\gamma\alpha\delta$ 重写（rewrite）为 $\gamma\beta\delta$ ，记作 $\gamma\alpha\delta \Rightarrow \gamma\beta\delta$ 。此时，称文法中的符号串 $\gamma\alpha\delta$ 直接推导（directly derive）出 $\gamma\beta\delta$ 。与之相对应，也可以称文法中的符号串 $\gamma\beta\delta$ 直接归约（directly reduce）出 $\gamma\alpha\delta$ 。

如果 $\alpha_0 \Rightarrow \alpha_1$ ， $\alpha_1 \Rightarrow \alpha_2$ ， $\alpha_2 \Rightarrow \alpha_3$ ，...， $\alpha_{n-1} \Rightarrow \alpha_n$ ，则可以记作 $\alpha_0 \Rightarrow \alpha_1 \Rightarrow \alpha_2 \Rightarrow \alpha_3 \Rightarrow \dots \Rightarrow \alpha_{n-1} \Rightarrow \alpha_n$ ，称符号串 α_0 经过 n 步推导出 α_n ，可简记为 $\alpha_0 \Rightarrow^n \alpha_n$ 。

这里需要注意， n 可以等于 0。经过 0 步推导实际上就是没有推导，所以 α 经过 0 步推导得到的还是 α ，即 $\alpha \Rightarrow^0 \alpha$ 。 $\Rightarrow^+$ 表示“经过至少 1 步推导”。 $\Rightarrow^*$ 表示“经过若干（可以是 0）步推导”。

例 2.3 对于例 2.2 中给出的文法，从文法开始符号 <句子>，经过若干步推导可以得到词串“little boy eats apple”（如下图所示）。从这个词串经过若干步归约，也可以归约出文法开始符号 <句子>。

<句子> $\Rightarrow$ <名词短语> <动词短语>

$\Rightarrow$ <形容词> <名词短语> <动词短语>
 $\Rightarrow$ little <名词短语> <动词短语>
 $\Rightarrow$ little <名词> <动词短语>
 $\Rightarrow$ little boy <动词短语>
 $\Rightarrow$ little boy <动词><名词短语>
 $\Rightarrow$ little boy eats <名词短语>
 $\Rightarrow$ little boy eats <名词>
 $\Rightarrow$ little boy eats apple (2.11)

而对于词串 “I’m I do, sorry that afraid Dave I’m can’t.” 无论是从推导的角度，还是从归约的角度，都无法给它构造出一个对应的过程。 □

有了推导和归约的概念，我们可以回答“如何判定一个终结符号串是否满足文法规则”的问题。如果从文法开始符号可以推导出这个符号串，那么该终结符号串就满足文法规则，它就是该语言的一个句子，这是从生成语言的角度；如果从这个符号串可以归约出文法开始符号，那么该终结符号串也满足文法规则，它就是该语言的一个句子，这是从识别语言的角度。由此，我们可以给出两个概念：句型和句子。

如果 $S \Rightarrow^* \alpha$, $\alpha \in (V_T \cup V_N)^*$, 则称 α 是 G 的一个句型 (sentential form)。可见，一个句型中既可以包含终结符，又可以包含非终结符，也可能是空串。如果 $S \Rightarrow^* w$, $w \in V_T^*$, 则称 w 是 G 的一个句子 (sentence)。可见，句子是不包含非终结符的句型。在例 2.3 的推导中，每一步得到的结果都是一个句型，但只有最后一步得到的结果是句子。

基于以上概念，给出语言的形式化的定义：由文法 G 的开始符号 S 推导出的所有句子构成的集合称为文法 G 生成的语言，记为 $L(G)$ 。即

$$L(G) = \{w | S \Rightarrow^* w, w \in V_T^*\} \quad (2.12)$$

其中， $w \in V_T^*$ 表示 w 是一个终结符号串， $S \Rightarrow^* w$ 表示 w 满足文法规则。

根据语言的定义可知，语言是一个集合，因此，可以进行集合上的各种运算，包括并运算、连接运算（相当于乘积运算）、幂运算，闭包运算等等。图 2-2 给出了这些运算的正式定义。

运算	定义和表示
L 和 M 的并	$L \cup M = \{s s \text{ 属于 } L \text{ 或者 } s \text{ 属于 } M\}$
L 和 M 的连接	$LM = \{st s \text{ 属于 } L \text{ 且 } t \text{ 属于 } M\}$
L 幂	$\begin{cases} L^0 = \{\epsilon\} \\ L^n = L^{n-1}L, n \geq 1 \end{cases}$

L 的 Kleene 闭包	$L^* = \bigcup_{i=0}^{\infty} L^i$
L 的正闭包	$L^+ = \bigcup_{i=1}^{\infty} L^i$

图 2-2 语言上的运算的定义

2.5 文法的分类

Chomsky 根据对产生式要求的不同, 将文法分成 4 类, 分别是 0 型文法 (type-0 grammar)、1 型文法 (type-1 grammar)、2 型文法 (type-2 grammar) 和 3 型文法 (type-3 grammar), 通常称为 Chomsky 体系。

(1) 0 型文法。这类文法的特点是对产生式 $\alpha \rightarrow \beta$ 的两边几乎无限制, 仅要求每个产生式的左部 α 至少含 1 个非终结符。因此这类文法又称为无限制文法 (unrestricted grammar), 有时也称短语结构文法 (phrase structure grammar, PSG)。由 0 型文法 G 生成的语言 $L(G)$ 称为 0 型语言。

(2) 1 型文法。这类文法要求 $\forall \alpha \rightarrow \beta \in P$, 均有 $|\alpha| \leq |\beta|$ 成立。1 型文法也称为上下文有关文法 (context-sensitive grammar, CSG)。之所以称为“上下文有关”是因为在该文法中, 产生式的一般形式为:

$$\alpha_1 A \alpha_2 \rightarrow \alpha_1 \beta \alpha_2 \quad (\beta \neq \epsilon) \quad (2.13)$$

即, 当一个非终结符 A 的前后分别是 α_1 和 α_2 时, A 可以被替换成 β 。 $\beta \neq \epsilon$ 是为了使 $|\alpha| \leq |\beta|$ 。由上下文有关文法 (1 型文法) G 生成的语言 $L(G)$ 称为上下文有关语言 (1 型语言)。

(3) 2 型文法。这类文法在 1 型文法的基础上, 进一步限制产生式的左部为单个非终结符, 也就是说, 产生式的一般形式为 $A \rightarrow \beta$ 。2 型文法也称为上下文无关文法 (context-free grammar, CFG)。之所以称为“上下文无关”是因为当用 β 替换 A 时, 与 A 的上下文环境无关。由上下文无关文法 (2 型文法) G 生成的语言 $L(G)$ 称为上下文无关语言 (2 型语言)。

例 2.4 一个用于生成标识符的 CFG

$$(1) S \rightarrow L \mid LT$$

$$(2) T \rightarrow L \mid D \mid TL \mid TD$$

$$(3) L \rightarrow a \mid b \mid c \mid d \mid \dots \mid z$$

$$(4) D \rightarrow 0 \mid 1 \mid 2 \mid 3 \mid \dots \mid 9$$

□

(4) 3 型文法。也称为正则文法 (regular grammar, RG)。正则文法分为两类, 一类是右线性 (right linear) 文法, 另一类是左线性 (left linear) 文法。

右线性文法在 2 型文法的基础上，进一步限制产生式的右部是要么是一个终结字符串 w ，是要么是一个终结字符串 w 右面跟一个非终结符，即 $A \rightarrow wB$ 或 $A \rightarrow w$ 。与右线性文法相对应，左线性文法要求产生式的右部要么是一个终结字符串 w ，要么是一个终结字符串 w 左面加一个非终结符，即 $A \rightarrow Bw$ 或 $A \rightarrow w$ 。把这两种情况概括起来，也就是说右部最多一个非终结符，且都得把一边。由于该文法要求规则的形式非常的规整（regular），因此叫正则文法。由正则文法（3 型文法） G 生成的语言 $L(G)$ 称为正则语言（3 型语言）。

例 2.5 一个右线性文法

$$(1) S \rightarrow a | b | c | d$$

$$(2) S \rightarrow aT | bT | cT | dT$$

$$(3) T \rightarrow a | b | c | d | 0 | 1 | 2 | 3 | 4 | 5$$

$$(4) T \rightarrow aT | bT | cT | dT | 0T | 1T | 2T | 3T | 4T | 5T$$

这里用 a 、 b 、 c 和 d 代表全部字母，用 0 、 1 、 2 、 3 、 4 和 5 代表全部数字。从形式上来看，每个产生式的右部要么是一个终结字符串，要么是一个终结字符串加上一个非终结符，它满足右线性文法的要求。

那么这个文法生成的是什么语言？先来看第 3 个产生式，可知一个字母或一个数字可以构成一个 T 。根据第 4 个产生式，可知一个字母或数字后接一个 T 可以构成一个更大的 T ，因此， T 生成的实际上是一个字母数字串

根据第 1 个产生式，我们知道一个字母可以构成一个句子。根据第 2 个产生式，可知一个字母后面加上一个 T （也就是说字母数字串）可以构成一个句子。因此，这个文法都是用来生成标识符的。同前面的 CFG 的例子是等价的。

事实上，这个右线性文法还可以写出与之等价的左线性文法，感兴趣的同同学可以在自行完成这个练习。 □

事实上，不仅标识符可以用正则文法描述，程序设计语言的多数单词都可以用正则文法来描述。我们在第三章词法分析中将详细介绍。

根据前面各类文法的定义可以看出，四种文法之间是逐级限制的关系。也就是说满足了 3 型文法的要求，必然能满足 2 型文法的要求；满足了 2 型文法的要求，必然能满足 1 型文法的要求；以此类推。因此、四种文法之间也是逐级包含的关系。也就是说一个文法如果是 3 型文法，那么它必然也是 2 型文法、1 型文法和 0 型文法，以此类推。

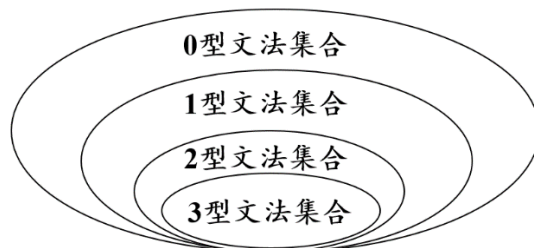


图 2-3 各类文法之间的关系

根据 CSG、CFG、RG 的定义可知，CSG、CFG、RG 中不含空产生式。所谓空产生式就是形如 $A \rightarrow \varepsilon$ 的产生式，即右部是空串 ε 。因为 $|\varepsilon|=0$ ，所以空产生式不满足 $|\alpha| \leq |\beta|$ 的条件。如果将形如 $A \rightarrow \varepsilon$ 的产生式排除在 CSG、CFG、RG 之外，也就是将 ε 排除在 CSL、CFL、RL 之外，这不能不说是一种缺憾。在语言处理中，如果对空产生式进行特殊的处理，而对其它的产生式均按照该类方法的一般情况处理，也是可以的。因此，在后面的讲述中约定，CSG、CFG、RG 中可以含有形如 $A \rightarrow \varepsilon$ 的产生式。

2.6 CFG 文法的分析树

对于一个高级程序设计语言来说，绝大部分成分都是可以用 CFG 描述的。基于 CFG 的推导过程可以用图的形式表示，即语法分析树（parse tree）。语法分析树的每个内部节点表示对一个产生式 $A \rightarrow \beta$ 的应用。该内部节点的标号是此产生式左部 A ，该节点的子节点的标号从左到右构成了产生式右部 β 。语法分析树的根节点的标号为文法开始符号。叶节点的标号既可以是非终结符，也可以是终结符。从左到右排列叶节点的符号就可以得到一个句型，称为是这棵树的产出（yield）或边缘（frontier）。

分析树是推导的图形化表示。给定一个推导 $S \Rightarrow \alpha_1 \Rightarrow \alpha_2 \Rightarrow \dots \Rightarrow \alpha_n$ ，对于推导过程中得到的每一个句型 α_i ，都可以构造出一个边缘为 α_i 的分析树。

例 2.6 一个简化版本的算术表达式文法。

- (1) $E \rightarrow E + E$
 - (2) $E \rightarrow E * E$
 - (3) $E \rightarrow (E)$
 - (4) $E \rightarrow \text{id}$
- (2.14)

$\text{id} + (\text{id} + \text{id})$ 是该文法的一个句子，其推导过程如下：

$$E \Rightarrow E + E$$

$$\begin{aligned}
&\Rightarrow \mathbf{id} + E \\
&\Rightarrow \mathbf{id} + (E) \\
&\Rightarrow \mathbf{id} + (E + E) \\
&\Rightarrow \mathbf{id} + (\mathbf{id} + E) \\
&\Rightarrow \mathbf{id} + (\mathbf{id} + \mathbf{id})
\end{aligned} \tag{2.15}$$

我们来看一下推导过程的树形表示（如图 2-4 所示）。我们从文法的开始符号 E 开始推导。初始的时候，分析树中只有一个节点，就是根节点，它对应着文法的开始符号 E ，如图(a)所示。

接下来，我们应用产生式(1)，从文法开始符号 E 推出 $E+E$ ，即 $E \Rightarrow E+E$ 。相应的，对分析树的根节点也应用第①条产生式，于是添加 3 个子节点，分别是“ E ”、“ $+$ ”和“ E ”，得到一棵新的分析树，树的边缘对应句型 $E+E$ ，如图(b)所示。

接下来，我们根据产生式(4)，将当前句型中的最左边的非终结符 E 替换成 $\mathbf{id}$ ，即 $E+E \Rightarrow \mathbf{id} + E$ 。相应的，在分析树中对最左边的标号为 E 的叶节点也应用产生式(4)，于是添加一个子节点 $\mathbf{id}$ ，得到一棵新的分析树，树的边缘对应句型 $\mathbf{id} + E$ ，如图(c)所示。

以此类推，可以得到推到每一步对应的分析树，如图(d)、(e)、(f)、(g)所示。

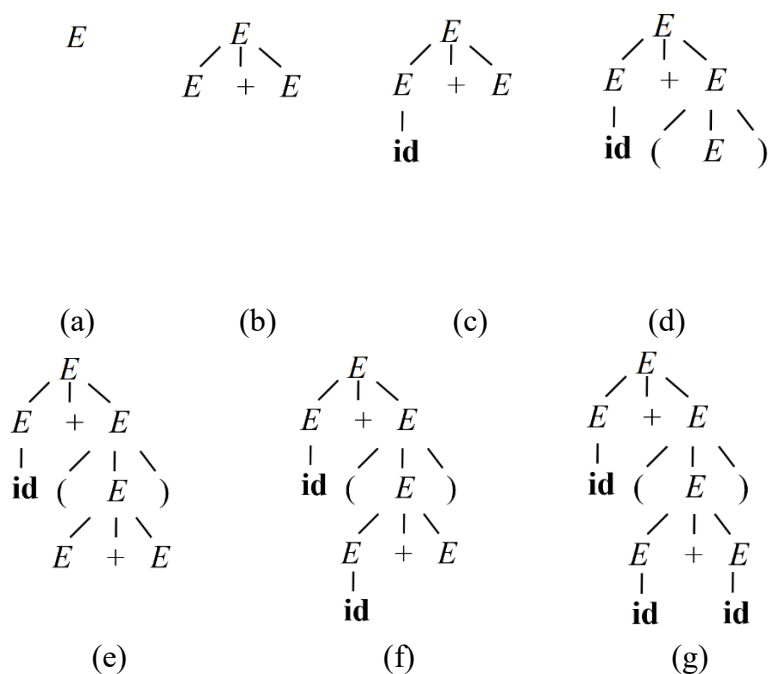


图 2-4 推导(2.15)的语法分析树序列

事实上，句子的推导并不是唯一的。例如， $\text{id} + (\text{id} + \text{id})$ 的另一种推导如下：

$$\begin{aligned}
 E &\Rightarrow E + E \\
 &\Rightarrow E + (E) \\
 &\Rightarrow E + (E + E) \\
 &\Rightarrow E + (E + \text{id}) \\
 &\Rightarrow E + (\text{id} + \text{id}) \\
 &\Rightarrow \text{id} + (\text{id} + \text{id})
 \end{aligned} \tag{2.16}$$

该推导最终得到的分析树与图(g)完全相同。还可以给出该句子的其它一些推导，如下方的推导(2.17)和推导(2.18)，其最终结果也对应图(g)所示的分析树。

$$\begin{aligned}
 E &\Rightarrow E + E \\
 &\Rightarrow E + (E) \\
 &\Rightarrow E + (E + E) \\
 &\Rightarrow E + (\text{id} + E) \\
 &\Rightarrow \text{id} + (\text{id} + E) \\
 &\Rightarrow \text{id} + (\text{id} + \text{id})
 \end{aligned} \tag{2.17}$$

$$\begin{aligned}
 E &\Rightarrow E + E \\
 &\Rightarrow \text{id} + E \\
 &\Rightarrow \text{id} + (E) \\
 &\Rightarrow \text{id} + (E + E) \\
 &\Rightarrow \text{id} + (E + \text{id}) \\
 &\Rightarrow \text{id} + (\text{id} + \text{id})
 \end{aligned} \tag{2.18}$$

□

尽管对句型中的非终结符的替换顺序不同，但是这些推导过程都对应着图 2.4(g) 中的句法分析树。可见，语法分析树忽略了替换句型中符号的不同顺序，也就是说，在推导和语法分析树之间具有多对一的关系。在推导(2.2)中，每次替换的是当前句型的最左边的非终结符。这样的推导称为最左推导（leftmost derivation），我们将其推导符号记为 $\Rightarrow_{lm}$ 。最左推导的逆过程称为最右归约（rightmost reduction），如下图所示(a)。在推导(2.3)中，每次替换的是当前句型的最右边的非终结符。这样的推导称为最右推导（rightmost derivation），我们将其推导符号记为 $\Rightarrow_{rm}$ 。最右推导的逆过程称为最左归约（leftmost reduction），如下图所示(b)。推导(2.4)和推导(2.5) 在非终结符的选择上没有什么规律和限制。

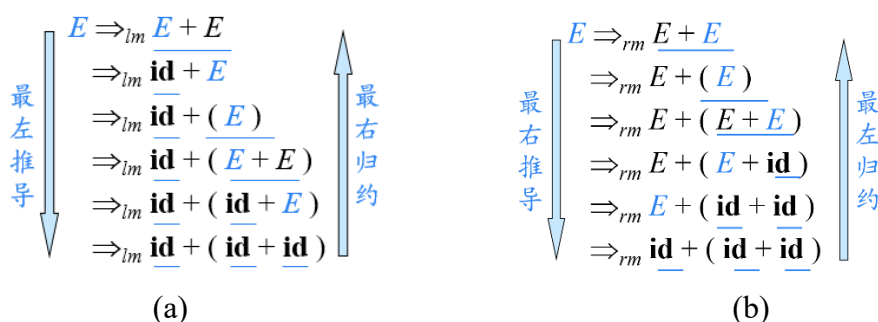


图 2-5 最左推导与最右归约、最右推导与最左归约举例

如果 $S \Rightarrow_{lm}^* \alpha$ ，则称 α 是当前文法的左句型（left-sentential form）。如果 $S \Rightarrow_{rm}^* \alpha$ ，则称 α 是当前文法的右句型（right-sentential form）。

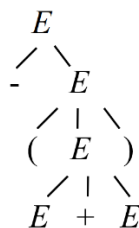
虽然一棵句法分析树可能对应多个推导过程，但是不难看出，在语法分析树和最左推导/最右推导之间存在一对一的关系。最左或最右推导都是以一种特定的顺序来替换句型中的符号。不难说明，每一棵语法分析树都和唯一的最左推导及唯一的最右推导相关联。

给定一个句型，其分析树中的每一棵子树的边缘称为该句型的一个短语（phrase）。如果子树只有父子两代节点（即子树的高度为 2），那么这棵子树的边缘称为该句型的一个直接短语（immediate phrase）。

例 2.7 对于文法

- $$\begin{aligned}
 (1) & E \rightarrow E + E \\
 (2) & E \rightarrow E * E \\
 (3) & E \rightarrow - E \\
 (4) & E \rightarrow (E) \\
 (5) & E \rightarrow id
 \end{aligned}
 \tag{2.19}$$

的句型 $-(E+E)$ 来说，其语法分析树如下图所示。

图 2-6 句型 $-(E+E)$ 的语法分析树

该分析树有 3 个内部节点，对应 3 个短语，分别是 $-(E+E)$ 、 $(E+E)$ 和 $E+E$ 。而高度为 2 的子树只有一棵，对应的直接短语为 $E+E$ 。□

由直接短语的定义可以看出，直接短语一定是某个产生式的右部，但反过

来，一个产生式的右部不一定是给定句型的直接短语。我们可以通过一个自然语言的例子来说明。

例 2.8 对于如下的文法：

- (1) $\langle \text{句子} \rangle \rightarrow \langle \text{动词短语} \rangle$
- (2) $\langle \text{动词短语} \rangle \rightarrow \langle \text{动词} \rangle \langle \text{名词短语} \rangle$
- (3) $\langle \text{名词短语} \rangle \rightarrow \langle \text{名词} \rangle \langle \text{名词短语} \rangle \mid \langle \text{名词} \rangle$
- (4) $\langle \text{动词} \rangle \rightarrow \text{提 高}$
- (5) $\langle \text{名词} \rangle \rightarrow \text{高 人} \mid \text{人 民} \mid \text{民 生} \mid \text{生 活} \mid \text{活 水} \mid \text{水 平}$ (2.20)

在该文法中，“高 人”、“人 民”、“民 生”、“生 活”、“活 水”、“水 平”都是名词。假如输入一个句子“提 高 人 民 生 活 水 平”，该句子对应的分析树如下：

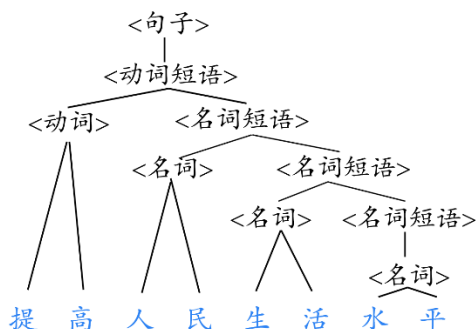


图 2-7 “提 高 人 民 生 活 水 平”的语法分析树

可以看出“人 民”、“生 活”、“水 平”分别是 3 棵高度为 2 的以 $\langle \text{名词} \rangle$ 为根结点的子树的边缘，它们都是这个句子的直接短语。而“高 人”、“民生”、“活 水”虽是也产生式右部，但是它们并不能构成该句子分析树中某棵子树的边缘，更不用提是高度为 2 的子树的边缘了，因此，它们不是该句子的直接短语。当然，它们在其它句型中可能会是一个直接短语。例如，“活 水”在“为 有 源 头 活 水 来”这个句子中是一个直接短语。可见，直接短语都是相对于一个句型而言的。一个产生式的右部是不是直接短语取决于当前的句型。这个性质对于第 4 章（语法分析）中讨论句柄的识别时会再次提到。

□

2.7 CFG 的二义性

如果一个文法可以为某个句子生成多棵分析树，则称这个文法是二义性的。

例 2.9 以下是一个简化版本的条件语句的文法：

$$\begin{array}{l}
 stmt \rightarrow \text{if } expr \text{ then } stmt \\
 \quad | \text{if } expr \text{ then } stmt \text{ else } stmt \\
 \quad | \text{other}
 \end{array} \quad (2.21)$$

这里, $stmt$ 表示句子, $expr$ 表示条件表达式, 前两个候选式表示条件语句, 第 3 个候选式 “other” 表示任何其它语句。条件语句有两种形式, 一种是带 **else** 的, 一种是不带 **else** 的。假如给定如下这样一个串

$$\text{if } E_1 \text{ then if } E_2 \text{ then } S_1 \text{ else } S_2 \quad (2.22)$$

这里 E_1 和 E_2 表示同一个非终结符, 只不过是为了便于讨论, 我们用下标来区分 E 在产生式不同位置的出现。该串具有如下图所示的两棵语法分析树。 □

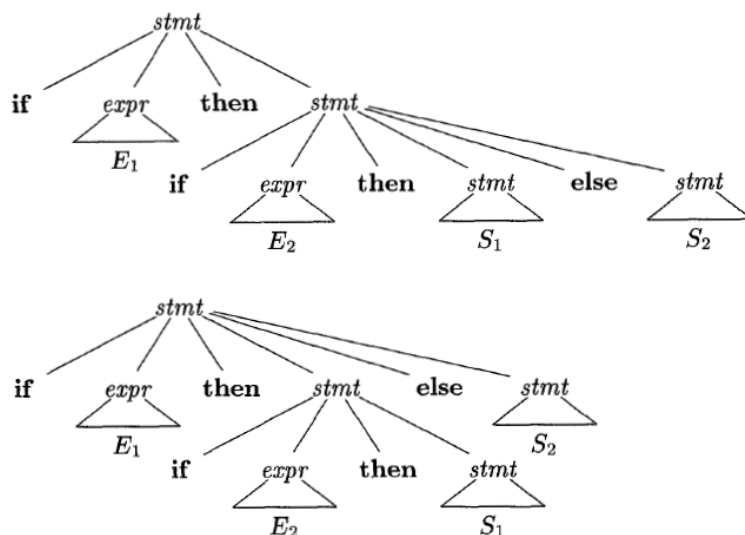


图 2-8 串 $\text{if } E_1 \text{ then if } E_2 \text{ then } S_1 \text{ else } S_2$ 的语法分析树

具有两棵以上语法分析树的符号串通常具有多个含义, 所以, 大部分语法分析器都期望文法是无二义性的, 否则当遇到歧义的时候, 机器不知道到底按照那种解释去理解这个句子。

在所有包含例 2.9 这种形式的条件语句的程序设计语言中, 都会采取就近匹配原则, 即 “每个 **else** 都与它前面最近的尚未匹配的 **then** 匹配”。因此, 总是会选择图 2-8 的第 1 棵语法分析树。从理论上讲, 这个消除二义性规则可以用一个文法直接表示, 但是在实践中很少用产生式来表示该规则。

例 2.10 我们可以将文法(2.21)改写成如下的无二义性文法。基本思想是: 根据就近匹配原则, 每个 **else** 都与它前面最近的尚未匹配的 **then** 匹配。也就是说, 一个 **then** 和一个 **else** 之间不会出现尚未匹配的语句。因此, 根据 **if** 语句中 **else** 与 **then** 的配对情况将其分为已配对的语句和尚未配对的语句两类。文法

(2.8)没有对这两个不同的概念加以区分，只是简单地将它们都定义为 *stmt*，从而导致该文法是二义性的。解决的办法很简单，就是通过引入新的非终结符来改造文法。

用非终结符 *unmatched_stmt* 表示尚未匹配的语句，用非终结符 *matched_stmt* 表示已匹配的语句。一个已匹配的语句要么是一个不包含尚未匹配语句的 if-then-else 语句，要么是一个非条件语句。我们可以使用下面这个文法。这个文法和前面的文法(2.8)生成同样的串集合，但是它只允许对串(2.9)进行一种语法分析，也就是将每个 else 和前面最近的尚未匹配的 then 匹配。

$$\begin{aligned}
 stmt &\rightarrow matched_stmt \\
 &\quad | \quad unmatched_stmt \\
 matched_stmt &\rightarrow \text{if } expr \text{ then } matched_stmt \text{ else } matched_stmt \\
 &\quad | \quad \text{other} \\
 unmatched_stmt &\rightarrow \text{if } expr \text{ then } stmt \\
 &\quad | \quad \text{if } expr \text{ then } matched_stmt \text{ else } unmatched_stmt
 \end{aligned} \tag{2.23}$$

□

2.8 本章小结

本章系统阐述了程序设计语言的形式化描述方法。首先回顾了语言的数学基础概念，随后形式化地定义了文法及其产生的语言。通过乔姆斯基文法分类体系，确立了上下文无关文法的核心地位。进而详细探讨了 CFG 的树形表示方法——分析树及其结构特性，并深入揭示了文法二义性这一关键问题，明确了其对语法分析的决定性影响。本章为后续词法与语法分析奠定了严格的理论基础，建立了从人类可读程序到机器可处理结构的数学桥梁。

本章要点如下：

(1) 基本概念

- 串：有穷符号序列
 - 串 s 的长度： $|s|$
 - 空串： ε
 - 串上的运算
 - 连接： xy
 - 幂： s^n
- 字母表：有穷符号集合

- 字母表上的运算
 - 乘积: $\Sigma_1 \Sigma_2$
 - 幂: Σ^n
 - 正闭包: $\Sigma^+ = \Sigma \cup \Sigma^2 \cup \Sigma^3 \cup \dots$
 - 克林闭包: $\Sigma^* = \Sigma^0 \cup \Sigma^+$

(2) 文法的定义

- $G = (V_T, V_N, P, S)$

(3) 语言的定义

- 语言: {句子}
 - 句子: 满足文法规则的终结字符串
- 语言上的运算
 - 并: $L_1 \cup L_2$
 - 连接: $L_1 L_2$
 - 幂: L^n
 - 正闭包: $L^+ = L \cup L^2 \cup L^3 \cup \dots$
 - 克林闭包: $L^* = L^0 \cup L^+$

(4) 文法的分类

- 0 型文法 (无限制文法/短语结构文法 PSG)
 - $\forall \alpha \rightarrow \beta \in P, \alpha$ 中至少包含 1 个非终结符
- 1 型文法 (上下文有关文法 CSG)
 - $\forall \alpha \rightarrow \beta \in P, |\alpha| \leq |\beta|$
 - 产生式的一般形式: $\alpha_1 A \alpha_2 \rightarrow \alpha_1 \beta \alpha_2 (\beta \neq \epsilon)$
- 2 型文法 (上下文无关文法 CFG)
 - $\forall \alpha \rightarrow \beta \in P, \alpha \in V_N$
 - 产生式的一般形式: $A \rightarrow \beta$
- 3 型文法 (正则文法 RG)
 - 右线性文法: $A \rightarrow wB$ 或 $A \rightarrow w$
 - 左线性文法: $A \rightarrow Bw$ 或 $A \rightarrow w$
 - 左线性文法和右线性文法都称为正则文法

第3章 词法分析

本章将带领读者深入编译器处理过程的第一个实质性阶段——词法分析。作为编译过程的“前线扫描器”，词法分析器负责将源程序的字符流转化为有意义的词法单元（token）序列，为后续的语法分析提供结构化的输入。我们将系统探讨如何用正则文法、正则表达式等形式化工具精确描述各类单词，并重点研究有穷自动机这一核心识别模型，涵盖其理论定义、转换算法与实际构造方法。最后，本章还将介绍词法分析器的自动生成技术，通过 Lex 等工具展示理论如何落地为高效、可靠的实践。

3.1 词法分析器的任务

词法分析器（也称为扫描器 scanner）的基本任务是识别单词。从左向右扫描每行源程序的字符，拼成单词，换成统一的内部表示 token 送给语法分析程序。

词法分析器的任务可具体分解为以下几个方面：

- (1) 组织源程序的输入；
- (2) 识别单词，并转换成二元式形式；
- (3) 过滤掉源程序中的注释、空格及无用符号（如回车、换行、字符常数的引号等）。它们在语法上通常没有意义，词法分析器会直接过滤掉它们，从而简化后续阶段的工作。
- (4) 发现并定位词法错误；
- (5) 行计数、列计数（用于定位词法错误）；
- (6) 列表打印源程序。

要想让计算机自动识别单词，我们首先需要告诉计算机单词长什么样子。因此，本章首先介绍单词的描述。

3.2 单词的描述

本节介绍几种用于描述单词的工具，分别是正则文法（regular grammar）、正则表达式（regular expression）和正则定义（regular definition）。

3.2.1 正则文法

我们在2.5节介绍过正则文法。如果把程序设计语言中的每类单词都看作一种语言，则多数程序设计语言单词的词法都能用正则文法来描述。词法是指语言中单词的组成规则。单词是定义在字符集上的字符的有穷序列。程序语言的字符集包括字母、数字、运算符、界限符等。例2.5是用正则文法描述标识符的例子。

例3.1 描述十进制常数的正则文法。

$$\begin{aligned} S &\rightarrow \text{digit} \mid \text{digit } A \\ A &\rightarrow \text{digit} \mid \text{digit } A \mid .B \mid \text{e}C \\ B &\rightarrow \text{digit} \mid \text{digit } D \\ D &\rightarrow \text{digit} \mid \text{digit } D \mid \text{e}C \\ C &\rightarrow +E \mid -E \mid E \\ E &\rightarrow \text{digit} \mid \text{digit } E \end{aligned}$$

其中，**digit**表示任意一个数字，**e**表示指数符号。 □

3.2.2 正则表达式

定义在字母表 Σ 上的正则语言也可以用该字母表上的正则表达式（**regular expression**——**RE**）描述。正则表达式是一种用来描述正则语言的更紧凑的表示方法。

正则表达式可以由较小的正则表达式按照特定规则递归地构建。每个正则表达式 r 表示一个语言，记为 $L(r)$ 。这个语言也是根据 r 的子表达式所表示的语言递归定义的。正则表达式中包括3中操作符：连接（ $\cdot$ ）、或（ $\mid$ ）和闭包（ $*$ ）。其中或运算也常用“+”表示。连接符“ $\cdot$ ”可以省略不写。

设 Σ 是一个字母表，则 Σ 上的正则表达式及其所表示的语言可递归地定义如下：

- (1) ε 是一个正则表达式，它表示的语言为 $\{\varepsilon\}$ ，即 $L(\varepsilon)=\{\varepsilon\}$ 。
- (2) 对于 $\forall a \in \Sigma$ ， a 是正则表达式，它表示的语言为 $\{a\}$ ，即 $L(a)=\{a\}$
- (3) 如果 r 和 s 都是正则表达式，分别表示语言 $L(r)$ 和 $L(s)$ ，则：
 - ① rs 是正则表达式， $L(rs)=L(r) L(s)$ ；

② $r|s$ 是正则表达式, $L(r|s) = L(r) \cup L(s)$;

③ r^* 是正则表达式, $L(r^*) = (L(r))^*$ 。

④ (r) 是正则表达式, $L(r) = L(r)$ 。

在正则表达式的3种运算中, 闭包运算的优先级最高, 连接运算次之, 或运算最低。

例3.2 令 $\Sigma = \{a, b\}$, 则

(1) $L(a|b) = L(a) \cup L(b) = \{a\} \cup \{b\} = \{a, b\}$ 。

(2) $L((a|b)(a|b)) = L(a|b) L(a|b) = \{a, b\} \{a, b\} = \{aa, ab, ba, bb\}$ 。

(3) $L(a^*) = (L(a))^* = \{a\}^* = \{\epsilon, a, aa, aaa, \dots\}$ 。

(4) $L((a|b)^*) = (L(a|b))^* = \{a, b\}^* = \{\epsilon, a, b, aa, ab, ba, bb, aaa, \dots\}$, 即由 a 和 b 构成的所有串的集合。

(5) $L(a|a^*b) = L(a) \cup L(a^*b) = \{a\} \cup (L(a^*) L(b)) = \{a\} \cup (\{a\}^* \{b\}) = \{a, b, ab, aab, aaab, \dots\}$ 。

(6) $L(a(a|b)^*(\epsilon|((\cdot|_)(a|b)(a|b)^*))) = \{a\} \{a, b\}^* (\{\epsilon\} \cup (\{., _\} \{a, b\} \{a, b\}^*))$ 。 □

例3.3 C语言无符号整数的正则表达式。

(1) 八进制整数的正则表达式: $0(1|2|3|4|5|6|7)(0|1|2|3|4|5|6|7)^*$

(2) 十进制整数的正则表达式: $(1|\dots|9)(0|\dots|9)^* | 0$

(3) 十六进制整数的正则表达式: $0x(1|\dots|9|a|\dots|f|A|\dots|F)(0|\dots|9|a|\dots|f|A|\dots|F)^*$

□

可以用正则表达式定义的语言叫做正则集合 (regular set) 或正则语言。如果两个正则表达式 r 和 s 表示同样的语言, 则称 r 和 s 等价 (equivalent), 记作 $r=s$ 。例如 $(a|b)=(b|a)$ 。

例3.2(2)中, 可表示同样语言的另一个正则表达式是 $aa|ab|ba|bb$, 即 $(a|b)(a|b)=aa|ab|ba|bb$ 。例3.2(4)中, 可表示同样语言的另一个正则表达式是 $(a^*|b^*)^*$, 即 $(a|b)^*=(a^*|b^*)^*$ 。

正则表达式遵循一些代数定律。设 e_1 、 e_2 、 e_3 为正则表达式, 下述各等式成立:

(1) 交换律

$$e_1|e_2=e_2|e_1 \quad (3.1)$$

(2) 结合律

$$\begin{aligned}
 e_1(e_2e_3) &= (e_1e_2)e_3 \\
 e_1|(e_2|e_3) &= (e_1|e_2)|e_3
 \end{aligned} \tag{3.2}$$

(3) 分配律

$$\begin{aligned}
 e_1(e_2|e_3) &= e_1e_2|e_1e_3 \\
 (e_1|e_2)e_3 &= e_1e_3|e_2e_3
 \end{aligned} \tag{3.3}$$

(4) 其它

$$\begin{aligned}
 \varepsilon e &= e\varepsilon = e \\
 e^* &= (e|\varepsilon)^* \\
 (e^*)^* &= e^* \\
 e^* &= e^+|\varepsilon \\
 e^+ &= e^*e = ee^*
 \end{aligned} \tag{3.4}$$

可以证明，正则文法与正则表达式之间存在着等价性。即，对任何正则文法 G ，存在正则表达式，使得 $L(r)=L(G)$ ；对任何正则表达式 r ，存在正则文法 G ，使得 $L(G)=L(r)$ 。因此，正则集合也称为正则语言。

3.2.3 正则定义

为方便表示，可以给某些正则表达式命名，并在之后的正则表达式中像使用符号一样使用这些名字。这种表示方法称为正则定义（**regular definition**）。如果 Σ 是基本符号的集合，那么一个正则定义是具有如下形式的定义序列：

$$\begin{aligned}
 d_1 &\rightarrow r_1 \\
 d_2 &\rightarrow r_2 \\
 &\dots \\
 d_n &\rightarrow r_n
 \end{aligned} \tag{3.5}$$

其中，每个 d_i 都是一个新符号，它们都不在字母表 Σ 中，而且各不相同；每个 r_i 是字母表 $\Sigma \cup \{d_1, d_2, \dots, d_{i-1}\}$ 上的正则表达式。

例3.4 标识符的正则定义

$$\begin{aligned}
 digit &\rightarrow 0|1|2|\dots|9 \\
 letter &\rightarrow A|B|\dots|Z|a|b|\dots|z
 \end{aligned}$$

$$id \rightarrow letter(letter|digit)^* \quad (3.6)$$

在这个定义中， $0|1|2|\dots|9$ 是一个正则表达式，我们给它起名字叫 $digit$ 。 $A|B|\dots|Z|a|b|\dots|z$ 也是一个正则表达式，我们给它起名字叫 $letter$ 。在第3个正则表达式中，我们像使用字母表中的符号那样使用前面定义的这两个名字，该表达式表示的语言是所有以字母开头的字母数字串组成的集合，这实际上表示的是标识符，因此给这个正则表达式起名字叫 id （标识符）。□

例3.5 十进制整数/实数的表示

$$\begin{aligned} digit &\rightarrow 0|1|2|\dots|9 \\ digits &\rightarrow digit\,digit^* \\ optionalFraction &\rightarrow .digits|\,\varepsilon \\ optionalExponent &\rightarrow (E(+|-|\,\varepsilon)digits)|\,\varepsilon \\ number &\rightarrow digits\,optionalFraction\,optionalExponent \end{aligned} \quad (3.7)$$

在这个定义中，根据第1个正则表达式， $digit$ 表示的是一个数字。根据第2个正则表达式， $digits$ 表示的是一个长度大于等于1的数字串。根据第3个正则表达式， $optionalFraction$ 表示的是在小数点后面加一个长度大于等于1的数字串，另外，它还可以为空，因此表示可选的小数部分。根据第4个正则表达式， $optionalExponent$ 表示的是在E后面连接一个正号或负号或空串后再连接一个长度大于等于1的数字串，另外，它还可以为空，因此表示可选的指数部分。根据第5个正则表达式， $number$ 表示的是在一个长度大于等于1的数字串后面连接一个可选的小数部分，再连接一个可选的指数部分。□

3.3 单词的识别

如果把程序设计语言中的每类语言都看作一种语言，则多数程序设计语言单词的词法都能用正则文法（正则表达式）来描述。根据形式语言与自动机理论，正则文法（正则表达式）定义的语言都可以用有穷自动机来识别。接下来将在3.3.1节介绍有穷自动机的相关技术，在3.3.2节介绍如何根据正则表达式构造有穷自动机。

3.3.1 有穷自动机

有穷自动机（finite automata, FA）是对一类处理系统建立的数学模型。这

类系统具有一系列离散的输入输出信息，同时具有有穷数目的内部状态。系统的状态概括了对过去输入信息处理的状况。系统只需要根据当前所处的状态和当前面临的输入信息就可以决定系统的后继行为。每当系统处理了当前的输入后，系统的内部状态也将发生改变。

电梯控制装置是 FA 的一个典型例子。系统的输入为顾客乘梯需求（即所要到达的层号），系统的状态为电梯所处的层数及运动方向。电梯控制装置并不需要记住先前全部的服务要求，只需要知道电梯当前所处的状态，以及还没有满足的所有服务请求。

FA 模型是由两位神经物理学家 McCulloch 和 Pitts 在 1948 首先提出来的。他们是在研究大脑神经功能时提出这一模型的。FA 模型如图 3-1 所示。

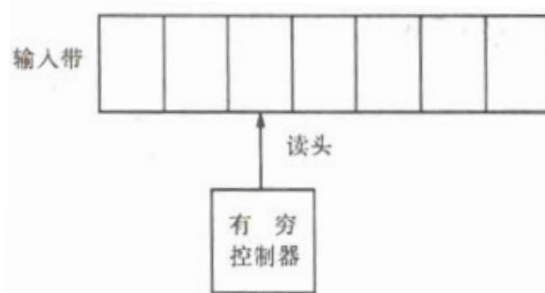


图3-1 FA模型

模型由一条有穷长度的输入带、一个读头和一个有穷控制器组成。在这个模型中，单个的输入信息被表示为一个符号，称为输入符号。输入带用来存放输入符号串，每个输入符号占据一个单元（方格），输入带的长度和输入串的长度相同。读头从左向右逐个读取输入符号，不能修改带符号（只读）、不能往返移动。有穷控制器具有有穷个状态数，根据当前的状态和当前输入符号控制转入下一个状态。

同各种具体机器一样，FA 也有它的初始状态和终止状态（也称为接收状态）。在初始状态下，读头指向输入带的最左单元，准备读入第一个输入符号。终止状态可以有若干个，表示输入串接收状态。如果读头在读取输入带上最后一个符号时，恰好进入某个终止状态，则宣布接收该输入串；否则，不接收。

（1）有穷自动机的表示

通常用转换图（transition graph）表示 FA。转换图是一种有向图，图中的节点对应于 FA 的状态。其中，初始状态由 *start* 箭头指向，终止状态用双圈表示。如果对于输入 *a*，存在一个从状态 *p* 到状态 *q* 的转换，就在转换图中画一

条有向边，并标记上 a 。如果存在一个对应于输入串 x 的从开始状态到某个接收状态的转换序列，则称串 x 被 FA 接收。

例 3.6 图 3-2 给出一个 FA 转换图的例子。该自动机有编号为 0~3 的 4 个状态。其中状态 0 由 *start* 箭头指向，表示初始状态。状态 3 为终止状态，用双圈表示。它识别输入串的过程如下：开始时处于状态 0。如果输入带上第 1 个输入符号是 a ，则读入该符号后转入终态 1；如果输入符号是 b ，则读入该符号后依然处于状态 0。在状态 1 下如果输入符号是 a ，则读入该符号后依然处于状态 1；如果输入符号是 b ，则读入该符号后转入状态 2。在状态 2 下如果输入符号是 a ，则读入该符号后转入状态 1；如果输入符号是 b ，则读入该符号后转入终止状态 3。此时如果输入带上没有符号了，则成功接收；如果输入带上下一个输入符号是 a ，则读入该符号后转入状态 1；如果输入带上下一个输入符号是 b ，则读入该符号后转入状态 0。

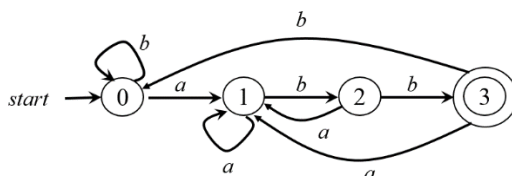


图3-2 FA转换图的例子

□

我们也可以将一个自动机表示为转换表（**transition table**）的形式。表的每一行对应于一个状态，每一列对应于一个输入符号。行与列的交汇处表示当前状态遇到输入符号后转入的后继状态。例如，对于图 3-2 中的自动机，其转换表如下图所示。

输入 状态	a	b
0	1	0
1	1	2
2	1	3
3	1	0

图 3-3 对应于图 3-2 的 FA 的转换表

（2）有穷自动机的形式化定义

参照文法的形式定义方法，可以把有穷自动机 M 定义为下面的五元组形式：

$$M=(S, \Sigma, \delta, s_0, F) \quad (3.8)$$

其中，

$S \rightarrow$ 有穷状态集合

$\Sigma \rightarrow$ 输入符号集合，本质上是一个字母表。

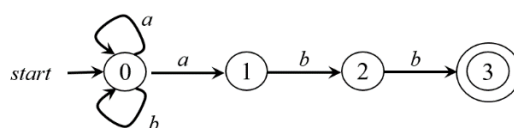
$\delta \rightarrow$ 转换函数。 $\delta: S \times \Sigma \rightarrow 2^S$ (2^S 是 S 的幂集)

$s_0 \rightarrow$ 唯一的初始状态, $s_0 \in S$

$F \rightarrow$ 终止状态集合。 $F \subseteq S$

如果 δ 函数是单值的, 即每次转向的状态是唯一的, 则这样的有穷自动机称为是确定的有穷自动机 (deterministic finite automata, DFA)。否则, 如果 δ 函数是不唯一, 而是一个状态集合的话, 这样的有穷自动机称为是不确定的有穷自动机 (nondeterministic finite automata, NFA)。

图 3-2 是 DFA 的例子, 下图给出一个 NFA 的例子。



(a) 转换图

输入 状态	a	b
0	{0, 1}	{0}
1	$\emptyset$	{2}
2	$\emptyset$	{3}
3	$\emptyset$	$\emptyset$

(b) 转换表

图3-4 一个NFA的例子

可见, 转换图和转换表实际上是将转换函数以图和表的形式表示。如果我们使用转换表来表示一个 NFA, 那么表中的每一个表项就是一个状态集合。如果转换函数没有给出对应于某个状态-输入对的信息, 就把空集放入转换表相应的表项中。如果我们使用转换表来表示一个 DFA, 那么表中的每一个表项就是一个状态。因此, 我们可以不适用花括号, 直接写出这个状态, 因为花括号只是用来说明表项的内容是一个集合。

(3) 有穷自动机接受 (识别) 的语言

由一个有穷自动机 M 接收的所有串构成的集合称为是该 FA 接受 (或识别) 的语言, 记为 $L(M)$ 。

DFA 在工作过程中遵循最长子串匹配原则, 即, 当输入串的多个前缀与一个或多个模式匹配时, 总是选择最长的前缀进行匹配。例如, 假设读入的两个字符为 $\leq$, 则第一个字符可与模式 $<$ 相匹配, 但 $<$ 不是能够匹配输入字符串的最长模式, 因此, $\leq$ 将被识别为一个单词 LE, 而不是两个单词 LT 和 EQ。因此, DFA 在到达某个终态之后, 只要输入带上还有符号, DFA 就继续前进, 以

便寻找尽可能长的匹配。

例如，假设读入的前两个字符为 $\leq$ ，

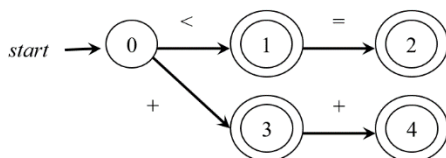


图 3-5 识别 $<$ 、 $\leq$ 、 $+$ 、 $++$ 的 DFA

(4) NFA 和 DFA 的等价性

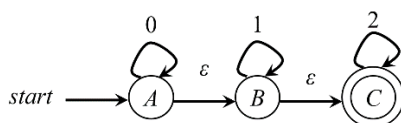
NFA 和 DFA 具有等价性。因为每个 DFA 都是一个 NFA，显然，NFA 接受的语言类将包含正规集合（即被 DAF 接受的语言）；反过来，对任何 NFA N ，存在识别同一语言的 DFAD，即 NFA 可以等价地转换成 DFA。由于篇幅所限，在此不对它们的等价性进行证明。

事实上，图 3-2 所示的 DFA 与图 3-4 所示的 NFA 识别的是同一种语言。图 3-4 所示的 NFA 的转换图可以很直观地显示该 NFA 识别已 abb 结尾的由符号 a 和 b 构成的串，即 $(a|b)^*abb$ 。通过分析图 3-2 所示的 DFA 可以看出，状态 0 为初始状态，不管再来多少个 b ，它都处于状态 0，直到某一时刻接收到一个符号 a 了，此时进入状态 1。状态 1 表示当前接收到的串以一个 a 结尾，因此，此后不管再来多少个 a ，它都处于状态 1，直到某一时刻接收到一个符号 b 了，此时进入状态 2。状态 2 表示当前接收到的串以 ab 结尾。此时如果再接收到一个 a ，就破坏了串以 ab 结尾这一条件，因此，就退回到状态 1，表示当前接收到的串以一个 a 结尾。如果在状态 2 接收到一个 b ，就进入状态 3。状态 3 表示当前接收到的串以 abb 结尾。状态 3 是终止状态，此时如果不再有新的符号出现，则成功接收一个以 abb 结尾的串。如果再有新的 a 或 b 出现，则分别进入状态 1 和状态 0。由此可见，这个 NFA 和 DFA 接收或识别的是同一种语言，即以 abb 结尾的串。如果用正则表达式来表示，就是 $(a|b)^*abb$ 。

通过对比图 3-2 和图 3-4 可以看出，NFA 更直观。但是从自动分析的角度，希望确定性（即每一步的操作都是确定的），因此 DFA 更实用。

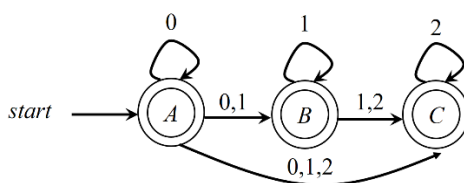
(5) 带有 ε -边的 NFA

事实上，NFA 的有向边上除了输入字母表中的符号以外，还可以标记为空串，这样的 NFA 称为是带有 ε -边的 NFA，其转换函数改为 $\delta: S \times (\Sigma \cup \{\varepsilon\}) \rightarrow 2^S$ 。下图给出一个带有 ε -边的 NFA 的例子。

图 3-6 一个带有 ϵ -边的 NFA 的例子

状态 A 到状态 B 之间有一条标记为 ϵ 的有向边，这表示状态 A 不需要遇到任何符号就可以直接进入状态 B 。但是状态 A 与状态 B 不是等价的。在状态 A 中还可以继续接受符号 0 ，但是一旦进入状态 B 中，将不再接受符号 0 。该自动机接受的语言 $L(M)=\{\text{由若干个 } 0 \text{ 连接若干个 } 1 \text{ 再连接若干个 } 2 \text{ 构成的串}\}$ ，用正则表达式表示即 $0^*1^*2^*$ 。

事实上，带有 ϵ -边和不带有 ϵ -边的 NFA 具有等价性。也就是说，给定一个带 ϵ -边的 NFA，我们可以构造出一个与它等价的不带 ϵ -边的 NFA。例如，对于上图所示的有 ϵ -边的 NFA，我们可以构造出与它等价的不带 ϵ -边的 NFA，如下图所示。

图 3-7 与图 3-6 等价的不带有 ϵ -边的 NFA

在这个 NFA 中，状态 A 表示当前接收到的串由若干个 0 构成，状态 B 表示当前接收到的串由若干个 0 连接若干个 1 构成（进入状态 B 后就不再接收 0 ），状态 C 表示当前接收到的串由若干个 0 连接若干个 1 再连接若干个 2 构成（进入状态 C 后就不再接收 0 和 1 ），用正则表达式表示也是 $0^*1^*2^*$ 。

通过对比这两个图可以看出，带 ϵ -边的 NFA 更直观，也更容易构造。

（6）DFA 的算法实现

前面提到，从自动分析的角度，DFA 更实用。下边的算法说明了如何将 DFA 用于串的识别。

算法 3.7： DFA 的实现算法。

输入： 以文件结束符 **eof** 结尾的字符串 x 。DFA D 的开始状态 s_0 ，接受状态集 F ，转换函数 $move$ 。

输出： 如果 D 接受 x ，则回答 “yes”，否则回答 “no”。

方法： 将下述算法应用于输入串 x 。函数 $nextChar()$ 返回输入串 x 的下一个符号。函数 $move(s, c)$ 表示从状态 s 出发，沿着标记为 c 的边所能到达的状态。

```
s = s0;  
c = nextChar();  
while ( c != eof ) {  
    s = move ( s, c );  
    c = nextChar();  
}  
if ( s 在 F 中 ) return "yes";  
else return "no";
```

图3-8 DFA的算法实现

该算法首先将 DFA 的开始状态 s_0 赋给变量 s , s 表示当前状态。然后将输入串中的下一个符号赋给变量 c 。只要 c 不等于文件结束符 **eof** 就循环以下操作, 也就是以当前状态 s 和输入符号 c 作为参数调用 *move* 函数。函数返回值是状态 s 相对于输入符号 c 的后继状态。移入该状态, 并调用 *nextChar* 函数读入下一个输入符号并赋给 c 。该循环直到遇到文件结束符 **eof** 为止。此时如果当前状态是一个终态, 则成功接受, 否则接收失败。

可见, DFA 的总控程序非常简单, 主要由一个循环构成。其中最核心的语句是 *move* 函数。*move* 函数模拟的是转换函数 δ 。转换函数 δ 在计算机中实际是以表的形式存储, *move* 函数实际上是一个查表函数, 根据当前状态 s 和输入符号 c , 查表可以得到后继状态, 即函数的返回值。□

(7) FA 与 RE 的等价性

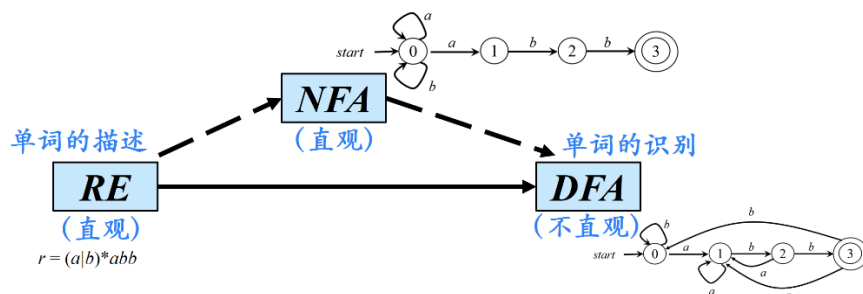
有穷自动机与正则表达式之间存在着等价性, 即, 对任何正则表达式 r , 存在有穷自动机 M , 使得 $L(M)=L(r)$; 对任何有穷自动机 M , 存在正则表达式 r , 使得 $L(r)=L(M)$ 。受篇幅所限, 这里不对它们的等价性进行证明。感兴趣的读者可以参考有关形式语言与自动机理论方面的书籍。

3.3.2 从正则表达式到有穷自动机

前面提到, 程序设计语言的多数单词都可以用正则文法来描述。而正则表达式是一种与正则文法等价的用来描述正则语言的更直观更紧凑的方法, 因此非常适合在用来描述单词。但是在构造词法分析器时, 我们真正实现或模拟的是 DFA。前面已经介绍过, 正则表达式与有穷自动机 (包括 DFA) 之间存在着等价性。也就是说。只要我们能够用正则表达式来描述一种单词, 我们就一定能构造出一个与它等价的 DFA 来识别这种单词。问题的关键是如何将用于描述单词构成的正则表达式 RE 转换为等价的 DFA, 从而实现对单词的识别。

将 RE 直接转换为 DFA 是比较困难的。我们在前面提到, DFA 本身并不像

正则表达式那样，是一种非常直观的表达形式。我们很难从给定的正则表达式一步到位地构造出与之等价地DFA。但是我们也看到，NFA相比于DFA是非常直观的。事实上，NFA跟正则表达式之间存在着一定的对应关系的。例如，图3-9中的正则表达式 $r = (a|b)^*abb$ 主要由4部分构成，第一部分是一个由a和b构成的串，后面三个部分分别对应的三个输入符号a、b、b。而图中的NFA的4个状态0、1、2、3分别跟正则表达式的4个部分对应。因此，给定一个正则表达式，我们去构造跟它相对应的NFA是非常容易的。前面我们也提到，NFA和DFA具有等价性。事实上，我们可以通过一种称为“子集构造法”的技术，让计算机自动地将NFA转化成与之等价的DFA。



这就相当于在RE和DFA之间引入NFA这样一个“桥梁”。也就是说，从RE到DFA的转化分为两步进行：首先，对于一个RE，先把它转换为等价的NFA；接下来，再利用一种称为“子集构造法”的技术将NFA转换为等价的DFA。

（1）从正则表达式构造NFA

这个“自动化”的理论引擎由1983年图灵奖得主肯·汤普森（Ken Thompson）提供。在1968年那篇名为《正则表达式搜索算法》（Regular expression search algorithm）的里程碑式论文中，汤普森展示了一个简洁而优美的算法，可以将任何正则表达式确定性地编译为一个等价的非确定性有限自动机（NFA）。

设 r 、 r_1 、 r_2 分别表示正则表达式， q_0 表示FA的初态， q_f 为终态。从正则表达式到NFA的转换算法如下：

①如果 $r = \varepsilon$ ，则 r 对应的FA中分别有一个初始状态和一个终止状态，二者之间的有向边上标记着符号 ε ，表示初始状态在未接收到任何符号的情况下可以

直接进入终止状态，即接收了一个空串，如图3-10所示。

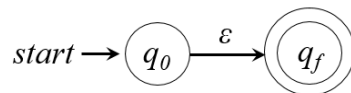


图3-10 正则表达式 ε 对应的NFA

②如果 $r = a$, $a \in V_T$, 则 r 对应的NFA中，在初态和终态之间的有向边上标记着符号 a ，如图3-11所示。

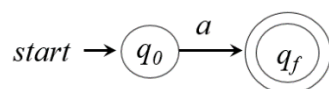


图3-11 正则表达式 a 对应的FA

③如果 $r = r_1 r_2$, 则 r 对应的NFA如图3-12所示。

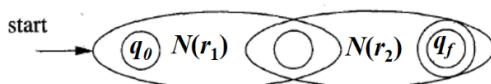


图3-12 正则表达式 $r_1 r_2$ 对应的FA

④ 如果 $r = r_1 | r_2$, 则 r 对应的NFA如图3-13所示。

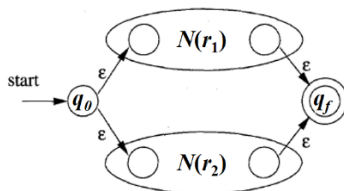


图3-13 正则表达式 $r_1 | r_2$ 对应的FA

⑤ 如果 $r = (r_1)^*$, 则 r 对应的NFA如图3-14所示。

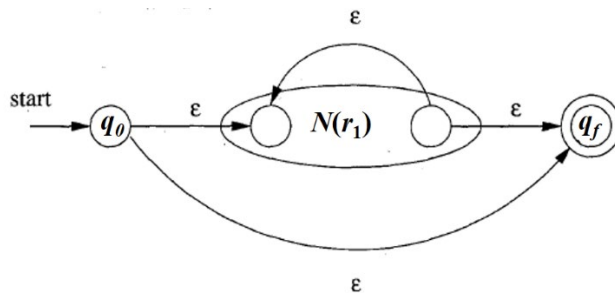


图3-14 正则表达式 $(r_l)^*$ 对应的FA

例3.8 识别C语言各进制无符号整数的NFA.

① 十进制整数（DEC）的正则表达式为：

$$(1|...|9)(0|...|9)^*|0$$

其NFA如图3-15所示。

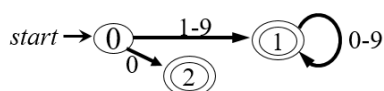


图3-15 识别十进制整数的NFA

② 八进制整数（OCT）的正则表达式为：

$$0(1|2|3|4|5|6|7)(0|1|2|3|4|5|6|7)^*$$

其NFA如图3-16所示。

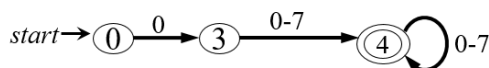


图3-16 识别八进制整数的NFA

③ 十六进制整数（HEX）的正则表达式为：

$$0x(1|...|9|a|...|f|A|...|F)(0|...|9|a|...|f|A|...|F)^*$$

其NFA如图3-17所示。

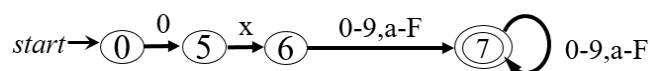


图3-17 识别十六进制整数的NFA

可以将以上3个转换图合并，构成图3-18所示的识别C语言无符号整数的NFA。

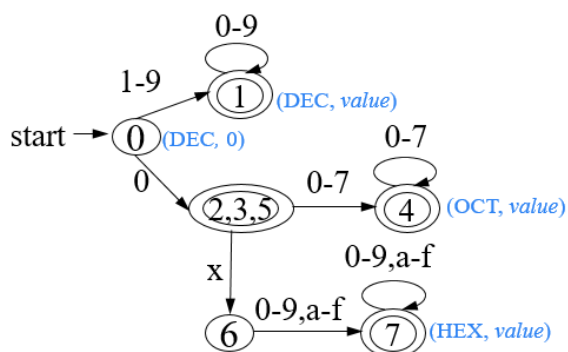


图3-18 识别C语言无符号整数的NFA

□

(2) 将NFA转化为DFA

子集构造法的基本思想是让构造得到的DFA的每个状态对应于NFA的一个状态集合。DFA在读入输入 $a_1a_2\cdots a_n$ 之后到达的状态对应于相应NFA从开始状态出发，沿着以 $a_1a_2\cdots a_n$ 为标号的路径能够到达的状态的集合。

我们先通过一个例子来演示一下子集构造法。对于例3.5给出的十进制整数/实数的正则定义，我们可以构造出它的NFA，如图3-19所示。

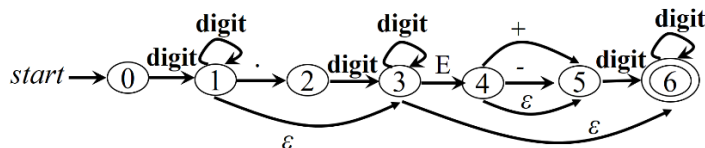


图 3-19 识别无符号十进制整数/实数的NFA

接下来，我们构造与之等价的DFA。如图3-20所示。

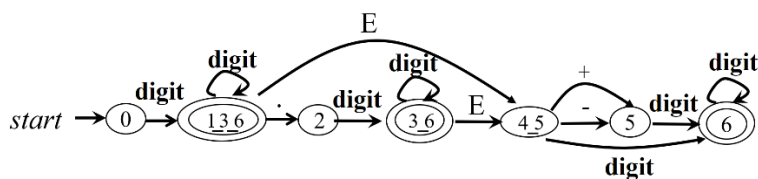


图 3-20 识别无符号十进制整数/实数的DFA

下面我们阐述一下子集构造法的基本原理。根据图3-20所示的NFA，初始状态0在遇到第1个符号（即数字`digit`）时，既可以进入状态1，又可以进入状态3，还可以进入状态6，具体是哪一个我们现在还不能确定。既然不能确定，我们就先不去确定它。等读入第2个符号，获得足够多信息后再做出正确的选择。我们先把能确定下来的事情确定下来。那么什么事情是能确定下来的呢？

那就是状态0在遇到数字`digit`时，肯定会进入状态1、3、6三者中的一个（绝对不会是2或4或6）。因此，我们在DFA中构造一个新的状态，它是由NFA状态1、3、6构成的集合。注意：这个DFA状态（它是一个由若干NFA状态构成的集合）并不是表示NFA的状态0在遇到数字`digit`时，会“同时”进入状态1、3、6，而是表示进入三者中的一个。具体是哪一个状态，可以根据下一个（即第2个）输入符号来判断。如果第2个符号是小数点，说明此时进入的是1，因为，在NFA中，只有状态1可以接收小数点，状态3和6并不接收小数点。如果第2个符号是E，说明此时进入的是1或3，因为，在NFA中，状态1或3可以接收E，但状态6并不接收E。如果第2个符号是数字`digit`，说明此时进入的是1或3或6，因为，在NFA中，状态1或3或6都可以接收数字`digit`。因此，在DFA中，虽然状态1_3_6后面有一条通往状态2的标记为“点”的弧，但并不表明NFA的状态3和6接收“点”，事实上，这里的“点”弧是由状态1“贡献”的。同理，虽然状态1_3_6后面有一条通往状态4_5的E弧，但并不表明NFA的状态6接收E，事实上，这里的E弧是由状态1和3“贡献”的。

最后再强调一下子集构造法的本质，实际上是通过生成一个新的DFA状态来推迟决定，等读入了足够多的输入、获得足够信息后再做出正确的选择。

算法3.9 由 NFA构造DFA的子集构造（subset construction）算法。

输入：一个NFA N 。

输出：一个接受同样语言的DFA D 。

方法：我们的算法为 D 构造个转换表 D_{ran} 。 D 的每个状态是个NFA状态集合，我们将构造 D_{ran} ，使得 D “并行地”模拟 N 在遇到一个给定输入串时可能执行的所有动作。我们面对的第一个问题是正确处理 N 的 ϵ 转换。在表3-1中我们可以看到一些函数的定义。这些函数描述了一些需要在这个算法中执行的 N 的状态集上的基本操作。请注意， s 表示 N 的单个状态，而 T 代表 N 的一个状态集。

操作	描述
$\epsilon\text{-closure}(s)$	能够从NFA的状态 s 开始只通过 ϵ 转换到达的NFA状态集合
$\epsilon\text{-closure}(T)$	能够从 T 中某个NFA状态 s 开始只通过 ϵ 转换到达的NFA状态集合，即 $\bigcup_{s \in T} \text{closure}(s)$
$\text{move}(T, a)$	能够从 T 中某个状态 s 出发通过标号为 a 的转换到达的NFA状态的集合

图 3-21 NFA状态集上的操作

我们必须找出当 N 读入某个输入串之后可能位于的所有状态集合。首先，在读入第一个输入符号之前， N 可以位于集合 $\epsilon\text{-closure}(s_0)$ 中的任何状态上，其中 s_0 是 N 的开始状态。下面进行归纳。假定 N 在读入输入串 x 之后可以位于集合 T 中的状态上。如果下一个输入符号是 a ，那么 N 可以立即移动到集合 $\text{move}(T,a)$ 中的任何状态。然而， N 可以在读入 a 后再执行几个 ϵ 转换，因此 N 在读入 xa 之后可以位于 $\epsilon\text{-closure}(\text{move}(T,a))$ 中的任何状态上。根据这些思想，我们可以得到图3-22中显示的方法，该方法构造了 D 的状态集合 $Dstates$ 和 D 的转换函数 $Dtran$ 。

```

初始时，  $\epsilon\text{-closure}(s_0)$ 是  $Dstates$  中的唯一状态，且它未加标记；
while (在  $Dstates$  中有一个未标记状态  $T$ )
{
    给  $T$  加上标记；
    for (每个输入符号  $a$ )
    {
         $U = \epsilon\text{-closure}(\text{move}(T,a))$ ；
        if ( $U$  不在  $Dstates$  中)
            将  $U$  加入到  $Dstates$  中，且不加标记；
         $Dtran[T,a] = U$ ；
    }
}

```

图3-22子集构造法

D 的开始状态是 $\epsilon\text{-closure}(s_0)$ ， D 的接受状态是所有至少包含了 N 的一个接受状态的状态集合。我们只需要说明如何对NFA的任何状态集合 T 计算 $\epsilon\text{-closure}(T)$ ，就可以完整地描述子集构造法。这个计算过程显示在图3-23中。它是从一个状态集合开始的一次简单的图搜索过程，不过此时假设这个图中只存在标号为 ϵ 的边。 □

```

将 $T$ 的所有状态压入 $stack$ 中；
将 $\epsilon\text{-closure}(T)$ 初始化为 $T$ ；
while ( $stack$ 非空)
{
    将栈顶元素 $t$ 弹出栈中；
    for (每个满足如下条件的 $u$ ：从 $t$ 出发有一个标号为 $\epsilon$ 的转换到达状态 $u$ )
        if ( $u$ 不在 $\epsilon\text{-closure}(T)$ 中)
        {
            将 $u$ 加入到 $\epsilon\text{-closure}(T)$ 中；
            将 $u$ 压入栈中；
        }
}

```


图3-23 计算 ϵ -closure (T)

3.4 词法分析阶段的错误处理

如果编译器只处理正确的程序，那么它的设计和实现将会大大简化。但是，人们还期望编译器能够帮助程序员定位和跟踪错误。因为不管程序员如何努力，程序中难免会有错误。

因为词法分析器不能从全局的角度考察源程序，所以能在词法层发现的错误是有限的。比如词法分析器在输入的程序中第一次遇到 **begen**，它无法区分 **begen** 究竟是关键字 **begin** 的错误拼写还是一个标识符。由于 **begen** 是合法的标识符，词法分析器必须返回这个标识符词法单元，而让编译器的其它阶段（该例子里是语法分析器）去处理这种错误。

再如，对于 C 程序片段

```
int i = 0x3G; float j = 1.05e;
```

0x3G 为错误的十六进制数，1.05e 为错误的指数形式的浮点数。但是这类错误未必都是在词法分析阶段处理。对于 0x3G 和 1.05e 这样的字符串，根据自动机工作中所要遵循的最长子串匹配原则，词法分析器会将 0x3 识别为十六进制，G 为标识符；将 1.05 识别为浮点类型，e 为标识符。在语法分析阶段根据语法规则可以检测出这类单词拼写错误。也就是说词法错误未必都是在词法分析阶段识别。

有时会出现由于剩余输入的前缀不能和任何单词的模式匹配而使词法分析器不能继续处理的情况，也就是说，当前状态与当前输入符号在转换表对应项中的信息为空，而当前状态又不是终止状态，此时说明检测到了词法错误，需要调用错误处理程序。

根据最长子串匹配原则，错误处理程序查找已扫描字符串中最后一个对应于某终态的字符。如果找到了，将该字符与其前面的字符识别成一个单词，然后将输入指针退回到该字符，扫描器重新回到初始状态，继续识别下一个单词。如果没找到，则确定出错。

检测到错误以后怎么办？是立刻中止分析吗？因为源程序中的错误可能不止一处，为了让分析器运行一次能够反馈尽可能多的错误从而提高程序员调式

的效率，应该以恰当的方式进行处理，使得分析器能够继续运行下去，从错误状态中恢复过来，以检测出源程序中更多错误。这种处理称为错误恢复。

最简单的错误恢复策略称为恐慌模式恢复（panic mode recovery），即从剩余的输入中不断删除字符，直到词法分析器能够在剩余的输入的开头发现一个正确的词法单元为止。这种恢复策略可能会给语法分析带来一些麻烦，但在交互计算环境中是非常有效的。

其它错误恢复动作包括：

- (1) 从剩余的输入中删除一个字符
- (2) 向剩余的输入中插入一个遗漏的字符
- (3) 用一个字符来替换另一个字符
- (4) 交换两个相邻的字符

这些变换可以在试图修复错误输入时进行。最简单的策略是看一下是否可以通过一次变换将剩余输入的某个前缀变成一个合法的词素。另外一种更加通用的改正策略是计算出最少需要多少次变换才能够把一个源程序转换成为一个只包含合法词素的程序。但是在实践中发现这种方法的代价太高，不值得使用。

3.5 词法分析器的自动生成

通常来说，扫描器的实现分为以下几步：

- (1) 确定所要识别的单词的种类，以及各类单词被识别出来以后应执行的动作
- (2) 写出各类单词的正则表达式
- (3) 根据正则式构造FA（画出FA对应的转换图）
- (4) 根据FA（转换图）编写程序

事实上，只要将第(1)步和第(2)步中定义的内容描述清楚，第(3)步和第(4)步是完全可以由计算机自动生成，从而实现整个词法分析器的自动生成。扫描器自动生成工具一方面可以加快分析器的实现速度，程序员只需在很高的模式层次上描述软件，就可以依赖自动生成工具来生成详细的代码。另一方面，自动生成工具也使修改扫描器的工作变得更加简单，只需修改那些受到影响的模式，无需改写整个程序。

在前面的章节中，花了较多的篇幅介绍词法的正则文法、正则表达式、NFA和DFA的基本概念，以及从正则表达式到NFA、从NFA到DFA的转换方法，

还介绍了DFA化简的方法。这些算法构成了词法分析器自动生成的基础。正则文法和正则表达式是词法定义的形式化技术。任何问题只有解决了形式化问题，才能实现问题求解的自动化。

目前存在很多词法分析器自动生成工具，但是 Lex 的应用最为广泛。Lex 是由美国贝尔实验室的 M. Lesk 等人用 C 语言编制的一个词法分析器自动生成工具。

3.5.1 Lex 的使用

就像 C 是由 C 语言和 C 编译器构成的一样，Lex 也由 Lex 语言和 Lex 编译器两部分组成。

(1) Lex 语言。从 Lex 工具使用者的角度，其目的是利用该工具为某语言 X 构造词法分析器，所以，他需要先描述一下该语言 X 的词法规则。Lex 语言正是用于描述各类单词的正则定义式，及各单词被识别出来以后应执行的动作（如输出单词的种别码等）。由 Lex 语言编写的程序叫做“Lex 源程序”，它是 Lex 编译器的输入。

(2) Lex 编译器。Lex 工具使用者编写的 Lex 源程序本质上是一系列正则表达式，Lex 编译器的任务就是将输入的正则表达式转化成一个用 C 语言编写的词法分析器（词法分析器的本质是 DFA）。正是因为正则表达式和 DFA 之间存在着等价关系，所以 Lex 编译器实际上是完成了这个等价变换（也就是翻译）的任务。

Lex 的使用过程如图 3-24 所示。用户用 Lex 语言编写 Lex 源程序（如图中 *lex.l*），用来描述某语言 X 的词法规则。因为源文件是由 Lex 语言编写的，因此其扩展名为“.l”。将 *lex.l* 输入给 Lex 编译器，Lex 编译器输出一个用 C 语言编写的 X 语言的词法分析器 *lex.yy.c*（由于是用 C 语言编写的，所以扩展名为“.c”）。由于 *lex.yy.c* 是用 C 语言编写的，不能直接在机器上运行，因此要通过 C 编译器编译后得到可运行的扫描器 *a.out*。将 X 语言的源程序 *p* 输入给词法分析器 *a.out*，输出的就是 *p* 的 token 序列。

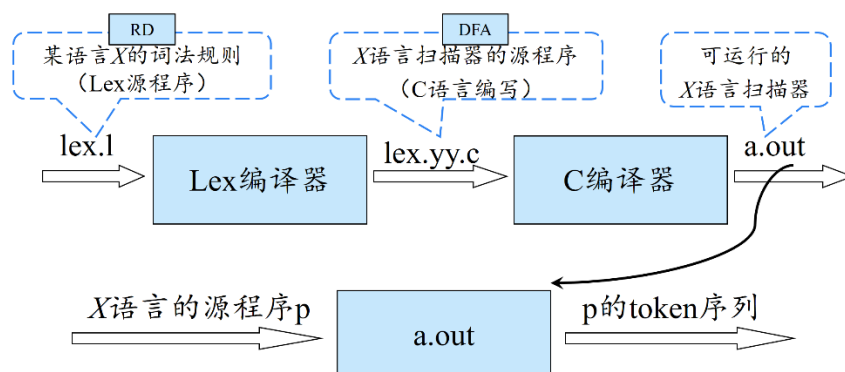


图3-24 Lex使用方法示意图

`a.out` 通常是一个语法分析器调用的子例程。这个子例程返回一个整数值，即可能出现的某个词法单元的种别码。而词法单元的属性值，不管它是一个数字编码，还是一个指向符号表的指针，或者什么都没有，都保存在全局变量 `yylval` 中。这个变量由词法分析器和语法分析器共享。顺便说一下，在 `yylval` 和 `lex.yy.c` 中出现的 `yy` 指的是我们将在 4.5 节中讨论的语法分析器生成工具 Yacc，它一般和 Lex 一起使用。

从 Lex 工具使用者的角度，只要用 Lex 语言编写一个源程序来描述 X 语言的词法规则就可以了，剩下的工作都是由 Lex 来完成的。接下来的关键问题是如何编写 Lex 程序。

3.5.2 Lex 程序的结构

一个 Lex 程序具有如下形式：

```
声明部分
%%
转换规则
%%
辅助过程
%
```

第一部分是声明部分，包括明示常量（manifest constant）和正则定义。

在第二部分中，每个转换规则具有如下形式：

```
模式 {动作}
```

其中，每个模式是一个正则表达式，它可以使用声明部分中给出的正则定义。

动作部分是代码片断。虽然人们已经创建了很多能使用其他语言的 Lex 的变体，但这些代码片断通常是用 C 语言编写的。

第三部分包含各个动作需要使用的所有辅助函数。这些过程由用户编写，Lex 将直接把它们复制到 lex.yy.c 中。

例 3.10 图 3-25 给出一个 Lex 程序的例子。

```
%{
    /*定义明示常量
    LT, LE, EQ, NE, GT, GE,
    IF, ELSE, ID, RELOP,... */
}%

/* 正则定义 */
delim      [ \t\n]
ws         {delim}+
digit      [0-9]
digits     {digit}{digit}*
fraction   \.{digits}
exponent   [eE][+-]?{digits}
letter     [_a-zA-Z]
id         {letter}({letter}|{digit})*

%%

{ws}       { /*没有动作和返回值*/ }
struct     { return STRUCT; }
return     { return RETURN; }
if         { return IF; }
else       { return ELSE; }
while      { return WHILE; }
{id}       {
            yylval =installID();
            return ID;
          }
{digits}   {
            yylval =installNum();
            return INT;
          }
{digits}{fraction}|{digits}{fraction}{exponent}|{digits}{exponent}
          {
            yylval=installNum();
            return FLOAT;
          }
";"       { return SEMI; }
","       { return COMMA; }
"="       { return ASSIGNOP; }
">"      { yylval=GT; return RELOP; }
```

```

"<"      { yylval=LT; return RELOP; }
">="     { yylval=GE; return RELOP; }
"<="     { yylval=LE; return RELOP; }
"=="     { yylval=EQ; return RELOP; }
"!="     { yylval=NE; return RELOP; }
"+"      { printf("PLUS\n"); return PLUS; }
"-"      { printf("SUB\n"); return SUB; }
"*"      { printf("MUL\n"); return MUL; }
"/"      { printf("DIV\n"); return DIV; }
"&&"     { return AND; }
"||"     { return OR; }
"."      { return DOT; }
"!"      { return NOT; }
"("      { return LP; }
")"      { return RP; }
"["      { return LB; }
"]"      { return RB; }
"{"      { return LC; }
"}"      { return RC; }

%%
installID()
{
    /*向符号表中填入词素的过程。yytext 指向词素的第一个字符，
    yyleng 表示词素的长度。函数返回指向该词素所在表项的指针*/
}
installNum()
{
    /* 与 installID 类似，只不过将数值常量放入一个单独的表格*/
}

```

图 3-25 一个 Lex 程序的例子

我们在声明部分看到一对特殊的括号：“%{”和“}%”。出现在括号内的所有内容将由 Lex 直接复制到 lex.yy.c 中。通常将明示常量的定义放置在该括号内，并利用 C 语言的#define 语句给每个明示常量赋予一个唯一的整数编码。在这个例子中，我们在一个注释中列出了 LT、IF 等明示常量，但没有显示它们被赋予哪些特定的整数。如果 Lex 同 Yacc 一起使用，那么明示常量通常会在 Yacc 程序中定义，并在 Lex 程序中不加定义就使用它们。因为 lex.yy.c 是和 Yacc 的输出一起编译的，因而这些常量在 Lex 程序的动作中也是可用的。

在声明部分还包含一个正则定义的序列。用方括号“[”“]”括起来的部分表示一个字符类。对于连续的字符可以用-进行缩写。例如[a-zA-Z0-9]可以匹配任意一个字母或数字。为了不致引起混淆，那些将在后面的定义中或某个转换规则的模式中使用的正则定义用花括号“{”“}”括起来。例如，digit 被定

义为表示一个包含了所有数字的字符类的缩写。于是，`digits` 通过正则表达式 `{digit}{digit}*` 定义为一个或多个数字组成的序列。

Lex 源程序中的正则表达式可以出现如下的运算符（优先级依次从高到低），

(), [], *, +, ?, |, ^, \$,

或控制符，

{ }, ., ", \, -, /, <, >

这些字符称为元字符。如果某个元字符需要以正文字符的形式出现在正则表达式中，则必须使用双引号 `"` 或反斜杠 `\` 作为转移字符将其变换成正文字符。

例如，在 `id` 的定义中，圆括号是用于分组的元符号，并不代表圆括号自身。相反，在 `exponent` 定义中的 `E` 代表其自身。如果我们希望 Lex 的某个元符号（比如括号、`+`、`*` 或 `?` 等）表示其自身，我们可以在它们前面加上个反斜线。例如，我们在 `fraction` 的定义中看到的 `\.` 就表示小数点本身。在它前面加上反斜线的原因是，和在 UNIX 正则表达式中一样，该字符在 Lex 中是一个代表“任一字符”的元符号。

在辅助函数部分，我们可以看到这样两个函数：`installID()` 和 `installNum()`。和位于 `%{.....%}` 中的声明部分一样，出现在辅助部分中的所有内容都被直接复制到文件 `lex.yy.c` 中。虽然它们位于转换规则部分之后，但这些函数可以在规则部分的动作定义中使用。

最后，让我们看一下例 3.10 的中间部分的一些模式和规则。首先，在第一部分中定义的名字 `ws` 有一个相关的空动作。如果我们发现了一个空白符，我们并不把它返给语法分析器，而是继续寻找另一个词素。

第 2 个词法单元有一个简单的正则表达式模式 `struct`。如果我们在输入中看到六个字母 `struct`，并且 `struct` 之后没有跟随其它字母或数位（如果有的话，词法分析器会去寻找一个和 `id` 模式匹配的最长输入前缀），然后词法分析器从输入中读入这六个字符，并返回词法单元名 `STRUCT`，也就是明示常量 `STRUCT` 所代表的整数值。关键字 `return`、`if`、`else` 和 `while` 的处理方法与此类似。

第 7 个词法单元的模式由 `id` 定义。注意，虽然像 `struct` 这样的关键字既和这个模式匹配，也和之前的一个模式匹配，但是当最长匹配前缀和多个模式匹配时，Lex 总是选择最先被列出的模式。当 `id` 被匹配时，相应的处理动作分为三步：

- (1) 调用函数 `installID()` 将找到的词素放入符号表中。

(2) 该函数返回一个指向符号表的指针。这个指针被放到全局变量 `yy1val` 中，并可被语法分析器或编译器的某个后续组件使用。注意，函数 `instalID()` 可以使用以下两个由 `Lex` 生成的、由词法分析器自动赋值的变量：

① `yytext` 是一个指向词素开头的指针。

② `ylleng` 存放刚找到的词素的长度。

(3) 将词法单元名 `ID` 返回到语法分析器。

当一个词素与模式 `digits` 和 `{digits}{fraction}|{digits}{fraction}{exponent}|{digits}{exponent}` 匹配时，执行的处理与此类似，它使用辅助函数 `instalNum()` 完成处理。□

3.5.3 Lex 中的冲突解决

前面我们已经间接提到了 `Lex` 语言具有的两项隐含规定：

(1) 最长子串匹配原则。

(2) 程序中位于前面的识别规则的优先级高于后面的识别规则。例如，`struct` 既可以和模式 `struct` 相匹配，又可以和模式 `{id}` 相匹配，则 `struct` 将被识别为关键字而不是标识符。一般地，通过将关键字的模式放到标识符的模式之前，可以简单、有效地将关键字从标识符中区分出来。

当输入的多个前缀与一个或多个模式匹配时，`Lex` 用以上两条规则选择正确的词素。

3.6 本章小结

本章系统构建了词法分析的理论与技术体系。我们首先明确了词法分析器作为编译器前端预处理模块的核心任务——将字符流转换为规范化的单词序列。随后，深入探讨了使用正则文法、正则表达式和正则定义等形式化工具对单词进行精确描述的数学方法。重点阐述了有穷自动机这一核心识别模型，详细分析了从正则表达式到非确定自动机（NFA）、再到确定自动机（DFA）的等价转换算法，揭示了形式语言理论与自动机模型的深刻联系。最后，本章介绍了词法分析器的系统化设计方法、错误处理机制，并通过 `Lex` 等自动生成工具展示了理论到工程实践的有效转化路径，为后续语法分析阶段奠定了坚实的输入基础。

本章要点如图 3-26 所示。

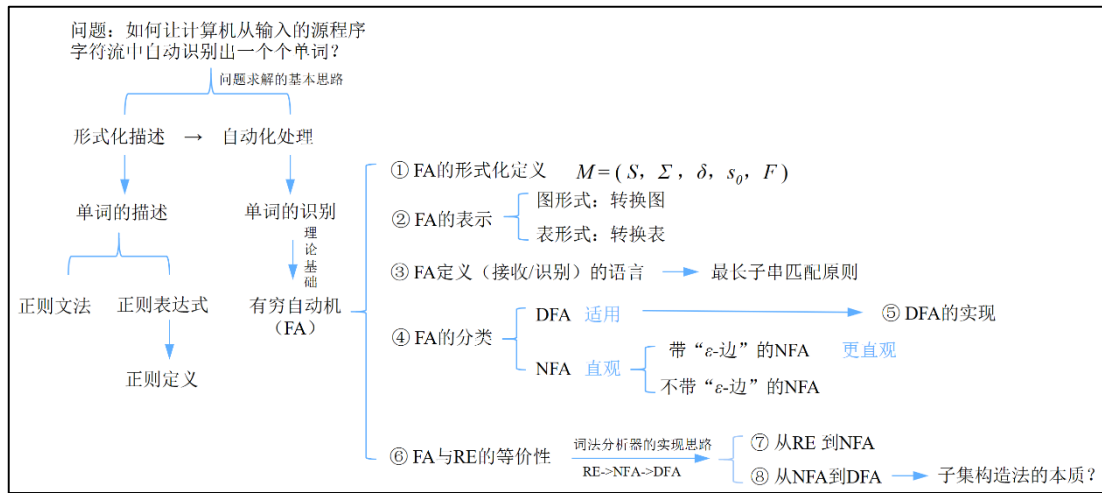


图 3-26 第 3 章要点

第4章 语法分析

如果说词法分析是识别语言的“单词”，那么语法分析的核心任务，就是识别由这些单词构成的“句子”。具体而言，它依据一套预先定义好的结构规则——即文法，对输入词法单元序列进行解析，识别出构成程序的各个语法成分（如表达式、语句、函数定义等），并将这种层次化的结构关系以一棵分析树的形式清晰地呈现出来。因此，语法分析的实质，就是在给定文法规则的指导下，为输入串构造其对应的分析树，从而验证其语法正确性，并为后续的语义分析奠定坚实的基础。

现代程序设计语言的语法结构绝大多数都可以用上下文无关文法（CFG）来精确描述。在编译的语法分析阶段，CFG中的终结符直接对应于词法分析器产出的词法单元，这使得两个分析阶段得以无缝衔接。为了保证计算机能够无歧义地进行自动化处理，语法分析器通常要求所使用的文法是无二义性的。否则，对于一个合法的输入句子，可能会推导出多棵不同的分析树，导致程序含义模糊不清，无法进行唯一的后续处理。

根据构造分析树的方向，语法分析方法主要分为两大策略：

(1) 自顶向下分析从代表整个程序的文法开始符号（即分析树的根节点）出发，通过不断选择并应用产生式，逐步推导出与输入词法单元序列相匹配的叶子节点。这个过程可以看作是“推导”出输入串。

(2) 自底向上分析则反其道而行之，从输入词法单元序列（即分析树的叶子节点）开始，通过不断寻找可归约的串并将其替换为相应的非终结符（即归约操作），逐步向上构造，直至归约到文法的开始符号（即根节点）。这个过程可以看作是“归约”出开始符号。

尽管最高效的（即确定性的、无需回溯的）自顶向下和自底向上分析方法只能处理特定的文法子类，但是其中的某些子类——特别是LL文法和LR文法——在实践中极具价值。它们所具备的表达能力，已足以覆盖现代主流程序设计语言几乎所有的语法构造。本章将系统阐述这两类核心分析方法的基本思想、关键算法、相关分析器的构造方法，以及它们各自的能力与局限，引领读者掌握让编译器“读懂”程序结构的核心技术。

4.1 自顶向下的语法分析概述

4.1.1 自顶向下分析的工作方式

前文已经提到，自顶向下的分析从分析树的顶部（根节点）向底部（叶节点）构造分析树，可以被看作是从文法开始符号 S 推导出串 w 的过程。推导的每一步，都需要做两个选择：

- (1) 替换当前句型中的哪个非终结符？
- (2) 用该非终结符的哪个候选式进行替换？

对于第一个问题，为了让计算机能够自动地进行自顶向下的分析，必须以一种特定的顺序来替换句型中的符号。由于语法分析器工作时通常都是自左向右扫描输入程序，这使得对句型按照从左到右的顺序进行分析是比较自然的，因此，自顶向下的分析在推导的每一步总是选择当前句型的最左非终结符进行替换，即。

对于第二个问题，由于语法分析的任务是“根据给定的文法，识别输入句子的各个成分，从而构造出句子的分析树”，因此，影响语法分析器的输出（即语法分析树）的因素有两个，一个是语法规则（通常以 CFG 来描述），另一个是输入的句子（词串） $w_1, w_2, \dots, w_n$ 。也就是说，在推导过程中，既要依据文法，也要依据输入的词串。因此，推导过程中会根据输入流中的当前终结符，选择最左非终结符的一个候选式。

例4.1 给定文法(4.1)，对于输入句子 $\text{id}+\text{id}*\text{id}$ ，其自顶向下的分析过程如图 4-1 所示。

$$\begin{aligned}
 (1) E &\rightarrow TE' \\
 (2) E' &\rightarrow +TE' \mid \varepsilon \\
 (3) T &\rightarrow FT' \\
 (4) T' &\rightarrow *FT' \mid \varepsilon \\
 (5) F &\rightarrow (E) \mid \text{id}
 \end{aligned} \tag{4.1}$$

初始时，输入指针指向输入句子中的第 1 个单词 **id**，从文法的开始符号 E 开始推导（如图 4-1 第 1 步）。根据产生式(1)， E 只有一个候选式，也就是说， E 由一个 T 和一个 E' 构成。将 E 替换成 TE' ，得到句型 TE' （如图 4-1 第 2 步）。

接下来选择当前句型的最左非终结符 T 进行分析。根据产生式(3)， T 也只有一个候选式，也就是说， T 由一个 F 和一个 T' 构成。将 T 替换成 FT' ，得到

句型 $FT'E'$ (如图 4-1 第 3 步)。

接下来继续选择当前句型的最左非终结符 F 进行分析。根据产生式(5), F 有两个候选式。第 1 个候选式表明 F 可以由一个 “(” 连接上一个 E 和一个 “)” 构成。第 2 个候选式表明 F 可以由一个 **id** 构成。由于当前输入指针指向的是单词 **id**, 与第 2 个候选式的第 1 个符号匹配, 说明当前句型中的这个 F 应该是由一个 **id** 构成的, 因此采用 F 的第 2 个候选式 **id** 进行替换, 得到符号串 **idT'E'** (如图 4-1 第 4 步)。当前的输入符号 **id** 匹配成功, 输入指针指向下一个单词 “+”。假设在第 3 步时当前输入指针指向的不是单词 **id**, 而是单词 “(”, 则与 F 的第 1 个候选式的第 1 个符号匹配, 说明当前句型中的这个 F 应该是由一个 “(” 连接上一个 E 和一个 “)” 构成。

接下来继续选择当前句型的最左非终结符 T' 进行分析。根据产生式(4), T' 有两个候选式。第 1 个候选式表明 T' 可以由一个 “*” 连接上一个 F 和一个 T' 构成。第 2 个候选式表明 T' 可以由一个空串 ε 构成。由于当前输入指针指向的是单词 “+”, 与第 1 个候选式的第 1 个符号不匹配, 说明当前句型中的这个 T' 应该不是由一个 “*” 连接上一个 F 和一个 T' 构成的。因此考虑采用 T' 的第 2 个候选式 ε 进行替换 (也就是说认为当前句型中的这个 T' 是由一个空串 ε 构成的), 得到符号串 **idE'** (如图 4-1 第 5 步)。当前的输入符号 “+” 没有得到匹配, 输入指针依然指向 “+”。

这个过程一直进行下去, 直至分析完毕。

步骤	输入	分析树
1	id + id * id ↑	E
2	id + id * id ↑	$\begin{array}{c} E \\ \swarrow \quad \searrow \\ T \quad E' \end{array}$
3	id + id * id ↑	$\begin{array}{c} E \\ \swarrow \quad \searrow \\ T \quad E' \\ \swarrow \quad \searrow \\ F \quad T' \end{array}$
4	id + id * id ↑	$\begin{array}{c} E \\ \swarrow \quad \searrow \\ T \quad E' \\ \swarrow \quad \searrow \\ F \quad T' \\ \\ \text{id} \end{array}$
5	id + id * id ↑ ↑	$\begin{array}{c} E \\ \swarrow \quad \searrow \\ T \quad E' \\ \swarrow \quad \searrow \\ F \quad T' \\ \quad \\ \text{id} \quad \varepsilon \end{array}$
6	id + id * id ↑ ↑	$\begin{array}{c} E \\ \swarrow \quad \searrow \\ T \quad E' \\ \swarrow \quad \searrow \quad + \quad \swarrow \quad \searrow \\ F \quad T' \quad T \quad E' \\ \quad \\ \text{id} \quad \varepsilon \end{array}$
7	id + id * id ↑ ↑ ↑	$\begin{array}{c} E \\ \swarrow \quad \searrow \\ T \quad E' \\ \swarrow \quad \searrow \quad + \quad \swarrow \quad \searrow \\ F \quad T' \quad T \quad E' \\ \quad \quad \swarrow \quad \searrow \\ \text{id} \quad \varepsilon \quad F \quad T' \end{array}$
8	id + id * id ↑ ↑ ↑	$\begin{array}{c} E \\ \swarrow \quad \searrow \\ T \quad E' \\ \swarrow \quad \searrow \quad + \quad \swarrow \quad \searrow \\ F \quad T' \quad T \quad E' \\ \quad \quad \swarrow \quad \searrow \\ \text{id} \quad \varepsilon \quad F \quad T' \\ \\ \text{id} \end{array}$
9	id + id * id ↑ ↑ ↑ ↑	$\begin{array}{c} E \\ \swarrow \quad \searrow \\ T \quad E' \\ \swarrow \quad \searrow \quad + \quad \swarrow \quad \searrow \\ F \quad T' \quad T \quad E' \\ \quad \quad \swarrow \quad \searrow \\ \text{id} \quad \varepsilon \quad F \quad T' \\ \quad \swarrow \quad \searrow \\ \text{id} \quad * \quad F \quad T' \end{array}$

10	$\text{id} + \text{id} * \text{id}$ $\uparrow \uparrow \uparrow \uparrow \uparrow$	
11	$\text{id} + \text{id} * \text{id}$ $\uparrow \uparrow \uparrow \uparrow \uparrow \uparrow$	
12	$\text{id} + \text{id} * \text{id}$ $\uparrow \uparrow \uparrow \uparrow \uparrow \uparrow$	

图 4-1 $\text{id} + \text{id} * \text{id}$ 的自顶向下的分析

计算机是如何自动地实现这个自顶向下的分析过程的？

计算机实现自顶向下语法分析的通用形式称为递归下降分析（由 1984 年图灵奖得主 Niklaus Wirth 提出）。递归下降分析由一组过程组成，每个过程对应文法的一个非终结符。从文法开始符号 S 对应的过程开始，其中递归调用文法中其它非终结符对应的过程。如果 S 对应的过程体恰好扫描了整个输入串，则成功完成语法分析。

过程 $S()$ 的示意图如图 4-2 所示。过程开始执行的时候，首先选择 S 的一个产生式，不妨设为 $S \rightarrow X_1 X_2 \cdots X_n$ 。这相当于给分析树的根节点 S 构造了 n 个子节点，如图 4-3(a) 所示。接下来从左到右依次分析每个文法符号 X_i ($i=1$ to n)。 X_i 无外乎两种情况，一种情况它是一个终结符 ($X_i \in V_T$)，另一种情况它是一个非终结符 ($X_i \in V_N$)。

```

void S(TOKEN)
{
    选择一个S产生式:  $S \rightarrow X_1 X_2 \dots X_n$  ;
    for (  $i = 1$  to  $n$  )
    {
        if  $X_i \in V_T$  {
            if  $X_i == \text{TOKEN}$  then  $\text{TOKEN} = \text{GetNextToken}()$ 
            else (即  $X_i \neq \text{TOKEN}$ )  $\text{Error}()$ 
        }
        else (即  $X_i \in V_N$ )  $X_i()$ 
    }
}

```

图 4-2 过程 S() 的代码示意图



图 4-3 递归下降构造分析树的过程示意图

如果 $X_i \in V_T$ ，将 X_i 与当前输入符号 **TOKEN** 进行匹配。如果匹配上了（即 $X_i == \text{TOKEN}$ ），则调用 *GetNextToken()* 函数，读取下一个输入符号，相当于把输入指针指向下一个输入符号。如果匹配不成功（即 $X_i \neq \text{TOKEN}$ ），说明选择的这个 *S* 产生式无法完成对该输入符号串的推导，此时需要调用 *error()* 函数报错。文法中如果还有其他的产生式可以选择，就退回到过程开始的地方，尝试其他的产生式。如果所有的产生式试下来都分析不下去，就说明输入的符号串是一个错误的句子。

如果 $X_i \in V_N$ ，因为文法中的每一个非终结符都有它对应的一个过程，那么我们就递归调用非终结符 X_i 对应的过程 $X_i()$ ， X_i 的过程体系跟 *S* 的过程体是类似的，它们的形式都是通用的，进入 $X_i()$ 以后，首先也是选择 X_i 的一个产生式，这里不妨设为 $X_i \rightarrow Y_1 Y_2 \dots Y_m$ ，它的右部由 *m* 个文法符号构成的。我们在选择该产生式的同时，就相当于为分析树中的 X_i 节点构造了 *m* 个子节点，如图 4-3(b) 所示。由此可见，分析树就是在递归调用的过程中，自顶向下构造出来的。每次给一个节点增加子节点的时候，其实都是因为它递归调用了非终结符对应的过程。这个过程可以往复循环下去，接下来从左到右依次分析 X_i 的每一个子节点 Y_j 。

递归下降分析中的“递归”指的就是非终结符对应的过程之间的递归调用，“下降”指的是自顶向下构造分析树。左到右依次分析产生式右部的每个文法符号体现了“最左推导”，也就是说排在左边的非终结符会优先被推导。

综上所述，对于任意非终结符 A ，其对应的过程 $A()$ 的程序框架如图 4-4 所示。

```

void A()
{
    选择一个  $A$  产生式,  $A \rightarrow X_1 X_2 \dots X_n$ ;
    for ( $i = 1$  to  $n$ )
    {
        if ( $X_i$  是一个非终结符号)
            调用过程  $X_i()$ ;
        else if ( $X_i$  等于当前的输入符号 TOKEN)
            读入下一个输入符号;
        else /* 发生了一个错误 */;
    }
}

```

图 4-4 递归下降分析器中非终结符 A 对应的典型过程

4.1.2 文法的改造

很多时候，最初编写好的文法并不一定适合于自顶向下的分析，在这之前，通常需要进行必要的文法改造。

(1) 提取左公因子

我们把文法中每个非终结符 A 的右部（可能有多个）称为 A 的候选式。如果一个非终结符的多个候选式存在共同前缀，则分析器无法根据当前输入符号确定选择哪个候选式进行推导，只能按照某个顺序逐个尝试各个候选式。当某个候选式尝试不成功时，就需要退回到上一步推导，继续尝试其他候选式。这个重新尝试的过程称为回溯（backtracking）。回溯会影响分析器的效率。

例 4.2 给定如下文法：

$$\begin{aligned} S &\rightarrow xAy \\ A &\rightarrow *|** \end{aligned} \quad (4.2)$$

对输入串 $x**y$ 进行分析。首先，从 G 的开始符号 S 出发，此时 S 只有一个候选式，因此得到推导

$$S \Rightarrow xAy \quad (4.3)$$

此时，当前句型中由终结符组成的最长前缀为 x ，与输入串的同等长度的前缀 x 正好匹配。此时，当前句型的最左非终结符 A 有两个候选式 “*” 和 “**”，按顺序首先选择第一个候选式 “*” 进行试探，得到推导

$$S \Rightarrow xAy \Rightarrow x*y \quad (4.4)$$

此时，当前句型中由终结符组成的最长前缀为 $x*y$ ，但是与输入串的同等长度

的前缀 x^{**} 不匹配，从而可以断定推导(4.4)是错误的，于是回退到推导(4.3)，并选用 A 的第二个候选式 ** 进行试探，从而得到推导

$$S \Rightarrow xAy \Rightarrow x^{**}y \quad (4.5)$$

此时，当前句型中由终结符组成的最长前缀为 $x^{**}y$ ，与输入串的同等长度的前缀 $x^{**}y$ 正好匹配。此时，由于已经与整个输入串完全匹配，因此分析结束。

□

从这个例子可以看出，对于某个给定的文法 G ，当试图采用自顶向下的方法为 G 的某个句子 w 建立一个最左推导时，可能会出现大量的回溯。特别是当 w 不是 G 的句子时，只有在穷尽所有可能的试探后才能断定 w 不是 G 的句子。回溯的存在使得人们不得不放弃大量已经进行的分析工作，这必然将大大降低分析的效率。即使在自然语言句法分析这样的场合，回溯也不是很高效，因此人们更加倾向于基于表格的方法，如动态规划算法或 Earley 算法。需要回溯的自顶向下分析称为不确定的自顶向下分析。不确定的自顶向下分析由于其代价很高、效率很低，在实际的编译程序中几乎不被采用，因此本书只讨论确定的自顶向下分析方法。自顶向下、自底向上的句法分析都需要解决分析的确定性问题。

对于公共前缀问题，可以采用提取左公因子的方法来改造文法。当不清楚应该选择非终结符 A 的哪个候选式进行替换时，可以将公共前缀提取出来。例如，将文法 4.2 中的 A 产生式改写为

$$A \rightarrow *A' \quad (4.6)$$

$$A' \rightarrow \epsilon | * \quad (4.7)$$

也就是说，通过改造文法使得 A 的各个候选式没有公共前缀，从而推后这个选择过程，等到读入足够多的输入、获得足够多的信息后再作出正确的选择。

算法 4.3 提取左公因子算法。

输入：文法 G

输出：等价的提取了左公因子的文法

方法：对于每个非终结符 A ，找出它的两个或多个选项之间的最长公共前缀 α 。如果 $\alpha \neq \epsilon$ ，即存在一个非平凡的 (nontrivial) 公共前缀，那么将所有 A -产生式 $A \rightarrow \alpha \beta_1 | \alpha \beta_2 | \dots | \alpha \beta_n | \gamma_1 | \gamma_2 | \dots | \gamma_m$ 替换为

$$A \rightarrow \alpha A' | \gamma_1 | \gamma_2 | \dots | \gamma_m \quad (4.8)$$

$$A' \rightarrow \beta_1 | \beta_2 | \dots | \beta_n \quad (4.9)$$

其中， γ_i 表示所有不以 α 开头的产生式体， A' 是一个新的非终结符。不断应用这个转换，直到每个非终结符的任意两个产生式体都没有公共前缀为止。 □

可以看出，消除回溯是要付出代价的，因为引进了一些非终结符和 ε 产生式。

(2) 消除左递归

我们先来看一个例子。

例 4.4 给定算术表达式文法 4.10:

$$\begin{aligned} E &\rightarrow E+T \mid E-T \mid T \\ T &\rightarrow T*F \mid T/F \mid F \\ F &\rightarrow (E) \mid \text{id} \end{aligned} \quad (4.10)$$

对输入串 $\text{id}+\text{id}*\text{id}$ 进行分析，并假定每一步推导总是选择非终结符的第一个候选式进行试探。首先，从文法的开始符号 E 出发，输入指针指向输入串中第一个输入符号 id 。选择 E 的第一个候选式 $E+T$ 进行替换，得到推导

$$E \Rightarrow E+T \quad (4.11)$$

由于此时句型的最左非终结符依然是 E ，而在这一过程中，当前输入符号 id 也没有被匹配，因此也就没有改变。在这种完全相同的条件下，意味着还将做同样的选择，所以接下来还是选择 E 的第一个候选式 $E+T$ 进行替换，如此重复下去，推导将变成如下的一个无限循环过程：

$$\begin{aligned} E &\Rightarrow E+T \\ &\Rightarrow E+T+T \\ &\Rightarrow E+T+T+T \\ &\Rightarrow \dots \end{aligned} \quad (4.12)$$

□

出现上述问题的根源是由于产生式右部的第一个符号与产生式左部符号相同，都是 E 。因此导致推导出的句型的最左非终结符没有改变，就会反复执行相同的操作，陷入无限循环。

如果一个文法中有一个非终结符 A 使得对某个串 α 存在一个推导 $A \Rightarrow^+ A\alpha$ ，那么这个文法就是左递归的 (left recursive)。含有 $A \rightarrow A\alpha$ 形式产生式的文法称为是直接左递归的 (immediate left recursive)。经过两步或两步以上推导产生的左递归称为是间接左递归的 (indirectly left recursive)。左递归文法会使递归下降分析器陷入无限循环。

针对以上的问题，我们需要对文法进行改造，使其适合于自顶向下分析的方法。

首先来看如何消除直接左递归。

对于一个形如 $A \rightarrow A\alpha \mid \beta (\alpha \neq \varepsilon, \beta \text{ 不以 } A \text{ 开头})$ 的直接左递归产生式 (假设非

终结符 A 有两个候选式)，我们先来看看，这个产生式都能生成什么样的串。通过分析，我们可以看出，这个产生式生成的是 β 后面连接若干个 α ，这里的若干可以为 0 个。于是我们可以想办法构造具有等价功能的不含左递归的产生式。引入一个新的非终结符 A' ，令 $A \rightarrow \beta A'$ ，说明从 A 推导出的串都是以 β 开头的。接下来， A' 必须能够产生若干个 α 。这很容易，我们可以写成 $A' \rightarrow \alpha A' \mid \varepsilon$ 。这样，我们就将左递归产生式 $A \rightarrow A\alpha \mid \beta(\alpha \neq \varepsilon, \beta \text{ 不以 } A \text{ 开头})$ 替换为如下的非左递归产生式

$$\begin{aligned} A &\rightarrow \beta A' \\ A' &\rightarrow \alpha A' \mid \varepsilon \end{aligned} \quad (4.13)$$

替换前后的产生式所生成的符号串集合是相同的，但是后者消除了直接左递归。可以看出，消除左递归是要付出代价的，因为引进了一些非终结符和 ε 产生式。事实上，这种消除过程就是把左递归转换成了右递归。

例 4.5 给定一个简化版本的算术表达式文法

$$\begin{aligned} E &\rightarrow E + T \mid T \\ T &\rightarrow T * F \mid F \\ F &\rightarrow (E) \mid \text{id} \end{aligned} \quad (4.14)$$

在消除 E 和 T 的直接左递归后，可以得到如下不含左递归的文法：

$$\begin{aligned} E &\rightarrow T E' \\ E' &\rightarrow + T E' \mid \varepsilon \\ T &\rightarrow F T' \\ T' &\rightarrow * F T' \mid \varepsilon \\ F &\rightarrow (E) \mid \text{id} \end{aligned}$$

这实际上就是文法(4.1)。

□

我们可以很容的将这个方法扩展到一般形式。设文法 G 中的非终结符 A 的所有产生式如下：

$$A \rightarrow A\alpha_1 \mid A\alpha_2 \mid \dots \mid A\alpha_n \mid \beta_1 \mid \beta_2 \mid \dots \mid \beta_m \quad (4.15)$$

其中， β_i ($i=1, 2, \dots, m$) 不以 A 开头，则用以下产生式替换式(4.16)

$$\begin{aligned} A &\rightarrow \beta_1 A' \mid \beta_2 A' \mid \dots \mid \beta_m A' \\ A' &\rightarrow \alpha_1 A' \mid \alpha_2 A' \mid \dots \mid \alpha_n A' \mid \varepsilon \end{aligned} \quad (4.16)$$

但是该方法不能消除那些由两步或两步以上推导构成的间接左递归。

考虑如下文法：

$$S \rightarrow A a \mid b \quad (4.17)$$

$$A \rightarrow A c \mid S d \mid \varepsilon \quad (4.18)$$

从表面上看， S 不存在直接左递归，但这不代表 S 就不存在左递归。事实上，由于 $S \Rightarrow Aa \Rightarrow Sda$ ，因此，它存在间接左递归。如何消除间接左递归？我们可以将 S 的定义代入 A -产生式，得：

$$A \rightarrow Ac | Aad | bd | \varepsilon \quad (4.19)$$

用产生式(4.19)替换(4.18)，使得 A 产生式右部开头不再出现 S ，也就是说， A 不会再推导出 S 开头的串了，这样， S 不会再存在间接左递归问题了。消除 A -产生式(4.19)的直接左递归，得：

$$\begin{aligned} A &\rightarrow bdA' | A' \\ A' &\rightarrow cA' | adA' | \varepsilon \end{aligned} \quad (4.20)$$

这样进一步用(4.20) 替换(4.18)，得到的最终的无左递归文法：

$$\begin{aligned} S &\rightarrow Aa | b \\ A &\rightarrow bdA' | A' \\ A' &\rightarrow cA' | adA' | \varepsilon \end{aligned} \quad (4.21)$$

根据上面的例子，可以总结出消除左递归的算法 4.6。其基本思想是：为非终结符编号，再采用代入法将间接左递归变为直接左递归，然后消除直接左递归。如果文法中不存在循环推导（即形如 $A \Rightarrow^+ A$ 的推导）和 ε -产生式，就保证能够消除左递归。事实上，循环推导和 ε -产生式都可以系统地从文法中消除，具体方法可以参阅形式语言与自动机理论方面的相关文献。

算法 4.6 左递归消除算法。

输入： 不含循环推导（即形如 $A \Rightarrow^+ A$ 的推导）和 ε -产生式的文法 G 。

输出： 与 G 等价的无左递归文法。

方法：

- 1 按照某个顺序将 G 的所有非终结符排序为 $A_1, A_2, \dots, A_n$
- 2 **for** $i=1$ to n {
- 3 **for** $j=1$ to $i-1$ {
- 4 将每个形如 $A_i \rightarrow A_j \gamma$ 的产生式替换为产生式组 $A_i \rightarrow \delta_1 \gamma | \delta_2 \gamma | \dots | \delta_k \gamma$ ，其中 $A_j \rightarrow \delta_1 | \delta_2 | \dots | \delta_k$ 是当前 A_j 的所有产生式；
- 5 }
- 6 消除 A_i 产生式中的直接左递归。
- 7 }

□

值得注意的是，经过改造后的文法在进行自顶向下分析的时候，依然可能会存在问题。事实上，以上文法改造只是进行有效地自顶向下分析的必要条件，

也就是说，不改造，就不能进行自顶向下分析或者是有效地自顶向下分析。但是，改造了，也未必就可以。

事实上，在递归下降分析中，最主要的是“选择一个 A 产生式”这一步。如果 A 的所有候选式都是以终结符开头的，那么经过文法改造（提取左公因子）后可以根据当前的输入符号决定选择哪个候选式，可以做确定的分析。但是如果 A 存在某个以非终结符打头的候选式，那么遇到什么输入符号时选择这个候选式呢？理论上无论当前输入符号是什么，都应该尝试一下。当尝试不成功时，就需要退回到上一步推导，看 A 是否还有其他的候选式，因此，这种情况下依然会产生回溯。需要回溯的分析也称为不确定的分析。

如果分析的每一步都能够预测出正确的选择，就不需要回溯。因此，我们通常只考虑不需要回溯的分析，称为预测分析，也叫做确定的分析。

预测分析是递归下降分析技术的一个特例，通过在输入中向前看固定个数（通常是一个）符号来选择正确的 A -产生式。可以对某些文法构造出向前看 k 个输入符号的预测分析器，该类文法有时也称为 $LL(k)$ 文法类。预测分析不需要回溯，是一种确定的自顶向下分析方法。

4.2 预测分析法

为便于讨论，首先给出预测分析的工作过程：从文法开始符号出发，在每一步推导过程中根据当前句型的最左非终结符 A 和当前输入符号 a ，选择正确的 A -产生式。为保证分析的确定性，选出的候选式必须是唯一的。

那么什么样的文法才能够进行预测分析呢？也就是我们接下来将要介绍的 $LL(1)$ 文法。

4.2.1 $LL(1)$ 文法

要想让分析器能够进行预测分析，我们希望文法最好满足以下两个条件：

- (1) 每个产生式都以终结符开始；
- (2) 同一非终结符的各个候选式的首终结符都不同。

满足以上条件的上下文无关文法称为 S _文法。Korenjak 和 Hopcroft 在 1966 年最先开始研究 S _文法。 S _文法不含 ϵ 产生式，对产生式的形式要求过于严格，用来描述语言有时不太方便。假如允许 S _文法包含 ϵ 产生式，将会产生什么问题？

例 4.7 给定文法(4.22):

- (1) $S \rightarrow aBCD$
 - (2) $B \rightarrow bC$
 - (3) $B \rightarrow dB$
 - (4) $B \rightarrow \varepsilon$
 - (5) $C \rightarrow c$
 - (6) $C \rightarrow a$
 - (7) $D \rightarrow e$
- (4.22)

对输入串 *adae* 的分析过程如图 4-5 所示。

推导过程		输入串				
1	S	a	d	a	e	$\$$
			↑			
2	$\xRightarrow{(1)} aBCD$	a	d	a	e	$\$$
			↑			
3	$\xRightarrow{(3)} adBCD$	a	d	a	e	$\$$
			↑			
4	$\xRightarrow{(4)} adCD$	a	d	a	e	$\$$
			↑			
5	$\xRightarrow{(6)} adaD$	a	d	a	e	$\$$
			↑			
6	$\xRightarrow{(7)} adae$	a	d	a	e	$\$$
					↑	

图 4-5 对输入串 *adae* 的分析过程

接下来我们再看另一种情况，当输入的符号串为 *adee* 时，分析过程如图 4-6 所示。

推导过程		输入串				
1	S	a	d	e	e	$\$$
			↑			

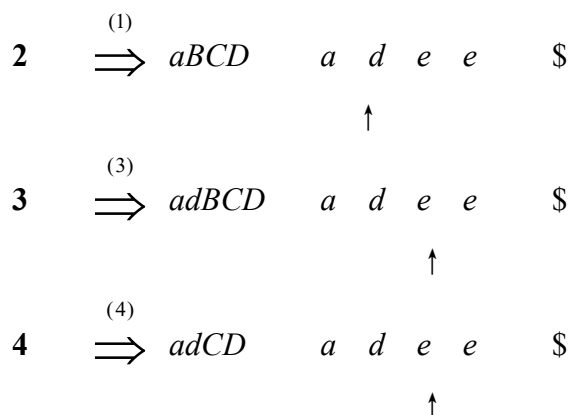
图 4-6 对输入串 *adee* 的分析过程

图 4-6 的前 3 步推导与图 4-5 是一样的。第 4 步推导结束时，当前句型的最左非终结符为 *C*，当前输入符号为 *e*。而 *C* 的两个候选式的第一个符号分别是终结符 *c* 和 *a*，并不是 *e*。因此，分析到这一步就无法继续下去了。为什么在图 4-5 输入串为 *adae* 的例子中应用 *B* 的空产生式推导成功了，而在图 4-6 输入串为 *adee* 的例子中应用 *B* 的空产生式就没有成功？看来，空产生式也不是任何情况下都可以用的。□

那么什么情况下可以使用 ϵ 产生式？事实上，在图 4-6 第 3 步推导结束时，接下来如果应用 *B* 的空产生式，就意味着我们认为 *B* 已经分析出来了（*B* 是由一个空串构成），接下来开始分析紧跟在 *B* 后面的成分。从文法(4.23)来看，*B* 后面可以紧跟着 *C*，而 *C* 可以替换成 *c* 或 *a* 开头的串，因此，如果当前输入符号是 *c* 或 *a*，我们这一步应用 *B* 的空产生式将 *B* 替换成空串是没问题的，也就是说，当前输入符号 *c* 或 *a* 是与紧跟在 *B* 后面的符号 *C* 匹配的（即，*c* 或 *a* 是 *C* 的某个候选式的第 1 个符号）。但是，此时当前输入符号并不是 *c* 或 *a*，而是 *e*。从文法(4.23)来看，*e* 不会紧跟在 *B* 后面（因为文法中紧跟在 *B* 后面的符号 *C* 不会推出 *e* 开头的串），因此，此时应用 *B* 的空产生式将 *B* 替换成空串是没有意义的。

问题的关键是要确定 *B* 后面都可以紧跟哪些终结符，这是由文法本身决定的，可以紧跟 *B* 后面出现的终结符构成一个集合。如果当前的输入符号属于这个集合，则可以使用 *B* 的空产生式，否则使用 *B* 的空产生式是没有意义的。

通过以上分析可知，如果当前句型的最左非终结符 *A* 与当前输入符 *a* 不匹配时，若存在 $A \rightarrow \epsilon$ ，可以通过检查 *a* 是否可以紧跟在 *A* 的后面出现，来决定是否使用产生式 $A \rightarrow \epsilon$ （若文法中无 $A \rightarrow \epsilon$ ，则应报错）。

为此，引入后继符号集的概念。非终结符 *A* 的后继符号集被定义为可能在

某些句型中紧跟在 A 后边的终结符 a 的集合, 记为 $\text{FOLLOW}(A)$ 。其形式化定义为: $\text{FOLLOW}(A) = \{a | S \Rightarrow^* \dots Aa \dots, a \in V_T\}$ 。后面将给出计算 FOLLOW 集的算法。

我们用一个特殊的符号 $\$$ 来表示输入结束标记。我们约定, 句型的后面连接着一个结束符, 用 $\$$ 符号表示, 因此, 如果 A 是某个句型的最右符号, 则将 $\$$ 添加到 $\text{FOLLOW}(A)$ 中。

在例 4.7 中, 我们已经分析过, 可能在某个句型中紧跟在 B 后面的终结符包括 a 和 c , 因此, B 的 FOLLOW 集中有两个元素 a 和 c 。这样, 当最左非终结符为 B 时, 如果当前输入符号为 b , 就选择第 2 条产生式; 如果当前输入符号为 d , 就选择第 3 条产生式; 如果当前输入符号为 a 或 c , 就选择第 4 条产生式。 B 的各个产生式对应的输入符号互不冲突, 因此可以对该文法进行确定的分析。

为了叙述上的方便, 定义一个概念: 产生式的**可选集** SELECT 。产生式 $A \rightarrow \beta$ 的可选集是指可以选用该产生式进行推导时对应的输入符号的集合, 记为 $\text{SELECT}(A \rightarrow \beta)$ 。也就是说, 当前句型最左非终结符为 A 时, 遇到 $\text{SELECT}(A \rightarrow \beta)$ 中的这些符号时, 可以选择产生式 $A \rightarrow \beta$ 。今后为了简便起见, 我们把 $\text{SELECT}(A \rightarrow \beta)$ 表示为 $\text{SELECT}(i)$ 。其中, i 是产生式在文法 G 中的编号。

如果一个产生式的右部以终结符打头, 则该产生式的可选集为该终结符, 即: $\text{SELECT}(A \rightarrow a\beta) = \{a\}$; 如果一个产生式的右部为 ε , 则该产生式的可选集为该非终结符的 FOLLOW 集, 即 $\text{SELECT}(A \rightarrow \varepsilon) = \text{FOLLOW}(A)$ 。

只要具有相同左部 A 的各个产生式的可选集互不相交, 那么分析器就可以根据当前输入符号和这些产生式的可选集来正确地选择 A 的某个候选式替换 A 。由此我们给出 q 文法的定义。

q 文法是满足以下条件的文法:

- (1) 每个产生式的右部或为 ε , 或以终结符开始。
- (2) 具有相同左部的产生式有不相交的可选集。

其实, q 文法的本质就是在 S 文法的基础上增加了一条限制: 如果某个非终结符存在一个 ε 产生式, 那么该非终结符的 FOLLOW 集中不能包含该非终结符的其它候选式的首终结符。 q 文法比 S 文法功能更强, 应用范围更广, 但对产生式右部的限制仍然较严 (不允许右部以非终结符开始)。这还是限制了它的应用范围。因此, 需要引入功能更强的文法, 这就是我们接下来要讲的 $\text{LL}(1)$ 文法。

$\text{LL}(1)$ 文法允许产生式右部以非终结符打头, 这就增加了计算可选集的复

杂性。为此，需要引入串首终结符的概念。串首终结符是串首第一个符号，并且是终结符。简称首终结符。给定一个文法符号串 α ， α 的串首终结符集 $\text{FIRST}(\alpha)$ 被定义为可以从 α 推导得到的串首终结符构成的集合。如果 $\alpha \Rightarrow^* \varepsilon$ ，那么 ε 也在 $\text{FIRST}(\alpha)$ 中。即

$$\forall \alpha \in (V_T \cup V_N)^+, \text{FIRST}(\alpha) = \{ a \mid \alpha \Rightarrow^* a\beta, a \in V_T, \beta \in (V_T \cup V_N)^* \}; \quad (4.23)$$

$$\text{如果 } \alpha \Rightarrow^* \varepsilon, \text{ 那么 } \varepsilon \in \text{FIRST}(\alpha) \quad (4.24)$$

下面我们来看一下 $\text{FIRST}(\alpha)$ 如何计算。假设串 α 由 k 个文法符号构成的，即 $\alpha = X_1 X_2 \cdots X_k$ ，那么直观上来讲，排在最前的符号 X_1 能推出的首终结符决定了串 α 能推出的首终结符，即 $\text{FIRST}(\alpha) = \text{FIRST}(X_1)$ 。接下来，如果 X_1 能推出空串的话，即 $X_1 \Rightarrow^* \varepsilon$ ，那么 X_2 推出的首终结符也有可能成为 α 的首终结符。因此，我们要把 $\text{FIRST}(X_2)$ 中的符号也并入到 $\text{FIRST}(\alpha)$ 。同理，如 X_2 也能推出空串的话，那么 X_3 推出的首终结符也有可能成为 α 的首终结符。因此，也要把 $\text{FIRST}(X_3)$ 中的符号并入到 $\text{FIRST}(\alpha)$ 。以此类推，即

$$\text{FIRST}(\alpha) = \text{FIRST}(X_1) \cup_{\text{if } X_1 \Rightarrow^* \varepsilon} \text{FIRST}(X_2) \cup_{\text{if } X_2 \Rightarrow^* \varepsilon} \text{FIRST}(X_3) \cdots \quad (4.25)$$

该过程就一直进行下去。一种极端的情况就是如果所有的 X_i ($1 \leq i \leq k$) 都能推出空串，那么整个 α 就能推出空串，此时，把空串 ε 也放到 $\text{FIRST}(\alpha)$ 里。

从式(4.25)可以看出，对一个串 α 的 FIRST 集的计算最终被分解为对构成该串的各个文法符号的 FIRST 集的计算。因为单个文法符号本身也可以看成是一个串，因此可以对它施加 FIRST 集这样一个运算。

接下来我们来看一下单个文法符号的 FIRST 集如何计算。我们说对于文法符号 X_i ，它无外乎有两种情况，要么它是一个终结符，要么它是一个非终结符。如果 X_i 是一个终结符的话，那么它能推出的首终结符就是它本身，即 $\text{FIRST}(X_i) = \{ X_i \}$ ；如果 X_i 是一个非终结符的话，那么它能推出的首终结符取决于它的产生式右部是如何构成的。下面我们通过一个例子来说明一下。

例 4.8 求例 4.1 给出的文法(4.1)中非终结符的 FIRST 集。为便于讨论，我们将文法(4.1)重新写在这里：

- (1) $E \rightarrow T E'$
- (2) $E' \rightarrow + T E' \mid \varepsilon$
- (3) $T \rightarrow F T'$
- (4) $T' \rightarrow * F T' \mid \varepsilon$
- (5) $F \rightarrow (E) \mid \text{id}$

该文法中有 5 个非终结符，我们分别计算每个非终结符的 FIRST 集。初始的时候所有文法符号的 FIRST 集都是空集 Φ ，接下来我们逐条产生式处理。

(1) 首先根据产生式(1)计算 E 的 FIRST 集。产生的右部由两个符号构成的, E 的 FIRST 集首先是取决于右部第 1 个符号 T 的 FIRST 集。目前 $\text{FIRST}(T) = \Phi$, 也就是说 $\text{FIRST}(T)$ 还没开始计算。我们暂且不做任何处理。

(2) 接下来根据产生式(2)计算 E' 的 FIRST 集。 E' 有两个候选式。第 1 个候选式的第 1 个符号是 “+”, 因此 E' 能够推出的首终结符可以是 “+”, 将 “+” 添加进 $\text{FIRST}(E')$ 。因为该候选式的第 1 个符号是终结符 (“+”), 终结符是推不出空串的, 因此, 候选式的第 2 个乃至后续符号 (如果有的话) 的 FIRST 集中的符号都不可能成为 E' 的首终结符, 因此不用考虑去考虑它们。第 2 个候选式是空串 ε , 根据式(4.25), 将 ε 也放入 $\text{FIRST}(E')$, 即 $\text{FIRST}(E') = \{ + \varepsilon \}$ 。

(3) 接下来根据产生式(3)计算 T 的 FIRST 集。产生的右部由两个符号构成的, T 的 FIRST 集首先是取决于右部第 1 个符号 F 的 FIRST 集。目前 $\text{FIRST}(F) = \Phi$, 因此我们暂且不做任何处理。

(4) 接下来根据产生式(4)计算 T' 的 FIRST 集。 T' 的情况与 (2) 中的 E' 类似, 可以得到 $\text{FIRST}(T') = \{ * \varepsilon \}$ 。

(5) 接下来根据第产生式(5)计算 F 的 FIRST 集。 F 有两个候选式。第 1 个候选式的第 1 个符号是 “(”, 因此 F 能够推出的首终结符可以是 “(”, 将 “(” 放添加进 $\text{FIRST}(F)$ 。因为该候选式的第 1 个符号是终结符, 依据 (2) 中的讨论, 不需要考虑候选式中的后续符号。第 2 个候选式的第 1 个符号是 **id**, 因此 F 能够推出的首终结符可以是 **id**, 将 **id** 添加进 $\text{FIRST}(F)$ 。因为 **id** 是一个终结符, 因此不需要考虑候选式中的后续符号。这样, $\text{FIRST}(F) = \{ (\text{ id } \}$ 。

至此, 第 1 轮处理结束, 计算结果如下:

$$\begin{aligned}\text{FIRST}(E) &= \Phi \\ \text{FIRST}(E') &= \{ + \varepsilon \} \\ \text{FIRST}(T) &= \Phi \\ \text{FIRST}(T') &= \{ * \varepsilon \} \\ \text{FIRST}(F) &= \{ (\text{ id } \}\end{aligned}$$

目前还有 $\text{FIRST}(E)$ 和 $\text{FIRST}(T)$ 没有完成计算, 因此继续进行第 2 轮。

(1) 根据产生式(1), E 的 FIRST 集首先取决于右部第 1 个符号 T 的 FIRST 集。目前 $\text{FIRST}(T) = \Phi$ 。我们依然暂且不做任何处理。

(
)

根据产生式(1)计算

推不出空串，因此，候选式中 F 之后的符号（如果有的话）的 FIRST 集中的符号都不可能成为 T 的首终结符，因此不用考虑去考虑它们。

(
)

根 至此，第 2 轮处理结束，计算结果如下：

根 $\text{FIRST}(E) = \Phi$
 据 $\text{FIRST}(E') = \{ + \ \varepsilon \}$
 座 $\text{FIRST}(T) = \{ (\ \text{id} \}$
 式 $\text{FIRST}(T') = \{ * \ \varepsilon \}$
 式 $\text{FIRST}(F) = \{ (\ \text{id} \}$

计 目前还有 $\text{FIRST}(E)$ 没有完成计算，因此继续进行第 3 轮。根据产生式(1)，算的 FIRST 集首先取决于右部第 1 个符号 T 的 FIRST 集。 $\text{FIRST}(T)$ 在第 2 轮已经算出来了，于是把 $\text{FIRST}(T)$ 中符号 “(” 和 id 加入到 $\text{FIRST}(E)$ 中。因为算 $\text{FIRST}(T)$ 中没有空串，也就是说 T 推不出空串，因此，候选式中 T 之后的符号 R (如果有的话) 的 FIRST 集中的符号都不可能成为 E 的首终结符，因此不用考

虑

此 $\text{FIRST}(E) = \{ (\ \text{id} \}$
 考 $\text{FIRST}(E') = \{ + \ \varepsilon \}$
 虑 $\text{FIRST}(T) = \{ (\ \text{id} \}$
 过 $\text{FIRST}(T') = \{ * \ \varepsilon \}$
 程 $\text{FIRST}(F) = \{ (\ \text{id} \}$ □

与 由此，给出计算单个文法符号 FIRST 集的算法。

算

法

法

算 方法：不断应用下列规则，直到没有新的终结符或 ε 可以被加入到任何 FIRST 集合中为止。

法

法

法 3) 如果 $X \rightarrow \varepsilon \in P$ ，则将 ε 加入到 $\text{FIRST}(X)$ 中。 □

第 接下来，我们可以利用算法 4.10 计算任何串 $\alpha = X_1 X_2 \cdots X_k$ 的 FIRST 集。

法

法

法

法

保持不变。

法

法

法

法

。

方法:

向 $\text{FIRST}(X_1X_2 \dots X_n)$ 加入 $\text{FIRST}(X_1)$ 中所有的非 ε 符号

如果 ε 在 $\text{FIRST}(X_1)$ 中, 再加入 $\text{FIRST}(X_2)$ 中的所有非 ε 符号; 如果 ε 在 $\text{FIRST}(X_1)$ 和 $\text{FIRST}(X_2)$ 中, 再加入 $\text{FIRST}(X_3)$ 中的所有非 ε 符号, 以此类推。

最后, 如果对所有的 i , ε 都在 $\text{FIRST}(X_i)$ 中, 那么将 ε 加入到 $\text{FIRST}(X_1X_2 \dots X_n)$

现在有了 FIRST 集的概念以后, 我们再来重新定义产生式的可选集。设 $A \rightarrow \alpha$ 的是文法 G 的任意产生式。则它的可选集 $\text{SELECT}(A \rightarrow \alpha)$ 定义如下:

如果 $\varepsilon \notin \text{FIRST}(\alpha)$, 那么 $\text{SELECT}(A \rightarrow \alpha) = \text{FIRST}(\alpha)$ (4.26)

如果 $\varepsilon \in \text{FIRST}(\alpha)$, 那么 $\text{SELECT}(A \rightarrow \alpha) = (\text{FIRST}(\alpha) - \{\varepsilon\}) \cup \text{FOLLOW}(A)$ (4.27)
也就是说, 当前句型的最左非终结符为 A 、当前输入符号为产生式右部 α 的首终结符时, 可以选用 $A \rightarrow \alpha$ 这条产生式。如果产生式右部 α 可以推出空串的话, 相当于 A 能推出空串。那么, 除了 $\text{FIRST}(\alpha)$ 中的终结符以外, 如果遇到 $\text{FOLLOW}(A)$ 中的终结符, 也可以选用该产生式 (参见例 4.7)。

通过以上定义可以看出, SELECT 集中不含空串 ε 。基于 SELECT 集的定义, 我们给出 LL(1)文法的定义。

一个上下文无关文法称为是 LL(1)文法, 当且仅当同一非终结符的各个产生式的可选集互不相交。

LL(1)文法的这一性质决定了我们可以为它构造预测分析器。因为在分析的每一步, 根据当前输入符号 a 以及当前句型最左非终结符 A 的各个候选式的 SELECT 集, 最多可以选出 A 的一个候选式, 因此做出的选择唯一的, 不会产生回溯。“LL(1)”中第一个“L”表示从左 (left) 向右扫描输入串, 第二个“L”表示采用最左 (left) 推导, “1”表示在每一步中只需要向前看一个输入符号来决定语法分析动作。

最后, 我们再来看一下非终结符 A 的后继符号集 $\text{FOLLOW}(A)$ 如何计算。前面提到, $\text{FOLLOW}(A)$ 被定义为可能在某些句型中紧跟在 A 后边的终结符构成的集合。非终结符 A 后面都会紧跟哪些终结符这是由文法决定的。具体来说, 文法中各产生式的右部描述了文法定义的语言中符号之间的相邻关系。通过分析每个非终结符在产生式右部出现的位置, 就能分析出它后面会紧跟什么样的终结符。接下来, 我们先通过一个例子来加以说明。另外, 如果 A 是某个句型的最右符号, 则将输入结束标记 “\$” 添加到 $\text{FOLLOW}(A)$ 中 (因为句型后面会紧跟一个输入结束标记 “\$”, 因此, “\$” 会紧跟在 A 后面)。文法开始符号 S

本身就是一个句型， S 是该句型的最右符号，因此，需要将结束符 “\$” 添加到 FOLLOW(S) 中。

例 4.11 求例 4.1 给出的文法(4.1)中非终结符的 FOLLOW 集。为便于讨论，我们将文法(4.1)重新写在这里：

- (1) $E \rightarrow T E'$
- (2) $E' \rightarrow + T E' \mid \varepsilon$
- (3) $T \rightarrow F T'$
- (4) $T' \rightarrow * F T' \mid \varepsilon$
- (5) $F \rightarrow (E) \mid \text{id}$

在例 4.8 中，我们已经计算出文法(4.1)中各个非终结符的 FIRST 集：

$$\begin{aligned}\text{FIRST}(E) &= \{ (\text{id} \} \\ \text{FIRST}(E') &= \{ + \varepsilon \} \\ \text{FIRST}(T) &= \{ (\text{id} \} \\ \text{FIRST}(T') &= \{ * \varepsilon \} \\ \text{FIRST}(F) &= \{ (\text{id} \}\end{aligned}$$

接下来计算各个非终结符的 FOLLOW 集。首先将 “\$” 添加到文法开始符号 E 的 FOLLOW 集中，即 FOLLOW(E) = { \$ }。接下来逐条分析各产生式。

(1) 产生式 $E \rightarrow T E'$

右部第 1 个非终结符是 T ，我们先来计算它的 FOLLOW 集。 T 后面紧跟 E' ，因此， E' 推出的首终结符，即 FIRST(E') 中的全部终结符（即 “+”）都可以紧跟在 T 后面，因此把它们放入 FOLLOW(T) 中，即 FOLLOW(T) = { + }。注意，FIRST(E') 中的 ε 不要放入 FOLLOW(T) 中，因为根据定义，FOLLOW 集中不包含 ε 。FIRST(E') 中包含 ε ，说明 E' 能推出空串。这样的话，将产生式 $E \rightarrow T E'$ 右部中的 E' 替换成空串，相当于文法中有了 $E \rightarrow T$ 这样一个产生式，也就是说， T 是构成 E 的最后一个成分。这样的话，能紧跟在 E 后面的终结符也能紧跟在 T 后面。所以我们要把 FOLLOW(E) 中的全部终结符（即 \$）也添加到 FOLLOW(T) 中，即 FOLLOW(T) = { + \$ }。

接下来计算右部第 2 个非终结符 E' 的 FOLLOW 集，这取决于紧跟在 E' 后面的符号。可是 E' 后面已经没有其它符号了，也就是说， E' 是构成 E 的最后一个成分。这样的话，能紧跟在 E 后面的终结符也能紧跟在 E' 后面。所以我们要把 FOLLOW(E) 中的全部终结符（即 \$）也添加到 FOLLOW(E') 中，即 FOLLOW(E') = { \$ }。

(2) $E' \rightarrow + T E' \mid \varepsilon$

右部第 1 个非终结符是 T ，我们先来计算它的 FOLLOW 集。 T 后面紧跟 E' ，因此， E' 推出的首终结符，即 $\text{FIRST}(E')$ 中的全部终结符（即“+”）都可以紧跟在 T 后面，因此把它们放入 $\text{FOLLOW}(T)$ 中，事实上，这一操作在分析产生式(1)的时候已经完成了。 $\text{FIRST}(E')$ 中包含 ε ，说明 E' 能推出空串。这样的话，将产生式 $E' \rightarrow +TE'$ 右部中的 E' 替换成空串，相当于文法中有了 $E' \rightarrow +T$ 这样一个产生式，也就是说， T 是构成 E' 的最后一个成分。这样的话，能紧跟在 E' 后面的终结符也能紧跟在 T 后面。所以我们要把 $\text{FOLLOW}(E')$ 中的全部终结符（即 $\$$ ）也添加到 $\text{FOLLOW}(T)$ 中，由于 $\$$ 已经在 $\text{FOLLOW}(T)$ 中了，所以这一步没有改变计算结果。

接下来计算右部第 2 个非终结符 E' 的 FOLLOW 集，这取决于紧跟在 E' 后面的符号。可是 E' 后面已经没有其它符号了，也就是说， E' 是构成 E' 的最后一个成分（即嵌套的情况）。这样的话，能紧跟在产生式左部非终结符 E' 后面的终结符也能紧跟在产生式右部非终结符 E' 后面。于是要把产生式左部非终结符 E' 的 FOLLOW 集中的符号加入到产生式右部非终结符 E' 的 FOLLOW 集。而事实上，不管 E' 是位于产生式左部还是右部，都是同一个文法符号，FOLLOW 集也是同一个，所以不用进行该操作。

(3) $T \rightarrow FT'$

F 和 T' 的 FOLLOW 集的计算过程分别与产生式(1)中 T 和 E' 类似，读者可

以 (4) $T' \rightarrow *FT' \mid \varepsilon$

自 与产生式(2)的情况类似，不再赘述。

行 (5) $F \rightarrow (E) \mid id$

推 先来分析第 1 个候选式。该候选式中只有一个非终结符 E ，其 FOLLOW 集取决于紧跟在 E 后面的符号。由于 E 后面紧跟着终结符“ $)$ ”，因此将“ $)$ ”也添加到 $\text{FOLLOW}(E)$ 中，即 $\text{FOLLOW}(E) = \{\$, \})\} = \{\$ \}$ 。第 2 个候选式中没有非终结符，所以不涉及 FOLLOW 集的计算。

算 可以看出，在分析产生式(5)的时候，向 $\text{FOLLOW}(E)$ 中追加了终结符“ $)$ ”，也就是说 $\text{FOLLOW}(E)$ 被更新了。而产生式(1)中的 T 和 E' 的 FOLLOW 集都会依赖 $\text{FOLLOW}(E)$ ，而产生式(2)中的 T 的 FOLLOW 集会依赖 $\text{FOLLOW}(E')$ ，相当于间接依赖 $\text{FOLLOW}(E)$ ；而产生式(3)中的 F 和 T' 的 FOLLOW 集会依赖 $\text{FOLLOW}(T)$ ，也相当于间接依赖 $\text{FOLLOW}(E)$ 。因此，需要再进行一轮迭代，对相关非终结符的 FOLLOW 进行更新，直到没有新的终结符可以被加入到任何 FOLLOW 集合中为止。

L

O

W

(

F

)

=

最终，各文法符号 FOLLOW 集的计算结果如下：

$$\text{FOLLOW}(E) = \{ \$ \}$$

$$\text{FOLLOW}(E') = \{ \$ \}$$

$$\text{FOLLOW}(T) = \{ + \$ \}$$

$$\text{FOLLOW}(T') = \{ + \$ \}$$

$$\text{FOLLOW}(F) = \{ * + \$ \}$$

□

基于以上过程，给出计算单个文法符号 FOLLOW 集的算法。

算

法 输

入 输

算 方法：不断应用下列规则，直到没有新的终结符可以被加入到任何 FOLLOW 集合中为止

法

如果存在一个产生式 $A \rightarrow \alpha B \beta$ ，那么 $\text{FIRST}(\beta)$ 中除 ε 之外的所有符号都在

入

如果存在一个产生式 $A \rightarrow \alpha B$ ，或存在产生式 $A \rightarrow \alpha B \beta$ 且 $\text{FIRST}(\beta)$ 包含 ε ，那么

入

基

于

算 例

设 $A \in V_N$

$$(1) E \rightarrow T E'$$

和

$$(2) E' \rightarrow + T E'$$

算

$$(3) E' \rightarrow \varepsilon$$

法

$$(4) T \rightarrow F T'$$

的

$$(5) T' \rightarrow * F T'$$

用

$$(6) T' \rightarrow \varepsilon$$

例

$$(7) F \rightarrow (E)$$

算

$$(8) F \rightarrow \text{id}$$

例 4.8 和例 4.11 中，我们已经计算出文法(4.1)中各个非终结符的 FIRST 集和 FOLLOW 集：

$$\text{FIRST}(E) = \{ (\text{id} \} \quad \text{FOLLOW}(E) = \{ \$ \}$$

$$\text{FIRST}(E') = \{ + \varepsilon \} \quad \text{FOLLOW}(E') = \{ \$ \}$$

$$\text{FIRST}(T) = \{ (\text{id} \} \quad \text{FOLLOW}(T) = \{ + \$ \}$$

是输入右端的结束标记。

法

由 $\text{FOLLOW}(A)$ ，可以根据式(4.27)和(4.28)计算每一个产生式 $A \rightarrow \alpha$ 的

入

B

中

)

$$\begin{aligned}\text{FIRST}(T') &= \{ * \ \varepsilon \} & \text{FOLLOW}(T') &= \{ + \$ \} \\ \text{FIRST}(F) &= \{ (\text{id} \} & \text{FOLLOW}(F) &= \{ * + \$ \}\end{aligned}$$

基
于
以
上
结
果
,
计
算

$$\begin{aligned}\text{SELECT}(1) &= \text{FIRST}(TE') = \text{FIRST}(T) = \{ (\text{id} \} \\ \text{SELECT}(2) &= \text{FIRST}(+TE') = \{ + \} \\ \text{SELECT}(3) &= \text{FOLLOW}(E') = \{ \$ \} \\ \text{SELECT}(4) &= \text{FIRST}(FT') = \text{FIRST}(F) = \{ (\text{id} \} \\ \text{SELECT}(5) &= \text{FIRST}(*FT') = \{ * \} \\ \text{SELECT}(6) &= \text{FOLLOW}(T') = \{ + \$ \} \\ \text{SELECT}(7) &= \text{FIRST}((E)) = \{ (\} \\ \text{SELECT}(8) &= \text{FIRST}(\text{id}) = \{ \text{id} \}\end{aligned}$$

产生式(2)和(3)具有相同的左部 E' , 且 $\text{SELECT}(2) \cap \text{SELECT}(3) = \Phi$; 产生式(5)和(6)具有相同的左部 T' , 且 $\text{SELECT}(5) \cap \text{SELECT}(6) = \Phi$; 产生式(7)和(8)具有相同的左部 F , 且 $\text{SELECT}(7) \cap \text{SELECT}(8) = \Phi$ 。由 LL(1)文法的定义可知, 式法(4.1)是 LL(1)文法。 $\square$

通过以上分析可知, 可以基于 LL(1)文法做预测分析。具体可以通过两种形式构造预测分析器: 一种是递归的方式, 另外一种是非递归的方式。

递归的预测分析顾名思义, 是对递归下降分析的扩展。具体来说是基于 SELECT 集对递归下降分析进行扩展。递归下降分析里最关键的一步就是做产生式的选择。如果选错了, 就需要回溯。有了 SELECT 集以后, 就不用盲目的选择了, 而是在 SELECT 集的指导下预测出正确的选择, 从而提高递归下降分析的效率。从图 4-3 显示的递归下降分析中分析树的构造过程可以看出, 它实际上是按照从左到右深度优先的顺序来递归调用文法中的非终结符对应的过程, 所以相当于是隐式地维护了一个栈结构(深度优先采用的数据结构是栈, 广度优先对应的数据结构是队列)。与递归的预测分析不同, 非递归的预测分析通过显式地维护一个栈结构来模拟最左推导过程。

4.2.2 递归的预测分析法

递归的预测分析是对递归下降分析框架的扩展。递归下降分析由一组过程组成, 每个过程对应一个非终结符。递归的预测分析中每个过程带有一个参数 TOKEN, 用来存放输入指针指向的当前的输入符号(如图 4-7 所示)。假设非终结符 A 有 n 个候选式:

$$A \rightarrow \alpha_1 | \alpha_2 | \dots | \alpha_n$$

则每个候选式 α_i 都对应一段代码 $code_i$ 。这样，过程 A 的代码主要分为 $n+1$ 个分支，前 n 个分支分别对应 $code_1$ 、 $code_2$ 、 $\dots$ 、 $code_n$ 。因为待分析的文法是一个 LL(1)文法，因此对于任意一个非终结符 A ，它的这 n 个候选式的 SELECT 集是互不相交的。因此当前输入符号 TOKEN 最多只能落入一个候选式的 SELECT 集。TOKEN 落入哪个候选式的 SELECT 集，就执行哪个候选式对应的代码 $code$ 。如果 TOKEN 没有落入任何一个候选式的 SELECT 集，则说明输入串是有问题的，则执行最后一个（第 $n+1$ 个）分支，即报错处理。

```

A(TOKEN)
{
    if TOKEN ∈ SELECT( $A \rightarrow \alpha_1$ )
         $code_1$ ;
    if TOKEN ∈ SELECT( $A \rightarrow \alpha_2$ )
         $code_2$ ;
    ...
    if TOKEN ∈ SELECT( $A \rightarrow \alpha_n$ )
         $code_n$ ;
    else error();
}

```

图 4-7 递归的预测分析器中非终结符 A 对应的典型过程

具体每一个候选式 α_i 对应的代码 $code_i$ ，与图 4-2 类似。不妨设 α_i 由 k 个符号构成，即 $\alpha_i = X_1 X_2 \dots X_k$ 。我们从左到右依次分析每个文法符号 X_j 。 X_j 无外乎两种情况（如图 4-8 所示），一种情况它是一个终结符（ $X_j \in V_T$ ），另一种情况它是一个非终结符（ $X_j \in V_N$ ）。

```

for ( $j = 1$  to  $k$ )
{
    if  $X_j \in V_T$ 
    {
        if  $X_j == \text{TOKEN}$  then  $\text{TOKEN} = \text{GetNextToken}()$ 
        else (即  $X_j \neq \text{TOKEN}$ )    error()
    }
    else (即  $X_j \in V_N$ )     $X_j()$ 
}

```

图 4-8 候选式 $\alpha_i = X_1 X_2 \dots X_k$ 对应的代码 $code_i$ 的示意图

如果 $X_j \in V_T$ ，将 X_j 与当前输入符号 TOKEN 进行匹配。如果匹配上了（即 $X_j == \text{TOKEN}$ ），则调用 $\text{GetNextToken}()$ 函数，读取下一个输入符号，并将其放入 TOKEN 中。如果匹配不成功（即 $X_j \neq \text{TOKEN}$ ），说明输入符号串有问题，此时需要调用 $\text{error}()$ 函数报错。

如果 $X_j \in V_N$ ，则以当前输入符号 TOKEN 为参数，递归调用非终结符 X_j 对应的过程 $X_j()$ 。

下面是描述某语言程序的文法。<PROGRAM>是开始符号。

- (1) <PROGRAM> $\rightarrow$ **program** <DECLIST> :<TYPE> ; <STLIST> **end**
- (2) <DECLIST> $\rightarrow$ **id** <DECLISTN>
- (3) <DECLISTN> $\rightarrow$, **id** <DECLISTN>
- (4) <DECLISTN> $\rightarrow \varepsilon$
- (5) <STLIST> $\rightarrow$ *s* <STLISTN>
- (6) <STLISTN> $\rightarrow$; *s* <STLISTN>
- (7) <STLISTN> $\rightarrow \varepsilon$
- (8) <TYPE> $\rightarrow$ **real**
- (9) <TYPE> $\rightarrow$ **int** (4.28)

<TYPE>表示类型，可以是 **real**，也可以是 **int**。*s* 表示语句（statement）。由产生式(5)(6)(7)可以看出，<STLIST>表示语句的序列。**id** 表示标识符。由产生式(2)(3)(4)可以看出，<DECLIST>表示所声明的标识符序列。<PROGRAM>表示程序。一个程序由关键字 **program** 依次连接上一个声明序列<DECLIST>、类型<TYPE>、语句序列<STLIST>及关键字 **end**。

接下来我们判断一下该文法是否是 LL(1)文法。产生式(3)和(4)具有相同的左部<DECLISTN>，产生式(6)和(7)具有相同的左部<STLISTN>，产生式(8)和(9)具有相同的左部< TYPE>，我们分别计算这几个产生式的 SELECT 集，得到：

SELECT(3)={ **id** }
 SELECT(4)={ : }
 SELECT(6)={ ; }
 SELECT(7)={ **end** }
 SELECT(8)={ **real** }
 SELECT(9)={ **int** }

可见，具有相同左部的产生式的 SELECT 集互不相交，该文法是 LL(1)文法，可以为其构造递归的预测分析器。

图4-9给出了主程序 DESCENT 的代码框架。首先调用 GETNEXT(TOKEN) 函数读入下一个输入符号，并将其存放在全局变量 TOKEN 里面。初始的时候，TOKEN 存放的就是输入串的第1个符号。接下来，以 TOKEN 作为参数，调用文法开始符号<PROGRAM>对应的过程。该过程执行完以后，正常情况下整个输入串就应该分析完了。此时输入指针应该指向输入结束标记\$。因此，如果此时 TOKEN 存放的不是\$，就说明输入是有问题的，则进行报错处理。

```

program DESCENT;
  begin
    TOKEN = GetNextToken ();
    PROGRAM(TOKEN);
    if TOKEN ≠ '$' then error();
  end

```

图 4-9 主程序 DESCENT 的代码框架

接下来，对于文法的第 1 个非终结符<PROGRAM>，我们可以根据其对应的产生式(1)，构造与<PROGRAM>对应的过程，如图 4-10 所示：

```

procedure PROGRAM(TOKEN);
  begin
    if TOKEN ≠ 'program' then error();
    TOKEN = GetNextToken();

    DECLIST(TOKEN);

    if TOKEN ≠ ':' then error();
    TOKEN = GetNextToken();

    TYPE(TOKEN)
    if TOKEN ≠ ';' then error();
    TOKEN = GetNextToken();
    STLIST(TOKEN);

    if TOKEN ≠ 'end' then error();
    TOKEN = GetNextToken();
  end

```

图 4-10 非终结符<PROGRAM>对应的过程

产生式(1)的右部有 7 个文法符号，因此，过程 *PROGRAM* 的代码由 7 个部分组成，分别对应这 7 个文法符号。右部第 1 个符号是一个终结符 **program**。对于终结符，其对应的代码如下：

```

if TOKEN ≠ 'program' then error();
TOKEN = GetNextToken();

```

也就是判断一下当前输入符号 *TOKEN* 与该终结符是否匹配。如果不匹配就要报错，如果匹配的话就调用 *GetNextToken* ()函数读取下一个输入符号到 *TOKEN*。产生式右部的第 2 个文法符号是一个非终结符<DECLIST>。对于非终结符，我们就以当前的 *TOKEN* 作为参数递归调用该非终结符对应的过程。接下来的几个文法符号也是采用类似的代码编写方式。

对于文法的第 2 个非终结符<DECLIST>，我们可以根据其对应的产生式

(2)，构造与<DECLIST>对应的过程，如图 4-11 所示。由于产生式(2)的右部有 2 个文法符号，因此，过程 *DECLIST* 的代码由 2 部分组成。

```

procedure DECLIST(TOKEN);
begin
    if TOKEN ≠ 'id' then error();
    TOKEN = GetNextToken();

    DECLISTN(TOKEN);
end

```

图 4-11 非终结符<DECLIST>对应的过程

对于文法的第 3 个非终结符<DECLISTN>，它有 2 个产生式(3)和(4)。所以过程<DECLISTN>的代码原则上主要由 3 个分支构成，每一个产生式对应一个分支，如果当前的输入符号 *TOKEN* 没有落入到任何候选式的 *SELECT* 集，则执行第 3 个分支，进行报错处理。由于产生式(4)是一个空产生式，也就是右部没有任何终结符或非终结符，因此不用为它生成任何代码。这样的话，过程<DECLISTN>的代码实际上可以由 2 个分支构成（如图 4-12 所示）：

```

procedure DECLISTN(TOKEN);
begin
    if TOKEN = ',' then
        begin
            TOKEN = GetNextToken();

            if TOKEN ≠ 'id' then error();
            TOKEN = GetNextToken();

            DECLISTN(TOKEN);
        end
    else if TOKEN ≠ ':' then error();
end

```

图 4-12 非终结符<DECLISTN>对应的过程

因为 *SELECT*(3)={,}, 所以当参数 *TOKEN* 是 “,” 时，执行第 1 个分支；产生式(4)是一个空产生式，且 *SELECT*(4)={:}, 此时，如果参数 *TOKEN* 是 “:” 时，不用生成和执行任何代码，否则的话，相当于 *TOKEN* 没有落入任何一个候选式的 *SELECT* 中，此时执行第 2 个分支，进行报错。

类似的，其余几个非终结符<STLIST>、<STLISTN>和<TYPE>对应过程的代码如图 4-13、4-14、4-15 所示。 □

```

procedure STLIST(TOKEN);
begin

```

```

    if TOKEN ≠ 's' then error();
    TOKEN = GetNextToken();

    STLISTN(TOKEN);

end

```

图 4-13 非终结符<STLIST>对应的过程

```

procedure STLISTN(TOKEN);
begin
    if TOKEN = ';' then
        begin
            TOKEN = GetNextToken();

            if TOKEN ≠ 's' then error();
            TOKEN = GetNextToken();

            STLISTN(TOKEN);
        end
    else if TOKEN ≠ 'end' then error();
end

```

图 4-14 非终结符<STLISTN>对应的过程

```

procedure TYPE(TOKEN);
begin
    if TOKEN ≠ 'real' and TOKEN ≠ 'int' then error();
    TOKEN = GetNextToken();
end

```

图 4-15 非终结符<TYPE>对应的过程

4.2.3 非递归的预测分析法

非递归的预测分析显式地维护一个栈结构，而不是通过递归调用的方式隐式地维护栈结构来模拟最左推导过程。非递归的预测分析器模型如图 4-16 所示：

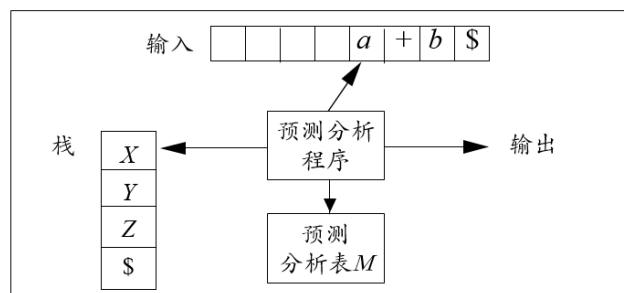


图 4-16 非递归的预测分析器模型

该模型由预测分析程序控制一个输入缓冲区、一个分析栈、一个预测分析表和一个输出流。输入缓冲区中存放待分析的输入串，串末尾连接着输入结束标记\$。分析栈中存放文法符号序列，我们复用符号\$标记栈底。分析过程中输出的产生式序列对应于一个最左推导。由于需要将预测分析表加载到分析器中，因此非递归的预测分析也称为表驱动的预测分析。

为便于计算机处理，将计算出来的 select 集转换成预测分析表的形式。预测分析表 M 是一个二维数组，其中每一行对应一个非终结符 A ，每一列对应一个输入符号 a （包括输入结束标记\$）。 $M[A,a]$ 表示在递归下降分析过程中，当前句型最左非终结符为 A ，当前输入符号为 a 时所采用的产生式。如果 $M[A,a]$ 中没有产生式，说明没有可以使用的产生式，此时需要报错。

例

4

.

1

5

例

4

.

1

根

3

据

通

文

法

计

算

个

产

生

式

4

的

集

中

各

产

生

式

的

集

产生式	SELECT 集
$E \rightarrow TE'$	(id
$E' \rightarrow +TE'$	+
$E' \rightarrow \varepsilon$	\$)
$T \rightarrow FT'$	(id
$T' \rightarrow *FT'$	*
$F \rightarrow (E)$	(
$F \rightarrow id$	id

非终结符	输入符号					
	id	+	*	(	)	\$
E	$E \rightarrow TE'$			$E \rightarrow TE'$		
E'		$E' \rightarrow +TE'$			$E' \rightarrow \varepsilon$	$E' \rightarrow \varepsilon$
T	$T \rightarrow FT'$			$T \rightarrow FT'$		
T'		$T' \rightarrow \varepsilon$	$T' \rightarrow *FT'$		$T' \rightarrow \varepsilon$	$T' \rightarrow \varepsilon$
F	$F \rightarrow id$			$F \rightarrow (E)$		

图 4-17 文法(4.1)的预测分析表

其中第 1 行对应于非终结符 E ，因此它是根据 E 的产生式的 SELECT 集构造的。 E 只有一个候选式（即产生式(1)），因此遇到该候选式的 SELECT 集中非空符号（即“(”和“id”）时，应用该候选式，因此。第 1 行中“(”和“id”对应的列中填入的是候选式 $E \rightarrow TE'$ ，其它列均空白，表示报错。

第 2 行对应于非终结符 E' ，因此它是根据 E' 的产生式的 SELECT 集构造的。 E' 有两个候选式（即产生式(2)和(3)）。产生式(2)的 SELECT 集为 $\{ + \}$ ，产生式(3)的 SELECT 集为 $\{ \$ \}$ 。“+”对应的列中填入的是产生式(2)，即 $E' \rightarrow +TE'$ 。

第 3~5 行的处理方式相同。□

算法输出方

下面我们首先通过一个例子来演示一下非递归的预测分析是如何模拟最左推导的。

例 4.17 对于例 4.1 中给出的算术表达式文法(4.1)以及例 4.15 构造出的该文的预测分析表（为便于讨论，将文法和预测分析表重新写在这里），假如待分析的句子为 $\text{id+id*id\$}$ ，分析过程如图 4-18 所示。

文法的每一个产生式

$A \rightarrow$, 对于 SELECT 中

- (1) $E \rightarrow TE'$
- (2) $E' \rightarrow +TE'$
- (3) $E' \rightarrow \varepsilon$
- (4) $T \rightarrow FT'$
- (5) $T' \rightarrow *FT'$
- (6) $T' \rightarrow \varepsilon$
- (7) $F \rightarrow (E)$
- (8) $F \rightarrow \text{id}$

非终结符	输入符号					
	id	+	*	(	)	\$
E	$E \rightarrow TE'$			$E \rightarrow TE'$		
E'		$E' \rightarrow +TE'$			$E' \rightarrow \varepsilon$	$E' \rightarrow \varepsilon$
T	$T \rightarrow FT'$			$T \rightarrow FT'$		
T'		$T' \rightarrow \varepsilon$	$T' \rightarrow *FT'$		$T' \rightarrow \varepsilon$	$T' \rightarrow \varepsilon$
F	$F \rightarrow \text{id}$			$F \rightarrow (E)$		

	栈	剩余输入	输出
(1)	$E \$$	$\text{id+id*id \$}$	
(2)	$TE' \$$	$\text{id+id*id \$}$	$E \rightarrow TE'$
(3)	$FT'E' \$$	$\text{id+id*id \$}$	$T \rightarrow FT'$
(4)	$\text{id}TE' \$$	$\text{id+id*id \$}$	$F \rightarrow \text{id}$
(5)	$T'E' \$$	$\text{+id*id \$}$	
(6)	$E' \$$	$\text{+id*id \$}$	$T' \rightarrow \varepsilon$
(7)	$\text{+}TE' \$$	$\text{+id*id \$}$	$E' \rightarrow \text{+}TE'$
(8)	$TE' \$$	$\text{id*id \$}$	
(9)	$FT'E' \$$	$\text{id*id \$}$	$T \rightarrow FT'$
(10)	$\text{id}TE' \$$	$\text{id*id \$}$	$F \rightarrow \text{id}$
(11)	$T'E' \$$	$\text{*id \$}$	
(12)	$\text{*}FT'E' \$$	$\text{*id \$}$	$T' \rightarrow \text{*}FT'$
(13)	$FT'E' \$$	$\text{id \$}$	
(14)	$\text{id}TE' \$$	$\text{id \$}$	$F \rightarrow \text{id}$
(15)	$T'E' \$$	$\$$	
(16)	$E' \$$	$\$$	$T' \rightarrow \varepsilon$
(17)	$\$$	$\$$	$E' \rightarrow \varepsilon$

图 4-18 句子 $\text{id+id*id \$}$ 的分析过程

初始的时候，如图中第(1)步所示，栈中只有唯一的一个符号，即文法开始符号 E ， $\$$ 是栈底标志，栈顶在左侧，栈底在右侧。输入带上是要进行语法分析的词串，串后面跟有结束标记 $\$$ ，输入指针指向串首第 1 个输入符号 id 。下面

开始推导。首先栈顶是非终结符 E ，也就是将当前句型的最左非终结符。根据预测分析表， E 遇到输入符号 id 时，将其替换成 TE' 。于是， E 出栈， TE' 进栈，如图中第(2)步所示。

此时，栈顶是非终结符 T ，它成为当前句型的最左非终结符。根据预测分析表， T 遇到输入符号 id 时，将其替换成 FT' 。于是， T 出栈， FT' 进栈，如图中第(3)步所示。

此时，栈顶是非终结符 F ，它成为当前句型的最左非终结符。根据预测分析表， F 遇到输入符号 id 时，将其替换成 id 。于是， F 出栈， id 进栈，如图中第(4)步所示。

此时，栈顶是终结符 id ，它与当前输入符号 id 匹配。于是， id 出栈，输入指针移向下一个符号“+”，如图中第(5)步所示。

此时，栈顶是非终结符 T' ，它成为当前句型的最左非终结符。根据预测分析表， T' 遇到输入符号“+”时，将其替换成空串 ε 。于是， T' 出栈，没有任何符号进栈，如图中第(6)步所示。

这个过程一直进行下去。在第(14)步时，栈顶是终结符 id ，它与当前输入符号 id 匹配。于是， id 出栈，输入指针移向下一个符号 $\$$ ，如图中第(15)步所示。

此时，栈顶是非终结符 T' ，它成为当前句型的最左非终结符。根据预测分析表， T' 遇到 $\$$ 符号时，将其替换成空串 ε 。于是， T' 出栈，没有任何符号进栈，如图中第(16)步所示。

此时，栈顶是非终结符 E' ，它成为当前句型的最左非终结符。根据预测分析表， E' 遇到 $\$$ 符号时，将其替换成空串 ε 。于是， E' 出栈，没有任何符号进栈，如图中第(17)步所示。

此时，栈被清空，输入串中的所有符号都分析完毕，表示成功接收了。另外在分析的每一步，如果应用了某个产生式，我们会将其输出。这样在分析结束成功接收时得到的产生式序列对应的就是最左推导。□

接下来，我们给出表驱动的预测分析的算法描述。

算

法 输入：一个串 w 和文法 G 的预测分析表 M 。

表 输出：如果 w 在 $L(G)$ 中，输出 w 的最左推导；否则给出错误指示。

驱 方法：初始时，语法分析器的格局如下：输入缓冲区中是 $w\$$ ， G 的开始符号位于栈顶，其下面是 $\$$ 。图 4-19 中的程序使用预测分析表 M 生成了处理这个输入的预测分析过程。□

```

设置  $ip$  使它指向  $w$  的第一个符号, 其中  $ip$  是输入指针;
令  $X$ =栈顶符号;
while (  $X \neq \$$  ) {      /* 栈非空 */
    if (  $X$  等于  $ip$  所指向的符号  $a$  ) 执行栈的弹出操作, 将  $ip$  向前移动一个位置;
    else if (  $X$  是一个终结符号 )  $error()$ ;
    else if (  $M[X, a]$  是一个报错条目 )  $error()$ ;
    else if (  $M[X, a] = X \rightarrow Y_1 Y_2 \dots Y_k$  ) {
        输出产生式  $X \rightarrow Y_1 Y_2 \dots Y_k$  ;
        弹出栈顶符号;
        将  $Y_k, Y_{k-1} \dots, Y_1$  压入栈中, 其中  $Y_1$  位于栈顶。
    }
    令  $X$ =栈顶符号
}

```

图 4-19 预测分析算法

前面我们分别介绍了递归的预测分析法和非递归的预测分析法, 下面我们来对比一下这两种方法。

从直观性上来看: 递归的预测分析器中每个非终结符对应的过程是由程序员根据该非终结符产生式右部的符号序列依次编写的, 因此非常直观, 易于理解。非递归的预测分析器的总控程序是独立于文法的, 也就是说, 任何一个 LL(1)文法的非递归的预测分析器的主控程序都是一样的。所不同的是预测分析表。也就是说不同的 LL(1)文法对应的预测分析表不同。将不同文法的预测分析表加载给预测分析器, 预测分析器就会去分析不同 LL(1)文法对应的句子。但是从主控程序本身看不出跟文法和句子结构相关的信息。

从程序规模上来讲, 非递归的预测分析器的总控程序的核心代码如算法 4.18 所示, 规模并不是很大, 但是需要加载预测分析表。递归的预测分析器其程序规模取决于文法中非终结符的个数和及其各个产生式的长度。程序员依据文法和预测分析表编写递归的预测分析器, 但是预测分析表本身不用加载到系统里面, 它已经隐含在程序里了。

从效率上来看, 在递归的预测分析器中, 高深度的递归调用需要花费时间在递归子程序之间的连接上, 影响了分析器的效率。而对于非递归的预测分析器, 分析时间大约正比于程序长度。

另外, 如果所采用的高级语言不允许递归, 则不能使用递归下降法, 也就不能使用递归的预测分析。

4.2.4 预测分析中的错误恢复

与词法分析阶段的错误处理类似，在预测分析过程中检测到错误后也要做必要的错误恢复。前面几个小节中我们多次提到报错，我们先来回顾一下哪些情况下会检测到错误。

我们以非递归的预测分析为例。根据算法 4.18，分析过程中一直是将栈顶的文法符号 X 与输入缓冲区中的当前输入符号 a 放到一起做分析。 X 无外乎分为两种情况：一种是终结符（即 $X \in V_T$ ），一种是非终结符（即 $X \in V_N$ ）。与之相对应，两种情况下可以检测到错误。

X (2) $X \in V_N$ ，且 X 与当前输入符号 a 在预测分析表对应项中的信息为空。也就是说，当前输入符号 a 没有落入 X 的任何一个候选式的可选集，当前成分 X 不可能以 a 开头。

r 与词法分析阶段的错误处理类似，在预测分析阶段检测到错误以后，也要采取适当的错误恢复措施。我们依然可以采取恐慌模式恢复。在预测分析中，恐慌模式错误恢复的基本思想是：忽略输入中的一些符号，直到出现由设计者选定的同步词法单元（synchronizing token）集合中的某个词法单元。其效果依赖于同步（词法单元）集合的选取。同步集合选取的原则是使得语法分析器能够从实际遇到的错误中快速恢复。下面是一些启发式规则。

输 (1) 可以把 $\text{FOLLOW}(A)$ 中的所有终结符放入非终结符 A 的同步词法单元集合。也就是说，当栈顶为非终结符 A 时，检测到错误了，说明 A 在识别过程中出现了问题，那么我们也不指望把它正确地识别出来，索性放弃对这个成分的识别。于是我们忽略输入中的一些符号，直到出现能紧跟在成分 A 后面的终结符匹配 $\text{FOLLOW}(A)$ 中的终结符），此时将 A 从栈顶弹出，使得栈中的下一个符号与输入缓冲区中的当前输入符号同步起来。这也是同步词法单元中“同步”二字的含义。

(2) 只使用 $\text{FOLLOW}(A)$ 作为 A 的同步集合是不够的。比如，C 语言用分号“;”表示一个语句结束，那么语句开头的关键字可能不会出现在代表表达式的非终结符 E 的 FOLLOW 集中。因此，在一个赋值语句（其形式可能是“ $\text{id}=\text{E};$ ”）中遗漏分号可能会使得语法分析器略过下一个语句开头的关键字从而略过对下一个语句的分析。实际上，一个语言的各个构造之间常存在某个层次结构。比如，表达式出现在语句内部，而语句出现在块内部，等等。我们可以把较高层构造的开始符号加入到较低层构造的同步集合中去。比如，把语句

开头的关键字加入到生成表达式的非终结符号 E 的同步集合中去。与(1)相比，(2)相当于除了放弃对栈顶非终结符 A 这个成分的识别以外，还放弃了对 A 所在的构造单元这样一个成分的识别。

(3) 把 $FIRST(A)$ 中的符号加入到 A 的同步集合中。与(1)相比，(3)实际上是不放弃对 A 的分析。为此，忽略输入缓冲区中的输入符号，直到出现能与栈顶非终结符 A 同步的词法单元。

(4) 如果一个非终结符可以生成空串 ε ，那么可以把推导出 ε 的产生式作为默认值使用。这么做可能会延迟对某些错误的检测，但是不会使错误被漏检。

(5) 如果终结符在栈顶而不能匹配，一个简单的办法就是弹出此终结符。(5)与(1)的策略是类似的。从效果上看，该方法相当于是将所有其它词法单元构成的集合作为栈顶终结符的同步集合。

例 4.19 对于例 4.1 中给出的算术表达式文法 4.1 以及例 4.14 构造出的该文法的预测分析表（为便于讨论，将文法和预测分析表重新写在这里），我们采用上述的基于 FOLLOW 集的策略为文法中的非终结符设置同步集合。根据例 4.11，文法 4.1 中各非终结符的 FOLLOW 集如下：

$$FOLLOW(E) = \{ \$ \}$$

$$FOLLOW(E') = \{ \$ \}$$

$$FOLLOW(T) = \{ + \$ \}$$

$$FOLLOW(T') = \{ + \$ \}$$

$$FOLLOW(F) = \{ * + \$ \}$$

我们用 synch 表示根据相应非终结符的 FOLLOW 集得到的同步词法单元，并将它们放入到预测分析表相应的表项中，得到带有同步词法单元的预测分析表（如图 4-19 所示）。

非终结符	输入符号					
	id	+	*	(	)	\$
E	$E \rightarrow TE'$			$E \rightarrow TE'$	synch	synch
E'		$E' \rightarrow +TE'$			$E' \rightarrow \varepsilon$	$E' \rightarrow \varepsilon$
T	$T \rightarrow FT'$	synch		$T \rightarrow FT'$	synch	synch
T'		$T' \rightarrow \varepsilon$	$T' \rightarrow *FT'$		$T' \rightarrow \varepsilon$	$T' \rightarrow \varepsilon$
F	$F \rightarrow \text{id}$	synch	synch	$F \rightarrow (E)$	synch	synch

图 4-19 将同步词法单元加入到文法 4.1 的预测分析表

因为 $FOLLOW(E) = \{ \$ \}$ ，所以将 \$ 和 “)” 设为 E 的同步词法单元。于是， $M[E, \$] = M[E,)] = \text{synch}$;

同理，因为 $\text{FOLLOW}(T) = \{ + \$) \}$ ，所以 $M[T, +] = M[T, \$] = M[T,)] = \text{synch}$;

因为 $\text{FOLLOW}(F) = \{ * + \$) \}$ ，所以 $M[F, *] = M[F, +] = M[F, \$] = M[F,)] = \text{synch}$;

虽然 $\text{FOLLOW}(E') = \{ \$) \}$ ，但是不可以将\$和“)”设为 E' 的同步词法单元。因为 E' 具有空产生式，因此，当栈顶非终结符为 E' 、输入缓冲区中的当前输入符号为 $\text{FOLLOW}(E')$ 中的符号时，是可以正常应用 E' 的空产生式的。非终结符 T' 也是类似的情况，不再赘述。

分析表 4-2 按照如下方式使用：如果 $M[A, a]$ 为空，表示检测到错误，根据恐慌模式，忽略输入符号 a ；如果 $M[A, a]$ 为 synch ，则弹出栈顶的非终结符 A ，试图继续分析后面的语法成分；如果栈顶的终结符和输入符号不匹配，则弹出栈顶的终结符。假设输入的词串为 $*id*+id$ ，它显然不是一个合法的句子。对于该词串的分析及错误恢复过程如图 4-20 所示。

步骤	栈	剩余输入	错误恢复
(1)	$E \$$	$*id*+id \$$	忽略 “*”
(2)	$E \$$	$id*+id \$$	
(3)	$TE' \$$	$id*+id \$$	
(4)	$FT'E' \$$	$id*+id \$$	
(5)	$idT'E' \$$	$id*+id \$$	
(6)	$T'E' \$$	$*+id \$$	
(7)	$*FT'E' \$$	$*+id \$$	
(8)	$FT'E' \$$	$+id \$$	同步
(9)	$T'E' \$$	$+id \$$	
(10)	$E' \$$	$+id \$$	
(11)	$+TE' \$$	$+id \$$	
(12)	$TE' \$$	$id \$$	
(13)	$FT'E' \$$	$id \$$	
(14)	$idT'E' \$$	$id \$$	
(15)	$T'E' \$$	$\$$	
(16)	$E \$$	$\$$	
(17)	$\$$	$\$$	

图 4-20 词串为 $*id*+id$ 的分析及错误恢复过程

在分析开始时，如第(1)步所示，栈顶非终结符为文法开始符号 E ，当前输入符号为 “*”。通过查预测分析表发现 $M[E, *]$ 为空，表示检测到错误。根据恐慌模式，忽略输入符号 “*”。

分析到第(8)步时，栈顶非终结符为 F ，当前输入符号为 “+”。通过查预测分析表发现 $M[F, +] = \text{synch}$ ，则弹出栈顶的非终结符 F ，试图继续分析后面的语法成分。

□

4.3 自底向上的语法分析概述

4.3.1 自底向上分析的工作方式

自底向上的语法分析与自顶向下的语法分析正好相反，它是从分析树的底部（叶节点）向顶部（根节点）方向构造分析树，可以看成是将输入串 w 归约为文法开始符号 S 的过程。自顶向下的语法分析采用最左推导方式，自底向上的语法分析采用最左归约方式（相当于反向构造句子的最右推导）。最左归约也称为规范归约，而最右推导相应地称为规范推导。

自底向上语法分析的通用框架为移入-归约分析(Shift-Reduce Parsing)。移进-归约分析器的结构与非递归的预测分析器类似。它也具有一个输入缓冲区、一个分析栈和一个输出流。输入缓冲区用来存放待分析的串，串尾连接输入结束标记 $\$$ 符号。分析栈用来存放文法符号序列，也是复用符号 $\$$ 标记栈底。与非递归的预测分析器不同的是，在讨论自底向上语法分析的时候，我们将栈顶显示在右侧，而不是像在自顶向下语法分析中那样显示在左侧。

初始时输入缓冲区为 $w\$$ （ w 是待分析的串），而栈是空的，即只有栈底符号 $\$$ 。分析器初始格局如下：

栈	输入
$\$$	$w\$$

移入-归约分析器按照如下的方式来工作：自左向右扫描输入串，一边将输入符号移入分析栈内，一边判断是否应该对栈顶的某个符号串进行归约。如果应该归约，该符号串称为句柄，则分析器执行归约动作。否则继续将输入符号移入栈内，并继续进行判断（这也是“移入-归约分析”名字的由来）。重复上述过程，直到发现错误或栈中只含有开始符号 S 且输入缓冲区为空为止，即进入如下格局：

栈	输入
$\$S$	$\$$

进入这一格局后，移进-归约分析器停止并宣告分析成功。

例4.20 给定算术表达式文法(4.29)，对于输入词串 $id + id * id$ ，其移入-归约分析过程如图4-21所示。移入-归约分析试图采用自底向上的方式为输入的词串构造最左归约，即反向构造最右推导。既然是推导，那么每一步的中间产物就

是一个句型。由于最右推导也称为规范推导，因此，最右推导得到的句型也称为规范句型。可以看出，规范句型的前一部分处于分析栈中，后一部分处于剩余输入缓冲区中，也就是说，分析栈的内容跟剩余输入缓冲区的内容放在一起构成一个规范句型，这个自底向上语法分析一个非常重要的性质。

$$\begin{aligned}
 (1) \quad & E \rightarrow E+E \\
 (2) \quad & E \rightarrow E * E \\
 (3) \quad & E \rightarrow (E) \\
 (4) \quad & E \rightarrow \text{id}
 \end{aligned}
 \tag{4.29}$$

栈	剩余输入	动作
\$	id+(id+id) \$	
\$ id	+(id+id) \$	移入
\$ E	+(id+id) \$	归约: $E \rightarrow \text{id}$
\$ E+	(id+id) \$	移入
\$ E+(	id+id) \$	移入
\$ E+(id	+id) \$	移入
\$ E+(E	+id) \$	归约: $E \rightarrow \text{id}$
\$ E+(E+	id) \$	移入
\$ E+(E+id	) \$	移入
\$ E+(E+E	) \$	归约: $E \rightarrow \text{id}$
\$ E+(E	) \$	归约: $E \rightarrow E+E$
\$ E+(E)	\$	移入
\$ E+E	\$	归约: $E \rightarrow (E)$
\$ E	\$	归约: $E \rightarrow E+E$

图 4-21 词串 id + id * id 的移入-归约分析过程

□

虽然移入-归约分析的主要操作是移入和归约，但实际上一个移入-归约分析器可能采取的动作包括以下 4 种。

- (1) 移入：将下一个输入符号移到栈的顶端。
- (2) 归约：被归约的符号串的右端必然处于栈顶。语法分析器在栈中确定这个串的左端，并决定用哪个非终结符来替换这个串。
- (3) 接收：宣布语法分析过程成功完成。
- (4) 报错：发现一个语法错误，并调用错误恢复子例程。

4.3.2 移入-归约分析中的冲突

与自顶向下语法分析的通用框架——递归下降分析类似，自底向上分析的通用框架——移入-归约分析也会面临一些问题。我们在 4.1.2 节中提到，递归

下降分析存在的一个主要问题是回溯。也就是说当前句型的最左非终结符如果存在多个候选式，这个时候就可能形成选择上的冲突。因此，回溯的本质是候选式选择上的冲突。

移入-归约分析面临的问题本质上也是动作选择上的冲突。具体分为两种，一种叫归约-归约冲突，一种叫移入-归约冲突。所谓归约-归约冲突是指某一个时刻栈顶有两个串分别对应两个产生式的右部，这个时候应该归约哪一个串？而移入-归约冲突是指某一个时刻栈顶的某个串对应着一个产生式的右部，这个时候是否对它进行归约，还是采取继续移入动作？

(1) 归约-归约冲突

例 4.21 归约-归约冲突。给定某语言的声明语句文法(4.30)，输入串 $\text{var } i_A, i_B : \text{real}$ 的移入归约分析过程如图 4-22 所示。

- $$\begin{aligned}
 (1) & \langle S \rangle \rightarrow \text{var } \langle IDS \rangle : \langle T \rangle \\
 (2) & \langle IDS \rangle \rightarrow i \\
 (3) & \langle IDS \rangle \rightarrow \langle IDS \rangle , i \\
 (4) & \langle T \rangle \rightarrow \text{real} \mid \text{int}
 \end{aligned}
 \tag{4.30}$$

步骤	动作	分析栈	剩余输入	分析树
0		\$	var $i_A, i_B : \text{real}$ \$	
1	移入	\$ var	$i_A, i_B : \text{real}$ \$	var
2	移入	\$ var i_A	, $i_B : \text{real}$ \$	var i_A
3	归约	\$ var $\langle IDS \rangle$	, $i_B : \text{real}$ \$	$ \begin{array}{c} \langle IDS \rangle \\ \\ \text{var} \quad i_A \end{array} $
4	移入	\$ var $\langle IDS \rangle$,	$i_B : \text{real}$ \$	$ \begin{array}{c} \langle IDS \rangle \\ \\ \text{var} \quad i_A \quad , \end{array} $
5	移入	\$ var $\langle IDS \rangle$, i_B	: real \$	$ \begin{array}{c} \langle IDS \rangle \\ \\ \text{var} \quad i_A \quad , \quad i_B \end{array} $
6	归约	\$ var $\langle IDS \rangle$, $\langle IDS \rangle$	: real \$	$ \begin{array}{c} \langle IDS \rangle \quad \langle IDS \rangle \\ \quad \quad \\ \text{var} \quad i_A \quad , \quad i_B \end{array} $

7	移入	\$ var <IDS> , <IDS> :	real \$	$\begin{array}{c} \text{var} \quad \begin{array}{c} \text{<IDS>} \\ \\ \mathbf{i}_A \end{array} \quad , \quad \begin{array}{c} \text{<IDS>} \\ \\ \mathbf{i}_B \end{array} : \end{array}$
8	移入	\$ var <IDS> , <IDS> : real	\$	$\begin{array}{c} \text{var} \quad \begin{array}{c} \text{<IDS>} \\ \\ \mathbf{i}_A \end{array} \quad , \quad \begin{array}{c} \text{<IDS>} \\ \\ \mathbf{i}_B \end{array} : \quad \text{real} \end{array}$
9	归约	\$ var <IDS> , <IDS> :<T>	\$	$\begin{array}{c} \text{var} \quad \begin{array}{c} \text{<IDS>} \\ \\ \mathbf{i}_A \end{array} \quad , \quad \begin{array}{c} \text{<IDS>} \\ \\ \mathbf{i}_B \end{array} : \quad \begin{array}{c} \text{<T>} \\ \\ \text{real} \end{array} \end{array}$

图 4-22 词串 var i_A, i_B : real 的第 1 种移入归约分析过程

初始时刻，栈中没有任何文法符号，剩余输入缓冲区中存放的待分析的句子（如图 4-20 第 0 步所示）。

首先，第 1 步，我们将当前的输入符号 var 移入到栈中。此时，栈顶的符号串构不成某个产生式的右部，因此，第 2 步我们继续移入一个输入符号 i_A 。这个时候，栈顶的 i 是产生式(2)的右部，于是，第 3 步将 i 归约成文法符号 <IDS>。此时，栈顶的符号串构不成某个产生式的右部，因此，第 4 步继续移入一个输入符号 “，”。此时，栈顶的符号串依然构不成某个产生式的右部，因此，第 5 步继续移入一个输入符号 i_B 。这个时候，栈顶的 i 是第 2 个产生式的右部，于是第 6 步将 i 归约成文法符号 <IDS>。此时，栈顶的符号串构不成某个产生式的右部，因此，第 7 步继续移入一个输入符号 “:”。此时，栈顶的符号串依然构不成某个产生式的右部，因此，第 8 步继续移入一个输入符号 “real”。此时，栈顶的 real 是产生式(4)的右部，于是，第 9 步将 real 归约成文法符号 <T>。此时，栈顶的符号串并不能构成某个产生式的右部，因此不能采取归约动作。而输入缓冲区中已经没有符号了，因此也不能采取移入动作。可是分析栈中目前还有一堆符号，并不是接收状态要求的只有一个文法开始符号，说明分析失败了。但事实上，可以看出，这个词串实际上是该语言的合法句子。那么，问题出在哪里了呢？

事实上，在分析的第 5 步结束时，栈顶除了 i_B 是某个产生式（即产生式(2)）的右部以外，“<IDS> , i_B ”也是某个产生式（即产生式(3)）的右部。因此，这里就形成了归约-归约冲突。图 4-20 中第 6 步选择用产生式(2)进行归约，看来无法成功构造分析树，因此，接下来我们尝试使用产生式(3)进行归约，则可以成功构造分析树，如图 4-23 所示。

步骤	动作	分析栈	剩余输入	分析树
----	----	-----	------	-----

0		\$	var $i_A, i_B : \text{real}$ \$	
1	移入	\$ var	$i_A, i_B : \text{real}$ \$	var
2	移入	\$ var i_A	, $i_B : \text{real}$ \$	var i_A
3	归约	\$ var $\langle IDS \rangle$	, $i_B : \text{real}$ \$	$\begin{array}{c} \langle IDS \rangle \\ \\ \text{var } i_A \end{array}$
4	移入	\$ var $\langle IDS \rangle$,	$i_B : \text{real}$ \$	$\begin{array}{c} \langle IDS \rangle \\ \\ \text{var } i_A \end{array} ,$
5	移入	\$ var $\langle IDS \rangle$, i_B	: real \$	$\begin{array}{c} \langle IDS \rangle \\ \\ \text{var } i_A \end{array} , i_B$
6	归约	\$ var $\langle IDS \rangle$	: real \$	$\begin{array}{c} \langle IDS \rangle \\ / \quad \quad \backslash \\ \langle IDS \rangle \quad , \quad i_B \\ \\ \text{var } i_A \end{array}$
7	移入	\$ var $\langle IDS \rangle$:	real \$	$\begin{array}{c} \langle IDS \rangle \\ / \quad \quad \backslash \\ \langle IDS \rangle \quad , \quad i_B \\ \\ \text{var } i_A \end{array} :$
8	移入	\$ var $\langle IDS \rangle : \text{real}$	\$	$\begin{array}{c} \langle IDS \rangle \\ / \quad \quad \backslash \\ \langle IDS \rangle \quad , \quad i_B \\ \\ \text{var } i_A \end{array} : \text{real}$
9	归约	\$ var $\langle IDS \rangle : \langle T \rangle$	\$	$\begin{array}{c} \langle IDS \rangle \\ / \quad \quad \backslash \\ \langle IDS \rangle \quad , \quad i_B \\ \\ \text{var } i_A \end{array} : \begin{array}{c} \langle T \rangle \\ \\ \text{real} \end{array}$
10	归约	\$ $\langle S \rangle$	\$	$\begin{array}{c} \langle S \rangle \\ / \quad \quad \backslash \\ \text{var } i_A \quad \langle IDS \rangle \quad \langle T \rangle \\ \quad / \quad \quad \backslash \quad \\ \text{var } i_A \quad \langle IDS \rangle \quad , \quad i_B \quad : \quad \text{real} \end{array}$

图 4-23 词串 $\text{var } i_A, i_B : \text{real}$ 的第 2 种移入归约分析过程

图 4-22 中未能成功构造分析树的原因是在第 6 步时错误地识别了句柄（即

应该归约的那个符号串，见 4.3.1 节）。移入-归约分析试图采用自底向上的方式为输入的句子构造最左归约，即反向构造最右推导。因此，分析每一步的中间产物是一个句型，也就是说，分析栈的内容跟剩余输入缓冲区的内容放在一起构成一个规范句型，如例 4.20 所示。因此，在图 4-40 第 5 步结束时，只要输入的词串是一个合法的句子，那么分析栈中的符号串 “**var** $\langle IDS \rangle$, i_B ” 与剩余输入缓冲区中的符号串 “**: real**” 连接在一起应该构成一个句型 “**var** $\langle IDS \rangle$, i_B : **real**”。只要是句型，它就对应着一棵分析树。虽然这棵分析树目前还没有建立起来，但其实它已经客观存在了，如图 4-24 所示：

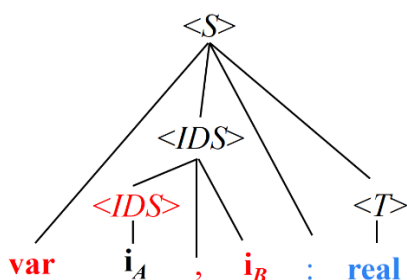


图4-24 句型 “**var** $\langle IDS \rangle$, i_B : **real**” 对应的的分析树

树的边缘（即叶节点构成的序列）对应的是当前的句型 “**var** $\langle IDS \rangle$, i_B : **real**”。其中 “**var** $\langle IDS \rangle$, i_B ” 在栈中，我们用分析树中的红色节点表示。“: **real**” 在剩余输入缓冲区中，我们用分析树中的蓝色节点表示。我们可以很直观地看出，此时应该把栈顶的 i_B 跟 $\langle IDS \rangle$ 和逗号放在一起归约为 $\langle IDS \rangle$ ，如图4-21第6步所示，而不能像图4-20第6步那样，只归约 i_B 自己。这里的 i_B 虽然是第2个产生式的右部，但在当前句型的分析树中，它是有兄弟节点的，它自己构不成一棵子树。这就涉及到我们在2.6节提到的“直接短语”的概念，即高度为2的子树的边缘。实际上要归约的串必须是一个直接短语。事实上，该句型有两个直接短语，一个是以内部节点 $\langle IDS \rangle$ 为根的子树的边缘 “ $\langle IDS \rangle$, i ”，另一个是以内部节点 $\langle T \rangle$ 为根的子树的边缘 “**real**”。自底向上分析过程中应该归约这两个串。由于我们采用的是最左归约，“ $\langle IDS \rangle$, i ” 在最左边，所以先归约它。因此，句柄实际上是当前句型的最左直接短语。□

既然句柄是一个直接短语，它具备直接短语同样的性质。也就是说句柄一定是一个产生式的右部，但是反过来，一个产生式的右部不一定是当前句型的句柄（如例 4.21 中的 i_B ）。在例 4.21 中，当我们人工构造出输入句子的那棵客观存在的分析树时，我们可以人工判断出分析的每一步的句柄是什么。但是计算机在自动分析的过程中在树还没有完全建立起来的前提下，它是如何去

判断应该归约哪个串？这就是如何正确识别句柄的问题。我们将在 4.4 节介绍。

(2) 移入-归约冲突

在移入-归约分析中，当栈顶的任何一个符号串都不能归约时，我们毫无选择地只能采取移入动作。但是如果栈顶某个符号串可以进行归约时，我们实际上是面临着选择的：除了归约以外，其实还可以采取移入动作。这就产生了移入-归约冲突。移入-归约分析过程中，我们总是优先选择归约操作，实际上，有时候这样处理是有问题的。

例 4.22 移入-归约冲突。对于例 4.5 中的文法(4.14)，输入串 $\text{id}_A * \text{id}_B$ 的移入归约分析过程如图 4-25 所示。为便于讨论，我们将文法(4.14)重新写在这里：

$$(1) E \rightarrow E + T$$

$$(2) E \rightarrow T$$

$$(3) T \rightarrow T * F$$

$$(4) T \rightarrow F$$

$$(5) F \rightarrow (E)$$

$$(6) F \rightarrow \text{id}$$

步骤	动作	分析栈	剩余输入	分析树
0		\$	$\text{id}_A * \text{id}_B \$$	
1	移入	\$ id_A	$* \text{id}_B \$$	id_A
2	归约	\$ F	$* \text{id}_B \$$	F id_A
3	归约	\$ T	$* \text{id}_B \$$	T F id_A
4	归约	\$ E	$* \text{id}_B \$$	E T F id_A

5	移入	$\$ E *$	$\text{id}_B \$$	$\begin{array}{c} E \\ \\ T \\ \\ F \\ \\ \text{id}_A \end{array} * $
6	移入	$\$ E * \text{id}_B$	$\$$	$\begin{array}{c} E \\ \\ T \\ \\ F \\ \\ \text{id}_A \end{array} * \text{id}_B$
7	归约	$\$ E * F$	$\$$	$\begin{array}{c} E \\ \\ T \\ \\ F \\ \\ \text{id}_A \end{array} * \begin{array}{c} F \\ \\ \text{id}_B \end{array}$
8	归约	$\$ E * T$	$\$$	$\begin{array}{c} E \\ \\ T \\ \\ F \\ \\ \text{id}_A \end{array} * \begin{array}{c} T \\ \\ F \\ \\ \text{id}_B \end{array}$
9	归约	$\$ E * E$	$\$$	$\begin{array}{c} E \\ \\ T \\ \\ F \\ \\ \text{id}_A \end{array} * \begin{array}{c} E \\ \\ T \\ \\ F \\ \\ \text{id}_B \end{array}$

图 4-25 词串 $\text{id}_A * \text{id}_B$ 的第 1 种移入归约分析过程

在分析的第 9 步结束时，栈顶的符号串并不能构成某个产生式的右部，因此不能采取归约动作。而输入缓冲区中已经没有符号了，因此也不能采取移入动作。可是分析栈中目前还有一堆符号，并不是接收状态要求的只有一个文法开始符号，说明分析失败了。但事实上，可以看出，这个词串实际上是该语言的合法句子。那么，问题出在哪里了呢？

事实上，在分析的第 3 步结束时，栈顶符号 T 虽然是产生式 (2) 的右部，但它并不是当前句型的句柄。虽然输入串 $\text{id}_A * \text{id}_B$ 的分析树目前还没有建立起来，但是它已经客观存在了，如图 4-26 所示。而在分析第 3 步结束时，当前句型为 “ $T * \text{id}_B$ ”。其对应分析树如图 4-27 所示。其中 “ T ” 在栈中，我们用分析树中

的红色节点表示。“ $* id_B$ ”在剩余输入缓冲区中，我们用分析树中的蓝色节点表示。我们可以很直观地看出，此时栈顶的 T 虽然是产生式(2)的右部，但在当前句型的分析树中，它自己构不成一棵子树的边缘，因此它不是当前句型的句柄，不能执行归约动作。在这种情况下，其实我们还有其他的选择，那就是执行移入动作。由此得到的后续分析步骤如图4-28所示。

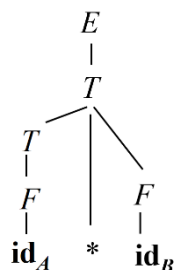


图4-26 句子“ $id_A * id_B$ ”对应的的分析树

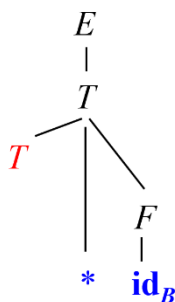


图4-27句型“ $T * id_B$ ”对应的的分析树

步骤	动作	分析栈	剩余输入	分析树
0		\$	$id_A * id_B$ \$	
1	移入	\$ id_A	$* id_B$ \$	id_A
2	归约	\$ F	$* id_B$ \$	F id_A
3	归约	\$ T	$* id_B$ \$	T F id_A

4	移入	$\$ T *$	$\text{id}_B \$$	$ \begin{array}{c} T \\ \\ F \\ \\ \text{id}_A \quad * \end{array} $
5	移入	$\$ T * \text{id}_B$	$\$$	$ \begin{array}{c} T \\ \\ F \\ \\ \text{id}_A \quad * \quad \text{id}_B \end{array} $
6	归约	$\$ T * F$	$\$$	$ \begin{array}{c} T \\ \quad \quad F \\ F \quad \quad \\ \quad \quad \\ \text{id}_A \quad * \quad \text{id}_B \end{array} $
7	归约	$\$ T$	$\$$	$ \begin{array}{c} T \\ / \quad \quad \backslash \\ T \quad * \quad F \\ \quad \quad \\ F \quad \quad \\ \quad \quad \\ \text{id}_A \quad * \quad \text{id}_B \end{array} $
8	归约	$\$ E$	$\$$	$ \begin{array}{c} E \\ \\ T \\ / \quad \quad \backslash \\ T \quad * \quad F \\ \quad \quad \\ F \quad \quad \\ \quad \quad \\ \text{id}_A \quad * \quad \text{id}_B \end{array} $

图 4-28 词串 $\text{id}_A * \text{id}_B$ 的第 2 种移入归约分析过程

□

那么在出现移入-归约冲突时，到底是该归约还是该移入呢？归根结底其实还是正确识别句柄的问题。如果当前栈顶的符号串是句柄，则应该归约；反之，如果不是句柄，就应采取移入动作。那么如何正确地识别句柄呢？接下来我们将在 4.4 节（LR 分析法）中将详细介绍。

4.4 LR 分析法

如果一个文法在自底向上的分析过程中既不存在归约-归约冲突，也不存在移入-归约冲突，它就能做确定的分析。什么样的文法不存在归约-归约冲突，也不存在移入-归约冲突？LR 文法就是这样的一类文法。

LR 文法（由 1974 年图灵奖得主 Donald Knuth 于 1963 年提出）是最大的、可以构造出相应移入-归约语法分析器的文法类。与 LL 文法类似，第 1 个字母表示扫描方向，即对输入进行从左（left）到右的扫描。第 2 个字母表示推导方式，当然，LR 分析不是一个推导过程，而是一个归约过程，即最左归约，也就是反向构造最右推导。所这里的推导方式 R 指的是最左归约对应的是最右（right）推导。

为准确识别句柄，在决定是否归约时，LR 分析器需要向前查看 k 个输入符号，这种 LR 分析器称为 $LR(k)$ 分析器。对于大多数描述程序设计语言的 CFG 来说，至多向前看一个输入符号就足够了，因此 $k = 0$ 和 $k = 1$ 这两种情况具有实践意义。当省略 (k) 时，表示 $k = 1$ 。能由 $LR(k)$ 分析器分析的 CFG，称为 $LR(k)$ 文法。

LR 分析法的基本原理

通过 4.3.2 节的分析可知，自底向上语法分析的关键问题是如何正确地识别句柄，也就是说当栈顶出现某个产生式的右部时，应不应该归约它。移入-归约分析过程中面临的两类冲突的本质也是如何正确识别句柄的问题。对于移入-归约冲突，如果栈顶的符号串是句柄就归约它，不是就移入下一个输入符号；对于归约-归约冲突，栈顶的哪个符号串是句柄就归约哪个符号串。

事实上，句柄是逐步形成的。我们知道，句柄是某个产生式的右部，而产生式的右部是由若干个符号构成的，所以我们可以用“项目（item）”表示句柄识别的进展程度。右部某位置标有圆点的产生式称为相应文法的一个 $LR(0)$ 项目（简称为项目）。

$$A \rightarrow \alpha_1 \cdot \alpha_2$$

例如，文法开始符号 S 表示语言中最大的成分，代表整个句子。对于产生式 $S \rightarrow bBB$ ，它的右部有 3 个符号，因此将 S 的识别过程划分为 4 个阶段。我们在产生式右部的不同的位置分别标记上圆点用来表示句柄识别的进展程度。

初始的时候，我们在期待构成 S 的第 1 个子成分（即终结符 b ）的出现。因此我们将圆点放置在产生式右部第 1 个符号前面，即

$$S \rightarrow \cdot bBB \quad (4.31)$$

此时，圆点后面紧跟着的是一个终结符，表示我们在等待该终结符出现以后将它从输入缓冲区中移入到栈中，因此这样的项目称为“移进项目”。

等到 b 移进栈里之后，句柄的识别向前进展一步，接下来期待识别出构成句柄的第 2 个子成分（即产生式右部的第 1 个“ B ”），因此将圆点移动到产生式右部第 2 个符号“ B ”前面，即

$$S \rightarrow b \cdot BB \quad (4.32)$$

等到归约出这个“B”之后，句柄的识别又向前进展一步，接下来期待识别出构成句柄的第3个子成分（还是一个“B”），因此将圆点移动到产生式右部第3个符号前面，即

$$S \rightarrow bB \cdot B \quad (4.33)$$

项目(4.32)和(4.33)都是将圆点放置在一个非终结符前面，表示等待归约出这个非终结符。这样的项目称为待约项目。

当第2个“B”归约出来以后，组成句柄的三个符号全部都已经识别出来了，此时将圆点移动到产生式右部末尾，即

$$S \rightarrow bBB \cdot \quad (4.34)$$

圆点处于产生式末尾，表示可以将产生式右部的符号串归约成左部非终结符S了，因此这样的项目称为“归约项目”。

对于产生式 $A \rightarrow \varepsilon$ ，它只生成一个项目 $A \rightarrow \cdot$ 。

项目描述了句柄识别的状态（进展程度）。LR分析器正是基于一系列状态（实际上是由项目组成的集合）来构造有穷自动机，从而正确地识别句柄。如果分析过程中某一步所处的状态指示我们将栈顶的某个符号串进行归约（对应于某产生式的归约项目），则说明该符号串是一个句柄。否则的话，即使栈顶的符号串是某个产生式的右部，如果状态没有指示我们对其归约，则说明它不是句柄。

LR分析器（自动机）的总体结构

LR分析器由一个输入缓冲区、一个分析栈、一个分析表和一个输出流构成，如图4-29所示。

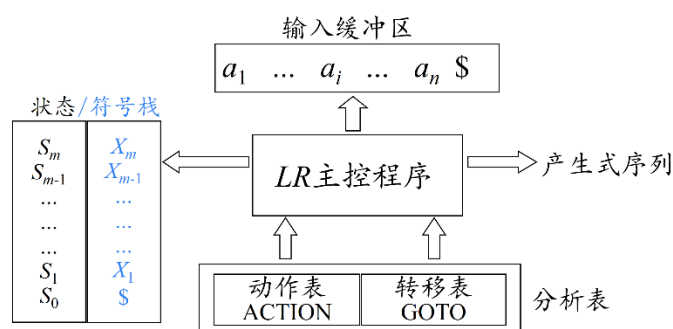


图 4-29 LR 分析器（自动机）的总体结构

(1) 分析栈

移入-归约分析器中包含一个符号栈。LR分析器除了符号栈以外，还包含一个与之并行的状态栈。事实上，符号栈是可以隐式存在的，因为从这些状态

能够复原出相应的文法符号。这里显式表示只是为了便于理解。这个栈称为下推存储器，起记忆功能。相应的，这样的自动机称为下推自动机（push down automata, PDA）。

下推自动机比有穷自动机 FA 的识别能力更强，FA 识别能力不强的主要原因是其记忆能力不强。比如说有一个语言，它的每个句子由 n 个 a 连接同样个数的 n 个 b 。因为要想识别这一语言，需要记忆读入的 a 的个数。而 FA 没有专门的存储器，只能利用不同的状态记忆所读入 a 的个数。由于 n 趋于无穷，而 FA 的状态数是有穷的，这是一对矛盾。因而，FA 无法识别该语言。

如果给 FA 增加一个下推存储器（即堆栈），那么每读入一个 a 就把它压入栈，这样，可以把 n 个 a 压入栈。之后每读入一个 b 就让一个 a 出栈，这样，当扫描到串尾时，如果恰好所有的 a 都出栈了，就表示 a 与 b 的个数相等，成功接收该串。

有穷自动机 FA 可以识别单词这样的正则语言，因为单词的构成规则比较简单，可以用正则文法描述。而句子结构远比单词结构复杂，需要用 CFG 来描述。在句子的识别过程中需要一个存储器来记录识别的进展程度（通过状态描述），因此需要构造下推自动机来完成。

(2) 分析表

LR 自动机的分析表是一个二维数组。其中每一行对应一个状态（正是因为有了状态的概念，LR 分析器才称之为是一个自动机。LL 分析器通常不称之为自动机，因为它的分析表中并没有涉及状态），每一列对应一个文法符号。根据文法符号类型的不同将分析表划分为两个子表。其中动作表 ACTION 每一列对应一个终结符（包括 $\$$ ），转移表 GOTO 的每一列对应一个非终结符。ACTION 表的每一个入口对应一个语法动作，包括移入 s （shift）、归约 r （reduce）、接收 acc 以及报错 err。GOTO 表的每一个入口对应一个后继状态。

LR 分析器的工作过程

接下来我们先通过一个例子来演示一下 LR 分析器是如何工作的。

例 4.23 图 4-30 显示了文法(4.35)的分析表。

$$(1) S \rightarrow BB$$

$$(2) B \rightarrow aB$$

$$(3) B \rightarrow b$$

(4.35)

状态	ACTION			GOTO	
	a	b	$\$$	S	B

0	s3	s4		1	2
1			acc		
2	s3	s4			5
3	s3	s4			6
4	r3	r3	r3		
5	r1	r1	r1		
6	r2	r2	r2		

图 4-30 文法(4.46)的分析表

其中动作 si 表示将当前输入符号移入符号栈，并转入 i 号状态（即，将 i 号状态压入状态栈）； ri 表示用第 i 个产生式进行归约； acc 表示成功接收；所有空白项表示报错。

给定输入串 bab ，LR 分析器的分析过程如图 4-31 所示。

步骤	动作	分析栈	剩余输入	分析树
0		0 \$	$bab\$$	
1	移入	04 \$b	$ab\$$	b
2	归约	02 \$B	$ab\$$	$\begin{array}{c} B \\ \\ b \end{array}$
3	移入	023 \$Ba	$b\$$	$\begin{array}{c} B \\ \\ b \end{array} \quad a$
4	移入	0234 \$Bab	$\$$	$\begin{array}{c} B \\ \\ b \end{array} \quad a \quad b$
5	归约	0236 \$BaB	$\$$	$\begin{array}{cc} B & B \\ & \\ b & a \end{array} \quad b$
6	归约	025 \$BB	$\$$	$\begin{array}{ccc} & B & \\ & & \diagdown \\ B & & B \\ & & \\ b & a & b \end{array}$

7	归约	01 \$S	\$	<pre> S / \ B B / \ / \ b a b </pre>
8	接收	01 \$S	\$	<pre> S / \ B B / \ / \ b a b </pre>

图 4-31 输入串 *bab* 的 LR 分析过程

初始时输入缓冲区中存放的是待分析的词串 *bab*（如图 4-20 第 0 步所示）。分析栈由两部分构成，上方是状态栈，下方符号栈。初始时符号栈存放的是栈底标记 \$，表示符号栈为空。状态栈初始时存放的是初始状态（0 号状态）。通过观察输入缓冲区，当前的输入符号是位于输入串首的终结符 *b*。根据 ACTION 表，ACTION[0, *b*]=s4，因此，接下来进行第 1 步，将当前输入符号 *b* 移入符号栈，并转入 4 号状态（即，将 4 号状态压入状态栈）。

此时，4 号状态位于栈顶，输入缓冲区当前的输入符号是 *a*。根据 ACTION 表，ACTION[4, *a*]=r3，因此，接下来进行第 2 步，将符号栈顶的 *b*（即产生式(3)的右部）归约为 *B*（即产生式(3)的左部），也就是说，将符号栈顶的 *b* 出栈，将 *B* 压入符号栈，由于状态栈与符号栈是平行同步的，因此 *b* 出栈时连同与之对应的状态栈中的 4 号状态一起出栈，这样，状态栈顶就露出了 0 号状态。通过查 GOTO 表，可知 GOTO[0, *B*]=2，即 0 号状态遇到刚刚归约出的 *B* 转入 2 号状态，因此，将 2 号状态压入状态栈。

此时，2 号状态位于栈顶，输入缓冲区当前的输入符号是 *a*。根据 ACTION 表，ACTION[2, *a*]=s3，因此，接下来进行第 3 步，将当前输入符号 *a* 移入符号栈，并转入 3 号状态（即，将 3 号状态压入状态栈）。

这个过程可以一直进行下去，直到第 7 步结束时，状态栈顶露出了 1 号状态，输入缓冲区中只剩输入结束标记 \$。通过查找 ACTION 表，ACTION[1, \$]=acc，说明输入的词串是一个合法的句子，因此进入第 9 步，成功接收。□

我们也可以将 LR 自动机的分析表用转换图的形式表示。图中的每个节点表示一个状态。连接状态 s_i 和状态 s_j 的边上的符号既可以是终结符，也可以是非终结符（即 LR 自动机的符号表由终结符集合和非终结符集合组成），但是对应的含义不一样。如果是终结符 *a*，表示状态 s_i 遇到一个移入的输入符号

a 进入状态 s_j 。如果是非终结符 A ，表示状态 s_i 遇到一个归约出来的非终结符 A 进入状态 s_j 。

我们来总结一下 LR 分析器的工作过程。

初始时，状态栈中只有一个状态，就是初始状态 s_0 。符号栈是空的，只有一个栈底标志 $\$$ 。输入缓冲区中存放待分析的符号串，后面连接着结束标记 $\$$ 。

$$\begin{array}{l} s_0 \\ \$ \qquad a_1 a_2 \dots a_n \$ \end{array}$$

一般情况下，假设分析器的当前格局为：

$$\begin{array}{l} s_0 s_1 \dots s_m \\ \$ X_1 \dots X_m \qquad a_i a_{i+1} \dots a_n \$ \end{array}$$

对于栈顶状态 s_m 和当前输入符号 a_i 和，查 ACTION 动作表。

①如果 ACTION $[s_m, a_i] = sx$ ，即对应表项是一个移入动作 sx ，那么格局变为：

$$\begin{array}{l} s_0 s_1 \dots s_m x \\ \$ X_1 \dots X_m a_i \qquad a_{i+1} \dots a_n \$ \end{array}$$

也就是说将当前输入符号 a_i 移入栈中，并进入状态 x 。

②如果 ACTION $[s_m, a_i] = rx$ ，表示用第 x 个产生式 $A \rightarrow X_{m-(k-1)} \dots X_m$ 进行归约，那么格局变为：

$$\begin{array}{l} s_0 s_1 \dots s_{m-k} \\ \$ X_1 \dots X_{m-k} A \qquad a_i a_{i+1} \dots a_n \$ \end{array}$$

也就是说将栈顶的 k 个符号连同它们对应的状态都出栈，然后让产生式左部非终结符 A 进栈。当前栈顶状态为 s_{m-k} ，在 GOTO 表中查找 s_{m-k} 和非终结符 A 的对应项。如果 GOTO $[s_{m-k}, A] = y$ ，则进入状态 y ，那么格局变为：

$$\begin{array}{l} s_0 s_1 \dots s_{m-k} y \\ \$ X_1 \dots X_{m-k} A \qquad a_i a_{i+1} \dots a_n \$ \end{array}$$

③如果 ACTION $[s_m, a_i] = acc$ ，那么分析成功

④如果 ACTION $[s_m, a_i] = err$ ，那么出现语法错误

最后，我们总结一下 LR 语法分析算法。

算法 4.24 LR 语法分析算法。

输入：串 w 和 LR 语法分析表，该表描述了文法 G 的 ACTION 函数和 GOTO 函数。

输出：如果 w 在 $L(G)$ 中，则输出 w 的自底向上语法分析过程中的归约步骤；否则给出一个错误指示。

方法：初始时，语法分析器栈中的内容为初始状态 s_0 ，输入缓冲区中的内容为 $w\$$ 。然后，语法分析器执行图 4-32 所示的程序。

```
令  $a$  为  $w\$$  的第一个符号;  
while(1) { /* 永远重复*/  
    令  $s$  是栈顶的状态;  
    if ( ACTION [ $s, a$ ] =  $st$  ) {  
        将  $t$  压入栈中;  
        令  $a$  为下一个输入符号;  
    }  
    else if ( ACTION [ $s, a$ ] = 归约  $A \rightarrow \beta$  ) {  
        从栈中弹出  $|\beta|$  个符号;  
        将 GOTO [ $t, A$ ] 压入栈中;  
        输出产生式  $A \rightarrow \beta$ ;  
    }  
    else if ( ACTION [ $s, a$ ] = 接受 )  
        break; /* 语法分析完成*/  
    else  
        调用错误恢复例程;  
}
```

图 4-32 LR 语法分析程序

□

事实上，LR 分析器的主控程序是通用的，也就是说，各种文法的 LR 分析器的主控程序是一样的，所不同的只是分析表而已。因此，构造 LR 语法分析器的关键是构造待分析语言的分析表。那么分析表是如何构造的呢？这里需要考虑两个问题。

(1) 分析表的每一行对应一个状态。因此需要给出这些状态的形式化定义，并据此列出所有的状态。

(2) 在每一个表项 ACTION[s, a] 中填入所要采取的动作。如前所述，这本质上是如何正确识别句柄的问题。也就是说，如果当前栈顶形成了句柄，则采取归约动作，否则采取移入或报错等动作。

为准确识别句柄，也就是说在决定是否归约时，LR 分析器需要向前查看 k 个输入符号，这种 LR 分析器称为 LR(k) 分析器。对于大多数描述程序设计语言的 CFG 来说，至多向前看一个输入符号就足够了，因此 $k=0$ 和 $k=1$ 这两种情况具有实践意义。接下来我们先介绍最简单的 LR 分析方法，即 LR(0) 分析。然后依次介绍 SLR(1) 分析、LR(1) 分析和 LALR(1) 分析。这些 LR 分析器的主控程序都是一样的（如图 4-32 所示）。所不同的是它们构造分析表的方法。

4.4.1 LR(0)分析法

LR(0)自动机通过维护一系列状态来表示语法分析的进展程度，从而做出移入-归约决定。为保证文法开始符号仅出现在一个产生式的左边，从而使得LR(0)自动机只有一个接收状态，需要对原始文法进行增广。如果 G 是一个以 S 为开始符号的文法，则 G 的增广文法（augmented grammar） G' 就是在 G 中加上新开始符号 S' 和产生式 $S' \rightarrow S$ 而得到的文法。容易看出，增广后的文法与原始文法是等价的。

例 4.25 对于例 4.5 中的文法(4.14)，其增广文法如下：

$$\begin{aligned}
 (0) \quad & E' \rightarrow E \\
 (1) \quad & E \rightarrow E + T \\
 (2) \quad & E \rightarrow T \\
 (3) \quad & T \rightarrow T * F \\
 (4) \quad & T \rightarrow F \\
 (5) \quad & F \rightarrow (E) \\
 (6) \quad & F \rightarrow \text{id}
 \end{aligned} \tag{4.36}$$

之前的文法(4.14)的开始符号 E 有两个产生式（分别是产生式(1)和(2)）。增广后，新的文法开始符号 E' 只出现在一个产生式的左部，这样可以使得后续构造出来的 LR(0) 自动机的接收状态是唯一的。

□

利用 LR(0)项目可以构造 LR(0)自动机。其基本原理是：根据文法反推句型的形成过程中需要经历哪些状态，并基于这些状态构造自动机。

例 4.26 对于例 4.21 中的文法(4.30)，其增广文法（略有简化，并加以缩写）及所有 LR(0)项目如图 4-33 所示。

产生式	项目
(0) $S' \rightarrow S$	(0) $S' \rightarrow \cdot S$
	(1) $S' \rightarrow S \cdot$
(1) $S \rightarrow \mathbf{v} I : T$	(2) $S \rightarrow \cdot \mathbf{v} I : T$
	(3) $S \rightarrow \mathbf{v} \cdot I : T$
	(4) $S \rightarrow \mathbf{v} I \cdot : T$
	(5) $S \rightarrow \mathbf{v} I : \cdot T$
	(6) $S \rightarrow \mathbf{v} I : T \cdot$
	(7) $I \rightarrow \cdot I, \mathbf{i}$
(2) $I \rightarrow I, \mathbf{i}$	(8) $I \rightarrow I \cdot, \mathbf{i}$
	(9) $I \rightarrow I, \cdot \mathbf{i}$
	(10) $I \rightarrow I, \mathbf{i} \cdot$

(3) $I \rightarrow i$	(11) $I \rightarrow \cdot i$
	(12) $I \rightarrow i \cdot$
(4) $T \rightarrow r$	(13) $T \rightarrow \cdot r$
	(14) $T \rightarrow r \cdot$

图 4-33 文法(4.30)的增广文法及所有 LR(0)项目

其中，项目(0)表示开始识别一个句子，称为初始项目。项目(1)表示整个句子已经识别完毕，称为接收项目。由于进行了文法增广，使得接收项目只有一个。项目(1)、(6)、(10)、(12)、(14)都是归约项目。同属于一个产生式的项目，但圆点的位置只相差一个符号，则称后者是前者的后继项目。即 $A \rightarrow \alpha \cdot X \beta$ 的后继项目是 $A \rightarrow \alpha X \cdot \beta$ 。例如，项目(4)是项目(3)的后继项目。

该文法对应的 LR(0)自动机构造过程如下。

首先构造自动机的初始状态（用 I_0 表示）。前面提到，LR(0)自动机的每个状态都是一个由项目组成的集合 I ，这里“ I ”是项目集合（item sets）的意思。同时，状态也刻画了分析栈中存放的符号信息（从初始状态 I_0 到某一状态 I_i 的路径上的符号序列表示处于状态 I_i 时符号栈中的符号串。根据 4.3.1 节的论述，符号栈中的符号串是分析到这一步时的规范句型的一个前缀，规范句型的其余部分处于剩余输入缓冲区中）。例如，初始状态 I_0 时符号栈是空的。

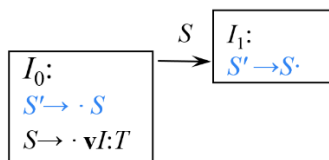
接下来，我们根据文法反推句型的形成过程中需要经历哪些状态。LR 分析的最终目标是识别出由文法开始符号 S 表示的一个句子。因此，分析一开始，我们期待着逐步将符号栈中后续出现符号最终归约为开始符号 S 。因此，我们将图 4-33 中的项目(0) $S' \rightarrow \cdot S$ 放入 I_0 ，意即等待归约出 S （如图 4-34 所示）。根据产生式(1)， S 是由符号串 $\mathbf{v}I:T$ 构成的。因此，等待 S 就相当于先要等待构成 S 第一个成分（即符号 $\mathbf{v}$ ）的出现，即项目(2) $S \rightarrow \cdot \mathbf{v}I:T$ 描述的情况（在这种情况下，称项目(2)是从项目(0)出发 ϵ -可达（ ϵ -reachable）的）。因此，我们将项目(2)也添加到 I_0 中（如图 4-34 所示）。

I_0 : $S' \rightarrow \cdot S$ $S \rightarrow \cdot \mathbf{v}I:T$
--

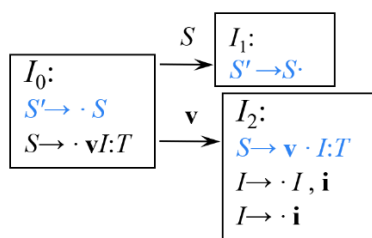
图 4-34 初始状态 I_0

刚刚提到，状态 I_0 中的项目(0) $S' \rightarrow \cdot S$ 表示等待归约出 S 。一旦这个 S 被归约出来，分析就向前进展一步，进入到 I_0 相对于 S 的后继状态 I_1 。因此，状态 I_1 应该包含项目(0)的后继项目（即项目(1) $S' \rightarrow S \cdot$ ），如图 4-35 所示。状态 I_1 中仅包含一个项目，并且是文法的接收项目，称为该自动机的接收状态，并且它没有后继状态（因为 I_1 中唯一的项目(1)作为一个归约项目没有后继项目）。由

于增广后的文法的接收项目是唯一的，因此，自动机的接收状态也是唯一的。根据图 4-35，从初始状态 I_0 到状态 I_1 的路径上的符号序列是“ S ”，表明进入状态 I_1 后，符号栈中存放的符号串是“ S ”。

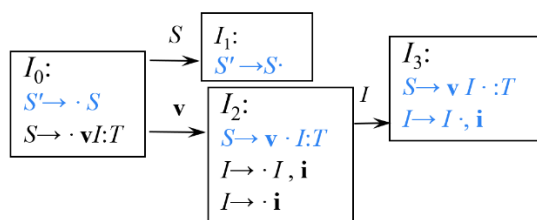
图 4-35 添加状态 I_1

状态 I_0 中的项目(2) $S \rightarrow \cdot v I : T$ 其圆点后面紧跟一个终结符 v ，表明当处于初始状态 I_0 时，如果剩余输入缓冲区中的当前输入符号（也是整个输入串的第 1 个输入符号）是 v ，则将它移入符号栈中以后，自动机将转入状态 I_0 相对于 v 的后继状态 I_2 。因此，状态 I_2 应该包含项目(2)的后继项目（即项目(3) $S \rightarrow v \cdot I : T$ ），如图 4-36 所示。此时（状态 I_2 ），符号栈中存放的符号串是“ v ”。对于新添加进状态 I_2 的项目(3) $S \rightarrow v \cdot I : T$ ，其圆点后面紧跟一个非终结符 I ，表示在等待归约出 I 。根据产生式(2)和(3)可知， I 要么是由符号串 I_i 构成的，要么是由单个符号 i 构成的。因此，等待 I 就相当于先要等待构成 I 的第 1 个子成分，即产生式(2)的右部第 1 个符号“ I ”（项目(7)描述的状况）或产生式(3)的右部第 1 个符号“ i ”（项目(11)描述的状况）。因此，项目(7)和项目(11)是从项目(3)出发 ε -可达的，将它们也添加进状态 I_2 。接下来再进一步考虑新添加进 I_2 的项目(7) $I \rightarrow \cdot I, i$ ，其圆点后面紧跟非终结符 I （与之前的项目(3)一样）。因此，从项目(7)出发 ε -可达的项目也是项目(7)和项目(11)。由于这两个项目已经在 I_2 中了，因此不需要重复添加。

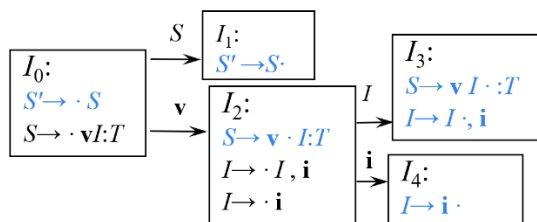
图 4-36 添加状态 I_2

状态 I_2 中的项目(3) $S \rightarrow v \cdot I : T$ 和项目(7) $I \rightarrow \cdot I, i$ 的圆点后面紧跟非终结符都是 I ，表示在等待归约出 I 。一旦这个 I 被归约出来，分析就向前进展一步，进入到状态 I_2 相对于文法符号 I 的后继状态 I_3 。因此，状态 I_3 应该包含项目(3) 和项目(7)的后继项目（即项目(4) $S \rightarrow v I \cdot : T$ 和(8) $I \rightarrow I \cdot, i$ ），如图 4-37 所示。此时（状态 I_3 ），符号栈中存放的符号串是“ $v I$ ”。根据状态 I_3 中的项目可知，当

符号栈中的符号串是“ vI ”时，如果输入缓冲区中的当前输入符号是“ $:$ ”（即项目(4) $S \rightarrow vI \cdot T$ 中紧跟圆点后的符号），则说明由 I 表示的标识符序列已经全部分析完了，接下来可以识别“ $:$ ”及紧跟其后的代表类型信息的 T 了；如果输入缓冲区中的当前输入符号是“ $,$ ”（即项目(8) $I \rightarrow I \cdot i$ 中紧跟圆点后的符号），则说明由 I 表示的标识符序列尚未分析完，接下来需要继续识别“ $,$ ”及由“ $,$ ”引导的下一个标识符 i 。

图 4-37 添加状态 I_3

状态 I_2 中的另一个项目(11) $I \rightarrow \cdot i$ 其圆点后面紧跟一个终结符 i ，表明当处于状态 I_2 时（此时符号栈中存放的符号串是 v ），如果剩余输入缓冲区中的当前输入符号是 i ，则将它移入符号栈中以后，自动机将转入状态 I_2 相对于 i 的后继状态 I_4 。因此，状态 I_4 应该包含项目(11)的后继项目（即项目(12) $I \rightarrow i \cdot$ ），如图 4-38 所示。此时（状态 I_4 ），符号栈中存放的符号串是“ $v i$ ”。状态 I_4 中包含归约项目(12) $I \rightarrow i \cdot$ ，表示当符号栈中存放的符号串是“ $v i$ ”时（即处于状态 I_4 时），栈顶的符号串“ i ”是一个句柄，应该将它归约成 I 。于是符号栈顶的符号串“ i ”出栈（与之对应，图 4-38 中的状态 I_4 沿着标记为“ i ”的导入边反向回退到状态 I_2 ）， I 进栈。状态 I_2 遇到新归约出的 I 转入状态 I_3 。

图 4-38 添加状态 I_4

按照这样的方式继续构造 I_3 的各个后继状态，往复下去，直到构造出自动机的全部状态，如图 4-39 所示。

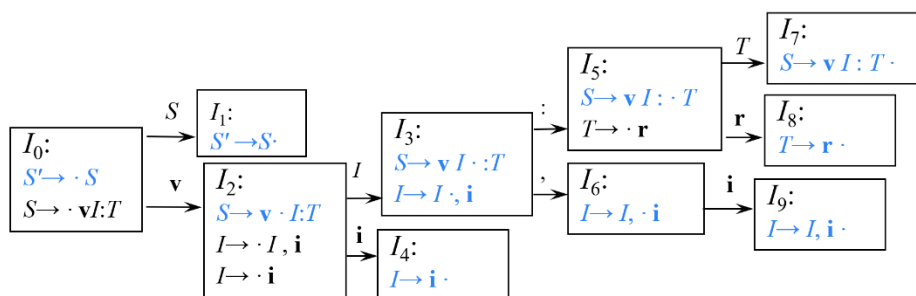


图 4-39 文法(4.36)的增广文法对应的 LR(0)自动机

□

正如我们在前面提到的，LR(0)自动机的状态刻画了分析栈中存放的符号信息，也就是说，从初始状态 I_0 到某一状态 I_i 的路径上的符号序列表示处于状态 I_i 时符号栈中的符号串。根据 4.3.1 节的论述，符号栈中的全部符号构成的串是分析到这一步时的规范句型的一个前缀（规范句型的其余部分处于剩余输入缓冲区中）。值得一提的是，符号栈中的这个规范句型的前缀不会包含句柄右侧的符号（因为一旦句柄在栈顶形成，它将立即被归约掉），这样的前缀称为活前缀。LR(0)自动机的工作过程实际上就是逐步产生规范句型的活前缀，可以看作是识别文法所有活前缀的自动机。

接下来，我们应用图 4-39 所示的 LR(0)自动机来重新分析一下例 4.21 中的输入串 $\text{var } i_A, i_B : \text{real}$ ，从而体会一下 LR(0)是如何帮我们正确地识别句柄。

例 4.27 应用图 4-39 所示的 LR(0)自动机重新分析输入串 $\text{var } i_A, i_B : \text{real}$ ，分析过程如图 4-40 所示。

步骤	动作	分析栈	剩余输入	分析树
0		0 \$	$\text{var } i_A, i_B : \text{real } \$$	
1	移入	0 2 \$ var	$i_A, i_B : \text{real } \$$	var
2	移入	0 2 4 \$ var i_A	$, i_B : \text{real } \$$	var i_A
3	归约	0 2 3 \$ var $\langle IDS \rangle$	$, i_B : \text{real } \$$	$\langle IDS \rangle$ var i_A

4	移入	0 2 3 6 \$ var <IDS> ,	$i_B : \text{real } \$$	$\begin{array}{c} \langle IDS \rangle \\ \\ \text{var } i_A , \end{array}$
5	移入	0 2 3 6 9 \$ var <IDS> , i_B	$: \text{real } \$$	$\begin{array}{c} \langle IDS \rangle \\ \\ \text{var } i_A , i_B \end{array}$
6	归约	0 2 3 \$ var <IDS>	$: \text{real } \$$	$\begin{array}{c} \langle IDS \rangle \\ / \quad \quad \backslash \\ \langle IDS \rangle \quad , \quad i_B \\ \\ \text{var } i_A \end{array}$
7	移入	0 2 3 5 \$ var <IDS> :	$\text{real } \$$	$\begin{array}{c} \langle IDS \rangle \\ / \quad \quad \backslash \\ \langle IDS \rangle \quad , \quad i_B : \\ \\ \text{var } i_A \end{array}$
8	移入	0 2 3 5 8 \$ var <IDS> : real	$\$$	$\begin{array}{c} \langle IDS \rangle \\ / \quad \quad \backslash \\ \langle IDS \rangle \quad , \quad i_B : \text{real} \\ \\ \text{var } i_A \end{array}$
9	归约	0 2 3 5 7 \$ var <IDS> : <T>	$\$$	$\begin{array}{c} \langle IDS \rangle \\ / \quad \quad \backslash \\ \langle IDS \rangle \quad , \quad i_B : \quad \langle T \rangle \\ \quad \quad \quad \\ \text{var } i_A \quad \text{real} \end{array}$
10	归约	0 1 \$ <S>	$\$$	$\begin{array}{c} \langle S \rangle \\ / \quad \quad \backslash \\ \langle IDS \rangle \quad , \quad \langle T \rangle \\ / \quad \quad \backslash \quad \\ \langle IDS \rangle \quad , \quad i_B : \quad \text{real} \\ \\ \text{var } i_A \end{array}$

图 4-40 词串 var $i_A, i_B : \text{real}$ 的 LR(0)分析过程

与图 4-22 相比，图 4-40 中“分析栈”一列增加了与符号栈并行的状态栈。

初始时刻，符号栈中除了栈底标记\$之外没有任何文法符号（如图 4-40 第 0 步所示）。相应的，状态栈中初始时只有 0 号状态（对应于自动机中的 I_0 ）。剩余输入缓冲区中存放待分析的句子，输入指针指向串首第 1 个符号 **var**（即图 4-39 中的 **v**）。

首先，第 1 步，根据图 4-39 所示的自动机，状态 I_0 遇到输入指针指向的当前输入符号 **var** 时，将 **var** 移入符号栈中，输入指针指向下一个输入符号 i_A ，

同时，自动机转入状态 I_2 ，如图 4-30 第 1 步所示。

接下来，根据图 4-39 所示的自动机，状态 I_2 遇到输入符号 i_4 时，将 i_4 移入符号栈中，输入指针指向下一个输入符号 “,”，同时，自动机转入状态 I_4 ，如图 4-30 第 2 步所示。

接下来，根据图 4-39 所示的自动机，状态 I_4 只包含归约项目 $I \rightarrow i \cdot$ ，表明此时符号栈顶的串 “ i ” 是一个句柄，需要对其进行归约。于是栈顶的符号串 “ i ” 出栈（与之对应，图 4-39 中的自动机从状态 I_4 沿着标记为 “ i ” 的导入边反向回退到状态 I_2 ）， I 进栈。栈顶的符号串 “ i ” 出栈时，与之并行的状态栈中的 4 号状态也跟着一起出栈。这样，状态栈顶露出 2 号状态（这与刚刚提到的 “图 4-39 中的自动机回退到状态 I_2 ” 是一致的）。根据图 4-39 所示的自动机，状态 I_2 遇到新归约出的 I 转入状态 I_3 。于是，将 3 号状态压入状态栈，如图 4-30 第 3 步所示。由于这一步采取的是归约动作，所以输入缓冲区中的当前输入符号没有改变，依然是 “,”。

按这种方式继续分析下去，当第 5 步结束时，自动机进入 9 号状态。根据图 4-39，状态 I_9 只包含归约项目 $I \rightarrow I, i \cdot$ ，表明此时符号栈顶的串 “ I, i ” 是一个句柄，需要对其进行归约。于是栈顶的符号串 “ I, i ” 出栈（与之对应，图 4-39 中的自动机从状态 I_9 依次沿着标记为 “ i ”、“,”、“ I ” 的导入边反向回退到状态 I_2 ），产生式左部符号 I 进栈。栈顶的符号串 “ I, i ” 出栈时，与之并行的状态栈中的 3 号、6 号和 9 号状态也跟着一起出栈。这样，状态栈顶露出 2 号状态（这与刚刚提到的 “图 4-39 中的自动机回退到状态 I_2 ” 是一致的）。根据图 4-39 所示的自动机，状态 I_2 遇到新归约出的 I 转入状态 I_3 。于是，将 3 号状态压入状态栈，如图 4-30 第 6 步所示。

值得一提的是，将图 4-30 与前面的图 4-22 对比，前面几步的操作都是一样的。但是在第 5 步结束时，虽然栈顶的单个符号 “ i ” 是产生式(3)的右部，但是自动机中的状态 I_9 没有指示将单个符号 “ i ” 进行归约，说明 “ i ” 不是句柄，不能对单个符号 “ i ” 进行归约。因此，图 4-30 的第 6 步没有像图 4-22 的第 6 步那样归约错误的符号串。由此可见，LR(0)可以正确地识别句柄。

接下来的几步推导方式与前面的步骤类似，这里不再赘述。 □

自动机构造过程可以通过程序自动实现。为此需要两个辅助函数，一个是 CLOSURE 函数，另一个是 GOTO 函数。CLOSURE 函数用来生成自动机的一个状态。GOTO 函数用来生成一个状态相对于某一个文法符号的后继状态。

CLOSURE 函数

从例 4.26 中 LR(0)自动机构造过程可以看出，LR(0)自动机的每个状态是一

个项目集合，其中的项目可以分为两类：

(1) 核心项目 (kernel items)

这类项目是在状态构造之初加入到状态中的项目，如图 4-39 中标记为蓝色字体的项目。可以看出，除了初始项目 $S' \rightarrow \cdot S$ 以外，所有核心项目的圆点都不在产生式右部的最左端。

(2) 非核心项目 (nonkernel items)

这些项目都是从内核项目出发 ε -可达的项目，如图 4-39 中标记为黑色字体的项目。可以看出，非核心项目的圆点都在产生式右部的最左端。

容易证明， ε -可达是传递的。利用求闭包的方法可以求出相应的“ ε -可达闭包”。CLOSURE 函数用于计算给定项目集 I 的闭包。

$$\text{CLOSURE}(I) = I \cup \{B \rightarrow \cdot \gamma \mid A \rightarrow \alpha \cdot B \beta \in \text{CLOSURE}(I), B \rightarrow \gamma \in P\} \quad (4.37)$$

式(4.37)表示，首先将项目集 I 加入到 $\text{CLOSURE}(I)$ 。接下来，如果 $\text{CLOSURE}(I)$ 中存在某个圆点后紧跟一个非终结符 B 的项目 $A \rightarrow \alpha \cdot B \beta$ ，说明在等待归约出一个 B 。由于 $B \rightarrow \cdot \gamma$ ，等待归约出 B 就相当于首先要等待构成 B 的第 1 个子成分出现，即项目 $B \rightarrow \cdot \gamma$ 描述的情形。因此，将项目 $B \rightarrow \cdot \gamma$ 作为由项目 $A \rightarrow \alpha \cdot B \beta$ 出发 ε -可达的项目添加进 $\text{CLOSURE}(I)$ 。随着新的项目（不妨记为 i ）不断添加进 $\text{CLOSURE}(I)$ ，又可能产生由 i 出发 ε -可达的项目（不妨记为 j ）并将其添加进 $\text{CLOSURE}(I)$ 。这个过程一直进行下去，直到没有新的项目可以添加进来，便形成的最终的闭包。

从例 4.26 中每个状态的构造过程可以看出，LR(0)自动机的每个状态实际上是由内核项的项目集闭包构成的。图 4-41 给出了 $\text{CLOSURE}(I)$ 函数的计算方法。

```

SetOfItems CLOSURE ( I ) {
    J = I;
    repeat
        for ( J 中的每个项  $A \rightarrow \alpha \cdot B \beta$  )
            for (  $G$  的每个产生式  $B \rightarrow \gamma$  )
                if ( 项  $B \rightarrow \cdot \gamma$  不在  $J$  中 )
                    将  $B \rightarrow \cdot \gamma$  加入  $J$  中;
    until 在某一轮中没有新的项被加入到  $J$  中;
    return J;
}

```

图 4-41 CLOSURE(I)函数的计算方法

GOTO 函数

GOTO 函数返回项目集 I 相对于文法符号 X 的后继项目集闭包，其数学

定义如下：

$$\text{GOTO}(I, X) = \text{CLOSURE}(\{A \rightarrow \alpha X \beta \mid A \rightarrow \alpha \cdot X \beta \in I\}) \quad (4.38)$$

函数的第 1 个参数的是一个项目集 I 。对于 I 中每个圆点后紧跟文法符号 X （第 2 个参数）的项目，计算它相对于 X 的后继项目。将这些后继项目放在一起构成一个集合，调用 **CLOSURE** 函数求该项目集的闭包，从而得到 I 相对于文法符号 X 的后继项目集闭包。直观地讲，项目集 I 是自动机的一个状态，**GOTO**(I, X) 函数用于生成状态 I 相对于文法符号 X 的后继状态（正如前面提到，**LR**(0) 自动机的每个状态是一个项目集闭包）。图 4-42 给出了 **GOTO**(I, X) 函数的计算方法。

```

SetOfItems GOTO ( I, X ) {
    将 J 初始化为空集;
    for ( I 中的每个项  $A \rightarrow \alpha \cdot X \beta$  )
        将项  $A \rightarrow \alpha X \beta$  加入到集合 J 中;
    return CLOSURE ( J );
}

```

图 4-42 **GOTO**(I, X) 函数的计算方法

构造 **LR**(0) 自动机的状态集

从初始项目 $S' \rightarrow \cdot S$ 出发，利用 **CLOSURE** 和 **GOTO** 函数可以构造 **LR**(0) 自动机的状态集。由于每一个状态是一个项目集和，因此状态的集合就是一个集族，称为规范 **LR**(0) 项集族（canonical **LR**(0) collection），用 C 表示。其数学定义如下：

$$C = \{I_0\} \cup \{J \mid \exists I \in C, X \in V_N \cup V_T, J = \text{GOTO}(I, X)\} \quad (4.39)$$

$$I_0 = \text{CLOSURE}(\{S' \rightarrow \cdot S\}) \quad (4.40)$$

式(4.39)表示，初始时， C 中只包含初始项目的项目集闭包（即状态 I_0 ）。接下来，对于 C 中的每个状态 I 以及每个文法符号 X ，将状态 I 相对于 X 的后继状态 **GOTO**(I, X) 也添加进 C 。对于新添加进 C 的状态，再进一步求该状态的后继状态。这个过程一直进行下去，直到没有新的状态可以添加进来。图 4-43 给出了规范 **LR**(0) 项集族的计算方法。

```

void items( G' ) {
    C = { CLOSURE ( {[ S' → · S ] } ) };
    repeat
        for ( C 中的每个项集 I )
            for ( 每个文法符号 X )
                if ( GOTO ( I, X ) 非空且不在 C 中 )
                    将 GOTO ( I, X ) 加入 C 中;
    until 在某一轮中没有新的项集被加入到 C 中;
}

```

}

图 4-43 规范 LR(0) 项集族的计算方法

LR(0)分析表的构造

根据文法 G 构造出规范 LR(0)项集族以及 GOTO 函数之后, 就可以构造该文法的 LR(0)分析表。

例 4.28 例 4.26 中图 4-39 描述了文法(4.36)的 LR(0)自动机的转换图。其对应的分析表如图 4-44 所示。

状态	ACTION						GOTO		
	v	:	,	i	r	\$	S	I	T
0	s2						1		
1						acc			
2				s4				3	
3		s5	s6						
4	r4	r4	r4	r4	r4	r4			
5					s8				7
6				s9					
7	r2	r2	r2	r2	r2	r2			
8	r5	r5	r5	r5	r5	r5			
9	r3	r3	r3	r3	r3	r3			

图 4-44 文法(4.36)的 LR(0)自动机的分析表

根据图 4-39, 状态 I_0 相对于非终结符 S 的后继状态是 I_1 , 因此 $GOTO[0,S]=1$ 。前面曾经提到过, 转移表 GOTO 的每一列对应一个非终结符。状态 I_0 相对于终结符 v 的后继状态是 I_2 , 因此 $ACTION[0,v]=s2$, 表示将 v 移入符号栈中以后, 自动机转入状态 I_2 。前面曾经提到过, 动作表 ACTION 的每一列对应一个终结符。

状态 I_1 没有后继状态, 因为它只包含了一个归约项目 $S' \rightarrow S \cdot$, 并且是文法唯一的接收项目。此时, 如果输入串的所有符号都已经分析完毕, 即输入指针指向 $\$$ 符号, 则表示输入串是该文法的一个合法的句子, 可以成功接收。因此, $ACTION[1,\$]=acc$ 。

状态 I_2 、 I_3 、 I_5 和 I_6 的处理方式与类似 I_0 类似, 这里不再赘述。

状态 I_4 中包含一个归约项目 $I \rightarrow i \cdot$, 表示此时应该应用产生式(4)进行归约(不管当前输入符号是什么)。因此, 对于任意一个终结符 a (包括 $\$$), $ACTION[4,a]=r4$ 。

状态 I_7 、 I_8 和 I_9 的处理方式与类似 I_4 类似, 这里不再赘述。 □

以下是 LR(0)分析表的构造算法。

算法 4.29 LR(0)分析表构造算法

输入：一个增广文法 G' 。

输出： G' 的 LR(0) 语法分析表函数 ACTION 和 GOTO。

方法：

(1) 构造 G' 的规范 LR(0)项集族 $C = \{ I_0, I_1, \dots, I_n \}$ 。

(2) 根据 I_i 构造得到状态 i 。状态 i 的语法分析动作按照下面的方法决定：

① 如果 $A \rightarrow \alpha \cdot a\beta \in I_i$ 并且 $GOTO(I_i, a) = I_j$ ，则 $ACTION[i, a] = sj$ ；

② 如果 $A \rightarrow \alpha \cdot B\beta \in I_i$ 并且 $GOTO(I_i, B) = I_j$ ，则 $GOTO[i, B] = j$ ；

③ 如果 $A \rightarrow \alpha \cdot \in I_i$ 且 $A \neq S'$ ，则对于 $\forall a \in V_T \cup \{\$, \}$ ， $ACTION[i, a] = rj$ (j 是产生式 $A \rightarrow \alpha$ 的编号)；

④ 如果 $S' \rightarrow S \cdot \in I_i$ ，则 $ACTION[i, \$] = acc$ 。

(3) 没有定义的所有条目都设置为“error”。

(4) 语法分析器的初始状态是由包含 $S' \rightarrow \cdot S$ 的项集构造得到的状态。 □

LR(0) 自动机的形式化定义

LR(0)分析技术的基本原理是根据文法

$$G = (V_N, V_T, P, S)$$

构造出对应的 LR(0)自动机 M

$$M = (C, V_N \cup V_T, GOTO, I_0, F)$$

其中，

M 的状态集合就是文法 G 的规范 LR(0)项集族 C ；

M 的符号表是文法非终结符和终结符的并集 $V_N \cup V_T$ ；

M 的转换函数就是我们前面定义的 GOTO 函数；

M 的初始状态就是初始项目的项目集闭包 I_0 ；

M 的终止状态集合只包含一个状态，就是接收项目所在的项目集闭包。

LR(0)分析过程中的冲突

并非所有的文法都能用 LR(0) 分析法进行分析。

例 4.30 对于下面的简单算数表达式文法(4.41)，其对应的 LR(0)自动机如图 4-45 所示。

$$(0) E' \rightarrow E$$

$$(1) E \rightarrow E + T$$

$$(2) E \rightarrow T$$

$$(3) T \rightarrow T * F$$

$$(4) T \rightarrow F$$

$$(5) F \rightarrow (E)$$

(6) $F \rightarrow id$

(4.41)

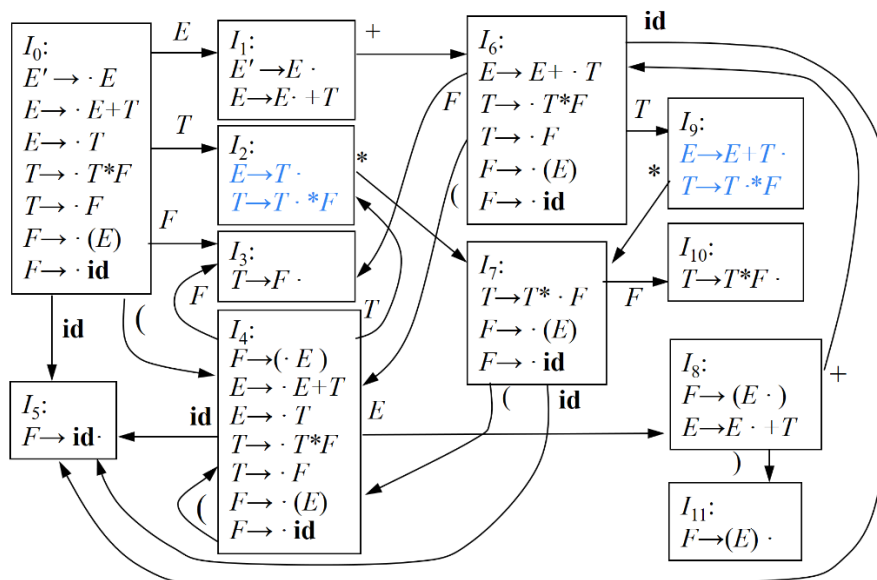


图 4-45 文法(4.41)的 LR(0)自动机

在构造 LR(0)分析表的过程中会遇到冲突的现象（如图 4-46 所示）。 I_2 和 I_9 中既有移入项目，又有归约项目，于是就产生了移入-归约冲突。

状态	ACTION						GOTO		
	id	+	*	(	)	\$	E	T	F
0	s5			s4			1	2	3
1		s6				acc			
2	r2	r2	r2/s7	r2	r2	r2			
3	r4	r4	r4	r4	r4	r4			
4	s5			s4			8	2	3
5	r6	r6	r6	r6	r6	r6			
6	s5			s4				9	3
7	s5			s4					10
8		s6			s11				
9	r1	r1	r1/s7	r1	r1	r1			
10	r3	r3	r3	r3	r3	r3			
11	r5	r5	r5	r5	r5	r5			

图 4-46 文法(4.41)的 LR(0)分析表

□

除了移进/归约冲突以外，有时还可能存在归约/归约冲突。

例 4.31 对于下面的文法(4.42)，其对应的 LR(0)自动机如图 4-47 所示。

(0) $S' \rightarrow T$

- $$\begin{aligned}
 (1) & T \rightarrow aBd \\
 (2) & T \rightarrow \varepsilon \\
 (3) & B \rightarrow Tb \\
 (4) & B \rightarrow \varepsilon
 \end{aligned}
 \tag{4.42}$$

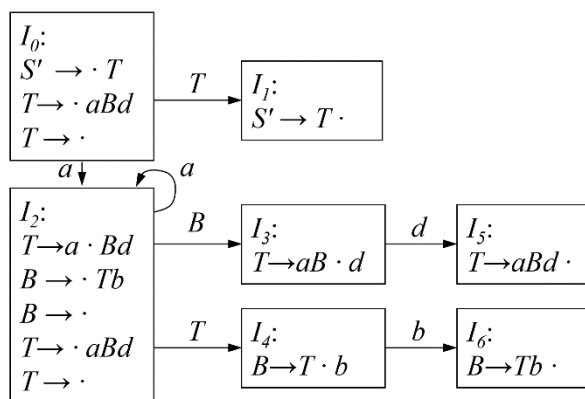


图 4-47 文法(4.42)的 LR(0)自动机

状态 I_2 中既有移入项目 ($T \rightarrow \cdot aBd$)，又有归约项目 ($B \rightarrow \cdot$ 和 $T \rightarrow \cdot$)，所以显然存在移入-归约冲突。移入项目 ($T \rightarrow \cdot aBd$) 表示在状态 I_2 时，如果输入缓冲区中当前的输入符号是 a ，则将 a 移入栈中，并继续留在状态 I_2 。而对于这两个归约项目，“ $B \rightarrow \cdot$ ”表明在状态 I_2 时，要将栈顶的一个空串归约为 B （无论输入缓冲区中当前的输入符号是什么，当然也包括是 a 的情况），即没有任何符号出栈， B 进栈；类似的，“ $T \rightarrow \cdot$ ”表明在状态 I_2 时，要将栈顶的一个空串归约为 T ，即没有任何符号出栈， T 进栈。那么，状态 I_2 时，到底是让 B 进栈还是让 T 进栈，这就构成了归约-归约冲突。更进一步，如果当前输入符号是 a 时，还会与移入项目 ($T \rightarrow \cdot aBd$) 构成移入-归约冲突，即到底是移入 a 进栈还是让 T 或 B 进栈。

□

由此给出 LR(0)文法的概念。对于给定的文法 G ，如果其 LR(0)分析表中没有语法分析动作冲突，那么文法 G 就称为 LR(0)文法。LR(0)文法要求每个项目集内（即自动机的一个状态）不存在冲突项。这个要求有点太苛刻，很难用这种文法描述各种语言成分。就连简单的算术表达式文法都还不是 LR(0)文法。如果项目中存在冲突，不同的解决方法导致了 SLR(1)文法和 LR(1)文法的出现。

4.4.2 SLR(1)分析法

4.4.1 节提到，并非所有的文法都能用 LR(0) 分析法进行分析，也就是说，对于某些文法，LR(0)自动机无法消解分析过程中的某些冲突。LR(0)自动机为什么消解不了这些冲突？

事实上，句柄都是相对一个句型而言的，因此应该将句柄的识别（归约）放在句型这样一个上下文环境中考虑。例如，在例 4.21 中，当分析到图 4-23 的第 5 步时，栈顶的 i_B 虽然是产生式(2)的右部，但是该不该对它归约（它是不是一个句柄）取决于它所处的句型。从句型对应的分析树（如图 4-24 所示）可以很直观的看出，要归约的串应该是一棵高度为 2 的子树的边缘（即直接短语）。虽然 i_B 是一个产生式右部，但是在当前句型中，它自己构不成一棵高度为 2 的子树的边缘，也就是说不能将 i_B 自己归约成 $\langle IDS \rangle$ ，因为 i_B 是有兄弟的，要把它和它的兄弟放在一起归约。因此，产生式右部未必都是句柄。句柄是相对一个句型而言的，因此应该将句柄的识别放在句型这样一个上下文环境中考虑。

对于 $A \rightarrow \beta$ ，是否应该将 β 归约为 A 取决于当前句型对应的分析树中 A 所处的上下文环境。上下文环境是指：当前句型对应的那棵客观存在的分析树，如图 4-48 所示。实际上我们肉眼也是根据这棵客观存在的分析树来决定该不该归约的。

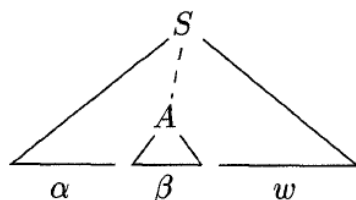


图 4-48 $\alpha\beta w$ 的语法分析树中的一个句柄 $A \rightarrow \beta$

LR(0)只考虑了 A 的上文（即规范句型的前缀）。事实上，前文信息已经通过前面的状态收集上来了。但未考虑 A 的下文（也就是说，构造分析表的过程中，当一个状态中含有归约项目时，无论下一个输入符号是什么，分析器都执行归约动作，相当于所做的决定跟下一个输入符号具体是啥没有关系，即没有考虑下文的信息，相当于向前看 0 个符号，这也是“LR(0)”中“0”的含义），因此消歧能力有限。

在例 4.30 的图 4-45 中，对于状态 I_2 ，此时，输入缓冲区中的下一个输入符号是“*”，假如我们把栈顶的 T 归约为 E 的话，那就意味着 E 后面将要紧跟着这个“*”。但是根据 E 的 follow 集可知， $\text{FOLLOW}(E) = \{) + \$ \}$ 中不包含

“*”，也就是说归约出的这个 E 后面不可能紧跟着 “*”，因此，这么归约是错误的，栈顶的这个 T 不是一个句柄。

这就是 SLR 分析法提出的基本思想：通过向前查看一个输入符号，并依据 FOLLOW 集来判断是否可以归约。具体来说，假设文法 G 的某个状态（项目集） I 中同时含有 m 个移进项目

$$\begin{aligned} A_1 &\rightarrow \alpha_1.a_1\beta_1 \\ A_2 &\rightarrow \alpha_2.a_2\beta_2 \\ &\dots \\ A_m &\rightarrow \alpha_m.a_m\beta_m \end{aligned}$$

和 n 个归约项目

$$\begin{aligned} B_1 &\rightarrow \gamma_1. \\ B_2 &\rightarrow \gamma_2. \\ &\dots \\ B_n &\rightarrow \gamma_n. \end{aligned}$$

如果集合 $\{a_1, a_2, \dots, a_m\}$ 和 $\text{FOLLOW}(B_1)$ 、 $\text{FOLLOW}(B_2)$ 、...、 $\text{FOLLOW}(B_n)$ 两两不相交，则项目集 I 中的冲突可以按以下原则解决：设 a 是输入缓冲区中的下一个输入符号，

- (1) 若 $a \in \{a_1, a_2, \dots, a_m\}$ ，则移进 a
- (2) 若 $a \in \text{FOLLOW}(B_i)$ ，则用产生式 $B_i \rightarrow \gamma_i$ 归约
- (3) 此外，报错。

因为这 $n+1$ 个集合互不相交，所以 a 最多只能属于其中的一个集合，所以采取的动作也是唯一的，不可能有冲突。这里我们只需向前查看一个符号，所以这个方法实际叫 SLR(1) 分析。由于 $k=1$ 时可以省略不写，因此叫 SLR 分析。这里的 S 是 simple（简单）的意思。之所以说简单，是因为它只需简单通过 FOLLOW 集就可以化解冲突。后面我们将会看到，有些冲突需要通过更复杂的方法才能化解。

根据以上原则，我们可以构造表达式文法(4.41)的 SLR 分析表（如图 4-49 所示）。可见，移入-归约冲突已被消解。

状态	ACTION						GOTO		
	id	+	*	(	)	\$	E	T	F
0	s5			s4			1	2	3
1		s6				acc			
2		r2	s7		r2	r2			
3		r4	r4		r4	r4			

4	s5			s4			8	2	3
5		r6	r6		r6	r6			
6	s5			s4				9	3
7	s5			s4					10
8		s6			s11				
9		r1	s7		r1	r1			
10		r3	r3		r3	r3			
11		r5	r5		r5	r5			

图 4-49 文法(4.41)的 SLR 分析表

类似的, 文法(4.42)的 SLR 分析表如图 4-50 所示。

状态	ACTION				GOTO	
	<i>a</i>	<i>b</i>	<i>d</i>	<i>\$</i>	<i>T</i>	<i>B</i>
0	s2	r2		r2	1	
1				acc		
2	s2	r2	r4	r2	4	3
3			s5			
4		s6				
5		r1		r1		

图 4-50 文法(4.42)的 SLR 分析表

算法 4.32 SLR 分析表构造算法。

输入: 一个增广文法 G' 。

输出: G' 的 LR(0) 语法分析表函数 ACTION 和 GOTO。

方法:

(1) 构造 G' 的规范 LR(0) 项集族 $C = \{I_0, I_1, \dots, I_n\}$ 。

(2) 根据 I_i 构造得到状态 i 。状态 i 的语法分析动作按照下面的方法决定:

① **if** $A \rightarrow \alpha \cdot a \beta \in I_i$ **and** $\text{GOTO}(I_i, a) = I_j$ **then** $\text{ACTION}[i, a] = sj$

② **if** $A \rightarrow \alpha \cdot B \beta \in I_i$ **and** $\text{GOTO}(I_i, B) = I_j$ **then** $\text{GOTO}[i, B] = j$

③ **if** $A \rightarrow \alpha \cdot \in I_i$ **and** $A \neq S'$ **then for** $\forall a \in \text{FOLLOW}(A)$ **do** $\text{ACTION}[i, a] = rj$ (j 是产生式 $A \rightarrow \alpha$ 的编号)

④ **if** $S' \rightarrow S \cdot \in I_i$ **then** $\text{ACTION}[i, \$] = \text{acc}$;

(3) 没有定义的所有条目都设置为 “error”。

(4) 语法分析器的初始状态是由包含 $S' \rightarrow \cdot S$ 的项集构造得到的状态。 □

SLR 分析表构造方法与 LR(0) 分析表不同之处仅在于归约项目的处理上。

构造 SLR 分析表时, 如果某个状态含有形如 $A \rightarrow \alpha \cdot$ 的归约项目, 则对于 $\text{FOLLOW}(A)$ 中的那些符号, 可以采取归约动作。而 LR(0) 分析表时, 是对于所有符号都采取归约动作。

如果给定文法的 SLR 分析表中不存在有冲突的动作，那么该文法称为 SLR 文法。SLR 分析法照 LR(0)分析法的性能更强大一些，但是在给某些文法构造 SLR 分析器的时候依然会产生冲突现象。

例 4.33 对于描述赋值语句的文法(4.43)

- (0) $S' \rightarrow S$
 - (1) $S \rightarrow L = R$
 - (2) $S \rightarrow R$
 - (3) $L \rightarrow *R$
 - (4) $L \rightarrow \text{id}$
 - (5) $R \rightarrow L$
- (4.43)

识别该文法的 SLR 自动机如图 4-51 所示。

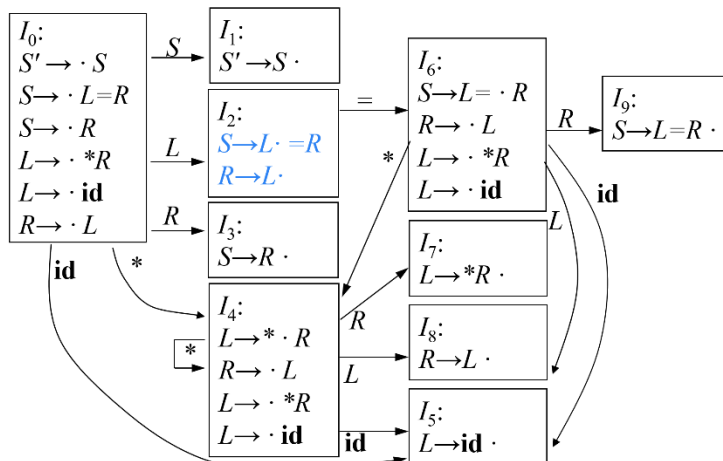


图 4-51 识别文法(4.43)的 SLR 自动机

对于状态 I_2 ， $\text{FOLLOW}(R) = \{ = \$ \}$ 中包含 “=”，因此遇到 “=” 时出现移入归约冲突。□

例 4.33 说明仅利用 FOLLOW 集的信息来解决冲突在某些情况下是不好使的，这就需要引入处理功能更强大的分析法，这就是我们接下来将要介绍的 LR(1)文法。

4.4.3 LR(1)分析法

我们先来分析一下 4.4.2 节介绍的 SLR 分析法为什么在某些情况下消解不了冲突。事实上，SLR 只是通过简单地考察下一个输入符号 b 是否属于与归约项目 $A \rightarrow \alpha$ 相关联的 $\text{FOLLOW}(A)$ 来决定是否采取归约动作，但 $b \in \text{FOLLOW}(A)$

实际上只是归约 α 的一个必要条件，而非充分条件。也就是说 b 不是 FOLLOW 集中的符号肯定不行，因为这是一个必要条件。但是是了也未必行，因为它不是充分条件。

事实上，对于产生式 $A \rightarrow \alpha$ 的归约，在不同使用位置， A 会要求不同的后继符号。例如在文法(4.43)中， $\text{FOLLOW}(R) = \{= \$\}$ 中包含 2 个符号。图 4-52(a) 是文法中的一个句型 $*L=L$ 对应的分析树。

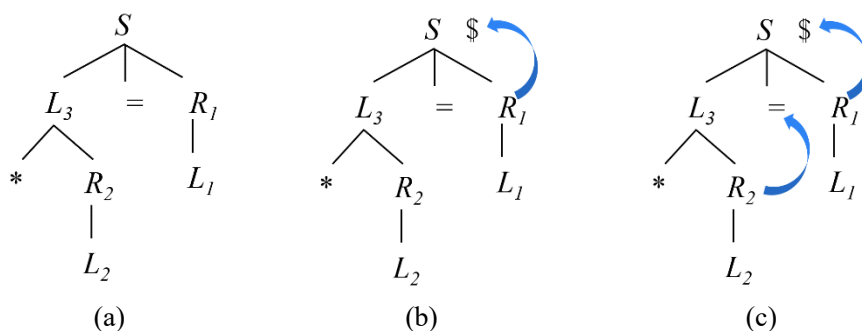


图 4-52 句型 $*L=L$ 对应的分析树

图中标记为 R 的节点有两处（分别是 R_1 和 R_2 。注意： R_i 跟 R 是同一个文法符号，只不过为了便于讨论，用下脚标来区分 R 在分析树不同位置的出现。 L 也是类似的情况）。这两处都是将 L 归约为 R ，但是， R_1 是 S 产生式右部最后一个符号，而 S 后面只能紧跟“\$”，因此 R_1 后面也只能紧跟“\$”。也就是说，对于 R_1 所处的这个位置，只有当输入缓冲区中的下一个输入符号是“\$”时，才可以把 L 归约为 R （如图 4-51(b)所示）。而“\$”也正是 $\text{FOLLOW}(R)$ 中的一个符号。

而对于 R_2 所处的这个位置， R_2 是产生式 $L_3 \rightarrow *R_2$ 右部的最后一个符号。而 L_3 后面紧跟的符号是“=”，也就是说，对于 R_2 所处的这个位置，只有当输入缓冲区中的下一个输入符号是“=”时，才可以把 L_2 归约为 R_2 （如图 4-51(c)所示）。而“=”也正是 $\text{FOLLOW}(R)$ 中的一个符号。

可见， $\text{FOLLOW}(A)$ 相当于是把各个位置能紧跟在 A 后面的符号合并起来构成的集合。但反过来，特定位置的后继符集合是 $\text{FOLLOW}(A)$ 的子集。所以，SLR 根据 FOLLOW 集来判定在各个位置是否可以归约实际上是扩大了可以归约的范围。

基于以上原因，LR(1)分析法在构造项目时，就直接考虑了后继符号信息。例如，为表示 R_1 的后继符号是“\$”，LR(1)分析法采用“ $R \rightarrow L \cdot, \$$ ”这种形式表示 R 产生式对应的归约项目；为表示 R_2 的后继符号是“=”，采用“ $R \rightarrow L \cdot, =$ ”这种形式表示 R 产生式对应的归约项目；为表示 L_3 的后继符号是“=”，

采用“ $L \rightarrow *R \cdot, =$ ”这种形式表示 L 产生式对应的归约项目；为表示 S 的后继符号是“\$”，采用“ $S \rightarrow L=R \cdot, \$$ ”这种形式表示 S 产生式对应的归约项目，如图 4-53 所示。

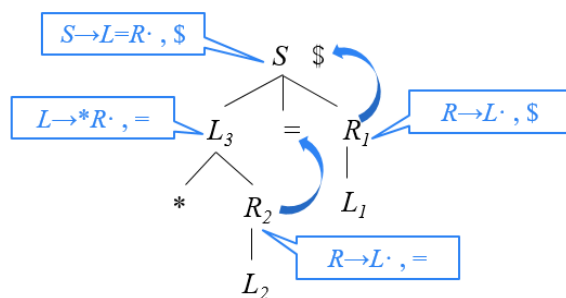


图 4-53 LR(1)项目示意图

由此，给出规范 LR(1)项目的通用形式：

$$[A \rightarrow \alpha \cdot \beta, a]$$

其中 $A \rightarrow \alpha \beta$ 是一个产生式， a 是一个终结符（这里将 $\$$ 视为一个特殊的终结符），它表示在当前状态下 A 后面必须紧跟的终结符，称为该项的展望符（lookahead）。LR(1) 中的 1 指的是项的第二个分量的长度。一个形如 $[A \rightarrow \alpha \cdot, a]$ 的项在只有在下一个输入符号是 a 时才可以按照 $A \rightarrow \alpha$ 进行归约。这样的 a 的集合总是 $\text{FOLLOW}(A)$ 的子集，而且它通常是一个真子集。在形如 $[A \rightarrow \alpha \cdot \beta, a]$ 且 $\beta \neq \varepsilon$ 的项中，展望符 a 没有任何作用。

与 LR(0) 自动机类似，LR(1) 自动机的每一状态也是用一个 LR(1) 项目集来表示。与 LR(0) 项目类似，有的 LR(1) 项目也存在 ε -可达的项目。对于形如 $[A \rightarrow \alpha \cdot B \beta, a]$ 的项目，假如 $B \rightarrow \gamma \in P$ ，那么，从项目 $[A \rightarrow \alpha \cdot B \beta, a]$ 出发 ε -可达的 LR(1) 项目的形式为 $[B \rightarrow \gamma \cdot, b]$ 。那么 b 应该是什么呢？

b 的含义是紧跟在 B 后面的终结符。由于在项目 $[A \rightarrow \alpha \cdot B \beta, a]$ 中， B 后面连接着符号串 β ，因此， $\text{FIRST}(\beta)$ 中的终结符就是项目 $[B \rightarrow \gamma \cdot, b]$ 中的展望符 b 。但是如果 β 可以为空的话，那么原项目 $[A \rightarrow \alpha \cdot B \beta, a]$ 中的展望符 a 就成为了项目 $[B \rightarrow \gamma \cdot, b]$ 的展望符。因此概括起来， b 就是 $\text{FIRST}(\beta a)$ 中的终结符。当 $\beta \Rightarrow^+ \varepsilon$ 时，此时 $b=a$ 叫继承的后继符，否则叫自生的后继符。

例 4.34 对于例 4.33 中的文法(4.43)，该文法对应的 LR(1) 自动机构造过程如下。

首先构造自动机的初始状态（用 I_0 表示）。首先将文法的初始项目

$$[S' \rightarrow \cdot S, \$] \quad (4.44)$$

放入 I_0 ，意即等待归约出 S ，并且紧跟在 S 后面的符号为“\$”，即该项目的展

望符。根据产生式(1)和(2)可知, S 要么由符号串 $L=R$ 构成, 要么由单个符号 R 构成。因此, 等待 S 就相当于先要等待构成 S 的第 1 个子成分, 即产生式(1)的右部第 1 个符号 L (该情况由项目

$$[S \rightarrow \cdot L = R, \$] \quad (4.45)$$

描述) 或产生式(2)的右部第 1 个符号 R (该情况由项目

$$[S \rightarrow \cdot R, \$] \quad (4.46)$$

描述)。因此, 将它们也添加进状态 I_0 。接下来再分别进一步考虑这两个新添加进 I_0 的项目。

对于项目(4.45), 其圆点后面紧跟非终结符 L , 意即等待归约出 L 。根据产生式(3)和(4)可知, L 要么由符号串 $*R$ 构成, 要么由单个符号 **id** 构成。因此, 等待 L 就相当于先要等待构成 L 的第 1 个子成分, 即产生式(3)的右部第 1 个符号 “*” (该情况由项目

$$[L \rightarrow \cdot *R, =] \quad (4.47)$$

描述) 或产生式(4)的右部第 1 个符号 **id** (该情况由项目

$$[L \rightarrow \cdot \text{id}, =] \quad (4.48)$$

描述)。因此, 将它们也添加进状态 I_0 。因为这两个项目中圆点后面紧跟的都是终结符, 因此没有新的 ϵ -可达项目添入 I_0 。

对于项目(4.46), 其圆点后面紧跟非终结符 R , 意即等待归约出 R 。根据产生式(5)可知, R 是由单个符号 L 构成的。因此, 等待 R 就相当于先要等待构成 R 的第 1 个子成分, 即产生式(5)的右部第 1 个符号 L (该情况由项目

$$[R \rightarrow \cdot L, \$] \quad (4.49)$$

描述)。因此, 将该项目也添加进状态 I_0 。对于项目(4.49), 其圆点后面紧跟非终结符 L , 意即等待归约出 L 。根据产生式(3)和(4)可知, L 要么由符号串 $*R$ 构成, 要么由单个符号 **id** 构成。因此, 等待 L 就相当于先要等待构成 L 的第 1 个子成分, 即产生式(3)的右部第 1 个符号 “*” (该情况由项目

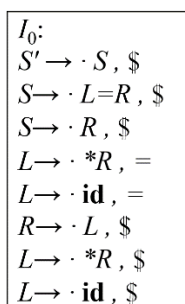
$$[L \rightarrow \cdot *R, \$] \quad (4.50)$$

描述) 或产生式(4)的右部第 1 个符号 **id** (该情况由项目

$$[L \rightarrow \cdot \text{id}, \$] \quad (4.51)$$

描述)。因此, 将它们也添加进状态 I_0 。因为这两个项目中圆点后面紧跟的都是终结符, 因此没有新的 ϵ -可达项目添入 I_0 。

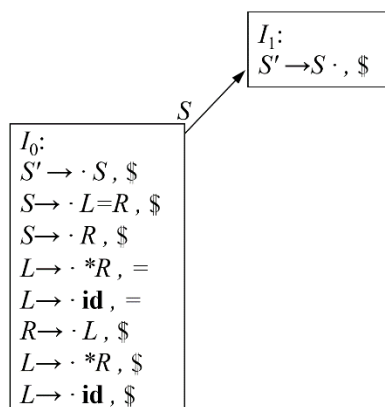
综上, 项目(4.44)~ (4.51)构成了 LR(1)自动机的初始状态 I_0 (如图 4-54 所示)。

图 4-54 初始状态 I_0

状态 I_0 中的第 1 个项目 $[S' \rightarrow \cdot S, \$]$ (即项目(4.44)) 表示等待归约出 S 。一旦这个 S 被归约出来, 分析就向前进展一步, 进入到 I_0 相对于 S 的后继状态 I_1 。因此, 状态 I_1 应该包含项目(4.44)的后继项目, 即项目

$$[S' \rightarrow S \cdot, \$] \quad (4.52)$$

注意, 一个 LR(1)项目 i 的后继项目 i' 与项目 i 本身的展望符是一样的, 不需要重新计算。只有在构造从某个 LR(1)项目 i 出发 ε -可达的 LR(1)项目 i' 时才需要重新计算展望符, 因为 i' 与 i 相比, 产生式左部符号表达的含义改变了, 因此展望符要重新计算。项目(4.52)是一个归约项目, 圆点后面没有紧跟任何非终结符, 因此没有从该项目出发 ε -可达的项目, 也就不会再有新的项目添加到 I_1 , 如图 4-55 所示。项目(4.52)是文法的接收项目, I_1 为该自动机的接收状态, 它没有后继状态。

图 4-55 添加状态 I_1

状态 I_0 中的项目(4.45)和(4.49)的圆点后面紧跟非终结符都是 L , 表示在等待归约出 L 。一旦这个 L 被归约出来, 分析就向前进展一步, 进入到状态 I_0 相对于文法符号 L 的后继状态 I_2 。因此, 状态 I_2 应该包含项目(4.45)和(4.49)的后继项目 (即项目

$$[S \rightarrow L \cdot = R, \$] \quad (4.53)$$

和

$$[R \rightarrow L \cdot, \$] \quad (4.54)$$

项目(4.53)和项目(4.54)的圆点后面没有紧跟任何非终结符，因此没有从它们出发 ϵ -可达的项目，也就不会再有新的项目添加到 I_2 ，如图 4-56 所示。

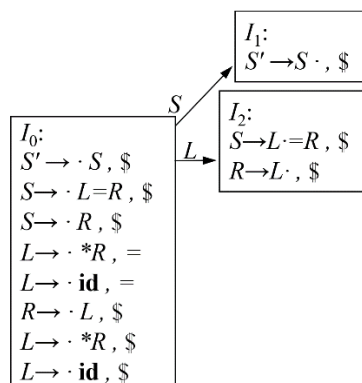


图 4-56 添加状态 I_2

按照这样的方式继续构造 I_0 的各个后继状态，往复下去，直到构造出自动机的全部状态，如图 4-57 所示。

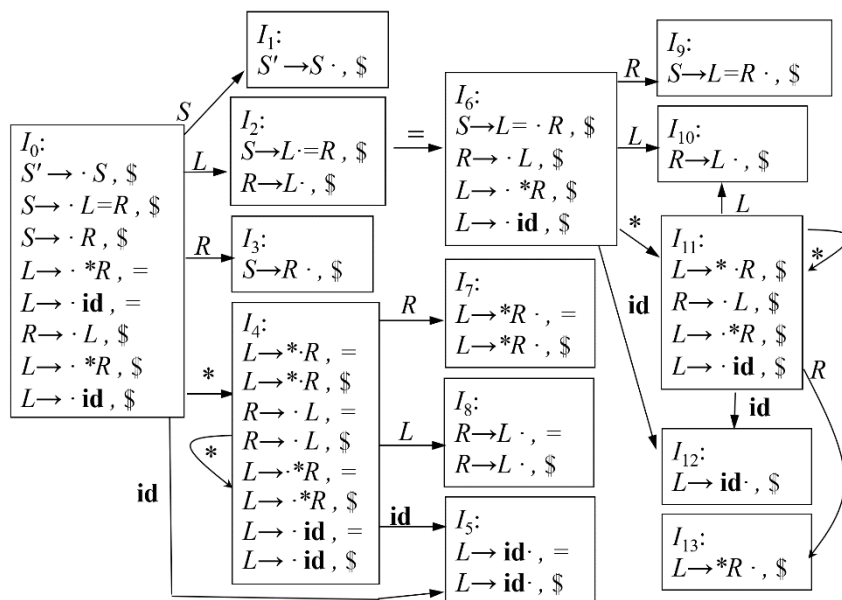


图 4-57 文法(4.43)对应的 LR(1)自动机

需要注意的是，状态 I_4 中的项目

$$[L \rightarrow \cdot * R, =] \quad (4.56)$$

和

$$[L \rightarrow \cdot *R, \$] \quad (4.57)$$

的圆点后面紧跟终结符“*”，表明当处于状态 I_4 时，如果剩余输入缓冲区中的当前输入符号是“*”，则将它移入符号栈中以后，自动机将转入状态 I_4 相对于“*”的后继状态（不妨设为 I_4' ）。因此，状态 I_4' 应该包含这两个项目的后继项目（即项目

$$[L \rightarrow * \cdot R, =] \quad (4.58)$$

和

$$[L \rightarrow * \cdot R, \$] \quad (4.59)$$

而这两个项目正是状态 I_4 的核心项目（核心项目的概念参见 4.4.1 节）。事实上，如果两个项目集的核心项目集合相同，那么这两个项目集闭包（即状态）就是相同的，因为项目集中的非核心项目都是从核心项目出发 ε -可达的项目（注意 ε -可达的可传递性）。因此， I_4 相对于“*”的后继状态 I_4' 与 I_4 是同一个状态。

根据图 4-57，可以构造文法(4.43)的 LR(1)分析表，如图 4-58 所示。

状态	ACTION				GOTO		
	*	id	=	\$	S	L	R
0	s4	s5			1	2	3
1				acc			
2			s6	r5			
3				r2			
4	s4	s5				8	7
5			r4	r4			
6	s11	s12				10	9
7			r3	r3			
8			r5	r5			
9				r1			
10				r5			
11	s11	s12				10	13
12				r4			
13				r3			

图 4-58 文法(4.43)的 LR(1)分析表

根据图 4-57，状态 I_0 相对于非终结符 S 的后继状态是 I_1 ，因此 $GOTO[0, S]=1$ 。状态 I_0 相对于非终结符 L 的后继状态是 I_2 ，因此 $GOTO[0, L]=2$ 。状态 I_0 相对于非终结符 R 的后继状态是 I_3 ，因此 $GOTO[0, R]=3$ 。状态 I_0 相对于终结符“*”的后继状态是 I_4 ，因此 $ACTION[0, *]=s4$ ，表示将“*”移入符号栈中以后，自动机转入状态 I_4 。状态 I_0 相对于终结符 id 的后继状态是 I_5 ，因此

$ACTION[0, id]=s5$, 表示将 id 移入符号栈中以后, 自动机转入状态 I_5 。

状态 I_1 没有后继状态, 因为它只包含了一个归约项目 $[S' \rightarrow S \cdot, \$]$, 并且是文法唯一的接收项目。此时, 如果输入串的所有符号都已经分析完毕, 即输入指针指向 $\$$ 符号, 则表示输入串是该文法的一个合法的句子, 可以成功接收。因此, $ACTION[1, \$]=acc$ 。

状态 I_2 中包含两个项目。第 1 个项目 $[R \rightarrow L \cdot, \$]$ 是一个归约项目, 该项目的展望符为 “ $\$$ ”, 表示如果输入缓冲区中的下一个输入符号是 “ $\$$ ”, 则应用产生式(5)进行归约。因此, $ACTION[2, \$]=r5$ 。第 2 个项目 $[S \rightarrow L \cdot = R, \$]$ 不是归约项目, 紧跟圆点后面是符号 “ $=$ ”。根据图 4-57, 状态 I_2 相对于终结符 “ $=$ ” 的后继状态是 I_6 , $ACTION[2, =]=s6$, 表示将 “ $=$ ” 移入符号栈中以后, 自动机转入状态 I_6 。

类似的, I_3 、 I_5 、 I_7 、 I_8 、 I_9 、 I_{10} 、 I_{12} 和 I_{13} 中都包含归约项目, 这些状态在输入缓冲区中的下一个输入符号为归约项目的展望符时才可以执行归约操作。

□

将图 4-57 所示的 LR(1)自动机与图 4-51 所示的 SLR/LR(0)自动机相比, LR(1)自动机多出来了几个状态 ($I_{10} \sim I_{13}$), 这是 LR(1)分析法为了细化输入信息而将状态进行分裂的结果。事实上, I_{10} 和 I_8 中的项目的第一个分量是相同的, 只是展望符集合不同, 但它们都是 $FOLLOW(R)$ 的子集。

如果除后继符外两个 LR(1)项目集是相同的, 则称这两个 LR(1)项目集是同心的。这里说的“心 (core)”, 实际就是 LR(0)项集。或者说, 如果两个 LR(1)项集的第一分量的集合是相同的, 则称这两个 LR(1)项目集是同心的。

I_{10} 和 I_8 是同心项目集。另外, I_{11} 与 I_4 号、 I_{12} 与 I_5 、 I_{13} 与 I_7 也分别是同心项目集。可见, LR(1)分析实际上是根据后继符集合的不同, 将原始的 LR(0)项目状态分裂成不同的 LR(1)状态。

构造 LR(1)项集族

构造有效 LR(1)项集族的方法实质上 and 构造规范 LR(0)项集族的方法相同。我们只需要修改两个过程: CLOSURE 和 GOTO 函数。

CLOSURE 函数用于计算 LR(1)项目集 I 的闭包。

$$CLOSURE(I) = I \cup \{ [B \rightarrow \gamma \cdot b] \mid [A \rightarrow \alpha \cdot B \beta, a] \in CLOSURE(I), B \rightarrow \gamma \in P, b \in \{ \beta a \} \} \quad (4.60)$$

GOTO 函数返回 LR(1)项目集 I 相对于文法符号 X 的后继项目集闭包, 其数学定义如下:

$$GOTO(I, X) = CLOSURE(\{ [A \rightarrow \alpha X \cdot \beta, a] \mid [A \rightarrow \alpha \cdot X \beta, a] \in I \}) \quad (4.61)$$

从初始 LR(1)项目 $[S' \rightarrow \cdot S, \$]$ 出发, 利用 CLOSURE 和 GOTO 函数可以构造 LR(1) 项集族。下面给出构造 LR(1) 项集族的算法。

算法 4.35 LR(1) 项集族的构造方法

输入: 一个增广文法 G' 。

输出: LR(1) 项集族 C 。

方法: 过程 CLOSURE 和 GOTO, 以及用于构造项集族的主例程 items, 见图 4-59。

```

SetOfItems CLOSURE ( I ) {
    repeat
        for ( I 中的每个项  $[A \rightarrow \alpha \cdot B\beta, a]$  )
            for (  $G'$  的每个产生式  $B \rightarrow \gamma$  )
                for ( FIRST( $\beta a$ ) 中的每个终结符  $b$  )
                    将  $[B \rightarrow \cdot \gamma, b]$  加入到集合  $I$  中;
    until 不能向  $I$  中加入更多的项;
    return I;
}

SetOfItems GOTO ( I, X ) {
    将  $J$  初始化为空集;
    for ( I 中的每个项  $[A \rightarrow \alpha \cdot X\beta, a]$  )
        将项  $[A \rightarrow \alpha X\beta, a]$  加入到集合  $J$  中;
    return CLOSURE ( J );
}

void items (  $G'$  ) {
    将  $C$  初始化为  $\{ \text{CLOSURE} ( \{ [S' \rightarrow \cdot S, \$] \} ) \}$  ;
    repeat
        for (  $C$  中的每个项集  $I$  )
            for ( 每个文法符号  $X$  )
                if ( GOTO( $I, X$ ) 非空且不在  $C$  中 )
                    将 GOTO( $I, X$ ) 加入  $C$  中;
    until 不再有新的项集加入到  $C$  中;
}

```

图 4-59 LR(1) 项集族的构造方法

LR(1)分析表的构造

下面给出从 LR(1)项目集构造 LR(1)分析表的算法, 其动作表和转移表的形式与 LR(0)的类似。

算法 4.36 LR(1)分析表构造算法

输入: 一个增广文法 G' 。

输出： G' 的 LR(1) 语法分析表函数 ACTION 和 GOTO。

方法：

- (1) 构造 G' 的 LR(1)项集族 $C = \{ I_0, I_1, \dots, I_n \}$ 。
- (2) 根据 I_i 构造得到状态 i 。状态 i 的语法分析动作按照下面的方法决定：
 如果 $[A \rightarrow \alpha \cdot a\beta, b] \in I_i$ 并且 $\text{GOTO}(I_i, a) = I_j$ ，那么 $\text{ACTION}[i, a] = \text{sj}$
 如果 $[A \rightarrow \alpha \cdot B\beta, b] \in I_i$ 并且 $\text{GOTO}(I_i, B) = I_j$ ，那么 $\text{GOTO}[i, B] = j$
 如果 $[A \rightarrow \alpha \cdot, a] \in I_i$ 且 $A \neq S'$ ，那么 $\text{ACTION}[i, a] = \text{rj}$
 如果 $[S' \rightarrow S \cdot, \$] \in I_i$ ，那么 $\text{ACTION}[i, \$] = \text{acc}$;
- (3) 没有定义的所有条目都设置为“error”。
- (4) 语法分析器的初始状态是由包含 $[S' \rightarrow \cdot S, \$]$ 的项集构造得到的状态。

□

4.4.4 LALR(1)分析法

我们在前面介绍 LR(1)分析的时候，举了赋值语句文法的例子（例 4.34），并且发现，LR(1)自动机中有很多同心项目集。也就是说，LR(1)分析实际上是根据后继符集合的不同，将原始的 LR(0)项目状态分裂成不同的 LR(1)状态。这样就使得 LR(1)自动机的状态数特别多。对于像 C 这样的语言，它的 LR(0)/SLR 分析表通常有几百个状态，而它的 LR(1)分析表通常有几千个状态。要想使 LR(1)分析技术实用化，必需想办法减少它的状态数。

于是我们就想，如果同心项目集对应的语法分析动作不冲突的话，可以考虑将它们合并，这样可以大大减少自动机的状态数，空间上更节省。例如，在例 4.34 的图 4-57 中， I_{10} 和 I_8 是同心项目集。 I_8 表示：当输入缓冲区的下一输入符号是=或\$时，将栈顶 L 归约为 R 。 I_{10} 表示：当输入缓冲区的下一输入符号是\$时，将栈顶 L 归约为 R 。 I_{10} 与 I_8 之间的动作并不冲突。在这种情况下，可以将它们合并。同理，同心项目集 I_{12} 与 I_5 、 I_{13} 与 I_7 也都是可以合并的。同心项目集 I_{11} 与 I_4 中没有归约项目，因此不会产生动作冲突（因为所有的冲突都是因归约动作而起），也是可以合并的。

事实上，合并同心项集并不总是一帆风顺的，下面来看一个例子。

例 4.37 对于下面的文法(4.62)，其对应的 LR(1)自动机如图 4-60 所示。

(0) $S' \rightarrow S$

(1) $S \rightarrow aAd|bBd|aBe|bAe$

(2) $A \rightarrow c$

(3) $B \rightarrow c$

(4.62)

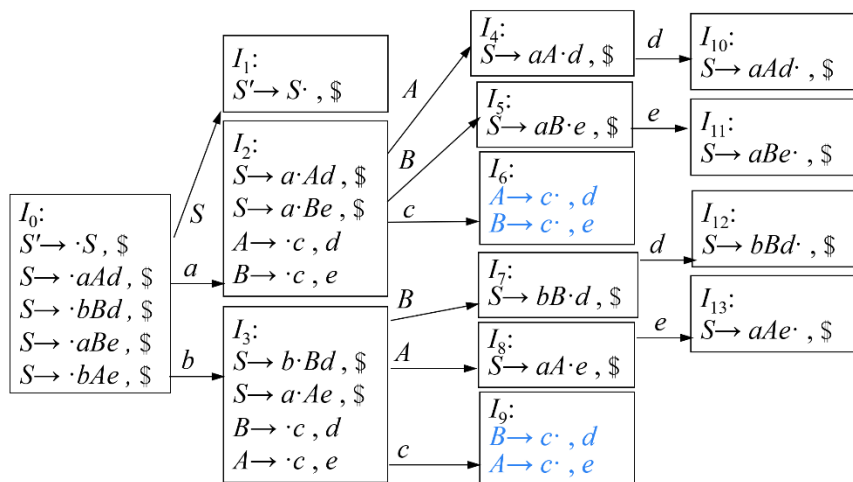


图 4-60 文法(4.62)对应的 LR(1)自动机

I_6 和 I_9 的第一分量的集合是相同的，因此它们是同心的。如果我把这两个同心项目集（状态）合并成一个状态，得到

$$A \rightarrow c \cdot, d|e$$

$$B \rightarrow c \cdot, d|e$$

那么在构造分析表的时候，对于合并后的状态，当输入缓冲区中下一个输入符号是 d 的时候， I_6 要求把栈顶的 c 归约成 A ，而 I_9 要求把栈顶的 c 归约成 B 。这就产生了冲突：到底是把栈顶 c 归约成 A 还是 B ？也就是说到底让 A 进栈还是让 B 进栈？遇到输入符号 e 的时候也是类似的情况。在这种情况下，同心项目集就不能合并。 □

合并同心项集会不会产生移进-归约冲突？答案是否定的。由于同心项集的心是相同的，所以合并的实际上是对应项的展望符集合。而展望符只在归约的时候起作用，移入时不起作用。因此，只要合并前的项目集中不存在移-归冲突，合并后也不会产生移-归冲突。

计算机科学家弗兰克·德雷默（Frank DeRemer）在其 1969 年的博士论文《实用 LR(k)翻译器》（Practical LR(k) Translators）中提出了 LR(k)理论的一个实用简化版本——LALR(1)（look-ahead LR(1)）。其基本思想是：寻找具有相同核心的 LR(1) 项集，并将这些项集合并为一个项集。根据合并后得到的项族构造语法分析表。如果分析表中没有语法分析动作冲突，给定的文法就称为 LALR(1)文法，就可以根据该分析表进行语法分析。下面给出 LALR(1)分析表

的构造算法。

算法 4.38 LALR(1)分析表的构造。

输入：一个增广文法 G' 。

输出： G' 的 LALR(1) 语法分析表函数 ACTION 和 GOTO。

方法：

(1) 构造 G' 的 LR(1)项集族 $C = \{I_0, I_1, \dots, I_n\}$ 。

(2) 对于 LR(1)项集中的每个心，找出所有具有这个心的项集，并将这些同心项目集替换为它们的并集。

(3) 令 $C' = \{J_0, J_1, \dots, J_m\}$ 为合并后的 LR(1)项目集族。状态 i 的语法分析动作是按照和算法 4.36 中的方法相同的方式根据 J_i 构造得到的。如果存在一个分析动作冲突，该算法就无法构造语法分析器，该文法就不是 LALR(1)的。

(4) 假设 J_i 是 $I_{i1}, I_{i2}, \dots, I_{il}$ 合并后的新项目集，由于这些项目集是同心的，所以 $\text{GOTO}(I_{i1}, X) = k_1, \text{GOTO}(I_{i2}, X) = k_2, \dots, \text{GOTO}(I_{il}, X) = k_l$ 也是同心的。令 K 是 $k_1, k_2, \dots, k_l$ 合并后的新项目集，则 $\text{GOTO}(J_i, X) = K$ 。

□

例 4.39 对于例 4.33 中的文法(4.43)，该文法对应的 LALR(1)自动机如图 4-61 所示。LALR(1)分析表如图 4-62 所示。

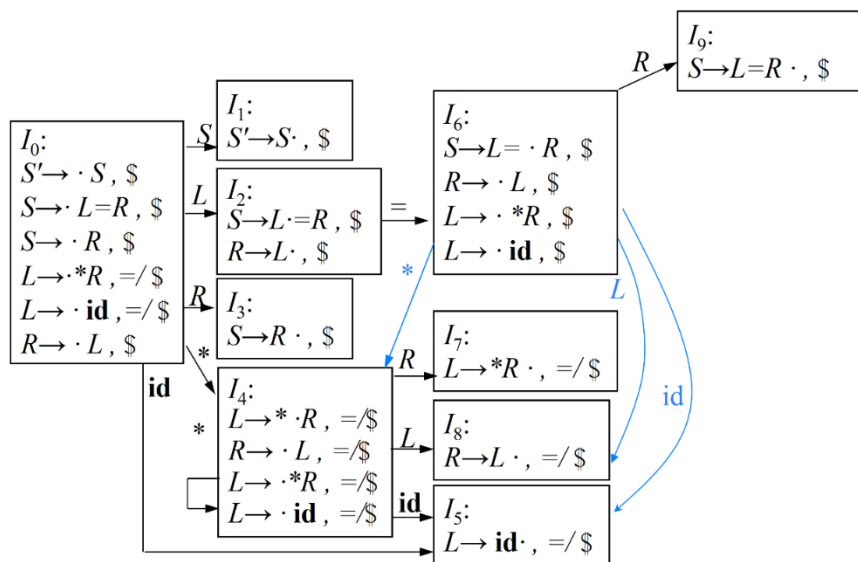


图 4-61 文法(4.43)对应的 LALR(1)自动机

状态	ACTION				GOTO		
	*	id	=	\$	<i>S</i>	<i>L</i>	<i>R</i>
0	s4	s5			1	2	3
1				acc			
2			s6	r5			
3				r2			
4	s4	s5				8	7
5			r4	r4			
6	s4	s5				8	9
7			r3	r3			
8			r5	r5			
9				r1			

图 4-62 文法(4.43)对应的 LALR(1)分析表

□

LALR 分析存在的问题

LALR 分析通过合并同心项目集虽然节省了空间，但是也会带来一定的问题，那就是遇到错误的时候，可能不能及时发现。也就是说，合并同心项目集以后可能会使本来不该归约的地方变成可以归约的了，本该报错的地方没有报错而是采取归约动作，我们通过一个例子来看一下这样操作会产生什么问题。

例 4.40 考虑文法(4.63)

- $$\begin{aligned}
 (0) \quad & S' \rightarrow S \\
 (1) \quad & S \rightarrow AcA \\
 (2) \quad & A \rightarrow aA \\
 (3) \quad & A \rightarrow d
 \end{aligned}
 \tag{4.63}$$

通过分析可以发现，该文法生成的正则语言如果用正则表达式表示即为

$$a^*dca^*d \tag{4.64}$$

该文法对应的 LR(1)自动机如图 4-63 所示。

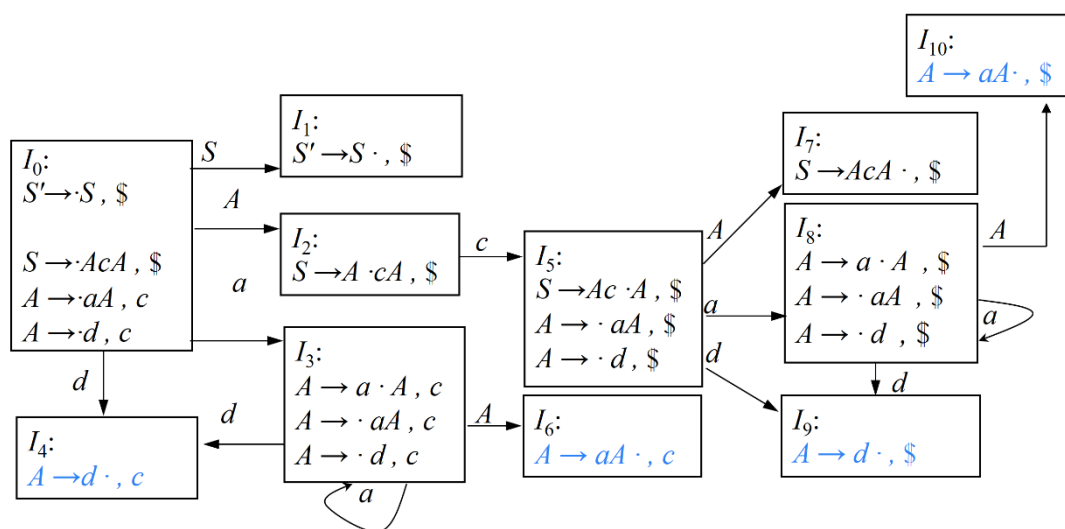


图 4-63 文法(4.63)对应的 LR(1)自动机

从输入串的第一个符号开始, 分析器在读入若干个 a 后, 一直处于状态 I_3 , 直到读入一个 d 后进入状态 I_4 。此时当输入缓冲区中下一输入符号是 c 时, 才可以将栈顶的 d 归约为 A 。虽然 $\$$ 也在 $\text{FOLLOW}(A)=\{c, \$\}$ 中, 但此时如果下一输入符号是 $\$$ 却不能归约, 因为根据正则表达式(4.64), 第一个 d 后面只能连接 c , 而第二个 d 后面只能连接 $\$$ 。

读入 d 进入状态 I_4 之后, 经过若干次归约, 最终将归约出 A 而进入状态 I_2 。 I_2 表示构成 S 产生式右部的第一个符号 A 已经识别出来了。接下来, 读入 c 进入状态 I_5 。在读入若干个 a 后, 一直处于状态 I_8 , 直到读入第二个 d 后进入状态 I_9 。此时当输入缓冲区中下一输入符号是 $\$$ 时, 才可以将栈顶的 d 归约为 A 。虽然 c 也在 $\text{FOLLOW}(A)=\{c, \$\}$ 中, 但此时如果下一输入符号是 c 就会报错。因为根据正则表达式(4.64), 句子中只能有一个 c 。而从初始状态 I_0 到状态 I_9 的路径上已经经过标记为 c 的边了, 因此, 状态 I_9 再遇到输入符号 c 就说明输入串不是合法的句子。

从图 4-63 可以看出, I_4 和 I_9 是同心项目集, I_6 和 I_{10} 是同心项目集, 并且不存在语法分析动作冲突。如果把 I_4 和 I_9 合并 (也就是说, 进入状态 I_4 后, 如果下一个输入符号为 $\$$ 时也执行归约), 会遇到什么问题?

假设现在输入一个错误的句子 $d\$$ 。之所以说它是错误的, 是因为根据正则表达式(4.64), 句子中必须包含两个 d , 而只有一个 d 就结束是不合法的。我们开始分析。初始状态 I_0 遇到输入符号 d , 将 d 移入符号栈, 进入状态 I_4 。此时, 输入缓冲区中下一个输入符号变成 $\$$ 。合并同心项集 I_4 和 I_9 之前, 正常此时会

报错，因为下一输入符号\$对于状态 I_4 来说是没有意义的。但合并同心项集之后，状态 I_4 遇到下一输入符号\$也执行归约动作，将栈顶的 d 连同状态 I_4 一起出栈，露出压在状态 I_4 下面的状态 I_0 。 A 进栈，状态 I_0 遇到刚归约出的 A 进入状态 I_2 。此时发现下一输入符号\$不合法。

由此可见，合并同心项集后，虽然在状态 I_4 没有立即发现错误，但是这个错误最终还是会被发现，只不过是推迟了一步进入状态 I_2 后才发现。实际上，这个错误会在移入任何新的输入符号之前就被发现，因为 LALR 分析法可能会作多余的归约动作，但绝不会作错误的移进操作。合并同心项集实际上是合并对应项的展望符集合，而移入动作跟展望符没有任何关系（正如 4.4.3 节提到的那样，展望符对移入项目或待约项目没有任何作用，展望符只对归约项目有意义），因此，合并展望符集合不会影响移入动作的正确性。

□

LALR(1)的特点

LALR(1)分析具有如下几个特点：

- (1) 形式上与 LR(1)相同，都带有展望符。
- (2) 大小上与 LR(0)/SLR 相当。由于 LALR(1)对 LR(1)的同心项目集进行了合并，导致状态数目大幅度的缩减，因此在规模上与 LR(0)/SLR 相当。
- (3) 分析能力介于 SLR 和 LR(1)二者之间。LALR(1)由于合并同心项集，可能会延迟发现一些错误，因此，分析能力不如 LR(1)。合并后的展望符集合仍为 FOLLOW 集的子集，也就是说，相对于 SLR 分析，LALR(1)进一步限制了可以归约的范围，也就是说，LALR(1)对于信息的划分比 SLR 更加精细，因此延迟发现的错误要比 SLR 少，分析能力强于 SLR。事实上，LR(0)、SLR、LALR(1)、LR(1)这几种分析方法的分析性能是依次增强的，即

$$LR(0) < SLR < LALR(1) < LR(1)$$

从分析表的构造过程可以看出，它们对归约项目的处理上，使得归约的范围在逐步缩小。在 LR(0)分析中，判定该不该归约的条件是对于所有的终结符都可以执行归约动作，即归约的范围是 $V_T \cup \{\$\}$ 。在 SLR 分析中，可以归约的范围进一步缩小到 FOLLOW(A)了。FOLLOW(A)是 $V_T \cup \{\$\}$ 的子集。在 LALR(1)中，合并后的展望符集合是 FOLLOW(A)的一个子集，所以可以归约的范围又进一步缩小。而 LR(1)中的每一个归约项目（合并同心项目集之前）是针对特定的展望符进行归约。所以从 LR(0)到 SLR，到 LALR(1)，到 LR(1)，归约的范围是在逐步地缩小，对信息的划分也就越来越精细，分析得也就越来越精准，性能就越来越强。

4.4.5 二义性文法的 LR 分析

任何二义性文法都不是 LR 文法，对于给定的输入词串可能存在多棵分析树，因此不能做确定的分析，其 LR 项目集族（不管是 LR(0)、SLR(1)、LALR(1)还是 LR(1)的项目集族）中都会存在动作冲突，但这并不意味着对二义性文法就都不能使用 LR 分析技术。

事实上，二义性文法并非都是缺点，它也有优点。某些二义性文法在语言的描述和实现中很有用。对于像表达式这样的语言构造，二义性文法(4.29)能够提供比任何等价的无二义性文法（如文法(4.41)）更简短、更直观的描述。简短的原因是：为消除二义性，需要引入更多的非终结符（如 T 、 F ），因而非二义性文法需要更多的产生式，其 LR 自动机的状态数也会较多。

下面通过两个例子来说明怎样基于二义性文法来构造 LR 分析器，其主要思想是出现冲突时引入消除二义性的规则。

例 4.41 将二义性文法(4.29)重写于此

$$(1) E \rightarrow E + E$$

$$(2) E \rightarrow E * E$$

$$(3) E \rightarrow (E)$$

$$(4) E \rightarrow id$$

该文法对应的 LR(0)自动机如图 4-64 所示。

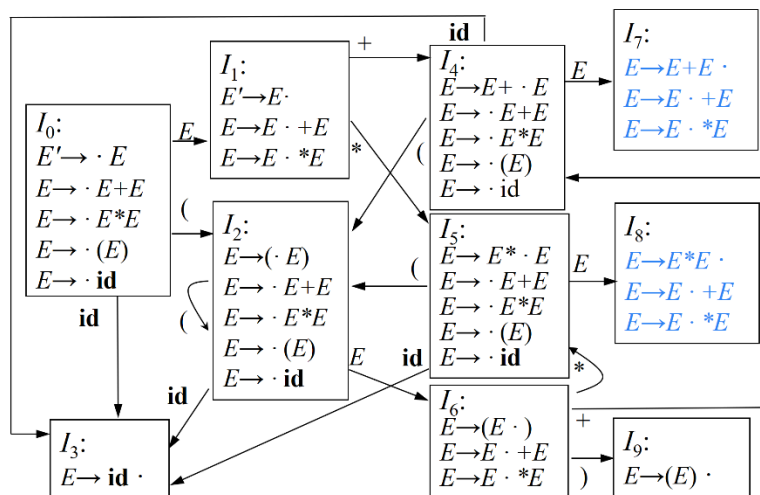


图 4-64 文法(4.29)对应的 LR(0)自动机（含冲突）

由于该文法具有二义性，因此在构造 LR 语法分析表时会出现分析动作冲突，如图 4-64 的状态 I_7 和 I_8 。

对于状态 I_7 ，其中第 1 个项目是归约项目，根据 SLR 分析法，由于 $\text{FOLLOW}(E) = \{ + *) \$ \}$ ，因此，当输入缓冲区中的下一个输入符号是 +、*、) 或 \$ 时应该采取归约动作。而第 1 个和第 3 个项目是移入项目，表示当输入缓冲区中的下一个输入符号是 + 或 * 时应该采取移入动作。这样就产生了冲突。 I_8 也有类似的问题。然而，这些冲突可以使用 + 和 * 的优先级与结合性信息来化解。

当处于状态 I_7 时，栈顶是一个符号串 $E+E$ ，如果栈外（输入缓冲区）下一个输入符号是 “*”，那么由于乘法的优先级高于加法，因此，应该先算乘法，后算加法。于是，我们应该采取移入动作，之后转入状态 I_5 。也就是说从 I_7 向 I_5 引出一条标记为 “*” 的边（如图 4-65 所示）。如果栈外下一个输入符号是 “+”，由于加法是左结合的，因此，应该先算栈内的加法，于是，我们采用归约动作。

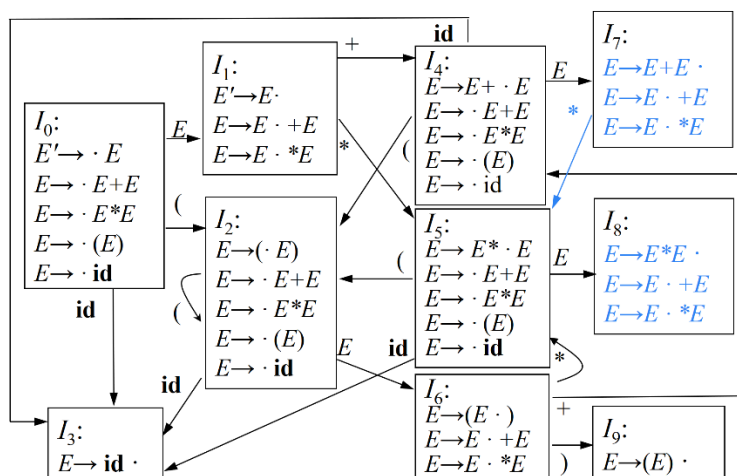


图 4-65 文法(4.29)对应的 LR(0)自动机（消解冲突后）

而当处于状态 I_8 时，栈顶是一个符号串 $E * E$ ，那么不管栈外下一个输入符号是 “*” 还是 “+”，无论是根据优先级还是结合性，都应该先算栈内的乘法，于是，我们采用归约动作，这样的话， I_8 不存在后继状态（如图 4-65 所示）。

由此得到的 LR(0)语法分析表如图 4-66 所示。

状态	ACTION						GOTO
	id	+	*	(	)	\$	E
0	s3			s2			1
1		s4	s5			acc	
2	s3			s2			6
3		r4	r4		r4	r4	

4	s3			s2			7
5	s3			s2			8
6		s4	s5		s9		
7		r1	s5		r1	r1	
8		r2	r2		r2	r2	
9		r3	r3		r3	r3	

图 4-66 二义性文法(4.29)对应的 LR 分析表

□

例 4.42 将二义性文法(2.8)重写于此

```

stmt → if expr then stmt
      | if expr then stmt else stmt
      | other

```

为简化讨论，我们考虑这个文法的一个抽象表示

$$\begin{aligned}
 S' &\rightarrow S \\
 S &\rightarrow iS \mid iSeS \mid a
 \end{aligned}
 \tag{4.65}$$

其中， i 表示 **if expr then**， e 表示 **else**， a 表示所有其它的产生式。该文法对应的 LR(0) 自动机如图 4-67 所示。

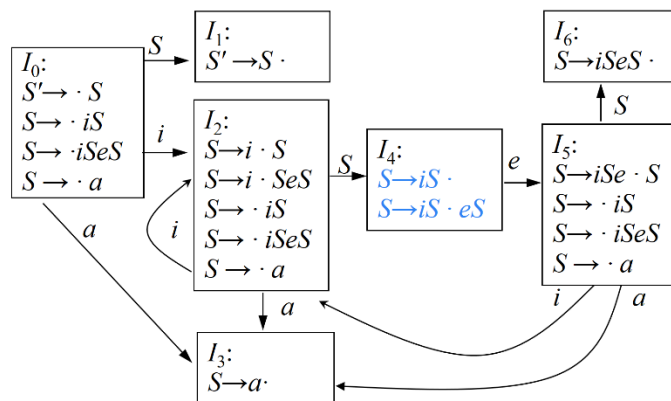


图 4-67 文法(4.65)对应的 LR(0) 自动机

容易看出，状态 I_4 存在分析动作冲突。对于其中的归约项目“ $S \rightarrow iS \cdot$ ”，根据 SLR 分析法，由于 $\text{FOLLOW}(S) = \{e, \$\}$ ，因此，当输入缓冲区中的下一个输入符号是 e 或 $\$$ 时应该采取归约动作。而第 2 个项目是移入项目，表示当输入缓冲区中的下一个输入符号是 e 时应该采取移入动作。这样就产生了冲突。

我们在第 2 章介绍二义性文法时提到过，可以通过引入最近匹配原则来消解该文法的歧义。那么根据最近匹配原则，进入状态 I_4 后，如果输入缓冲区中

下一输入符号是 e ，应该选择移进 **else**，以便让 **else** 与它前面最近的 **then** 配对。于是得到图 4-68 所示的语法分析表。

状态	ACTION				GOTO
	i	e	a	$\$$	S
0	s2		s3		1
1				acc	
2	s2		s3		4
3		r3		r3	
4		s5		r1	
5	s2		s3		6
6		r2		r2	

图 4-68 文法(4.65)对应的 LR 分析表

□

值得注意的是，应该保守地使用二义性文法，并且必须在严格控制之下使用，因为稍有不慎就会导致语法分析器所识别的语言出现偏差。

4.4.6 LR 分析中的错误处理

当 LR 语法分析器在查询语法分析动作表 ACTION 并发现一个报错条目（即表项为空）时，就检测到了一个语法错误。空白的表项没有对应的语义动作，说明输入的词串存在错误，是不合法。

与词法分析和 LL 文法的预测分析类似，LR 语法分析器在检测到语法错误后需要进行错误恢复。这里我们介绍两种错误恢复策略：一种是恐慌模式错误恢复，另一种是短语层次错误恢复。

恐慌模式错误恢复

假设在 LR 分析的某一步，分析器的格局如图 4-69 所示。

$s_0 s_1 \dots s_i s_{i+1} \dots s_m$	
$\$ X_1 \dots X_i X_{i+1} \dots X_m$	$a_j a_{j+1} \dots a_{j+k} a_{j+k+1} \dots a_n \$$

图 4-69 LR 分析某一步时分析器的格局

左边是分析栈（由并行的状态栈和符号栈构成），右边是输入缓冲区。分析到某一步的时候，栈顶的状态是 s_m ，输入缓冲区中下一个输入符号是 a_j 。此时查 ACTION 表。如果 $\text{ACTION}[s_m, a_i] = \text{err}$ ，则检测到了语法错误，说明已经

进栈的输入符号串与剩余的符号串无法正常衔接起来构成一个句型，也就是说在这个“衔接处”存在错误的成分。不妨假设这个错误的成分是 A ，那么可以认为 A 的一部分已经在栈顶($X_{i+1} \dots X_m$)，一部分仍在输入缓冲区中($a_i a_{i+1} \dots a_{i+k}$)，如图 4-70 所示。

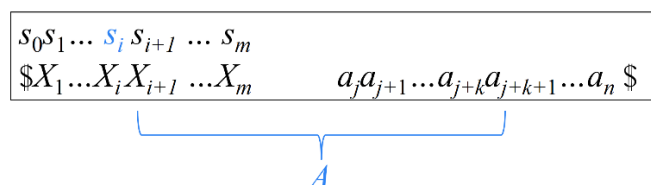


图 4-70 LR 分析中的恐慌模式错误恢复示意图

如果成分 A 内部不存在错误的话，就能归约出 A ，那么 s_i 遇到归约出的 A 就会转入一个新的状态。但是现在成分 A 存在错误，为了使分析器能够继续运行下去，可以采取恐慌模式进行处理，使得分析器能够尽快从错误状态中恢复过来。基本思想就是：放弃对 A 的识别，因此从栈顶和输入缓冲区两个方面分别忽略掉一些符号，直到可以将两部分衔接起来，并假装已经成功归约出 A ，因此可以将符号 A 和状态 $\text{GOTO}(s_i, A)$ 压入栈中，继续进行正常的语法分析。

实践中可能会选择多个这样的非终结符 A 。通常这些非终结符代表了主要的程序段，比如表达式、语句或块。比如，如果 A 是非终结符 $stmt$ ， a 就可能是分号或者 $\}$ 。其中， $\}$ 表示一个语句序列的结束。

从栈顶向下扫描，直到发现某个状态 s_i ，它有一个对应于某个非终结符 A 的 GOTO 目标，可以认为从这个 A 推导出的串中包含错误。然后丢弃 0 个或多个输入符号，直到发现一个可能合法地紧跟在 A 之后的符号 a 为止。之后将 $s_j = \text{GOTO}(s_i, A)$ 压入栈中，继续进行正常的语法分析。

短语层次错误恢复

短语层次错误恢复的基本思想是：检查 LR 分析表中的每一个报错条目，并根据语言的使用方法来决定程序员所犯的何种错误最有可能引起这个语法错误。然后构造出适当的恢复过程。

例 4.43 再次考虑例 4.41 中的表达式文法(4.29)

$$(1) E \rightarrow E + E$$

$$(2) E \rightarrow E * E$$

$$(3) E \rightarrow (E)$$

$$(4) E \rightarrow \text{id}$$

该文法对应的 LR(0)自动机和分析表如图 4-65 和 4-66 所示。

图 4-66 所示的分析表中有一些空白的表项，表示报错条目。我们在每个空

白表项中填写一个指向错误处理例程（error routines）的指针。该例程执行编译器设计者所选定的恢复动作。

首先来看状态 I_0 。 I_0 在期待读入一个运算分量的第一个符号，这个符号可能是 **id** 或 “(”。但是如果读入的是 +、*、\$，就表明出现了“缺少运算分量”的错误。于是我们调用错误处理例程 e_1 。 e_1 将状态 I_3 （即状态 I_0 相对于 **id** 的后继状态）压入栈中，也就是假设缺少的运算分量已经被我们补充上了，然后可以继续正常的语法分析。如果读入的是右括号，就表明出现了“不匹配的右括号”的错误。于是我们调用错误处理例程 e_2 ， e_2 从输入中删除 “)”，发出诊断信息“不匹配的右括号”。

I_2 、 I_4 和 I_5 也是同样的处理。

I_1 在期待读入一个运算符（“+”或“*”）或者是\$（此时分析成功结束）。但是如果读入的是“(”或 **id**，就表明出现了“缺少运算符”的错误。于是我们调用错误处理例程 e_3 。 e_3 将状态 I_4 （即状态 I_1 相对于“+”的后继状态）压入栈中，也就是假设缺少的运算符已经被我们补充上了，然后可以继续正常的语法分析。如果读入的是“)”，也表明出现了“不匹配的右括号”的错误。于是我们调用错误处理例程 e_2 ， e_2 从输入中删除 “)”，发出诊断信息“不匹配的右括号”。

I_6 在期待读入一个运算符（“+”或“*”）或一个“)”。但是如果读入的是“(”或 **id**，则调用错误处理例程 e_3 。 e_3 将状态 I_4 （即状态 I_6 相对于“+”的后继状态）压入栈中，也就是假设缺少的运算符已经被我们补充上了，然后可以继续正常的语法分析。如果读入的是\$，表明出现了“缺少右括号”的错误。于是我们调用错误处理例程 e_4 ， e_4 将状态 I_9 （即状态 I_4 相对于“)”的后继状态）压入栈中，也就是假设缺少的右括号已经被我们补充上了，然后可以继续正常的语法分析。

I_3 、 I_7 、 I_8 和 I_9 中含有归约项目，在遇到非法符号时，我们将采用这个归约动作。这种修复可能会使得报错推迟，但错误仍然会在任何移入动作发生之前被发现。

以 I_7 为例，遇合法符号*时采取移入动作；遇合法符号+、)、\$时采取归约动作；**id** 和 “(” 是非法符号，因为它们既不是 I_7 中某个移入项目的圆点后的终结符（因此不能采取移入动作），也不在 $\text{FOLLOW}(E)=\{+*) \$\}$ 中（因此不能采取归约动作）。对于这两个非法符号，即使我们采取了归约动作归约出 E 以后，也会发现它们不可能跟在 E 的后面，因此在进行错误处理之前不会把它们移入栈中。事实上，归约后分析器会进入状态 I_1 ，**id** 和 “(” 对 I_1 来说依

然是非法符号，这个错误在这里最终会被发现。此时会调用错误处理例程 e3 进行处理。

带有错误处理子程序的 LR 分析表如图 4-71 所示。

状态	ACTION						GOTO
	id	+	*	(	)	\$	<i>E</i>
0	s3	e1	e1	s2	e2	e1	1
1	e3	s4	s5	e3	e2	acc	
2	s3	e1	e1	s2	e2	e1	6
3	r4	r4	r4	r4	r4	r4	
4	s3	e1	e1	s2	e2	e1	7
5	s3	e1	e1	s2	e2	e1	8
6	e3	s4	s5	e3	s9	e4	
7	r1	r1	s5	r1	r1	r1	
8	r2	r2	r2	r2	r2	r2	
9	r3	r3	r3	r3	r3	r3	

图 4-71 带有错误处理子程序的 LR 分析表

其中，错误处理例程 e1~e4 执行的操作总结如下：

- e1：将状态 3 压入栈中，发出诊断信息“缺少运算分量”；
- e2：从输入中删除“)”，发出诊断信息“不匹配的右括号”；
- e3：将状态 4 压入栈中，发出诊断信息“缺少运算符”；
- e4：将状态 9 压入栈中，发出诊断信息“缺少右括号”。

□

4.5 语法分析器自动生成工具

本节将介绍如何使用语法分析器生成工具来帮助构造一个编译器的前端。我们将使用 LALR 语法分析器生成工具 Yacc 作为我们讨论的基础，因为它实现了我们在前一节中讨论的很多概念，并且这个工具很容易获得。Yacc 表示“yet another compiler-compiler”，即“又一个编译器的编译器”。这个名字反映出当 S.C. Johnson 在 20 世纪 70 年代早期创建出 Yacc 的第一个版本时，语法分析器生成工具非常流行。Yacc 在 UNIX 系统中是以命令的方式出现的，它已经用于实现多个编译器产品。

4.5.1 Yacc 的使用

就像 LEX 是由 Lex 语言和 Lex 编译器两部分组成，YACC 也由 YACC 语

言和 Yacc 编译器两部分组成。

(1) Yacc 语言。从 Yacc 工具使用者的角度，其目的是利用该工具为某语言 X 构造语法分析器，所以，他需要先描述一下该语言 X 的语法规则，也就是用 Yacc 语言编写源程序来描述语言 X 的语法定义（本质上是 CFG）。由 Yacc 语言编写的程序叫做“Yacc 源程序”，它是 Yacc 编译器的输入。

(2) Yacc 编译器。Yacc 工具使用者编写的 Yacc 源程序本质上是一个 LR 文法。Yacc 编译器的任务就是将输入的 LR 文法转化成一个用 C 语言编写的语法分析器（本质上是一个 LALR 自动机）。

Yacc 的使用过程如图 4-72 所示。用户用 Yacc 语言编写 Yacc 源程序（如图中 `translate.y`），用来描述某语言 X 的语法规则。因为源文件是由 Yacc 语言编写的，因此其扩展名为“.y”。将 `translate.y` 输入给 Yacc 编译器，Yacc 编译器输出一个用 C 语言编写的 X 语言的语法分析器 `y.tab.c`（由于是用 C 语言编写的，所以扩展名为“.c”）。由于 `y.tab.c` 是用 C 语言编写的，不能直接在机器上运行，因此要通过 C 编译器编译后得到可运行的扫描器 `a.out`。将 X 语言的源程序输入给语法分析器 `a.out`，输出的就是语法分析结果。

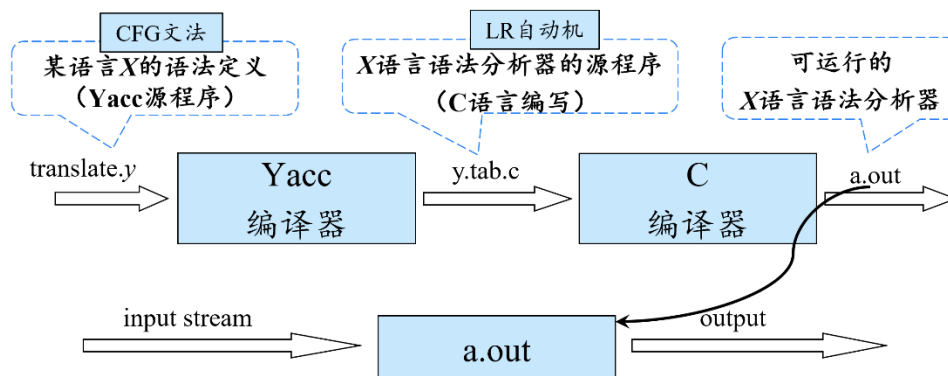


图4-72 Yacc使用方法示意图

从 Yacc 工具使用者的角度，只要用 Yacc 语言编写一个源程序来描述 X 语言的语法规则就可以了，剩下的工作都是由 Yacc 来完成的。接下来的关键问题是如何编写 Yacc 程序。

4.5.2 Yacc 程序的结构

一个 Yacc 源程序由三个部分组成（各部分之间由一个%%对分割）：
声明部分

%%

翻译规则

%%

辅助性 C 语言过程

%

例 4.44 为了说明如何编写一个 Yacc 源程序，我们构造一个简单的桌上计算器。该计算器读入一个算术表达式，对表达式求值，然后打印出表达式的结果。我们将从下面的算术表达式文法(4.14)开始构造这个桌上计算器：

$$\begin{aligned} E &\rightarrow E + T \mid T \\ T &\rightarrow T * F \mid F \\ F &\rightarrow (E) \mid \text{digit} \end{aligned}$$

其中的词法单元 **digit** 是一个 0-9 之间的数字。根据这个文法得到的 Yacc 桌上计算器程序显示在图 4-73 中。

```
%{
#include <ctype.h>
%}

%token DIGIT

%%
line : expr '\n'      { printf("%d\n", $1); }
    ;
expr : expr '+' term  { $$ = $1 + $3; }
    | term
    ;
term : term '*' factor { $$ = $1 * $3; }
    | factor
    ;
factor : '(' expr ')'  { $$ = $2; }
    | DIGIT
    ;

%%
yylex() {
    int c;
    c = getchar();
    if (isdigit(c)) {
        yylval = c-'0';
        return DIGIT;
    }
    return c;
}
```

图4-73 一个简单的桌上计算器的Yacc源程序

□

声明部分

一个 Yacc 程序的声明部分分为两节，它们都是可选的。在第一节中放置通常的 C 声明，这个声明用 `%{` 和 `%}` 括起来。那些由第二和第三部分中的翻译规则及过程使用的临时变量都在这里声明。在图 4-73 中，这一节只包含 `include` 语句

```
#include <ctype.h>
```

这个语句使得 C 语言的预处理器将标准头文件 `<ctype.h>` 包含进来，这个头文件中包含了断言 `isdigit()`。

在声明部分中还包括对词法单元的声明。在图 4-73 中，语句

```
%token DIGIT
```

声明 `DIGIT` 是一个词法单元。在这一部分中声明的词法单元可以在第二和第三部分中使用。如果向 Yacc 语法分析器传送词法单元的词法分析器是使用 Lex 创建的，那么如 3.5.2 节中讨论的，Lex 生成的词法分析器也可以使用这里声明的词法单元。

翻译规则部分

每个规则由一个文法产生式和一个相关联的语义动作组成。文法产生式用于构造语法分析器，而语义动作用于语法分析之后的语义翻译。

我们通过分号来分割文法中的不同的非终结符（如图 4-73 中的 `line`、`expr`、`term` 和 `factor`）对应的产生式及其语义动作。如果一个非终结符它有多个候选式的话，那么候选式之间通过竖线来进行分割。前面写作

$$\langle \text{非终结符} \rangle \rightarrow \langle \text{候选式} \rangle_1 | \langle \text{候选式} \rangle_2 | \dots | \langle \text{候选式} \rangle_n$$

的一组产生式在 Yacc 中被写成：

```

<非终结符>: <候选式>_1 { <语义动作>_1 }
           | <候选式>_2 { <语义动作>_2 }
           ...
           | <候选式>_n { <语义动作>_n }
           ;

```

在一个 Yacc 产生式中，如果一个由字母和数字组成的字符串没有加引号且未被声明为词法单元，它就会被当作非终结符号处理（如图 4-73 中的 `line`、`expr`、`term` 和 `factor`）。带引号的单个字符，比如 `'c'`，会被当作终结符号 `c` 以及它所代表的词法单元所对应的整数编码（即 Lex 将把 `c` 的字符编码当作整数返回给语法分析器）。第一个产生式的左部非终结符被看作开始符号。

一个 Yacc 语义动作是一个 C 语句的序列。由于 Yacc 后续可以做语义翻译，所以可以为产生式中的文法符号设置语义属性。 $$$$ 表示的是产生式左部文法符号对应的语义属性， $\$i$ 表示所在产生式右部第 i 个文法符号对应的属性。当我们按照一个产生式进行归约时就会执行和该产生式相关联的语义动作，因此语义动作通常根据 $\$i$ 的值来计算 $$$$ 的值。在图 4-73 中的语义动作

```
term : term '*' factor{ $$=$1* $3;}
      | factor
      ;
```

中，花括号里面给出的 C 语言语句用来描述产生式中的这些文法符号的属性值是如何计算出来的。其中 $$$$ 表示的是产生式左部文法符号 `term` 对应的语义属性，由于该产生式右部第 1 个文法符号是 `term`，第 2 个是 `*`，第 3 个是 `factor`，所以 $\$1$ 是产生式右部第 1 个文法符号 `term` 的语义属性， $\$3$ 是 `factor` 的语义属性。该语义动作将产生式体中的 `term` 和 `factor` 的值相加，并把结果赋给产生式左部非终结符 `term`。我们省略了第二个产生式的语义动作，因为对于体中只包含一个文法符号的产生式，默认的语义动作就是复制属性值。总的来说，默认动作是 $\{ $$ = \$1; \}$ 。

请注意，我们向这个 Yacc 程序中加入了一个新的开始符号产生式

```
line: expr '\n' {printf( " %d\n " , $1);}
```

这个产生式说明桌面计算器的输入是一个跟着换行符的表达式。和这个产生式相关的语义动作打印出了输入表达式的十进制取值和一个换行符。

辅助性 C 语言例程部分

这一部分必须提供一个名为 `yylex()` 的词法分析器。用 Lex 来生成 `yylex()` 是一个常用的选择。在需要时可以添加错误恢复例程这样的过程。

词法分析器 `yylex()` 返回一个由词法单元名（即种别码）和相关属性值组成的词法单元。如果要返回一个词法单元名字，比如 `DIGIT`，那么这个名字必须先要在 Yacc 源程序的第一部分进行声明。一个词法单元的相关属性值通过一个 Yacc 定义的变量 `yylval` 传送给语法分析器。

图 4-73 中的词法分析器 `yylex()` 是非常原始的。其中 `c` 是一个整型变量，调用 `getchar()` 函数读入一个字符并赋给 `c`。接下来调用 `isdigit()` 函数判断该字符是不是一个数字。如果是的话返回种别码 `DIGIT`。因为常数是多词一码，所以要用属性值字段来区分不同的常数，因此将该数字的值（即其属性值）存放在变量 `yylval` 中。如果 `c` 所存放的字符不是数字的话，则字符本身被当作词法单元名返回，也就是把该字符本身的 ASCII 作为该词法单元的种别码返回。

4.6 本章小结

本章系统阐述了语法分析的核心理论与方法。首先，介绍了自顶向下分析的基本思想，重点讲解了通过消除左递归和提取左公共因子对文法进行改造，并深入探讨了 LL(1)文法及其递归与非递归的预测分析法。其次，阐释了自底向上的移入-归约框架，进而详细论述了功能强大的 LR 分析家族，包括 LR(0)、SLR(1)、LR(1)和 LALR(1)分析法，分析了其能力与构建过程，并探讨了二义性文法的处理及错误恢复策略。最后，对语法分析器自动生成工具进行了简要介绍。自顶向下分析与自底向上分析之间的对比如图 4-73 所示。本章内容为理解与构建高效的语法分析器奠定了坚实基础。

对比项	自顶向下的分析	自底向上的分析
通用框架	递归下降分析	移入-归约分析
主要问题	候选式冲突	归约-归约冲突 移入-归约冲突
关键问题	如何正确选择候选式	如何正确识别句柄
确定性分析技术	LL(1)分析	LR 分析

图4-73 自顶向下分析与自底向上分析之间的对比

第5章 语法制导翻译

在完成了语法分析——即识别出程序语句的层次化结构之后，编译器的下一个关键任务，是理解这些结构所蕴含的语义。词法分析与语法分析确保了程序“形式”的正确，而语义分析则旨在揭示程序“含义”的合法与逻辑。由于语义分析的结果通常直接表示成中间代码的形式，因此，语义分析与中间代码生成两个阶段通常放在一起实现，称为是语义翻译。

我们在绪论中曾经提到，语法分析（识别句子成分）是语义分析的基础，从而也是语义翻译的基础。因此，通常使用上下文无关文法来引导对语言的翻译，也就是将语法分析和语义翻译结合起来，这一技术称为语法制导翻译，是一种面向文法的翻译技术。

在语义翻译中需要解决两个核心问题：

第一，如何表示语义信息？既然要把语法分析和语义翻译结合起来，那么我们可以将语义信息同语法信息关联起来。所谓语义，简而言之就是句子的含义。而句子的含义通常是由构成句子的各个成分的含义组合起来的。那么如何表示各个成分的语义？实际上，每个成分通常是用一个文法符号表示。因此，可以为 CFG 中的文法符号设置语义属性，用来表示语法成分对应的语义。例如，在日常生活中，当我们在学校里提到一个学生，那么跟他相关的一些信息除了他的姓名之外，还包括他的性别、年龄、寝室等。类似的，对于程序中的一个变量，我们会关心它的类型（类似于学生的性别）、值（类似于学生的年龄）、存储位置（类似于学生的寝室）等信息。我们分析一个成分的含义，无非就是要得到类似这样的一些信息。对于一个文法符号所对应的成分，我们可以把与之相关的信息都设置成该文法符号的语义属性。也就是说，我们关心什么信息，就把什么信息设置成语义属性。文法符号属于语法信息，语义属性属于语义信息。这样就将语义信息同语法信息联系起来。

第二，如何计算语义信息（语义属性）？可以通过设置语义规则的方法。事实上，构成句子的各个成分之间不是相互独立的。一个成分的语义受它周边成分的影响。比如说“我打你”和“你打我”这两个句子里都有“我”，假设我现在关心的属性就是“我”是打人者还是被打者。单凭“我”本身无法推断，要根据“我”前后的成分来分析。“我”是打人的还是挨打的取决于“打”出现在“我”的前面还是“我”的后面。既然一个成分的语义属性值与它所处的上下文（兄弟、甚至父亲或儿子节点）有关，而成分所处的上下文可以通过产生式

（形如 $A \rightarrow X_1 X_2 \dots X_n$ ）来描述，因此我们可以为产生式关联语义规则，用来表明产生式中的文法符号的属性值是如何通过其周边的成分计算的。这样，我们就将语义规则同语法规则（也就是产生式）关联起来了。

综合以上思想，我们引出一个形式化的基础概念——语法制导定义（syntax-directed definitions, SDD）。本章将系统阐述 SDD 及其实现机制语法制导翻译方案，探讨如何利用它们进行类型检查、中间代码生成等关键翻译任务，从而架起从程序语法结构到其实际含义与可执行代码的坚实桥梁。

5.1 语法制导定义（SDD）

语法制导定义（SDD）是对 CFG 的扩展（如图 5-1 所示）。给 CFG 增加了语义属性和语义规则就构成了 SDD。具体来说，一方面，为文法符号设置语义属性，从而将语义信息同语法信息联系起来；另一方面，为产生式关联语义规则，从而将语义规则同语法规则关联起来。这里的“定义”既包括定义了语义属性，也包括定义了语义规则。



图 5-1 SDD 是对 CFG 的扩展

如果 X 是一个文法符号， a 是 X 的一个属性，则用 $X.a$ 表示属性 a 在某个标号为 X 的分析树结点上的值。

例 5.1 图 5-1 所示的 SDD 其基础文法是一个用来描述声明语句的 CFG。文法符号 T 用来生成一个类型关键字，它有一个属性 $type$ ，当产生式右部是 **int** 时， T 的 $type$ 值为 **integer**；当产生式右部是 **real** 时， T 的 $type$ 值为 **real**。文法符号 L 用来生成一个标识符序列，它有一个属性 inh 。与产生式 “ $D \rightarrow T L$ ” 关联的语义规则指示属性 $L.inh$ 是根据属性 $T.type$ 计算得到的。与产生式 “ $L \rightarrow L_1, id$ ” 关联的语义规则指示属性 $L.inh$ 是根据属性 $L_1.inh$ 计算得到的。注意： L_1 跟 L 是同一个文法符号，只不过为了便于讨论，用下角标来区分 L 在产生式不同位置的出现。

产生式	语义规则
$D \rightarrow T L$	$L.inh = T.type$

$T \rightarrow \text{int}$	$T.type = integer$
$T \rightarrow \text{real}$	$T.type = real$
$L \rightarrow L_1, \text{id}$	$L_1.inh = L.inh$
...	...

图 5-1 简单类型声明的 SDD

□

语义属性决定了我们需要分析哪方面信息，语义规则决定了如何计算这些语义属性的值。把这些语义属性的值计算出来（或者说分析出来）了，语义分析的任务就完成了。

属性可以按照如下方式求值。对于给定的输入串 x ，构建 x 的语法分析树，并应用语义规则计算分析树中各结点对应的属性值。语义属性分为两种：一种是综合属性（synthesized attribute），一种是继承属性（inherited attribute）。

5.1.1 非终结符的语义属性

非终结符既可以有综合属性，也可以有继承属性。

在分析树结点 N 上的非终结符 A 的综合属性只能通过 N 的子结点或 N 本身的属性值来定义。

在分析树结点 N 上的非终结符 A 的继承属性只能通过 N 的父结点、 N 的兄弟结点或 N 本身的属性值来定义。

下面先来看一个带有综合属性的 SDD 的例子。

例 5.2 图 5-2 所示的 SDD 用于对一个以 n 作为结尾标记的表达式求值。**digit** 是一个终结符，它具有综合属性 $lexval$ ，其值是由词法分析器提供的词法值。文法中的非终结符 E 表示表达式（expression）， T 表示项（term）， F 表示因子（factor）。为它们都设置了 val 属性，分别表示表达式、项和因子的值。由产生式(2)和(3)的语义规则可以看出，非终结符 E 的 val 属性值是由产生式右部符号（即 E 的子节点）的属性值定义的，因此 val 是 E 的综合属性。同理 T 和 F 的 val 属性也都是综合属性。

产生式	语义规则
(1) $L \rightarrow E n$	$print(E.val)$
(2) $E \rightarrow E_1 + T$	$E.val = E_1.val + T.val$
(3) $E \rightarrow T$	$E.val = T.val$
(4) $T \rightarrow T_1 * F$	$T.val = T_1.val \times F.val$
(5) $T \rightarrow F$	$T.val = F.val$

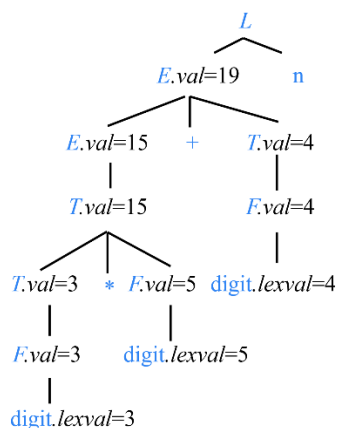
(6) $F \rightarrow (E)$	$F.val = E.val$
(7) $F \rightarrow \text{digit}$	$F.val = \text{digit.lexval}$

图 5-2 一个简单的桌面计算器的 SDD

语义规则通常被写成表达式的形式，用来计算文法符号的属性值。实际上，表达式本质上可以看作是一个计算函数。例如，图 5-2 中的产生式(2)对应的语义规则可以表示为 $E.val = ADD(E_1.val, T.val)$ 。其中函数的名字 ADD 是运算符“+”对应的名字，参数是参与运算的对象对应的属性，返回值是被计算的对象的属性。

但有时，SDD 中的某个规则的目的就是为了产生一些副作用（side effect），即完成一个特定的操作，比如打印一个计算结果或者填查符号表等。这种语义规则由于不涉及到计算，所以没有写成表达式的形式，而是一般直接被写成过程调用或程序段。例如产生式(1)的语义规则是一个过程调用 $print(E.val)$ 。过程名表示采取的动作是什么，而参数表示要对哪些属性进行操作。因为不涉及计算，因此没有显式地给产生式左部非终结符定义一个用来存放计算结果的属性。可以把过程 $print$ 看作是给产生式左部文法符号 L 定义了一个虚综合属性（dummy synthesized attribute）的规则。之所以说是虚属性，是因为函数没有返回值，相当于没有显式地定义一个要被计算的属性；之所以说是一个综合属性，是因为规则中使用了 L 的子节点 E 的属性值

有了这样一个 SDD，对于给定输入串的语法分析树，就可以利用这些与产生式相关联的语义规则来计算分析树中各节点对应的属性值。如果一棵语法分析树的各个节点上标记了相应的属性值，那么这棵语法分析树就称为注释语法分析树（annotated parse tree，简称注释分析树）。对于上述文法，假设输入符号串为：3*5+4n，图 5-3 显示了它对应的注释分析树。树上的每个节点的属性值都已经标记出来了。例如，树中三个标记为 **digit** 的叶节点对应着输入串中的三个常数（终结符），因此，它们的综合属性 $lexval$ 的值分别是它们的词法值，即 3、5 和 4。

图 5-3 $3*5+4n$ 的注释分析树

三个标记为 F 的节点表示对产生式(7)的应用。根据产生式(7)对应的语义规则， F 的 *value* 属性的值是由子节点的 **digit** 的词法值计算得到的。

节点 $*$ 的父节点表示对产生式(4)的应用。根据产生式(4)对应的语义规则， T 的 *value* 属性的值是由两个子节点 T 和 F 的 *value* 值相乘得到的（如图中指向节点 $*$ 的父节点的虚线箭头所示）。

根节点表示对产生式(1)的应用。产生式(1)对应的语义规则是一个过程调用，该过程使用了 L 的子节点 E 的属性值 *val*（如图中指向 L 节点的虚线箭头所示），因此，我们可以把该规则看作是定义 L 的一个虚综合属性的规则。

□

下面再来看一个带有继承属性的 SDD 的例子。

例 5.3 图 5-4 所示的 SDD 是将图 5-1 所示的 SDD 中省略号部分补充完整后的版本。

产生式	语义规则
(1) $D \rightarrow T L$	$L.inh = T.type$
(2) $T \rightarrow \text{int}$	$T.type = integer$
(3) $T \rightarrow \text{real}$	$T.type = real$
(4) $L \rightarrow L_1, \text{id}$	$L_1.inh = L.inh$ $addtype(\text{id.lexeme}, L.inh)$
(5) $L \rightarrow \text{id}$	$addtype(\text{id.lexeme}, L.inh)$

图 5-4 简单类型声明的完整的 SDD

非终结符 T 表示类型，它有一个属性 *type*。根据产生式(2)和(3)的语义规则可以看出， T 的 *type* 属性值是根据子节点 **int** 和 **real** 定义的，因此，*type* 是 T

的综合属性。

非终结符 L 用来生成一个标识符序列，它有一个属性 inh 。由产生式(1)和(4)的语义规则可以看出， L 的 inh 属性值都是由其兄弟或父亲节点的属性值定义的，因此， inh 是 L 的继承属性。根据产生式(1)的语义规则， $L.inh = T.type$ 。因此， $L.inh$ 表示的是标识符序列中标识符的类型。

产生式(4)和(5)的语义规则集合中都包含一个副作用 $addtype(id.lexeme, L.in)$ 。其中，标识符 id 是一个终结符，它具有综合属性 $lexeme$ ，其值是由词法分析器提供的词法值。标识符 id 的 $token$ 可以表示成： $\langle ID, lexeme \rangle$ 的形式。其中， ID 是标识符的种别码， $lexeme$ 就是这个词法值，它表示构成这个名字本身的字符串。过程 $addtype$ 的任务是：将 $L.in$ 所表示的类型值填入到符号表中标识符 $id.lexeme$ 对应的表项中。由于该过程使用了 L 的子节点 id 和 L 自身的属性值，因此可以把它看作是定义产生式左部非终结符 L 的虚综合属性的规则。

有了这样一个 SDD，对于输入句子的语法分析树，我们就可以根据与产生式关联的这些语义规则来计算语法分析树中各个节点的属性值。假设输入符号串为 “**real** a, b, c ”，图 5-5 显示了它的注释分析树。

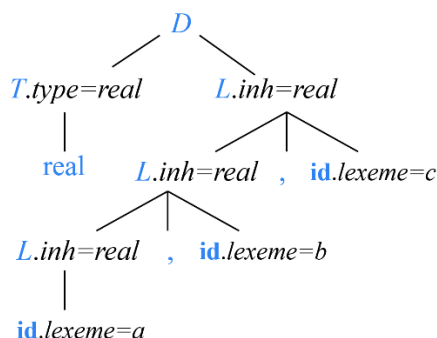


图 5-5 声明 **real** a, b, c 的注释分析树

树中三个标记为 **id** 的叶节点对应着输入中的三个单词 a 、 b 、 c （终结符）。它们的综合属性值是由词法分析器提供的词法值，分别是三个单词的字面值 a 、 b 、 c 。这里我们重点关注继承属性 $L.in$ 。

树中的根节点表示对产生式(1)的应用。根据产生式(1)的语义规则， L 的 inh 属性是由它的兄弟节点 T 的 $type$ 属性定义的。也就是说，将 T 的 $type$ 属性值 $real$ 传递给 L 的 inh 属性。

根节点的第 2 个子节点表示对产生式(4)的应用。根据产生式(4)的第 1 条语义规则，将 inh 属性的值（在该例中为 $real$ ）从父节点传递给子节点。第 2 条语义规则用来生成一个副作用，在符号表中为 **id.lexeme**（当前为 c ）创建一条

记录，并且将 c 的类型字段的值设置为由 $L.in$ 所代表的类型（即 *real*）。

从图 5-5 可以看出，产生式(4)的语义规则将 L 的 *in* 属性沿着分析树向下传递，这样由 L 生成的标识符序列中的每一个标识符都可以获得这个类型信息。

□

5.1.2 终结符的语义属性

终结符可以具有综合属性，但是不能有继承属性。终结符的综合属性值是由词法分析器提供的词法值，在 SDD 中没有计算终结符的属性值的语义规则。词法分析器的输出是一个 token 序列，每一个 token 有两部分组成，一个是种别码，一个是属性值。那么这个属性值就是词法分析器提供词法值。

事实上，在词法分析的输出结果 token 的两个分量中，种别码是语法分析关注的信息，属性值是语义分析关注的信息。

例 5.4 对于如下这样一个源程序片段

```
int a, i;
int p();
a=p[i];
```

标识符 a 和 i 被声明为两个整型变量， p 是一个过程的名字。然后接下来有一条赋值语句 $a=p[i]$ 。对于第 3 条语句

$a=p[i];$ (5.1)

其词法分析结果如图 5-6 所示。

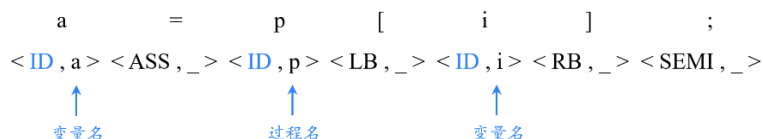


图 5-6 “ $a=p[i];$ ”的词法分析结果

接下来我们来做语法分析。语法分析过程只关心种别码。假如说文法中有如下这样一条产生式

$S \rightarrow id = id[id];$ (5.2)

那么语句(5.1)从语法分析的视角来看，它完全是一个合法的句子，因为图 5-6 的种别码序列

$\langle ID \rangle \quad \langle ASS \rangle \quad \langle ID \rangle \quad \langle LB \rangle \quad \langle ID \rangle \quad \langle RB \rangle \quad \langle SEMI \rangle$

与产生式(5.2)完全匹配。

而语义分析阶段关注的是 **token** 的第二个分量，即属性值字段。对于源程序片段中的词法单元 **a**、**p** 和 **i**，它们的种别码相同（都是标识符 **ID**）。但是，不同标识符的种别（如变量的名字、数组的名字、过程的名字等）、类型（如整型、实型、布尔型、字符型等）、长度、存放地址等信息都不同，这些信息可以通过对声明该标识符的语句进行分析得到，并且将它们存入符号表。而词法单元 **a**、**p**、**i** 各自的属性值实际上是它们各自在符号表上对应的那条记录的入口地址。有了这个入口地址，就可以访问到这个标识符的各个语义信息。这里我们用各个标识符的具体名字来表示这个地址。

对于产生式(5.2)，从语义分析的角度来看，由于 “[]” 是数组访问操作符，因此要求位于 “[]” 之前的 **id** 必须是一个数组的名字。但是在语句(5.1)中，对于 “[]” 之前的标识符 **p**，通过其 **token** 中的属性值去查符号表，发现 **p** 是一个过程的名字，所以从语义分析的视角来看，语句(5.1)是不合法的，即存在语义错误。 □

5.1.3 属性文法

例 5.2 和例 5.3 的 SDD 中都含有副作用。如果一个 SDD 中没有副作用，则这样的 SDD 称为属性文法（attribute grammars，1974 年图灵奖得主 Donald Knuth 于 1968 年提出）。也就是说属性文法的语义规则均被写成表达式的形式，而不会出现过程调用的情况。作为属性文法的一个例子，我们对例 5.2 中的 SDD 进行修改。我们为 **L** 设置一个属性 **val**，并且将产生式(1)的语义规则中的副作用改为计算属性 **L.val** 的值，这样我们就得到了一个属性文法，如图 5-7 所示。

产生式	语义规则
(1) $L \rightarrow E \mathbf{n}$	$L.val = E.val$
(2) $E \rightarrow E_1 + T$	$E.val = E_1.val + T.val$
(3) $E \rightarrow T$	$E.val = T.val$
(4) $T \rightarrow T_1 * F$	$T.val = T_1.val \times F.val$
(5) $T \rightarrow F$	$T.val = F.val$
(6) $F \rightarrow (E)$	$F.val = E.val$
(7) $F \rightarrow \mathbf{digit}$	$F.val = \mathbf{digit.lexval}$

图 5-7 一个属性文法的例子

属性文法这一名字也从侧面反映出 SDD 是对上下文无关文法 CFG 的扩展这一事实。

5.1.4 SDD 的求值顺序

SDD 为 CFG 中的文法符号设置语义属性。对于给定的输入串 x ，应用语法规则计算分析树中各节点对应的属性值。那么按照什么顺序计算属性值？

依赖图

事实上，语义规则定义了属性之间的依赖关系。在对语法分析树节点的一个属性求值之前，必须首先求出这个属性值所依赖的所有属性值。为此，我们先介绍一个称之为依赖图（dependency graph）的工具，它可以用来描述分析树中节点属性间的依赖关系，从而帮助我们确定分析树中各属性的求值顺序。

具体来说，分析树中每个标号为 X 的结点的每个属性 a 都对应着依赖图中的一个结点。如果和某产生式 p 关联的语义规则通过属性 $Y.b$ 的值定义了属性 $X.a$ 的值，则相应的依赖图中有一条从 $Y.b$ 的结点指向 $X.a$ 的结点的有向边。作为惯例，我们把分析树的边显示为实线，而把依赖图的边显示为虚线并且为了区别起见，将继承属性标记在文法符号的左侧，将综合属性标记在文法符号的右侧。

例 5.5 图 5-5 所示的注释分析树对应的依赖图如图 5-8 所示。

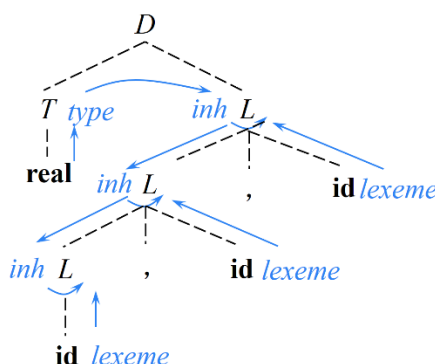


图 5-8 对应于图 5-5 中的注释分析树的依赖图

根节点表示对图 5-4 中产生式(1)的应用。根据产生式(1)的语义规则， L 的 inh 属性值依赖于 T 的 $type$ 属性值，因此图中有一条从 T 的 $type$ 属性指向 L 的 inh 属性的边。由于 $type$ 是 T 的一个综合属性，因此把它标记在 T 的右侧。而 inh 是 L 的一个继承属性，因此把它标记在 L 的左侧。

根节点的第 1 个子节点表示对产生式(2)的应用。根据产生式(2)的语义规则， T 的 $type$ 属性值依赖于终结符 **real** 的属性值，因此图中有一条从 **real** 指向 L 的 inh 属性的边。

根节点的第 2 个子节点表示对产生式(4)的应用。根据产生式(4)的第 1 条语

义规则，子节点 L 的 inh 属性值依赖于父节点 L 的 inh 属性值，因此图中有一条从父节点 L 的 inh 属性指向子节点 L 的 inh 属性的边。产生式(4)的第 2 条语义规则产生了一个副作用 $addtype(id.lexeme, L.in)$ 。它表示将 $L.in$ 所表示的类型值填入到符号表中标识符 $id.lexeme$ 对应的符号表表项中。由于该过程使用了子节点 id 的属性值和 L 节点自身的属性值，因此，可以把它们看作是定义 L 的虚综合属性的规则。我们在依赖图中为 L 设置一个虚节点，该节点上没有给出属性的名字（因为是虚属性），但是标记了从 $L.inh$ 和 $id.lexeme$ 到该节点的有向边。

其余各节点的处理方式类似，不再赘述。 □

属性的求值顺序

有了依赖图，我们就可以设计属性值的计算顺序了。一个可行的求值顺序是满足下列条件的结点序列 $N_1, N_2, \dots, N_k$ ：如果依赖图中有一条从结点 N_i 到 N_j 的边($N_i \rightarrow N_j$)，那么 $i < j$ （即：在节点序列中， N_i 排在 N_j 前面）。这样的排序将一个有向图变成了一个线性排序，这个排序称为这个图的**拓扑排序**（topological sort）。

例 5.6 我们为图 5-8 所示的依赖图中的节点进行编号，如图 5-9 所示。

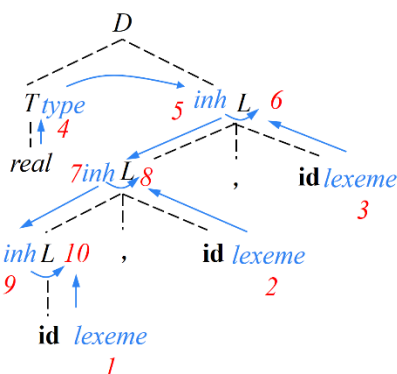


图 5-9 根据声明语句 **real a, b, c** 的依赖图得到的一个属性求值顺序

这个编号顺序事实上也正好就是这个依赖图的一个拓扑排序。也就是说，可以先计算节点 1~4 对应的属性，然后节点 5 对应的属性依赖于节点 4 的属性，节点 6 的属性依赖于节点 3 和 5 的属性，节点 7 的属性依赖于节点 5 的属性，节点 8 的属性依赖于节点 2 和 7 的属性，节点 9 的属性依赖于节点 7 的属性，节点 10 的属性依赖于节点 1 和 9 的属性。也就是说依赖者的序号都是大于被依赖者的序号。在计算依赖者的属性的时候，被依赖者的属性值都已经计算出来了。

当然还存在着其它的拓扑排序，比如说 1~4 这四个节点它们之间是互不依

赖的，因此它们四个的顺序是可以任意调换的。另外节点 6 和 7 都依赖于节点 5，但是节点 6 和 7 之间是互不依赖的，所以它们之间的顺序也是可调的。节点 8 和 9 也是类似的情况。例如，对于图 5-9 来说，4、3、2、1、5、7、6、9、8、10 也是一种可行的拓扑排序。□

是不是所有的依赖图都可以给出拓扑排序？答案是否定的。

根据 SDD 中是否含继承属性，可以将 SDD 分为两类，一类是 SDD 中不含继承属性，言外之意就是所有的属性都是综合属性，另外一类就是包含继承属性。

对于第一类 SDD，由于它只包含综合属性，而综合属性的值只依赖于子节点的属性值，可以按照任何自底向上的顺序计算它们的值，也就是先计算子节点的属性值，再计算父节点的属性值。

对于第二类 SDD，也就是说 SDD 中含有继承属性，则不能保证存在一个顺序来对各个节点上的属性进行求值。考虑图 5-10 所示的这种情况。

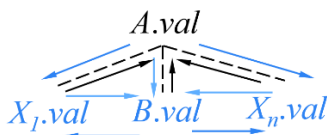


图 5-10 循环依赖示意图

其中， A 是父节点， B 以及 X_1 、 X_2 、 $\dots$ 、 X_n 是子节点。假设父节点 A 有一个综合属性 val ，它的值会依赖于子节点 B 或 X_1 、 X_2 、 $\dots$ 、 X_n 的属性值（如图中黑色箭头所示）。对于子节点 B ，假设它有一个继承属性 val ，也就是说该属性在计算过程中可能会用到它父节点 A 的属性值 val ，也会用到它前后兄弟节点 X_1 、 X_2 、 $\dots$ 、 X_n 的属性值（如图中蓝色虚线箭头所示）。类似的， X_1 、 X_2 、 $\dots$ 、 X_n 这些文法符号，它们也含有继承属性 val ，也就是说计算过程中会用到父节点的属性值，或者兄弟节点的属性值，比如说 $X_1.val$ 或 $X_n.val$ 依赖于 $B.val$ 。这样，从竖直方向（纵向）来看，父节点 A 的综合属性 $A.val$ 可能会依赖于子节点 B 的继承属性 $B.val$ ，而子节点 B 的继承属性 $B.val$ 反过来又依赖于父节点 A 的综合属性 $A.val$ 。这样在竖直方向就会造成循环依赖，即在依赖图中形成了环。这个时候就无法确定先计算谁。到底是先计算父节点的综合属性，还是先计算子节点的继承属性。

类似的，从水平方向（横向）来看， X_1 、 X_2 、 $\dots$ 、 X_n 这些子节点的继承属性值之间也会存在循环依赖。因为一个文法符号的继承属性可以依赖于它的兄弟节点的属性值，也就是说哥哥的继承属性可以依赖于弟弟的继承属性（比如

说 $X_1.val$ 依赖于 $B.val$ ），弟弟的继承属性也可以依赖于哥哥的继承属性（比如说 $B.val$ 依赖于 $X_1.val$ ）。这样在依赖图的水平方向上也会形成环。

所以当 SDD 中包含继承属性的时候，会使得问题变得比较复杂，无法保证有一个求值顺序。只有当依赖图中不存在环的时候，才能保证至少存在一个拓扑排序。

从计算的角度看，给定一个 SDD，很难确定是否存在某棵语法分析树，使得 SDD 的属性之间存在循环依赖关系。幸运的是，存在一个 SDD 的有用子类，它们能够保证对每棵语法分析树都存在一个求值顺序，因为它们不允许产生带有环的依赖图。不仅如此，接下来介绍的两类 SDD 可以和自顶向下及自底向上的语法分析过程一起高效地实现。

5.2 S-属性定义与 L-属性定义

5.2.1 S-属性定义

仅仅使用综合属性的 SDD 称为 S 属性的 SDD，或 S -属性定义（ S -attributed definitions, S -SDD）。

例 5.7 图 5-2 和图 5-7 中的 SDD 都是 S -SDD 的例子。其中的每个属性（ $L.val$ 、 $L.val$ 、 $L.val$ 、和 $L.val$ ）都是综合属性。□

我们在 5.1.5 节讨论过，对于只具有综合属性的 SDD（即 S -SDD）可以按照语法分析树节点的任何自底向上顺序来计算它的各个属性值。即 S -SDD 可以在自底向上的语法分析过程中实现。

5.2.2 L-属性定义

我们在 5.1.1 节中提到，当 SDD 中都包含继承属性的时候，会使问题变得比较复杂，无法保证有一个求值顺序。原因是继承属性的存在使得依赖图中可能形成环。那么满足什么样的条件可以确保依赖图中不会产生环呢？

对于产生式 $A \rightarrow X_1 X_2 \dots X_n$ ，根据继承属性的定义，其右部符号 X_i ($1 \leq i \leq n$) 的继承属性既可以依赖于父节点的属性，又可以依赖于兄弟节点的属性。

首先，从依赖于父节点的属性这一方面来看，正如 5.1.1 节中分析的那样，如果子节点 X_i 的继承属性依赖于父节点 A 的综合属性就可能导致在依赖图的纵向上形成环（因为 A 的综合属性可以依赖于子节点的属性，当然也包括依赖于子节点 X_i 的这个继承属性）。所以为了避免依赖图在纵向上出现环，我们做出

如下限制：

非终结符 X_i 的继承属性只能依赖父节点 A 的继承属性。 (5.3)

另一方面，从依赖于兄弟节点的属性这一方面来看，有可能在依赖图的水平方向上形成环。为了避免依赖图在水平方向上出现环，我们做出如下限制：

一个非终结符 X_i ($1 \leq i \leq n$) 的继承属性只能依赖于产生式中 X_i 左边的符号 $X_1, X_2, \dots, X_{i-1}$ 的属性。也就是说， X_i 只能依赖于左兄弟节点的属性，不能依赖于右兄弟节点的属性，这样使得依赖图中水平方向的箭头只能从左侧指向右侧，从而使得水平方向不会形成环。

由此，我们给出 L-属性定义 (L-attributed definitions, L-SDD) 的概念。

一个 SDD 是 L-属性定义，当且仅当它的每个属性要么是一个综合属性，要么是满足如下条件的继承属性：假设存在一个产生式 $A \rightarrow X_1 X_2 \dots X_n$ ，其右部符号 X_i ($1 \leq i \leq n$) 的继承属性仅依赖于下列属性：

- (1) A 的继承属性。
- (2) 产生式中 X_i 左边的符号 $X_1, X_2, \dots, X_{i-1}$ 的属性。
- (3) X_i 本身的属性，但 X_i 的全部属性不能在依赖图中形成环路。

它的直观含义是：在一个产生式所关联的各属性之间，依赖图的边可以从左到右，但不能从右到左。这也是 L 属性定义名字中 “L” 的由来，也就是说依赖图水平方向所有的箭头都是从左侧 (left) 指向右侧。

有了以上条件的约束，可以保证 L-SDD 无论是纵向上还是水平方向上，依赖图中都不会出现环，从而保证一定会构造出一个属性求值顺序。

每个 S-SDD 都是 L-SDD，因为 S-SDD 中只有综合属性，而 L-SDD 对综合属性没有任何限制。S-SDD 中没有继承属性，因此不会违反 L-SDD 对继承属性的限制条件。

例 5.8 对于图 5-11 所示的 SDD，判定它是否为 L-SDD。

产生式	语义规则
(1) $T \rightarrow F T'$	$T'.inh = F.val$ $T.val = T'.syn$
(2) $T' \rightarrow * F T_1'$	$T_1'.inh = T'.inh \times F.val$ $T'.syn = T_1'.syn$
(3) $T' \rightarrow \epsilon$	$T'.syn = T'.inh$
(4) $F \rightarrow \text{digit}$	$F.val = \text{digit.lexval}$

图 5-11 一个 L-SDD 的例子

首先我们需要判别一个 SDD 中设置的这些属性分别是文法符号的综合属性还是继承属性，然后再根据 L-SDD 的定义来判定它是否为 L-SDD。

那么，给定一个 SDD，如何判别其中的某个属性是文法符号的综合属性还

是继承属性呢？

对于图 5-11 中定义的以下三个属性 $T.val$, $T'.syn$ 、 $F.val$ （图中蓝色字体所示），它们的一个共同特点就是它们都是所在产生式左部文法符号的属性，因此它们都是综合属性。否则，如果它们是继承属性，由于继承属性计算过程中要用到兄弟节点或者父节点的属性值，但是由于 T 、 T' 、 F 都是所在产生式左部的文法符号，因此从产生式中看不到它们的兄弟节点和父节点，而只能看到它们的子节点。因此，产生式左部这些文法符号的属性肯定是综合属性。从图 5-11 中的语义规则可以看出， $T.val$, $T'.syn$ 、 $F.val$ 确实都依赖于子节点的属性值。

而属性 $T'.inh$, $T_l'.inh$ （图中红色字体所示），它们的一个共同特点就是它们都是所在产生式右部文法符号的属性，因此它们都是继承属性。否则，如果它们是综合属性，由于综合属性计算过程中要用到子节点的属性值，但是由于 T' 和 T_l' 都是所在产生式右部的文法符号，因此从产生式中看不到它们的子节点，而只能看到它们的父节点和兄弟节点。因此，产生式右部这些文法符号的属性肯定是继承属性。从图 5-11 中的语义规则可以看出， $T'.inh$, $T_l'.inh$ 确实都依赖于父节点或兄弟节点的属性值。

综上，根据文法符号在产生式中所处的位置就能判断出其属性是继承属性还是综合属性。

接下来我们来判定它是否为 L-SDD。L-SDD 对于综合属性没有限制，它只是限制了继承属性。因此这个 SDD 是不是 L-SDD 关键取决于这两个继承属性它们所依赖的属性值。

从规则(1)关联的语义规则可以看出，继承属性 $T'.inh$ 依赖于它左边兄弟的属性值，因此它不违反 L-SDD 对于继承属性的限制。从规则(2) 关联的语义规则可以看出，继承属性 $T_l'.inh$ 依赖于它父亲的继承属性 $T'.inh$ 以及它左边兄弟的属性值 $F.val$ ，也不违反 L-SDD 中对继承属性的限制。因此这个 SDD 是一个 L-SDD。□

我们再来看一个非 L-SDD 的例子。

例 5.9 对于图 5-12 所示的 SDD，根据产生式(1)的第 1 条语义规则， L 的继承属性 i 依赖于其父节点的继承属性；根据第 2 条语义规则， M 的继承属性 i 依赖于其左兄弟节点的属性 $L.s$ ；根据第 3 条语义规则， A 的综合属性 s 依赖于它子节点 M 的属性 s ，这些都不违反 L-SDD 的要求。

根据产生式(2)的第 1 条语义规则， R 的继承属性 i 依赖于其父节点 A 的继承属性 i ；根据第 3 条语义规则， A 的综合属性 s 依赖于其子节点 Q 的属性 s ，这些都不违反 L-SDD 的要求。但是根据第 2 条语义规则， Q 的继承属性 i 依赖

于其右兄弟节点的属性 $R.s$ ，这违反了 L-SDD 的限制。因此这个 SDD 不是一个 L-SDD。 □

产生式	语义规则
(1) $A \rightarrow LM$	$L.i = l(A.i)$ $M.i = m(L.s)$ $A.s = f(M.s)$
(2) $A \rightarrow QR$	$R.i = r(A.i)$ $Q.i = q(R.s)$ $A.s = f(Q.s)$

图 5-12 一个非 L-SDD 的例子

L-SDD 设计思路举例

接下来我们分析一下图 5-11 所示的这个 L-SDD 的设计思路，也就是为什么要设置这些属性、定义这些规则？设计的依据是什么？

为方便讨论，我们将这个 L-SDD 重新写在下面。

产生式	语义规则
(1) $T \rightarrow F T'$	$T'.inh = F.val$ $T.val = T'.syn$
(2) $T' \rightarrow * F T'_1$	$T'_1.inh = T'.inh \times F.val$ $T'.syn = T'_1.syn$
(3) $T' \rightarrow \varepsilon$	$T'.syn = T'.inh$
(4) $F \rightarrow \text{digit}$	$F.val = \text{digit.lexval}$

我们先来看这个 L-SDD 的基础文法，它其实是一个改造后的文法，原始文法为

$$\begin{aligned} T &\rightarrow T * F \mid F \\ F &\rightarrow \text{digit} \end{aligned} \quad (5.5)$$

用来生成“项（term）”。项是由若干个因子（factor）相乘得到的。根据产生式(1)， T 由一个 F 连接上一个 T' 构成。 F 表示因子。根据产生式(2)， T' 用来生成由“ $*$ ”构成的序列。

我们消除了原始文法中的左递归，显然是要做自顶向下的分析。设计 SDD 的目的是为了完成语义翻译的任务。对于“项”这一语言，语义翻译的主要任务之一是计算 T （也就是项）的值。 T 的值是由构成 T 的各个 F 的值相乘得到的。因此，首先为 T 和 F 分别设置一个综合属性 val ，用来存放项和因子的值。 F 的 val 值就等于它的子节点 **digit** 的 val 值。而我们最终要得到项 T 的值 $T.val$ ，又该如何计算？

在自顶向下的分析过程中，首先应用产生式(1) $T \rightarrow F T'$ 生成第一个因子 F ，

如图 5-13(a)所示。接下来再应用产生式(2) $T' \rightarrow *F T_1'$ 若干次生成若干个 $*F$ ，如图 5-13(b)所示。每次应用产生式(2)时，都应该把新生成的 F 与已经生成的各个 F 的值累积到一起，因此这里为 T' 设置一个继承属性 inh ，用来存放这个累积的值，如图 5-13(c)所示。将父节点 T' 的累积值乘上新生成的 F 值赋给子节点 T_1' 的 inh 属性。也就是说子节点 T_1' 的累积值等于在父节点 T' 的累积值的基础上乘上新生成的 F 值。如果把累积值看作是财富的话，相当于子节点“继承”了父节点的财富（子承父业）。而在使用产生式(1) $T \rightarrow F T'$ 的时候，生成了整个项 T 的第 1 个因子 F ，因此，该产生式的 $T'.inh$ 值（用于存放已生成的 F 的乘积）等于已经生成的这第 1 个因子 F 的 val 值。

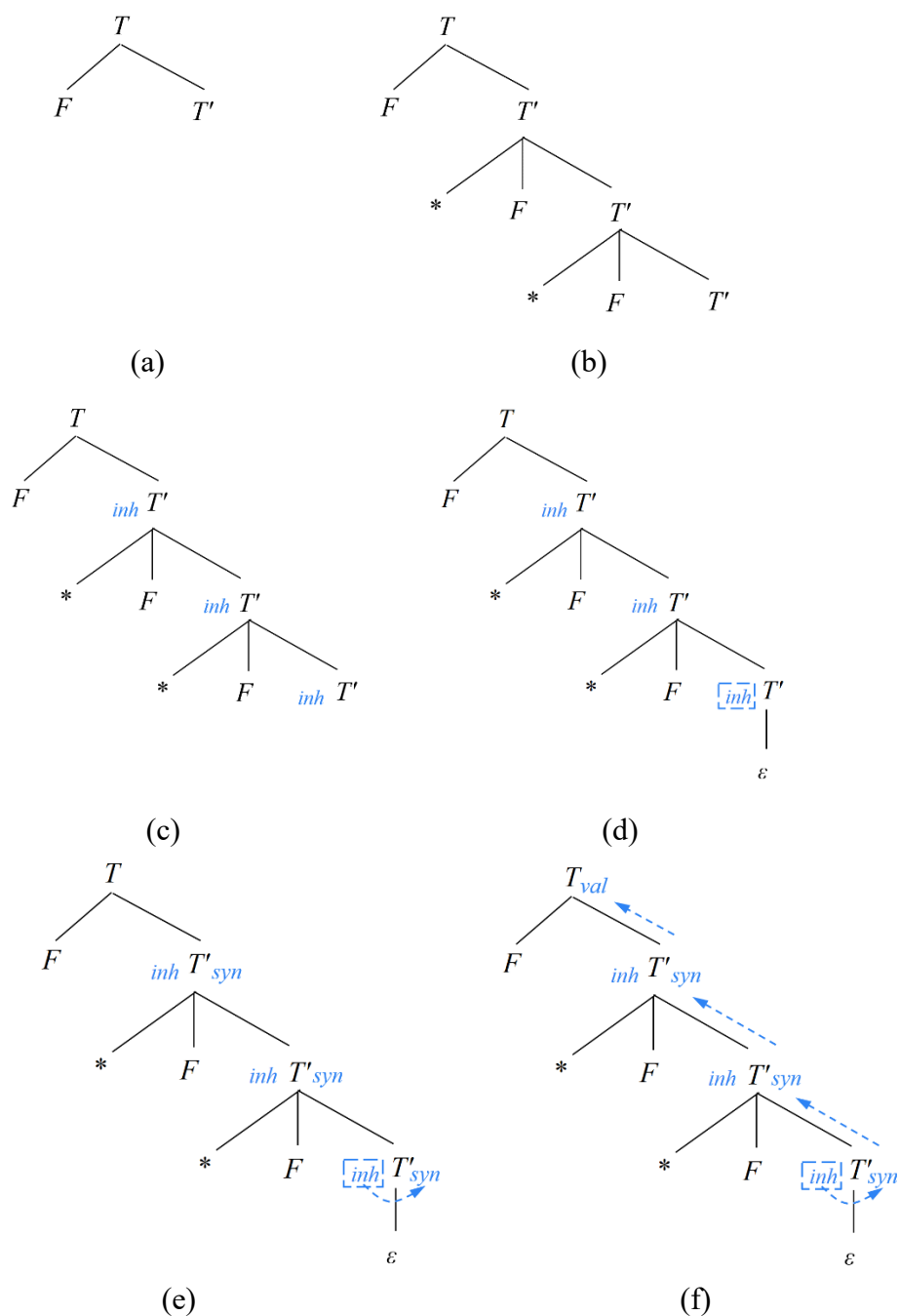


图 5-13 例 5.8 中 L-SDD 的设计思路示意

当所有因子都处理完毕后（也就是应用产生式(3) $T' \rightarrow \varepsilon$ 结束递归生成时），位于分析树的最底部的 $T'.inh$ 中存放了整个 T 中全部因子的累计值，如图 5-13(d)所示。我们要把这个最终的累计值 $T'.inh$ 赋给根节点 T 的 val 属性（表示

整个项的值)，即传递到根节点。要想将值自下而上传递，需要使用综合属性，因此为 T' 设置综合属性 syn 。在产生式(3) $T' \rightarrow \varepsilon$ 对应的规则里，执行的操作就是将这个累计值 $T'.inh$ 复制给综合属性 $T'.syn$ ，如图 5-13(e)所示。

在产生式(2)对应的规则里，通过规则 $T'.syn = T_1'.syn$ 将累积结果从子节点 T_1' 传给父节点 T' 。这样，通过综合属性将累积结果向上传递到最顶层的 T' ，并最终在产生式(1)对应的规则里将这个累积值复制给根节点 T 的综合属性 val ，如图 5-13(f)所示。

总结一下，就是为 T' 设置继承属性 inh ，用来存放已生成的各个因子的累积值。每次应用产生式(2)时，都将父节点 T' 的累计值乘上新生成的 F 值赋给儿子 T_1' 的 inh 属性，这样，这个累计值就从上向下传递到树的底部。最后通过设置综合属性，将其传回树的根节点，赋给 T 的 val 属性。

5.3 语法制导翻译方案 SDT

SDD 为每个产生式关联语义规则，用以说明产生式中的文法符号的属性值是如何通过计算的。由于属性之间存在依赖关系，就存在着先算谁后算谁的问题。S-SDD 和 L-SDD 能够保证肯定存在一个求值顺序，那么如何表示这个求值顺序呢？换句话说，各个属性具体在什么时刻计算？这就是本节要介绍的语法制导翻译方案（syntax-directed translation scheme, SDT），即在产生式右部中嵌入了程序片段（称为语义动作）的上下文无关文法。

例 5.10 图 5-14(a)是图 5-11 所示的 SDD，(b)是与之对应的 SDT。

产生式	语义规则
(1) $T \rightarrow F T'$	$T'.inh = F.val$ $T.val = T'.syn$
(2) $T' \rightarrow * F T_1'$	$T_1'.inh = T'.inh \times F.val$ $T'.syn = T_1'.syn$
(3) $T' \rightarrow \varepsilon$	$T'.syn = T'.inh$
(4) $F \rightarrow \text{digit}$	$F.val = \text{digit.lexval}$

(a)

(1) $T \rightarrow F \{ T'.inh = F.val \} T' \{ T.val = T'.syn \}$
(2) $T' \rightarrow * F \{ T_1'.inh = T'.inh \times F.val \} T_1' \{ T'.syn = T_1'.syn \}$
(3) $T' \rightarrow \varepsilon \{ T'.syn = T'.inh \}$
(4) $F \rightarrow \text{digit} \{ F.val = \text{digit.lexval} \}$

(b)

图 5-14 图 5-11 所示的 SDD 对应的 SDT

其中，花括号中括起的是一些语义动作，用来计算文法符号的属性值。一个语

义动作在产生式中的位置决定了相应属性值的计算时机，从而定义了属性值的计算顺序。例如，根据(b)中的产生式(1)，当分析出 F 以后，就可以计算 T 的 inh 属性值了。□

SDT 可以看作是 SDD 的具体实施方案。SDD 定义了各个属性的计算方法，也就是说每一个属性怎么计算。而 SDT 进一步明确了各个属性何时计算，也就是计算时机。SDD 定义了计算规则，SDT 在此基础上还定义了计算顺序。

我们在 5.1.4 节提到，任何一个无循环依赖的 SDD 都能够确保存在一个属性的求值顺序，因此，无循环依赖的 SDD 都可以给出其具体实施方案 SDT。这里面我们重点关注如何使用 SDT 实现两类重要的 SDD:

(1) 基础文法 (underlying grammar) 可以使用 LR 分析技术，且 SDD 是 S 属性的。

(2) 基础文法可以使用 LL 分析技术，且 SDD 是 L 属性的。
因为在这两种情况下，不仅可以确保 SDD 存在一个计算顺序，而且其对应的 SDT 可在语法分析过程中实现，也就是说可以在语法分析的同时把所有的语义属性计算出来。

5.3.1 S-属性定义的 SDT

在一个 S-SDD 中，所有的属性都是综合属性，而综合属性的值通常依赖于子节点的属性值，因此，需要等所有子节点都分析完毕才可以计算父节点的综合属性。因此，将一个 S-SDD 转换为 SDT 时，只需将每个语义动作都放在产生式的末尾。所有动作都在产生式末尾的 SDT 称为后缀翻译方案 (postfix SDT)。

例 5.11 图 5-15(a)是一个与图 5-2 类似的 S-SDD (只是将产生式 $L \rightarrow E n$ 替换成了 $E' \rightarrow E$ ，本质上相当于将产生式左部文法符号 L 替换为 E')。只需将其中的每一条语义规则直接连接到对应产生式的尾部，即可得到对应的后缀 SDT，如图 (b) 所示。□

产生式	语义规则
(1) $E' \rightarrow E$	$\text{print}(E.val)$
(2) $E \rightarrow E_1 + T$	$E.val = E_1.val + T.val$
(3) $E \rightarrow T$	$E.val = T.val$
(4) $T \rightarrow T_1 * F$	$T.val = T_1.val \times F.val$
(5) $T \rightarrow F$	$T.val = F.val$
(6) $F \rightarrow (E)$	$F.val = E.val$

(7) $F \rightarrow \text{digit}$	$F.val = \text{digit.lexval}$
----------------------------------	-------------------------------

(a)

(1) $E' \rightarrow E \{ \text{print}(E.val) \}$
(2) $E \rightarrow E_1 + T \{ E.val = E_1.val + T.val \}$
(3) $E \rightarrow T \{ E.val = T.val \}$
(4) $T \rightarrow T_1 * F \{ T.val = T_1.val \times F.val \}$
(5) $T \rightarrow F \{ T.val = F.val \}$
(6) $F \rightarrow (E) \{ F.val = E.val \}$
(7) $F \rightarrow \text{digit} \{ F.val = \text{digit.lexval} \}$

(b)

图 5-15 图 5-7 所示的 S-SDD 对应的后缀 SDT

5.3.2 S-属性的定义的自底向上翻译

如果一个 S-SDD 的基础文法可以使用 LR 分析技术，那么它的 SDT 可以在 LR 语法分析过程中实现，也就是说，当归约发生时执行相应的语义动作。

要想利用语法分析器实现语义翻译，需要扩展语法分析器，使得在语法分析的同时能够进行语义翻译。语义分析照语法分析增加了语义属性信息，各个文法符号的属性值可以放到栈中的某个位置，使得执行归约时可以找到它们。最好的方法是将属性和文法符号（或者表示文法符号的 LR 状态）一起放在栈中的记录里。为此，我们对语法分析栈进行扩充。假设语法分析栈存放在一个被称为 *stack* 的记录数组中，*top* 是指向栈顶的游标，如图 5-16 所示。这样，*stack[top]* 指向这个栈的栈顶记录（即 *Z* 对应的记录），*stack[top-1]* 指向栈顶记录的下一个记录（即 *Y* 对应的记录），以此类推。假设每个记录有一个被称为 *symbol* 的字段，用来存放该记录所代表的文法符号（如图中的 *X*、*Y* 和 *Z*）。*state* 表示与 *symbol* 符号栈平行的状态栈，*s_X*、*s_Y* 和 *s_Z* 分别表示文法符号 *X*、*Y* 和 *Z* 对应的 LR 状态。在此基础上，为每个栈记录增加属性值字段 *value*，存放文法符号的综合属性值（如图中的 *x*、*y* 和 *z* 分别是 *X*、*Y* 和 *Z* 的综合属性值）。图 5-16 中蓝色区域表示为进行语义翻译而扩展的部分。

					<i>top</i> ↓
<i>value</i>		...	<i>X.x</i>	<i>Y.y</i>	<i>Z.z</i>
<i>symbol</i>	\$	...	<i>X</i>	<i>Y</i>	<i>Z</i>
<i>state</i>	<i>s</i> ₀	...	<i>s</i> _{<i>X</i>}	<i>s</i> _{<i>Y</i>}	<i>s</i> _{<i>Z</i>}

图 5-16 带有助于存放综合属性的字段的语法分析栈

分析器在每次归约时执行计算综合属性值的语义动作。例如，对于如下的产生式及其对应的语义动作

$$A \rightarrow XYZ \{ A.a = f(X.x, Y.y, Z.z) \} \quad (5.6)$$

当将栈顶的 X 、 Y 和 Z 归约为 A 时执行语义动作 $A.a = f(X.x, Y.y, Z.z)$ 计算综合属性 $A.a$ 的值。接下来，栈顶的 X 、 Y 和 Z 对应的记录出栈， A 的记录进栈，将计算出来的 $A.a$ 的值存放在记录 A 的属性值字段 $value$ 中。

一般来说，一个文法符号可能有多个属性，为此，需要使栈的记录足够大，或者在栈记录中存放指针。对于小型的属性，将记录变得足够大是比较简单的方法，即使有些时候有些字段不会被用到也没有太大关系。如果一个或多个属性的大小没有限制（比如它们是字符串），那么最好把属性值存放在栈之外的某个比较大的共享存储区域中，并把指向属性值的指针放到栈记录中。

到目前为止，所有的语义动作中给出的都是抽象的定义式，如式(5.6)中的 $A.a = f(X.x, Y.y, Z.z)$ 。需要将它们改写成具体可执行的栈操作，这样可以由语法分析器自动完成。

例如，对于式(5.6)，图 5-16 显示的是使用产生式 $A \rightarrow XYZ$ 进行归约前栈中的格局。可见， $X.x$ 存放在 $stack[top-2].value$ 中， $Y.y$ 存放在 $stack[top-1].value$ 中， $Z.z$ 存放在 $stack[top].value$ 中。归约后， X 、 Y 和 Z 三个符号对应的记录出栈， A 对应的记录进栈。因此， A 将出现在 X 现在所处的位置，即 $stack[top-2].symbol$ 中， $A.a$ 将存放在 $stack[top-2].value$ 中。 $top-2$ 的位置变成新的栈顶。于是，我们将式(5.6)中的抽象定义式改写为如下的具体可执行的显式栈操作

$$\begin{aligned} & stack[top-2].symbol = A \\ & stack[top-2].value = f(stack[top-2].value, stack[top-1].val, stack[top].value) \\ & top = top-2; \end{aligned} \quad (5.7)$$

如图 5-17 所示。

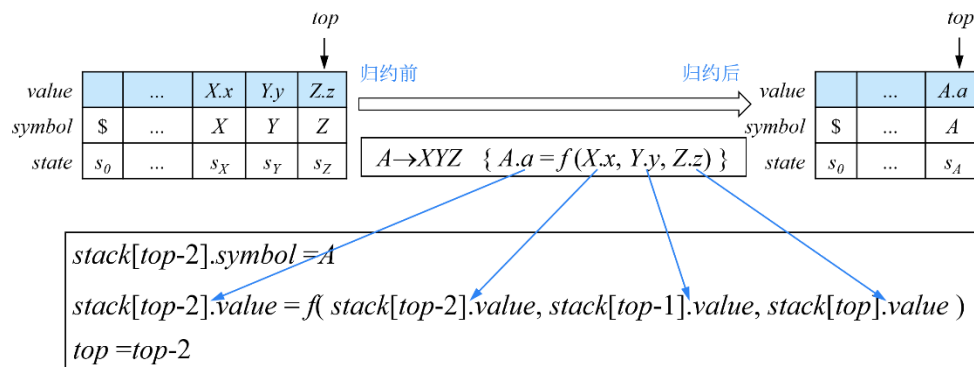


图 5-17 将语义动作中的抽象定义式改写成具体可执行的栈操作

例 5.12 我们将例 5.11 中桌面计算器 SDT 中的动作改写为具体可执行的显式栈操作，如图 5-18 所示。

产生式	语义动作
(1) $E' \rightarrow E$	{ $print(stack[top].value);$ }
(2) $E \rightarrow E_1 + T$	{ $stack[top-2].value = stack[top-2].value + stack[top].value;$ $top=top-2;$ }
(3) $E \rightarrow T$	
(4) $T \rightarrow T_1 * F$	{ $stack[top-2].value = stack[top-2].value \times stack[top].value;$ $top=top-2;$ }
(5) $T \rightarrow F$	
(6) $F \rightarrow (E)$	{ $stack[top-2].value = stack[top-1].value;$ $top=top-2;$ }
(7) $F \rightarrow \text{digit}$	

图 5-18 在一个自底向上语法分析栈中实现桌面计算器

对于产生式(2)，归约时，栈顶 $stack[top]$ 对应 T 的记录， $stack[top-1]$ 对应 “+” 的记录， $stack[top-2]$ 对应 E_1 的记录。将栈顶的 3 个对应于产生式右部 3 个文法符号的记录出栈，产生式左部文法符号 E 的记录进栈，因此， E 进栈后将出现在 $stack[top-2]$ 的位置。语义动作 $stack[top-2].value = stack[top-2].value + stack[top].value$ 实际是将 E_1 和 T 的值相加，计算结果存放在即将进栈的 E 记录的属性值字段中。语义动作 $top=top-2$ 将 top 指向归约后的新的栈顶。产生式(4)和(6)与产生式(2)的情况类似。这里我们只列出了属性值字段 $value$ 对应的语义动作，而省略了符号 $symbol$ 和状态 $state$ 字段，因为符号栈和状态栈相关内容已经在语法分析一章介绍过了。

这里我们注意到产生式(3)并未附加栈操作形式的语义动作。这是因为产生式右部只有一个文法符号。归约时，相当于子节点对应的文法符号出栈，父节点对应的文法符号进栈。根据与该产生式所关联语义动作的抽象定义式，将子节点的属性值赋给父节点，相当于栈顶记录的属性值字段没有改变，因此无需采取任何语义动作。产生式(5)和(7)也是类似的情况。

该 SDT 的基础文法 G 与例 4.30 中的文法(4.41)接近，只是将文法(4.41)的产生式(7) $F \rightarrow id$ 中的标识符 **id** 替换为了常数 **digit**。与文法(4.41)一样，文法 G 也是 LR 文法，其 LR 自动机与图 4-45 类似，如图 5-19 所示。

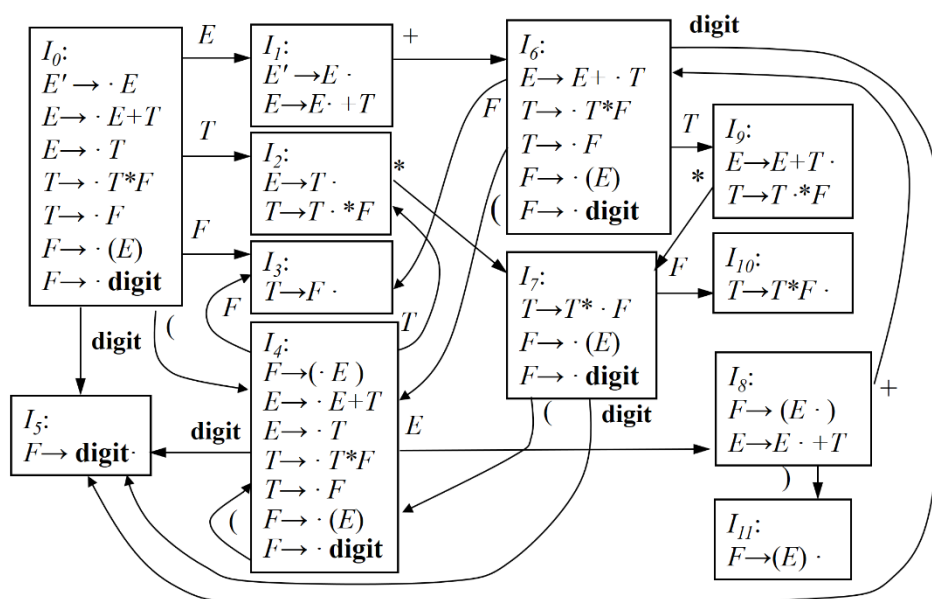
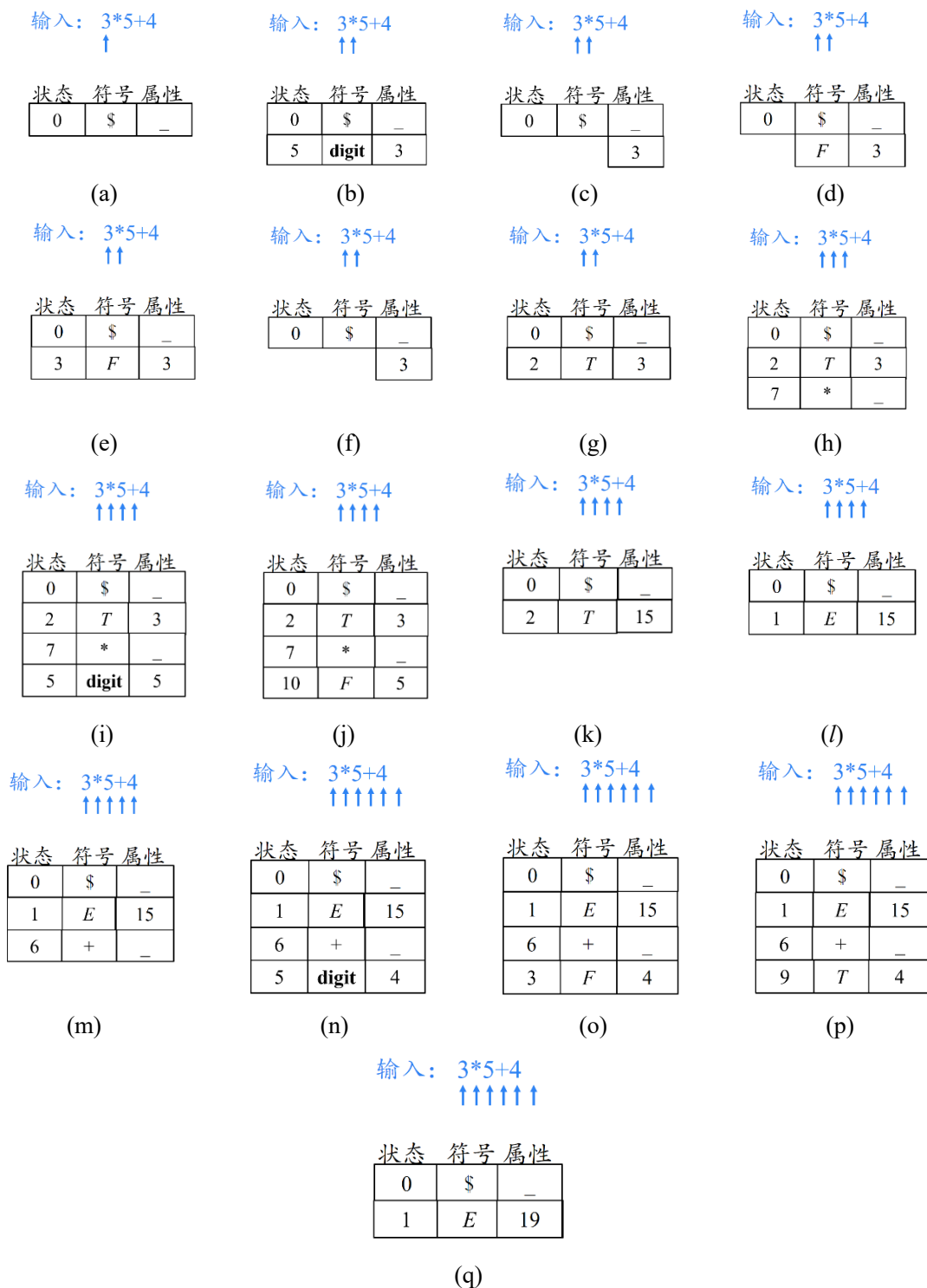


图 5-19 桌面计算器 SDT 基础文法的 LR 自动机

对于图 5-18 所示的 SDT，假设输入符号串为：3*5+4，图 5-20 显示了在 LR 语法分析过程中实现语义翻译的过程。

图 5-20 在 LR 语法分析过程中实现 $3*5+4$ 的语义翻译过程

第 1 步：初始时刻分析栈的格局如图(a)所示。状态栈中只有初始状态 0。与之平行的符号栈中只有栈底标记\$。符号\$没有属性，因此，该记录的属性值字段为空。此时，输入指针指向串首第 1 个输入符号 3，即一个 **digit**。根据图 5-19，状态 0 遇到终结符 **digit**，则将 **digit** 移入符号栈，并转入状态 5，同时输入指针指向下一输入符号*。终结符 **digit** 进栈时带着它的属性值（即 3），如图(b)所示。

第 2 步：根据图 5-19，状态 5 是一个归约状态，应用产生式(7) $F \rightarrow \text{digit}$ 进行归约，符号栈中 **digit** 带着与之对应的状态 5 出栈，状态栈的栈顶露出状态 0，如图(c)所示。非终结符 F 进入符号栈，如图(d)所示。状态 0 遇到新归约出的非终结符 F 进入状态 3，如图(e)所示。根据图 5-18，产生式(7)关于属性值字段不需要执行任何语义动作，即栈顶记录（对应文法符号 F ）的属性值字段的值不变，依然是 3。

第 3 步：根据图 5-19，当前栈顶的状态 3 依然是一个归约状态，应用产生式(5) $T \rightarrow F$ 进行归约，符号栈中 F 带着与之对应的状态 3 出栈，状态栈的栈顶露出状态 0，如图(f)所示。非终结符 T 进入符号栈，状态 0 遇到新归约出的非终结符 T 进入状态 2，如图(g)所示。根据图 5-18，产生式(5)关于属性值字段不需要执行任何语义动作，即栈顶记录（对应文法符号 T ）的属性值字段的值不变，依然是 3。

第 4 步：根据图 5-19，状态 2 遇到终结符*，则将*移入符号栈，并转入状态 7，同时输入指针指向下一输入符号 5，即一个 **digit**。运算符*是一词一（种别）码的词法单元，它没有属性值，因此，该记录的属性值字段为空。这一步执行完如图(h)所示。

第 5 步：根据图 5-19，状态 7 遇到终结符 **digit**，则将 **digit** 移入符号栈，并转入状态 5，同时输入指针指向下一输入符号+。终结符 **digit** 进栈时带着它的属性值（即 5），如图(i)所示。

第 6 步：与第 2 步类似，这里不再赘述。这一步执行完如图(j)所示。

第 7 步：根据图 5-19，状态 10 是一个归约状态，应用产生式(4) $T \rightarrow T_1 * F$ 进行归约，符号栈中的 T_1 、*和 F 带着与之对应的状态出栈，状态栈的栈顶露出状态 0，非终结符 T 进入符号栈。状态 0 遇到新归约出的非终结符 T 进入状态 2。根据图 5-18 中与产生式(4)关联的语义动作，将 $stack[top-2].val$ （即 3）与 $stack[top].val$ （即 5）相乘得到 15，并将其存入 $stack[top-2]$ 的 val 字段，这一步执行完如图(k)所示

第 8 步：根据图 5-19，状态 2 遇到输入符号+执行归约动作，应用产生式

(3) $E \rightarrow T$ 进行归约，与第 3 步类似，这里不再赘述。这一步执行完如图(l)所示。

第 9 步：与第 4 步类似，这里不再赘述。这一步执行完如图(m)所示。

第 10 步：与第 5 步类似，这里不再赘述。这一步执行完如图(n)所示。

第 11 步：与第 2 步类似，这里不再赘述。这一步执行完如图(o)所示。

第 12 步：与第 3 步类似，这里不再赘述。这一步执行完如图(p)所示。

第 3 步：根据图 5-19，状态 9 遇到除*以外的其它输入符号都采取归约动作，应用产生式(1) $E \rightarrow E_1 * T$ 进行归约。具体操作与第 7 步类似，这里不再赘述。这一步执行完如图(q)所示。栈顶的状态 1 是接收状态，属性值 19 是整个表达式的值，这样，就在语法分析的同时完成了语义翻译的任务。 □

5.3.3 L-属性定义的 SDT

在 L-SDD 中，既可以有综合属性，又可以有继承属性。将一个 L-SDD 转换为 SDT 的规则如下：

(1) 将计算一个产生式左部符号的综合属性的动作放置在这个产生式右部的最右端。

(2) 将计算某个非终结符号 A 的继承属性的动作插入到产生式右部中紧靠在 A 的本次出现之前的位置上。这是因为 L-SDD 要求产生式右部某符号 A 的继承属性仅依赖于它左兄弟节点的属性值以及产生式左部非终结符的继承属性，因此， A 的继承属性要等到所有的左兄弟都分析完毕再计算，因此，将计算 A 的继承属性的动作插入到产生式右部中紧靠在 A 的本次出现之前（也就是 A 的所有左兄弟刚刚分析完）的位置上。

例如在图 5-14 (a)中， $T.val$ 、 $T'.syn$ 和 $F.val$ 都是综合属性，因此将计算它们的值的语义动作放置在产生式的末尾。而 $T'.inh$ 和 $T_1'.inh$ 是继承属性，因此将计算它们的值的语义动作放置在紧靠在 T' 和 T_1' 的本次出现之前。

如果一个 L-SDD 的基本文法可以使用 LL 分析技术，那么它的 SDT 可以在 LL 语法分析过程中实现。

例如，对于图 5-14(a)所示的 L-SDD，其基础文法的各个产生式的 SELECT 集如下：

```
SELECT (1)= { digit }
SELECT (2)= { * }
SELECT (3)= { $ }
```

SELECT (4)= { **digit** }

可见，具有相同左部的产生式(2)和(3)的 SELECT 集{*}和{\$}互不相交，因此，该基础文法是 LL(1)文法。那么这个 SDT 可以在 LL 语法分析的同时完成语义翻译。

具体来说分为两种情况：

- (1) 在非递归的预测分析过程中进行语义翻译
- (2) 在递归的预测分析过程中进行语义翻译

接下来在 5.4 节将介绍在这种两种方式下，分别如何改造语法分析器，使它能进行语义翻译。

另外值得一提的是，这样的 L-SDD 甚至还可以在自底向上的 LR 分析中实现语义翻译，我们将在 5.5 节介绍具体的实现方法。

5.4 L-属性定义的自顶向下翻译

我们在 5.3 节中提到，实现 L-SDD 的自顶向下翻译的前提是其基础文法是 LL(1)文法。我们先通过一个例子演示一下 L-SDD 的自顶向下翻译过程。

例 5.13 我们可以将图 5-14(b)所示的 SDT 看作是扩展后的 CFG（为方便讨论，我们将其重新写在下面）。

- (1) $T \rightarrow F \{ T'.inh = F.val \} T' \{ T.val = T'.syn \}$
- (2) $T' \rightarrow *F \{ T_l'.inh = T'.inh \times F.val \} T_l' \{ T'.syn = T_l'.syn \}$
- (3) $T' \rightarrow \varepsilon \{ T'.syn = T'.inh \}$
- (4) $F \rightarrow \text{digit} \{ F.val = \text{digit.lexval} \}$

也就是说，我们将 SDT 中的语义动作（即花括号括起来的蓝色字体显示的部分）看成是一个特殊的文法符号。这样，文法中包含三类符号：终结符、非终结符、动作符号。这样的话，自顶向下的语义翻译实际上可以看作是基于这个特殊的 CFG 做语法分析。

假设输入的词串是 3*5，经过词法分析以后得到的 token 序列是 **digit** * **digit**。自顶向下分析时，首先应用产生式(1)，为树的根结点 T 生成 4 个子节点，如图 5-22 第 1 步所示。当然， F 和 T' 是正常的子节点。除此之外，我们把两个蓝色的动作符号作为附加子节点也添加到分析树中，也用蓝色的字体显示，以区别于黑色字体显示的原始语法分析树中的子节点。这样就得到了句型 $F \{ T'.inh = F.val \} T' \{ T.val = T'.syn \}$ 。

接下来选择当前句型的最左非终结符 F 进行分析。根据产生式(4), F 只有一个候选式。将 F 替换成 **digit** $\{ F.val = \text{digit.lexval} \}$, 得到句型 **digit** $\{ F.val = \text{digit.lexval} \} \{ T'.inh = F.val \} T' \{ T.val = T'.syn \}$ (如图 5-22 第 2 步所示, 其中分析树中的“(3)”表示终结符 **digit** 的词法值)。

此时, 句型中的前缀 **digit** 与当前的输入符号匹配成功, 输入指针指向下一个输入符号 “*”。接下来, 句型中露出了动作符号 $\{ F.val = \text{digit.lexval} \}$ 。对于动作符号, 我们采取的操作就是直接去执行它。这个语义动作的任务是计算父节点 F 的综合属性 val 的值, 计算过程中用到了子节点 **digit** 的词法值 $\text{lexval} = 3$ 。该动作执行完以后, F 的综合属性值就计算出来了。根据我们的约定, 把综合属性放在文法符号的右下角, 如图 5-22 第 3 步所示。

接下来句型中露出的符号还是一个动作符号 $\{ T'.inh = F.val \}$ 。这个动作符号对应的语义动作的任务是计算 T' 的继承属性 inh , 计算过程中用到了其左兄弟节点 F 的属性值 val 。该语义动作执行完了以后, T' 的继承属性 inh 就算出来了 (等于 3), 我们把它标记在, 好确定的 T' 的左下方, 如图 5-22 第 4 步所示。

接下来, 句型中露出了非终结符 T' , 根据当前输入符号 “*”, 选择产生式(2)进行替换, 得到句型 **digit** $\{ F.val = \text{digit.lexval} \} \{ T'.inh = F.val \} *F \{ T_1'.inh = T'.inh \times F.val \} T_1' \{ T'.syn = T_1'.syn \} \{ T.val = T'.syn \}$, 如图 5-22 第 5 步所示。

此时, 句型中的 “*” 与当前的输入符号匹配成功, 输入指针指向下一个输入符号 **digit**。接下来, 句型中露出非终结符 F 。接下来选择合适的 F 产生式进行进一步分析。

这个过程一直进行下去, 直至分析完毕。

步骤	输入	分析树
1	3 * 5 digit * digit ↑	
2	3 * 5 digit * digit ↑	

3	$\begin{array}{c} 3 \quad * \quad 5 \\ \text{digit} \quad * \quad \text{digit} \\ \uparrow \quad \uparrow \end{array}$	Partial parse tree for step 3. Root T has children F (with $val=3$), T', and an empty child. F has child digit (with $lexval$). Annotations: $\{T'.inh = F.val\}$, $\{T.val = T'.syn\}$.
4	$\begin{array}{c} 3 \quad * \quad 5 \\ \text{digit} \quad * \quad \text{digit} \\ \uparrow \quad \uparrow \end{array}$	Partial parse tree for step 4. Similar to step 3, but T' now has $inh=3$. Annotations: $\{T'.inh = F.val\}$, $\{T.val = T'.syn\}$.
5	$\begin{array}{c} 3 \quad * \quad 5 \\ \text{digit} \quad * \quad \text{digit} \\ \uparrow \quad \uparrow \end{array}$	Partial parse tree for step 5. T' has children F and T''. F has child digit (with $lexval$). Annotations: $\{T'.inh = F.val\}$, $\{T.val = T'.syn\}$, $\{T'.inh = T'.inh * F.val\}$, $\{T'.syn = T'.syn\}$.
6	$\begin{array}{c} 3 \quad * \quad 5 \\ \text{digit} \quad * \quad \text{digit} \\ \uparrow \quad \uparrow \quad \uparrow \end{array}$	Partial parse tree for step 6. T'' has children F and T'''. F has child digit (with $lexval$). Annotations: $\{T'.inh = F.val\}$, $\{T.val = T'.syn\}$, $\{T'.inh = T'.inh * F.val\}$, $\{T'.syn = T'.syn\}$.
7	$\begin{array}{c} 3 \quad * \quad 5 \\ \text{digit} \quad * \quad \text{digit} \\ \uparrow \quad \uparrow \quad \uparrow \quad \uparrow \end{array}$	Partial parse tree for step 7. T''' has children F and T''''. F has child digit (with $lexval$). Annotations: $\{T'.inh = F.val\}$, $\{T.val = T'.syn\}$, $\{T'.inh = T'.inh * F.val\}$, $\{T'.syn = T'.syn\}$.

8	$\begin{array}{c} 3 * 5 \\ \text{digit} * \text{digit} \\ \uparrow \uparrow \uparrow \uparrow \end{array}$	Parse tree structure for step 8: T $F_{val=3}$ $\text{digit} \{Fval = \text{digit.lexval}\} (3)$ $\{T'.inh = Fval\}$ $T'_{inh=3}$ $*$ $F_{val=5}$ $\text{digit} \{Fval = \text{digit.lexval}\} (5)$ $T'_{inh=15}$ $\{T'.syn = T'.syn\}$
9	$\begin{array}{c} 3 * 5 \\ \text{digit} * \text{digit} \\ \uparrow \uparrow \uparrow \uparrow \end{array}$	Parse tree structure for step 9: T $F_{val=3}$ $\text{digit} \{Fval = \text{digit.lexval}\} (3)$ $\{T'.inh = Fval\}$ $T'_{inh=3}$ $*$ $F_{val=5}$ $\text{digit} \{Fval = \text{digit.lexval}\} (5)$ $T'_{inh=15}$ $\{T_1'.inh = T'.inh \times Fval\}$ $T'_{syn=T_1'.syn}$ $\{T'.syn = T_1'.syn\}$
10	$\begin{array}{c} 3 * 5 \\ \text{digit} * \text{digit} \\ \uparrow \uparrow \uparrow \uparrow \end{array}$	Parse tree structure for step 10: T $F_{val=3}$ $\text{digit} \{Fval = \text{digit.lexval}\} (3)$ $\{T'.inh = Fval\}$ $T'_{inh=3}$ $*$ $F_{val=5}$ $\text{digit} \{Fval = \text{digit.lexval}\} (5)$ $T'_{inh=15}$ $\{T_1'.inh = T'.inh \times Fval\}$ $T'_{syn=T_1'.syn}$ $\{T'.syn = T_1'.syn\}$
11	$\begin{array}{c} 3 * 5 \\ \text{digit} * \text{digit} \\ \uparrow \uparrow \uparrow \uparrow \end{array}$	Parse tree structure for step 11: T $F_{val=3}$ $\text{digit} \{Fval = \text{digit.lexval}\} (3)$ $\{T'.inh = Fval\}$ $T'_{inh=3}$ $*$ $F_{val=5}$ $\text{digit} \{Fval = \text{digit.lexval}\} (5)$ $T'_{inh=15}$ $\{T_1'.inh = T'.inh \times Fval\}$ $T'_{syn=15}$ $\{T'.syn = T_1'.syn\}$
12	$\begin{array}{c} 3 * 5 \\ \text{digit} * \text{digit} \\ \uparrow \uparrow \uparrow \uparrow \end{array}$	Parse tree structure for step 12: T $F_{val=3}$ $\text{digit} \{Fval = \text{digit.lexval}\} (3)$ $\{T'.inh = Fval\}$ $T'_{inh=3}$ $*$ $F_{val=5}$ $\text{digit} \{Fval = \text{digit.lexval}\} (5)$ $T'_{inh=15}$ $\{T_1'.inh = T'.inh \times Fval\}$ $T'_{syn=15}$ $\{T'.syn = T_1'.syn\}$

图 5-21 $3 * 5$ 的自顶向下翻译过程演示

因为自顶向下的语法分析是按照从左向右深度优先的顺序分析各个文法符号，所以我们相当于是按照从左向右深度优先的顺序来执行 SDT 中的这些语义动作（如图 5-23 红色箭头指示的顺序），从而完成语义属性的计算。

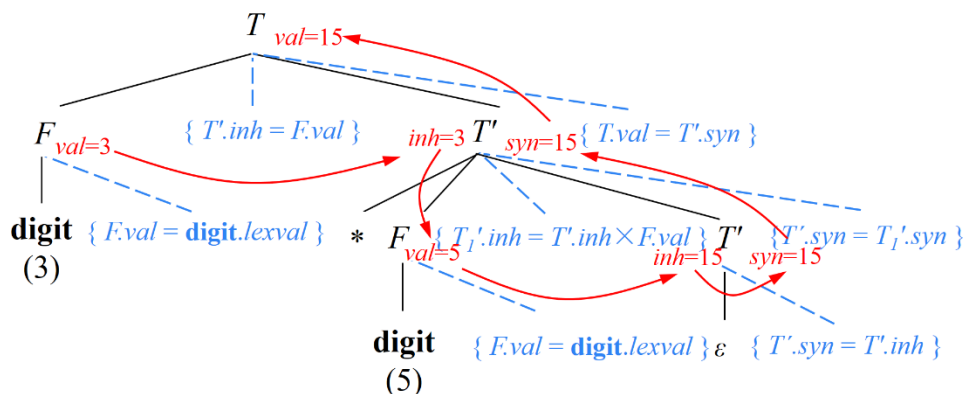


图 5-22 自顶向下翻译实际上是按照从左向右深度优先的顺序执行 SDT 中的语义动作

□

例 5.13 演示了 L-SDD 的自顶向下翻译原理。正如 5.3 节最后提到的，计算机具体实现时分为两种情况：

- (1) 在非递归的预测分析（即 LL(1)分析）过程中进行语义翻译。
- (2) 在递归的预测分析过程中进行语义翻。

接下来我们分别介绍。

5.4.1 在 LL(1)语法分析过程中进行语义翻译

要想在 LL(1)语法分析过程中进行语义翻译，需要扩展语法分析栈（如图 5-23 所示）。之前的 LL(1)语法分析器中只有符号栈，扩展过程中主要考虑以下因素：

(1) 首先要扩展出属性值字段。SDD 在 CFG 的基础上设置了语义属性，因此在分析栈中要有相应的字段（如图 5-23 中标记为 *val* 的区域）来存放属性值。

(2) 在 L-SDD 中，对于一个非终结符 A 来说，它既可以有继承属性，又可以有综合属性。但是综合属性和继承属性的计算时机不同（继承属性在 A 出现之前就可以计算，而综合属性在 A 的子节点都分析完毕之后才能计算），因此，将它们存放在同一个记录中是不合适的。于是，需要将非终结符 A 的继承属性和综合属性存放在不同的记录中。具体来说，继承属性就存放在 A 本身记录的

属性值字段中。另外增加一种新的类型的记录——综合记录 $Asyn$ ，用于专门存放非终结符 A 的综合属性值。 A 的综合记录在栈中就紧压在 A 本身记录的下面。

(3) 终结符只有综合属性，其综合属性就存放在其记录本身的属性值字段中。

(4) 由于 SDT 在产生式右部中嵌入了一些语义动作，因此还增加一种动作记录 $action$ ，用来存放指向被执行的语义动作代码的指针。

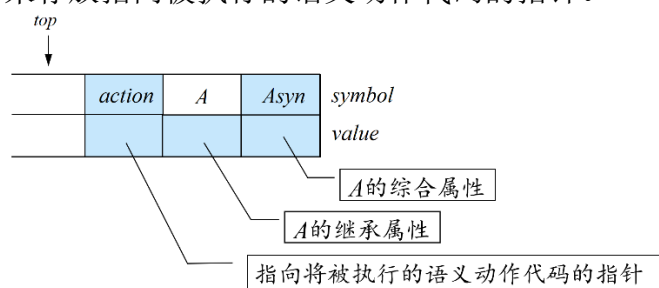


图 5-23 用于语义翻译的扩展后的 LL(1)语法分析栈

下面我们通过一个例子来演示如何利用这个扩展后的语法分析器进行语义翻译。

例 5.14 对于图 5-14(b)所示的 SDT，为了便于讨论，将其改写成图 5-23 所示的形式：

	$a_1 : T'.inh = F.val$
(1) $T \rightarrow F \{ a_1 \} T' \{ a_2 \}$	$a_2 : T.val = T'.syn$
(2) $T' \rightarrow *F \{ a_3 \} T_1' \{ a_4 \}$	$a_3 : T_1'.inh = T'.inh \times F.val$
(3) $T' \rightarrow \varepsilon \{ a_5 \}$	$a_4 : T'.syn = T_1'.syn$
(4) $F \rightarrow \text{digit} \{ a_6 \}$	$a_5 : T'.syn = T'.inh$
	$a_6 : F.val = \text{digit}.lexval$

图 5-23 将图 5-14(b)所示 SDT 的改写成等价的形式

也就是说，将原始 SDT 中包含的 6 个语义动作分别命名为 a_1 、 a_2 、……、 a_6 ，并将它们被嵌入到产生式右部的不同位置。改写后的 SDT 是一种特殊的 CFG，它包含三种符号：终结符，非终结符和动作符号。各文法符号的属性情况如图 5-24 所示。

	继承属性	综合属性
T		val
T'	inh	syn
F		val

图 5-24 图 5-23 中各文法符号的属性情况

假设输入的词串还是 $3*5$ ，经过词法分析以后得到的 token 序列是 **digit** *

digit。初始时刻，栈中存放着文法开始符号 T 对应的记录。由于 T 没有继承属性，因此 T 本身记录中的属性字段不存放任何信息。但是 T 具有综合属性，因此 T 有一个综合记录 $Tsyn$ ，记录的属性字段用来存放 T 的综合属性 val 。 T 的综合记录 $Tsyn$ 位于栈中紧靠在 T 本身的记录之下，如图 5-25 所示。

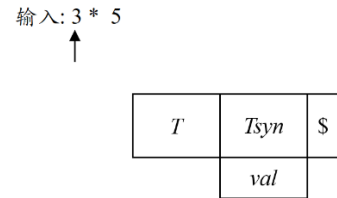


图 5-25 串 3*5 的翻译过程：分析器的初始格局

第(1)步：此时栈顶的非终结符 T 表示当前句型的最左非终结符。根据产生式(1)，将栈顶非终结符 T 替换成 $F\{a_1\}T'\{a_2\}$ 。于是， T 本身的记录出栈，但是它的综合记录并不出栈（因为 T 的综合属性要等 T 的即将进栈的儿子们分析完毕才能计算出来）， F 、 $Fsyn$ 、 $\{a_1\}$ 、 T' 、 $T'syn$ 、 $\{a_2\}$ 进栈，如图 5-26 所示。

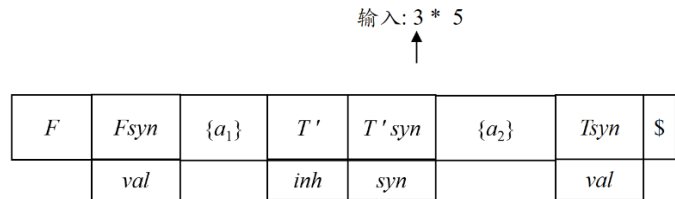


图 5-26 串 3*5 的翻译过程：第(1)步

第(2)步：对于栈顶的非终结符 F ，根据产生式(4)，将栈顶非终结符 F 替换成 **digit** $\{a_6\}$ 。于是， F 本身的记录出栈，**digit**、 $\{a_6\}$ 进栈，如图 5-27 所示。

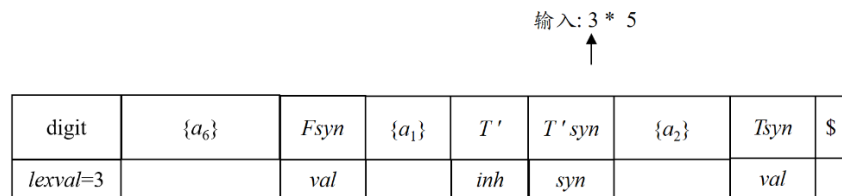


图 5-27 串 3*5 的翻译过程：第(2)步

第(3)步：此时，栈顶是一个终结符 **digit**，它与当前输入符号 3 匹配，于是，**digit** 出栈。但是，根据产生式(4)，紧跟在 **digit** 后面的语义动作 a_6 将使用

digit 的属性值 *lexval* 来计算左部非终结符 *F* 的综合属性 *val*，因此，**digit** 出栈前要把这个属性值复制到动作记录 *a₆* 中的相应字段。也就是说，*a₆* 中设置了一个字段用来对 **digit** 的 *lexval* 值进行备份，如图 5-28 所示。

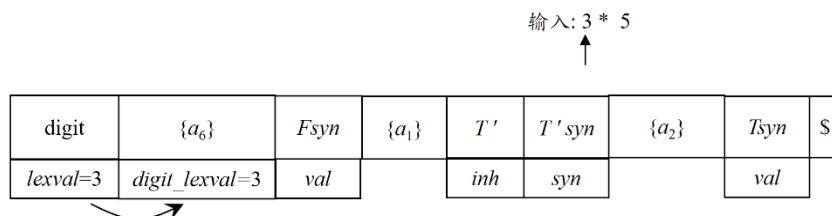


图 5-28 串 3*5 的翻译过程：第(3)步

第(4)步：备份后，**digit** 记录出栈，输入指针移向下一个符号 “*” ，如图 5-29 所示。

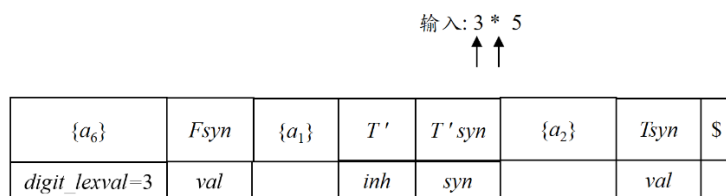


图 5-29 串 3*5 的翻译过程：第(4)步

第(5)步：此时，栈顶出现了动作记录 {*a₆*}。*a₆* 作为产生式(4)末尾的语义动作，其任务就是计算产生式左部符号（也就是父节点）*F* 的综合属性 *val* 的值。这里，我们将语义动作 *a₆* 中的抽象定义式改写成可具体执行的栈操作。根据 *a₆* 的抽象定义式

$$F.val = \mathbf{digit.lexval} \quad (5.8)$$

F 的综合属性 *val* 的值是通过 **digit** 的综合属性 *lexval* 计算的。而 **digit.lexval** 已经在栈顶的动作记录 {*a₆*} 中做了备份，因此我们可以将它改写为 *stack[top].digit_lexval*。*F* 的综合属性 *val* 的值计算出来以后需要存放在 *F* 的综合记录 *Fsyn* 里，而 *Fsyn* 此时就位于 *stack[top-1]* 这个位置（因为根据产生式(4)，{*a₆*} 是跟着它前面的 **digit** 一起进栈来替换当时位于栈顶的 *F*。所以 {*a₆*} 进栈以后就位于 *F* 之前所在的位置。而 *F* 的综合记录 *Fsyn* 紧压在 *F* 本身的记录下面。因此，{*a₆*} 进栈以后，*Fsyn* 就紧压在 {*a₆*} 下面，即当前 *stack[top-1]* 这个位置）。因此，将式(5.8)所示的抽象定义式改写为式(5.9)所示的可具体执行的栈操作：

$$\mathbf{stack[top-1].val = stack[top].digit_lexval} \quad (5.9)$$

这样的话，{*a₆*} 中除了备份必要的属性值字段外，还会存放指向语义动作代码(5.9)的指针，如图 5-30 所示。

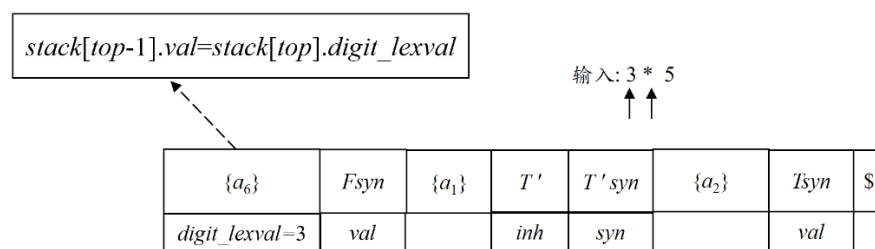


图 5-30 串 3*5 的翻译过程：第(5)步

第(6)步：执行 $\{a_6\}$ 指向的代码，将计算出来的 F 的综合属性的值存放到 $Fsyn$ 的属性值字段，如图 5-31 所示。

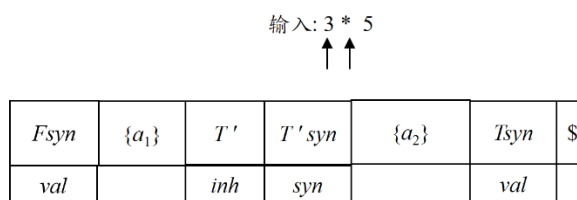


图 5-31 串 3*5 的翻译过程：第(6)步

第(7)步：现在， F 的综合属性计算完毕，它可以像 F 本身的记录那样出栈了。我们知道，当前栈顶的 $Fsyn$ 是在第(1)步应用产生式(1)时进栈的。根据产生式(1)，紧跟在 F 后面的语义动作 a_1 将使用 F 的这个综合属性 $Fsyn$ 的值来计算 T 的继承属性 inh ，因此， $Fsyn$ 出栈前要把这个属性值复制到动作记录 $\{a_1\}$ 中的相应字段，也就是说， $\{a_1\}$ 中设置了一个字段用来对 F 的综合属性 val 值进行备份，如图 5-32 所示。

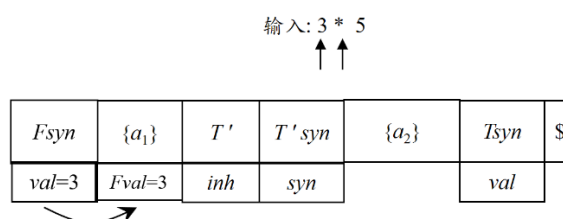


图 5-32 串 3*5 的翻译过程：第(7)步

第(8)步：备份后， $Fsyn$ 记录出栈，栈顶出现了动作记录 $\{a_1\}$ ，如图 5-33 所示。

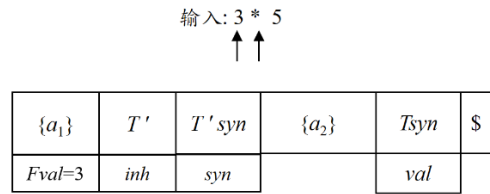


图 5-33 串 3*5 的翻译过程：第(8)步

第(9)步：根据产生式(1)，仅靠在 T' 前面的语义动作 $\{a_1\}$ 的任务是计算 T' 的继承属性 inh ，并将计算出的结果存放在 T' 本身的记录里（即 $stack[top-1]$ 所在的位置）。而计算过程中用到的 F 的综合属性 val 的值已经在栈顶 $\{a_1\}$ 的记录中做了备份。因此可以将图 5-24 中 a_1 的抽象定义式改写为式(5.10)所示的可具体执行的栈操作：

$$stack[top-1].inh = stack[top].Fval \quad (5.10)$$

这样， $\{a_1\}$ 中除了备份必要的属性值字段外，还会存放指向语义动作代码(5.10)的指针，如图 5-34 所示。

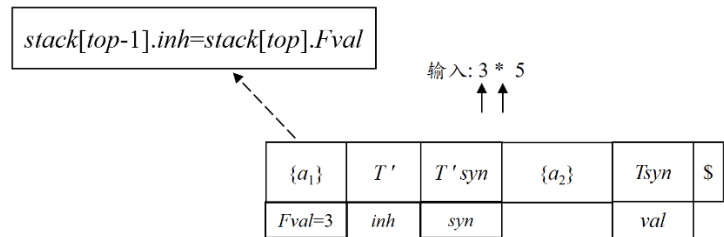


图 5-34 串 3*5 的翻译过程：第(9)步

第(10)步：执行 a_1 对应的代码，将计算出来的 T' 的继承属性 inh 的值存放到 T' 的属性值字段，如图 5-35 所示。

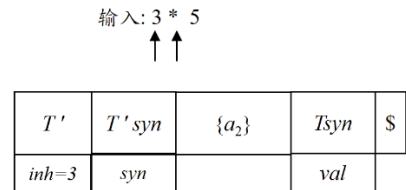


图 5-35 串 3*5 的翻译过程：第(10)步

第(11)步：对于栈顶的非终结符 T' ，和当前的输入符号 $*$ ，选择产生式(2)，将栈顶非终结符 T' 替换成 $*F\{a_3\}T_1'\{a_4\}$ 。于是， T' 本身的记录出栈。根据产生式(2)，动作记录 $\{a_3\}$ 将使用存放在记录 T' 中的继承属性 inh 的值来计算子节点 T_1' 的继承属性 inh 。因此，记录 T' 出栈前要把这个属性值备份到即将进栈的动作记录 $\{a_3\}$ 的相应字段中，如图 5-36 所示。

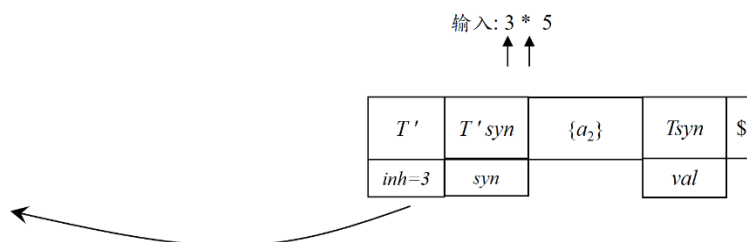


图 5-36 串 3*5 的翻译过程: 第(11)步(a)

那么, $\{a_3\}$ 进栈后将位于什么位置呢? 我们知道, T' 出栈以后, 位于产生式(2)末尾的 $\{a_4\}$ 将处于 T' 当前所处的位置, 即 $stack[top]$ 。因为产生式(2)中的 T_1' 有综合属性, 所以 T_1' 的综合记录 $T_1'syn$ 也要跟 T_1' 本身的记录进栈, 分别位于 $stack[top+1]$ 和 $stack[top+2]$, 这样, $\{a_3\}$ 就位于 $stack[top+3]$ 。所以 T' 出栈前要把它的继承属性 inh 的值备份到 $stack[top+3]$ 中。然后, T' 出栈, $*$ 、 F 、 $Fsyn$ 、 $\{a_3\}$ 、 T_1' 、 $T_1'syn$ 、 $\{a_4\}$ 进栈, 如图如图 5-37 所示。

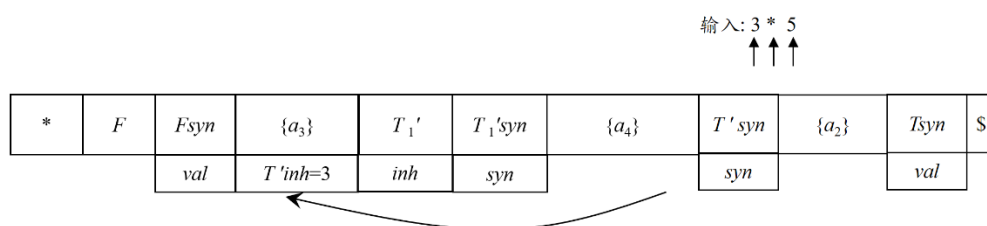


图 5-37 串 3*5 的翻译过程: 第(11)步(b)

第(12)步: 此时, 栈顶是终结符 “*” 对应的记录, 与当前输入符号 “*” 匹配, 于是, “*” 对应的记录出栈, 输入指针指向下一个输入符号, 即输入结束标记 “\$”。由于终结符 “*” 没有属性值, 因此, “*” 对应的记录出栈时无需备份, 此时的分析器格局如图 5-38 所示。

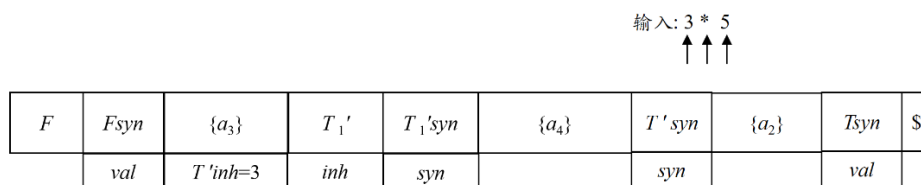
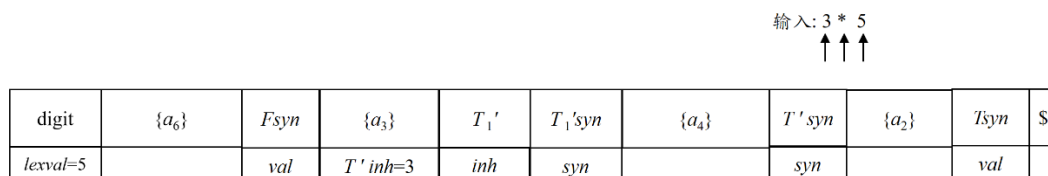
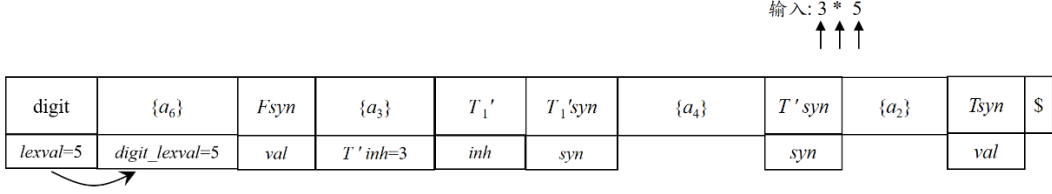


图 5-38 串 3*5 的翻译过程: 第(12)步

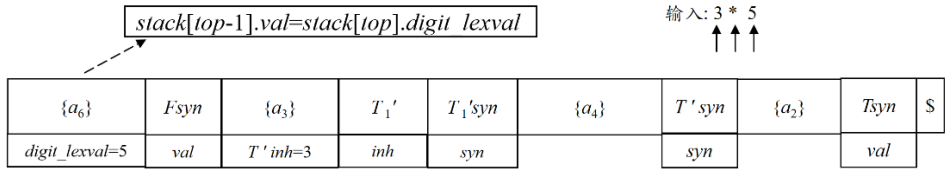
第(13)步: 这个过程一直进行下去, 如图 5-39 所示。



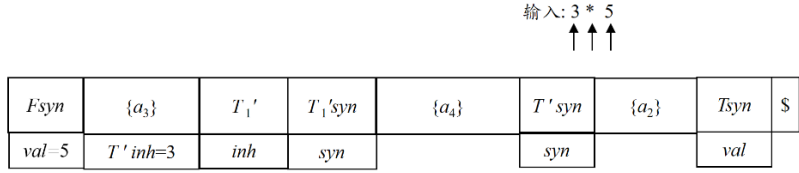
(a)



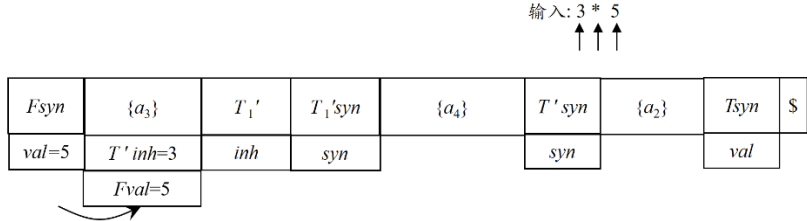
(b)



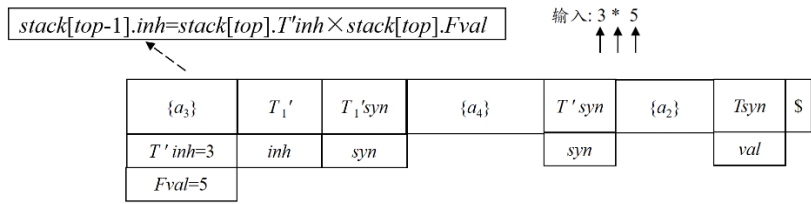
(c)



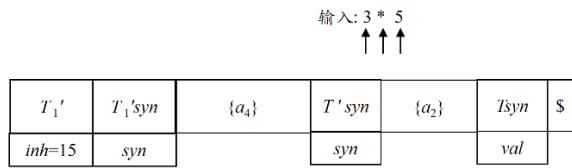
(d)



(e)



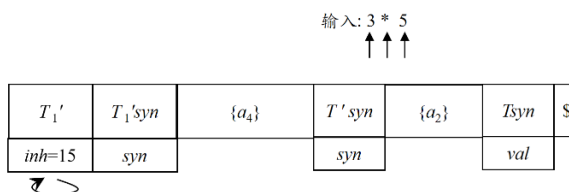
(f)



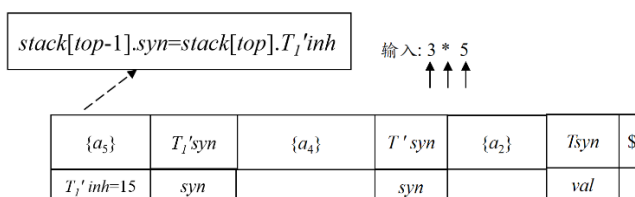
(g)

图 5-39 串 3*5 的翻译过程：第(13)步

第(14)步：此时，对于栈顶的非终结符 T' 和当前的输入符号“\$”，选择产生式(3)，将栈顶非终结符 T' 替换成 $\{a_5\}$ 。于是， T' 本身的记录出栈。根据产生式(3)，动作记录 $\{a_5\}$ 将使用存放在记录 T' 中的继承属性 inh 的值来计算 T' 的综合属性 syn 。因此，记录 T' 出栈前要把这个属性值备份到即将进栈的动作记录 $\{a_5\}$ 的相应字段中。与第(11)步的分析类似，我们可以推算出 $\{a_5\}$ 进栈后将位于记录 T' 此时所处的位置，因此， T' 的继承属性 inh 的值还保留在栈顶记录的属性值字段中，如图 5-40 所示。



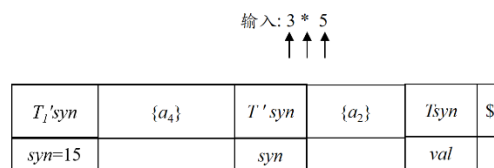
(a) T' 的继承属性 inh 的值还保留在栈顶记录的属性值字段中



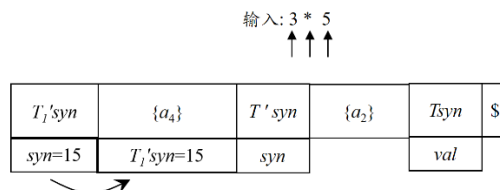
(b) 动作记录 $\{a_5\}$ 中备份了 T' 的继承属性 inh 的值

图 5-40 串 3*5 的翻译过程：第(14)步

这个过程一直进行下去，直至分析结束。如图 5-41 显示剩余各个步骤。



(a)



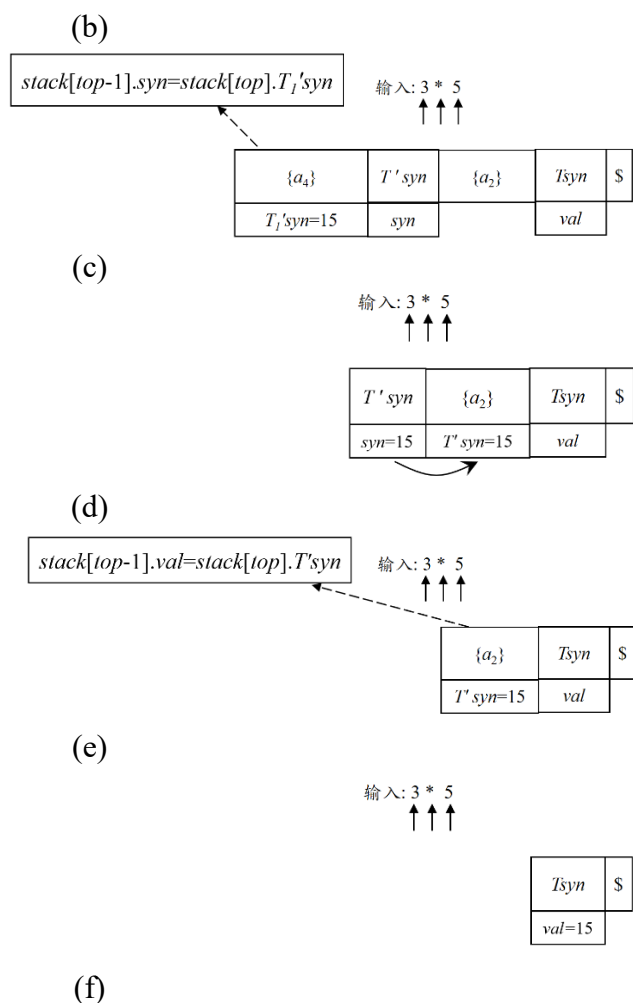


图 5-41 3*5 的翻译过程：剩余步骤

□

通过例 5.15 可以看出，分析栈中的每一个记录（不光是动作记录，也包括综合记录和文法符号本身的记录）都可能对应着一段执行代码，因为涉及到属性值的备份。具体来说，

- (1) 综合记录出栈时，要将综合属性值复制给后面特定的语义动作。
- (2) 变量展开时（即变量本身的记录出栈时），如果其含有继承属性，则要将继承属性值复制给后面特定的语义动作。

这就要求我们在设计 SDT 的时候，要把每一个文法符号对应的记录的语义动作（具体可执行的栈操作）设计好。

例 5.15 对于图 5-24 所示的 SDT，为其每一条产生式中的各个文法符号对应的记录设计的语义动作如图 5-42 所示。

符号	属性	执行代码 (val 字段)
F		$top=top+1;$
F_{syn}	val	$stack[top-1].Fval = stack[top].val; top=top-1;$
a_1	$Fval$	$stack[top-1].inh = stack[top].Fval; top=top-1;$
T'	inh	根据当前输入符号选择产生式进行推导 若选产生式(2): $stack[top+3].T'inh = stack[top].inh; top=top+6;$ 若选产生式(3): $stack[top].T'inh = stack[top].inh;$
T'_{syn}	syn	$stack[top-1].T'syn = stack[top].syn; top=top-1;$
a_2	$T'syn$	$stack[top-1].val = stack[top].T'syn; top=top-1;$
(a) 产生式(1) $T \rightarrow F \{a_1:T'.inh=F.val\} T' \{a_2:T.val=T'.syn\}$		
符号	属性	执行代码 (val 字段)
$*$		$top=top-1;$
F		$top=top+1;$
F_{syn}	val	$stack[top-1].Fval = stack[top].val; top=top-1;$
a_3	$T'inh;$ $Fval$	$stack[top-1].inh = stack[top].T'inh \times stack[top].Fval; top=top-1;$
T'_1	inh	根据当前输入符号选择产生式进行推导 若选产生式(2): $stack[top+3].T'inh = stack[top].inh; top=top+6;$ 若选产生式(3): $stack[top].T'inh = stack[top].inh;$
T'_1_{syn}	syn	$stack[top-1].T'_1syn = stack[top].syn; top=top-1;$
a_4	T'_1syn	$stack[top-1].syn = stack[top].T'_1syn; top=top-1;$
(b) 产生式(2) $T' \rightarrow *F\{a_3:T'_1.inh=T'.inh \times Fval\} T'_1\{a_4:T'.syn=T'_1.syn\}$		
符号	属性	执行代码 (val 字段)
a_5	$T'inh$	$stack[top-1].syn = stack[top].T'inh;$ $top=top-1;$
(c) 产生式(3) $T' \rightarrow \varepsilon \{a_5:T'.syn = T'.inh\}$		

符号	属性	执行代码 (val 字段)
digit	lexval	$stack[top-1].digitlexval = stack[top].lexval;$ $top = top - 1;$
a_6	digitlexval	$stack[top-1].val = stack[top].digitlexval;$ $top = top - 1;$

(d) 产生式(4) $F \rightarrow digit \{a_6: F.val = digit.lexval\}$

图 5-42 图 5-24 中各文法符号对应记录的语义动作

□

5.4.2 在递归的预测分析过程中进行语义翻译

我们在第 4 章递归的预测分析中介绍过，递归的预测分析器中的每一个非终结符 A 都对应一个过程 $A(token)$ 。我们可以将一个递归的预测分析器扩展为一个翻译器。扩展过程中主要考虑以下因素：

(1) SDD 在 CFG 的基础上设置了语义属性，因此对与出现在 A 产生式右部中的每个文法符号的每个属性都要在过程 A 中为其设置一个局部变量用来存放属性的值。

(2) 我们在 4.2.1 节最后提到，递归下降分析实际上是按照从左到右、深度优先的顺序递归调用文法中特定非终结符对应的过程。而在 L-SDD 中，一个非终结符 A 既可以有继承属性，又可以有综合属性。对于一个 L-SDD 的 SDT 来说， A 的继承属性在调用过程 A 之前已经被计算出来了（因为计算 A 的继承属性的语义动作被放置在仅靠 A 的本次出现之前的位置上），而 A 的综合属性要在调用完过程 A （也就是 A 的所有子节点都分析完毕）之后才能计算出来。因此，我们将非终结符 A 的每个继承属性作为过程 A 的一个形参（因为继承属性在调用过程 A 之前已经被计算出来了），而将非终结符 A 的综合属性值作为过程 A 的返回值（也就是说在过程调用结束时计算出综合属性）。由于过程 A 带有返回值，所以我们实际上已经将非终结符 A 对应的过程扩展成函数了。

(3) 对于 SDT 中的每个语义动作，将其代码复制到语法分析器，并把对属性的引用改为对相应变量的引用。

例 5.16 对于图 5-14(b)所示的 SDT（为方便讨论，我们将该 SDT 及各文法符号的属性情况重新写在下面）：

- (1) $T \rightarrow F \{ T'.inh = F.val \} T' \{ T.val = T'.syn \}$
- (2) $T' \rightarrow *F \{ T_1'.inh = T'.inh \times F.val \} T_1' \{ T'.syn = T_1'.syn \}$

(3) $T' \rightarrow \varepsilon \{ T'.syn = T'.inh \}$

(4) $F \rightarrow \text{digit} \{ F.val = \text{digit.lexval} \}$

	继承属性	综合属性
T		val
T'	inh	syn
F		val

扩展后的递归的预测分析器如图 5-43 所示。其中图(a)是文法中非终结符 T 对应的函数，(b)是非终结符 T' 对应的函数，(c)是非终结符 F 对应的函数。图中黑色字体部分是原有的语法分析器中 T 对应的过程，蓝色字体是扩充的部分。

```

Tval T(token)
{
    D: Fval, T'inh, T'syn;
    Fval = F(token);
    T'inh = Fval;
    T'syn = T'(token, T'inh);
    Tval = T'syn;
    return Tval;
}

```

(a)非终结符 T 对应的函数

```

T'syn T'(token, T'inh)
{
    D: Fval, T1'inh, T1'syn;
    if token = "*" then
    {
        GetNextToken(token);
        Fval = F(token);
        T1'inh = T'inh × Fval;
        T1'syn = T1'(token, T1'inh);
        T'syn = T1'syn;
        return T'syn;
    }
    else if token = "$" then
    {
        T'syn = T'inh;
    }
}

```

```

        return T'syn;
    }
    else error();
}

```

(b)非终结符 T 对应的函数

```

Fval F(token)
{
    D: digitlexval ;

    if token ≠ “digit” then Error();
    digitlexval = token.lexval;
    GetNextToken (token);
    Fval= digitlexval;
    return Fval;
}

```

(c)非终结符 F 对应的函数

```

Desent()
{
    D: Tval;

    GetNextToken (token);
    Tval = T(token);

    if token ≠ “$” then error();
    return;
}

```

(d)主程序 $Desent()$

图 5-43 用一个递归的预测分析器实现项的翻译

我们以图(b)非终结符 T 对应的函数为例进行讲解。

第 1 行：函数 $T()$ 的返回值 $T'syn$ 对应着非终结符 T 的综合属性 syn ，函数 $T()$ 的第 2 个参数 $T'inh$ 对应着非终结符 T 的继承属性 inh 。

第 2 行：非终结符 T 的对应着产生式(2)和产生式(3)。对于在这两个产生式右部中出现的每个文法符号 (F 和 T_1') 的每个属性 ($Fval$ 、 $T_1'.inh$ 和 $T_1'.syn$)，都为其设置了局部变量 (分别是 $Fval$ 、 $T_1'inh$ 和 $T_1'syn$)。

第 3~15 行： T 有两个候选式，我们根据这两个候选式的 SELECT 集来决定

执行哪段代码。因为 $\text{SELECT}(2) = \{*\}$ ，所以当输入的参数 `token` 为 “*” 时，应用产生式(2)，执行第 4~9 行代码。又因为 $\text{SELECT}(3) = \{\$ \}$ ，所以当输入的参数 `token` 为 “\$” 时，应用产生式(3)，执行第 12~13 行代码。除此之外，表明当前输入的单词不合法，因此执行第 15 行代码报错。

第 4 行：对应于产生式(2)右部第 1 个文法符号，即终结符 “*”。由于当前输入符号 `token` 与产生式中的终结符 “*” 匹配成功，所以调用 `GetNextToken()` 函数读入下一个输入符号。

第 5 行：对应于产生式(2)右部第 2 个文法符号 F 。于是调用 F 对应的函数。由于 F 具有综合属性，因此调用函数 $F()$ 之后会得到一个返回值，我们把它赋给 F 的综合属性 val 对应的局部变量 $Fval$ 。

第 6 行：对应于产生式(2)右部第 3 个符号，即内嵌的语义动作 $\{ T_1'.inh = T'.inh \times Fval \}$ 。我们将其代码复制到语法分析器，并把对属性的引用改为对相应变量的引用。其中 T' 的继承属性 inh 是作为函数 $T'()$ 的参数传进来的，因此将语义动作中的 $T'.inh$ 替换成参数 $T'inh$ 。然后把对属性 $Fval$ 和 $T_1'.inh$ 的引用替换成对局部变量 $Fval$ 和 $T_1'inh$ 的引用。

第 7 行：对应于产生式(2)右部第 4 个文法符号 T_1' ，于是调用 T_1' 对应的函数。因为 T_1' 具有继承属性，因此将 T_1' 的继承属性对应的局部变量 $T_1'inh$ 作为参数传递给函数 $T_1'()$ 。又因为 T_1' 具有综合属性，因此调用函数 $T_1'()$ 之后会得到一个返回值，我们把它赋给 T_1' 的综合属性 syn 对应的局部变量 $T_1'syn$ 。

第 8 行：对应于产生式(2)右部第 5 个符号，即内嵌的语义动作 $\{ T'.syn = T_1'.syn \}$ 。将其代码复制到语法分析器时，我们把对属性 $T_1'.syn$ 的引用替换成对局部变量 $T_1'syn$ 的引用。而 T' 的综合属性 syn 对应于函数 $T'()$ 的返回值 $T'syn$ ，因此，用 $T'syn$ 替换 $T'.syn$ 。

第 9 行：返回 $T'syn$ 。 □

下面给出构造递归的预测翻译器的算法。

算法 5.17 构造递归的预测翻译器。

输入：一个带有适于预测分析的基础文法的 SDT。

输出：递归的预测翻译器的代码。

方法：

1. 为每个非终结符 A 构造一个函数， A 的每个继承属性对应应该函数的一个形参，函数的返回值是 A 的综合属性值。对出现在 A 产生式中的每个文法符号的每个属性都设置一个局部变量

2. 非终结符 A 的代码根据当前的输入决定使用哪个产生式

3. 与每个产生式有关的代码执行如下动作：从左到右考虑产生式右部的终结符（词法单元）、非终结符及语义动作

(1) 对于终结符（词法单元） a ，如果它带有综合属性 x ，把 x 的值保存在局部变量 $a.x$ 中；然后产生一个匹配 X 的调用，并继续输入。

(2) 对于非终结符 B ，产生一个右部带有函数调用的赋值语句 $c := B(b_1, b_2, \dots, b_k)$ ，其中， $b_1, b_2, \dots, b_k$ 是代表 B 的继承属性的变量， c 是代表 B 的综合属性的变量。

(3) 对于每个语义动作，将其代码复制到语法分析器，并把对属性的引用改为对相应变量的引用。□

5.5 L-属性定义的自底向上翻译

我们在 5.3 节中提到，如果一个 L-SDD 的基本文法可以使用 LL 分析技术，那么可以在自底向上的 LR 分析中实现语义翻译。由于 L-SDD 的产生式中会存在内嵌的语义动作，也就是说不是所有的语义动作都位于产生式末尾，因此，不能直接采用自底向上的方法进行翻译。于是，我们需要修改文法，使之适合于自底向上翻译。

例 5.18 对于图 5-14 所示的 SDD 与 SDT，为方便讨论，我们将它们重新写在下面：

产生式	语义规则
(1) $T \rightarrow F T'$	$T'.inh = F.val$ $T.val = T'.syn$
(2) $T' \rightarrow * F T'_1$	$T'_1.inh = T'.inh \times F.val$ $T'.syn = T'_1.syn$
(3) $T' \rightarrow \varepsilon$	$T'.syn = T'.inh$
(4) $F \rightarrow \text{digit}$	$F.val = \text{digit.lexval}$

(a)

(1') $T \rightarrow F \{ T'.inh = F.val \} T' \{ T.val = T'.syn \}$
(2') $T' \rightarrow *F \{ T'_1.inh = T'.inh \times F.val \} T'_1 \{ T'.syn = T'_1.syn \}$
(3') $T' \rightarrow \varepsilon \{ T'.syn = T'.inh \}$
(4') $F \rightarrow \text{digit} \{ F.val = \text{digit.lexval} \}$

(b)

我们在 5.5.3 节计算过其基础文法的各个产生式的 SELECT 集：

$$\text{SELECT}(1) = \{ \text{digit} \}$$

$$\text{SELECT (2)} = \{ \quad * \quad \}$$

$$\text{SELECT (3)} = \{ \quad \$ \quad \}$$

$$\text{SELECT (4)} = \{ \text{digit} \}$$

我们将该 SDT 修改后为图 5-44 所示的形式。

$(1.1') \quad T \rightarrow F M T' \{ T.val = T'.syn \}$ $(1.2') \quad M \rightarrow \varepsilon \{ M.T'inh = F.val \}$ $(2.1') \quad T' \rightarrow * F N T_1' \{ T'.syn = T_1'.syn \}$ $(2.2') \quad N \rightarrow \varepsilon \{ N.T_1'inh = T'.inh \times F.val \}$ $(3') \quad T' \rightarrow \varepsilon \{ T'.syn = T'.inh \}$ $(4') \quad F \rightarrow \text{digit} \{ F.val = \text{digit.lexval} \}$

图 5-44 对图 5-14 中的 SDT 进行修改后 SDT

也就是说，对于产生式(1)中内嵌的语义动作，向文法中引入一个非终结符 M 来替换它。对于产生式(2)中内嵌的语义动作，向文法中引入一个非终结符 N 来替换它。为保证文法的等价性， M 和 N 不能推导出任何文法符号，而只能推导出空串。因此 M 和 N 分别对应着一个空产生式 $M \rightarrow \varepsilon$ 和 $N \rightarrow \varepsilon$ 。这种只能推导出空串的非终结符称为标记非终结符，用来标记这个位置原本是一个语义动作。产生式 $M \rightarrow \varepsilon$ 关联着一段语义子程序，它的任务就是执行 M 所替换的那个语义动作要完成的任务。产生式 $N \rightarrow \varepsilon$ 也对应着一段语义子程序，它的任务就是执行 N 所替换的那个语义动作要完成的任务。 N 所替换的那个语义动作要完成的工作是计算 T_1' 的继承属性值 $T_1'.inh$ ，计算过程中要用到 T' 的继承属性值 $T'.inh$ 和 F 的综合属性值 $F.val$ 。于是，我们为 N 设置一个综合属性 $N.T_1'inh$ （因为要想做自底向上的翻译，所有的属性都必须是综合属性。而名字 $T_1'inh$ 则体现了 N 所替换的语义动作的任务），并令它的值等于原语义动作中的运算结果。这样，与 $N \rightarrow \varepsilon$ 关联的这段语义子程序实际上就完成了与 N 所替代的那个语义动作相同的工作，计算出来的结果就放在 N 的综合属性里。类似的， M 所替换的那个语义动作要完成的任务是计算 T' 的继承属性值 $T'.inh$ ，计算过程中要用到 F 的综合属性值 $F.val$ 。为此，我们为 M 设置综合属性 $M.T'inh$ ，并令它的值等于原语义动作中的运算结果。

修改后的 SDT，功能上与原 SDT 是等价的，形式上，所有语义动作都位于产生式末尾，可以进行自底向上的翻译。但是这个变换看起来是非法的，因为和这两个空产生式相关的语义动作将不得不访问没有出现在这两个产生式中的文法符号 F 和 T' 的属性。然而，我们将基于 LR 语法分析栈实现各个语义动作，因此，这些必要的属性总是位于栈顶之下的已知位置上。

我们首先证明一下，如果原始文法是 LL 文法，则修改后的文法是 LR 的。

例如，我们已知(a)中给出的基础文法（产生式(1)~(4)）是 LL 文法，因此，在自顶向下的语法分析时，我们从文法开始符号 T 开始分析，如图 5-45 第 1 步 (a)列所示。另一方面，从自底向上分析的角度，我们期待最终识别（归约）出文法开始符号 T ，因此，从初始项目 $S' \rightarrow \cdot T$ 出发，如图中第 1 步(b)列所示。

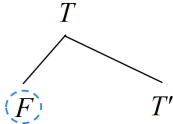
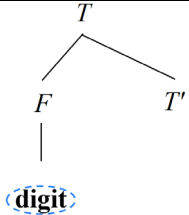
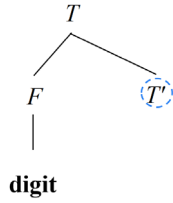
步骤	(a) 自顶向下的语法分析	(b) 自底向上的语法分析
1		$I_0:$ $S' \rightarrow \cdot T$
2		$I_0:$ $S' \rightarrow \cdot T$ $T \rightarrow \cdot FMT'$
3		$I_0:$ $S' \rightarrow \cdot T$ $T \rightarrow \cdot FMT'$ $F \rightarrow \cdot d$
4		$I_0:$ $S' \rightarrow \cdot T$ $T \rightarrow \cdot FMT'$ $F \rightarrow \cdot d$ $\xrightarrow{F}$ $I_2:$ $T \rightarrow F \cdot MT'$ $M \rightarrow \cdot , */\\$ $\xrightarrow{M}$ $I_4:$ $T \rightarrow FM \cdot T'$ $T' \rightarrow \cdot *FNT'$ $T' \rightarrow \cdot , \\$ $I_3:$ $F \rightarrow d \cdot , */\\$

图 5-45 示例：如果一个文法是 LL 文法，则它必然也是 LR 的

第 2 步：接下来，从自顶向下分析的角度，对于非终结符 T ，根据产生式 (1)， T 由 F 和 T' 构成的。因此，要想分析 T 就得先分析构成 T 的第一个成分 F ，如图中第 2 步(a)列所示。另一方面，从自底向上分析的角度，我们要归约出 T ，首先需要等待归约出构成 T 的第一个成分 F ，如图中第 2 步(b)列所示。

第 3 步：接下来，从自顶向下分析的角度，对于非终结符 F ，根据产生式 (4)， F 由一个终结符 **digit** 构成，如图中第 3 步(a)列所示。另一方面，从自底向上分析的角度，我们要归约出 F ，首先需要期待识别出构成 F 的第一个成分 **digit**，如图中第 3 步(b)列所示。

第 4 步：此时，如果当前输入符号是 **digit**，那么，从自顶向下分析的角度，

当前输入符号匹配成功。输入指针指向下一个输入符号，并继续分析构成 T 的第 2 个成分 T' ，如图中第 4 步(a)列所示。 T' 有两个候选式，分别是产生式(2)和(3)。由于该文法是 LL(1)文法，因此，根据当前的输入符号和产生式(2)和(3)的 SELECT 集，可以决定接下来应用哪个产生式扩展 T' 。如果当前的输入符号是 “*”，则应用产生式(2)；如果当前的输入符号是 “\$”，则应用产生式(3)。另一方面，从自底向上分析的角度，如果输入中出现了 **digit**，那么从状态 I_0 沿着标记为 **d**（即 **digit** 的首字母，这里为了节省空间）的边进入状态 I_3 。状态 I_3 是一个归约状态，也就是说将栈顶的 **digit** 归约为 F 并退回到状态 I_0 。状态 I_0 遇到刚刚归约出的这个 F 进入状态 I_2 ，并等待归约出紧跟在 F 之后的 M ，如图中第 4 步(b)列所示。 M 对应于空串 ε ，因此，当下一个输入符号为该项目的展望符*或者\$时可以将栈顶的一个空串 ε 归约为 M ，并沿着标记为 M 的边进入状态 I_4 ，等待着识别出紧跟在 M 后面的成分 T' 。根据展望符的定义可知，该项目的展望符集合是当前状态下紧跟在 M 之后的终结符集合，实际上就是 $\text{FISRT}(T')$ 。而 $\text{FISRT}(T')$ 也正是原始文法中 T' 的两个产生式(2)和(3)的 SELECT 集的并集。

综上，在第 4 步，从自顶向下分析的角度，将继续“分析”紧跟在 F 后面的成分 T' 。而从自底向上分析的角度，等待着“识别”出紧跟在 M （实际上就是 F ，因为 M 是空串）后面的成分 T' 。因此，二者的任务实际上是等价的。而从自底向上分析的角度，因为 T' 有产生式(2)和(3)两个候选式，所以为项目 $\mathbf{[T \rightarrow FM \cdot T']}$ 生成两个等价项目 $\mathbf{[T' \rightarrow \cdot * FNT']}$ 和 $\mathbf{[T' \rightarrow \cdot]}$ 。可以看出，前者是移入项目，后者是归约项目，这样就构成了移入-归约冲突。现在，我们要想证明该文法也是 LR 文法，则只需证明这个移入-归约冲突可以被消解即可。事实上，这个移入-归约冲突的本质也是候选式的选择，即，如果采取“移入”*的动作，相当于是选择了产生式(2)；如果采取将栈顶的一个空串“归约”成 T' 的动作，相当于是选择了产生式(3)。既然给定的文法是一个 LL(1)文法，也就是说这个产生式选择上的冲突一定能消解，那么这个移入-归约冲突就可以参照 LL(1)分析中的冲突消解策略进行消解。也就是说，当输入符号是 “*”（即 $\text{SELECT}(2)$ 中的符号）时，则采取移入动作，相当于是选择了产生式(2)；当输入符号是 “\$”（即 $\text{SELECT}(3)$ 中的符号）时，则采取归约动作，相当于是选择了产生式(3)。(b)列中所有蓝色字体表示归约项目的展望符集合。为简洁起见，这里我们忽略掉非归约项目的展望符集合，因为展望符对非归约项目不起任何作用。

这个过程继续下去，可以构造出这个修改后的文法的完整的 LR 自动机，如图 5-46 所示。这实际上也证明了只要基础文法是 LL(1)文法，我们就可以构

造出它的 LR 自动机，从而证明该文法也是 LR 文法。

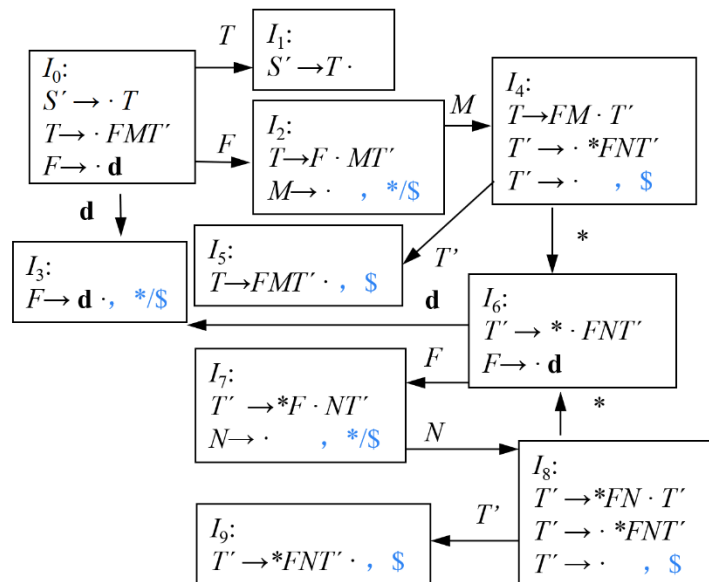


图 5-46 图 5-44 所示 SDT 对应的 SLR 自动机

接下来我们来看如何在 LR 分析过程中实现语义翻译。假设输入的符号串依然是 $3*5$ ，经过词法分析以后得到的 token 序列是 **digit * digit**。

第 1 步：初始时刻，符号栈存放的是栈底标记“\$”，表示符号栈为空。状态栈初始时存放的是初始状态（0 号状态）。输入指针指向第 1 个输入符号 **digit**（词法值为 3），如图 5-47 第 1 步所示。

步骤	输入	分析器																					
1	3 * 5 digit * digit ↑	<table><tr><td>0</td></tr><tr><td>\$</td></tr></table>	0	\$																			
0																							
\$																							
2	3 * 5 digit * digit ↑ ↑	<table><tr><td>0</td><td>3</td></tr><tr><td>\$</td><td>d</td></tr><tr><td></td><td>3</td></tr></table>	0	3	\$	d		3															
0	3																						
\$	d																						
	3																						
3	3 * 5 digit * digit ↑ ↑	<table><tr><td>0</td><td>2</td></tr><tr><td>\$</td><td>F</td></tr><tr><td></td><td>3</td></tr></table>	0	2	\$	F		3															
0	2																						
\$	F																						
	3																						
4	3 * 5 digit * digit ↑ ↑	<table><tr><td>0</td><td>2</td><td>4</td></tr><tr><td>\$</td><td>F</td><td>M</td></tr><tr><td></td><td>3</td><td>T'inh=3</td></tr></table>	0	2	4	\$	F	M		3	T'inh=3												
0	2	4																					
\$	F	M																					
	3	T'inh=3																					
5	3 * 5 digit * digit ↑ ↑ ↑	<table><tr><td>0</td><td>2</td><td>4</td><td>6</td></tr><tr><td>\$</td><td>F</td><td>M</td><td>*</td></tr><tr><td></td><td>3</td><td>T'inh=3</td><td></td></tr></table>	0	2	4	6	\$	F	M	*		3	T'inh=3										
0	2	4	6																				
\$	F	M	*																				
	3	T'inh=3																					
6	3 * 5 digit * digit ↑ ↑ ↑ ↑	<table><tr><td>0</td><td>2</td><td>4</td><td>6</td><td>3</td></tr><tr><td>\$</td><td>F</td><td>M</td><td>*</td><td>d</td></tr><tr><td></td><td>3</td><td>T'inh=3</td><td></td><td>5</td></tr></table>	0	2	4	6	3	\$	F	M	*	d		3	T'inh=3		5						
0	2	4	6	3																			
\$	F	M	*	d																			
	3	T'inh=3		5																			
7	3 * 5 digit * digit ↑ ↑ ↑ ↑	<table><tr><td>0</td><td>2</td><td>4</td><td>6</td><td>7</td></tr><tr><td>\$</td><td>F</td><td>M</td><td>*</td><td>F</td></tr><tr><td></td><td>3</td><td>T'inh=3</td><td></td><td>5</td></tr></table>	0	2	4	6	7	\$	F	M	*	F		3	T'inh=3		5						
0	2	4	6	7																			
\$	F	M	*	F																			
	3	T'inh=3		5																			
8	3 * 5 digit * digit ↑ ↑ ↑ ↑	<table><tr><td>0</td><td>2</td><td>4</td><td>6</td><td>7</td><td>8</td></tr><tr><td>\$</td><td>F</td><td>M</td><td>*</td><td>F</td><td>N</td></tr><tr><td></td><td>3</td><td>T'inh=3</td><td></td><td>5</td><td>T₁'inh=15</td></tr></table>	0	2	4	6	7	8	\$	F	M	*	F	N		3	T'inh=3		5	T ₁ 'inh=15			
0	2	4	6	7	8																		
\$	F	M	*	F	N																		
	3	T'inh=3		5	T ₁ 'inh=15																		
9	3 * 5 digit * digit ↑ ↑ ↑ ↑	<table><tr><td>0</td><td>2</td><td>4</td><td>6</td><td>7</td><td>8</td><td>9</td></tr><tr><td>\$</td><td>F</td><td>M</td><td>*</td><td>F</td><td>N</td><td>T'</td></tr><tr><td></td><td>3</td><td>T'inh=3</td><td></td><td>5</td><td>T₁'inh=15</td><td>syn=15</td></tr></table>	0	2	4	6	7	8	9	\$	F	M	*	F	N	T'		3	T'inh=3		5	T ₁ 'inh=15	syn=15
0	2	4	6	7	8	9																	
\$	F	M	*	F	N	T'																	
	3	T'inh=3		5	T ₁ 'inh=15	syn=15																	
10	3 * 5 digit * digit ↑ ↑ ↑ ↑	<table><tr><td>0</td><td>2</td><td>4</td><td>5</td></tr><tr><td>\$</td><td>F</td><td>M</td><td>T'</td></tr><tr><td></td><td>3</td><td>T'inh=3</td><td>syn=15</td></tr></table>	0	2	4	5	\$	F	M	T'		3	T'inh=3	syn=15									
0	2	4	5																				
\$	F	M	T'																				
	3	T'inh=3	syn=15																				
11	3 * 5 digit * digit ↑ ↑ ↑ ↑	<table><tr><td>0</td><td>1</td></tr><tr><td>\$</td><td>T</td></tr><tr><td></td><td>val=15</td></tr></table>	0	1	\$	T		val=15															
0	1																						
\$	T																						
	val=15																						

图 5-47 $3*5$ 的自底向上翻译过程

第 2 步：根据图 5-46，初始状态 I_0 遇到 d 后， d 带着它的词法值 3 移入栈中，并转入状态 I_3 ，输入指针指向下一输入符号 “*”，如图 5-47 第 2 步所示。

第 3 步：此时栈顶的 I_3 是一个归约状态，于是应用产生式(4)将栈顶的 d 归约为 F ，同时执行对应的语义动作。该语义动作的任务是计算即将进栈的 F 的综合属性 val 的值。这样，栈顶的 d 带着与之对应的状态 I_3 出栈，露出状态 I_0 。 F 带着计算出来的综合属性 val 的值 3 进栈。根据图 5-46，状态 I_0 遇到归约出的 F 以后转入状态 I_2 ，如图 5-47 第 3 步所示。

第 4 步：根据图 5-46，状态 I_2 遇到 “*” 的时候执行归约动作，把栈顶的一个空串归约为 M ，相当于没有符号出栈， M 进栈。归约的时候执行对应的语义动作，该语义动作的任务是计算即将进栈的 M 的综合属性 $T'inh$ 的值。从属性的名字上可以看出，它实际是要计算 T 的继承属性 inh 的值（因为标记非终结符 M 位于 T 之前），计算的过程中需要用到 F 的 val 属性的值， F 此时就位于栈顶（因为 M 在产生式中紧跟在 F 之后。 M 即将进栈，那么 F 当前就在栈顶），也就是位置 $stack[top]$ 。因此，该语义动作就是把栈顶 $stack[top]$ 里面存放的 F 的 val 值赋给即将进栈的 M ， M 将位于 $stack[top+1]$ 。于是，产生式(1.2)中的语义动作对应的具体栈操作为：

$$stack[top+1].T'inh = stack[top].val$$

$$top = top+1$$

这样， M 带着计算出来的综合属性 $T'inh$ 进栈。栈顶的状态 I_2 遇到归约出的 M 以后转入状态 I_4 ，如图 5-47 第 4 步所示。

第 5 步：根据图 5-46，状态 I_4 遇到 “*” 采取移入动作，并转入状态 I_6 ，输入指针指向下一输入符号 **digit**（词法值为 5），如图 5-47 第 5 步所示。

第 6 步：根据图 5-46，状态 I_6 遇到 d 后， d 带着它的词法值 5 移入栈中，并转入状态 I_3 ，输入指针指向下一输入符号 “\$”，如图 5-47 第 6 步所示。

第 7 步：与第 3 步类似，此时栈顶的 I_3 是一个归约状态，于是应用产生式(4)将栈顶的 d 归约为 F ，同时执行对应的语义动作。该语义动作的任务是计算即将进栈的 F 的综合属性 val 的值。这样，栈顶的 d 带着与之对应的状态 I_3 出栈，露出状态 I_6 。 F 带着计算出来的综合属性 val 的值 5 进栈。根据图 5-46，状态 I_6 遇到归约出的 F 以后转入状态 I_7 ，如图 5-47 第 7 步所示。

第 8 步：根据图 5-46，状态 I_7 遇到 “\$” 的时候执行归约动作，把栈顶的一个空串归约为 N ，相当于没有符号出栈， N 进栈。归约的时候执行对应的语义动作，该语义动作的任务是计算即将进栈的 N 的综合属性 $T'_l inh$ 的值。从属性的名字上可以看出，它实际是要计算 T_l 的继承属性 inh 的值（因为标记非终结

符 N 位于 T_1 之前），计算的过程中需要用到 T 的 inh 属性值和 F 的 val 属性值。 T 和 F 虽然没在 N 的空产生式右部中出现，但是这两个属性值其实已经在栈里了。事实上， N 进栈以后要与产生式(2)中位于它左边的 F 、“*”以及后续识别出来的 T_1 放在一起，归结成 T' 。也就是说，*、 F 、 N 、 T_1 出栈后， T' 将进栈并位于“*”当前所处的位置。而 T 的继承属性 inh 的值其实在即将识别 T' 之前（比如说在产生式(1)的 M 中）已经计算出来了。因此，当前栈顶 $stack[top]$ 处是产生式(2)中位于 N 左边的 F ， $stack[top-1]$ 处是“*”，而 $stack[top-2]$ 处就是用于计算 T 的继承属性 inh 的记录， $T.inh$ 的值就存放在这里。于是，产生式(2.2)中的语义动作对应的具体栈操作为：

$$\begin{aligned} stack[top+1].T.inh &= stack[top-2].T.inh \times stack[top].val; \\ top &= top+1; \end{aligned}$$

这样， N 带着计算出来的综合属性 $T.inh$ 的值进栈。栈顶的状态 I_7 遇到归约出的 N 以后转入状态 I_8 ，如图 5-47 第 8 步所示。

第 9 步：根据图 5-46，状态 I_8 遇到“\$”的时候执行归约动作，把栈顶的一个空串归约为 T' ，相当于没有符号出栈， T' 进栈。归约的时候执行对应的语义动作，该语义动作的任务是计算即将进栈的 T' 的综合属性 syn 的值。计算的过程中需要用到 T 的继承属性 inh 的值。而 $T.inh$ 的值其实当前就位于栈顶记录 $stack[top]$ 中，因为根据 L-SDD 的翻译方案设计原则，用于计算非终结符的继承属性的记录要紧挨在非终结符左侧的位置摆放。于是，产生式(3)中的语义动作对应的具体栈操作为：

$$\begin{aligned} stack[top+1].syn &= stack[top].T.inh; \\ top &= top+1; \end{aligned}$$

这样， T' 带着计算出来的综合属性 syn 的值进栈。栈顶的状态 I_8 遇到归约出的 T' 以后转入状态 I_9 ，如图 5-47 第 9 步所示。

第 10 步：此时栈顶的 I_9 是一个归约状态。于是应用产生式(2)将栈顶的*、 F 、 N 、 T_1 归约为 T' ，同时执行对应的语义动作。该语义动作的任务是计算即将进栈的 T' 的综合属性 syn 的值。这样，栈顶的*、 F 、 N 、 T_1 带着与之对应的状态 I_6 、 I_7 、 I_8 、 I_9 出栈，露出状态 I_4 。 T' 带着计算出来的综合属性 syn 的值 15 进栈。根据图 5-46，状态 I_4 遇到归约出的 T' 以后转入状态 I_5 ，如图 5-47 第 10 步所示。

第 11 步：此时栈顶的 I_5 是一个归约状态。于是应用产生式(1)将栈顶的 F 、 M 、 T' 归约为 T ，同时执行对应的语义动作。该语义动作的任务是计算即将进栈的 T 的综合属性 val 的值。这样，栈顶的 F 、 M 、 T' 带着与之对应的状态 I_2 、

I_4 、 I_5 出栈，露出状态 I_0 。 T 带着计算出来的综合属性 val 的值 15 进栈。根据图 5-46，状态 I_0 遇到归约出的 T 以后转入状态 I_1 ，即自动机唯一的终止状态，如图 5-47 第 11 步所示，表示成功完成语义翻译。

将图 5-44 所示的 SDT 中的各个语义动作改写为具体可执行的栈操作后的结果如图 5-48 所示。

(1.1') $T \rightarrow F M T' \{stack[top-2].val = stack[top].syn; top = top-2;\}$

(1.2') $M \rightarrow \varepsilon \{stack[top+1].T'inh = stack[top].val; top = top+1;\}$

(2.1') $T' \rightarrow * F N T'_1 \{stack[top-3].syn = stack[top].syn; top = top-3;\}$

(2.3') $N \rightarrow \varepsilon \{stack[top+1].T'inh = stack[top-2].T'inh \times stack[top].val; top = top+1;\}$

(3') $T' \rightarrow \varepsilon \{stack[top+1].syn = stack[top].T'inh; top = top+1;\}$

(4') $F \rightarrow \text{digit} \{stack[top].val = stack[top].lexval;\}$

图 5-48 将图 5-44 所示的 SDT 中的各个语义动作改写为具体可执行的栈操作

□

综上，给定一个以 LL 文法为基础的 L-SDD，可以修改这个文法，并在 LR 语法分析过程中计算这个新文法之上的 SDD。具体方法如下：

(1) 首先构造 SDT，在各个非终结符之前放置语义动作来计算它的继承属性，并在产生式后端放置语义动作计算综合属性。

(2) 对每个内嵌的语义动作，向文法中引入一个标记非终结符来替换它。每个这样的位置都有一个不同的标记，并且对于任意一个标记 M 都有一个产生式 $M \rightarrow \varepsilon$ 。

(3) 如果标记非终结符 M 在某个产生式 $A \rightarrow \alpha \{a\} \beta$ 中替换了语义动作 a ，对 a 进行修改得到 a' ，并且将 a' 关联到 $M \rightarrow \varepsilon$ 上。动作 a' 按照 a 中的方法计算各个属性，但是将计算得到的这些属性作为 M 的综合属性。

5.6 本章小结

本章系统阐述了将语法分析与语义翻译相结合的语法制导翻译技术。首先，介绍了其形式化基础语法制导定义（SDD），明确了文法符号的语义属性及其

计算规则，并讨论了 SDD 的求值顺序问题。进而，重点讲解了两类实用的属性定义：适用于自底向上分析的 S-属性定义，以及适用于自顶向下分析的 L-属性定义。在此基础上，介绍了可执行的语法制导翻译方案（SDT），并详细探讨了基于 S-SDD 的自底向上翻译实现，以及 L-SDD 在递归下降、LL(1)分析等自顶向下方法和特殊技术支持的自底向上方法中的翻译实现机制。本章内容为理解语义信息如何沿语法结构流动与计算，以及构建各类翻译器提供了系统的理论与方法基础。

本章要点如图 5-49 所示。

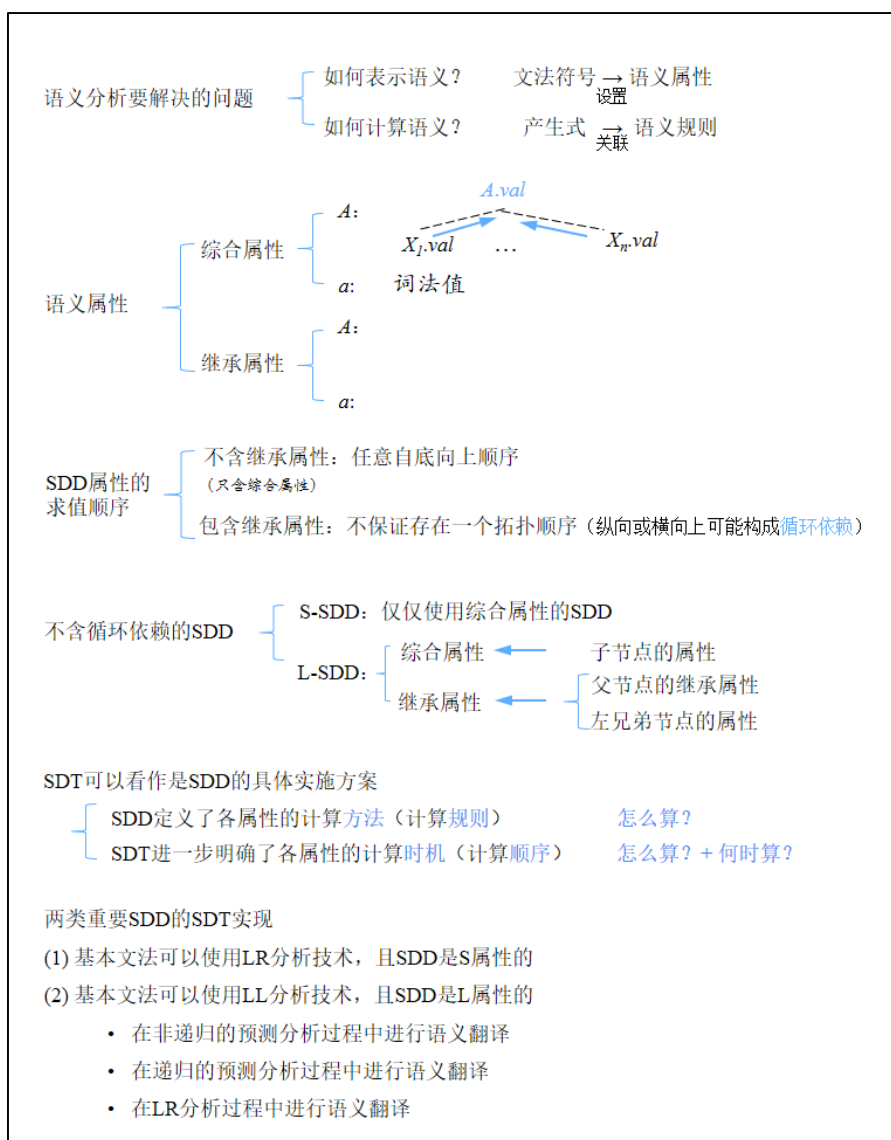


图 5-49 第 5 章要点

第6章 中间代码生成

本章是对第 5 章（语法制导翻译）内容的延伸。这两章都是在讲语义翻译，但是侧重点不同。

在第 5 章中，我们系统学习了语法制导翻译的核心实现机制，掌握了如何将形式化的语义规则嵌入到语法分析栈或递归下降过程中，从而构建出能够执行翻译动作的程序框架。这好比我们深入理解了翻译引擎的精密构造原理与操作方法。

然而，掌握引擎的构建与操作，尚不足以完成一项完整的翻译任务。我们还需要知道，对于程序中形形色色的语法结构——从变量声明、算术赋值，到复杂的条件分支、循环控制，乃至过程调用——具体应该生成什么样的中间代码，以及如何设计对应的语义动作来生成这些代码。这正是本章的核心目标。

可见，上一章侧重讲解 SDT 的实现机制，而本章侧重于如何设计具体语法结构的 SDT。通过本章的学习，我们能够将抽象的翻译理论与具体的语言特性相结合，设计出关键程序结构的语义动作，从而完成从源程序语法树到可被后续优化与代码生成阶段处理的中间表示这一实质性飞跃。这标志着我们正式从“理解结构”进入了“赋予含义并转换表达”的编译新阶段。

6.1 声明语句的翻译

要想设计声明语句的翻译方案，首先需要明确翻译的任务是什么。声明语句翻译的任务主要任务是：收集标识符的种属（**kind**）、类型（**type**）等属性信息，并为每一个名字分配一个相对地址。名字的类型和相对地址等信息保存在相应的符号表记录中。种属（**kind**）包括简单变量、数组、指针、过程、结构体等。类型（**type**）包括整型、实型、字符型、布尔型等等。

根据一个名字的类型，编译器可以确定这个名字在运行时刻需要多大的存储空间。在这一节中，我们将考虑在程序中声明的名字的类型及存储空间布局问题。一个过程调用或对象的实际存储空间是在运行时刻（当该过程被调用或该对象被创建时）进行分配的。然而，当我们在编译时刻检查局部声明时，可以进行相对地址（**relative address**）的布局，一个名字或某个数据结构分量的相对地址是指它相对于数据区域开始位置的偏移量（**offset**）。

种属和类型这两类属性将整合成一个统一的类型表达式（**type expression**）。

6.1.1 类型表达式

类型自身也有结构，我们使用类型表达式来表示这种结构。类型表达式可能是基本类型，也可能通过把被称为类型构造符的运算符作用于类型表达式而得到。基本类型的集合和类型构造符根据被分析的具体语言而定。

我们将使用如下的类型表达式的定义：

(1) 简单变量的类型表达式就是这些基本类型本身。一种语言的基本类型通常包括 integer、real、char、boolean 以及 void 等。

(2) 将数组类型构造符 array 作用于一个数字 n 和一个类型表达式 T 可以构成一个数组类型表达式 $\text{array}(n, T)$ ，其中 n 表示数组中元素的个数， T 表示数组中每个元素的类型。

(3) 将指针类型构造符 pointer 作用于一个类型表达式可以构成一个指针类型表达式 $\text{pointer}(T)$ 。

(4) 过程的类型表达式形如 $T_1 \times T_2 \times \dots \times T_n \rightarrow R$ 。 T_1 、 T_2 、 $\dots$ 、 T_n 是过程的各个参数的类型表达式， R 是返回值的类型表达式。

(5) 结构体的类型表达式形如 $\text{record}((N_1 \times T_1) \times \dots \times (N_n \times T_n))$ 。 record 是结构体构造符。 n 是字段的个数。每个字段包括两部分， N_i 是第 i 个字段的名称， T_i 是 N_i 的类型表达式。

从变量的类型表达式可以知道该变量在运行时刻所需的存储单元数量，称为类型的宽度（width）。在编译时刻，我们可以根据类型的宽度为每个名字分配一个相对地址。

本章重点介绍简单变量、数组和指针声明的翻译方案，过程和结构体声明的翻译方案将在第 7 章介绍。

6.1.2 变量声明语句的翻译

图 6-1 所示的 SDT 处理形如 $T \text{ id}$ 的变量声明序列。

- (1) $P \rightarrow \{ \text{offset} = 0 \} D$
- (2) $D \rightarrow T \text{ id}; \{ \text{enter}(\text{id.lexeme}, T.\text{type}, \text{offset}); \text{offset} = \text{offset} + T.\text{width}; \} D$
- (3) $D \rightarrow \epsilon$

- (4) $T \rightarrow B \{ t = B.type; w = B.width; \} C \{ T.type = C.type; T.width = C.width; \}$

(5) $T \rightarrow \uparrow T_1 \{ T.type = pointer(T_1.type); T.width = 4; \}$

(6) $B \rightarrow \text{int} \{ B.type = \text{integer}; B.width = 4; \}$

(7) $B \rightarrow \text{float} \{ B.type = \text{real}; B.width = 8; \}$

(8) $C \rightarrow \varepsilon \{ C.type = t; C.width = w; \}$

(9) $C \rightarrow [\text{num}] C_1 \{ C.type = \text{array}(\text{num.val}, C_1.type); C.width = \text{num.val} * C_1.width; \}$

图 6-1 变量声明序列的 SDT

先来看一下基础文法（图 6-1 中的黑色字体）。其中 P 表示程序， D 表示声明的序列。每个声明由一个类型表达式 T 连接上一个标识符 **id** 和一个分号构成。根据产生式(4)，一个类型表达式 T 可以由一个 B 连接上一个 C 构成。 B 表示基本类型， C 用来生成一个 **[num]** 序列，每个 **num** 对应着 n 维数组的某一维的维长信息。当然， C 也可以是个空串。当产生式(4)中 B 后面的 C 为空串时表示 T 生成的是一个基本类型；否则， T 生成的是一个数组类型。

为文法符号 T 和 B 设置属性

前面提到，翻译的主要任务是分析标识符的类型，并根据类型表达式进行地址分配。为此，为 T 设置综合属性 $type$ ，表示 T 对应的类型表达式。 T 的 $type$ 依赖于 B 所表示的基本类型，因此为 B 也设置综合属性 $type$ 。 $B.type$ 很好计算，根据子节点。如果 B 产生式右部是关键字 **int**，则 B 表示整型，其 $type$ 属性值为 **integer**；因此在产生式(6)中设置语义动作

$$B.type = \text{integer} \quad (6.1)$$

如果 B 产生式右部是关键字 **float**，则 B 表示实型，其 $type$ 属性值为 **real**，因此产生式(7)中设置语义动作

$$B.type = \text{real} \quad (6.2)$$

另一方面，由于地址需要做全局统一分配，因此没有为各个文法符号设置单独的属性，而是设置了一个全局变量 $offset$ ，用来跟踪下一个可用的相对地址。前后连续声明的两个名字的 $offset$ 值之间存在如下关系：第 $i+1$ 个标识符的偏移地址等于第 i 个标识符的偏移地址加上第 i 个标识符所需占用的存储空间的大小 $T_i.width$ ，即

$$offset_{i+1} = offset_i + T_i.width \quad (6.3)$$

为此，我们为与类型有关的非终结符 T 和 B 设置 $width$ 属性，用来表示该类型

所需的存储空间的大小。存储空间的大小与标识符的类型表达式有关。如果 B 是整型，我们假设 1 个整型占用 4 个字节，因此产生式(6)设置语义动作

$$B.width = 4 \quad (6.4)$$

另外，我们假设 1 个实型占用 8 个字节，因此产生式(7)设置语义动作

$$B.width = 8 \quad (6.5)$$

在符号表中为名字建立记录

在开始对整个程序进行分析时，需要首先为全局变量 $offset$ 赋初值 0，也就是说从相对地址 0 开始分配。因此在产生式(1)的右部 D 之前嵌入语义动作

$$offset = 0 \quad (6.6)$$

每识别出一个 id ，意味着分析出该 id 的类型表达式，在符号表中为该 id 建立一条记录，将相关的属性信息登记到记录中的对应字段中。其相对地址被设置为 $offset$ 的当前值。随后，更新 $offset$ 的值，将 id 的类型宽度加到 $offset$ 上。因此，在产生式(2)的“;”与 D 之间嵌入语义动作

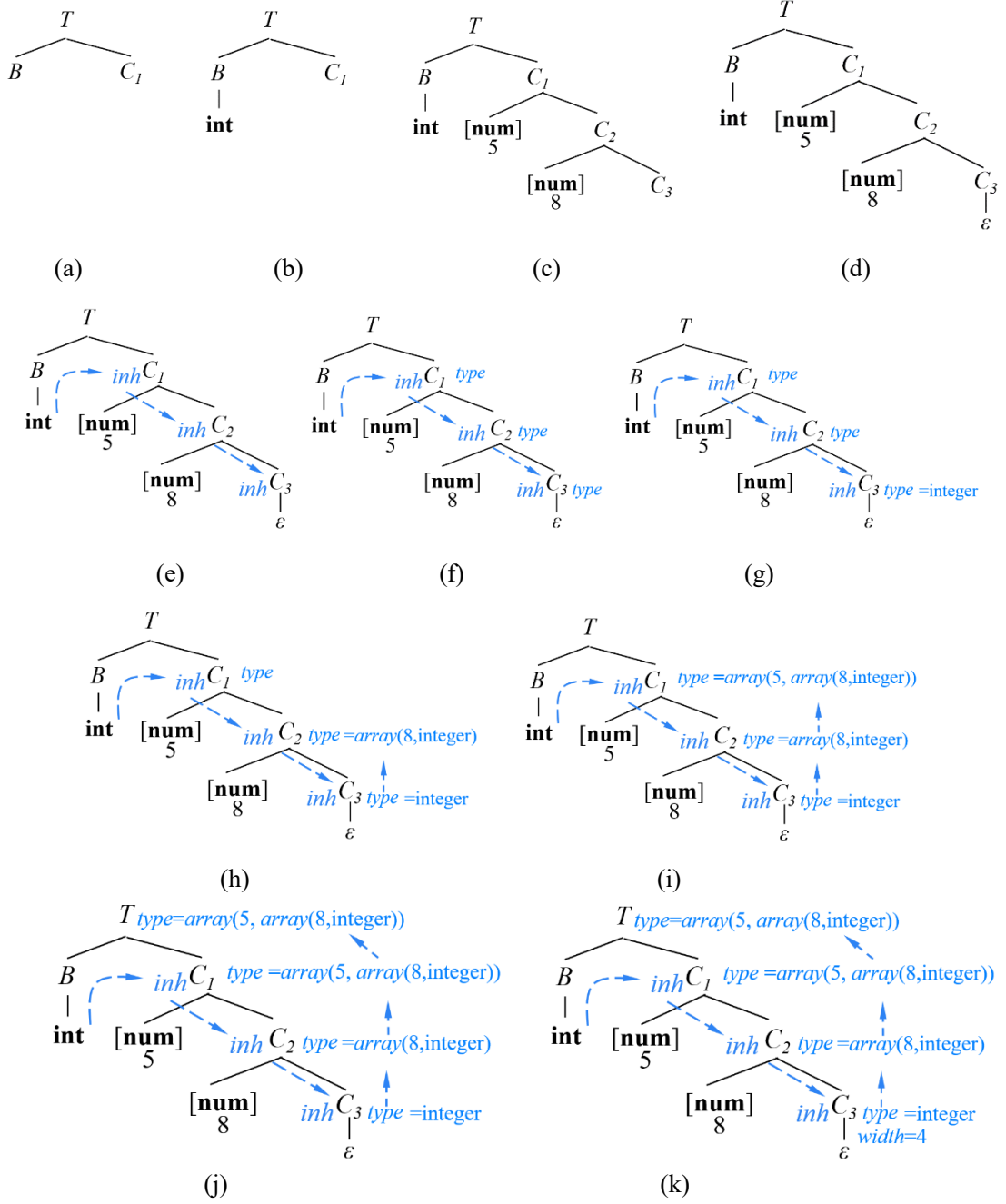
$$enter(id.lexeme, T.type, offset) \quad (6.7)$$

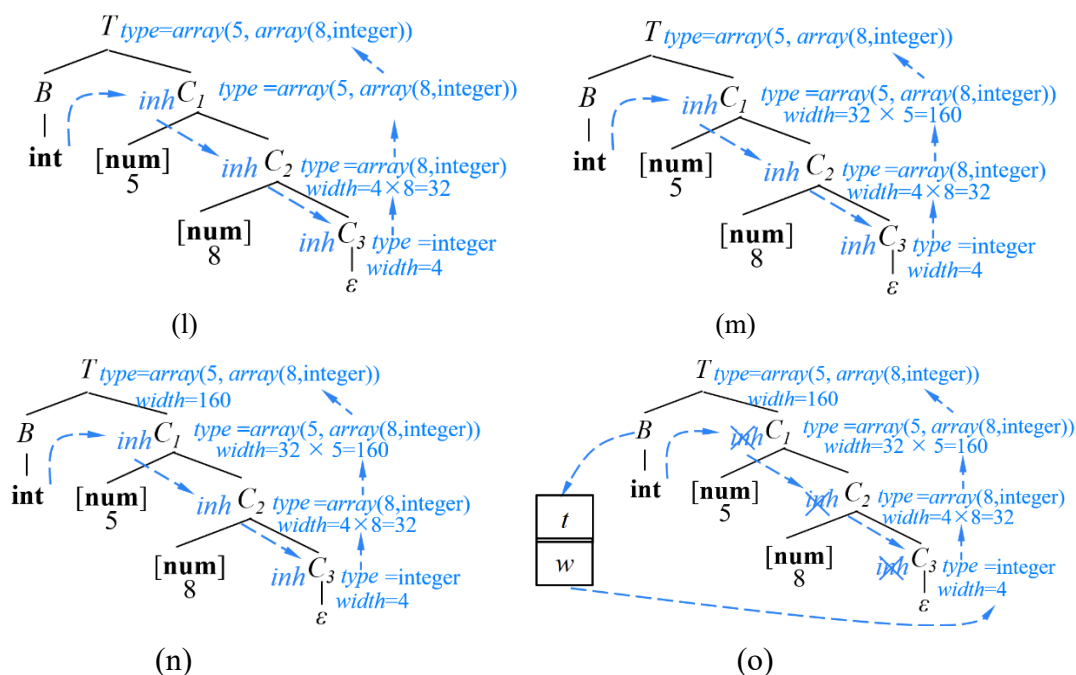
$$offset = offset + T.width \quad (6.8)$$

其中 $enter(id.lexeme, T.type, offset)$ 函数在符号表中为 $id.lexeme$ 建立一条记录，将 $T.type$ 填入类型字段，将 $offset$ 填入地址字段。然后通过式(6.8)更新 $offset$ 的值。

T 的属性的计算

接下来讨论 $T.type$ 的计算。首先看一个例子。假设有一个表示类型的程序片段 $int[5][8]$ ，对应着文法符号 T 。在自顶向下的分析过程中，首先根据 T 的两个产生式(4)和(5)的 SELECT 集以及当前的输入符号 int ，决定应用产生式(4) $T \rightarrow BC$ 生成基本类型 B ，如图(a)所示。



图 6-2 T 的属性的计算

对于当前句型的最左非终结符 B ，根据 B 的两个产生式(6)和(7)的 SELECT 集以及当前的输入符号 int ，决定应用产生式(6)在分析树中建立节点 int ，如图(b)所示。

接下来，对于当前句型的最左非终结符 C ，根据 C 的两个产生式(8)和(9)的 SELECT 集以及当前输入符号 “[”，决定应用产生式(9)，生成第 1 个 $[num]$ ，该 num 的词法值为 5。类似的，再次应用产生式(9)，生成第 2 个 $[num]$ ，该 num 的词法值为 8，如图(c)所示。

当所有 $[num]$ 都处理完毕后，应用 C 的空产生式(8)结束，如图(d)所示。当应用 C 的空产生式(8)时，意味着所有的 num 都处理完了。而我们最终需要得到的类型表达式为 $array(5, array(8, integer))$ 。因此，首先需要将最后一个 num （也就是本例中的 8）同基本类型 int 结合起来，生成最内层的一维数组的类型表达式 $array(8, int)$ 。但是最后一个 num 所在的产生式(9)中访问不了 B 的 $type$ 属性值 $integer$ ，因此需要设法把 B 的 $type$ 属性值传递到产生式 $C \rightarrow \epsilon$ 对应的节点。显然可以为 C 设置继承属性 inh 将 B 的 $type$ 属性值在分析树中从左向右、从上向下传递，如图(e)所示。

此时，分析树中最下方的 C_3 对应着基本变量（可以视为 0 维数组），其父节点 C_2 对应一维数组， C_1 对应二维数组。因此，为 C 设置一个 $type$ 属性表示各维数组的类型表达式，如图(f)所示。

C_3 的 *type* 属性值等于它的继承属性 *inh* 的值，即基本变量类型 *int*，如图(g)所示。 C_2 是一个一维数组，它的 *type* 属性值依赖于它的两个儿子。其中元素个数的值依赖于其左儿子 *num* 的词法值，元素类型表达式依赖于其右儿子 C_3 的类型表达式 *type*，即图 6-1 中产生式(9)的语义动作

$$C.type = \text{array}(\text{num.val}, C_1.type) \quad (6.9)$$

计算结果如图(h)所示。类似地，应用式(6.9)所示的语义动作计算二维数组 C_1 的 *type* 属性，结果如图(i)所示。

此时， $C_1.type$ 中实际上已经形成了输入词串 *int*[5][8]对应的类型表达式，我们通过产生式(4)中的语义动作

$$T.type = C.type \quad (6.10)$$

将它复制给 *T.type*，结果如图(j)所示。

分析出各维数组的类型表达式后，就可以计算各维数组的宽度信息了。为此，我们为 C 也设置综合属性 *width*。位于分析树最底部的 C_3 的 *width* 属性依赖于它的 *type* 属性的值。根据我们前面的假设，1 个整型 (*int*) 变量占用 4 个字节，因此，

$$C_3.width = 4 \quad (6.11)$$

如图(k)所示。

C_2 是一个一维数组，它的宽度 (*width* 属性) 取决于它一共有多少个元素，以及每个元素的宽度。其中元素个数的值依赖于其左儿子 *num* 的词法值，每个元素的宽度依赖于其右儿子 C_3 的 *width* 属性，因此，通过图 6-1 中产生式(9)的语义动作

$$C.width = \text{num.val} * C_1.width \quad (6.12)$$

可以计算出其结果，如图(l)所示。类似地，应用式(6.12)所示的语义动作计算二维数组 C_1 的 *width* 属性，结果如图(m)所示。

此时， $C_1.width$ 中实际上已经计算出了 *int*[5][8]的宽度，我们通过产生式(4)中的语义动作

$$T.width = C.width \quad (6.13)$$

将它赋给 *T.width*，如图(n)所示。

注意：这里之所以采用综合属性来数组的累积维度的信息是因为整个类型表达式的是采用的右结合的形式。只有读入最后一个 *num*，才知道一维数组中元素的个数，才可以计算一维数组的宽度，然后依次计算二维、三维等数组的宽度。而读入最后一个 *num* 时，也就是到达分析树的底部，这时候再沿着由这些综合属性构成的链依次构造一维、二维、三维等数组的类型表达式。

回忆在 5.2.2 节关于“项 (term)”的例子中，采用继承属性积累各个因子 F 的乘积，生成整个项的值。那是因为因子的乘积可以采用左结合的方式计算累加值。当然实际上因子的乘积也可以像这里这个例子一样，采用综合属性以右结合的方式计算各因子 F 的累积值。5.2.2 节的例子之所以采用继承属性累积主要是为了讲解继承属性的用法。

设置临时变量 t 和 w

前面提到，可以采用继承属性传递 B 的 $type$ 值。事实上，这个继承属性值自上而下传递的过程中只是简单的复制操作，并没有积累进来新的信息，因此，没有必要一遍遍地重复这个同样的复制，影响分析的效率。于是，在分析树中的节点 B 处，也就是说，在应用产生式(4)分析完 B 以后，把 B 的 $type$ 值赋给临时变量 t ，即

$$t = B.type \quad (6.14)$$

然后在分析空产生式 $C \rightarrow \varepsilon$ 时，将这个临时变量的值赋给 C 的 $type$ 属性，即在产生式(8)中嵌入语义动作

$$C.type = t \quad (6.15)$$

对于属性 $width$ 也是采取类似的处理方法。即在产生式(4)中设置语义动作

$$w = B.width \quad (6.16)$$

在产生式(8)中设置语义动作

$$C.width = w \quad (6.17)$$

这样就可以删除继承属性 inh 了，如图(o)所示。当数组维度比较高时（例如 n 维），这种处理方式可以显著提高效率因为可以。

对于产生式(5) $T \rightarrow \uparrow T_i$ ，根据它的语义动作，将 T 的类型定义为 T_i 类型的指针。另外，我们假设一个指针占用 4 个字节，因此，宽度定义为 4。

将各维数组的宽度信息存储在符号表中

在上方这个例子中，对于输入的源程序片段 `int[5][8]`，如果将其中的关键信息提取出来，形式为 `int...5...8`。而对于语义分析后得到的类型表达式 `type=array(5, array(8,int))`，如果将其中的关键信息提取出来，形式为 `5...8...int`。这样的话，似乎没必要用上述这么复杂翻译方案去构造类型表达式，而是可以采用某种更简单的方法直接将 `int` 追加到 `5...8` 后面。但事实上，构造各维数组的类型表达式并不是主要目的，主要目的是要求出各维数组的宽度，进行地址分配。这里之所以设置类型表达式属性只是为了直观地表示数组宽度的计算过程。事实上，这一节介绍的是数组的声明，在后面章节中会介绍数组元素的引用，其中会涉及到数组元素的寻址。例如，对于赋值语句 `b=a[3][4]`，在执行时

需要首先计算出数组元素 $a[3][4]$ 存放的地址。具体计算公式为

$$\text{addr}(a[3][4]) = a + 3 \times w_1 + 4 \times w_0 = a + 3 \times 32 + 4 \times 4 = a + 112 \quad (6.18)$$

其中 a 表示该数组的基地址（该基地址在分析完 a 的声明语句后登记到符号表中。当然，符号表中不可能存放每一个数组元素的地址，而只是存放数组的基地址。程序执行时具体引用哪个元素时再去应用地址计算公式来计算其存放地址）。 w_0 是 0 维数组（即基本变量）的宽度， w_1 是一维数组的宽度。类似的，如果是 n 维数组，数组元素寻址时还要用到 $n-1$ 、 $n-2$ 、……、0 维数组的宽度。因此我们需要把每一维数组的宽度信息都保存在符号表中。这样在程序中引用数组中的任何一个元素时，可以直接从符号表中读取这些宽度信息，进行数组元素地址的计算。

将各维数组的宽度存储在符号表中，这将涉及到符号表的组织问题。不同种属的名字所需存放的属性信息在数量上的差异会造成符号表空间的浪费。

例 6.1 假设有如下程序片段：

```
int abc;
int i;
int myarray[3][5][8];
```

其中 abc 和 i 都是简单变量，所以符号表中只需存储它们的种属、类型和地址等信息就可以了。但是对于数组 $myarray$ ，除了以上信息以外，还需要额外存储数组的维数以及各个维度的宽度信息。再如，对于过程的名字，还需要额外存储参数的个数以及各个参数的类型信息。 □

那么如何设计记录的数据结构呢？一种可行的方案是将符号表中的属性分割成基本属性和扩展属性两部分：基本属性是各种符号均具有的属性（如种属、类型、地址等），基本属性直接存放在符号表中。扩展属性则用一段动态申请的内存存放。例如，对于数组，扩展属性存放在称为内情向量的表中。因此，在符号表的基本属性部分要设置一个指针项，对具有扩展属性的符号对应的表项，它指向扩展属性的指针；对于没有扩展属性的符号对应的表项，则将该指针项置空。例如，例 6.1 中程序片段对应的符号表组织形式如图 6-3 所示。

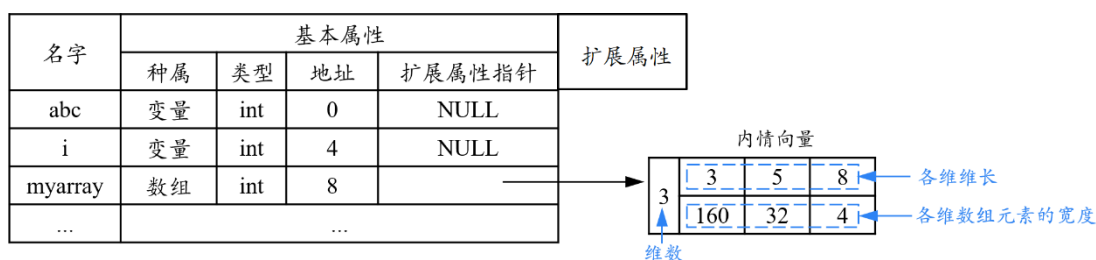


图 6-3 例 6.1 中程序片段对应的符号表组织形式

6.2 赋值语句的翻译

这一部分主要包含两方面内容：简单赋值语句的翻译和数组引用的翻译。与声明语句类似，赋值语句翻译的难点也在数组的翻译。

6.2.1 赋值语句的 SDD

赋值语句的基本文法如下：

- (1) $S \rightarrow \text{id} = E;$
- (2) $E \rightarrow E_1 + E_2$
- (3) $E \rightarrow E_1 * E_2$
- (4) $E \rightarrow -E_1$
- (5) $E \rightarrow (E_1)$
- (6) $E \rightarrow \text{id}$ (6.19)

产生式(2)~(6)是表达式对应的产生式。产生式(1)将一个表达式的值赋给一个变量 id 。赋值语句翻译的主要任务是生成对表达式求值的三地址码。

例 6.2 假设输入的源程序片段为：

$$x = (a + b) * c; \quad (6.20)$$

则应该生成如下的三地址码

$$t_1 = a + b \quad (6.21)$$

$$t_2 = t_1 * c \quad (6.22)$$

$$x = t_2 \quad (6.23)$$

首先通过式(6.21)计算 $a+b$ 的值，计算结果放到临时变量 t_1 中。然后通过式(6.22)将 t_1 与 c 相乘，计算的结果放在临时变量 t_2 中。此时， t_2 中存放的就是(6.20)中赋值号右边的表达式的值。接下来通过式(6.23)将该值赋给变量 x 。

注意，式(6.22)和(6.23)不能合并成一条三地址指令

$$x = t_1 * c \quad (6.24)$$

因为 x 和 “*” 分别处于两个不同个产生式中，所以不能直接生成一条同时包含这两个符号的三地址指令。□

赋值语句的 SDT 如图 6-4 所示。正如我们前面提到的，赋值语句翻译的主要任务是生成对表达式求值的三地址码，因此我们为 S 和 E 分别设置综合属性 *code*，用来表示这两个文法符号对应的三地址码。表达式 E 的值需要通过一段

三地址码来计算，赋值语句 S 涉及到将一个表达式的值赋给一个变量。

(1) $S \rightarrow id = E;$	{	$p = lookup(id.lexeme);$ $if\ p == nil\ then\ error();$ $S.code = E.code \parallel gen(p '=' E.addr);$
	}	
(2) $E \rightarrow E_1 + E_2$	{	$E.addr = newtemp();$ $E.code = E_1.code \parallel E_2.code \parallel gen(E.addr '=' E_1.addr '+' E_2.addr);$
	}	
(3) $E \rightarrow E_1 * E_2$	{	$E.addr = newtemp();$ $E.code = E_1.code \parallel E_2.code \parallel gen(E.addr '=' E_1.addr '*' E_2.addr);$
	}	
(4) $E \rightarrow -E_1$	{	$E.addr = newtemp();$ $E.code = E_1.code \parallel gen(E.addr '=' 'uminus' E_1.addr);$
	}	
(5) $E \rightarrow (E_1)$	{	$E.addr = E_1.addr;$ $E.code = E_1.code;$
	}	
(6) $E \rightarrow id$	{	$E.addr = lookup(id.lexeme);$ $if\ E.addr == nil\ then\ error();$ $E.code = '';$
	}	

图 6-4 赋值语句的 SDT

表达式的值计算出来以后需要有一个地方存放。为此，我们为文法符号 E 设置综合属性 $addr$ ，表示该表达式的值的存放地址。与产生式(2)~(6)关联的语义动作的任务就是计算 E 的两个综合属性 $E.addr$ 和 $E.code$ 的值。

对于产生式(6)，由于该表达式 E 是由一个标识符 id 构成，这个 id 实际上是一个基本变量的名字。因此，基本变量 id 的值就是这个表达式 E 的值，基本变量 id 的值的存放地址就是该表达式 E 的值的存放地址。那么 id 的值的存放地址是哪里呢？事实上，我们之所以在这里能够引用这个标识符 id ，那么原则上之前肯定已经在某个声明语句声明过它。那么在分析该 id 的声明语句的时候，会给它分配相对地址，并且登记在符号表中。因此，我们以标识符 id 的词法值 $lexeme$ ，即标识符的名字作为参数调用 $lookup()$ 函数去查符号表，函数的返回值就是该标识符 id 在符号表中的入口地址，也就是符号表中标识符 id 对应的那条记录。由此，我们可以读取到该标识符 id 的所有属性信息，当然也包括它的相对地址，我们把它付给 $E.addr$ 。如果函数的返回值为空，则说明符号表中没有该标识符 id 对应的记录，也就是说我们检测到了一个“变量未经声明就引用”的语义错误，于是调用 $error()$ 函数报错。可见，我们可以通过设置特定的语义动作进行语义错误检测。因为产生式(1)中不涉及任何运算，因此不需要生成三地址指令， $E.code$ 值为空串。

对于产生式(2), $E.addr$ 为表达式 E 的值的存放地址, 于是我们调用 $newtemp()$ 函数, 申请一个新的临时变量 t 来存放这个表达式的值。该函数的返回值即为该临时变量 t 的地址 (这里用临时变量的名字 t 表示该地址)。 $E.code$ 的代码结构布局如图 6-5 所示, 共由三部分组成。首先是 $E_1.code$ 和 $E_2.code$, 其中 $E_1.code$ 用来计算表达式 E_1 的值, $E_2.code$ 用来计算表达式 E_2 的值。 E_1 和 E_2 的值计算出来以后分别存放在 $E_1.addr$ 和 $E_2.addr$ 中。接下来调用 $gen()$ 函数生成一条三地址指令 $E.addr \leftarrow E_1.addr + E_2.addr$ 。该指令表示从 $E_1.addr$ 和 $E_2.addr$ 中将 E_1 和 E_2 的值取出来做加法运算, 然后将运算结果存放到 $E.addr$ 中。语义动作中的 “||” 表示字符串的连接运算符。引号内的字符串按照字面值传递。

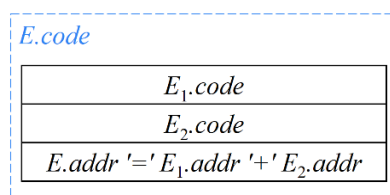


图 6-5 $E \rightarrow E_1 + E_2$ 的代码结构布局

产生式(3)与产生式(2)的语义动作类似, 只是将运算符 “+” 改为 “*”。

对于产生式(5), 对 E 的翻译与对子表达式 E_1 的翻译相同, 因此, 属性值也相同, 即 $E.addr = E_1.addr$, $E.code = E_1.code$;

产生式(4)的翻译过程与产生式(2)的翻译过程类似, 区别仅在于执行的是单目运算。

对于产生式(1), $S.code$ 是在 $E.code$ 后面连接上一条将 E 的值赋给标识符 **id** 的指令。为此, 首先调用 $lookup()$ 函数返回 **id** 的地址。

6.2.2 增量翻译

由于表达式的 $code$ 值是由子表达式的 $code$ 值连接起来的, 子表达式的 $code$ 值是由子表达式的子表达式的 $code$ 值连接起来的, 以此类推。因此, $code$ 属性可能是很长的字符串。事实上, 从图 6-4 中的这些语义动作可以看出, 产生式左部符号对应的代码总是将产生式右部符号对应的代码按顺序连接以后, 在后面追加新的三地址指令。因此通常用增量的方式生成三地址码, 也就是说不用构造 $code$ 属性去复制已经生成的三地址码, 而是在已有三地址码后面追加新的三地址指令。指令最终都是由 $gen()$ 函数生成的。在增量方法中, $gen()$ 函

数不仅要构造出一个新的三地址指令，还要将它添加到至今为止已生成的指令序列之后，如图 6-6 所示。

(1) $S \rightarrow \mathbf{id} = E;$	{	$p = \text{lookup}(\mathbf{id.lexeme});$ if $p == \text{nil}$ then $\text{error}();$ $\text{gen}(p := E.addr);$
	}	
(2) $E \rightarrow E_1 + E_2$	{	$E.addr = \text{newtemp}();$ $\text{gen}(E.addr := E_1.addr '+' E_2.addr);$
	}	
(3) $E \rightarrow E_1 * E_2$	{	$E.addr = \text{newtemp}();$ $\text{gen}(E.addr := E_1.addr '*' E_2.addr);$
	}	
(4) $E \rightarrow -E_1$	{	$E.addr = \text{newtemp}();$ $\text{gen}(E.addr := 'uminus' E_1.addr);$
	}	
(5) $E \rightarrow (E_1)$	{	$E.addr = E_1.addr; \}$
(6) $E \rightarrow \mathbf{id}$	{	$E.addr = \text{lookup}(\mathbf{id.lexeme});$ if $E.addr == \text{nil}$ then $\text{error}();$
	}	

图 6-6 赋值语句的增量翻译

例 6.3 对于图 6-6 所示的 SDT，我们在 4.4.5 节中讨论过，其基础文法虽不是 LR 文法（因为存在二义性），但是通过应用算符优先级和结合率可以消解其中的动作冲突，所以该文法依然可以使用 LR 分析技术。该文法对应的 LR 自动机如图 6-7 所示。

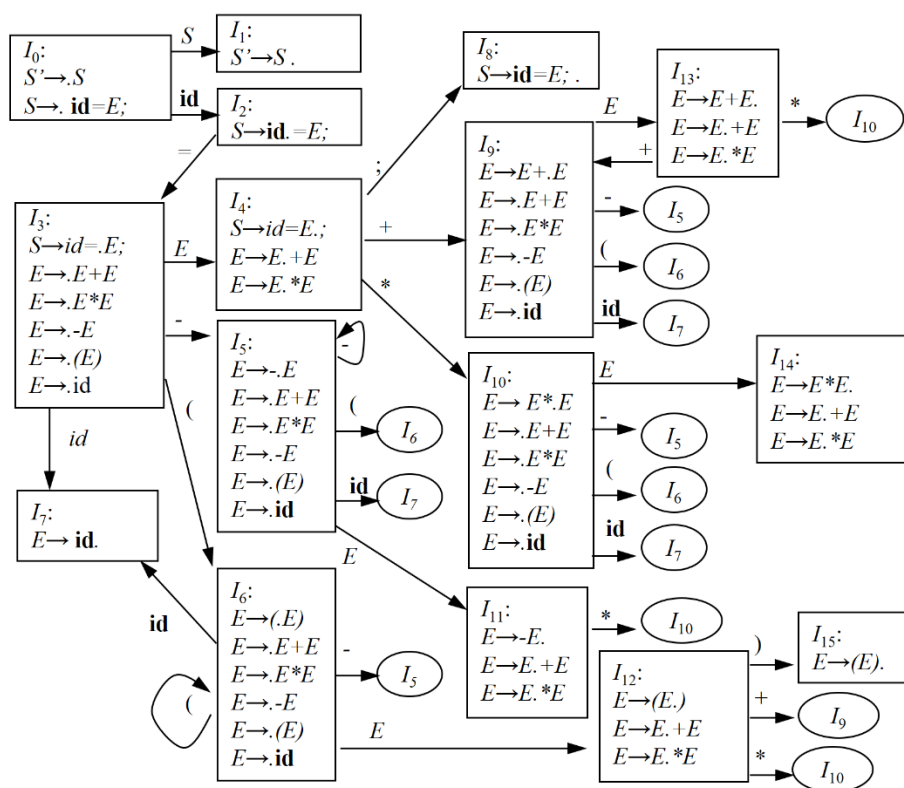


图 6-7 赋值语句文法的 LR 自动机

假设输入的源程序片段为“ $x = (a + b) * c;$ ”，则基于图 6-6 所示的 SDT 进行增量翻译的过程如图 6-8 所示。

初始时分析器的格局如图中第 1 步所示。符号栈中只有栈底符号“\$”，状态栈中是初始状态（0 号状态）。输入指针指向第 1 个输入符号 id （词法值为 x ）。

根据图 6-7，状态 I_0 遇到终结符 id 时，将 id 移入栈中并转入到状态 I_2 ，输入指针指向下一个输入符号“ $=$ ”，如图中第 2 步所示。

根据图 6-7，状态 I_2 遇到终结符“ $=$ ”时，将“ $=$ ”移入栈中并转入到状态 I_3 ，输入指针指向下一个输入符号“ $($ ”，如图中第 3 步所示。

根据图 6-7，状态 I_3 遇到终结符“ $($ ”时，将“ $($ ”移入栈中并转入到状态 I_6 ，输入指针指向下一个输入符号 id （词法值为 a ），如图中第 4 步所示。

根据图 6-7，状态 I_6 遇到终结符 id 时，将 id 移入栈中并转入到状态 I_7 ，输入指针指向下一个输入符号“ $+$ ”，如图中第 5 步所示。

I_7 是归约状态，应用产生式(6)进行归约，并执行关联的语义动作。其任务是计算即将进栈的 E 的综合属性 $addr$ 的值。由于该表达式 E 仅由一个标识

符 a 构成，因此，标识符 a 的值就是该表达式的值，那么，标识符 a 的值的存放地址就是该表达式的值的存放地址。因此，以 id 的词法值 a 作为参数调用 $lookup()$ 函数，获得 a 的地址并将它赋给 $E.addr$ 。为方便起见，可以用源程序中的名字（也就是标识符）作为三地址指令中的地址。因此，我们用名字 a 表示标识符 a 的地址，于是， $E.addr$ 等于 a 。这样，栈顶的 id 带着与之对应的状态 I_7 出栈，栈顶露出状态 I_6 。 E 带着其属性值 $E.addr=a$ 进栈。根据图 6-7，状态 I_6 遇到新归约出的 E 进入状态 I_{12} ，如图中第 6 步所示。

根据图 6-7，状态 I_{12} 遇到终结符 “+” 时，将 “+” 移入栈中并转入到状态 I_9 ，输入指针指向下一个输入符号 id （词法值为 b ），如图中第 7 步所示。

根据图 6-7，状态 I_9 遇到终结符 id 时，将 id 移入栈中并转入到状态 I_7 ，输入指针指向下一个输入符号 “)”，如图中第 8 步所示。

I_7 是归约状态，与第 6 步类似，应用产生式(6)进行归约，并执行关联的语义动作。 E 带着其属性值 $E.addr=b$ 进栈。根据图 6-7，状态 I_9 遇到新归约出的 E 进入状态 I_{13} ，如图中第 9 步所示。

I_{13} 遇到终结符 “)” 则应用产生式(2)进行归约，并执行关联的语义动作。首先调用 $newtemp()$ 函数申请一个临时变量用来存放即将进栈的 E 的值（即两个子表达式 E_1 和 E_2 相加后的结果）。假设本次 $newtemp()$ 函数返回的地址是 t_1 ，则 $E.addr=t_1$ 。接下来，调用函数 $gen(E.addr '=' E_1.addr '+' E_2.addr)$ 生成一条三地址指令，其中 $E.addr=t_1$ 。根据第 8 步， $E_1.addr=a$ ， $E_2.addr=b$ 。于是，生成的指令为：

$$t_1 = a + b \quad (6.25)$$

执行归约动作后，栈顶的 E_1 、+、 E_2 带着与之对应的状态 I_{12} 、 I_9 、 I_{13} 出栈，栈顶露出状态 I_6 。 E 带着其属性值 $E.addr=t_1$ 进栈。根据图 6-7，状态 I_6 遇到新归约出的 E 进入状态 I_{12} ，如图中第 10 步所示。

根据图 6-7，状态 I_{12} 遇到终结符 “)” 时，将 “)” 移入栈中并转入到状态 I_{15} ，输入指针指向下一个输入符号 “*”，如图中第 11 步所示。

I_{15} 是归约状态，应用产生式(5)进行归约，并执行关联的语义动作。其任务是计算即将进栈的 E 的综合属性 $addr$ 的值。该值依赖于其子节点 E_1 的 $addr$ 值（即 t_1 ）。栈顶的 “(”、 E_1 、“)” 带着与之对应的状态 I_6 、 I_{12} 、 I_{15} 出栈，栈顶露出状态 I_3 。 E 带着其属性值 $E.addr=t_1$ 进栈。根据图 6-7，状态 I_3 遇到新归约出的 E 进入状态 I_4 ，如图中第 12 步所示。

根据图 6-7，状态 I_4 遇到终结符 “*” 时，将 “*” 移入栈中并转入到状态 I_{10} ，输入指针指向下一个输入符号 id （词法值为 c ），如图中第 13 步所示。

根据图 6-7，状态 I_{10} 遇到终结符 id 时，将 id 移入栈中并转入到状态 I_7 ，输入指针指向下一个输入符号 “;”，如图中第 14 步所示。

I_7 是归约状态，与第 6 步类似，应用产生式(6)进行归约，并执行关联的语义动作。 E 带着其属性值 $E.addr=c$ 进栈。根据图 6-7，状态 I_{10} 遇到新归约出的 E 进入状态 I_{14} ，如图中第 15 步所示。

I_{14} 是归约状态，应用产生式(3)进行归约，并执行关联的语义动作。首先调用 $newtemp()$ 函数申请一个临时变量用来存放即将进栈的 E 的值（即两个子表达式 E_1 和 E_2 相乘后的结果）。假设本次 $newtemp()$ 函数返回的地址是 t_2 ，则 $E.addr=t_2$ 。接下来，调用函数 $gen(E.addr='E_1.addr * E_2.addr')$ 生成一条三地址指令，其中 $E.addr=t_1$ 。根据第 15 步， $E_1.addr=t_1$ ， $E_2.addr=c$ 。于是，生成的指令为：

$$t_2 = t_1 * c \quad (6.26)$$

执行归约动作后，栈顶的 E_1 、 $*$ 、 E_2 带着与之对应的状态 I_4 、 I_{10} 、 I_{14} 出栈，栈顶露出状态 I_3 。 E 带着其属性值 $E.addr=t_2$ 进栈。根据图 6-7，状态 I_3 遇到新归约出的 E 进入状态 I_4 ，如图中第 16 步所示。

根据图 6-7，状态 I_4 遇到终结符 “;” 时，将 “;” 移入栈中并转入到状态 I_8 ，输入指针指向下一个输入符号 “\$”，如图中第 17 步所示。

I_8 是归约状态，应用产生式(1)进行归约，并执行关联的语义动作。首先以 id 的词法值 x 作为参数调用 $lookup()$ 函数，获得 x 的地址并将它赋给变量 p 。这里，我们依然是用 x 的名字表示 x 的地址，这样， $p=x$ 。接下来，调用函数 $gen(p='E.addr')$ 生成一条三地址指令，其中 $E.addr=t_2$ 。根于是，生成的指令为：

$$x = t_2 \quad (6.27)$$

执行归约动作后，栈顶的 id 、 $=$ 、 E 和 “;” 带着与之对应的状态 I_2 、 I_3 、 I_4 和 I_8 出栈，栈顶露出状态 I_0 。 S 进栈。根据图 6-7，状态 I_0 遇到新归约出的 S 进入状态 I_1 ，即自动机唯一的终止状态，分析成功结束，如图中第 18 步所示。

步骤	输入	分析器																					
1	$x = (a + b) * c ;$ ↑	<table><tr><td>0</td></tr><tr><td>\$</td></tr></table>	0	\$																			
0																							
\$																							
2	$x = (a + b) * c ;$ ↑ ↑	<table><tr><td>0</td><td>2</td></tr><tr><td>\$</td><td>id</td></tr><tr><td></td><td>x</td></tr></table>	0	2	\$	id		x															
0	2																						
\$	id																						
	x																						
3	$x = (a + b) * c ;$ ↑ ↑ ↑	<table><tr><td>0</td><td>2</td><td>3</td></tr><tr><td>\$</td><td>id</td><td>=</td></tr><tr><td></td><td>x</td><td></td></tr></table>	0	2	3	\$	id	=		x													
0	2	3																					
\$	id	=																					
	x																						
4	$x = (a + b) * c ;$ ↑ ↑ ↑ ↑	<table><tr><td>0</td><td>2</td><td>3</td><td>6</td></tr><tr><td>\$</td><td>id</td><td>=</td><td>(</td></tr><tr><td></td><td>x</td><td></td><td></td></tr></table>	0	2	3	6	\$	id	=	(		x											
0	2	3	6																				
\$	id	=	(																				
	x																						
5	$x = (a + b) * c ;$ ↑ ↑ ↑ ↑ ↑	<table><tr><td>0</td><td>2</td><td>3</td><td>6</td><td>7</td></tr><tr><td>\$</td><td>id</td><td>=</td><td>(</td><td>id</td></tr><tr><td></td><td>x</td><td></td><td></td><td>a</td></tr></table>	0	2	3	6	7	\$	id	=	(	id		x			a						
0	2	3	6	7																			
\$	id	=	(	id																			
	x			a																			
6	$x = (a + b) * c ;$ ↑ ↑ ↑ ↑ ↑	<table><tr><td>0</td><td>2</td><td>3</td><td>6</td><td>12</td></tr><tr><td>\$</td><td>id</td><td>=</td><td>(</td><td>E</td></tr><tr><td></td><td>x</td><td></td><td></td><td>a</td></tr></table>	0	2	3	6	12	\$	id	=	(	E		x			a						
0	2	3	6	12																			
\$	id	=	(	E																			
	x			a																			
7	$x = (a + b) * c ;$ ↑ ↑ ↑ ↑ ↑ ↑	<table><tr><td>0</td><td>2</td><td>3</td><td>6</td><td>12</td><td>9</td></tr><tr><td>\$</td><td>id</td><td>=</td><td>(</td><td>E</td><td>+</td></tr><tr><td></td><td>x</td><td></td><td></td><td>a</td><td></td></tr></table>	0	2	3	6	12	9	\$	id	=	(	E	+		x			a				
0	2	3	6	12	9																		
\$	id	=	(	E	+																		
	x			a																			
8	$x = (a + b) * c ;$ ↑ ↑ ↑ ↑ ↑ ↑ ↑	<table><tr><td>0</td><td>2</td><td>3</td><td>6</td><td>12</td><td>9</td><td>7</td></tr><tr><td>\$</td><td>id</td><td>=</td><td>(</td><td>E</td><td>+</td><td>id</td></tr><tr><td></td><td>x</td><td></td><td></td><td>a</td><td></td><td>b</td></tr></table>	0	2	3	6	12	9	7	\$	id	=	(	E	+	id		x			a		b
0	2	3	6	12	9	7																	
\$	id	=	(	E	+	id																	
	x			a		b																	
9	$x = (a + b) * c ;$ ↑ ↑ ↑ ↑ ↑ ↑ ↑	<table><tr><td>0</td><td>2</td><td>3</td><td>6</td><td>12</td><td>9</td><td>13</td></tr><tr><td>\$</td><td>id</td><td>=</td><td>(</td><td>E</td><td>+</td><td>E</td></tr><tr><td></td><td>x</td><td></td><td></td><td>a</td><td></td><td>b</td></tr></table>	0	2	3	6	12	9	13	\$	id	=	(	E	+	E		x			a		b
0	2	3	6	12	9	13																	
\$	id	=	(	E	+	E																	
	x			a		b																	
10	$x = (a + b) * c ;$ ↑ ↑ ↑ ↑ ↑ ↑ ↑	<table><tr><td>0</td><td>2</td><td>3</td><td>6</td><td>12</td></tr><tr><td>\$</td><td>id</td><td>=</td><td>(</td><td>E</td></tr><tr><td></td><td>x</td><td></td><td></td><td>t₁</td></tr></table>	0	2	3	6	12	\$	id	=	(	E		x			t ₁						
0	2	3	6	12																			
\$	id	=	(	E																			
	x			t ₁																			
11	$x = (a + b) * c ;$ ↑ ↑ ↑ ↑ ↑ ↑ ↑ ↑	<table><tr><td>0</td><td>2</td><td>3</td><td>6</td><td>12</td><td>15</td></tr><tr><td>\$</td><td>id</td><td>=</td><td>(</td><td>E</td><td>)</td></tr><tr><td></td><td>x</td><td></td><td></td><td>t₁</td><td></td></tr></table>	0	2	3	6	12	15	\$	id	=	(	E	)		x			t ₁				
0	2	3	6	12	15																		
\$	id	=	(	E	)																		
	x			t ₁																			
12	$x = (a + b) * c ;$ ↑ ↑ ↑ ↑ ↑ ↑ ↑ ↑	<table><tr><td>0</td><td>2</td><td>3</td><td>4</td></tr><tr><td>\$</td><td>id</td><td>=</td><td>E</td></tr><tr><td></td><td>x</td><td></td><td>t₁</td></tr></table>	0	2	3	4	\$	id	=	E		x		t ₁									
0	2	3	4																				
\$	id	=	E																				
	x		t ₁																				
13	$x = (a + b) * c ;$ ↑ ↑ ↑ ↑ ↑ ↑ ↑ ↑	<table><tr><td>0</td><td>2</td><td>3</td><td>4</td><td>10</td></tr><tr><td>\$</td><td>id</td><td>=</td><td>E</td><td>*</td></tr><tr><td></td><td>x</td><td></td><td>t₁</td><td></td></tr></table>	0	2	3	4	10	\$	id	=	E	*		x		t ₁							
0	2	3	4	10																			
\$	id	=	E	*																			
	x		t ₁																				
14	$x = (a + b) * c ;$ ↑ ↑ ↑ ↑ ↑ ↑ ↑ ↑	<table><tr><td>0</td><td>2</td><td>3</td><td>4</td><td>10</td><td>7</td></tr><tr><td>\$</td><td>id</td><td>=</td><td>E</td><td>*</td><td>id</td></tr><tr><td></td><td>x</td><td></td><td>t₁</td><td></td><td>c</td></tr></table>	0	2	3	4	10	7	\$	id	=	E	*	id		x		t ₁		c			
0	2	3	4	10	7																		
\$	id	=	E	*	id																		
	x		t ₁		c																		
15	$x = (a + b) * c ;$ ↑ ↑ ↑ ↑ ↑ ↑ ↑ ↑	<table><tr><td>0</td><td>2</td><td>3</td><td>4</td><td>10</td><td>14</td></tr><tr><td>\$</td><td>id</td><td>=</td><td>E</td><td>*</td><td>E</td></tr><tr><td></td><td>x</td><td></td><td>t₁</td><td></td><td>c</td></tr></table>	0	2	3	4	10	14	\$	id	=	E	*	E		x		t ₁		c			
0	2	3	4	10	14																		
\$	id	=	E	*	E																		
	x		t ₁		c																		
16	$x = (a + b) * c ;$ ↑ ↑ ↑ ↑ ↑ ↑ ↑ ↑	<table><tr><td>0</td><td>2</td><td>3</td><td>4</td></tr><tr><td>\$</td><td>id</td><td>=</td><td>E</td></tr><tr><td></td><td>x</td><td></td><td>t₂</td></tr></table>	0	2	3	4	\$	id	=	E		x		t ₂									
0	2	3	4																				
\$	id	=	E																				
	x		t ₂																				

17	$x = (a + b) * c;$ ↑ ↑ ↑ ↑ ↑ ↑ ↑ ↑ ↑	<table><tr><td>0</td><td>2</td><td>3</td><td>4</td><td>8</td></tr><tr><td>\$</td><td>id</td><td>=</td><td>E</td><td>;</td></tr><tr><td></td><td>x</td><td></td><td>t₂</td><td></td></tr></table>	0	2	3	4	8	\$	id	=	E	;		x		t ₂	
0	2	3	4	8													
\$	id	=	E	;													
	x		t ₂														
18	$x = (a + b) * c;$ ↑ ↑ ↑ ↑ ↑ ↑ ↑ ↑ ↑	<table><tr><td>0</td><td>1</td></tr><tr><td>\$</td><td>S</td></tr></table>	0	1	\$	S											
0	1																
\$	S																

图 6-8 “ $x = (a + b) * c;$ ” 增量翻译过程

在以上翻译过程中生成的 3 条三地址指令(6.25)~(6.27)构成了源程序片段 “ $x = (a + b) * c;$ ” 的三地址码。可见，增量翻译可以生成正确的翻译结果。 □

6.2.3 数组引用的翻译

为了进行数组引用的翻译，我们文法(6.19)进行扩充，增加非终结符 L ，得到如下文法（其中蓝色字体表示扩充的部分）：

$$\begin{aligned}
 S &\rightarrow \text{id} = E; \mid L = E; \\
 E &\rightarrow E_1 + E_2 \mid -E_1 \mid (E_1) \mid \text{id} \mid L \\
 L &\rightarrow \text{id} [E] \mid L_1 [E]
 \end{aligned} \tag{6.28}$$

L 生成一个数组名字再加上一个下标表达式的序列，用来表示数组引用。在声明语句的文法中（见图 6-1），方括号里是常数 **num**；在数组引用的文法中，方括号里是表达式。

根据文法(6.28)，数组引用本身也可以作为一个表达式（就像标识符 **id** 所表示的基本变量本身可以作为一个表达式一样），而且也可以把一个表达式的值赋给一个数组元素。无论数组引用出现在赋值号的左边还是的右边，都涉及到数组元素存放地址的确定，即数组元素的寻址。这是将数组引用翻译成三地址码时要解决的主要问题。

数组元素的寻址

对于一维数组来说，假设每个数组元素的宽度是 w ，则数组元素 $a[i]$ 的相对地址是：

$$base + i \times w \tag{6.29}$$

其中， $base$ 是数组的基地址， $i \times w$ 是偏移量。

对于二维数组来说，假设一行的宽度是 w_1 ，同一行中每个数组元素的宽度是 w_2 ，则数组元素 $a[i_1][i_2]$ 的相对地址是：

$$base + i_1 \times w_1 + i_2 \times w_2 \tag{6.30}$$

其中， $i_1 \times w_1 + i_2 \times w_2$ 是偏移量。

式(6.29)和(6.30)可以被推广到 k 维数组，即数组元素 $a[i_1][i_2] \dots [i_k]$ 的相对地址是：

$$base + i_1 \times w_1 + i_2 \times w_2 + \dots + i_k \times w_k \quad (6.31)$$

$w_1 \rightarrow k-1$ 维数组 $a[i_1]$ 的宽度

$w_2 \rightarrow k-2$ 维数组 $a[i_1][i_2]$ 的宽度

...

$w_k \rightarrow 0$ 维数组（即基本元素） $a[i_1][i_2] \dots [i_k]$ 的宽度

其中， $i_1 \times w_1 + i_2 \times w_2 + \dots + i_k \times w_k$ 是偏移量。

带有数组引用的赋值语句的翻译

我们先来看一下带有数组引用的赋值语句的三地址代码的结构。

例 6.4 假设在某声明语句中声明了一个三维整型数组 a ，其类型表达式为 $type(a) = array(3, array(5, array(8, int)))$ 。接下来我们将源程序中的一条赋值语句

$$c = a[i_1][i_2][i_3]; \quad (6.32)$$

翻译成三地址码。这条赋值语句将一个数组元素的值赋给一个变量 c ，其翻译过程主要包括两步：

(1) 计算数组元素 $a[i_1][i_2][i_3]$ 的存放地址。

(2) 生成一条三地址指令，从第 (1) 步中计算出的地址中取出元素的值，放到地址 c 中。

根据地址计算公式(6.23)，有

$$\begin{aligned} addr(a[i_1][i_2][i_3]) &= base + i_1 * w_1 + i_2 * w_2 + i_3 * w_3 \\ &= base + i_1 * 160 + i_2 * 32 + i_3 * 4 \end{aligned} \quad (6.33)$$

其中，因为 a 是一个三维整型数组，所以 w_3 是数组基本元素（即整型变量）的宽度 4， w_2 是一维数组（类型表达式为 $array(8, int)$ ）的宽度 32， w_1 是二维数组（类型表达式为 $array(5, array(8, int))$ ）的宽度 160。

在式(6.33)中， $i_1 * 160 + i_2 * 32 + i_3 * 4$ 是偏移量 $offset$ 。为此，首先生成一段三地址码来计算它，即

$$\begin{aligned} t_1 &= i_1 * 160 \\ t_2 &= i_2 * 32 \\ t_3 &= t_1 + t_2 \\ t_4 &= i_3 * 4 \\ t_5 &= t_3 + t_4 \end{aligned} \quad (6.34)$$

此时， t_5 中存放的就是偏移量 $offset$ 。

接下来，生成一条数组访问指令

$$t_6 = a[t_5] \quad (6.35)$$

该指令的含义是：从距离基地址 a 处 t_5 个内存单元的位置把数组元素的值取出

来赋给临时变量 t_6 。 t_6 就是数组元素 $a[i_1][i_2][i_3]$ 作为一个表达式 E ，它的值的存放地址。

再接下来，根据图 6-6 中产生式(1)的语义动作，生成一条赋值指令，把这个表达式的值赋给变量 c ，即

$$c = t_6 \quad (6.36)$$

注意，与例 6.2 类似，式(6.34)和(6.35)也不能合并成一条三地址指令

$$c = a[t_5] \quad (6.37)$$

因为 $[]$ 在 L 产生式中， c 在 S 产生式中，它们未同时出现在同一个产生式中，因此无法直接生成一条同时包含这两个符号的三地址指令，而是通过 E 产生式进行过渡衔接。当然这条冗余的指令可以在代码优化时被删除掉。

在以上过程中生成的三地址指令(6.34)~(6.36)构成了源程序片段“ $c = a[i_1][i_2][i_3];$ ”的三地址码。□

为了给赋值语句“ $c = a[i_1][i_2][i_3];$ ”生成 (6.34)~(6.36)这样的三地址码，应该如何为文法(6.28)构造 SDT？翻译的关键问题是：如何将地址计算公式(6.31)和数组引用的文法(6.28)关联起来。

根据基础文法(6.28)，我们将采用自底向上的翻译方法。对于例 6.4 中的输入词串

$$a \quad [\quad i_1 \quad] \quad [\quad i_2 \quad] \quad [\quad i_3 \quad] \quad (6.38)$$

a 和各个 i 都是标识符 **id**。根据文法(6.28)， $FOLLOW[E] = \{ ; +) \}$ ，因此 i_1 、 i_2 和 i_3 都将归约为 E ，如图 6-9 所示。

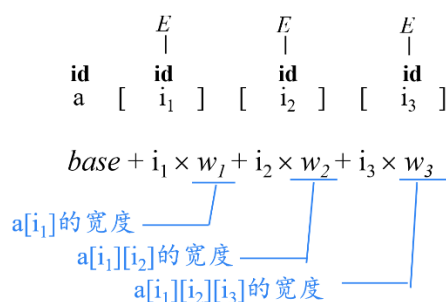


图 6-9 数组引用翻译过程示例（第 1 步）

读入 a 以后，通过调用 $lookup(a)$ 函数可以返回 a 在符号表中对应的记录，从而获得数组 a 的基地址，即式(6.31)中的 $base$ 。读入 i_1 以后，公式中 i_1 的值也可以确定了，如图 6-10 所示。

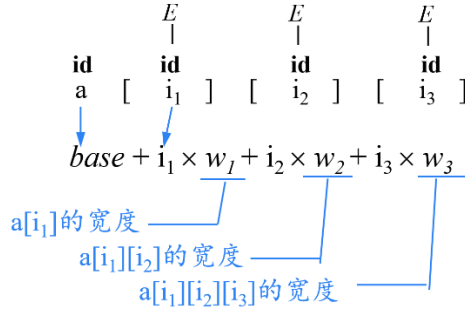


图 6-10 数组引用翻译过程示例（第 2 步）

为 L 设置 `type` 属性

当读入第一个 “`]`” 后，应用产生式 $L \rightarrow \text{id } [E]$ 进行归约，如图 6-11 所示。

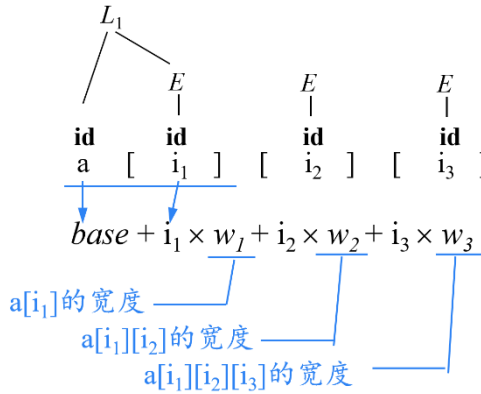


图 6-11 数组引用翻译过程示例（第 3 步）

其中， L_1 表示子数组 $a[i_1]$ ，而公式中的 w_1 是 L_1 所表示的子数组 $a[i_1]$ 的宽度。要想求 $a[i_1]$ 的宽度，就需要知道 $a[i_1]$ 的类型表达式。事实上，读入 a 以后，通过调用 `lookup(a)` 函数可以返回 a 在符号表中对应的记录，从而获得数组 a 的类型表达式，也就是 `array(3, array(5, array(8, int) ) )`。对于任何数组 arr ，假设 $arr.type$ 表示其类型表达式。则在该例中，

$$a.type = \text{array}(3, \text{array}(5, \text{array}(8, \text{int}))) \quad (6.39)$$

而 $a[i_1]$ 是 a 的一个元素，因此， $a[i_1]$ 的类型表达式就等于 a 的类型表达式中元素的类型表达式。对于任何数组类型 t ，假设 $t.elem$ 给出了其数组元素的类型。则

$$\begin{aligned} a[i_1].type &= a.type.elem \\ &= \text{array}(3, \text{array}(5, \text{array}(8, \text{int}))).elem \\ &= \text{array}(5, \text{array}(8, \text{int})) \end{aligned} \quad (6.40)$$

即， $a[i_1]$ 是一个二维数组。为此，我们为 L 设置属性 `type`，用来表示 L 生成的

子数组的类型。在图 6-11 中, L_1 生成的子数组是 $a[i_1]$, 因此它的类型表达式为

$$L_1.type = \text{array}(5, \text{array}(8, \text{int})) \quad (6.41)$$

如图 6-12 所示。

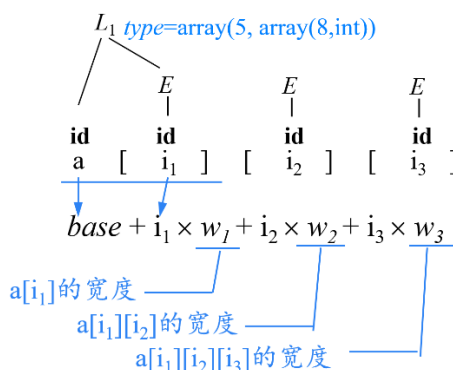


图 6-12 数组引用翻译过程示例 (第 4 步)

对于任何类型 t , 假设其宽度由 $t.width$ 给出。则

$$w_1 = L_1.type.width = 160 \quad (6.42)$$

这样, 我们就可以计算出 $i_1 \times w_1$ 所表示的一部分偏移量。

当读入第二个 “]” 后, 应用产生式 $L \rightarrow L[E]$ 进行归约, 如图 6-13 所示。

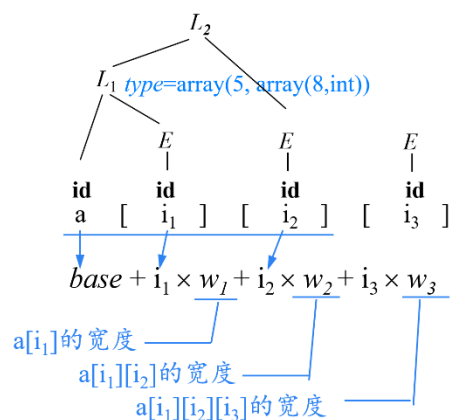


图 6-13 数组引用翻译过程示例 (第 5 步)

其中, L_2 表示子数组 $a[i_1] a[i_2]$, 而公式中的 w_2 是子数组 $a[i_1] [i_2]$ 的宽度。要想求 $a[i_1] [i_2]$ 的宽度, 就需要知道 $a[i_1] [i_2]$ 的类型表达式, 即 $L_2.type$ 的值。根据产生式 $L \rightarrow L_1[E]$, L 表示的是 L_1 的一个元素。因此, $L.type$ 可以根据 $L_1.type$ 来求。于是, 我们为产生式 $L \rightarrow L_1[E]$ 关联如下语义动作:

$$L.type = L_1.type.elem \quad (6.42)$$

这样, 图 6-13 中 $L_2.type$ 属性值计算如下:

$$L_2.type = L_1.type.elem$$

$$\begin{aligned}
&= \text{array}(5, \text{array}(8, \text{int})).\text{elem} \\
&= \text{array}(8, \text{int})
\end{aligned} \tag{6.43}$$

即, L_2 表示一个一维数组, 如图 6-14 所示。

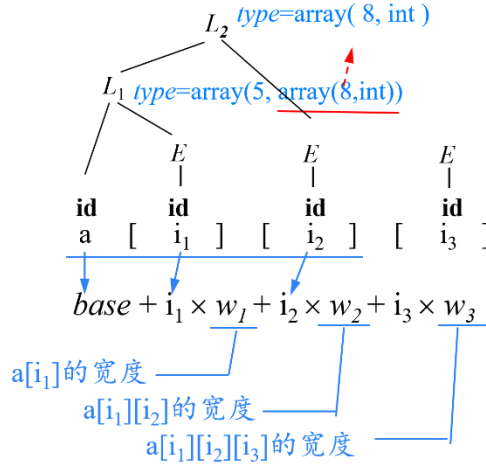


图 6-14 数组引用翻译过程示例 (第 6 步)

同时, 可以计算出

$$w_2 = L_2.type.width = 32 \tag{6.44}$$

这样, 我们就可以得到当前累积的偏移量 $i_1 * w_1 + i_2 * w_2$ 。

类似地, 当读入第三个 “]” 后, 仍然应用产生式 $L \rightarrow L[E]$ 进行归约, 同时计算出 $L_3.type$ (如图 6-15 所示) 和 w_3 , 从而得到全部的偏移量 $i_1 * w_1 + i_2 * w_2 + i_3 * w_3$ 。

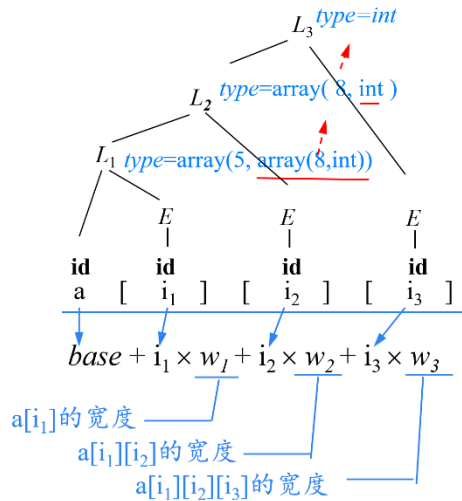


图 6-15 数组引用翻译过程示例 (第 7 步)

$$L_3.type = L_2.type.elem$$

$$= \text{array}(8, \text{int}).\text{elem}$$

$$= \text{int} \quad (6.45)$$

$$w_3 = L_3.\text{type.width} = 4 \quad (6.46)$$

为 L 设置 *offset* 属性

可以看出，式(6.31)中的偏移量可以随着翻译的进展逐步累加起来，也就是说，每读入一对方括号 $[]$ ，就可以归约出一个 L ，并计算出一部分偏移量 $i_j \times w_j$ 。为此，我们为 L 设置 *offset* 属性，它指示一个临时变量，该临时变量用于累加公式中的 $i_j \times w_j$ 项，从而计算数组元素的偏移量，如图 6-16 所示。

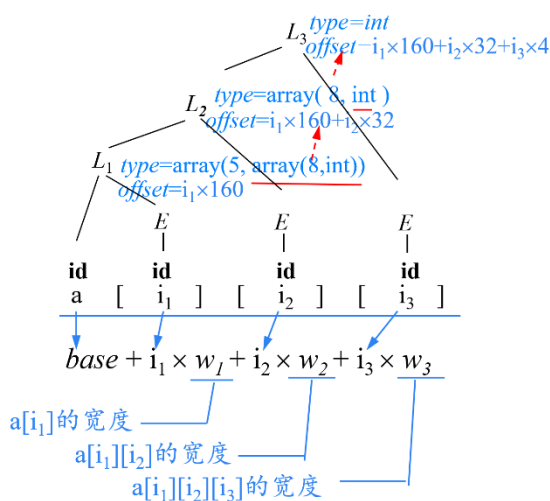


图 6-16 数组引用翻译过程示例（第 8 步）

可见，对于产生式 $L \rightarrow L_1 [E]$ 来说，父节点 L 的 *offset* 属性值是在子节点 L_1 的 *offset* 属性值的基础上增加新计算出来的项 $i_j \times w_j$ 的值。从图 6-16 可以看出， $L_3.\text{offset}$ 中形成了式(6.33)中完整的偏移量。此时，如果在节点 L_3 处能获得数组 a 的基地址 base 的话，就可以生成式(6.35)所示的数组访问指令。那么，如何让节点 L_3 获得基地址信息？

为 L 设置 *array* 属性

事实上，图 6-16 中的节点 L_1 表示对产生式 $L \rightarrow \text{id } [E]$ 的应用。而在 $L \rightarrow \text{id } [E]$ 的语义动作中，可以通过调用 $\text{lookup}(\text{id}.\text{lexeme})$ 函数返回数组 id 在符号表中对应的记录，获得数组 id 的基地址。为此，为 L 设置 *array* 属性，表示数组 id 在符号表的入口地址（为方便起见，我们用数组名表示这个入口地址）。同时，为产生式 $L \rightarrow \text{id } [E]$ 关联语义动作：

$$L.\text{array} = \text{lookup}(\text{id}.\text{lexeme}); \quad (6.47)$$

数组的基地址由 $L.\text{array}.\text{base}$ 给出。这样，如图 6-17 所示， $L_1.\text{array}$ 中存放了数

组 a 在符号表的入口地址。

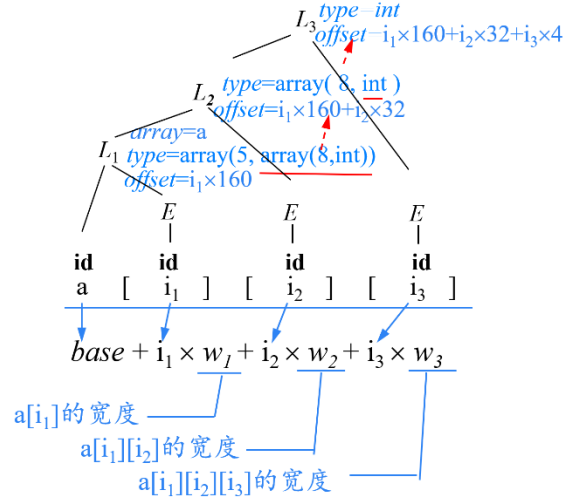


图 6-17 数组引用翻译过程示例（第 9 步）

同时，为产生式 $L \rightarrow L_1[E]$ 关联语义动作：

$$L.array = L_1.array; \quad (6.48)$$

这样，就可以将 $array$ 属性沿着由若干个 L 组成的链向上传递到树顶部的 L ，如图 6-18 所示。

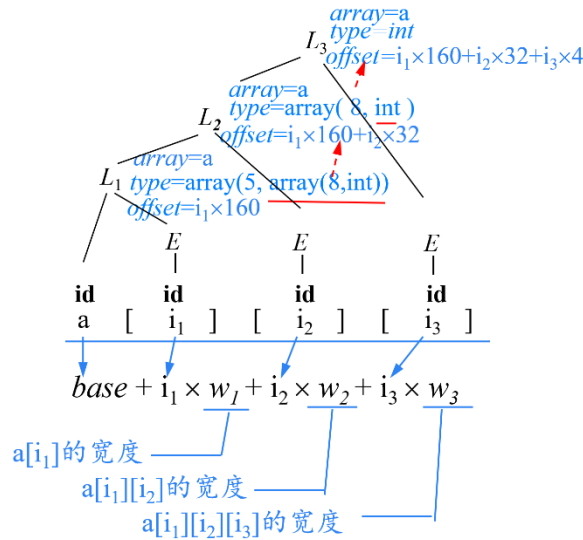


图 6-18 数组引用翻译过程示例（第 10 步）

这样，在树顶部的 L_3 中可以同时获得式(6.33)中的基地址 $base$ 和偏移量 $offset$ ，从而可以生成数组访问指令。

通过以上分析，可以得到带有数组引用的赋值语句的翻译方案，如图 6-

19 所示。所有的语义动作都放置在产生式右部末尾。 L 产生式末尾的语义动作主要是用来计算 L 的 3 个综合属性。

$S \rightarrow id = E;$	{	$p = lookup(id.lexeme);$ $if\ p == nil\ then\ error();$ $gen(p\ '='\ E.addr);$	
	}		
$ L = E;$	{	$gen(L.array.base\ '['\ L.offset\ ']\ '='\ E.addr);$	}
$E \rightarrow E_1 + E_2$	{	$E.addr = newtemp();$ $gen(E.addr\ '='\ E_1.addr\ '+'\ E_2.addr);$	
	}		
$ -E_1$	{	$E.addr = newtemp();$ $gen(E.addr\ '='\ 'uminus'\ E_1.addr);$	
	}		
$ (E_1)$	{	$E.addr = E_1.addr;$	}
$ id$	{	$E.addr = lookup(id.lexeme);$ $if\ E.addr == nil\ then\ error();$	
	}		
$ L$	{	$E.addr = newtemp();$ $gen(E.addr\ '='\ L.array.base\ '['\ L.offset\ ']);$	
	}		
$L \rightarrow id [E]$	{	$L.array = lookup(id.lexeme);$ $if\ L.array == nil\ then\ error();$ $L.type = L.array.type.elem;$ $L.offset = newtemp();$ $gen(L.offset\ '='\ E.addr\ '*' L.type.width);$	
	}		
$ L_1[E]$	{	$L.array = L_1.array;$ $L.type = L_1.type.elem;$ $t = newtemp();$ $gen(t\ '='\ E.addr\ '*' L.type.width);$ $L.offset = newtemp();$ $gen(L.offset\ '='\ L_1.offset\ '+'\ t);$	
	}		

图 6-19 带有数组引用的赋值语句的翻译方案

对于例 6.4 中声明的数组 a 以及的输入词串

$$a \quad [\quad i_1 \quad] \quad [\quad i_2 \quad] \quad [\quad i_3 \quad] \quad (6.49)$$

应用图 6-19 所示 SDT 进行自底向上的翻译。 i_1 、 i_2 和 i_3 最终被归约为 E 时，根据产生式 $E \rightarrow id$ 对应的语义动作， E 的 $addr$ 属性值分别是 i_1 、 i_2 和 i_3 ，也就是说，这些表达式的值分别存放在变量 i_1 、 i_2 和 i_3 中，如图 6-20 所示。

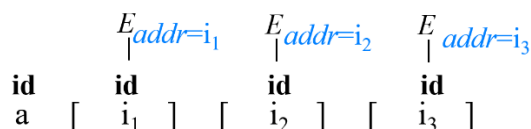


图 6-20 将 i_1 、 i_2 、 i_3 归约为 E 时的翻译结果

将 $a[i_1]$ 归约为 L_1 时, 执行产生式 $L \rightarrow \text{id}[E]$ 对应的语义动作, 如图 6-21 所示。其中, 在第 2 步分析出 $a[i_1]$ 的类型表达式 (即 $L_1.type$) 之后, 就可以获得 $a[i_1]$ 的宽度 w_1 (即 $L_1.type.width$, 在翻译数组 a 的声明语句时已经将该信息存入符号表中 a 的内情向量中), 从而计算 $i_1 \times w_1$ 这一部分偏移量。 $i_1 \times w_1$ 的值计算出来以后需要找一个地方存放, 因此, 第 3 步调用 $newtemp()$ 函数申请一个临时变量用来存放该值。将函数返回值 (即新申请的临时变量的地址, 假设为 t_1) 赋给 $L_1.offset$, 并在第 4 步生成计算 $i_1 \times w_1$ 的三地址指令。这样, 地址 $L_1.offset$ 中存放的就是 $i_1 \times w_1$ 的值。

步骤	语义动作	翻译结果
1	$L.array = \text{lookup}(\text{id.lexeme});$ $\text{if } L.array == \text{nil} \text{ then error}();$	
2	$L.type = L.array.type.elem;$	
3	$L.offset = \text{newtemp}();$	
4	$\text{gen}(L.offset '=' E.addr '*' L.type.width);$	三地址码: $t_1 = i_1 * 160$

图 6-21 将 $a[i_1]$ 归约为 L_1 时的翻译结果

将 $L_1[E]$ 归约为 L_2 时, 执行产生式 $L \rightarrow L_1[E]$ 对应的语义动作, 如图 6-22 所示。其中, 在第 2 步分析出 $a[i_1][i_2]$ 的类型表达式 (即 $L_2.type$) 之后, 就可以获得 $a[i_1][i_2]$ 的宽度 w_2 (即 $L_2.type.width$, 在翻译数组 a 的声明语句时已经将该信息存入符号表中 a 的内情向量中), 从而计算 $i_2 \times w_2$ 这一部分偏移量。 $i_1 \times w_1$ 的值计算出来以后需要找一个地方存放, 因此, 第 3 步调用 $newtemp()$ 函数申请一个临时变量用来存放该值。将函数返回值 (即新申请的临时变量的地址, 假设为 t_2) 赋给临时变量 t , 并在第 4 步生成计算 $i_2 \times w_2$ 的三地址指令。这样, 地址 t_2 中存放的就是 $i_2 \times w_2$ 的值。接下来, 需要将新计算出的这部分偏移量与 $L_1.offset$ 中存放的之前的累积偏移量累加在一起, 累加后的结果需要找一

个地方存放。因此，第 5 步再次调用 *newtemp()* 函数申请一个临时变量用来存放累加后的结果。将函数返回值（即新申请的临时变量的地址，假设为 t_3 ）赋给 $L_2.offset$ ，并在第 6 步生成计算累加值的三地址指令。这样，地址 $L_2.offset$ 中存放的就是 $i_1 \times w_1 + i_2 \times w_2$ 的值。

步骤	语义动作	翻译结果
1	$L.array = L_1.array;$	<i>array=a</i> <pre> graph TD L2 --> L1 L2 --> E3["E addr=i3"] L1 --> L2_1["L2 array=a, type=array(5, array(8,int)), offset=t1"] L1 --> E1["E addr=i1"] L2_1 --> id_a["id a"] L2_1 --> id_i1["id i1"] L2_1 --> id_i2["id i2"] </pre>
2	$L.type = L_1.type.elem;$	<i>array=a</i> <i>type=array(8, int)</i>
3	$t = newtemp();$	$t = t_2$
4	$gen(t := E.addr * L.type.width);$	三地址码: $t_2 = i_2 * 32$
5	$L.offset = newtemp();$	<i>array=a</i> <i>type=array(8, int)</i> <i>offset=t3</i>
6	$gen(L.offset := L_1.offset + t);$	三地址码: $t_3 = t_1 + t_2$

图 6-22 将 $L_1[E]$ 归约为 L_2 时的翻译结果

类似的，将 $L_2[E]$ 归约为 L_3 时，也执行产生式 $L \rightarrow L_1[E]$ 对应的语义动作，如图 6-23 所示。

步骤	语义动作	翻译结果
----	------	------

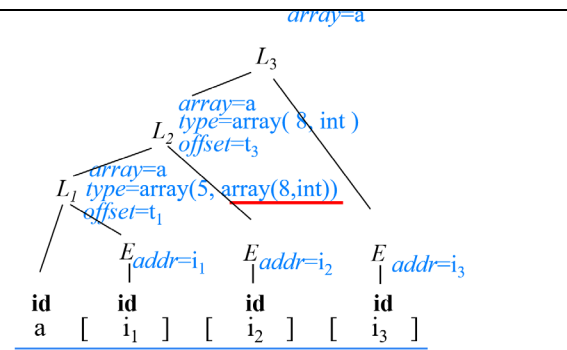
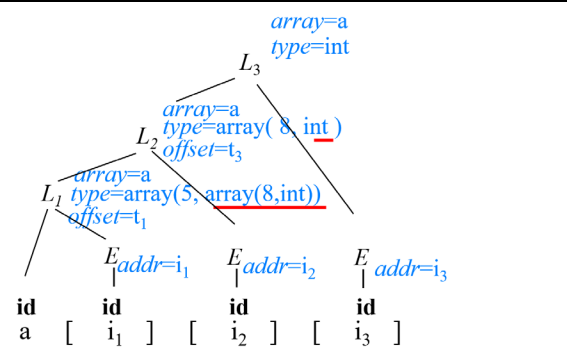
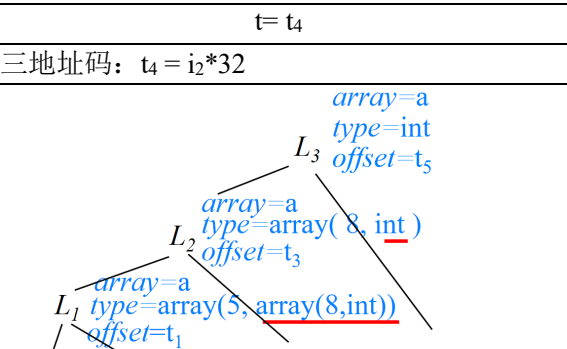
1	$L.array = L_1.array;$	
2	$L.type = L_1.type.elem;$	
3	$t = newtemp();$	$t = t_4$
4	$gen(t := E.addr * L.type.width);$	三地址码: $t_4 = i_2 * 32$
5	$L.offset = newtemp();$	
6	$gen(L.offset := L_1.offset + t);$	三地址码: $t_5 = t_3 + t_4$

图 6-23 将 $L_2[E]$ 归约为 L_3 时的翻译结果

这样, $L_3.offset$ 中存放的就是最终的偏移量 $i_1 \times w_1 + i_2 \times w_2 + i_3 \times w_3$ 的值。通过 $L_3.array$ 可以获得数组的基地址, 从而可以通过数组访问指令读取数组元素。

接下来, 应用产生式 $E \rightarrow L$ 进行归约 (如图 6-24 所示), 也就是说, 数组引用本身可以作为表达式 (就像标识符 **id** 所表示的基本变量本身可以作为一个表达式一样), 那么就需要有一个地址用来存放该表达式的值。因此,

产生式 $E \rightarrow L$ 的语义动作再次调用 *newtemp()* 函数申请一个临时变量用来存放累加后的结果。将函数返回值（即新申请的临时变量的地址，假设为 t_6 ）赋给 $E.addr$ ，并调用 $gen(E.addr := L.array.base [L.offset])$ 函数生成数组访问三地址指令，即

$$t_6 = a[t_5] \quad (6.50)$$

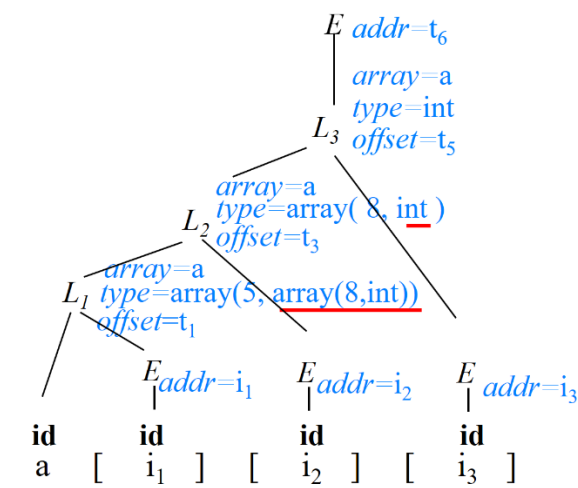


图 6-24 将 L_3 归约为 E 时的翻译结果

将图 6-21~图 6-23 中的三地址码以及式(6.50)连接到一起，即构成输入词串 (6.49) 对应的三地址码：

$$\begin{aligned} t_1 &= i_1 * 160 \\ t_2 &= i_2 * 32 \\ t_3 &= t_1 + t_2 \\ t_4 &= i_3 * 4 \\ t_5 &= t_3 + t_4 \\ t_6 &= a[t_5] \end{aligned} \quad (6.51)$$

当然，正如文法(6.28)中产生式 “ $S \rightarrow L = E;$ ” 表示的那样，我们也可以把一个表达式的值赋给数据元素，如图 6-25 所示。

$$S \rightarrow \text{if } B \text{ then } S_1 \text{ else } S_2 \quad (6.53)$$

为例， S 的代码主要由 $B.code$ 、 $S_1.code$ 和 $S_2.code$ 三部分顺序连接构成，如图 6-26 所示。

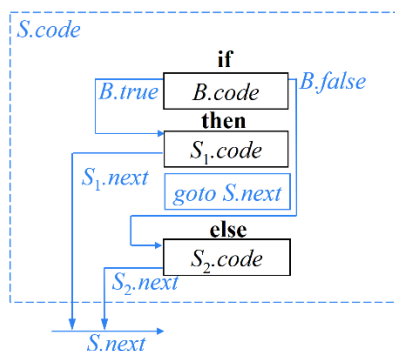


图 6-26 if-then-else 语句的代码结构

其中，布尔表达式 B 被翻译成由跳转指令构成的跳转代码。例如， $a < b$ 被翻译成一对跳转指令

$$\text{if } a < b \text{ goto } S_1.first \quad (6.54)$$

$$\text{goto } S_2.first \quad (6.55)$$

其中， $S_1.first$ 是 S_1 的第一条指令的标号（我们用指令的标号标识一条三地址指令，标号本身是一个常量）， $S_2.first$ 是 S_2 的第一条指令的标号。这对指令表达的语义为：当 B 值为真时，跳转到 S_1 的第一条指令（称为 B 的真出口）；否则，即当 B 值为假时，跳转到 S_2 的第一条指令（称为 B 的假出口）。但是，在生成这对跳转指令的时候， $S_1.first$ 和 $S_2.first$ 的值还不知道。因为 $S_2.first$ 的值要等 S_1 的代码生成完毕才能确定，而如果 B 是一个比较复杂的布尔表达式，如 $x < 100 \parallel x > 200 \ \&\& \ x \neq y$ ，那么 $S_1.first$ 的值在生成 B 的第 1 条跳转指令时也无法确定。解决这一问题的办法是：为布尔表达式 B 临时生成一对跳转指令

$$\text{if } a < b \text{ goto } B.true \quad (6.56)$$

$$\text{goto } B.false \quad (6.57)$$

其中， $B.true$ 和 $B.false$ 是两个变量，用来存放 $S_1.first$ 和 $S_2.first$ 的值。也就是说，在生成这对跳转指令时，虽然无法知道 $S_1.first$ 和 $S_2.first$ 的值，但是可以先申请两个变量用来存放 $S_1.first$ 和 $S_2.first$ 的值。把这两个变量的地址作为属性传递到 $S_1.first$ 和 $S_2.first$ 的值可以确定的地方。当分析完 B 、确定下来 $S_1.first$ 的值以后，再将其填入到地址 $B.true$ 中；当分析完 S_1 、确定下来 $S_2.first$ 的值以后，再将其填入到地址 $B.false$ 中。

最后，从 $B.true$ 和 $B.false$ 中取出标号 $S_1.first$ 和 $S_2.first$ ，生成最终（正式）

的三地址指令，即

$$\text{if } a < b \text{ goto } (B.true) \quad (6.58)$$

$$\text{goto } (B.false) \quad (6.59)$$

其中，“ $(B.true)$ ”表示地址 $B.true$ 中存放的内容，即 $S_1.first$ 的值，因此，式(6.58)与式(6.54)是同一条指令。“ $(B.false)$ ”也是类似的情况。

这就相当于在旅店为客人预先预定一个房间，等客人到达以后再入住。这里，标号 $S_1.first$ 和 $S_2.first$ 相当于客人， $B.true$ 和 $B.false$ 相当于房间地址。为标号 $S_1.first$ 预定的房间地址是 $B.true$ ；为标号 $S_2.first$ 预定的房间地址是 $B.false$ 。这样，我们可以为产生式(6.53)定义语义动作：当标号 $S_1.first$ 和 $S_2.first$ 确定下来以后，让它“入住”到自己的房间 $B.true$ 和 $B.false$ 中。具体来说，将 $B.true$ 和 $B.false$ 作为 B 的两个属性传递到 $S_1.first$ 和 $S_2.first$ 的值可以确定的地方，将值填写到变量里。由于要在分析 B 之前完成“预定房间”的工作，因此将 $B.true$ 和 $B.false$ 设置成 B 的继承属性：

- $B.true$ ：是一个地址，该地址用来存放当 B 为真时控制流转向的指令的标号。
- $B.false$ ：是一个地址，该地址用来存放当 B 为假时控制流转向的指令的标号。

根据 if-then-else 语句的语义， S_1 执行完需要跳转到紧跟在 S 代码之后执行的指令，因此这里需要生成一条跳转指令。但是， S 代码之后的指令的标号目前还不能确定，需要等 S_2 的代码生成完毕才能确定。解决这一问题的法：与 B 的继承属性类似，为 S 也设置继承属性： $S.next$

- $S.next$ ：是一个地址，该地址用来存放紧跟在 S 代码之后执行的指令的标号。

这样，可以临时生成一条跳转指令 $\text{goto } S.next$ 。当紧跟在 S 代码之后执行的指令的标号确定下来以后，将它填入到 $S.next$ 中。

既然为 S 设置了继承属性 $next$ （因为 S 的代码中可能会有一些跳转指令的目标地址是紧跟在 S 的代码后执行的指令），那么， S_1 和 S_2 其实是同样的文法符号， S_1 和 S_2 的代码中可能也会有一些跳转指令的目标地址是紧跟在 S_1 （ S_2 ）的代码后执行的指令（比如说 S_1 或 S_2 中嵌套了 if 分支语句），因此， S_1 和 S_2 也有 $next$ 属性。由于 S_1 和 S_2 执行完，都将执行紧跟在 S 之后执行的第一条指令（也就是 $S.next$ 指向的指令），因此 $S_1.next$ 和 $S_2.next$ 与 $S.next$ 指向的是同一个指令。

$P \rightarrow S$ 的翻译方案

根据该产生式，程序 P 可以由一个可执行语句序列 S 构成。前面提到，我们为 S 设置了继承属性 $S.next$ ，因此，在 S 出现之前需要设置语义动作计算该属性值。 $S.next$ 是一个地址，用来存放紧跟在 S 之后执行的那条指令的标号。为此，调用 $newlabel()$ 函数生成一个用于存放标号的新的临时变量 L ，并将函数的返回值，也就是新生成的临时变量的地址赋给 $S.next$ 。存放标号的地址已经确定了，但是标号具体是什么在分析 S 之前还不能确定。事实上，这个标号的值要等 S 分析完毕， S 的代码生成完毕才能确定。因此，在 S 之后，执行一个语义动作，调用 $label(S.next)$ 函数，将下一条三地址指令的标号存放到 $S.next$ 中。即

$$P \rightarrow \{ S.next = newlabel(); \} S \{ label(S.next); \} \quad (6.60)$$

由此可见， S 前的语义动作只是先确定 S 的后继指令标号的存放地址，而标号的具体值要等 S 分析完毕后在紧跟其后的语义动作中填入。

- $newlabel()$: 生成一个用于存放标号的新的临时变量 L ，返回变量地址。
- $label(L)$: 将下一条三地址指令的标号存放到地址 L 中。

顺序结构的翻译方案

我们首先根据产生式 $S \rightarrow S_1 S_2$ 来构造代码结构图。根据顺序结构的语义， $S.code$ 由 $S_1.code$ 和 $S_2.code$ 顺序连接起来。另外，还需要在图中标记出各文法符号的属性信息。这里涉及到的语义属性有三个，即 $S.next$ 、 $S_1.next$ 和 $S_2.next$ 。 $S.next$ 是父节点的继承属性，存放的是紧跟在 $S.code$ 之后执行的那条指令的标号，如图 6-27 所示。 $S_1.next$ 中存放的是紧跟在 $S_1.code$ 之后执行的那条指令的标号。由于 $S_1.code$ 执行完以后就立即执行 $S_2.code$ ，所以 $S_1.next$ 指向 $S_2.code$ 的第 1 条指令。 $S_2.next$ 中存放的是紧跟在 $S_2.code$ 之后执行的那条指令的标号。由于 $S_2.code$ 执行完毕就意味着整个 $S.code$ 执行完毕，因此 $S_2.next$ 与 $S.next$ 指向的是同一条指令。

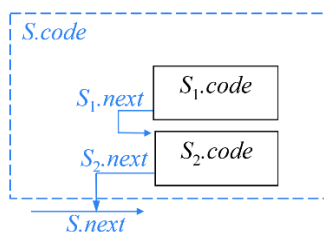


图 6-27 顺序结构的代码结构

根据图 6-27 所示的代码结构图，可以编写该产生式对应的语义动作：

$$S \rightarrow \{ S_1.next = newlabel(); \} S_1 \{ S_2.next = S.next; label(S_1.next); \} S_2 \quad (6.61)$$

该产生式对应的语义动作的主要任务就是完成相关语义属性的计算。在开始分

析 S_1 之前, 首先设置语义动作计算其继承属性, 即 $S_1.next$ 。根据图 6-27, $S_1.next$ 中存放的是 $S_2.code$ 的第 1 条指令的标号。但是在 S_1 未分析完之前, 还不能确定 $S_2.code$ 的第 1 条指令的标号是多少, 因为我们尚不知道 $S_1.code$ 中有多少条指令。为此, 调用 $newlabel()$ 函数生成一个用于存放标号的新的临时变量 L , 并将函数的返回值, 也就是新生成的临时变量的地址赋给 $S_1.next$ 。

在开始分析 S_2 之前, 首先设置语义动作计算其继承属性, 即 $S_2.next$ 。根据图 6-27, $S_2.next$ 与 $S.next$ 指向的是同一条指令, 因此不用再申请临时变量来存放紧跟在 S_2 之后执行的那个指标的标号, 而是令 $S_2.next = S.next$, 即 $S_2.next$ 也指向紧跟在 $S.code$ 之后的指令。也就是说, $S_2.next$ 的属性值是从父节点的属性值 $S.next$ 继承来的。从图中还可以看出, $S_2.code$ 的入口处有一个导入的箭头, 也就是说 $S_1.next$ 在等待 $S_2.code$ 的第 1 条指定的标号。此时即将分析 S_2 , 那么 $S_2.code$ 的第 1 条指令的标号就可以确定下来了。于是调用 $label(S_1.next)$ 函数, 将下一条三地址指令 (即 $S_2.code$ 的第 1 条指令) 的标号存放到 $S_1.next$ 中。

if-then-else 语句的翻译方案

if-then-else 语句的代码结构图已经在图 6-26 中给出。据此可以给出该产生式对应的语义动作:

$$\begin{aligned}
 S \rightarrow & \text{if } \{B.true = newlabel(); B.false = newlabel();\} B \\
 & \text{then } \{S_1.next = S.next; label(B.true);\} S_1 \{gen('goto' \ S.next)\} \\
 & \text{else } \{S_2.next = S.next; label(B.false);\} S_2 \quad (6.62)
 \end{aligned}$$

在开始分析产生式右部第 1 个非终结符 B 之前, 需要先计算其继承属性, 即 $B.true$ 和 $B.false$ 。根据图 6-26, $B.true$ 中存放的是 $S_1.code$ 的第 1 条指令的标号, $B.false$ 中存放的是 $S_2.code$ 的第 1 条指令的标号。但是在分析 B 之前, 还不能确定这两个标号具体是多少。为此, 分别调用 $newlabel()$ 函数生成两个用于存放标号的临时变量, 并将函数的返回值, 也就是新生成的临时变量的地址分别赋给 $B.true$ 和 $B.false$ 。

在开始分析产生式右部第 2 个非终结符 S_1 之前, 需要先计算其继承属性, 即 $S_1.next$ 。根据图 6-26, $S_1.next$ 与 $S.next$ 指向的是同一条指令, 因此令 $S_1.next = S.next$ 。从图中还能看出, $S_1.code$ 的入口处有一个导入的箭头, 也就是说 $B.true$ 在等 $S_1.code$ 的第 1 条指定的标号。于是调用 $label(B.true)$ 函数, 将下一条三地址指令 (即 $S_1.code$ 的第 1 条指令) 的标号存放到 $B.true$ 中。

根据图 6-26, 在紧跟 $S_1.code$ 之后, 需要显示生成一条跳转指令, 因此在产生式中紧跟 S_1 之后要插入语义动作, 调动 $gen()$ 函数, 生成一条跳转指令 $goto \ S.next$ 。

在开始分析产生式右部第 3 个非终结符 S_2 之前，需要先计算其继承属性，即 $S_2.next$ 。根据图 6-26， $S_2.next$ 与 $S.next$ 指向的是同一条指令，因此令 $S_2.next = S.next$ 。从图中还可以看出， $S_2.code$ 的入口处有一个导入的箭头，也就是说 $B.false$ 在等 $S_2.code$ 的第 1 条指定的标号。于是调用 $label(B.false)$ 函数，将下一条三地址指令（即 $S_2.code$ 的第 1 条指令）的标号存放到 $B.false$ 中。

if-then 语句的翻译方案

我们首先根据产生式 $S \rightarrow \text{if } B \text{ then } S_1$ 来构造代码结构图。 S 的代码主要由 $B.code$ 和 $S_1.code$ 两部分顺序连接构成。另外，还需要在图中标记出各文法符号的属性信息。这里涉及到的语义属性有四个，即 $B.true$ 、 $B.false$ 、 $S_1.next$ 和 $S.next$ 。 $S.next$ 是父节点的继承属性，存放的是紧跟在 $S.code$ 之后执行的那条指令的标号，如图 6-28 所示。根据 if-then 语句的语义，当 B 值为真时，跳转到 S_1 的第一条指令，因此 $B.true$ 指向 S_1 的第一条指令。当 B 值为假时，需要跳出 if-then 语句，即整个 S 执行完毕，接下来执行紧跟在 S 之后执行的第一条指令（也就是 $S.next$ 指向的指令），因此 $B.false$ 与 $S.next$ 指向的是同一个指令。 $S_1.next$ 中存放的是紧跟在 $S_1.code$ 之后执行的那条指令的标号。由于 $S_1.code$ 执行完毕就意味着整个 $S.code$ 执行完毕，因此 $S_1.next$ 与 $S.next$ 指向的是同一条指令。

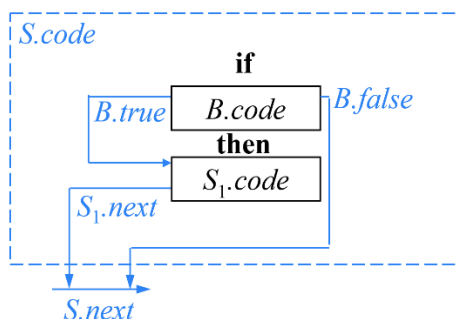


图 6-28 if-then 语句的代码结构

根据图 6-28 所示的代码结构图，可以编写该产生式对应的语义动作：

$$\begin{aligned}
 S \rightarrow & \text{if } \{ B.true = \text{newlabel}(); B.false = S.next; \} B \\
 & \text{then } \{ S_1.next = S.next; label(B.true); \} S_1
 \end{aligned} \quad (6.63)$$

在开始分析产生式右部第 1 个非终结符 B 之前，需要先计算其继承属性，即 $B.true$ 和 $B.false$ 。根据图 6-28， $B.true$ 中存放的是 $S_1.code$ 的第 1 条指令的标号。但是在 B 未分析完之前，还不能确定 $S_1.code$ 的第 1 条指令的是多少。为此，调用 $\text{newlabel}()$ 函数生成一个用于存放标号的临时变量，并将函数的返回值，也就是新生成的临时变量的地址赋给 $B.true$ 。从图中还可以看出， $B.false$

与 $S.next$ 指向的是同一条指令，因此令 $B.false = S.next$ 。

在开始分析产生式右部第 2 个非终结符 S_1 之前，需要先计算其继承属性，即 $S_1.next$ 。根据图 6-28， $S_1.next$ 与 $S.next$ 指向的是同一条指令，因此令 $S_1.next = S.next$ 。从图中还能看出， $S_1.code$ 的入口处有一个导入的箭头，也就是说 $B.true$ 在等 $S_1.code$ 的第 1 条指定的标号。于是调用 $label(B.true)$ 函数，将下一条三地址指令（即 $S_1.code$ 的第 1 条指令）的标号存放到 $B.true$ 中。

while-do 语句的翻译方案

我们首先根据产生式 $S \rightarrow \text{while } B \text{ do } S_1$ 来构造代码结构图。 S 的代码主要由 $B.code$ 和 $S_1.code$ 两部分顺序连接构成。另外，还需要在图中标记出各文法符号的属性信息。这里涉及到的语义属性有四个，即 $B.true$ 、 $B.false$ 、 $S_1.next$ 和 $S.next$ 。 $S.next$ 是父节点的继承属性，存放的是紧跟在 $S.code$ 之后执行的那条指令的标号，如图 6-29 所示。根据 while-do 语句的语义，当 B 值为真时，跳转到 S_1 的第一条指令，因此 $B.true$ 指向 S_1 的第一条指令。当 B 值为假时，需要跳出 while-do 语句，即整个 S 执行完毕，接下来执行紧跟在 S 之后执行的第一条指令（也就是 $S.next$ 指向的指令），因此 $B.false$ 与 $S.next$ 指向的是同一个指令。 $S_1.next$ 中存放的是紧跟在 $S_1.code$ 之后执行的那条指令的标号。由于 $S_1.code$ 执行完毕需要回过头来重新执行 $B.code$ ，因此，需要在 $S_1.code$ 之后生成一条跳转指令，跳转到 $B.code$ 的第 1 条指令。为此，我们为 B 设置另一个继承属性 $begin$ ，用来指向 $B.code$ 的第 1 条指令，并在 $S_1.code$ 之后生成跳转指令 $\text{goto } B.begin$ 。 $S_1.next$ 中存放的是紧跟在 $S_1.code$ 之后执行的那条指令的标号，即 $B.code$ 的第 1 条指令。因此 $S_1.next$ 与 $B.begin$ 指向的是同一个指令。

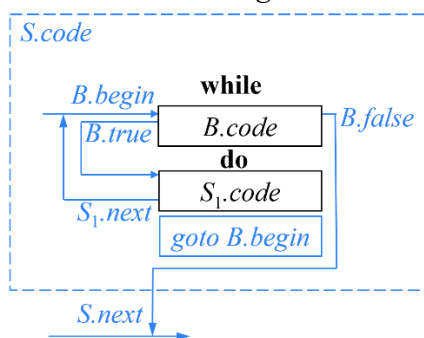


图 6-29 while-do 语句的代码结构

根据图 6-29 所示的代码结构图，可以编写该产生式对应的语义动作：

```

 $S \rightarrow \text{while} \{ B.true = \text{newlabel}(); B.false = S.next; \\
B.begin = \text{newlabel}(); \text{label}(B.begin); \} B \\
\text{do } \{ S_1.next = B.begin; \text{label}(B.true); \} S_1$ 

```

`{ gen('goto' B.begin); }` (6.64)

在开始分析产生式右部第 1 个非终结符 B 之前，需要先计算其继承属性，即 $B.true$ 、 $B.false$ 和 $B.begin$ 。根据图 6-29， $B.true$ 中存放的是 $S_1.code$ 的第 1 条指令的标号。但是在 B 未分析完之前，还不能确定 $S_1.code$ 的第 1 条指令的标号是多少。为此，调用 `newlabel()` 函数生成一个用于存放标号的临时变量，并将函数的返回值，也就是新生成的临时变量的地址赋给 $B.true$ 。从图中还可以看出， $B.false$ 与 $S.next$ 指向的是同一条指令，因此令 $B.false = S.next$ 。而 $B.begin$ 中存放的是 $B.code$ 的第 1 条指令的标号，因此，首先调用 `newlabel()` 函数生成一个用于存放标号的临时变量，并将函数的返回值，也就是新生成的临时变量的地址赋给 $B.begin$ 。接下来调用 `label(B.begin)` 函数，将下一条三地址指令（即 $B.code$ 的第 1 条指令）的标号存放到 $B.begin$ 中。

在开始分析产生式右部第 2 个非终结符 S_1 之前，需要先计算其继承属性，即 $S_1.next$ 。根据图 6-29， $S_1.next$ 与 $B.begin$ 指向的是同一条指令，因此令 $S_1.next = B.begin$ 。从图中还可以看出， $S_1.code$ 的入口处有一个导入的箭头，也就是说 $B.true$ 在等待 $S_1.code$ 的第 1 条指定的标号。于是调用 `label(B.true)` 函数，将下一条三地址指令（即 $S_1.code$ 的第 1 条指令）的标号存放到 $B.true$ 中。

根据图 6-29，在紧跟 $S_1.code$ 之后，需要显示生成一条跳转指令，因此在产生式中紧跟 S_1 之后要插入语义动作，调动 `gen()` 函数，生成一条跳转指令 `goto B.begin`。

控制流语句 SDT 编写要点

首先绘制代码结构图，整个语句的代码主要由产生式右部各非终结符对应的代码顺序连接构成。另外，需要在根据语义图中标记出各文法符号的属性信息。

接下来根据代码结构图编写产生式对应的语义动作。在每个非终结符之前嵌入语义动作：首先计算继承属性，再观察代码结构图中该非终结符对应的方框顶部是否有导入箭头。如果有，调用 `label()` 函数。当上一个代码框执行完后不顺序执行下一个代码框时，需要显式生成一条跳转指令。当有自下而上的箭头时，需要设置 `begin` 属性，且调用 `newlabel()` 函数申请临时变量后直接调用 `label()` 函数将下一条指令标号填入。

6.3.2 布尔表达式的翻译

6.3.2 节中讲到的不管是分支语句还是循环语句，里面都嵌套着布尔表达

式。布尔表达式的文法如下：

$$\begin{aligned}
 B \rightarrow & B \text{ or } B \\
 & | B \text{ and } B \\
 & | \text{not } B \\
 & | (B) \\
 & | E \text{ relop } E \\
 & | \text{true} \\
 & | \text{false}
 \end{aligned} \tag{6.65}$$

文法符号 B 表示布尔表达式。布尔常量 **true** 和 **false** 本身都是布尔表达式。**relop** 表示关系运算符，包括 $<$ 、 $\leq$ 、 $>$ 、 $\geq$ 、 $==$ 、 $!=$ 。 E 为算术表达式。将关系运算符作用于算术表达式得到一个关系表达式。关系表达式本身也是布尔表达式。**and**（与）、**or**（或）和 **not**（非）都是逻辑运算符，其优先级为 **not** $>$ **and** $>$ **or**。将括号或逻辑运算符作用于布尔表达式得到一个新的布尔表达式。

在布尔表达式的翻译中，逻辑运算符 **and**、**or** 和 **not** 被翻译成跳转指令。运算符本身不出现在代码中，布尔表达式的值是通过代码序列中的位置来表示的。

例 6.5 语句

$$\begin{aligned}
 & \text{if (} a < 10 \text{ or } a > 20 \text{ and } a != b \text{)} \\
 & \quad a = 0;
 \end{aligned} \tag{6.66}$$

可以被翻译成如下的三地址码：

$$\begin{aligned}
 & \text{if } a < 10 \text{ goto } L_2 \\
 & \quad \text{goto } L_3 \\
 L_3 : & \text{if } a > 20 \text{ goto } L_4 \\
 & \quad \text{goto } L_1 \\
 L_4 : & \text{if } a != b \text{ goto } L_2 \\
 & \quad \text{goto } L_1 \\
 L_2 : & a = 0 \\
 L_1 : &
 \end{aligned} \tag{6.67}$$

在该翻译中， L_2 是整个布尔表达式的真出口（执行赋值语句 $a=0$ ）， L_1 是其假出口。对于指令 $\text{if } a < 10 \text{ goto } L_2$ ， $a < 10$ 为真时直接跳往真出口，则说明 $a < 10$ 后面的逻辑运算符为 **or**，否则，它必须继续判断与 $a < 10$ 对应的另一个逻辑运算分量是否为真才能决定是否跳往真出口。而对于指令 $\text{if } a > 20 \text{ goto } L_4$ ， $a > 20$ 为真时既未跳往真出口 L_2 ，也未跳往假出口 L_1 ，则说明 $a > 20$ 后面的逻辑运算符

为 **and**，因为必须继续判断与 $a > 20$ 对应的另一个运算分量的真值来确定逻辑表达式的真值。也就是说，跳转指令的目标跳转位置隐含地表示了对应的逻辑运算符。□

$B \rightarrow E_1 \text{ relop } E_2$ 的翻译方案

对于由一个关系表达式构成的布尔表达式，我们将它翻译成一对跳转指令，如式(6.56)和(6.57)所示。因此该产生式对应的语义动作为：

$$B \rightarrow E_1 \text{ relop } E_2 \{ \text{gen}(\text{'if' } E_1.\text{addr relop } E_2.\text{addr 'goto' } B.\text{true}); \\ \text{gen}(\text{'goto' } B.\text{false}); \} \quad (6.68)$$

$B \rightarrow \text{true}$ 和 $B \rightarrow \text{false}$ 的翻译方案

对于由布尔常量 **true** 构成的产生式，由于对应的布尔表达式的值确定是真，因此该产生式对应的语义动作只生成一条跳往真出口的指令即可，即

$$B \rightarrow \text{true} \{ \text{gen}(\text{'goto' } B.\text{true}); \} \quad (6.69)$$

类似的，由布尔常量 **false** 构成的产生式，其翻译方案为

$$B \rightarrow \text{false} \{ \text{gen}(\text{'goto' } B.\text{false}); \} \quad (6.70)$$

$B \rightarrow (B_1)$ 的翻译方案

产生式 $B \rightarrow (B_1)$ 的语义动作的主要任务就是在分析 B_1 之前，完成 B_1 的两个继承属性 $B_1.\text{true}$ 和 $B_1.\text{false}$ 的计算。其翻译方案如下：

$$B \rightarrow (\{ B_1.\text{true} = B.\text{true}; B_1.\text{false} = B.\text{false}; \} B_1) \quad (6.71)$$

其中， $B_1.\text{true}$ 中存放 B_1 的值为真时前往的指令的标号。而 $B_1.\text{code}$ 本身构成了 $B.\text{code}$ ，因此 B_1 为真时前往的指令实际就是 B 为真时前往的指令，因此， $B_1.\text{true} = B.\text{true}$ 。同理， $B_1.\text{false} = B.\text{false}$ ；

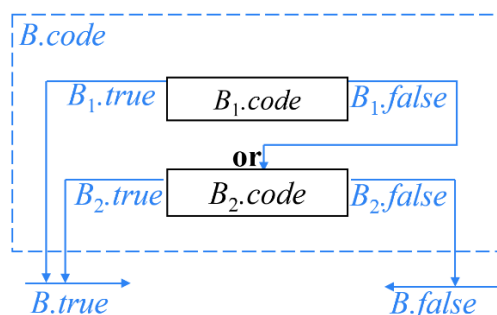
$B \rightarrow \text{not } B_1$ 的翻译方案

根据 $B \rightarrow \text{not } B_1$ 的语义， B 与 B_1 的值相反，因此，不需要为 $B \rightarrow \text{not } B_1$ 产生新的代码，只需将 B 中的真假出口对换，即可分别得到 B_1 的真假出口，即

$$B \rightarrow \text{not} \{ B_1.\text{true} = B.\text{false}; B_1.\text{false} = B.\text{true}; \} B_1 \quad (6.72)$$

$B \rightarrow B_1 \text{ or } B_2$ 的翻译方案

我们首先根据产生式 $B \rightarrow B_1 \text{ or } B_2$ 来构造代码结构图。 B 的代码主要由 $B_1.\text{code}$ 和 $B_2.\text{code}$ 两部分顺序连接构成。另外，还需要在图中标记出各文法符号的属性信息。这里涉及到的语义属性有 6 个，即 $B.\text{true}$ 、 $B.\text{false}$ 、 $B_1.\text{true}$ 、 $B_1.\text{false}$ 、 $B_2.\text{true}$ 和 $B_2.\text{false}$ 。 $B.\text{true}$ 和 $B.\text{false}$ 是父节点的两个继承属性，分别指向 B 的真出口和假出口，如图 6-30 所示。

图 6-30 $B \rightarrow B_1 \text{ or } B_2$ 的代码结构

$B_1.true$ 里存放的是 B_1 为真时应该前往的指令的标号。根据 $B \rightarrow B_1 \text{ or } B_2$ 的语义, B_1 为真时, 整个 B 就为真, 因此不用继续执行 $B_2.code$ 去判断 B 是否为真, 而是直接前往 B 的真出口, 因此 $B_1.true$ 与 $B.true$ 指向的是同一个指令。

$B_1.false$ 里存放的是 B_1 为假时应该前往的指令的标号。根据 $B \rightarrow B_1 \text{ or } B_2$ 的语义, B_1 为假时, 需要根据 B_2 判断 B 的真值, 因此 $B_1.false$ 指向 $B_2.code$ 的第 1 条指令。所以只要控制进入 $B_2.code$ 就隐含说明 B_1 的值为假。

$B_2.true$ 里存放的是 B_2 为真时应该前往的指令的标号。根据 $B \rightarrow B_1 \text{ or } B_2$ 的语义, B_2 为真时, 整个 B 就为真, 因此 $B_2.true$ 与 $B.true$ 指向的是同一个指令。

$B_2.false$ 里存放的是 B_2 为假时应该前往的指令的标号。 B_2 为假时, 整个 B 就为假 (只要控制进入 $B_2.code$ 就隐含说明 B_1 的值为假), 因此 $B_2.false$ 与 $B.false$ 指向的是同一个指令。

根据图 6-30 所示的代码结构图, 可以编写该产生式对应的语义动作:

$$B \rightarrow \{ B_1.true = B.true; B_1.false = newlabel(); \} B_1 \\ \text{or } \{ B_2.true = B.true; B_2.false = B.false; label(B_1.false); \} B_2 \quad (6.73)$$

在开始分析产生式右部第 1 个非终结符 B_1 之前, 需要先计算其继承属性, 即 $B_1.true$ 和 $B_1.false$ 。根据图 6-30, $B_1.true$ 与 $B.true$ 指向的是同一条指令, 因此令 $B_1.true = B.true$ 。

$B_1.false$ 中存放的是 $B_2.code$ 的第 1 条指令的标号。但是在 B_1 未分析完之前, 还不能确定 $B_2.code$ 的第 1 条指令的标号是多少。为此, 调用 `newlabel()` 函数生成一个用于存放标号的临时变量, 并将函数的返回值, 也就是新生成的临时变量的地址赋给 $B_1.false$ 。

根据图 6-30, $B_2.true$ 与 $B.true$ 指向的是同一条指令, $B_2.false$ 与 $B.false$ 指向的是同一个指令。因此令 $B_2.true = B.true$, $B_2.false = B.false$ 。从图中还可以看出, $B_2.code$ 的入口处有一个导入的箭头, 也就是说 $B_1.false$ 在等待 $B_2.code$ 的第 1 条指定的标号。于是调用 `label( $B_1.false$ )` 函数, 将下一条三地址指令 (即

$B_2.code$ 的第 1 条指令) 的标号存放到 $B_1.false$ 中。

$B \rightarrow B_1 \text{ and } B_2$ 的翻译方案

$B \rightarrow B_1 \text{ and } B_2$ 的翻译方案与 $B \rightarrow B_1 \text{ or } B_2$ 的翻译方案类似, 这里不再赘述。代码结构图如图 6-31 所示。翻译方案为

$$B \rightarrow \{ B_1.true = newlabel(); B_1.false = B.false; \} B_1 \\ \text{and} \{ B_2.true = B.true; B_2.false = B.false; label(B_1.true); \} B_2 \quad (6.74)$$

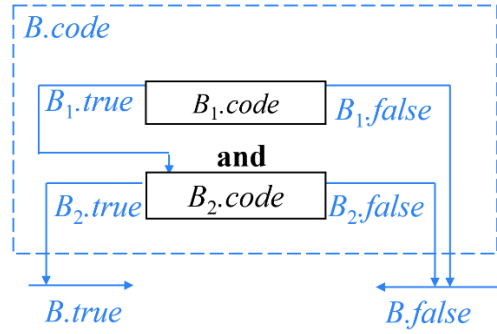


图 6-31 $B \rightarrow B_1 \text{ and } B_2$ 的代码结构

以上介绍了所有控制流语句的 SDT, 汇总后如图 6-32 所示。

- (1) $P \rightarrow \{ S.next = newlabel(); \} S \{ label(S.next); \}$
- (2) $S \rightarrow \{ S_1.next = newlabel(); \} S_1 \{ S_2.next = S.next; label(S_1.next); \} S_2$
- (3) $S \rightarrow \text{if } \{ B.true = newlabel(); B.false = newlabel(); \} B$
 $\quad \text{then } \{ S_1.next = S.next; label(B.true); \} S_1 \{ gen('goto' S.next) \}$
 $\quad \text{else } \{ S_2.next = S.next; label(B.false); \} S_2$
- (4) $S \rightarrow \text{if } \{ B.true = newlabel(); B.false = S.next; \} B$
 $\quad \text{then } \{ S_1.next = S.next; label(B.true); \} S_1$
- (5) $S \rightarrow \text{while } \{ B.true = newlabel(); B.false = S.next;$
 $\quad B.begin = newlabel(); label(B.begin); \} B$
 $\quad \text{do } \{ S_1.next = B.begin; label(B.true); \} S_1 \{ gen('goto' B.begin); \}$
- (6) $B \rightarrow E_1 \text{ relop } E_2 \{ gen('if' E_1.addr \text{ relop } E_2.addr 'goto' B.true);$
 $\quad gen('goto' B.false); \}$
- (7) $B \rightarrow \text{true} \{ gen('goto' B.true); \}$
- (8) $B \rightarrow \text{false} \{ gen('goto' B.false); \}$
- (9) $B \rightarrow (\{ B_1.true = B.true; B_1.false = B.false; \} B_1)$
- (10) $B \rightarrow \text{not} \{ B_1.true = B.false; B_1.false = B.true; \} B_1$
- (11) $B \rightarrow \{ B_1.true = B.true; B_1.false = newlabel(); \} B_1$
 $\quad \text{or } \{ B_2.true = B.true; B_2.false = B.false; label(B_1.false); \} B_2$
- (12) $B \rightarrow \{ B_1.true = newlabel(); B_1.false = B.false; \} B_1$
 $\quad \text{and } \{ B_2.true = B.true; B_2.false = B.false; label(B_1.true); \} B_2$

图 6-32 控制流语句的 SDT

将图 6-19 和图 6-32 所示的 SDT 作为一个整体考虑, 从其设计过程来看,

它对应的 SDD 是一个 L-SDD。图 6-19 所示的赋值语句的 SDD 只定义了综合属性 ($E.addr$ 、 $L.array$ 、 $L.type$ 和 $L.offset$)，图 6-32 所示的分支、循环语句只定义了继承属性 ($B.true$ 、 $B.false$ 和 $S.next$)，内嵌的语义动作都是用来计算其后非终结符的继承属性的，这些继承属性的值都只依赖父节点的继承属性或左兄弟节点的属性值（未依赖右兄弟的属性）。由于这个 SDT 的基础文法并不是 LL(1)文法，因此不能在语法分析过程中实现这个 SDT。我们在 4.4.5 节中已经分析过，该基础文法可以使用 LR 分析技术，因此，我们使用 SDT 的通用实现方法，即首先建立一棵语法分析树，然后按照从左到右的深度优先顺序来执行这些动作。

例 6.6 假设要翻译如下程序片段：

```

while  a < b  do
    if  c < d  then
        x = y + z;
    else
        x = y - z;

```

(6.75)

通过前面的分析，可以通过修改图 6-32 所示的 SDT，引入标记非终结符，从而在 LR 语法分析过程中实现这个 SDT。但是这里为了更加直观地说明该 SDT，我们将基于 SDT 的通用的实现方法（即首先建立一棵语法分析树，然后按照从左到右的深度优先顺序来执行这些动作，即，将语法分析跟语义分析分开进行）来说明这个 SDT。

该程序片段对应的语法分析树如图 6-33 所示。令 $a < b$ 对应于非终结符 B ， $c < d$ 对应于非终结符 B_1 ， $x = y + z$ 对应于非终结符 S_1 ， $x = y - z$ 对应于非终结符 S_2 ，整个 if 语句对应于非终结符 S_3 。按照从左到右的深度优先顺序来执行图 6-19 和图 6-32 所示翻译方案中的语义动作。

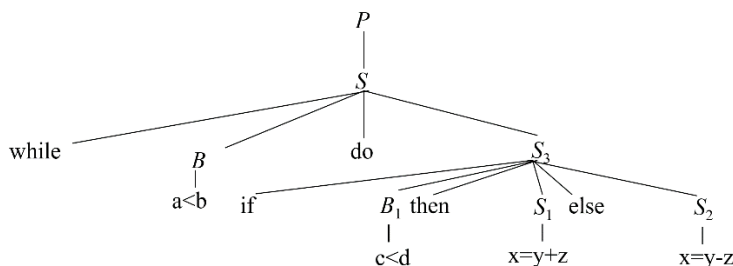


图 6-33 语句(6.75)的翻译过程(1)

根节点 P 对应于对图 6-32 中产生式 “ $P \rightarrow \{S.next = newlabel();\} S \{label(S.next);\}$ ” 的应用。该产生式中涉及到的语义动作的会作为附加子节点

加入到分析树中（如图 6-34 所示）。按照从左到右的深度优先的顺序，首先会执行 S 之前的语义动作 $S.next = newlabel()$ 来计算 S 的继承属性值。假设 $newlabel()$ 的返回值（即申请的临时变量）为 L_1 ，则 $S.next = L_1$ 。按照惯例，我们将计算好的继承属性值标注在 S 的左下方。图中为节省空间，将 $S.next$ 简写为 $S.n$ 。

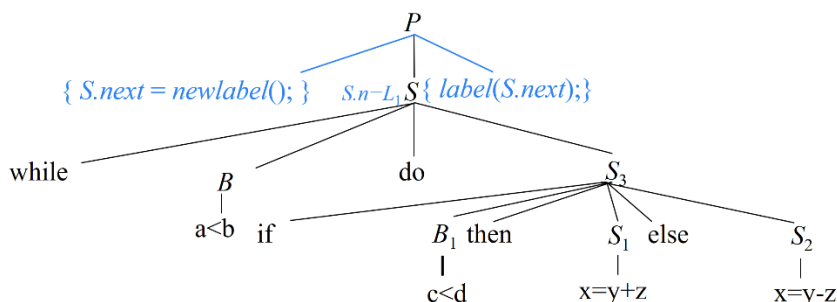


图 6-34 语句(6.75)的翻译过程(2)

接下来，按顺序开始分析 S ，对应于对图 6-32 中产生式(5)的应用。根据该产生式，读入 $while$ 之后执行一段语义动作。首先计算 B 的继承属性 $B.begin = newlabel()$ 。假设本次调用 $newlabel()$ 的返回值为 L_2 ，则 $B.begin = L_2$ ，如图 6-35 所示。然后调用 $label(B.begin)$ 函数将下一条指令的标号（不妨设为 1）存入 $B.begin$ ，即 $B.begin = 1$ 。接下来计算 B 的继承属性 $B.true = newlabel()$ 。假设本次调用 $newlabel()$ 的返回值为 L_3 ，则 $B.true = L_3$ 。最后计算 $B.false = S.next = L_1$ 。

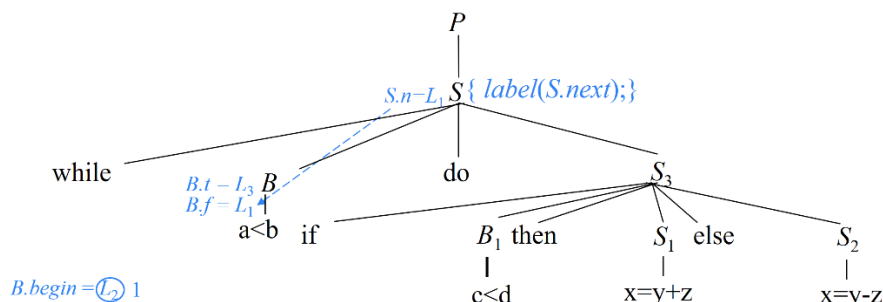


图 6-35 语句(6.75)的翻译过程(3)

这段语义动作执行完之后，按顺序接下来分析 B ，对应于对图 6-32 中产生式(6)的应用。该产生式对应的语义动作生成一对跳转指令（指令标号为 1 和 2），分别跳转到 $B.true$ （即 L_3 ）和 $B.false$ （即 L_1 ）：

1: if $a < b$ goto L_3 (6.76)

2: goto L_1 (6.77)

分析完 B 之后，继续读入 do ，并执行紧跟其后的一段语义动作。首先调用

$label(B.true)$ 函数将下一条指令的标号（即标号 3）存入 $B.true$ ，如图 6-36 所示。接下来计算 S_3 的继承属性 $next$ 的值，该值依赖于 $B.begin$ ，即 $S_3.next = L_2$ 。

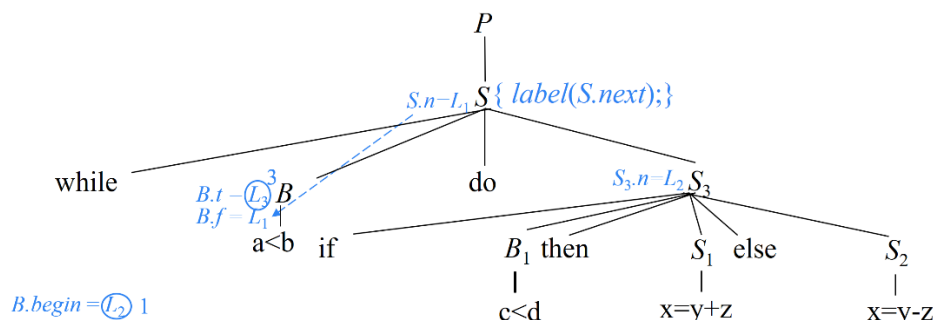


图 6-36 语句(6.75)的翻译过程(4)

这段语义动作执行完之后，按顺序接下来分析 S_3 ，对应于对图 6-32 中产生式(3)的应用。根据该产生式对应的翻译方案，读入 if 之后，首先执行一段语义动作计算 B_1 的两个继承属性 $B_1.true = newlabel()$ 和 $B_1.false = newlabel()$ 。假设这两次调用 $newlabel()$ 的返回值分别为 L_4 和 L_5 ，则 $B_1.true = L_4$ ， $B_1.false = L_5$ ，如图 6-37 所示。

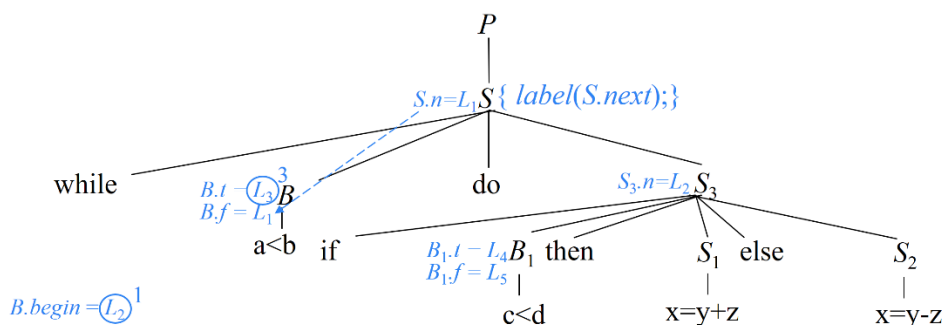


图 6-37 语句(6.75)的翻译过程(5)

这段语义动作执行完之后，按顺序接下来分析 B_1 ，对应于对图 6-32 中产生式(6)的应用。该产生式对应的语义动作生成一对跳转指令（按顺序，指令标号为 3 和 4），分别跳转到 $B_1.true$ （即 L_4 ）和 $B_1.false$ （即 L_5 ）：

3: if $c < d$ goto L_4 (6.78)

4: goto L_5 (6.79)

分析完 B_1 之后，继续读入 then，并执行紧跟其后的一段语义动作。首先调用 $label(B_1.true)$ 函数将下一条指令的标号（即标号 5）存入 $B_1.true$ ，如图 6-38 所示。接下来计算 S_1 的继承属性 $next$ 的值，该值继承于父节点 S_3 的属性值 $next$ ，因此， $S_1.next = L_2$ 。

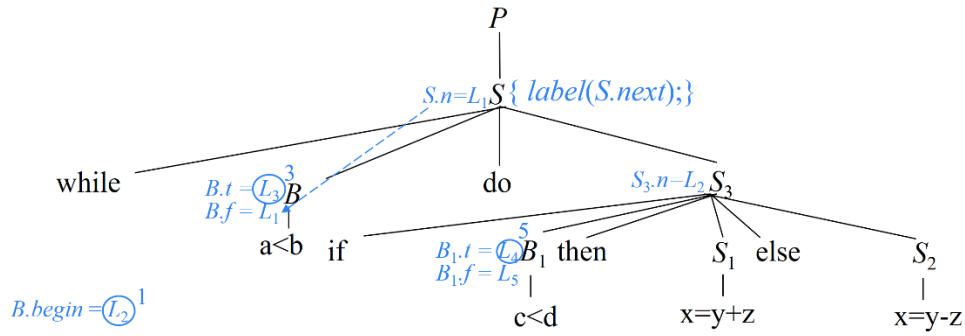


图 6-38 语句(6.75)的翻译过程(6)

这段语义动作执行完之后，按顺序接下来分析 S_1 。根据图 6-19 中赋值语句的翻译方案，将 $x=y+z$ 翻译成两条三地址指令

$$5: t_1 = y + z \quad (6.80)$$

$$6: x = t_1 \quad (6.81)$$

分析完 S_1 之后，接下来执行紧跟其后的语义动作 $gen('goto' S.next)$ 。其中的 S 表示 S_1 的父节点（即 S_3 ），其 $next$ 属性值为 L_2 ，因此，生成的指令为

$$7: goto L_2 \quad (6.82)$$

接下来，继续读入 $else$ ，并执行紧跟其后的一段语义动作。首先调用 $label(B_1.false)$ 函数将下一条指令的标号（即标号 8）存入 $B_1.false$ ，如图 6-39 所示。接下来计算 S_2 的继承属性 $next$ 的值，该值继承于父节点 S_3 的属性值 $next$ ，因此， $S_2.next = L_2$ 。

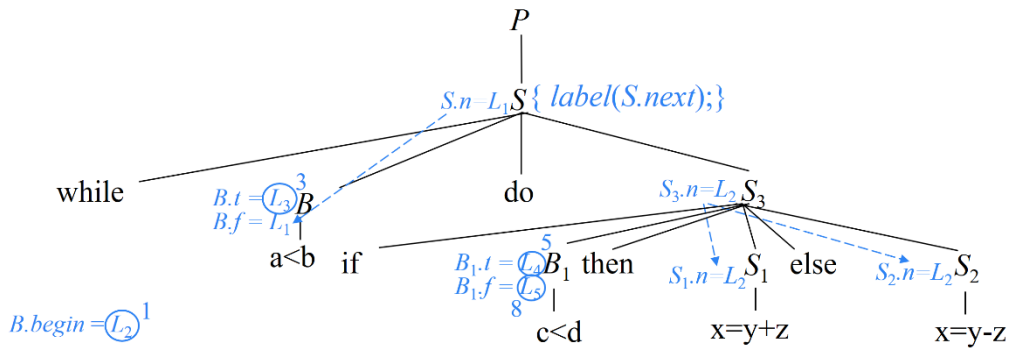


图 6-39 语句(6.75)的翻译过程(7)

这段语义动作执行完之后，按顺序接下来分析 S_2 。根据图 6-19 中赋值语句的翻译方案，将 $x=y-z$ 翻译成两条三地址指令

$$8: t_2 = y - z \quad (6.83)$$

$$9: x = t_2 \quad (6.84)$$

这样，整个 S_3 语句（即 if-then-else 语句）就分析完了，控制又回到 S_3 的

父节点 S 所在的语句（即 while-do 语句）。根据图 6-32 中产生式(5)的翻译方案，分析完 S_3 之后，接下来执行紧跟其后的语义动作 $gen('goto' B.begin)$ ，因此生成指令

10: goto L_1 (6.85)

这样，整个 S 语句（即 while-do 语句）就分析完了，控制又回到 S 的父节点 P 所在的语句。根据图 6-32 中产生式(1)的翻译方案，分析完 S 之后，接下来执行紧跟其后的语义动作 $label(S.next)$ 将下一条指令的标号（即标号 11）存入 $S.next$ ，如图 6-40 所示。

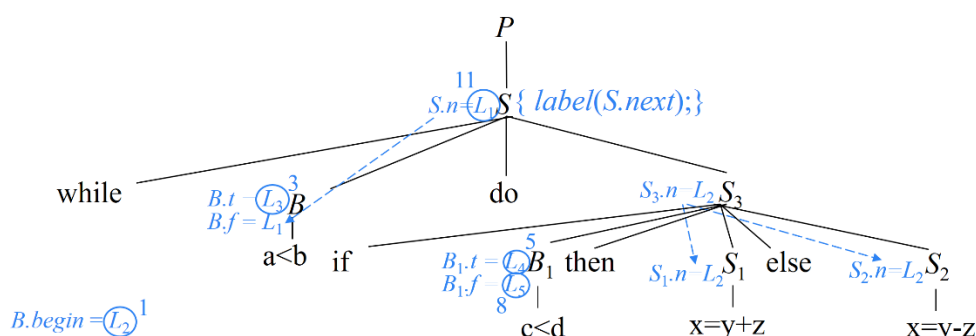


图 6-40 语句(6.75)的翻译过程(8)

正如前面提到的，(6.76)~(6.85)都是一些临时的指令，其中的 $L_1 \sim L_5$ 代表的都是标号的存放地址，而并非标号本身。而图 6-40 中绿色的数字是翻译过程中填入到 $L_1 \sim L_5$ 中的标号。接下来，从 $L_1 \sim L_5$ 中将这些标号取出来生成正式的三地址指令，如图 6-41 所示。可以看出，该翻译结果语句(6.75)的语义是完全一致的。

三地址码	四元式
1: if a < b goto 3	1: (j<, a, b, 3)
2: goto 11	2: (j, -, -, 11)
3: if c < d goto 5	3: (j<, c, d, 5)
4: goto 8	4: (j, -, -, 8)
5: $t_1 = y + z$	5: (+, y, z, t_1)
6: $x = t_1$	6: (=, t_1 , -, x)
7: goto 1	7: (j, -, -, 1)
8: $t_2 = y - z$	8: (-, y, z, t_2)
9: $x = t_2$	9: (=, t_2 , -, x)
10: goto 1	10: (j, -, -, 1)
11:	11:

图 6-41 语句(6.75)的三地址码及四元式序列

□

6.3.3 避免生成冗余的 goto 指令

在例 6.6 中, while 语句(6.75)被翻译成图 6-41 所示的三地址码。根据图 6-29 所示的 while 语句的代码结构图, 指令 1 对应 B 的第一条指令 (如图 6-42 所示, 由 $B.first$ 指示), 指令 3 是 B 的真出口 (即 S_1 的第一条指令, 由 $S_1.first$ 指示)。1 和 2 这一对指令表示: 如果 B 为真, 则跳往 B 的真出口 (即紧跟在 $B.code$ 后面的指令 3); 否则, 跳往 B 的假出口 (即指令 11)。换句话说, 也就是 B 为假时跳走, B 为真时顺序执行后面的指令。因此, 我们可以将 1 和 2 这两条跳转指令替换成一条跳转指令 “ifFalse $a < b$ goto 11”。该指令表示: 布尔表达式 $a < b$ 为假时跳走, 否则不用跳, 顺序执行后面的指令 (即 “if $c < d$ goto 5”), 从而省略了一条跳转指令。ifFalse 指令利用了控制流在指令序列中会从一条指令自然流动到下一条指令的性质, 使得在这个例子中, 当 B 的值为真时, 控制流直接 “穿越” 到标号 3, 从而减少了一条跳转指令。

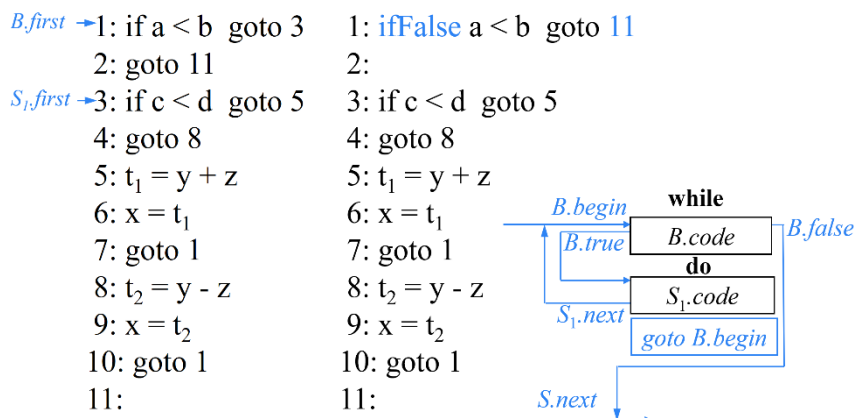


图 6-42 避免生成冗余的 goto 指令示例(1)

类似的, 图 6-41 中的指令 3~9 对应着的 while 语句(6.75)中内嵌的 if-then-else 分支语句。根据图 6-26 所示的 if-then-else 语句的代码结构图, 指令 3 对应 B 的第一条指令 (如图 6-43 所示, 由 $B.first$ 指示), 指令 5 是 B 的真出口 (即 S_1 的第一条指令, 由 $S_1.first$ 指示), 指令 8 是 B 的假出口 (即 S_2 的第一条指令, 由 $S_2.first$ 指示)。3 和 4 这一对指令表示: 如果 B 为真, 则跳往 B 的真出口 (即紧跟在 $B.code$ 后面的指令 5); 否则, 跳往 B 的假出口 (即指令 8)。换句话说, 也就是 B 为假时跳走, B 为真时顺序执行后面的指令。因此, 我们可以将 3 和 4 这两条跳转指令替换成一条跳转指令 “ifFalse $c < d$ goto 8”。该指令表示: 布尔表达式 $c < d$ 为假时跳走, 否则不用跳, 顺序执行后面的指令 (即 “ $t_1 = y + z$ ”), 从而省略了一条跳转指令。

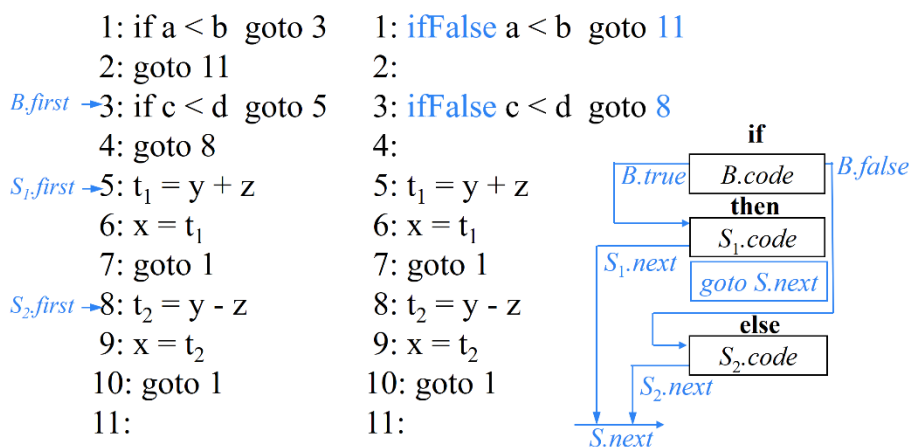


图 6-43 避免生成冗余的 goto 指令示例(2)

可见，在 if 和 while 这类语句的代码结构图中， S_1 的代码紧跟在 B 的代码之后，因此不用生成这条跳往真出口指令，只生成一条 B 值为假的跳转指令， B 值为真时直接顺序执行下一条指令。

为了达到这样的效果，我们需要修改图 6-32 中的 SDT。

修改 if-then 语句的翻译方案

对于产生式 $S \rightarrow \text{if } B \text{ then } S_1$ ，根据其代码结构图 6-28， B 的真出口（即 $S_1.code$ 的第 1 条指令）就紧跟在 $B.code$ 之后，因此不用生成跳往真出口的指令，而只生成跳往假出口的指令即可。也就是说， B 为假时跳走， B 为真时不用跳转，顺序执行后面的指令即可。为此，令 $B.true = \text{fall}$ 。fall 表示不生成（即省略）相应的跳转指令。其他语义动作之不变，即，新的语义规则为

$$\begin{aligned}
 S \rightarrow \text{if } \{ & B.true = \text{fall}; B.false = S.next; \} B \\
 & \text{then } \{ S_1.next = S.next; \} S_1
 \end{aligned} \quad (6.86)$$

修改 $B \rightarrow B_1 \text{ or } B_2$ 的翻译方案

对于产生式 $B \rightarrow B_1 \text{ or } B_2$ ，新的语义规则为

$$\begin{aligned}
 B \rightarrow \{ & B_1.true = B.true == \text{fall} ? \text{newlabel}(): B.true; B_1.false = \text{fall}; \} B_1 \text{ or} \\
 & \{ B_2.true = B.true; B_2.false = B.false; \} B_2 \\
 & \{ \text{if } B.true == \text{fall} \text{ then label}(B_1.true); \}
 \end{aligned} \quad (6.87)$$

根据 $B \rightarrow B_1 \text{ or } B_2$ 的代码结构图 6-30，原先的方案令 $B_1.true = B.true$ ，也就是说 $B_1.true$ 继承了父节点 B 的 $true$ 属性值。但是如果 $B.true = \text{fall}$ ，那么令 $B_1.true$ 也跟着 $B.true$ 等于 fall 就不合适了。因为 $B_1.code$ 后面紧跟着 $B_2.code$ ，如果令 $B_1.true = \text{fall}$ 就会导致 $B_1.code$ 执行完毕之后不跳走而继续执行 $B_2.code$ 。这显然与 or 运算的语义不符。因此当 $B.true = \text{fall}$ 时， $B_1.true$ 需要“另起炉灶”，调用

`newlabel()`生成一个标号指针，使之指向 B 的真出口。因此，在 B_2 分析完毕之后（即整个 B 分析完毕），如果 $B.true == fall$ ，则说明下一条指令就是 B 的真出口（因为无需生成跳往 B 的真出口的跳转指令），那么根据代码结构图 6-30，它也是 B_1 的真出口，因此调用 `label( $B_1.true$ )` 函数将下一条指令的标号存入 $B_1.true$ 。

从图 6-30 还可以看出， B_1 的假出口（即 $B_2.code$ 的第 1 条指令）就紧跟在 $B_1.code$ 之后，因此 B_1 不用生成跳往假出口的指令。也就是说， B 为假时不用跳转，顺序执行后面的指令（即 $B_2.code$ ）即可。因此，令 $B_1.false = fall$ 。

从图 6-30 中还可以看出， B_2 的两个属性 `true` 和 `false` 可以继承父节点 B 的对应属性，即使 $B.true$ 或 $B.false$ 是 `fall` 也不受影响，因为 $B_2.code$ 的末尾就是 $B.code$ 的末尾。

修改 $B \rightarrow B_1 \text{ and } B_2$ 的翻译方案

对于产生式 $B \rightarrow B_1 \text{ and } B_2$ ，新的语义规则为

$$\begin{aligned}
 B \rightarrow \{ & B_1.true = fall; B_1.false = B.false == fall ? newlabel() : B.false; \} B_1 \text{ and} \\
 & \{ B_2.true = B.true; B_2.false = B.false; \} B_2 \\
 & \{ \text{if } B.false == fall \text{ then } label(B_1.false); \} \quad (6.88)
 \end{aligned}$$

根据 $B \rightarrow B_1 \text{ and } B_2$ 的代码结构图 6-31， B_1 的真出口（即 $B_2.code$ 的第 1 条指令）就紧跟在 $B_1.code$ 之后，因此 B_1 不用生成跳往真出口的指令。也就是说， B 为真时不用跳转，顺序执行后面的指令（即 $B_2.code$ ）即可。因此，令 $B_1.true = fall$ 。

原翻译方案令 $B_1.false = B.false$ ，也就是说 $B_1.false$ 继承了父节点 B 的 `false` 属性值。但是如果 $B.false = fall$ ，那么令 $B_1.false$ 跟着 $B.false$ 等于 `fall` 就不合适了。因为 $B_1.code$ 后面紧跟着 $B_2.code$ ，如果令 $B_1.false = fall$ 就会导致 $B_1.code$ 执行完毕之后不跳走而继续执行 $B_2.code$ 。这显然与 `and` 运算的语义不符。因此当 $B.false = fall$ 时， $B_1.false$ 需要“另起炉灶”，调用 `newlabel()` 生成一个标号指针，使之指向 B 的假出口。因此，在 B_2 分析完毕之后（即整个 B 分析完毕），如果 $B.false == fall$ ，则说明下一条指令就是 B 的假出口（因为无需生成跳往 B 的假出口的跳转指令），那么根据代码结构图 6-31，它也是 B_1 的假出口，因此调用 `label( $B_1.false$ )` 函数将下一条指令的标号存入 $B_1.false$ 。

修改 $B \rightarrow E_1 \text{ relop } E_2$ 的翻译方案

对于产生式 $B \rightarrow E_1 \text{ relop } E_2$ ，新的语义规则为

$$\begin{aligned}
 B \rightarrow & E_1 \text{ relop } E_2 \\
 \{ & \text{if } B.true \neq fall \text{ and } B.false \neq fall \text{ then}
 \end{aligned}$$

```

    gen('if' E1.addr relop E2.addr 'goto' B.true); gen('goto' B.false);
  else if B.true ≠ fall then gen('if' E1.addr relop E2.addr 'goto' B.true);
  else if B.false ≠ fall then gen('ifFalse' E1.addr relop E2.addr 'goto' B.false);
  else " }

```

(6.89)

对于 $B.true$ 和 $B.false$ 的取值，无外乎以下 4 种情况：

(1) $B.true \neq fall$ 且 $B.false \neq fall$ ：即，跳往 B 的真出口和假出口的指令都不能省略。因此将产生和原翻译方案一样的两条指令。

(2) $B.true \neq fall$ 且 $B.false = fall$ ：即，跳往真出口的指令不能省略，那么跳往假出口的指令省略。因此只生成一条跳往真出口的 `if` 指令 “`if E1.addr relop E2.addr goto B.true`”，而得当 B 为假时控制穿越到下一条指令。

(3) $B.true = fall$ 且 $B.false \neq fall$ ：即，跳往假出口的指令不能省略，跳往真出口的指令省略。因此只产生一条 `ifFalse` 指令 “`ifFalse E1.addr relop E2.addr goto B.false`”，而得当 B 为真时控制穿越到下一条指令。

(4) $B.true = fall$ 且 $B.false = fall$ ：否则的话，即跳往 B 的真出口和假出口的指令都省略，因此将不生成任何指令，语义动作为空。

例 6.7 假设要翻译如下程序片段：

```

if ( x<100 or x>200 and x!=y )
    x=0;

```

(6.90)

该程序片段对应的语法分析树如图 6-44 所示（注意，该分析树中省略了各语义动作对应的附加子节点）。接下来，按照从左到右的深度优先顺序来执行翻译方案中的语义动作。

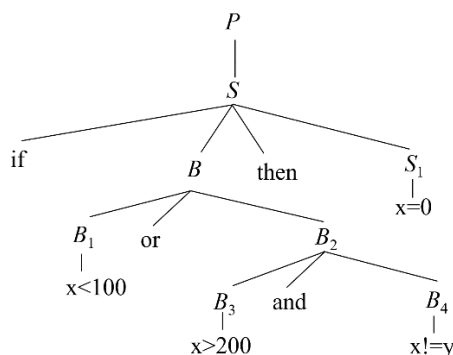


图 6-44 语句(6.75)的翻译过程(1)

根节点 P 对应于对图 6-32 中产生式(1)的应用。按照从左到右的深度优先的顺序，首先会执行 S 之前的语义动作 $S.next = newlabel()$ 来计算 S 的继承属性值。假设 $newlabel()$ 的返回值为 L_1 ，则 $S.next = L_1$ ，如图 6-45 所示。

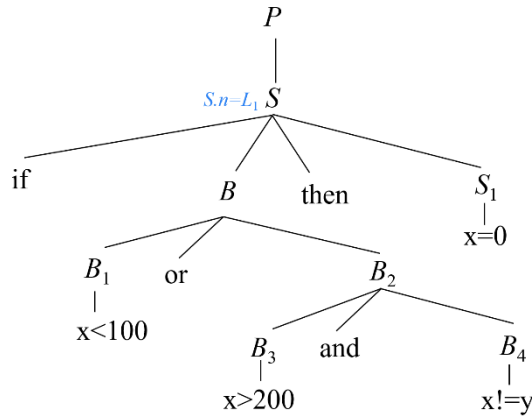


图 6-45 语句(6.75)的翻译过程(2)

接下来，按顺序开始分析 S ，对应于对翻译方案(6.86)的应用。读入 if 之后执行一段语义动作。首先计算 B 的两个继承属性， $B.true = \text{fall}$ ， $B.false = S.next = L_1$ ，如图 6-46 所示。

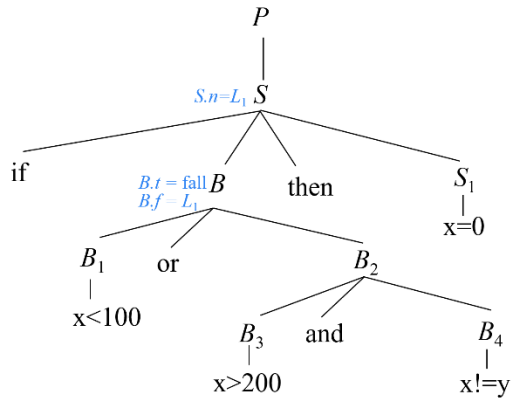


图 6-46 语句(6.75)的翻译过程(3)

按顺序接下来分析 B ，对应于翻译方案(6.87)的应用。首先计算 B_1 的两个继承属性。因为 $B.true = \text{fall}$ ，所以 $B_1.true = \text{newlabel}()$ 。假设本次调用 $\text{newlabel}()$ 的返回值为 L_2 ，则 $B_1.true = L_2$ ，如图 6-47 所示。根据翻译方案(6.87)， $B_1.false = \text{fall}$ 。

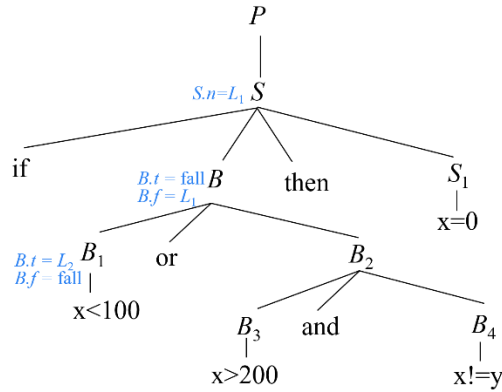


图 6-47 语句(6.75)的翻译过程(4)

按顺序接下来分析 B_1 ，对应于翻译方案(6.89)的应用。由于 $B_1.false = fall$ ，因此只生成一条跳往真出口 L_2 的指令，即

$$\text{if } x < 100 \text{ goto } L_2 \quad (6.91)$$

分析完 B_1 之后，继续读入 or ，并执行紧跟其后的一段语义动作，计算 B_2 的两个继承属性： $B_2.true = B.true = fall$ ， $B_2.false = B.false = L_1$ ，如图 6-48 所示。

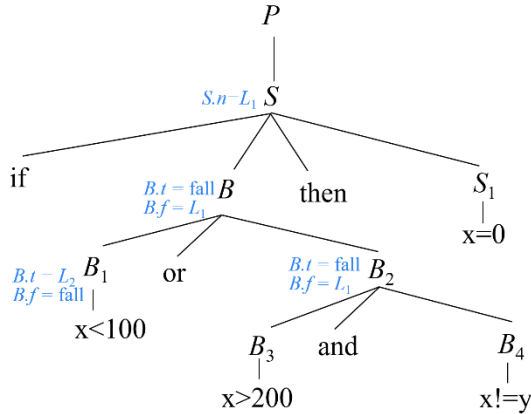


图 6-48 语句(6.75)的翻译过程(5)

按顺序接下来分析 B_2 ，对应于翻译方案(6.88)的应用。首先计算 B_3 的两个继承属性。首先令 $B_3.true = fall$ 。由于 $B_2.false \neq fall$ ，因此， $B_3.false = B_2.false = L_1$ ，如图 6-49 所示。

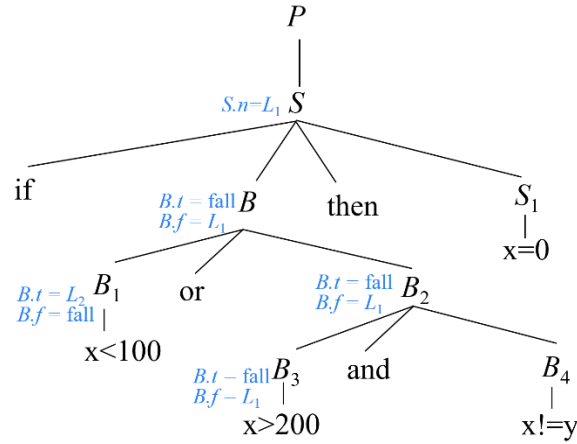


图 6-49 语句(6.75)的翻译过程(6)

按顺序接下来分析 B_3 ，对应于翻译方案(6.89)的应用。由于 $B_4.true=fail$ ，因此只生成一条跳往假出口 L_1 的指令，即

$$ifFalse\ x>200\ goto\ L_1 \quad (6.92)$$

分析完 B_3 之后，继续读入 `and`，并执行紧跟其后的一段语义动作，计算 B_4 的两个继承属性： $B_4.true = B_2.true=fail$ ， $B_4.false = B_2.false = L_1$ ，如图 6-50 所示。

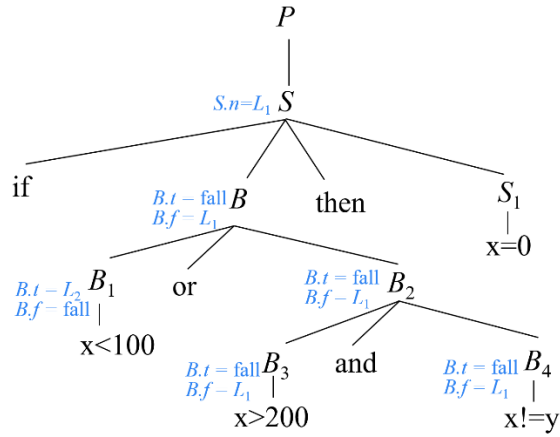


图 6-50 语句(6.75)的翻译过程(7)

按顺序接下来分析 B_4 ，对应于翻译方案(6.89)的应用。由于 $B_4.true=fail$ ，因此只生成一条跳往假出口 L_1 的指令，即

$$ifFalse\ x!=y\ goto\ L_1 \quad (6.93)$$

分析完 B_4 之后，控制回到 B_4 的父节点 B_2 所在的语句。根据翻译方案(6.88)，分析完 B_4 之后，接下来执行一段语义动作。因为父节点 $B_2.false!=fail$ ，因此，无需采取任何操作。

分析完 B_2 之后，控制回到 B_2 的父节点 B 所在的语句。根据翻译方案(6.87)，

分析完 B_2 之后，接下来执行一段语义动作。因为父节点 $B.true == fall$ ，因此，调用 $label(B_1.true)$ 函数将下一条指令（即紧跟在指令(6.93)后的指令）的标号存入 $B_1.true = L_2$ ，即 L_2 指向紧跟在指令(6.93)后的指令。

分析完 B 之后，继续读入 $then$ ，并执行紧跟其后的语义动作，计算 S_1 的继承属性： $S_1.next = S.next = L_1$ ，如图 6-51 所示。

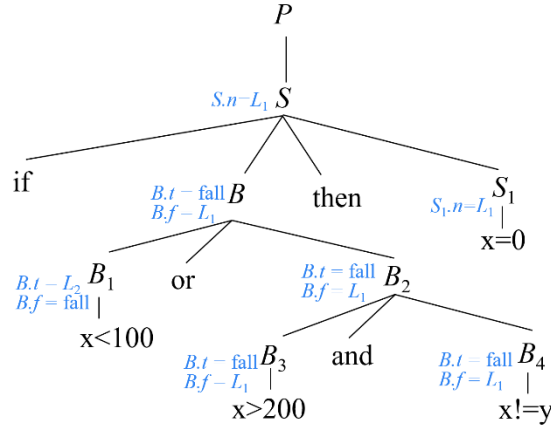


图 6-51 语句(6.75)的翻译过程(8)

按顺序接下来分析 S_1 ，根据图 6-19 中赋值语句的翻译方案，将 $x=0$ 翻译成三地址指令

$$L_2: x = 0 \quad (6.94)$$

分析完 S_1 之后，整个 S 就分析完毕，控制回到 S 的父节点 P 所在的语句。根据图 6-32 中关于 P 产生式的翻译方案，分析完 S 之后，接下来执行语义动作，调用 $label(S.next)$ 函数将下一条指令（即紧跟在指令(6.94)后的指令）的标号存入 $S.next = L_1$ ，即 L_1 指向紧跟在指令(6.94)后的指令。

将指令(6.91)~(6.94)连接起来，得到语句(6.90)的翻译结果

$$\begin{aligned} & \text{if } x < 100 \text{ goto } L_2 \\ & \text{ifFalse } x > 200 \text{ goto } L_1 \\ & \text{ifFalse } x \neq y \text{ goto } L_1 \\ & L_2: x = 0 \\ & L_1: \end{aligned} \quad (6.95)$$

□

6.4 回填

为布尔表达式和控制流语句生成中间代码时，关键问题之一是确定跳转指

令的目标标号。但是在生成跳转指令时，目标标号可能还不能确定。在 6.3 节介绍的控制流语句的 SDT 中，我们解决这个问题的方法是：将标号的存放地址作为继承属性传递到标号可以确定的地方，也就是说，当目标标号的值确定下来以后再填入到相应的地址中。但是这样的做法需要再进行一趟处理，将指令中的标号存放地址改为具体的标号，从而得到正式的三地址指令。

本节介绍一种被称为“回填（backpatching）”的补充性技术。它把一个由跳转指令组成的列表以综合属性的形式进行传递。具体来说，生成一个跳转指令时，暂时不指定该指令跳转的目标标号。这样的指令都被放入由跳转指令组成的列表中，同一个列表中的所有跳转指令具有相同的目标标号。等到能够确定正确的目标标号时，才去填充这些指令的目标标号。

以布尔表达式 B 为例。在为布尔表达式 B 生成中间代码的时候，所生成的跳转指令无外乎分为两类：一类是要跳往 B 的真出口，也就是 B 的值为真的时候前往的指令；一类是要跳往 B 的假出口，也就是 B 的值为假的时候前往的指令。为此，为 B 设置两个综合属性：

- $B.truelist$ ：指向一个包含跳转指令的列表，这些指令最终获得的目标标号就是当 B 为真时控制流应该转向的指令的标号。
- $B.falselist$ ：指向一个包含跳转指令的列表，这些指令最终获得的目标标号就是当 B 为假时控制流应该转向的指令的标号。

类似的，为语句 S 设置如下综合属性：

$S.nextlist$ ：包含所有跳转到按照运行顺序紧跟在 S 代码之后的指令的条件或无条件转移指令。

为处理跳转指令的列表，使用下面三个函数：

- $makelist(i)$ ：创建一个只包含 i 的列表， i 是跳转指令的标号，函数返回指向新创建的列表的指针。
- $merge(p_1, p_2)$ ：将 p_1 和 p_2 指向的列表进行合并，返回指向合并后的列表的指针。
- $backpatch(p, i)$ ：将 i 作为目标标号插入到 p 所指列表中的各指令中。

6.4.1 布尔表达式的回填

布尔表达式的回填翻译方案如图 6-52 所示。

```
(1)  $B \rightarrow E_1 \text{ relop } E_2$ 
    {
         $B.truelist = makelist(nextquad);$ 
         $B.falselist = makelist(nextquad+1);$ 
    }
```



```

        gen('if E1.addr relop E2.addr 'goto _');
        gen('goto _');
    }
(2) B → true
    {   B.truelist = makelist(nextquad);
        gen('goto _');
    }
(3) B → false
    {   B.falselist = makelist(nextquad);
        gen('goto _');
    }
(4) B → (B1)
    {   B.truelist = B1.truelist ;
        B.falselist = B1.falselist ;
    }
(5) B → → not B1
    {   B.truelist = B1.falselist;
        B.falselist = B1.truelist;
    }
(6) B → → B1 or M B2
    {   B.truelist = merge(B1.truelist, B2.truelist );
        B.falselist = B2.falselist ;
        backpatch(B1.falselist, M.quad );
    }
(7) B → → B1 and M B2
    {   B.truelist = B2.truelist;
        B.falselist = merge( B1.falselist, B2.falselist );
        backpatch( B1.truelist, M.quad );
    }
(8) M → ε {M.quad = nextquad ; }

```

图 6-52 布尔表达式的回填翻译方案

 $B \rightarrow E_1 \text{ relop } E_2$ 的翻译方案

对于该产生式，翻译的主要任务是：生成一对跳转指令，分别跳往 B 的真出口和假出口。但跳转的目标标号尚不确定，因此暂不填写，等待回填。等待回填的指令需要放到相应的列表里，也就是说，跳往 B 的真出口的指令放到 $B.truelist$ 中，跳往 B 的假出口的指令放到 $B.falselist$ 中。为此，调用 $makelist(nextquad)$ 函数，创建一个只包含 $nextquad$ 的列表（ $nextquad$ 是即将生成的下一条指令、也就是跳往 B 的真出口的指令的标号），并将函数的返回值、

也就是新创建的列表的地址赋给 $B.truelist$ 。

接下来，调用 $makelist(nextquad+1)$ 创建一个只包含 $nextquad+1$ 的列表（显然， $nextquad+1$ 是跳往 B 的假出口的指令的标号），并将函数的返回值、也就是新创建的列表的地址赋给 $B.falselist$ 。

最后，调用 $gen()$ 函数生成这对待回填的跳转指令。

$B \rightarrow true$ 与 $B \rightarrow false$ 的翻译方案

根据产生式 $B \rightarrow true$ ， B 的值已经确定为真，因此，接下来只需生成一条跳往 B 的真出口的指令。为此，调用 $makelist()$ 函数，创建一个只包含 $nextquad$ 的列表，将函数返回的指向新创建的列表的指针赋给 $B.truelist$ 。接下来生成一条待回填的无条件跳转指令 “goto _”。

产生式 $B \rightarrow false$ 的处理方式与 $B \rightarrow true$ 的类似。

$B \rightarrow (B_1)$ 的翻译方案

对于该产生式，翻译的主要任务就是计算 B 的两个综合属性 $B.truelist$ 和 $B.falselist$ 。由于 B 的代码就是由 B_1 的代码本身构成的，因此要跳往 B_1 的真出口的指令实际上就是要跳往 B 的真出口的指令。因此不用为 B 建立新的列表，让 $B.truelist$ 直接指向 $B_1.truelist$ 即可，即 $B.truelist = B_1.truelist$ 。 $B.falselist$ 也是类似地处理。

$B \rightarrow not B_1$ 的翻译方案

由于 B 的取值与 B_1 的取值恰好相反，要跳往 B 的真出口的指令实际上就是要跳往 B_1 的假出口的指令，而要跳往 B_1 的假出口的指令都放在 $B_1.falselist$ 中。因此也不用为 $B.truelist$ 建立新的列表，让它直接指向 $B_1.falselist$ 即可，即 $B.truelist = B_1.falselist$ 。 $B.falselist$ 也是类似地处理。

$B \rightarrow B_1 or B_2$ 的翻译方案

我们首先根据产生式 $B \rightarrow B_1 or B_2$ 来构造代码结构图，并在图中标记出各文法符号的属性信息，如图 6-53 所示。这里涉及到的语义属性有 6 个，即 $B.truelist$ 、 $B.falselist$ 、 $B_1.truelist$ 、 $B_1.falselist$ 、 $B_2.truelist$ 和 $B_2.falselist$ 。

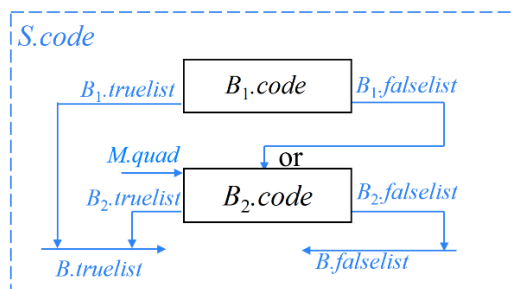


图 6-53 $B \rightarrow B_1 \text{ or } B_2$ 的代码结构

$B.truelist$ 中存放的是跳往 B 的真出口（即 B 的值为真时前往的指令）的指令。都有哪些指令会跳往 B 的真出口？

我们知道， $B_1.truelist$ 中存放的是要跳往 B_1 真出口的指令。由于 B_1 为真时，整个布尔表达式 B 就为真，因此， B_1 中要跳往 B_1 真出口的指令实际上也正是 B 中的一些要跳往 B 的真出口的指令（ $B_1.code$ 是 $B.code$ 的一部分）。因此 $B_1.truelist$ 中的指令也应该放在 $B.truelist$ 中。图中标记为 $B_1.truelist$ 的箭头表示 $B.truelist$ 中的一部分指令来自 $B_1.truelist$ 。

同理， $B_2.truelist$ 中的指令的跳转目标都是 B_2 的真出口。由于 B_2 为真时， B 就为真，因此， $B_2.truelist$ 中的指令实际上就是要跳到 B 的真出口，因此 $B_2.truelist$ 中的指令也在 $B.truelist$ 中。

由此可见， $B.truelist$ 的来源有两个，一个是 $B_1.truelist$ ，一个是 $B_2.truelist$ 。因为导致 B 值为真的原因有两种，要么是 B_1 的值为真，要么是 B_2 的值为真。而 $B.falselist$ 的来源只有一个，即 $B_2.falselist$ 。也就是说， B 的值是否为假取决于 B_2 的值是否为假，而 B_1 的值是否为假决定不了 B 的值是否为假。事实上，如果 B_1 的值为假，需要通过执行 B_2 来进一步判断 B_2 的值，从而决定整个表达式 B 的真值。因此， $B_1.falselist$ 中的指令的跳转目标都是 B_2 的第一条指令。这样，在即将分析 B_2 的时候， B_2 的第一条指令的标号就可以确定下来了。此时，应该用 B_2 的第一条指令的标号去回填 $B_1.falselist$ 中的指令的跳转目标。当然，我们也可以记下 B_2 的第一条指令的标号，然后在归约的时候完成这个回填的动作。

为此，我们在 B_2 之前放置一个标记非终结符 M ，并为 M 设置一个空产生式 $M \rightarrow \epsilon$ ，与该空产生式关联的语义动作的任务就是记下下一条指令的标号。于是，为 M 设置属性 $quad$ ，并且令 $M.quad = nextquad$ 。因为 M 位于 B_2 之前，因此 $M.quad$ 指向的是 B_2 的第一条指令。

接下来，根据代码结构图编写语义动作。该语义动作的主要任务就是计算 B 的综合属性，并执行回填动作。根据代码结构图， $B.truelist$ 由两部分构成，一部分来自于 $B_1.truelist$ ，另一部分来自于 $B_2.truelist$ 。因此，调用 $merge()$ 函数将 $B_1.truelist$ 和 $B_2.truelist$ 合并，将合并后的列表赋给 $B.truelist$ 。而 $B.falselist$ 就等于 $B_2.falselist$ 。另外，调用 $backpatch()$ 函数用 $M.quad$ 回填 $B_1.falselist$ 中的指令的跳转目标。

$B \rightarrow B_1 \text{ and } B_2$ 的翻译方案

$B \rightarrow B_1 \text{ and } B_2$ 的翻译方案的设计思路与 $B \rightarrow B_1 \text{ or } B_2$ 的类似，这里不再赘

述。其代码结构图如图 6-54 所示。

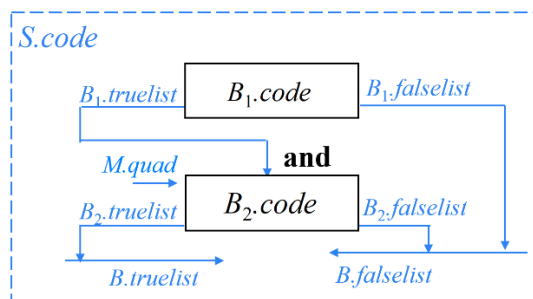


图 6-54 $B \rightarrow B_1 \text{ and } B_2$ 的代码结构

例 6.8 假设要翻译如下布尔表达式：

$$a < b \text{ or } c > d \text{ and } e < f \quad (6.96)$$

图 6-52 所示的 SDT 只定义了综合属性，因此可以采用自底向上翻译，从左向右扫描输入。首先应用产生式(1)，将 $a < b$ 归约为 B_1 ，如图 6-55 所示。根据语义动作，创建一个只包含 *nextquad* 的列表，并将这个列表赋给 $B_1.truelist$ 。假设下一条指令的标号为 100，则 $B_1.truelist$ 中只包含标号为 100 的指令（图中为节省空间，用单词首字母代表该单词，如用 *t* 代表 *truelist*，*f* 代表 *falselist*）。接下来再创建一个只包含 *nextquad*+1 的列表，并将这个列表赋给 $B_1.falselist$ ，则 $B_1.falselist$ 中只包含标号为 101 的指令。接下来生成一对跳转指令

$$100: \text{if } a < b \text{ goto } _ \quad (6.97)$$

$$101: \text{goto } _ \quad (6.98)$$

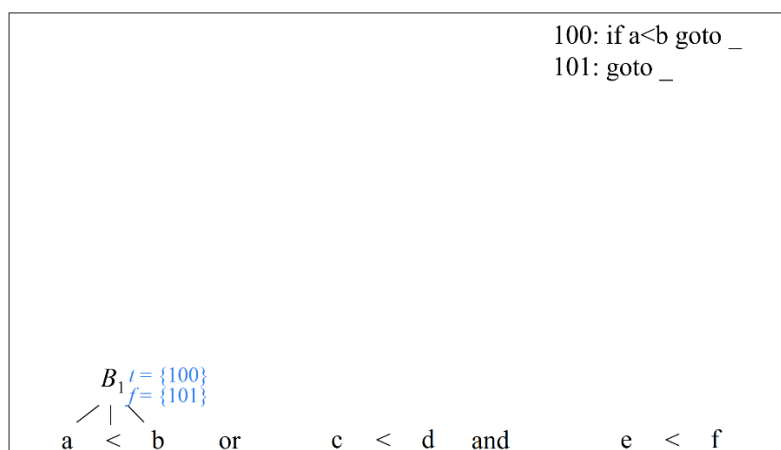


图 6-55 语句(6.96)的翻译过程(1)

接下来继续读入下一输入符号 *or*，这时将触发对产生式(7)的应用。根据该产生式，接下来将归约出标记非终结符 M_1 。归约时，执行 M_1 产生式(9)对应的

语义动作，记录下一条产生式标号 $nextquad$ ，赋给 $M_1.quad$ 。此时，下一条指令标号为 102，因此 $M_1.quad=102$ ，如图 6-56 所示（ q 为 $quad$ 的首字母）。

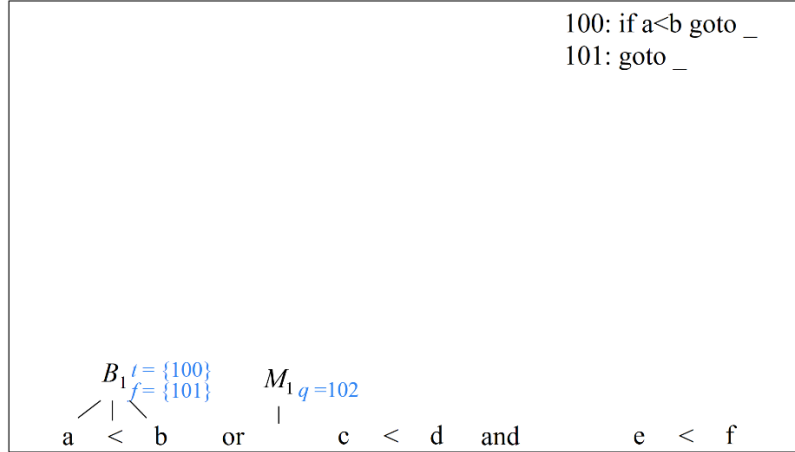


图 6-56 语句(6.96)的翻译过程(2)

接下来，将 $c<d$ 归约为 B_2 。根据产生式(1)对应的语义动作， $B_2.truelist=\{102\}$ ， $B_2.falselist=\{103\}$ ，如图 6-57 所示，并生成一对跳转指令

102: if $c<d$ goto (6.99)

103: goto (6.100)

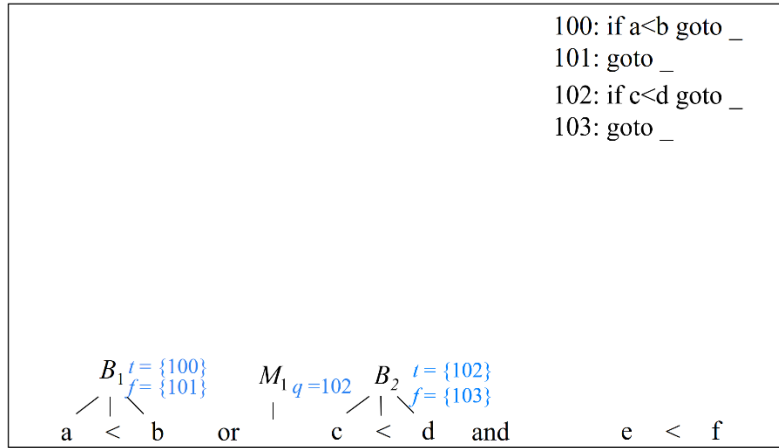


图 6-57 语句(6.96)的翻译过程(3)

接下来读入下一输入符号 and 。根据优先级， and 运算优先级高于 or 运算，因此，当栈顶为符号串 $B_1 or M_1 B_2$ 、栈外为输入符号 and 时，采取移入动作，先执行 and 运算，再执行 or 运算，因此，触发对产生式(8)的应用。根据该产生式，在 and 之后归约出标记非终结符 M_2 。归约时，执行 M_2 产生式(9)对应的语义动作，则 $M_2.quad=104$ ，如图 6-58 所示。

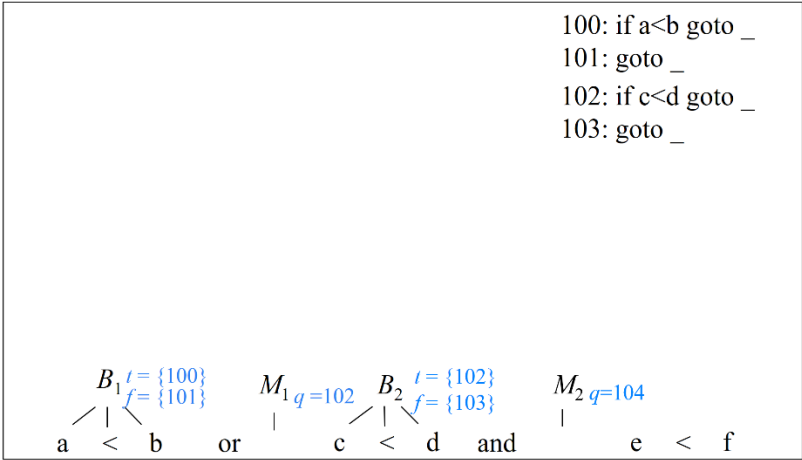


图 6-58 语句(6.96)的翻译过程(4)

接下来, 将 $e \leq f$ 归约为 B_3 。根据产生式(1)对应的语义动作, $B_3.truelist = \{104\}$, $B_3.falselist = \{105\}$, 如图 6-59 所示, 并生成一对跳转指令

104: if e<f goto (6.101)

105: goto (6.102)

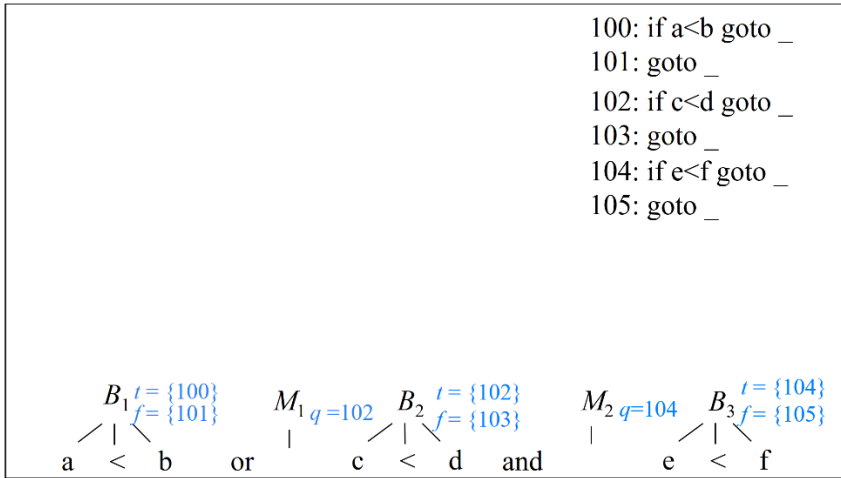


图 6-59 语句(6.96)的翻译过程(5)

接下来, 将 B_2 and $M_2 B_3$ 归约为 B_4 , 如图 6-60 所示。执行产生式(8)对应的语义动作, 首先计算 B_4 的两个综合属性

$$B_4.truelist=B_3.truelist=\{104\} \quad (6.103)$$
$$B_4.falselist = merge(B_2.falselist, B_3.falselist) = \{103, 105\} \quad (6.104)$$

接下来，调用 *backpatch()* 函数，用 *M₂.quad*（也就是 104）回填 *B₂.truelist* 中的跳转指令（即 102）的目标地址。于是有

102: if c<d goto 104 (6.105)

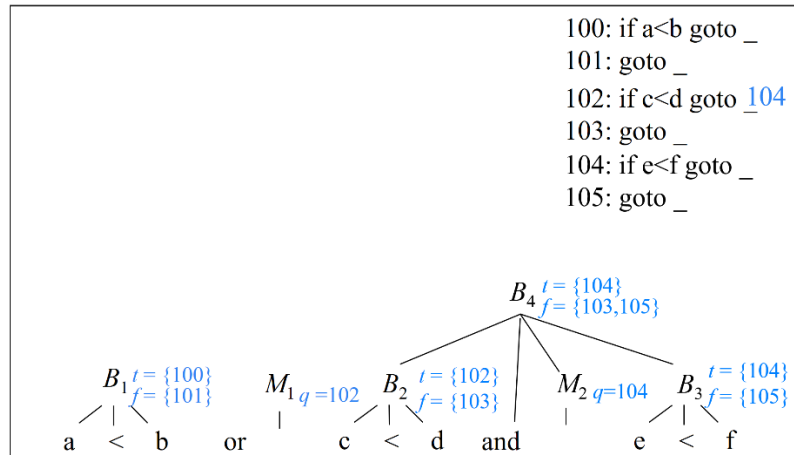


图 6-60 语句(6.96)的翻译过程(6)

接下来，将 B_1 **or** M_1 B_4 归约为 B_5 ，如图 6-61 所示。执行产生式(7)对应的语义动作，得到

$$B_5.truelist = merge(B_1.truelist, B_4.truelist) = \{100, 104\} \quad (6.106)$$

$$B_5.falselist = B_4.falselist = \{103, 105\} \quad (6.107)$$

用 $M_1.quad$ （也就是 102）回填 $B_1.falselist$ 中的跳转指令（即 101）的目标地址。于是有

$$101: goto 102 \quad (6.108)$$

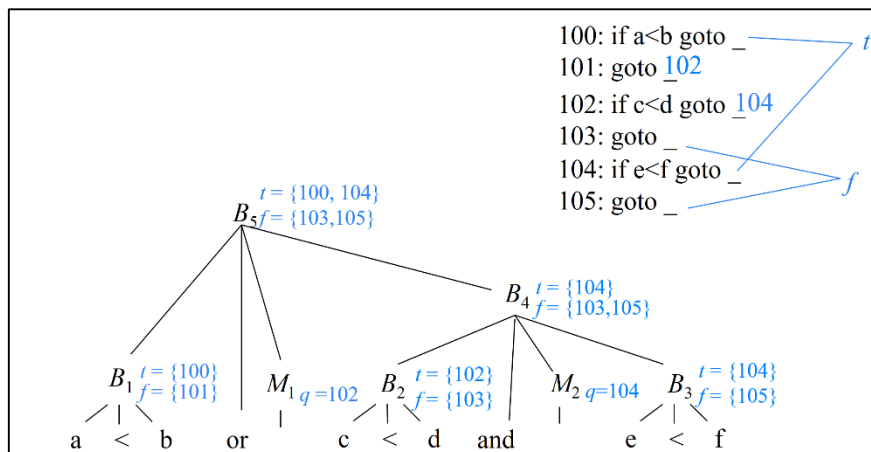


图 6-61 语句(6.96)的翻译过程(7)

这样，布尔表达式(6.96)就被翻译成 6 条跳转指令，其中有 4 条指令的目标标号是空着的，也就是说等待回填。其中 100 和 104 构成了 $B_5.truelist$ ，即这两条指令要跳往整个布尔表达式 B_5 的真出口，等真出口的指令标号确定之后才能回填。类似的，103 和 105 构成了 $B_5.falselist$ ，即这两条指令要跳往整个布尔表

达式 B_5 的假出口，等假出口的指令标号确定之后才能回填。事实上， B_5 的真出口和假出口目前可能还不能确定。因为 B_5 可能是一个更大的语句，比如说 $\text{if } B_5 \text{ then } S_1 \text{ else } S_2$ 语句的一部分。那么刚刚分析完 B_5 、尚未开始分析 S_1 时，无法确定 B_5 的假出口。再比如，假设 B_5 是一个更大的布尔表达式，比如说 $B_5 \text{ or } B_6 \text{ and } B_7$ 的一部分。那么刚刚分析完 B_5 、尚未开始分析 B_5 之后成分时，是无法确定 B_5 的真出口的。□

6.4.2 控制流语句的回填

对于文法(6.52)，翻译布尔表达式 B 的时候，会生成一些跳转指令，这些跳转指令会被放到相应的列表里。事实上，语句 S 中也可能会包含跳转指令。例如， S 本身是一个 if 语句或 while 语句，只要其中包含 B ，就必然会包含跳转指令。其中某些跳转指令可能要跳往紧跟在 S 之后执行的指令，这样的指令我们也要把它放入相应的列表 $list$ 中。为此，我们为 S 设置综合属性 $S.nextlist$ ： $S.nextlist$ 指向一个包含跳转指令的列表，这些指令最终获得的目标标号就是按照运行顺序紧跟在 S 代码之后的指令的标号。

控制流语句的回填翻译方案如图 6-62 所示。

```

(1)  $S \rightarrow \text{if } B \text{ then } M S_1$ 
    {
         $S.nextlist = \text{merge}(B.falselist, S_1.nextlist);$ 
         $\text{backpatch}(B.true\text{list}, M.\text{quad});$ 
    }

(2)  $S \rightarrow \text{if } B \text{ then } M_1 S_1 \text{ else } N M_2 S_2$ 
    {
         $S.nextlist = \text{merge}(\text{merge}(S_1.nextlist, N.nextlist), S_2.nextlist);$ 
         $\text{backpatch}(B.true\text{list}, M_1.\text{quad});$ 
         $\text{backpatch}(B.false\text{list}, M_2.\text{quad});$ 
    }

(3)  $S \rightarrow \text{while } M_1 B \text{ do } M_2 S_1$ 
    {
         $S.nextlist = B.falselist;$ 
         $\text{backpatch}(S_1.nextlist, M_1.\text{quad});$ 
         $\text{backpatch}(B.true\text{list}, M_2.\text{quad});$ 
         $\text{gen}(\text{'goto' } M_1.\text{quad});$ 
    }

(4)  $S \rightarrow S_1 M S_2$ 
    {
         $S.nextlist = S_2.nextlist;$ 
         $\text{backpatch}(S_1.nextlist, M.\text{quad});$ 
    }

(5)  $S \rightarrow \text{id} = E;$ 
    {
         $p = \text{lookup}(\text{id.lexeme});$ 
        if  $p == \text{nil}$  then  $\text{error}();$ 
         $\text{gen}(p \text{'=' } E.\text{addr});$ 
         $S.nextlist = \text{null};$ 
    }
```

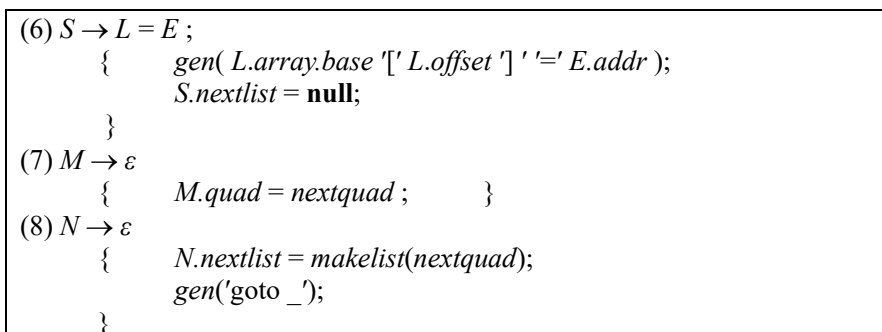



图 6-62 控制流语句的回填翻译方案

if-then 语句的翻译方案

我们首先根据产生式 $S \rightarrow \text{if } B \text{ then } S_1$ 来构造代码结构图，并在图中标记出各文法符号的属性信息，如图 6-63 所示。这里涉及到的语义属性有 4 个，即 $B.true\text{list}$ 、 $B.false\text{list}$ 、 $S_1.next\text{list}$ 和 $S.next\text{list}$ 。

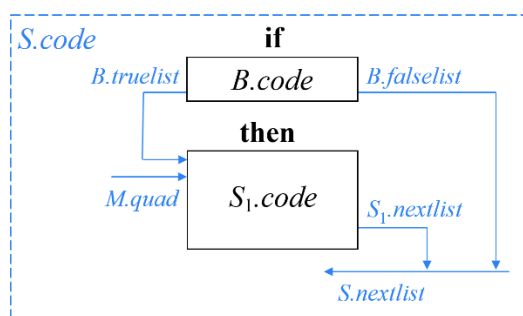


图 6-63 if-then 语句的代码结构

根据 if-then 语句的语义，如果 B 值为真，则执行 $S_1.code$ 的第 1 条指令，所以 $B.true\text{list}$ 中的跳转指令都要跳往 $S_1.code$ 的第 1 条指令。于是，图中标记为 $B.true\text{list}$ 的箭头指向 $S_1.code$ 的第 1 条指令。当即将分析 S_1 的时候， S_1 的第 1 条指令的标号就可以确定下来。此时，可以用 S_1 的第 1 条指令的标号去回填 $B.true\text{list}$ 中的指令的跳转目标。当然，我们也可以记下 S_1 的第一条指令的标号，然后在归约的时候完成这个回填的动作。为此，我们在 S_1 之前放置标记非终结符 M ，其空产生式对应的语义动作用来记录下下一条指令（即 S_1 的第 1 条指令）的标号。

如果 B 值为假，整个 if-then 语句就执行完了，因此， $B.false\text{list}$ 中的指令都是要跳往紧跟在 S 之后执行的第一条指令，因此，也要将它们放入 $S.next\text{list}$ 中。

S_1 中可能也会有一些跳转指令要跳往紧跟在 S_1 之后执行的那条指令。由于 S_1 执行完之后整个 S 就执行完毕，所以紧跟在 S_1 之后执行的那条指令实际上就是紧跟在 S 之后执行的那条指令。因此， $S_1.next\text{list}$ 中的指令也要放如 $S.next\text{list}$

中。

接下来，根据代码结构图编写语义动作。该语义动作的主要任务就是计算 S 的综合属性，并执行回填动作。根据代码结构图， $S.nextlist$ 由两部分构成，一部分来自于 $B.falselist$ ，另一部分来自于 $S_1.nextlist$ 。因此，调用 $merge()$ 函数将 $B.falselist$ 和 $S_1.nextlist$ 合并，将合并后的列表赋给 $S.nextlist$ 。另外，调用 $backpatch()$ 函数用 $M.quad$ 回填 $B.truelist$ 中的指令的跳转目标。

if-then-else 语句的翻译方案

我们首先根据产生式 $S \rightarrow \text{if } B \text{ then } S_1 \text{ else } S_2$ 来构造代码结构图，如图 6-64 所示。这里涉及到的语义属性有 5 个，即 $B.truelist$ 、 $B.falselist$ 、 $S_1.nextlist$ 、 $S_2.nextlist$ 和 $S.nextlist$ 。

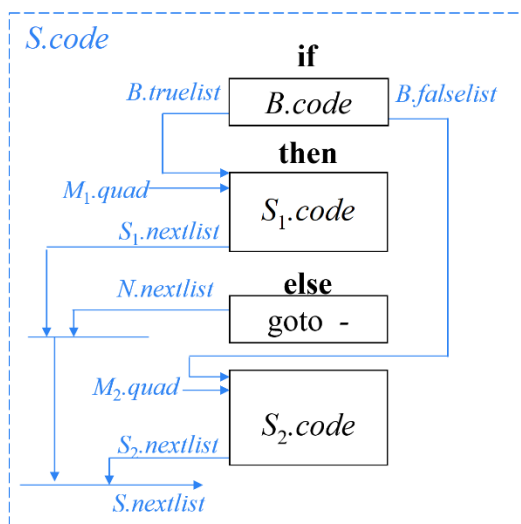


图 6-64 if-then-else 语句的代码结构

根据 if-then-else 语句的语义， S_1 执行完之后整个语句 S 就执行完毕，因此需要生成一条跳往紧跟在 S 之后执行的语句的跳转指令。为了执行这个语义动作，我们在 **else** 之后设置一个标记非终结符 N ，并为 N 设置一个空产生式。与该空产生式关联的语义动作的任务是生成这条跳转指令。但是由于跳转的目标地址需要等待回填，待回填的指令都要放入一个列表中。因此，为 N 也设置一个综合属性 $N.nextlist$ ，并将这条跳转指令放入该列表中。由于该指令要跳往紧跟在 S 之后执行的第 1 条指令，因此， $N.nextlist$ 中的指令也应该放入 $S.Nextlist$ 中。

S_1 中可能有一些跳转指令要跳往紧跟在 S_1 之后执行的那条指令。由于 S_1 执行完之后整个 S 就执行完毕，所以紧跟在 S_1 之后执行的那条指令实际上就是紧跟在 S 之后执行的那条指令。因此， $S_1.nextlist$ 中的指令也要放如 $S.nextlist$ 中。

S_2 及 $S_2.nextlist$ 也是类似的处理。

接下来, 根据代码结构图编写语义动作。该语义动作的主要任务就是计算 S 的综合属性, 并执行回填动作。根据代码结构图, $S.nextlist$ 由 3 部分构成, 分别来自于 $S_1.nextlist$ 、 $S_2.nextlist$ 和 $N.nextlist$ 。因此, 调用 $merge()$ 函数两次将它们合并, 将合并后的列表赋给 $S.nextlist$ 。另外, 调用 $backpatch()$ 函数用 $M_1.quad$ 和 $M_2.quad$ 分别回填 $B.truelist$ 和 $B.falselist$ 中的指令的跳转目标。

while-do 语句的翻译方案

我们首先根据产生式 $S \rightarrow \text{while } B \text{ do } S_1$ 来构造代码结构图, 如图 6-65 所示。这里涉及到的语义属性有 4 个, 即 $B.truelist$ 、 $B.falselist$ 、 $S_1.nextlist$ 和 $S.nextlist$ 。

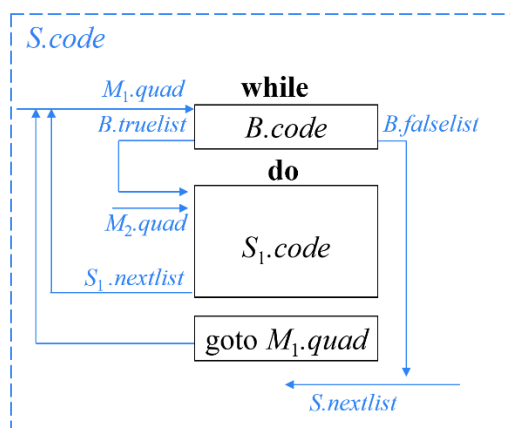


图 6-65 while-do 语句的代码结构

根据 while-do 语句的语义, S_1 执行完之后再重新执行 B , 因此需要生成一条跳 $B.code$ 第 1 条指令的跳转指令。因此, 我们在 B 之前放置标记非终结符 M , 其空产生式对应的语义动作用来记录下下一条指令 (即 $B.code$ 的第 1 条指令) 的标号。

接下来, 根据代码结构图编写语义动作。该语义动作的主要任务就是计算 S 的综合属性, 并执行回填动作, 另外还要生成必要的跳转指令。根据代码结构图, $S.nextlist$ 等于 $B.falselist$ 。接下来调用 $backpatch()$ 函数用 $M_1.quad$ 和 $M_2.quad$ 分别回填 $S_1.nextlist$ 和 $B.truelist$ 中的指令的跳转目标。最后生成跳往 $B.code$ 第 1 条指令的跳转指令。

顺序结构的翻译方案

我们首先根据产生式 $S \rightarrow S_1 S_2$ 来构造代码结构图, 如图 6-66 所示。这里涉及到的语义属性有 3 个, 即 $S_1.nextlist$ 、 $S_2.nextlist$ 和 $S.nextlist$ 。其中 $S_1.nextlist$ 中存放的是要跳往紧跟在 $S_1.code$ 之后执行的那条指令的指令。根据

顺序结构的语义， S_1 执行完毕之后顺序执行 S_2 。因此，图中标记为 $S_1.nextlist$ 的箭头指向 $S_2.code$ 的第 1 条指令。当即将分析 S_2 的时候， S_2 的第 1 条指令的标号就可以确定下来。此时，可以用 S_2 的第 1 条指令的标号去回填 $S_1.nextlist$ 中的指令的跳转目标。当然，我们也可以记下 S_2 的第一条指令的标号，然后在归约的时候完成这个回填的动作。为此，我们在 S_2 之前放置标记非终结符 M ，其空产生式对应的语义动作用来记录下下一条指令（也就是 S_2 的第 1 条指令）的标号。

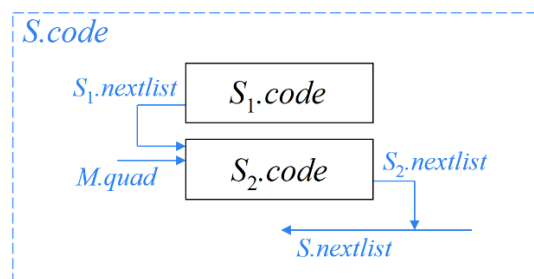


图 6-66 顺序结构的代码结构

$S_2.nextlist$ 中存放的是要跳往紧跟在 $S_2.code$ 之后执行的那条指令的指令。根据顺序结构的语义， S_2 执行完毕之后整个 S 就执行完毕，所以紧跟在 S_2 之后执行的那条指令实际上就是紧跟在 S 之后执行的那条指令。因此， $S_2.nextlist$ 中的指令也要放如 $S.nextlist$ 中。

接下来，根据代码结构图编写语义动作。该语义动作的主要任务就是计算 S 的综合属性，并执行回填动作。根据代码结构图， $S.nextlist$ 等于 $S_2.nextlist$ 。接下来调用 `backpatch()` 函数用 $M.quad$ 回填 $S_1.nextlist$ 中的指令的跳转目标。

赋值语句的翻译方案

对于产生式 “ $S \rightarrow id = E ; | L = E$ ”，面向回填技术的翻译任务就是计算 $S.nextlist$ 。 $S.nextlist$ 中存放的是 S 中要跳往紧跟在 S 之后执行的那条指令的指令。由于 S 生成的是赋值语句，其本身不包含跳转指令，所以 $S.nextlist$ 为空。

回填技术的编写要点

(1) 首先进行文法改造，在 `list` 箭头指向的位置（即需要回填的位置）设置标记非终结符 M 。

(2) 接下来，在每个产生式末尾的语义动作中主要完成以下操作：

- 计算综合属性。
- 调用 `backpatch()` 函数回填各个列表 `list`。
- 生成必要的附加指令。

例 6.9 假设要翻译如下程序片段：

```

while a < b do
  if c < 5 then
    while x > y do z = x + 1;
  else
    x = y;

```

(6.109)

图 6-62 所示的 SDT 只定义了综合属性，因此可以采用自底向上翻译。从左向右扫描输入。当读入关键字 `while` 以后，触发了对产生式(3)的应用从而识别 `while` 语句。首先为 `while` 创建叶节点，然后归约出 M_1 。 M_1 的任务是记录下一条指令的标号。我们不妨假设下条指令从 100 号开始，则 $M_1.q=100$ ，如图 6-67 中红色字体所示。为便于理解上更加直观，我们在图中画出了完整的分析树，使读者可以对分析树的全貌有一个宏观了解，虽然实际上它还没有完全生成。接下来我们将用蓝色字体来显示该分析树自底向上的生成轨迹。

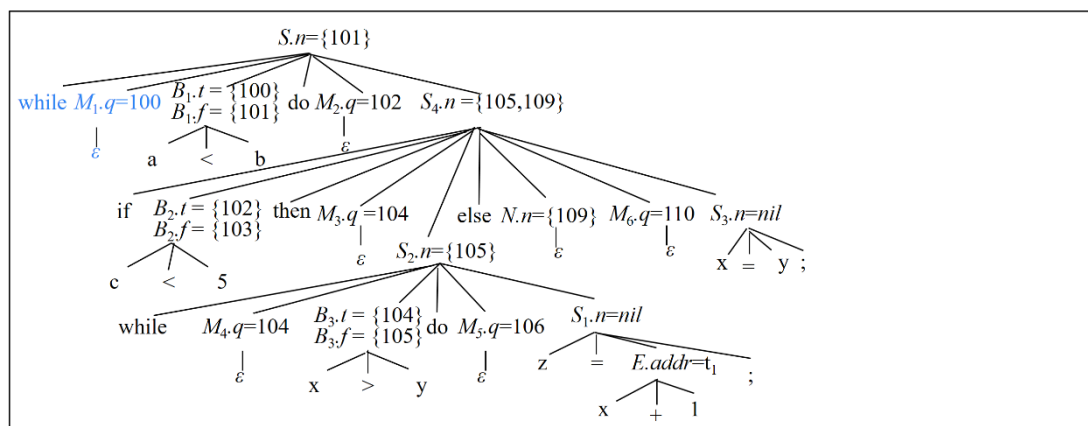


图 6-67 语句(6.109)的翻译过程(1)

接下来读入 `a<b`，并应用图 6-52 中的产生式(1)，将其归约为 B_1 ，如图 6-68 所示。根据该产生式的语义动作， $B_1.truelist=\{100\}$ ， $B_1.falselist=\{101\}$ ，并生成一对跳转指令

100: if a<b goto _ (6.110)

101: goto _ (6.111)

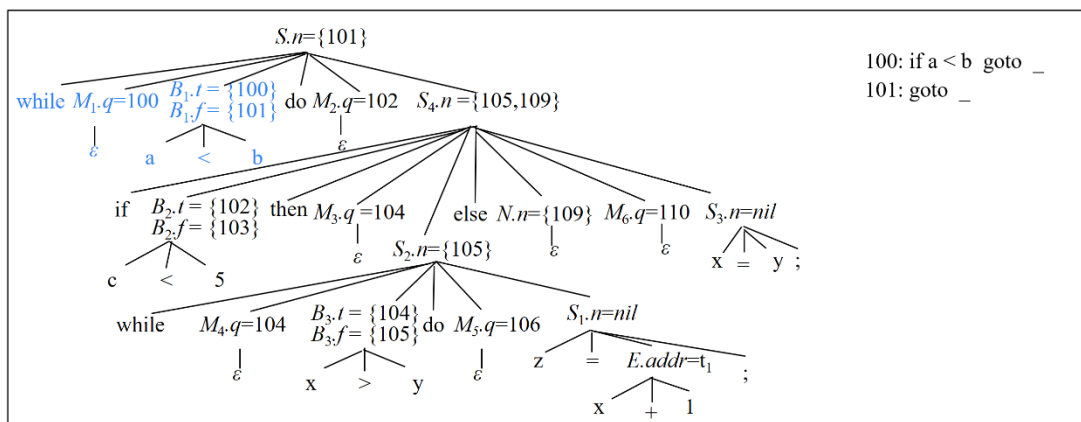


图 6-68 语句(6.109)的翻译过程(2)

接下来继续读入下一输入符号 **do**，并归约出 M_2 ，如图 6-69 中所示。

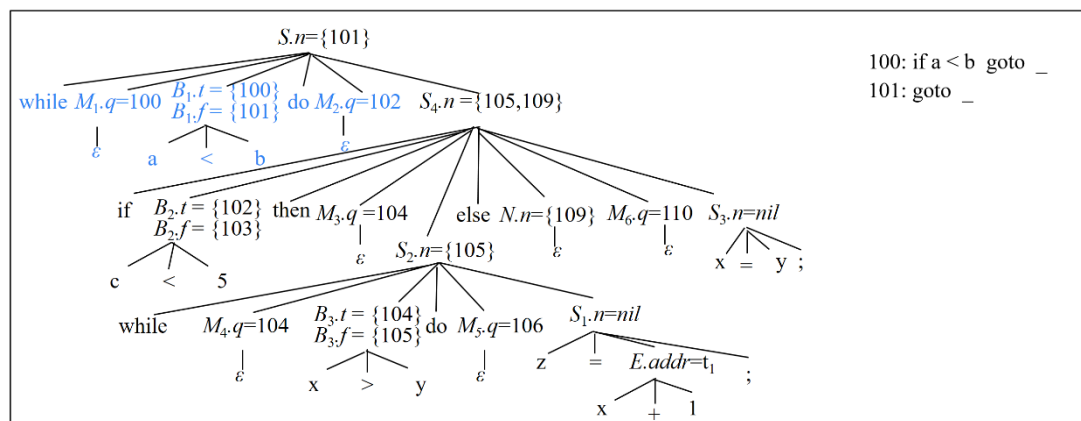


图 6-69 语句(6.109)的翻译过程(3)

接下来继续读入输入符号 **if** 以及 $c < 5$ ，并将 $c < 5$ 归约为 B_2 ，如图 6-70 所示，并生成一对跳转指令

102: if c < 5 goto _ (6.112)

103: goto _ (6.113)

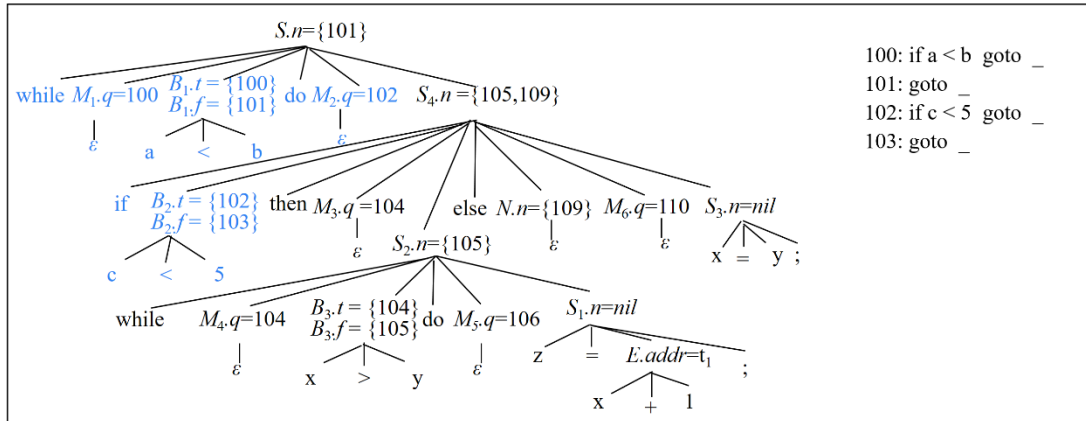


图 6-70 语句(6.109)的翻译过程(4)

接下来继续读入下一输入符号 `then`，并归约出 M_3 ，如图 6-71 中所示。

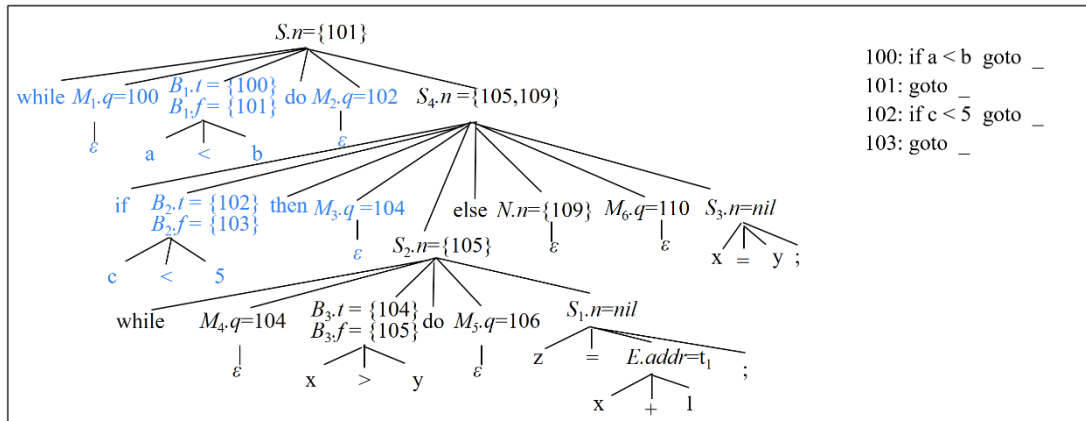


图 6-71 语句(6.109)的翻译过程(5)

接下来继续读入下一输入符号 `while`，并嵌套应用产生式(3)。为 `while` 创建叶节点之后，归约出 M_4 ，如图 6-72 所示。

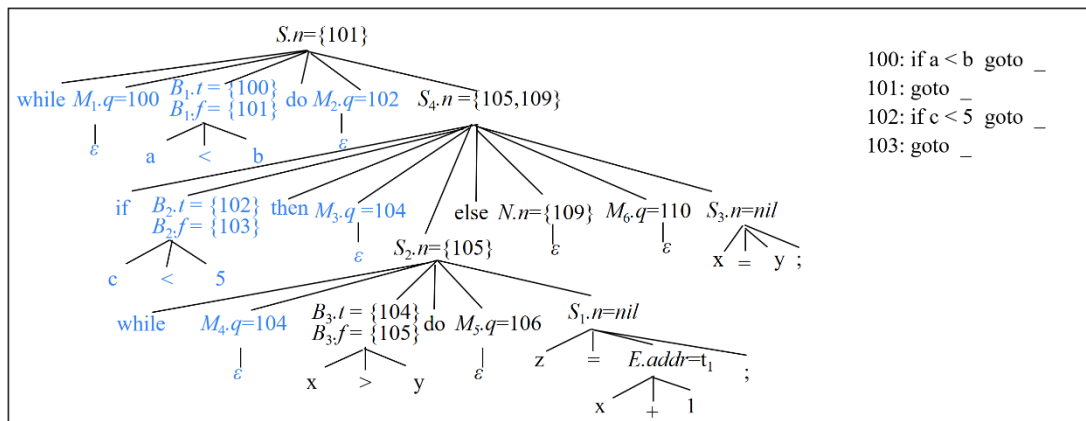
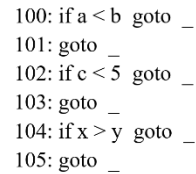
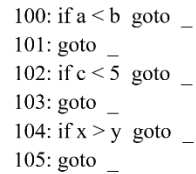


图 6-72 语句(6.109)的翻译过程(6)

$$104: \text{ if } x > y \text{ goto } _ \quad (6.114)$$


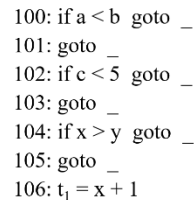
接下来继续读入下一输入符号 do，并归约出 M_5 ，如图 6-74 中所示。

$$S, n = \{101\}$$


接下来继续读入输入符号 $z=x+1$ ，并将 $x+1$ 归约为 E。根据图 6-6 中产生)对应的语义动作， $E.addr=t_1$ ，如图 6-75 所示，并生成一条赋值指令

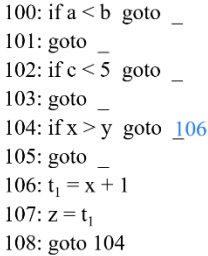
$$106: t_1 = x + 1 \quad (6.116)$$

- CCC -

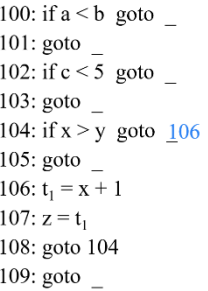

$$107: z = t_1 \quad (6.117)$$
[illegible]

接下来，将“**while** $M_4 B_3$ **do** $M_5 S_1$ ”归约为 S_2 。根据图 6-62 中产生式(3)对应的语义动作， $S_2.nextlist=\{105\}$ ，如图 6-77 所示。接下来，调用 *backpatch()* 函数，用 $M_4.quad$ （也就是 104）回填 $S_1.nextlist$ 中的跳转指令的目标地址。由于 $S_1.nextlist$ 为空，相当于不必执行任何回填操作。接下来，用 $M_5.quad$ （也就是 106）回填 $B.truelist$ 中的跳转指令（即 104）的目标地址。于是有

最后，生成一条跳往 $M_4.quad$ （即 104）的跳转指令，即



这样，整个 S_2 语句（即 while-do 语句）就分析完了，控制又回到 S_2 的父节点 S_4 所在的语句（即 if-then-else 语句）。继续读入下一输入符号 else，并归约出 N ，如图 6-78 所示。执行 N 产生式对应的语义动作，得到 $N.nextlist = \{109\}$ ，并生成 109 号待回填的跳转指令

$$(6.119)$$


接下来归约出 M_6 ，如图 6-79 中所示。

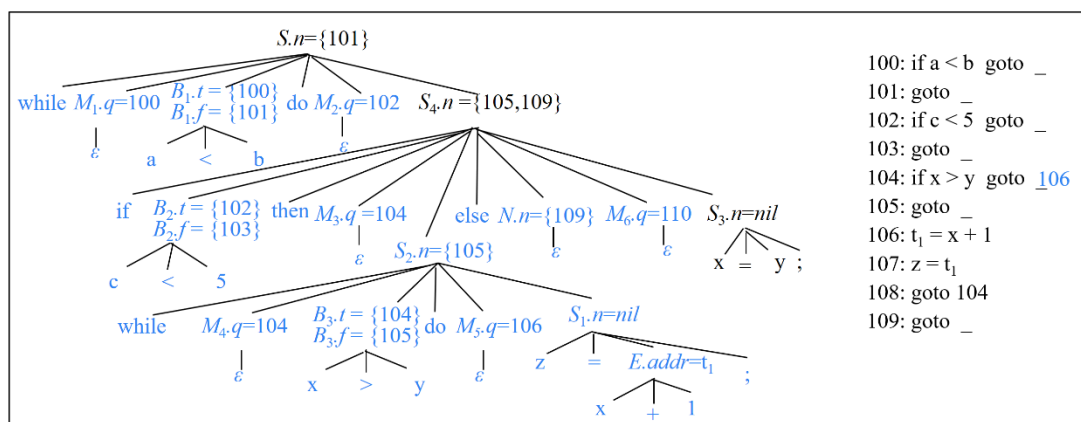


图 6-79 语句(6.109)的翻译过程(13)

接下来继续读入输入符号序列“x=y;”，并将其归约为 S_3 。根据图 6-62 中产生式(5)对应的语义动作，生成一条赋值指令

110: x=y (6.120)

并计算 $S_3.nextlist = nil$ ，如图 6-80 所示。

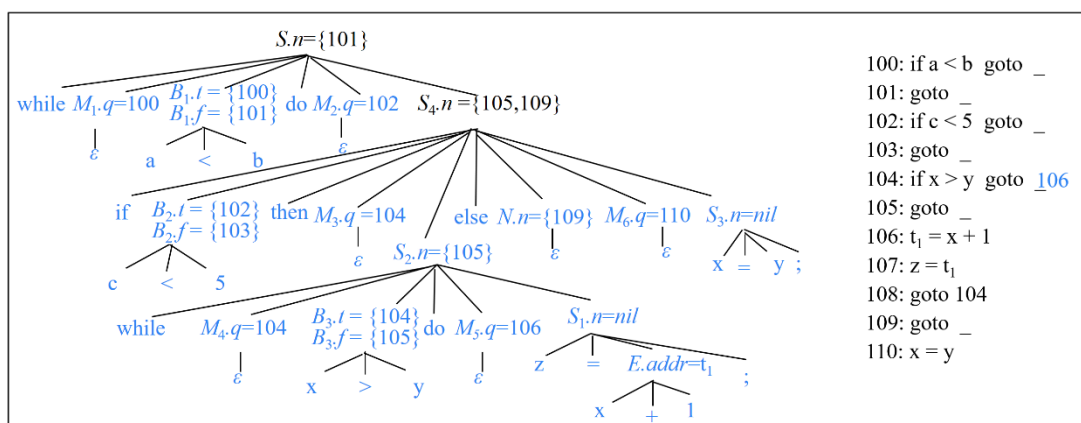


图 6-80 语句(6.109)的翻译过程(14)

接下来，将“if B_2 then M_3 S_2 else N M_6 S_3 ”归约为 S_4 。根据图 6-62 中产生式(2)对应的语义动作， $S_4.nextlist = \{105, 109\}$ ，如图 6-81 所示。接下来，调用 *backpatch()* 函数，用 $M_3.quad$ （也就是 104）回填 $B.truelist$ 中的跳转指令（即 102）的目标地址，用 $M_6.quad$ （也就是 110）回填 $B.falselist$ 中的跳转指令（即 103）的目标地址。于是有

102: if c < 5 goto 104 (6.121)

103: goto 110 (6.122)

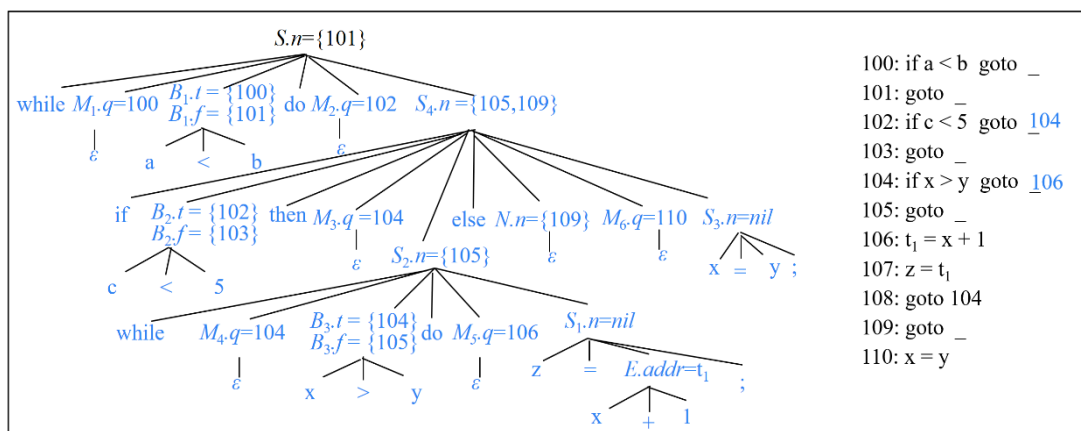


图 6-81 语句(6.109)的翻译过程(15)

这样，整个 S_4 语句（即 if-then-else 语句）就分析完毕，控制又回到 S_4 的父节点 S 所在的语句（即 while-do 语句）。

将 “while $M_1 B_1$ do $M_2 S_4$ ” 归约为 S 。根据图 6-62 中产生式(3)对应的语义动作， $S.nextlist=B_1.falselist=\{101\}$ ，如图 6-82 所示。接下来，调用 *backpatch()* 函数，用 $M_1.quad$ （也就是 100）回填 $S_4.nextlist$ 中的跳转指令（也就是 105 和 109）的目标地址，用 $M_2.quad$ （也就是 102）回填 $B_1.truelist$ 中的跳转指令（即 100）的目标地址。于是有

100: goto 102 (6.123)

105: goto 100 (6.124)

109: goto 100 (6.125)

最后，生成一条跳往 $M_1.quad$ （即 100）的跳转指令，即

111: goto 100 (6.126)

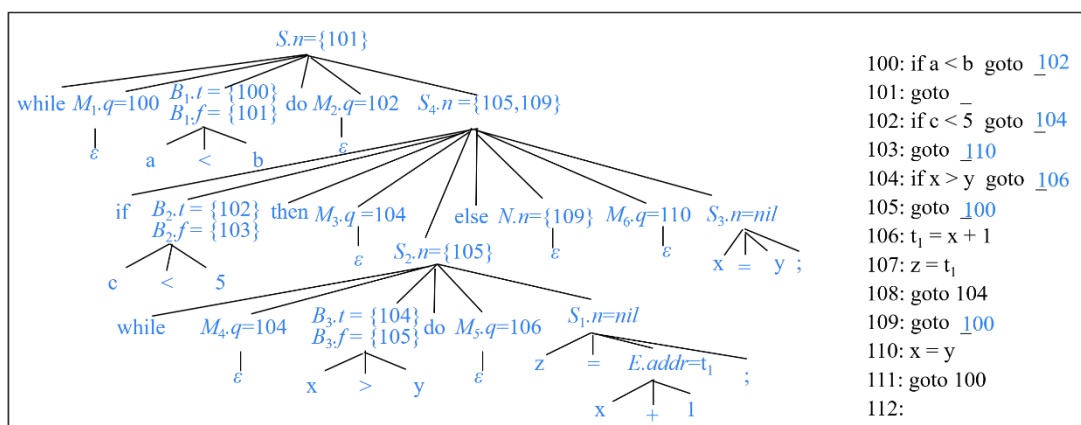


图 6-82 语句(6.109)的翻译过程(16)

值得注意的是，根据该翻译方案生成的代码中，第 109 号指令是一条冗余

指令。因为第 104 条指令是无条件跳转指令，而 109 又不是任何一条跳转指令的跳转目标，所以该指令永远不会被执行。实际上，从图 6-83 可以看出，第 104~108 条指令对应于 if 语句的第 1 各分支（即内层的 while 语句），110 对应于 if 语句的第 2 各分支。执行完第 108 条指令以后，需要执行一条显式跳转指令（即 109 号指令），跳往紧跟在 if 语句之后执行的语句。由于 if 语句内嵌在 while 语句中，因此，if 语句执行完要执行布尔表达式 $a < b$ 对应的 100 号指令，因此，109 号指令为：goto 100。冗余指令 109 在代码优化时可以被删除。

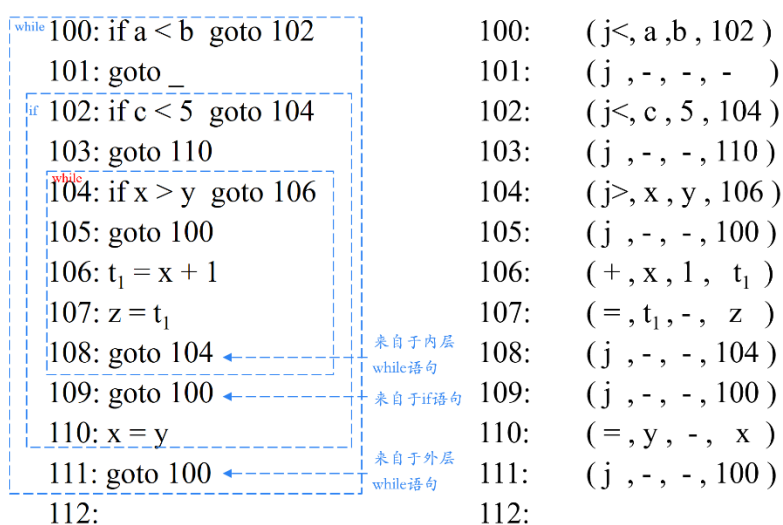


图 6-83 语句(6.109)的翻译结果

□

6.5 switch 语句的翻译

很多语言都使用 switch 或 case 语句。switch 语句的语法结构如下：

```

switch E
begin
    case  $V_1$ :  $S_1$ 
    case  $V_2$ :  $S_2$ 
    ...
    case  $V_{n-1}$ :  $S_{n-1}$ 
    default:  $S_n$ 
end

```

(6.127)

语句中包含一个待求值的选择表达式 E ，后面是 E 可能取的 n 个常量值。语句中也可能包含一个默认值，当其它值都不和选择表达式匹配时，就用这个默认值来匹配。根据 `switch` 语句的语义，画出代码结构图如图 6-84 所示。`switch` 语句的代码主要是由各个分支语句的代码 $S_1.code$ 、 $S_2.code$ 、……、 $S_n.code$ 顺序连接而成。在生成完 $E.code$ 之后，生成一条赋值指令将表达式 E 的值存放到临时变量 t 中。这里为了更直观地表示，直接使用 $E.addr$ 。在生成完每个 $S_i.code$ 的代码之后，都要生成一条无条件跳转指令 `goto next` 跳出 `switch` 语句。 $next$ 是紧跟在 $S_n.code$ 之后执行的第 1 条指令，也就是紧跟在 `switch` 语句之后执行的第 1 条指令。该翻译方案在生成跳转指令时，四元式的第四个分量不是目标标号，而是存放目标标号的变量地址，因此需要再进行一趟处理，从这些地址中取出具体的标号来生成正式的三地址指令。

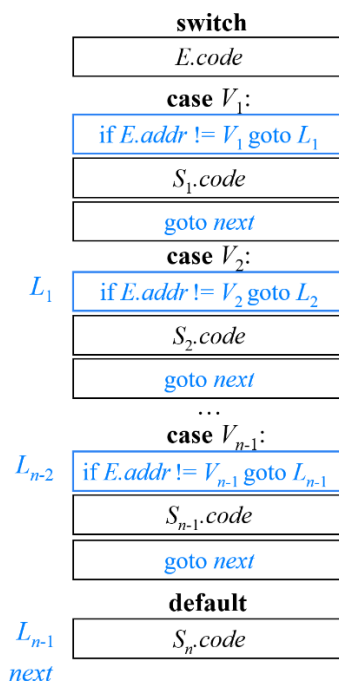


图 6-84 `switch` 语句的代码结构

该翻译方案将分支测试指令（图 6-84 蓝框中的部分）和各个 S 的代码混在了一起。事实上，当分支较多时，可以将分支测试指令集中在一起，这样，在代码生成阶段，便于生成较好的分支测试代码，如图 6-85 所示。

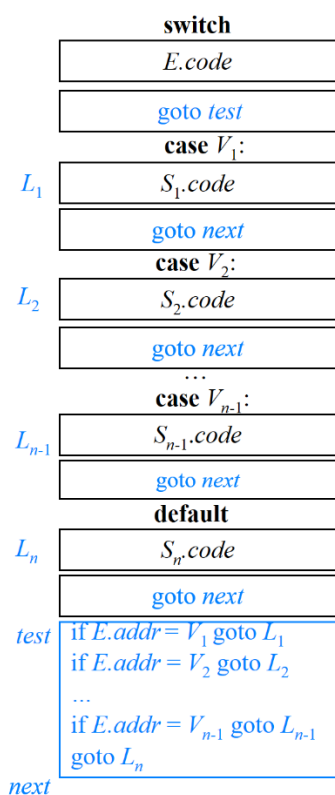


图 6-85 switch 语句的另一种代码结构

当编译器读入关键字 **switch** 时，生成两个新的标号 *test* 和 *next*。处理完 *E* 之后，产生跳转指令 **goto test**。每当读入一个 **case** 关键字时，就创建一个新的标号 L_i 并将其加入符号表。我们将在一个仅用于存放 **case** 分支的队列中放入一个值-标号对。这个值-标号对由常量值 V_i 和 L_i （或者是指向符号表中 L_i 的条目的指针）组成。我们逐个处理语句 **case V_i : S_i** ，生成附加于 S_i 的代码上的标号 L_i 。最后生成跳转指令 **goto next**。

当编译器到达 **switch** 语句的末端时，可以生成 n 路分支的代码。读取值-标号对的队列，我们可以生成形如图 6-86 所示的三地址语句序列。

<i>test</i> :	case	t	V_1	L_1
	case	t	V_2	L_2
			...	
	case	t	V_{n-1}	L_{n-1}
	case	t	t	L_n
<i>next</i> :				

图 6-86 用于翻译 switch 语句的 case 指令

指令 **case $E.addr$ V_i L_i** 和图 6-85 中的 **if $E.addr = V_i$ goto L_i** 含义相同，但是

case 指令更加容易被最终的代码生成器探测到，从而对这些指令进行某种特殊处理。在代码生成阶段，根据分支的个数以及这些值是否在一个较小的范围内，这种条件跳转指令序列可以被翻译成最高效的 n 路分支。

图 6-85 所示的方案之所以高效是因为当 $E.addr$ 与 V 的值多次不匹配时，图 6-84 所示的方案需要多次长距离跳转（涉及到程序计数器 PC 值的维护），而图 6-85 所示的方案只需顺序执行下一条指令（程序计数器 PC 值正常加 1 即可）。

之所以将所有的测试都出现在代码末端是因为在一趟式编译器中，将分支语句放在开始的位置会造成不便，因为编译器此时还没有碰到各个语句 S_i ，无法生成转向各个语句的代码。

6.6 过程调用语句的翻译

过程调用语句的文法如下：

$$\begin{aligned} S &\rightarrow \text{call id} (Elist) \\ Elist &\rightarrow Elist, E \\ Elist &\rightarrow E \end{aligned} \quad (6.128)$$

关键字 **call** 表示过程调用，**id** 是过程的名字， $Elist$ 是参数表达式的列表。根据过程调用语句的语义可以画出代码结构图如图 6-87 所示。过程调用语句的代码主要是由各参数表达式的代码 $E_1.code$ 、 $E_2.code$ 、……、 $E_n.code$ 构成。各参数表达式的值分别存放在 $E_1.addr$ 、 $E_2.addr$ 、……、 $E_n.addr$ 中。在每个 $E_i.code$ 之后，生成一条 **param** 指令，将 $E_i.addr$ 设置为实参。最后生成一条过程调用指令。

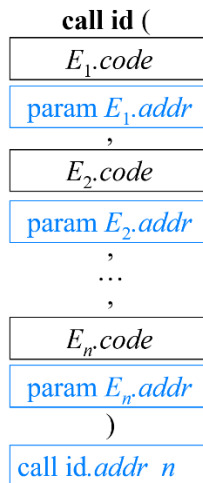


图 6-87 过程调用语句的代码结构

如果不想将 `param` 指令和参数表达式计算指令混在一起，也可以将 `param` 指令集中在一起，放在所有的参数表达式计算指令之后，如图 6-88 所示。因此，可以将每个表达式的属性 `E.addr` 存放到一个数据结构（比如队列）中。一旦所有的表达式都翻译完成，就可以在清空队列的同时生成 `param` 指令。

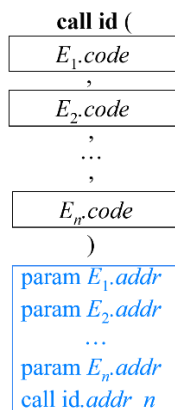


图 6-88 过程调用语句的另一种代码结构

过程调用语句的翻译方案如图 6-89 所示。其中，`n` 作为计数器用来记录参数的个数。

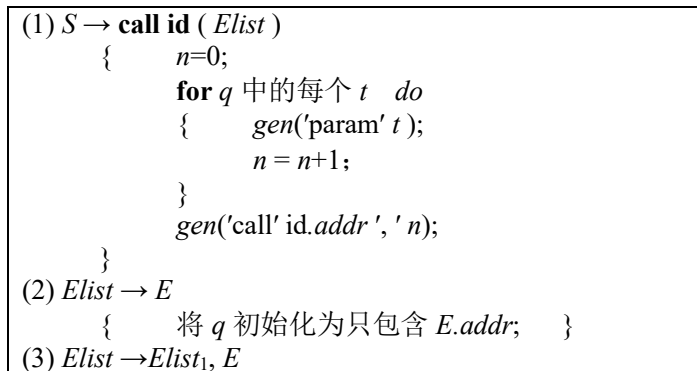


图 6-88 过程调用语句的翻译方案

例 6.10 假设要翻译如下程序片段：

$$f(b*c-1, x+y, x, y) \quad (6.129)$$

应用图 6-89 所示的翻译方案将其翻译成如图 6-90 所示的三地址码（不妨设指令标号从 100 开始）。其中指令(100)和(101)用来计算第 1 个参数表达式 `b*c-1` 的值，计算结果存放在临时变量 `t2` 中。指令(102)用来计算第 2 个参数表达式 `x+y` 的值，计算结果存放在临时变量 `t3` 中。第 3 和第 4 个参数表达式分别是由变量 `x` 和 `y` 本身构成的，所以这两个参数表达式的值的存放地址分别就是 `x` 和

y 的地址。指令(103)~(106)分别将 t_2 、 t_3 、x 和 y 设置成参数。最后，指令(107)调用过程 f，参数的个数是 4。

(100)	$t_1 = b * c$
(101)	$t_2 = t_1 - 1$
(102)	$t_3 = x + y$
(103)	param t_2
(104)	param t_3
(105)	param x
(106)	param y
(107)	call f, 4

图 6-88 语句(6.129)的翻译结果

□

6.7 本章小结

本章系统地介绍了中间代码生成的核心技术。首先讨论了声明语句与赋值语句的翻译方法，包括类型表达式的处理、数组引用以及增量翻译策略。接着重点阐述了控制语句的翻译，涵盖布尔表达式和控制流语句的中间代码生成与优化，并引入了“回填”技术以高效管理转移指令的目标标号。最后，对 switch 语句和过程调用语句的翻译机制进行了分析。通过本章学习，读者将掌握将各种语法结构转换为中间表示的关键方法与实现思想，为后续的代码优化和目标代码生成奠定坚实基础。

第7章 运行存储分配

在前面的章节中，我们系统地探讨了编译器的前端技术，已经完成了对程序静态结构和逻辑语义的分析。然而，程序最终需要在目标机上运行，编译器就需要创建并管理一个目标程序的运行时环境，该环境处理诸如出现在源程序中的各种对象的存储空间的布局 and 分配问题、目标程序所使用的变量访问机制、过程之间的链接、参数传递机制以及与操作系统、输入输出设备和其他程序的接口问题。本章将深入探讨运行存储分配策略，它是后续目标代码生成阶段的基础，因为生成何种机器指令来访问变量，直接取决于我们为这些变量选择了何种存储分配方案。可以说，运行存储分配是将静态的程序文本映射到动态的机器行为所不可或缺的一环。编译器可能需要额外生成部分代码来管理运行时刻环境，实现运行存储分配策略，这些代码可能是中间代码或目标代码。

通过本章的学习，不仅能为最终的目标代码生成做好准备，而且你也将成为一个程序运行机制的“洞察者”，理解对象生命周期的本质，掌握函数调用的底层实现，并真正明白诸如“栈溢出”、“内存泄漏”等问题的根源。这也将为你后续学习程序优化乃至开发高性能软件打下坚实的基础。

7.1 存储组织

对于编译器来说，通常目标程序是在逻辑地址空间中运行的，程序的所有数据和代码都在此空间中拥有自己的逻辑地址，而编译器在生成目标代码时要明确程序的各类对象在逻辑地址空间内是如何组织和存储的，在目标代码运行时，又是如何使用和管理逻辑存储空间的。编译器的存储组织策略在中间代码生成时就需要开始规划和布局，直至目标代码生成时实施完成。

存储组织主要涉及到内存的划分和管理，一种典型的目标程序运行时内存的组织方式如图 7-1 所示，主要包括代码区和数据区，数据区又分为静态数据区、栈区和堆区等关键部分。

代码区存储目标程序的指令代码，通常组织在低地址空间。编译器在生成目标代码时，会将这些指令按照特定的格式排列在代码区中，以便处理器能够顺序或跳转地执行它们。

静态数据区存储全局变量、静态变量和常量。这些数据在程序的生命周期内保持存在。

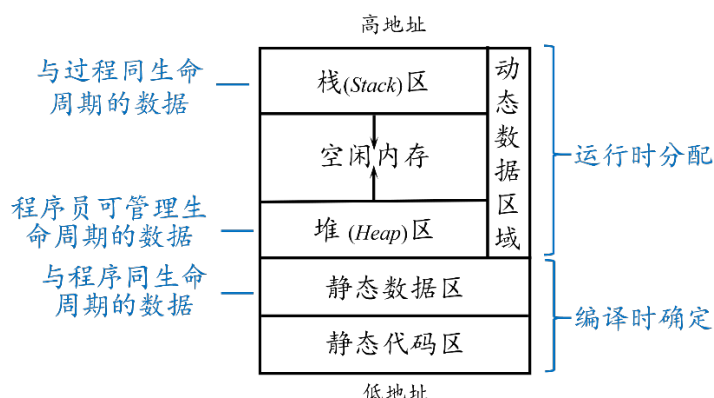


图 7-1 目标程序运行时存储空间布局的典型方式

动态数据区域包括运行时动态分配的栈区和堆区。通常栈区和堆区分列在动态数据区的两端，栈区从高地址向低地址增长，而堆区则从低地址向高地址变化，这样可以有效地利用动态数据区空间。随着目标程序的执行和空间消耗，堆区和栈区很可能会相遇甚至产生冲突，因此，每次栈区或堆区的增长都要检查并避免冲突发生。

栈区用于存放与过程同生命周期的局部数据对象以及支持过程调用与返回的信息。这些数据空间会随着过程的调用而创建，随过程的返回而回收。

堆区用于管理生命周期比创建它的过程的生命周期更长的数据对象，在数据对象被创建时分配空间，在数据对象不再使用时释放空间。

对应不同的数据区管理，运行存储空间的组织主要有两类策略，静态存储分配（管理静态数据区）和动态存储分配（管理栈式和堆区）。静态（**static**）对应编译时刻，而动态（**dynamic**）对应运行时刻。对于那些在编译时刻通过静态的代码分析就可以确定大小的数据对象，可以在编译时刻为它们分配存储空间，这样的分配策略称为静态存储分配。反之，如果不能在编译时完全确定数据对象的大小，而依赖运行时的程序状态的数据对象，采用动态存储分配的策略，即在编译时仅产生各种必要的信息，而在运行时刻，能确定数据对象大小时，再动态地分配数据对象的存储空间。

动态存储分配又分为栈式存储分配（管理栈区）和堆式存储分配（管理堆区）。需要注意的是，现代的主流编程语言（如 C/C++、Java、Go 等）的编译器大多使用静态存储、栈式存储和堆式存储分配三种策略的混合模型，以在效率与灵活性之间取得平衡。栈式存储分配原则是后进先出，即后分配的内存空间会先被释放，这种机制能有效地支持过程嵌套调用关系中过程存储空间的分

配需求，即后运行（后分配存储空间，即后进栈）的被调用过程会先结束（先回收存储空间，即先出栈）。堆区内存的分配和释放可以由程序员管理（如 C、C++ 等），在程序运行过程中，当需要动态地创建或销毁对象时，程序员通过程序指令申请（如 `malloc`、`new` 等）或释放（如 `free`、`delete` 等）内存空间。堆区的管理相对复杂，为了提高内存使用的安全性同时提升内存使用效率，有些编译器会自动地管理堆区，例如，Java 语言的垃圾自动回收机制，Python 语言将所有的对象空间都分配在堆区，并在对象不再被引用时自动回收空间。

7.2 静态分配策略

7.2.1 静态存储分配

在静态存储分配中，数据的内存在编译时一次性分配，运行期间不释放，直至程序结束。编译器需要在编译时为每个过程确定其局部数据在目标程序中的固定位置。这样，过程中每个名字¹的存储位置就确定了，因此，这些名字的存储地址可以被编译到目标代码中。过程每次执行时，它的名字都绑定到同样的存储单元。适合使用完全静态存储分配的语言必须满足以下条件。

（1）数组上下界必须是常数。

数组的大小在程序编译时就必须是已知的、固定的数值，而不能是变量或运行时才能计算的表达式。编译器需要在编译时就为整个数组分配一块固定大小的内存，如果数组大小在程序运行时才能确定，那么编译器就无法确定应该分配多少空间，从而无法实现静态分配。

（2）不允许过程的递归调用

过程的每一次递归调用都需要在内存中为其分配新的数据空间。而递归的深度在编译时是无法预知的，因此编译器无法在编译时预先为所有可能的调用分配固定大小的内存。

（3）不允许用户动态建立数据实体

程序在运行时动态创建的数据实体，其数量、大小和生命周期在编译时完全未知，而静态分配策略要求所有数据结构的数量和内存布局在程序开始执

¹ 术语“名字（name）”是一个抽象的语义概念，指的是由标识符表示的、在程序中可被引用的语言实体。主要包括：变量名、常量名、过程名、类型名及标签名等。

行前就已完全确定。

完全静态存储分配因其灵活性差而难以满足现代软件开发的需求，其主要缺陷包括，因无法支持递归而限制了算法设计和问题求解的方式；因无法创建动态数据结构而无法处理运行时才能确定大小的数据；为了应对最坏情况，程序可能需要预先分配远超实际需要的内存，因而内存使用效率低。

完全或主要采用静态存储分配的典型编程语言如早期的 FORTRAN 及 COBOL 语言等，也被称为静态语言。

7.2.2 常用的静态存储分配方法

（1）顺序分配法

顺序分配法指按照过程在源程序中出现的先后顺序逐段分配存储空间，各过程占用互不相交的存储空间。

例 7.1 假设源程序声明了 6 个过程，各过程所需存储空间分别为 22、15、18、17、10 和 23；过程之间的调用关系如图 7-2 所示。图中每个节点对应一个过程，节点标记由“过程编号/数据空间大小”组成。图中的边表示调用关系，即父节点调用子节点，子节点按照被调用顺序从左到右排列。

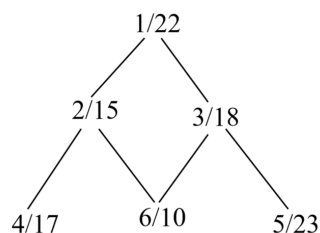


图 7-2 过程调用关系图

顺序分配法并不考虑过程间的调用关系，只按照过程定义的先后顺序逐段分配空间，使得各过程的空间互不相交。假设从逻辑地址 0 开始分配，一种可能的分配结果如表 7-1 所示。

表 7-1 一种顺序分配方案示例

过程	存储区域
1	0~21
2	22~36
3	37~54
4	55~71
5	72~94

6	95~104
---	--------

顺序分配法实现简单，但对内存空间的使用不够经济合理，过程执行结束后，即使空间不再使用，依然会保留到整个程序结束。

(2) 层次分配法

层次分配法的主要思想是让那些无相互调用关系的并列过程，尽可能地共享存储空间，以提升内存的利用率。具体来说，层次分配法首先分析源程序过程间的调用关系，由于不允许递归调用，调用关系图中不存在环路；然后按照自下而上的顺序依次为各层次的过程分配数据空间。层次分配法实现了位于相同调用层次的并列过程共享存储空间的目标。

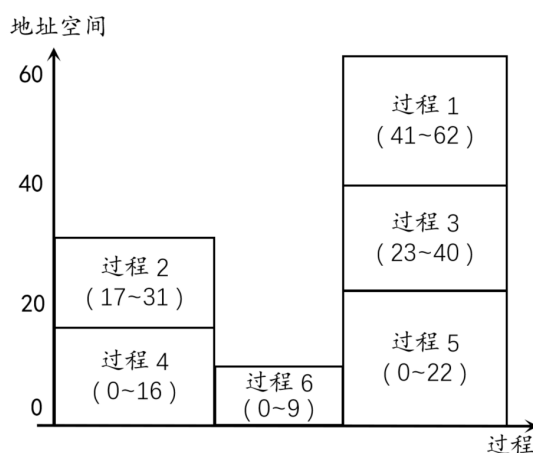


图 7-3 一种层次分配方案示例

例 7.2 分析图 7-2，自下而上地为不同层次的过程分配空间。首先最底层过程 4、5 和 6 之间无相互调用关系，3 个过程可以共享存储空间。从存储空间的同一起点（假定采用相对地址 0 号单元）为它们划定数据区，于是，过程 4、5、6 的数据区分别占用了单元 0-16、0-22、0-9，如图 7-3 所示。其次考虑中间层过程 2 和 3，由于过程 2 调用过程 4 和 6，因此过程 2 与过程 4 所占存储空间不能重叠，过程 2 与过程 6 亦是如此。已知过程 4 比过程 6 的空间上界更大，已经占用了单元 0-16，因此从单元 17 开始为过程 2 划分数据区，这样就能保证过程 2 在调用过程 4 或 6 时不会相互破坏其数据区中的内容。同理应该从单元 23 开始为过程 3 设置数据区，因此，过程 2 和 3 将分别以单元 17-31 和 23-40 为其数据区。最后，再考虑最顶层过程 1，显然，应该从单元 41 开始为其设置数据区，且所占用的单元为 41-62。由最终的分配方案图 7-3 可见，处于相同调用层次的过程 4、5 和 6 实现了存储空间共享；处于相同调用层次的过程 2

和过程 3 也实现了部分存储空间的共享。

层次分配算法如算法 7.1，其总体思想是先为不调用其他过程的过程分配内存，再为调用的所有过程都已分配内存的过程分配内存。假设一个程序由 N 个过程组成，那么算法使用了如下四个数组。

(1) N 阶布尔方阵 $B[N][N]$ ：表示各过程间的调用关系，对于上文中例子，其调用关系矩阵如图 7-4 所示。其中，

$B[i][j]=1$ ，表示第 i 个过程调用第 j 个过程，且第 j 个过程未分配内存；

$B[i][j]=0$ ，表示第 i 个过程不调用第 j 个过程或第 j 个过程已分配内存。

	1	2	3	4	5	6
1	0	1	1	0	0	0
2	0	0	0	1	0	1
3	0	0	0	0	1	1
4	0	0	0	0	0	0
5	0	0	0	0	0	0
6	0	0	0	0	0	0

图 7-4 过程调用关系矩阵

(2) 长度为 N 的数组 **units**：记录各过程数据区所需占用的存储单元个数；

(3) 长度为 N 的数组 **base**：记录每个过程的数据区的基地址；

(4) 长度为 N 的数组 **allocated**：记录每个过程是否完成存储分配的标志，1 为已分配，0 为未分配。如果某个过程 i 还未分配，即 $allocated[i]=0$ ，而他并没有调用其他过程，或者它调用的其他过程都已分配完成，那么就可以为过程 i 分配空间了。

另外，在实现分配时必须满足如下条件，若第 i 个过程调用第 j 个过程，那么，过程 i 与过程 j 的内存空间不能有重叠，先为过程 j 分配内存空间，过程 i 的内存空间的下界要大于过程 j 的内存空间的上界。

算法 7.1 层次分配算法。

输入： 存储每个过程所需空间单元数的数组 **Units**，过程总数量 N

输出： 存储每个过程分配的基地址的数组 **base**

方法：

/*初始化所有过程的基地址和分配标记*/

for(每个过程 i) {

 过程 i 的基地址 $base[i]$ 初始化为 0;


```

    过程 i 标记 allocated[i] = 0 为未分配;
}
for( 若还有过程未分配内存 ) {
    for( 每个过程 i ) { /*找到可以分配的所有过程 */
        if( 矩阵 B 的第 i 行的和 sum = 0 ) {
            /*若过程 i 调用的所有过程已分配*/
            为 i 分配内存, 标记过程 i 已分配 allocated[i] = 1;
            for( 每个调用 i 的过程 k ) {
                if( k 的基地址在过程 i 的数据区)
                    设置 k 的基地址在 i 的数据区之上
                B[k][i] = 0; /* 标记 k 调用的 i 已分配内存 */
            }
        }
    }
}
}

```

7.3 栈式分配策略

与静态存储分配不同, 栈式存储分配策略不仅允许活跃时段不交叠的多个过程调用之间共享空间, 而且非局部变量的相对地址总是固定的, 和过程调用序列无关。另外, 栈式存储分配也为递归算法的实现提供了支持。

7.3.1 活动树

可以将过程 (procedure) 看作是对程序代码中用户自定义动作单元的一种抽象。对于不同的编程语言, 过程的形式多种多样, 包括函数 (function)、方法 (method)、子例程 (subroutine)、处理函数 (handler) 等。

对于支持过程定义的编程语言, 过程的一次执行称为该过程的一个活动 (activation), 栈式存储分配策略通常以活动为单位来管理栈空间。用来描述程序运行期间控制进入和离开各个活动的情况的树称为活动树 (activation tree)。树的每个节点对应一个活动, 根节点对应启动程序执行的主过程的活动。对于表示过程 p 的某个活动的节点, 其子节点对应于被 p 的这次活动调用的各个过程的活动, 并且按照这些活动被调用的顺序, 自左向右地排列它们,

一个子节点必须在其右兄弟节点的活动开始之前结束。

例 7.3 给出一个快速排序程序的概要如图 7-5。该程序将 9 个整数读入到数组 a ，并使用递归的快速排序算法将整数排序。主函数 *main* 首先调用函数 *readArray* 读入 9 个整数，再调用函数 *quicksort* 将数组 $a[1]$ 至 $a[9]$ 之间的整数进行快速排序。函数 *quicksort* 调用分割函数 *partition* 将数组 $a[1]$ 至 $a[9]$ 之间的整数分割为三部分，即选择一个分割值 v ，使得 $a[1 \cdots p-1]$ 小于 v ， $a[p] = v$ ， $a[p+1 \cdots 9]$ 大于等于 v ，并返回 p ；然后再递归地调用函数 *quicksort* 将数组 $a[1]$ 至 $a[p-1]$ 之间和数组 $a[p+1]$ 至 $a[9]$ 之间的整数分别进行快速排序，以此类推，直接排序完成。

```
int a[11]; /*全局数组*/
void readArray() /*将9个整数读入到a[1],...,a[9]中*/
{
    int i;
    ...
}
int partition(int m, int n)
{
    /*选择一个分割值v, 划分a[m...n], 使得a[m...p-1]小于v, a[p]=v,
    a[p+1...n]大于等于v。返回p*/
    int i, v;
    ...
}
void quicksort(int m, int n)
{
    int i;
    if(n > m)
    {
        i=partition(m, n);
        quicksort(m, i-1);
        quicksort(i+1, n);
    }
}
main()
{
    readArray();
    a[0] = -9999;
    a[10] = 9999;
    quicksort(1, 9);
}
```

图 7-5 一个快速排序程序的概要

给出该程序在某次执行对应的活动树如图 7-6，活动树的节点使用对应函数名的首字母标识。在此次执行中，对 $p(1,9)$ 调用返回 4，因此， $a[1]$ 至 $a[3]$ 存放了小于被选定的分割值 $a[4]$ 的元素，而较大的元素被存放在 $a[5]$ 至 $a[9]$ 中。接下来，递归调用 $q(1,3)$ 和 $q(5,9)$ 。 $q(1,3)$ 中对 $p(1,3)$ 调用返回 2，因此， $a[1]$ 存放了小于被选定的分割值 $a[2]$ 的元素，而较大的元素被存放在 $a[2]$ 至 $a[3]$ 中，接下来，递归调用 $q(1,0)$ 和 $q(2,3)$ 如此执行下去，直到子数组只包含一个值或没

有任何值，开始逐层返回计算结果，最终完成数组排序。

由此程序的执行过程可见，过程调用在时间上是嵌套的，支持栈式分配。

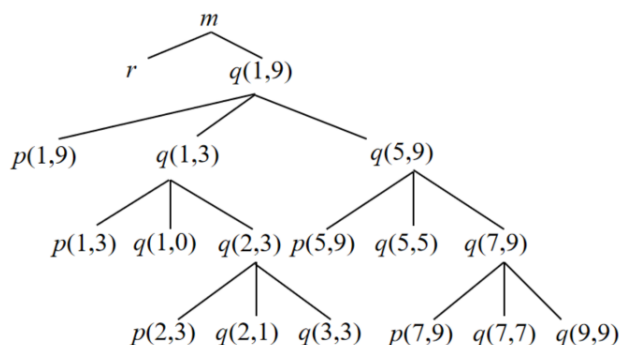


图 7-6 概要程序某次执行对应的活动树

7.3.2 活动记录

对于每一个过程的执行，即每一个过程的活动，编译器都为它在栈区分配一块连续存储区，用来管理过程本次执行所需的所有上下文信息，这段连续的存储区域称为活动记录（activation record）。其核心内容可以概括为以下几个关键部分，如图 7-7 活动记录的一般形式所示：

实参：用于存放调用过程提供给被调用过程的参数，所有的参数在存储器中需连续存放。参数也可以使用寄存器传递，但有些参数必须保存内存空间。

返回值：用于存放被调用过程返回给调用过程的值。返回值也可以使用寄存器传递，但为了保证活动记录的通用性，保留这个空间。

控制链：存放指向被调用过程的调用者的活动记录的指针，以便被调用过程结束时能找到调用者的活动记录。控制链也称为“动态链”。现代编译器也可以使用帧指针寄存器代替控制链指向调用者活动记录。

访问链：当过程需要访问其他过程管理的数据时需要使用访问链来定位该数据所在的活动记录。访问链也称为“静态链”。

机器状态：通常包括返回地址（被调用过程运行结束后，需要继续运行的调用过程的指令地址）和一些寄存器中的值。这些都是调用过程使用的内容，被调用过程执行结束返回时必须恢复这些内容，以便调用过程继续运行。

局部数据：在该过程中声明的数据对象。

临时变量：编译器生成的，用于存储中间运算结果的变量。在生成目标代码时会尽量将它们放在寄存器中，但若寄存器分配紧张时，就不得不将它们放

在内存中。

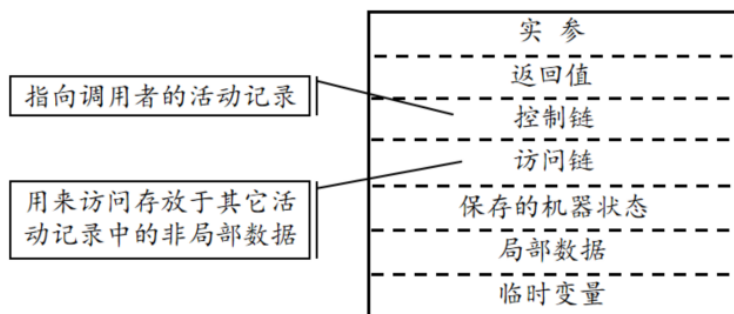


图 7-7 活动记录的一般形式

值得注意的是，以上只是活动记录的一般形式，不同计算机系统的不同编程语言其活动记录的具体内容和布局会有所不同。例如，根据现代计算机系统的参数传递约定，一个函数的前 k 个参数 ($k=4$ 或 $k=6$)，放在寄存器中传递，剩余的参数则放在存储器中传递。而某些被取了地址的特殊实参，或跨过程使用的变量也需要保存在存储器中。一个过程也可能把所有的局部变量和临时变量都存放在寄存器中。

栈式存储分配使用运行时刻栈动态地实现过程的调用和返回，当一个过程被调用时，该过程的活动记录被压入栈；当过程结束时，该活动记录被弹出栈。因此，这个运行时刻栈简称为运行栈，也称为控制栈，活动记录也被称为栈帧 (stack frame)。

对于图 7-6 中某次程序的执行，控制栈的分配变化情况如图 7-8 所示。图中活动树的兰色边表示活跃活动的路径，黑色边表示运行已结束活动的路径。程序从 *main* 函数开始执行，*main* 的活动记录入栈，如图 7-8(a)所示。接下来，*main* 调用函数 *readArray*，其活动记录（包含局部变量 i ）入栈，如图 7-8(b)所示，控制进入 *readArray* 函数，其局部数据 i 可以正常访问。当函数 *readArray* 的此次活动执行结束，其活动记录（包含局部变量 i ）出栈，程序控制返回 *main* 活动，控制栈中只包含 *main* 的活动记录，如图 7-8(c)所示。由于 *readArray* 的局部数据 i 存储空间已被回收， i 不能再访问。接下来，*main* 调用函数 *quicksort*(1,9)，其活动记录（包含局部变量 i ）入栈，如图 7-8(d)所示。然后 *quicksort*(1,9)又调用函数 *partition*(1,9)，*partition*(1,9)活动记录（包含局部变量 i 和 v ）入栈，如图 7-8(e)所示。当函数 *partition*(1,9)的此次活动执行结束，返回分割位置 4，其活动记录出栈，程序控制返回 *quicksort* 活动，此时控制栈布局如图 7-8(f)所示。接着 *quicksort*(1,9)又调用 *quicksort*(1,3)进行子数组的快

速排序， $quicksort(1,3)$ 又调用 $partition(1,3)$ 进行数组分割，如图 7-8(g)所示，此时，控制栈中同时包含函数 $quicksort$ 的两个活动记录，分别对应此函数的两次调用。程序如此执行下去，直至排序结束。

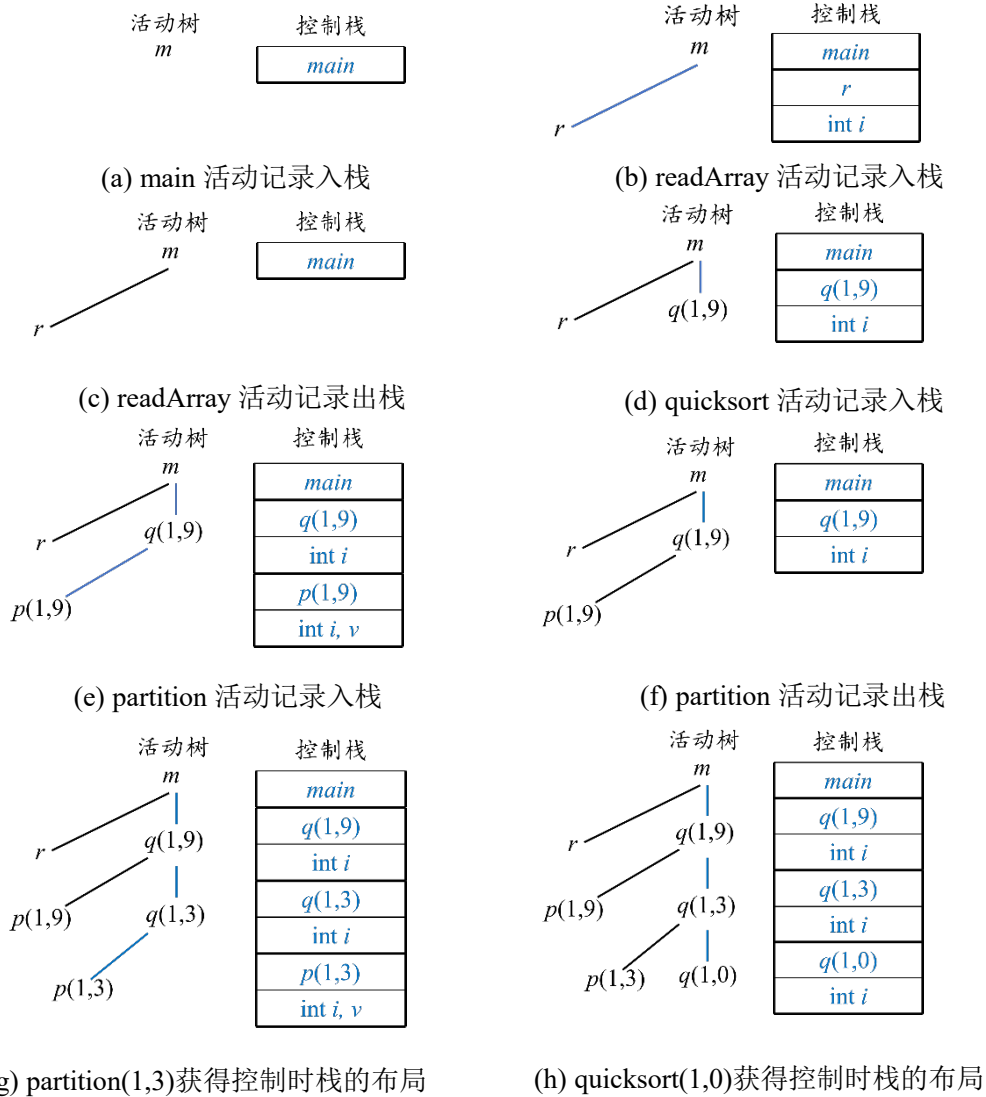


图 7-8 栈式存储分配过程示意图

基于以上过程可见，首先，栈式分配策略以活动为单位来分配栈空间，每个活跃的活动都有一个位于控制栈中的活动记录。其次，活动树的根位于栈底，程序控制所在的活动的记录位于栈顶，栈中全部活动记录的序列对应于活动树从根节点到达当前控制所在的活动节点的路径。再次，这种策略允许活跃时段不交叠的多个过程调用之间共享空间，如图 7-8(b)和图 7-8(c)，过程 $readArray$

的活动记录空间即被 *quicksort*(1,9)的活动记录复用。最后，当一个过程包含递归时，执行时常常会有该过程的多个活动记录同时出现在栈中，如图 7-8(h)的控制栈中包含 *quicksort* 函数的三个独立的活动记录，分别对应它的三次调用。栈式分配策略保证了无论函数被递归地调用多少次，都能够为每次执行分配独立的数据管理空间，从而保证了递归函数的正确执行。

活动记录的布局设计

活动记录的布局要考虑指令集的体系结构特征和被编译的程序设计语言的特性。图 7-9 展示一种典型的活动记录布局，考虑到栈区地址的变化是自上向下逐渐增大的，按照惯例，把栈底画在上方，把栈顶画在下方。活动记录的布局设计可以遵循以下一些基本原则：

(1) 需要在调用者和被调用者之间传递的值（如实参和函数返回值）一般被放在被调用者的活动记录的开始位置，即靠近调用者活动记录的位置。这样一来，调用者在不知道被调用者的活动记录的整体布局的情况下也可以准确地将实参放在被调用者的参数位置；同样的，从被调用函数返回后，调用者也可以准确地找到返回值的存放位置。

(2) 固定长度的项被放置在中间位置。这样的项包括控制链、访问链和机器状态字。如果每次调用中需要保存的机器状态字段相同，那么这些项都可以采用基于栈顶指针的相同的偏移量来访问，也可以使用相同的保存和恢复机器状态的代码。

(3) 在早期不知道大小的项被放置在活动记录的尾部。大部分局部变量具有固定的长度，编译时即可确定。然而，有些局部变量的大小只有在程序运行时才能确定，最常见的例子是变长数组，如某些 C 语言的版本支持使用变量作数组大小来声明数组，数组大小是由运行时变量的值确定的。临时变量所需空间的大小在早期也不确定，它通常依赖于代码生成阶段能将多少临时变量放在寄存器中。

(4) 必须谨慎地设置栈顶指针所指的位置。为了实现活动记录中数据的访问，通常以活动记录中某个字段位置作为基地址，使用基于这个地址的偏移量来访问活动记录中的数据。栈顶指针通常就指向栈顶活动记录的基地址。因此，栈顶指针所指位置直接决定了对数据的访问方式是否便捷。一种常用的方法是将栈顶指针 *top_sp* 指向活动记录固定长度字段的末端，如图 7-9，即局部数据开始的位置。这样，固定长度的数据域总是可以使用固定的偏移量来访问。局部数据也可以直接使用基于基址（栈顶指针）的偏移量实现访问。值得注意的是，由于栈是从高地址向低地址增长的，访问活动记录中栈顶指针上面的数

据域需使用正的偏移量，而访问栈顶指针下面的数据域需使用负的偏移量。

另外，由于控制链总是指向其所在活动记录之下（将栈底方向称为下）紧挨着的栈帧，现代编译器可能会使用帧指针取代控制链，让它始终指向栈顶之下的第二个栈帧。

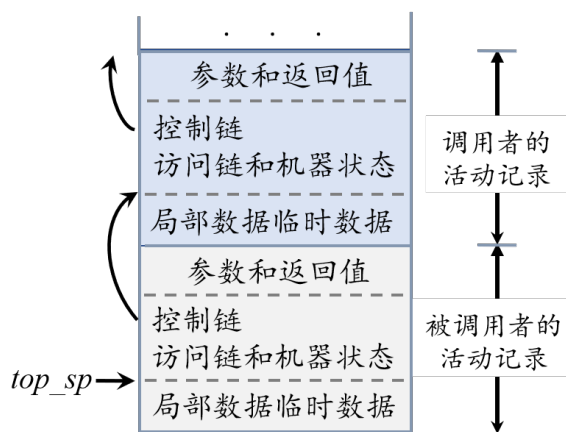


图 7-9 一种典型的活动记录布局

7.3.3 调用序列和返回序列

过程调用和过程返回都需要执行一些代码来管理活动记录栈，保存或恢复机器状态等。过程调用和返回是通过在目标代码中生成调用序列和返回序列来实现的。调用序列指实现过程调用的代码段，负责为活动记录在栈上分配空间，并在此记录的字段中填写信息并保存机器状态；返回序列负责恢复机器状态，使得调用过程能够在调用结束后继续执行。调用序列和返回序列中的代码通常被分割到调用过程（调用者）和被调用过程（被调用者）中，如何划分则依赖源语言、操作系统和目标机器，并没有明确的标准。总的来说，如果一个过程在多个不同点上被调用，分配给调用者的那部分调用代码序列就会被生成多次；由于被调用过程的代码定义只有一份，分配给被调用者的部分只会被生成一次。因此，我们期望将尽可能多的调用代码和返回代码分割给被调用者。根据这样的原则，返回代码主要由被调用者负责。然而对于调用序列，由于有些信息被调用者根本无法确定，因此它不可能完成调用序列的所有任务。图 7-10 给出了一个可行的调用者与被调用者合作管理调用栈时任务划分的例子。此例中调用代码序列具体划分描述如下：

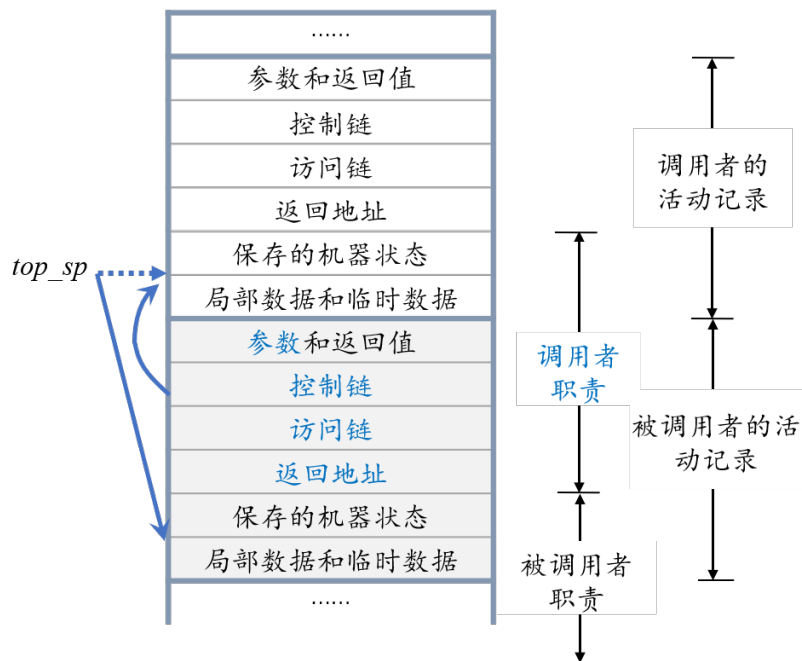


图 7-10 调用者和被调用者之间的任务划分

(1) 调用者计算实际参数的值，并将参数值存放在被调用者的参数空间。由于调用者知道自己的局部数据和临时数据空间大小，调用者能够基于当前的栈顶指针 top_sp （指向调用者的活动记录）计算出被调用者参数空间的相对位置，完成任务。

(2) 调用者将返回地址放到被调用者的机器状态字段中。将当前的栈顶指针 top_sp 值（指向调用者的活动记录）存放到被调用者的控制链中。然后，增加栈顶指针 top_sp 的值，使其指向被调用者的活动记录即局部数据开始的位置，为被调用者分配栈空间。由于调用者知道自己的局部数据、临时数据以及参数与返回值所占空间的大小，而其他项均为固定长度的项，调用者可以计算出被调用者的控制链、机器状态字段及其活动记录指针 top_sp 应指向位置相对于调用者的活动记录指针（当前 top_sp ）的位置。由于无法确定调用者局部数据和临时数据的空间大小，被调用者无法完成这个任务。

(3) 被调用者保存寄存器值和其他机器状态信息。

(4) 被调用者初始化其局部数据并开始执行。

另外，用于访问非局部数据的访问链的设置也是调用代码序列的一部分，然而设置方法稍显复杂，后面会单独讨论，根据小节 7.4.4 中的方法，它只能由调用者负责设置。

返回代码序列的职责主要由被调用者负责，具体如下：

- (1) 被调用者将返回值放到与参数相邻的位置。
- (2) 被调用者根据控制链恢复调用者的 `top_sp`，恢复寄存器信息，然后跳转到机器状态字段中的返回地址。
- (3) 调用者知道返回值相对于自己的 `top_sp` 值的位置。因此，调用者可以使用返回值。

值得注意的是，上文给出的调用序列和返回序列支持过程的参数数量可变的情况。参数域的一种排列方案是将参数列表的第一个参数存放在紧挨着控制链的栈底一侧的位置，其他参数依次连续向栈底方向排列。在编译时刻，调用者知道它提供给被调用者的参数数量和类型，所以调用者知道参数域中各个参数的偏移地址，并负责将实参存放到被调用者活动记录的参数域。被调用者的目标代码必须能够处理参数数量不同的各种调用情况，所以被调用者可以通过检查第一个参数来确定所有参数的数量及位置。由于第一个参数设置在固定域（控制链）相邻的地方，因此，被调用者能很容易地确定第一个参数的偏移位置，便可根据第一个参数提供的信息找到其他参数。例如，C 语言的标准库函数 `printf`，通过解析它的第一个参数（格式字符串）中的格式说明符就能确定需要读取多少个参数及参数类型，`printf` 就能够找到其余的参数。

7.3.4 栈中的变长数据

在现代程序设计语言中，在运行时刻才能确定大小的数据对象通常被分配在堆区。但是，如果它们是过程的局部对象，也可以将它们分配在运行时刻栈中，称为栈中的变长数据。这样它们的存储空间会随着声明它们的过程的结束而自动释放，节省了空间回收的开销。例如，C99 标准引入了变长数组，允许在栈上分配长度在运行时确定的数组，称为变长数组。由于变长数据的长度不定，也会带来运行时栈溢出的安全风险。注意，只有一个数据对象是局限于某个过程，且当此过程结束时它就变得不可访问时，才可以使用栈为这个对象分配空间。

图 7-11 给出了一种为变长数组分配栈中空间常用策略。总体思路是在编译时在活动记录的局部变量区只为变长数组分配一个存放指向数组空间的指针单元。而在运行时，待数组大小确定后，再在栈顶空闲空间为这些数组分配空间，并把空间的起始地址放置在活动记录中的数组指针单元里。这样通过活动记录中的数组指针就可以间接对这些数组数据进行访问了。

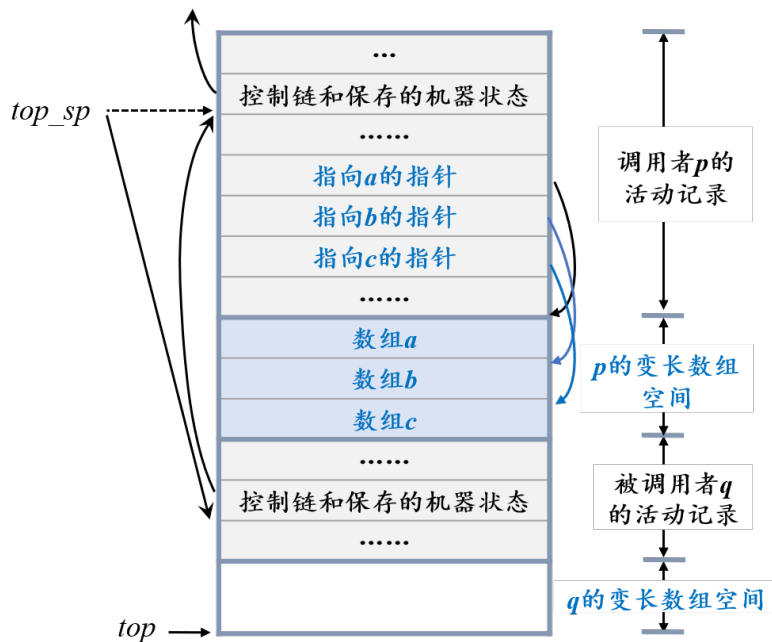


图 7-11 栈中变长数组的一种分配策略

例 7.4 如图 7-11 所示，假设过程 *p* 声明了三个变长数组，在 *p* 的活动记录中只为它们保留了三个数组指针单元。在运行时当确定了这三个动态数组的大小后，就在栈顶分别为它们分配存储空间。尽管这些数组的存储空间在栈中，但它们并不是 *p* 的活动记录的一部分。

当 *p* 执行时，三个数组指针的位置相对于 *top_sp* 的偏移量是已知的，因而目标代码可以通过这些指针间接访问数组元素。

图中还给出了一个被 *p* 调用的过程 *q* 的活动记录。*q* 的这个活动记录从 *p* 的数组之后开始，*q* 的所有变长数组被分配在 *q* 的活动记录之外。当过程 *p* 执行结束后，其活动记录连同变长数组空间一同出栈。

另外，除了指向栈顶活动记录的指针 *top_sp* 之外，还需要指针 *top* 组合使用实现此策略。指针 *top* 始终指向控制栈空闲空间的起始位置。

7.4 非局部数据的访问

一个过程除了可以使用过程自身声明的局部数据以外，还可以使用过程外声明的非局部数据，一类是全局数据，另一类是外围过程声明的数据。对于全局数据通常使用静态分配策略，将它们存储在静态区，它们的位置在编译时便

已确定，无论在何处引用这些数据，目标代码都使用确定的地址访问它们。然而对于允许过程嵌套定义的语言，访问外围过程声明的数据的机制将变得复杂，本小节重点探讨栈式存储分配策略中的外围过程声明数据的访问策略。

7.4.1 静态作用域与动态作用域

作用域规则是一组确定程序中标识符（变量、函数、类等）的可见性和生命周期的规则体系。具体来说，它解决以下核心问题，第一，绑定问题。当一个标识符在代码中被引用时，它应该绑定到哪个定义。第二，可见性问题。在程序的哪个部分可以访问某个标识符。第三，生命周期问题。标识符对应的实体何时创建、何时销毁。本节重点关注的是将变量的引用绑定到变量的定义的规则。根据作用域规则确定的时机分为静态作用域规则和动态作用域规则。

静态作用域规则指变量的作用域在程序编译时就根据代码的物理结构（嵌套块、函数定义的位置）确定了，与程序的运行时调用栈无关。编译器通过分析代码的结构就能确定一个标识符引用的是哪个变量。绝大多数现代编程语言都采用静态作用域，例如：C, C++, Java, C#, Python, JavaScript, Go, Rust, Pascal 等。

动态作用域规则指变量的作用域在程序运行时才确定，它依赖于函数的调用顺序和调用链。一个变量引用绑定到哪个定义，取决于当前执行的是哪个函数，以及它是被谁调用的。现在很少有主流语言完全采用动态作用域，但一些语言支持或曾经支持，如早期 LISP、Perl 和 APL 等。

本章重点关注静态作用域规则。在不允许过程嵌套定义的程序设计语言中，静态作用域规则相对简单，如 C、C++ 和 Java 等。以 C 语言为例，C 语言不允许在函数内部再定义函数，因此 C 函数内部只能访问在所有过程之外定义的外部变量，这类变量就是全局变量，其作用域包含此变量定义之后的所有函数体。在函数内部定义的名字的作用域就是该函数或函数内部由花括号界定的程序块。C 语言允许同名变量，遵循小作用域变量优先被引用的原则。

在允许过程嵌套定义的程序设计语言中，静态作用域规则要复杂一些，如早期的 Alogl 60、Pascal 和 ML 等。以 Pascal 语言为例，Pascal 语言允许过程的嵌套定义，即在过程内部还可以定义过程。Pascal 语言中的过程实体包括主程序（program）、过程（procedure）和函数（function）。Pascal 语言的主要作用域规则描述如下：

- （1）要求标识符必须先声明后使用；

- (2) 一个过程可以引用该过程直接定义的标识符;
- (3) 一个过程可以引用该过程所有外围过程直接定义的标识符;
- (4) 一个过程不能引用该过程内部过程定义的标识符。

(5) 最近嵌套规则，一个标识符 x 的使用对应的声明被解析为离使用点最近且作用域包含该使用点的 x 声明。也就是说，从 x 出现的作用域开始，从内向外检查各个作用域时找到的第一个对 x 的声明。

例 7.5 如图 7-12 给出了快速排序算法的一个 Pascal 程序概要。程序包含 5 个过程的定义，每个过程由标识符定义和可执行的过程体两部分组成。标识符定义包含变量定义、数组定义或过程定义。过程体部分位于 `begin` 和 `end` 关键字之间。花括号之间为注释。每个过程的代码范围如兰色的虚线标示。那么，根据 Pascal 语言的作用域规则，过程 `sort` 的过程体（位于 `begin` 与 `end` 之间的部分）可以访问 `sort` 直接定义的变量 x 、数组 a 以及过程 `readarray`、`exchange` 和 `quicksort`；不能访问过程 `partition`（属于间接定义）。过程 `partition` 有两个外围过程即 `sort` 和 `quicksort`，因此，`partition` 可以访问它们直接定义的标识符，如数组 a ，变量 x 以及过程 `readarray`、`exchange`、`quicksort` 以及 `partition` 自身；但不能访问 `readarray` 定义的变量 i ，因 `readarray` 并不是 `partition` 的外围过程。

```

program sort ( input, output );
  var a: array[0..10] of integer;
      x: integer;
  procedure readarray;
    var i: integer;
    ②    begin ... a ... end. {readarray};
  procedure exchange( i, j: integer);
    ③    begin x=a[i];a[i]=a[j];a[j]=x; end. {exchange};
  ① procedure quicksort(m, n:integer);
    var k, v: integer;
    function partition( y, z: integer):integer;
      ④      var i, j: integer;
      ⑤      begin ... a ... v ... exchange(i,j) ... end. {partition};
      begin ... a ... v ... partition ... quicksort ... end. {quicksort};
    begin ... a ... readarray ... quicksort ... end. {sort};
  
```

图 7-12 快速排序的 Pascal 语言程序概要

另外，C 语言虽然不支持过程嵌套声明，但是它的程序块机制遵循同样的嵌套作用域规则。在 C 语言的程序块中定义的变量，可以被该程序块内嵌的所有子程序块访问，这在一定程度上模拟了过程嵌套定义中数据访问的特性，但相较于真正的嵌套过程定义语言，其灵活性和复杂度都要低一些。

我们使用嵌套深度（nesting depth）用于量化包含过程嵌套定义的程序结构中作用域的嵌套层次。过程的嵌套深度可以递归计算，我们把程序中不内嵌在任何其他过程中的过程的嵌套深度设定为 1；而嵌套深度为 i 的过程 q 直接定义的过程 p 的嵌套深度为 $i+1$ 。名字的嵌套深度即为定义该名字的过程的嵌套深度。

由于嵌套层次较深的过程可以访问外围多个过程定义的非局部数据，而这些数据并不存储在该过程的活动记录中，就必须制定策略支持正确地定位到定义这些非局部数据的过程的活动记录，才能实现正确的访问。下面的几个小节将重点探讨不同的实现策略。

7.4.2 无嵌套过程定义中的数据访问

在 C 语言这类不支持过程嵌套定义的语言中，由于其非局部名字主要是全局名字，这些全局名字的存储分配与访问策略变得十分简单。

（1）全局名字的静态分配：在所有过程外部定义的全局名字，会被分配在静态存储区。这些存储单元在程序启动时便被创建，并持续存在至程序终止，其最终内存地址在编译阶段即可确定。因此，当任一过程需要访问一个非局部名字时，编译器只需生成访问该固定静态地址的代码即可。

（2）局部名字的栈式管理：过程内部定义的局部名字，则通过运行时的栈进行管理，通过指向局部名字所在的活动记录 SP 栈指针来访问。

对于过程中的某个名字的引用，首先在过程局部符号表中查找这个名字，若能找到则此名字是一个局部数据并存放于运行栈顶活动记录，通过基于 SP 栈指针的偏移量即可访问此数据。若未找到，那么一定是全局名字，直接根据符号表中静态分配的地址即可访问。

对非局部名字采用静态分配的一个关键优势在于，它天然支持过程作为参数传递或作为结果返回（在 C 语言中，通过函数指针实现）。由于作用域规则是静态的，且不存在过程嵌套，任何一个过程内部所能引用的非局部名字，必然是全局名字。这些全局名字的静态地址对于程序中的所有过程都是统一且永远有效的，无论该过程当前是否处于调用链中。

基于这一特性，即使一个函数通过指针被传递到其他作用域调用，或作为另一个函数的结果返回，其内部代码对全局名字的引用，依然能无误地定位到静态存储空间中的固定位置。这种确定性极大地简化了编译器的实现。

7.4.3 有过程嵌套定义中的数据访问

根据栈式存储分配策略，每个活动的局部数据都存放在其活动记录里，要访问这些数据使用基于此活动记录某个位置的偏移地址即可访问它们。然而对于允许过程嵌套定义的语言，一个过程还可以访问其外围过程定义的数据，称为非局部数据。此时非局部数据的偏移量通过查找符号表便可确定（包含嵌套定义过程的符号表见小节 7.6.4）。但想找到非局部数据所在的活动记录就不那么容易了。例如，当过程 p 访问非局部数据 x ，而 x 是 p 的某个外围过程 q 定义的名字，那么数据 x 就在过程 q 的最新活动记录里，就需要找到 q 的最新活动记录在控制栈中的位置。然而，即使在编译时刻知道 p 与 q 的嵌套关系，编译器也无法由此确定某个时刻它们的活动记录在运行栈中的相对位置。实际上，因为 p 或 q 甚至两者都可能是递归的，某个时刻的运行栈中可能有多个 p 或 q 的活动记录。由此可见，根据 p 的当前活动记录找到过程 q 的活动记录是一个动态的决定过程，依赖运行时刻的活动调用链，需要更多的有关活动的运行时刻信息。解决这个问题的方法有多种，主要包括使用“访问链”或“嵌套层次显示表”。

7.4.4 访问链

7.4.4.1 访问链的指向

为了能够确定包含非局部数据的活动记录的位置，可以在具有直接嵌套关系的过程的活动记录之间建立访问链（access link），使得内嵌过程的活动可以通过访问链找到直接外围过程活动的活动记录。

具体来说，就是在活动记录中增加一个访问链指针单元，访问链始终指向当前过程的直接外围过程的最近的活动记录。例如，如果过程 p 在源代码中直接嵌套在过程 q 中（即 p 的嵌套深度比 q 的嵌套深度多 1），那么 p 的任何活动中的访问链都指向最近的 q 的活动记录。由于访问链的指向主要由过程的静态嵌套关系决定，因此也称为静态链。

若过程 p 访问了非局部变量 x ，为了沿访问链找到非局部数据 x ，将 x 的地址表示为二元组 $(n_p - n_x, x \text{ 的相对地址})$ ，其中， n_p 和 n_x 分别为过程 p 和变量 x 的嵌套深度。在运行时刻，当 p 使用 x 时，只需要沿着 p 的访问链经过 $n_p - n_x$ 步就能到达 q 的活动记录，再结合 x 的偏移量，就能够找到 x 的数据空间。

下面通过示例讨论一下程序执行中访问链的指向情况以及是如何沿访问链

找到非局部数据所在的活动记录的。

例 7.6 对于图 7-12 中的 pascal 语言编写的快速排序法的程序概要，已知程序概要中各过程的嵌套深度如图 7-13。给出在该程序的某次执行中得到的一棵活动树如图 7-14。

过程	嵌套深度
<i>sort</i>	1
<i>readarray</i>	2
<i>exchange</i>	2
<i>quicksort</i>	2
<i>partition</i>	3

图 7-13 概要程序的过程嵌套深度

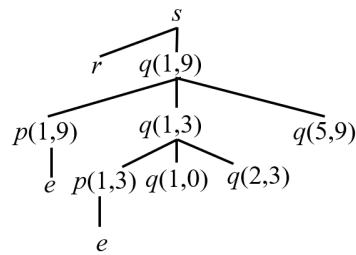


图 7-14 概要程序某次运行的活动树

此处假设访问链已被建立，看一下如何通过访问链找到非局部数据所在的活动记录。如图 7-15(a)所示，当过程 *sort* 启动时，其访问链为空（*sort* 无外围过程）。当过程 *sort* 调用过程 *readarray* 时，*readarray* 的活动记录进栈，根据访问链的定义，*readarray* 的访问链指向栈中直接外围过程 *sort* 的活动记录。*readarray* 中非局部数据 *a* 的地址二元组为 $(1, a \text{ 的偏移量})$ ，这样沿 *readarray* 访问链经过 $n_r - n_a$ 即 1 步便能到达 *sort* 的活动记录，再使用偏移量找到 *a*。

再如图 7-15(b)所示，当 *partition(1,9)* 的活动记录进栈，其访问链指向 *partition* 的直接外围过程 *quicksort(1,9)* 的活动记录，而 *quicksort(1,9)* 的访问链指向 *quicksort* 的直接外围过程 *sort* 的活动记录。*partition(1,9)* 中非局部数据 *a* 的地址二元组为 $(2, a \text{ 的偏移量})$ ，这样沿 *partition(1,9)* 的访问链经过 $n_p - n_a$ 即 2 步就能到达 *sort* 的活动记录，再使用偏移量找到 *a*。

如图 7-15(c)所示，当运行栈中有两个 *quicksort* 的活动记录时，*partition(1,3)* 的访问链指向它的直接外围过程 *quicksort* 的离 *partition(1,3)* 活动记录最近的 *quicksort(1,3)* 的活动记录。在图 7-15(d)中，当 *exchange* 的活动记录进栈时，它

的访问链绕过了过程 *partition* 和过程 *quicksort* 的活动记录直接指向了 *exchange* 的直接外围 *sort* 的活动记录，这样，过程 *exchange* 访问 *a* 时，只需沿访问链跳转 1 步就能到达 *a* 所在的 *sort* 的最新活动记录。

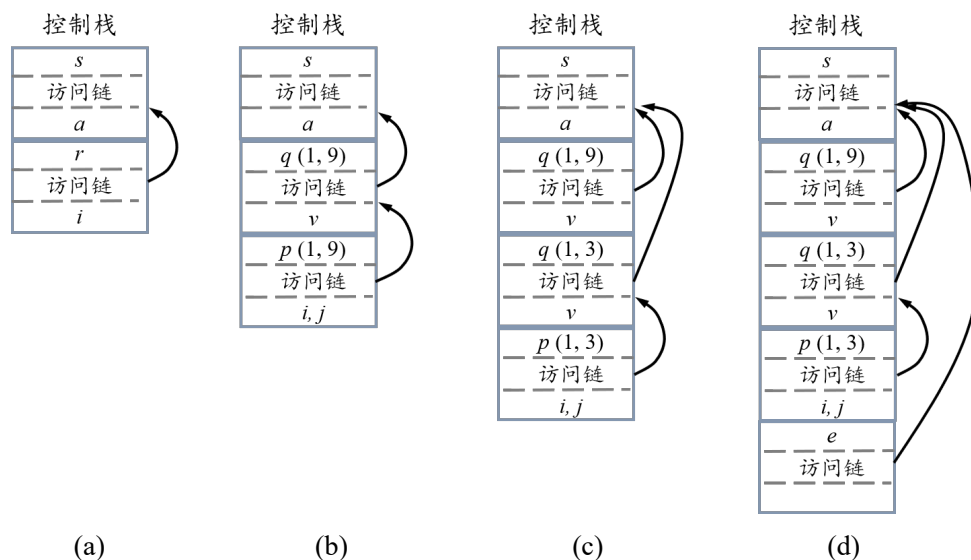


图 7-15 访问链的指向示意图

7.4.4.2 访问链的建立

访问链的建立属于调用代码序列，当一个过程 *q* 通过名字显式地调用另一个过程 *p* 时，由调用者负责维护访问链，方法相对简单，下面分三种情况讨论。

(1) 若调用过程 *q* 的嵌套深度 n_q 小于被调用过程 *p* 的嵌套深度 n_p ，即 $n_q < n_p$ 。可知 *q* 是外围过程，*p* 是内层过程，按照静态作用域规则，一定是过程 *q* 直接包含过程 *p*，即 $n_q = n_p - 1$ ，否则 *q* 不能调用 *p*。因此，按照访问链的定义，*p* 的访问链一定指向调用者 *q* 的活动记录。

这种情况的例子如图 7-15(a) 中过程 *sort* 调用过程 *readarray* 时，图 7-15(b) 中的过程 *sort* 调用过程 *quicksort*(1,9)，以及过程 *quicksort*(1,9) 调用过程 *quicksort*(1,3) 时，访问链的建立如图中所示。

(2) 若调用过程 *q* 的嵌套深度 n_q 大于被调用过程 *p* 的嵌套深度 n_p ，即 $n_q > n_p$ 。按照静态作用域规则可知此时一定是内层过程 *q* 调用外围过程 *p*，为了设置 *p* 的访问链，需找到过程 *p* 的直接外围过程 *r*，而过程 *r* 一定直接定义了过程 *p*，而过程 *p* 内嵌套定义了过程 *q*。此时，为了找到 *r* 的活动记录，只需沿着 *q* 的活动记录访问链经过 $n_q - n_p$ 步，即可到达与过程 *p* 同层的某个活动的活动记录，再沿着访问链经过一步即可到达 *r* 的活动记录。因此，总计沿着调用过程

q 的活动记录访问链经过 $n_q - n_p + 1$ 步即可找到被调用过程 p 的直接外围过程 r 的最新活动记录，最后将被调用过程 p 的访问链指向此活动记录。

这种情况的例子，如图 7-15(d)中的 $partition(1,3)$ 调用 $exchange(1,3)$ 。此时 $partition$ 的嵌套深度为 3，过程 $exchange$ 的嵌套深度为 2，符合 $n_p > n_e$ 的情况，此时，想找到过程 $exchange$ 的直接外围过程 $sort$ 的活动记录，只需沿着调用过程 $partition(1,3)$ 的访问链经过 $n_p - n_e$ 即 1 步即可到达与 $exchange$ 同层的 $quicksort$ 的活动记录，再经过 1 即可到达 $sort$ 的活动记录，可令 $exchange$ 的访问链指向此 $sort$ 的活动记录。

(3) 若调用过程 q 的嵌套深度 n_q 等于被调用过程 p 的嵌套深度 n_p ，即 $n_q = n_p$ 。可知调用过程 q 与被调用过程 p 处于同一层次，因此过程 q 的直接外围过程便是过程 p 的直接外围过程，因此，只需将 q 活动的访问链赋值给 p 活动的访问链即可。

这种情况的例子如图 7-15(d)中的 $quicksort(1,9)$ 调用 $quicksort(1,3)$ ， $quicksort(1,9)$ 与 $quicksort(1,3)$ 嵌套深度相同，具有相同的直接外围过程 $sort$ 。因此，只需将 $quicksort(1,3)$ 的访问链与 $quicksort(1,9)$ 的访问链指向相同的活动记录即可。

下面考虑过程作参数被调用的情况，在支持将过程作为参数传递的编程语言中，若过程 q 包含一个过程参数 f ，过程 q 在过程体中显示地调用过程 f ，就需要为其建立访问链。由于过程 q 不知道将会传递给它的真正的过程名字，而且不同的调用对应的实参过程名字总是不同的，因此过程 q 无法为被调用过程 f 建立访问链。然而，若是某过程 s 调用了过程 q 且传递给它过程参数 p ，那么 s 一定知道过程 p 的访问链信息，因此由 q 的调用者 s 计算过程 p 的访问链，然后将其作为参数与过程 p 一并传递给过程 q ，在过程 q 调用过程 p 时（通过参数 f 调用）就可以为 p 建立访问链了。

7.4.5 嵌套层次显示表

通过访问链访问非局部数据的问题之一是访问链的建立和沿访问链的数据访问都要沿着访问链路追踪，追踪次数都与嵌套深度有关，如果嵌套深度很大，就必须沿着一段很长的访问链路才能找到需要的数据。为了解决这一问题，另一种实现方法是引入嵌套层次显示表来提高访问外围名字的访问效率。

嵌套层次显示表（display）简称显示表，是一个全局指针数组 d ，其长度为程序中所有过程嵌套深度(层号)的最大值。在任何时刻， $d[i]$ 均指向运行栈中

最新建立的嵌套深度为 i 的过程的活动记录（在运行栈中可能存在多个嵌套深度为 i 的过程的活动记录）。

如果要访问某个嵌套深度为 i 的非局部名字 x ，只要沿着指针 $d[i]$ 即可找到 x 所属过程的活动记录，此时 x 的地址可以使用如下二元组表示：（嵌套深度，偏移量）。

为什么沿着指针 $d[i]$ 一定能找到嵌套深度为 i 的非局部名字 x 所属过程的活动记录？这是由静态作用域规则决定的。若一个过程 q 访问了非局部名字 x ，那么它的每个外围过程至少都有一个活动记录在控制栈中，而且过程 q 的每个外围过程的嵌套深度值都是唯一的。因此，根据 q 访问的非局部数据 x 的嵌套深度 k 就表明了它一定是 q 的嵌套深度为 k 的外围过程 p 定义的，因此， $d[k]$ 所指向过程 p 的最新的活动记录一定是 x 所在的活动记录。

例 7.7 如图 7-16(c) 所示，控制当前在 *exchange* 的活动，*exchange* 访问数组 a ，因 a 的嵌套深度为 1，因此，一定有一个 *exchange* 的外围过程 s 的活动记录在栈中，而且 $d[1]$ 一定指向这个最新的 s 的活动记录。

在过程调用或返回时，必须正确地维护显示表。在嵌套深度为 n_p 的过程 p 被调用时， p 的活动记录入栈，需要修改 $d[n_p]$ 指向 p 的活动记录，但为了能在 p 返回时恢复 $d[n_p]$ 的原值，需要先将 $d[n_p]$ 的原值保存在 p 的活动记录中；然后再修改 $d[n_p]$ 指针指向过程 p 的活动记录。而在过程 p 返回时，需先恢复保存的 $d[n_p]$ 原值，然后 p 的活动记录才能出栈。

例 7.8 对于图 7-14 中的概要程序的某次运行，图 7-16 给出了显示表的部分状态。如图 7-16(a) 所示，当嵌套深度为 1 的过程 *sort* 调用了嵌套深度为 2 的 *quicksort(1,9)*。*quicksort(1,9)* 的活动记录入栈，成为最新的深度为 2 的活动记录，在更新显示表 $d[2]$ 的值之前，先将 $d[2]$ 的原值保存在 *quicksort(1,9)* 的活动记录中，然后再更新 $d[2]$ 的值指向 *quicksort(1,9)* 的活动记录。实际上，在 *quicksort(1,9)* 的活动记录进栈前，并没有深度为 2 的活动记录，因此 $d[2]$ 的原值是空指针。

如图 7-16(b) 所示，当嵌套深度为 2 的过程 *quicksort(1,9)* 调用嵌套深度为 2 的过程 *quicksort(1,3)*，*quicksort(1,3)* 的活动记录入栈，成为最新的深度为 2 的活动记录，先将 $d[2]$ 的原值保存在 *quicksort(1,3)* 的活动记录中，然后将 $d[2]$ 指向 *quicksort(1,3)* 的活动记录。实际上，保存在 *quicksort(1,3)* 的活动记录中的 $d[2]$ 的原值是指向栈中前一个深度为 2 的过程 *quicksort(1,9)* 的活动记录。

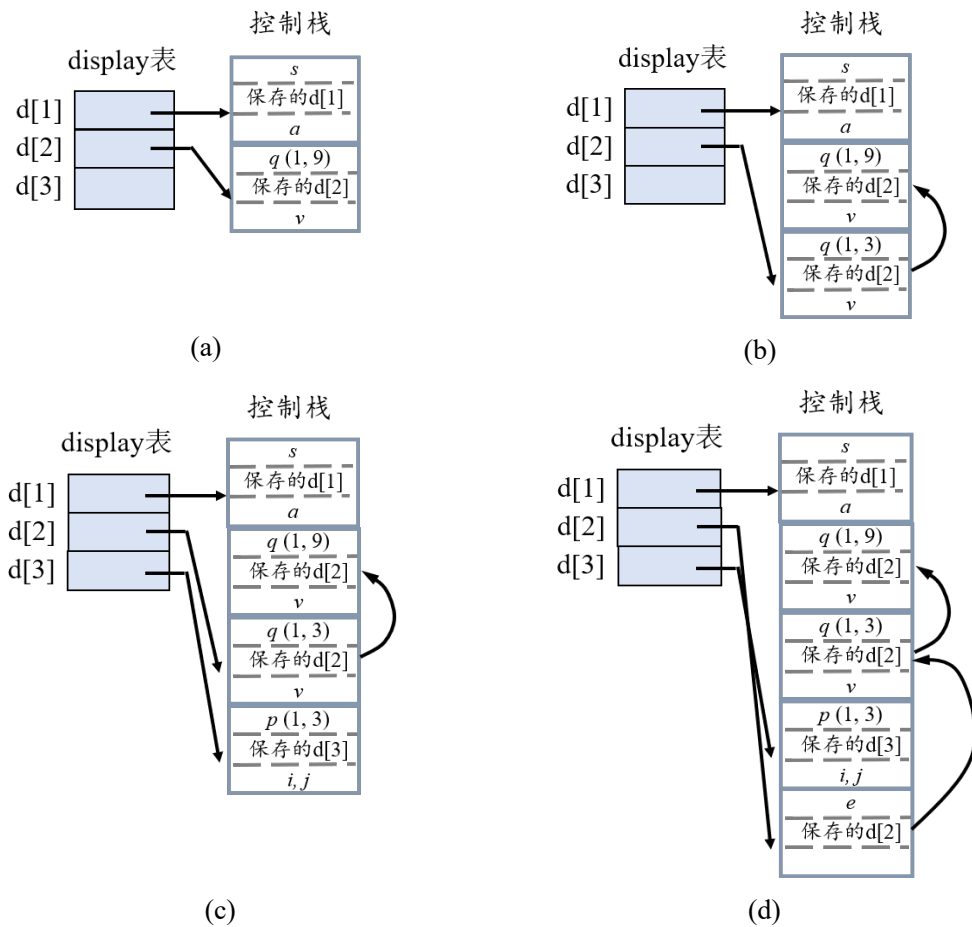


图 7-16 维护显示表

接下来，如图 7-16(c)所示，当嵌套深度为 2 的过程 *quicksort*(1,3)调用嵌套深度为 3 的过程 *partition*(1,3)，*partition*(1,3)的活动记录入栈，成为最新的深度为 3 的活动记录，需更新显示表 *d*[3]的值，先将 *d*[3]的原值（空指针）保存在 *partition*(1,3)的活动记录中，然后将 *d*[3]指向 *partition*(1,3)的活动记录。

再如图 7-16(d)所示，当前栈中共有三个嵌套深度为 2 的过程的活动记录，*exchange*(1,3)、*quicksort*(1,3)以及 *quicksort*(1,9)，显示表的 *d*[2]直接指向 *exchange*(1,3)的活动记录，*exchange*(1,3)的活动记录中“保存的 *d*[2]”单元指向了 *quicksort*(1,3)的活动记录，同样它的“保存的 *d*[2]”单元指向了 *quicksort*(1,9)的活动记录。可见，使用这种策略，无论栈中有多少个具有相同嵌套深度 *i* 的活动记录，都可以使用“保存的 *d*[*i*]”单元将它们链接在一起，并在需要时恢复到显示表 *d*[*i*]的元素位置。

分析上面的显示表策略可见，调用过程时显示表项的保存和修改以及调用返回时显示表项的恢复都只需要常数时间可完成，与嵌套深度无关。使用显示表访问非局部数据的操作也只需要常数时间可完成。另外，显示表通常作为全局的静态数组保存在静态数据区。

7.5 堆式分配策略

7.5.1 堆式分配的主要任务

堆式存储分配是一种更为灵活和通用的动态存储管理策略。它负责管理一片称为“堆”的内存区域。程序可以在运行时显式地（如通过 `malloc`, `new`）或隐式地从堆中申请任意大小的内存块，并且这块内存的生命周期不再与某个函数的调用周期绑定。一个在函数内部分配的对象，只要其引用（或指针）被正确传递，可以在函数返回后继续存在。这种特性使得堆式分配成为实现动态数据结构（如链表、树、图等）以及生命周期不确定的数据对象的基石。堆式管理的核心挑战在于如何高效地满足大小不一、随机到来的内存申请与释放请求，同时尽量减少内存碎片并保持较低的时间开销。

堆式分配管理器的两个主要任务是分配和回收。分配是指当收到一个内存申请请求时，在堆中找到一块合适大小的空闲内存满足请求；回收是指当一块内存不再使用时，将其有效地归还给堆，以便后续重新分配。回收策略根据其自动化程度，主要分为两大类，显式回收和隐式回收。显式回收由程序员手动管理（如 C/C++ 中的 `free` / `delete`）；隐式回收：由运行时系统自动管理，即垃圾回收。

7.5.2 显式堆式分配方法

在显式回收模型中，程序员负责手动释放不再使用的堆内存。分配器需要高效地组织和管理空闲内存块。分配器通常维护一个空闲链表，将所有空闲内存块链接起来。根据链表的组织方式，主要有以下方法：

（1）隐式空闲链表

在每一个内存块（无论是已分配还是空闲）的头部和尾部嵌入边界标签，包含块大小和分配状态位。所有块在内存中顺序排列，通过头部的大小字段可以隐式地遍历整个堆。当寻找空闲块时，需要顺序扫描整个隐式链表，直到找到足够大的块。优点是实现简单；缺点是分配时间为 $O(n)$ ，效率较低。

(2) 显式空闲链表

将所有空闲块用前驱和后继指针链接成一个双向链表。分配器只需遍历这个空闲链表，而无需查看已分配的块。分配时，从显式空闲链表中寻找合适的块。优点是分配时间与空闲块数量成正比，而非总块数，效率高于隐式链表。缺点是空闲块需要额外空间存储指针。合并空闲块时更为复杂。

(3) 分离空闲链表

维护多个空闲链表，每个链表中的块大小在一个特定的范围内。例如，一个链表专门管理小尺寸块，另一个管理中等尺寸块，等等。分配申请大小为 n 的块时，首先找到对应尺寸范围的链表，然后在该链表中寻找合适的块。优点是大幅减少了寻找合适块所需的时间，是实践中广泛采用的高效方法。

常见策略：

简单分离存储：每个链表中的块大小固定。分配和释放非常快，但可能造成内部碎片。

分离适配：著名的 `dlmalloc` 等分配器采用此策略。每个链表包含一个大小类的块。分配时，先在对大小类的链表中查找；若找不到，则分割一个更大的块，并将剩余部分放入其他链表中。释放时进行合并。这是速度与空间效率的极佳折衷。

寻找匹配块的策略

当为一个请求寻找空闲块时，分配器采用以下策略之一。

(1) 首次适应

流程：从链表起始处开始扫描，选择第一个满足大小的空闲块。

优点：倾向于将大块保留在链表尾部。

缺点：链表头部会积累许多小碎片，增加头部扫描开销。

(2) 下次适应

流程：从上一次搜索结束的位置开始扫描。

优点：比首次适应更均匀地遍历链表。

缺点：内存利用率通常低于首次适应。

(3) 最佳适应

流程：扫描整个空闲链表，选择满足要求的最小空闲块。

优点：试图保留大块内存，减少内部碎片。

缺点：通常性能最差，因为需要全程扫描，且会产生大量无用的外部小碎片。

空闲块合并

为了减少外部碎片，当释放一个块时，需要检查其相邻的块是否也是空闲的。如果是，则将它们合并成一个更大的空闲块。

立即合并：在释放一个块时，立即与相邻空闲块合并。

推迟合并：直到某个时刻（如分配失败时）才进行全面的合并操作。这可以避免某些“抖动”情况，但可能使碎片暂时存在。

7.5.3 垃圾回收

在支持垃圾回收的语言中（如 Java, Python, Go），程序员无需手动释放内存。运行时系统会自动识别并回收不再使用的内存（即“垃圾”）。

垃圾回收器的两个基本任务是垃圾识别和垃圾回收。垃圾识别是确定哪些内存对象是程序永远无法再访问的。垃圾回收指回收这些垃圾对象占用的内存。

垃圾识别策略有以下几种。

（1）引用计数法

在每个对象中维护一个引用计数器，记录指向该对象的引用数量。当引用关系建立时，计数器加 1；当引用关系解除时，计数器减 1。当计数器减至 0 时，立即回收该对象。优点是垃圾可以被立即回收，延迟低。缺点是无法处理循环引用（如对象 A 引用 B，B 引用 A，但无外部引用，两者计数器均为 1，无法回收）。另外，计数器更新开销较大。

（2）跟踪式回收器

这类回收器会周期性地暂停程序（“Stop-The-World”），通过一组称为“GC Roots”的根对象（如全局变量、栈上的局部变量等）出发，遍历所有可达对象。不可达的对象即为垃圾。

标记-清除流程：标记阶段为从 GC Roots 开始遍历对象图，对所有可达对象进行标记。清除阶段，线性遍历整个堆，回收所有未被标记的对象（即垃圾）到空闲链表。优点是解决了循环引用问题。缺点是执行期间需要暂停整个程序；会产生内存碎片。

标记-整理流程：标记阶段与“标记-清除”相同。整理阶段为将所有存活的对象向内存的一端“滑动”，紧凑地排列，从而消除所有外部碎片。优点是完全消除了外部碎片。缺点是对对象移动的开销较大，且需要更新所有指向被移动对象的引用。复制式回收是将堆空间分为两个相等的半区（From-space 和 To-space）。程序只在 From-space 中分配内存。当 From-space 快满时，启动垃

圾回收。其流程是从 GC Roots 出发，将所有 From-space 中的存活对象复制到 To-space 中，并紧凑地排列在 To-space 的一端。

复制完成后，交换 From-space 和 To-space 的角色。

优点：分配速度极快（仅需一个指针递增）；在回收的同时完成了内存整理，无碎片。

缺点：内存利用率只有 50%；复制大量存活对象开销大。

（3）现代高级垃圾回收技术

为了提升性能，现代垃圾回收器引入了更复杂的思想：

分代收集。核心假设：弱分代假说——绝大多数对象都是“朝生夕死”的。方法：将堆划分为多个“代”（通常是两个：年轻代和老年代）。新创建的对象在年轻代。年轻代采用高效的复制算法进行频繁的、快速的回收。经历过多次年轻代回收仍存活的对象会被提升到老年代。老年代中对象死亡率低，因此使用频率较低但吞吐量高的算法（如标记-清除或标记-整理）。优点：极大地降低了垃圾回收的平均开销，是现代回收器的基石。

增量回收与并发回收。目标：解决“Stop-The-World”导致的程序长时间暂停问题。增量回收：将回收工作分成多个小步骤，与用户程序交替执行。并发回收：回收器的大部分工作（如标记）与用户程序同时运行。挑战：在用户程序修改对象引用关系的同时进行回收，需要复杂的同步机制（如读/写屏障）来保证正确性。

堆式存储分配为程序提供了最大的灵活性，是动态数据结构的生命线。理解堆式存储分配的原理与方法，对于编写高效、健壮的系统软件和应用软件至关重要。

7.6 符号表

编译器在编译过程中需要不断地汇集和反复查证出现在源程序中各种名字的属性、特征和值等有关信息，他们通常被记录在一个或几个符号表中。这些信息的收集与使用通常贯穿编译器从分析到综合的多个阶段。本小节将讨论符号表在组织以及在编译器各个阶段的作用和使用方法。

7.6.1 符号表的信息

符号表（symbol table）是编译器用于存储和管理源程序中名字的各种属性信息的数据结构。这些信息主要包括名称（标识符）、种类、类型、存储地址、

值以及作用域等。其中名称是符号表查询和检索的唯一依据，根据名字的不同种类，需要收集的具体信息有所不同。种类主要包括符号常量、变量、数组、类型、函数等。

- (1) 符号常量和枚举符号通常只存储值的信息；
- (2) 普通变量存储类型（全局或静态等）、地址等信息；
- (3) 数组通常存储维度、各维度的宽度、上下界及类型等信息；
- (4) 函数需要保存参数数量、名称、类型及传递方式，返回值的类型等信息；
- (5) 结构体类型需要记录其成员列表，每个成员的名称、类型和在结构体内部的偏移量；

符号表中的大部分信息通常在语义分析阶段填入。当编译器在进行语义分析时，若遇到一个声明语句，会立即向符号表插入一个新条目，并填充大部分属性。例如，分析常量、变量或数组声明、分析类型声明和分析过程定义时。详见第 6 章。

然而，还有些信息无法在声明时立即确定，需要在后续阶段补充或回填到已存在的符号表条目中。例如当编译器为函数的活动记录（栈帧）进行内存布局时，它会根据活动记录的布局以及栈顶指针（或帧指针）的位置，计算每个局部变量和形参相对于活动记录基地址（栈指针或帧指针）的偏移量，然后将这个值回填到符号表对应条目的“地址/偏移量”字段中。全局变量和静态变量的绝对地址可能在链接时才能最终确定。另外，变量的地址描述符也可以存在符号表中，它们是在目标代码生成阶段才能确定并填入的信息。

7.6.2 符号表的作用

符号表的作用是将名字的相关信息从声明的位置传递到使用它们的位置。这些信息主要用于语义分析中的类型检查、一致性检查以及中间代码和目标代码生成阶段。

在语义分析阶段，符号表是进行上下文相关检查的依据。

(1) 标识符声明检查。当在代码中使用一个标识符时，编译器会查询符号表，确认该标识符是否已在当前作用域或外围作用域中声明。如果未找到，则报“未声明的标识符”错误。

(2) 重复声明检查。当遇到一个声明语句时，编译器会查询当前作用域的符号表，检查是否已存在同名的标识符。如果存在，则报“重复声明”错误。

(3) 类型检查。这是语义分析最核心的工作之一。编译器利用符号表中存储的类型信息来验证程序的语义是否正确。主要包括赋值类型相容性、表达式运算分量类型相容性以及函数调用实参类型相容性。如果类型不同但相容，编译器还需要生成类型转换代码。

(4) 作用域管理。符号表需要高效地处理不同作用域（如全局作用域、过程作用域、块作用域）中的标识符。一方面是管理作用域的进入与退出。当编译器进入一个新的作用域（如一个过程或一个语句块）时，它会“开启”一个新的子符号表或作用域层级。当退出该作用域时，会“关闭”或销毁该层符号表。另一方面是实现名字解析。当使用一个标识符时，符号表的查询机制会遵循“最近嵌套原则”，首先在当前作用域查找，如果找不到，则逐级向外围作用域查找，直到找到为止。这确保了使用的是正确作用域内的正确实体。

在中间代码生成阶段，符号表的作用主要包括以下几个方面。

(1) 提供类型信息，指导运算指令的选择。中间代码指令并非与机器指令一一对应，但其操作必须遵循类型的约束。符号表提供的精确数据类型信息，决定了应生成何种类型的中间操作并判断是否需要添加类型转换指令。

(2) 根据符号表中的信息计算数据访问的相对地址。编译器需要决定活动记录中各个区域的逻辑布局，并使用相对于活动记录区某个位置（栈帧指针）的“相对地址”来引用这些数据，也称为“偏移量”。符号表在此时提供了存储布局的关键信息。对于简单变量，符号表记录该变量在其活动记录中的偏移量。对于复合类型（数组、结构体），符号表可以存储数组的基地址、元素类型、维数、各维宽度和各维边界等。这些信息是某些编程语言实现数组引用的安全性检查和生成数组引用地址计算代码的重要信息依据。符号表存储了结构体类型的成员列表和每个成员的偏移量。代码在访问结构体变量的某个成员时，中间代码生成器通过查询符号表，基于变量相对地址和该成员在变量中偏移量计算出此成员基于栈帧指针的相对地址，从而生成准确的访问指令。

(3) 管理函数调用，确保接口一致性。函数调用是程序的基本结构，其正确的中间代码生成严重依赖于符号表中的信息。当遇到函数调用时，根据符号表中参数传递方式，生成相应的参数压栈或设置指令。例如，传引用需要传递变量的地址，而这个地址信息同样来源于符号表。最后，返回值的处理。如果函数有返回值，其类型信息也记录在符号表中。这决定了调用过程如何接收返回值，以及如何将其存储到目标变量中。

(4) 存储类别的确定。根据符号表中记录的 `static`、`auto` 等属性，决定变量应分配到静态区还是栈帧。

在目标代码生成阶段，符号表的主要作用。

(1) 将相对地址或偏移量绑定到物理寄存器。为全局变量、静态变量分配绝对地址或生成重定位入口；编译器将中间代码阶段计算的逻辑布局，与目标机器的具体寄存器关联起来。例如，它决定使用 `%ebp` 作为栈帧指针，那么符号表中那个 `offset: -8` 的逻辑地址就被具体化为 `-8(%ebp)` 这样的物理地址。如果使用 `%fp`，则变成 `-8(%fp)`。

(2) 支持过程调用序列的生成。过程调用涉及复杂的栈帧管理（活动记录），符号表为此提供了依据。符号表中过程的条目记录了局部变量总大小，用于在栈上分配空间；参数信息包括参数的个数、类型以及在栈帧或寄存器中的传递位置。符号表信息帮助生成正确的栈帧销毁和返回指令。

(3) 生成符号信息。编译器将符号表输出，供调试器（如 GDB）使用，形成调试符号表，建立源代码符号与机器指令地址间的映射，这是实现源代码级调试的基础。

除此之外，符号表还需支持符号的重定位与链接信息生成。

7.6.3 单作用域符号表的组织

单作用域指所有名字都在同一个作用域中，此时无需考虑作用域的嵌套关系，实现简单。单作用域符号表的组织可以使用一张大表保存所有种类的符号，也可以使用多张小表分别保存不同种类的符号。

当各种类型的符号共用一张符号表时，如图 7-17，优点是数据结构统一，实现容易。但由于不同种类符号的属性信息数量及组成具有很大的差异会造成符号表空间的浪费。例如常量符号通常只有类型和值信息；而数组需要保存类型、基址、维度、宽度或上下界等信息。因此常量符号的对应的符号表条目大量的空间会闲置。

另外，标识符名称的长度差异也较大，为了节省存储空间，可以引入单独的字符串表，将符号表中的全部符号的名称集中放在这个字符串表中，而在符号表表项的名字域只保留指向字符串表中标识符的首字符，以及标识符的长度。

单作用域符号表的另一种组织方式就是建立多个符号表来管理源程序中出现的各种符号，如常数表、变量表、函数表、数组表等。这种管理方式虽然可以较好地解决空间利用问题，但又会导致不同种类符号出现重名的问题。为避免重名问题，插入或查找某个符号时需要查看所有的符号表，显然，相比共用一张符号表的情况而言，多张表符号表上的插入和查找操作要更费时一些。

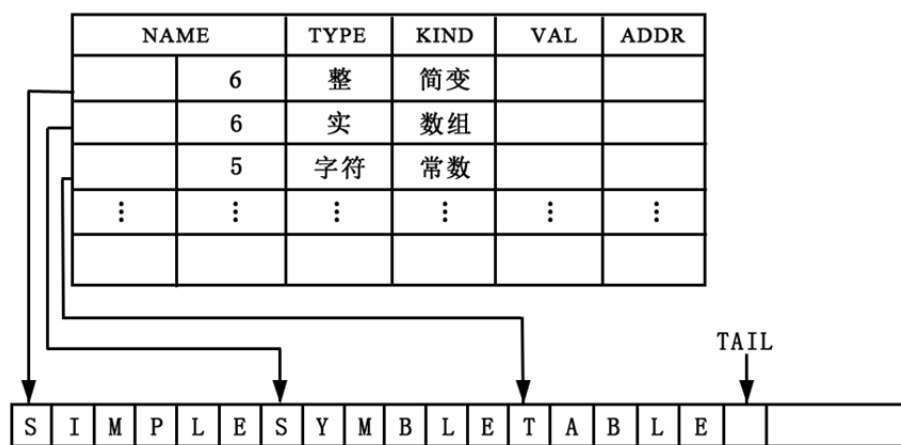


图 7-17 单作用域符号表的单表组织

综合上述两种情况，从空间效率上改造存放多种符号的共用表应该是处理符号表属性的更佳方案。一种可行的策略是将符号表中的属性分割成基本属性和扩展属性两部分：基本属性（名称、类型、地址等）直接存放在符号表中，扩展属性则用一段动态申请的内存存放。在符号表的基本属性部分要设置一个指针域，对具有扩展属性的符号对应的表项，它指向扩展属性的指针；对于没有扩展属性的符号对应的表项，则将该指针项置空。例如，对于数组，扩展属性存放在称为内情向量的表中，表中存储与维度相关的信息。如图 7-18 所示。

名字	种类	类型	地址	扩展属性指针
sum	变量	int	0	NULL
average	变量	float	4	NULL
score	数组	int	8	维数 各维维长 2 3 4
...	...	...	...	

图 7-18 带有扩展属性域的单符号表

7.6.4 嵌套作用域符号表的组织

在允许过程嵌套定义的编程语言中，过程的嵌套定义关系一般决定了作用域的嵌套关系；另外，在与 C 语言类似的编程语言中，通过语句块嵌套定义也实现了作用域的嵌套关系。都可以使用本节类似的方法实现。包含嵌套作用域的符号表的组织需重点考虑的两个问题，首先是如何刻画过程之间的嵌套关系

即作用域信息，然后是如何支持不同作用域符号重名。

作用域信息对于非局部数据的地址确定非常重要，例如，要通过查找符号表确定非局部数据所在的活动记录以及相对于活动记录的偏移地址，这些信息是通过过程之间的作用域嵌套关系信息获得的。

例 7.12 假设过程 p 要访问过程 q 中的数据对象 x ， x 存放在过程 q 的活动记录里， x 的逻辑地址由 q 的活动记录位置和 x 的偏移地址两部分决定。 q 的活动记录可以通过访问链确定。而 x 的偏移地址需要通过符号表表示的作用域关系来确定 x 本次使用点对应的是哪个 x 声明点（对应符号表中哪个条目）。

因此，符号表中需要维护过程间的嵌套关系这一信息（指针）实际上就是作用域信息。另外，不同过程中可能出现具有相同名字的局部变量，将所有过程中的名字登记在同一张表上显然不合适。因此，常用的组织方式是为每个过程建立一个符号表，同时需要建立起这些符号表之间的联系，用来刻画过程之间的嵌套关系。

例 7.13 对于图 7-12 中的 pascal 语言编写的快速排序法的程序概要，其符号表组织如图 7-19。程序包含五个不同作用域的过程 *sort*、*readarray*、*exchange*、*quicksort* 和 *partition*，因此为每个过程建立一个符号表。在每个过程的符号表中包含此作用域中声明的名字，即变量、数组或过程。每个符号表条目包含名字、类型和地址信息。每个符号表表头还保存了对应过程的局部数据区的大小，此值可用于为栈中此过程对应的活动记录预留局部数据空间。例如过程 *sort* 符号表中包含它声明的一个数组 a 、一个整型变量 x 和三个过程 *readarray*、*exchange* 和 *quicksort*；表头记录了 *sort* 过程的局部数据区的大小为 48。再如过程 *partition* 符号表中包含了此作用域中声明的两个变量 i 和 j ，局部数据区的大小为 8。

由图 7-19 可见，不同的两个过程 *readarray* 和 *partition* 中可以包含重名的变量 i 声明。而作用域之间的直接嵌套关系是通过符号表之间的一对指针来表示。一个指针由外围过程指向内层过程的符号表，另一个指针由内层过程的符号表指向直接外围过程符号表。

在图 7-19 中，过程 *sort* 直接包含内层过程 *readarray*，因此，由 *readarray* 在 *sort* 符号表中的条目的地址域设置指针指向 *readarray* 过程的符号表（如图中兰色箭头）；同时在 *readarray* 过程的符号表表头设置指针指向直接外围过程 *sort* 符号表（如图中红色箭头），以此表示它们之间的直接嵌套关系。对于 *sort* 声明的其它两个过程 *exchange* 和 *quicksort* 也是一样的。

当在 *partition* 过程中出现变量 i 的使用点时，按照最近嵌套规则，首先从

partition 的符号表开始查找变量 *i*，便可直接在符号表中查找到 *i* 在活动记录上的偏移地址。当出现非局部数据 *a* 的使用时，按照最近嵌套规则，首先从 *partition* 的符号表开始查找，未找到，再沿着 *partition* 的符号表表头中指向直接外围作用域（过程）的指针到达 *quicksort* 过程符号表，仍未找到，再沿着 *quicksort* 的符号表表头中指向直接外围作用域（过程）的指针到达 *sort* 过程符号表，便会成功找到 *a* 的偏移地址。

另一方面，还可以将过程的嵌套深度信息保存在表头，这样就可以根据 *a* 的嵌套深度 n_a 与 *partition* 的嵌套深度 n_p ，沿访问链追踪到 *a* 所在的 *sort* 过程活动记录（或使用显示表 $d[n_a]$ 直接定位）了，从而实现了对 *a* 的访问。

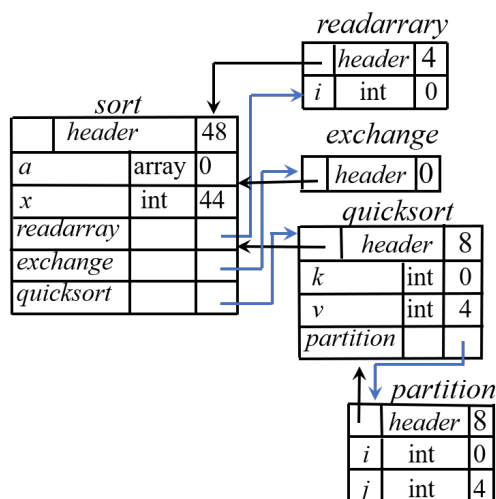


图 7-19 嵌套作用域的符号表组织

事实上，过程的参数也可以采用与局部变量类似的方法，登记在独立的符号表中，参数的相对地址不参加局部变量的顺序分配，因为如 7.3.2 小节所述，参数区在活动记录中的位置与局部变量不同。

7.6.5 嵌套作用域符号表的建立

在允许嵌套过程声明的语言中，局部于每个过程的名字可以使用第 6 章介绍的方法分配相对地址；当遇到内嵌在过程 *q* 内的过程 *p* 时，应暂时挂起对外围过程 *q* 声明语句的翻译，转而翻译过程 *p* 的声明语句。*p* 处理完后，再接着翻译过程 *q* 的剩余代码。过程定义亦满足嵌套性，因此可以使用栈来管理不同过程的符号表指针，实现在不同过程符号表之间的转换。

给定嵌套过程声明语句的文法(7.1):

$$\begin{aligned} (1) P &\rightarrow D \\ (2) D &\rightarrow D D \mid \text{proc id}; D S \mid \text{id}: T; \end{aligned} \quad (7.1)$$

其中, T 是表示类型定义的语法成分, 其产生式如第 6 章。

我们约定, 主程序外面还有一个语言预定义过程。实现嵌套过程声明语句翻译的后缀 SDT 如图 7-20, 此 SDT 专注构建类似图 7-19 的嵌套符号表结构。

$$\begin{aligned} (1) P &\rightarrow M D && \{ \text{addwidth}(\text{top}(\text{tblptr}), \text{top}(\text{offset})); \\ & && \text{pop}(\text{tblptr}); \\ & && \text{pop}(\text{offset}); \} \\ (2) M &\rightarrow \varepsilon && \{ t = \text{mktable}(\text{nil}); \\ & && \text{push}(t, \text{tblptr}); \\ & && \text{push}(0, \text{offset}); \} \\ (3) D &\rightarrow D_1 D_2 && \\ (4) D_p &\rightarrow \text{proc id}; N D_1 S && \{ t = \text{top}(\text{tblptr}); \\ & && \text{addwidth}(t, \text{top}(\text{offset})); \\ & && \text{pop}(\text{tblptr}); \\ & && \text{pop}(\text{offset}); \\ & && \text{enterproc}(\text{top}(\text{tblptr}), \text{id.lexeme}, t); \} \\ (5) D_v &\rightarrow \text{id}: T; && \{ \text{enter}(\text{top}(\text{tblptr}), \text{id.lexeme}, T.\text{type}, \text{top}(\text{offset})); \\ & && \text{top}(\text{offset}) = \text{top}(\text{offset}) + T.\text{width}; \} \\ (6) N &\rightarrow \varepsilon && \{ t = \text{mktable}(\text{top}(\text{tblptr})); \\ & && \text{push}(t, \text{tblptr}); \text{push}(0, \text{offset}); \} \end{aligned}$$

图 7-20 嵌套过程声明语句的 SDT

此 SDT 中使用两个栈来管理在符号表之间的转换, 其中, tblptr 栈用于保存因遇到新的过程定义, 处理被中断而被挂起的过程的符号表的指针; offset 栈用于保存被挂起过程的可用偏移地址。

根据需要预定义几个处理函数, 它们的功能说明如下:

(1) $\text{mktable}(\text{previous})$: 创建一个新符号表, 其直接外围过程指针为 previous , 并返回指向新表的指针。

(2) $\text{addwidth}(\text{table}, \text{width})$: 将符号表中所有局部数据宽度之和 width 记录在 table 指向的符号表表头中。

(3) $\text{enterproc}(\text{table}, \text{name}, \text{newtable})$: 在 table 指向的符号表中为过程 name 创建一个新条目, newtable 为指向过程 name 的符号表的指针。

(4) $\text{enter}(\text{table}, \text{name}, \text{type}, \text{offset})$: 在 table 指向的符号表中为变量 name 创建一个新条目, 类型为 type , 相对地址为 offset 。

例 7.14 下面以图 7-12 中的程序概要为例, 说明此 SDT 的处理过程。

在开始读入和分析代码之前, 会先执行产生式(2)关联的语义动作, 初始化

全局作用域的符号表和偏移地址，为后续声明做准备，如图 7-21(a)。具体操作如下，

t = mktable(nil): 创建一个新的符号表，其直接外围过程指针为 nil（表示没有外围，即全局作用域），并返回新符号表的指针 t。

push(t, tblptr): 将新符号表指针 t 压入 tblptr 栈，表示现在开始处理这个全局符号表。

push(0, offset): 将偏移地址 0 压入 offset 栈，表示全局过程的局部变量从偏移量 0 开始分配。

接下来，从输入中读入并识别了“**proc id;**”表示开始了一个新的过程定义，即进入了一个新的作用域，则执行产生式(6)关联的语义动作，新建并初始化新的符号表和偏移地址，为新过程的分析做准备。具体操作如下，

t = mktable(top(tblptr)): 创建一个新的符号表，其直接外围过程指针为当前 tblptr 栈顶的指针（即外围过程的符号表），返回新符号表指针 t。这表示新过程继承外围作用域。

push(t, tblptr): 将新符号表指针 t 压入 tblptr 栈，表示现在开始处理 t 符号表对应过程。

push(0, offset): 将偏移地址 0 压入 offset 栈，表示过程体内的局部变量从偏移量 0 开始分配。

然后，从输入中读入“**var a:int[11];**”和“**x:int;**”识别两个声明语句，执行产生式(5)关联的语义动作，处理声明并将它们的信息和分配地址填入当前过程符号表中，如图 7-21(b)。具体操作如下，

enter(top(tblptr), id.lexeme, T.type, top(offset)): 在当前符号表（tblptr 栈顶）中为变量 id 创建一个条目，记录其类型 T.type 和相对地址 top(offset)（即当前偏移地址）。

top(offset) = top(offset) + T.width: 更新当前偏移地址，增加 T.width（类型 T 的宽度），表示下一个变量将从这个新偏移量开始分配。

接着，读入并分析代码“**proc readarray;**”，表示开始了一个新的过程定义，执行产生式(6)关联的语义动作，新建新的符号表并将指向符号表的指针入栈，初始化偏移地址并入栈，为新过程的分析做准备。然后分析此过程的声明代码，执行产生式(5)关联的语义动作，在符号表中填入变量 i 的条目。如图 7-21(c)。最后，分析 readarray 可执行过程体，这些分析可能会涉及符号表查找操作，但不会影响符号表的结构。

readarray 过程体分析结束之后，表示当前过程处理结束，执行产生式(4)关

联的语义动作，当前过程符号表栈，并在外围符号表中注册该过程，实现作用域的嵌套管理。如图 7-21(d)，具体操作如下：

$t = \text{top}(\text{tblptr})$: 获取当前符号表指针（即 tblptr 栈顶指针），在它出栈之前先备份在变量 t 里。

$\text{addwidth}(t, \text{top}(\text{offset}))$: 将当前过程符号表的总宽度（即当前偏移量栈顶值）记录在符号表 t 中。

$\text{pop}(\text{tblptr})$: 从 tblptr 栈中弹出当前过程符号表指针，表示离开当前过程符号表，新栈顶为其直接外围过程符号表指针。

$\text{pop}(\text{offset})$: 从 offset 栈中弹出当前过程偏移量，新栈顶为其直接外围过程的偏移量。

$\text{enterproc}(\text{top}(\text{tblptr}), \text{id.lexeme}, t)$: 在外围符号表（ tblptr 栈顶指针）中为过程 id.lexeme 创建一个条目，新条目指向该过程的符号表，从而建立嵌套作用域的连接。

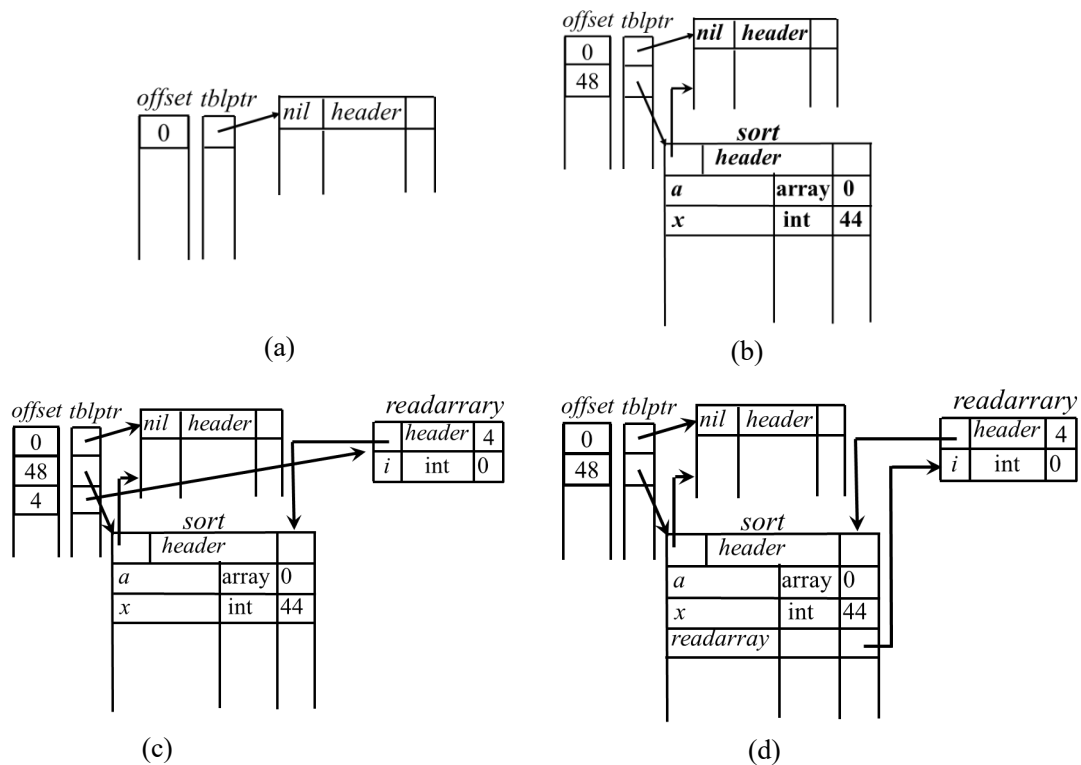


图 7-21 快速排序嵌套过程符号表构建

接下来，继续分析 `exchange` 和 `quicksort` 以及 `partition` 过程的代码，过程与

此类似，不再赘述。

以上的 SDT 未考虑相关语义检查，若加入标识符声明和使用的语义检查，那么，当在某一层的声明语句中识别出一个标识符时(id 的定义性出现)，以此标识符查找相应于本层的符号表，如果找到，则报错并发出诊断信息“id 重复声明”。否则，在符号表中加入新登记项，将标识符及有关信息填入。当在可执行语句部分扫视到标识符时(id 的应用性出现)，首先在该层符号表中查找该 id，如果找不到，则到直接外围符号表中去查，如此等等，一旦找到，则在表中取出有关信息并作相应处理。如果查遍所有外围符号表均未找到该 id，则报错并发出诊断信息“id 未声明”。

7.7 本章小节

本章系统阐述了程序运行时存储分配的核心策略。首先介绍了典型的内存布局，区分了静态、栈式与堆式存储分配的基本概念及适用场景。在静态分配部分，说明了其适用于无递归、数组定长的程序，并概述了顺序分配与层次分配两种编译时布局方法。栈式分配是本章重点，通过活动树和活动记录的概念，剖析了过程调用中栈的动态管理机制，包括活动记录的布局、调用/返回序列的协作，并探讨了在嵌套过程中通过访问链或显示表访问非局部数据的策略。堆式分配则概述了其灵活管理对象生命周期的特性，简要介绍了显式空闲链表管理与垃圾回收的基本思想。最后，本章简述了符号表在存储分配中的作用，及其在单作用域与嵌套作用域下的不同组织方式。通过本章学习，读者将深入理解程序运行时数据的组织与访问机制，为后续的目标代码生成及理解程序性能奠定基础。

第8章 代码优化

在编译器的整体结构中，代码优化与代码生成共同构成了编译器的后端，属于综合阶段。这一阶段的核心任务是将前端生成的中间代码转换为高效的目标代码，从而提升程序的执行性能。本章将聚焦于中间代码优化，系统介绍其基本原理、关键技术和实践方法，为读者奠定编译器优化的坚实基础。

代码优化本质上是一系列程序变换的过程，旨在改进目标代码的质量，使其运行速度更快、占用存储空间更少，或者在这两方面取得平衡。需要强调的是，任何优化变换都必须保持程序的语义等价性，即优化后的程序与原始程序在输入输出行为上完全一致。这意味着优化不能改变程序的功能，而是通过重组代码、消除冗余或利用硬件特性来提升效率。

一个优化编译器能够将程序 P 转换为与其语义等价的程序 P' ，但 P' 在性能或规模上优于 P 。理想情况下，我们可能期望存在一个“完全优化编译器”，它能将每一个程序 P 转换为语义等价的最小程序 P' ，其中“最小”可能指运行时间最短或代码体积最小。然而，可计算性理论（如基于图灵机模型的理论）已经证明，这样的完全优化编译器是不可实现的。这一结论源于计算问题的本质限制，例如停机问题的不可判定性，它表明无法通过算法为所有程序找到最优解。

进一步地，编译理论中的“充分就业定理”指出，对于任何给定的优化编译器，总存在另一个编译器能够产生更优的代码。这一定理揭示了优化过程的无限性，即编译器设计始终是一个不断改进的领域，不存在绝对完美的优化方案。因此，在实际编译器中，优化通常采用启发式方法和折中策略，在可计算范围内追求尽可能好的效果。

值得欣慰的是，优化领域的研究揭示了一个积极趋势，通过精心设计的优化策略，编译器往往能够以相对较小的代价获得显著的性能提升。许多简单而高效的优化技术能够在有限的分析开销下，消除程序中的明显低效问题。例如，常见的常量传播、死代码删除等优化，其实现复杂度相对较低，却能为程序性能带来可观的改善。这种代价与效益之间的有利关系，使得优化编译技术在实践应用中具有很高的价值，也指引着我们在设计优化算法时寻求效率与效果的最佳平衡。

本章将首先介绍中间代码的表示形式，然后深入讨论各类优化技术，包括局部优化、全局优化和循环优化等，并通过丰富的实例分析帮助读者掌握优化

算法的设计与实现。通过本章的学习，读者将理解优化在编译器中的关键作用，并具备初步的优化实践能力。

8.1 流图

8.1.1 流图的基本概念

为了进行有效且可靠的代码优化，优化编译器必须理解程序的控制流程，即指令的执行顺序和分支路径。为此，需要一种能够清晰展现程序控制流结构的中间表示方法。本节将介绍这种关键的数据结构——流图。

流图是一种用图结构来表示程序的方法。它将程序分解为一系列基本块，每个基本块是一个连续的指令序列，包含一个入口点和一个出口点。这些基本块通过有向边连接起来，边代表了控制流可能转移的路径，例如条件分支、无条件跳转或顺序执行。通过这种方式，一个复杂的程序被抽象为一个由节点（基本块）和边（控制流）构成的有向图。

流图对于代码优化是关键的重要表示，首先，流图为优化提供了理想的分析单元，清晰地定义了这些分析的边界和依赖关系，使得编译器能够系统性地追踪数值和控制的流动。其次，流图有助于识别程序中的关键结构循环，循环是程序中消耗大部分执行时间的区域，因此是优化的重点目标。最后，在流图上进行优化转换比在线性代码上更为直观和安全，确保了在改变代码的同时，能够清晰地维持程序正确的控制流语义。

8.1.2 流图的构建

流图的构建过程分为两个明确的步骤，首先将线性的中间代码序列划分为基本块，然后根据控制流信息将这些基本块连接起来，形成流图。

一个基本块是满足以下条件的、最大的连续三地址指令序列：

单一入口点：控制流只能从该基本块的第一条指令（称为首指令）进入该块。不存在任何跳转指令的目标点是该基本块中间或末尾的某条指令；

单一出口点：除了基本块的最后一条指令外，控制流在执行过程中不会在该块内发生跳转或停机。这意味着，一旦开始执行一个基本块，其中的所有指令都会被顺序执行一次，直到最后一条指令。

这个定义确保了基本块内部不存在分支结构，这为后续的分析和优化提供了极大的便利。

给定一个三地址指令序列，通过识别每个基本块的首指令来系统地划分基本块，具体方法如算法 8.1 基本块划分算法。

算法 8.1 基本块划分算法

输入： 三地址指令序列

输出： 输入序列对应的基本块列表

方法：

(1) 首先，确定指令序列中哪些指令是首指令(leaders)，即某个基本块的第一个指令。

- ① 指令序列的第一个三地址指令是一个首指令；
- ② 任意一个条件或无条件转移指令的目标指令是一个首指令；
- ③ 紧跟在一个条件或无条件转移指令之后的指令是一个首指令。

(2) 然后，每个首指令对应的基本块包括了从它自己开始，直到下一个首指令(不含)或者指令序列结尾之间的所有指令。

例 8.1 图 8-1 是快速排序算法划分函数 `partition` 的程序概要，其对应的中间代码基本块划分结果如图 8-2 所示。蓝色标注的代码为首指令。其中，指令 (1) 为指令序列的第一个三地址指令；指令 (5)、指令 (9) 和指令 (23) 为转移指令的目标指令；指令 (9)、指令 (13)、指令 (14) 和指令 (23) 是转移指令之后的指令。基于这些找到的首指令，将代码序列划分为 6 个基本块。

```

i = m - 1; j = n; v = a[n];
while (1) {
    do i = i + 1; while(a[i] < v);
    do j = j - 1; while (a[j] > v);
    if (i >= j) break;
    x = a[i]; a[i] = a[j]; a[j] = x;
}
x = a[i]; a[i] = a[n]; a[n] = x;

```

图 8-22 `partition` 函数的程序概要

在划分出所有基本块之后，便可以构建流图。流图的每个节点是一个基本块，使用有向边将基本块连接起来，代表控制流可能转移的路径，有向边构建方法为从基本块 B 到基本块 C 之间有一条边当且仅当基本块 C 的第一个指令可能紧跟在 B 的最后一条指令之后执行。此时称 B 是 C 的前驱，C 是 B 的后继。在划分出所有基本块之后，便可以构建流图。流图的每个节点是一个基本

块，使用有向边将基本块连接起来，代表控制流可能转移的路径，有向边构建方法为从基本块 B 到基本块 C 之间有一条边当且仅当基本块 C 的第一个指令可能紧跟在 B 的最后一条指令之后执行。此时称 B 是 C 的前驱，C 是 B 的后继。

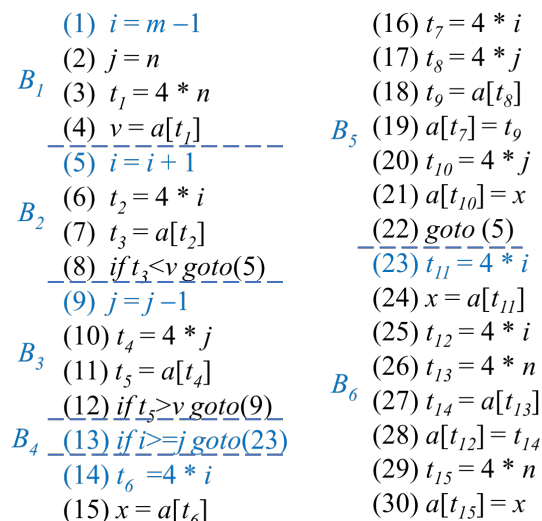


图 8-23 partition 函数中间代码基本块划分

有两种方式可以确认这样的边：

(1) 显式跳转，即存在一个从 B 的结尾跳转到 C 的开头的条件或无条件跳转指令；

(2) 顺序后继，即按照原来的三地址指令序列中的顺序，C 紧跟在 B 之后，且 B 的结尾不存在无条件跳转指令。

例 8.2 对于图 8-2 中的中间代码，按照原来的三地址语句序列中的顺序， B_2 紧跟在 B_1 之后， B_3 紧跟在 B_2 之后， B_4 紧跟在 B_3 之后， B_5 紧跟在 B_4 之后，且 B_1 、 B_2 、 B_3 和 B_4 的结尾均不存在无条件跳转语句，因此，画一条从 B_1 到 B_2 的边，一条从 B_2 到 B_3 的边，一条从 B_3 到 B_4 的边和一条从 B_4 到 B_5 的边；有一个从 B_2 的结尾跳转到 B_2 的开头的跳转语句，因此，从 B_2 画一条到 B_2 自己的边；有一个从 B_3 的结尾跳转到 B_3 的开头的跳转语句，因此，从 B_3 画一条到 B_3 自己的边；有一个从 B_4 的结尾跳转到 B_6 的开头的跳转语句，因此，从 B_4 画一条到 B_6 的边；有一个从 B_5 的结尾跳转到 B_2 的开头的跳转语句，因此，从 B_5 画一条到 B_2 的边。最终构建的流图如图 8-3 所示。

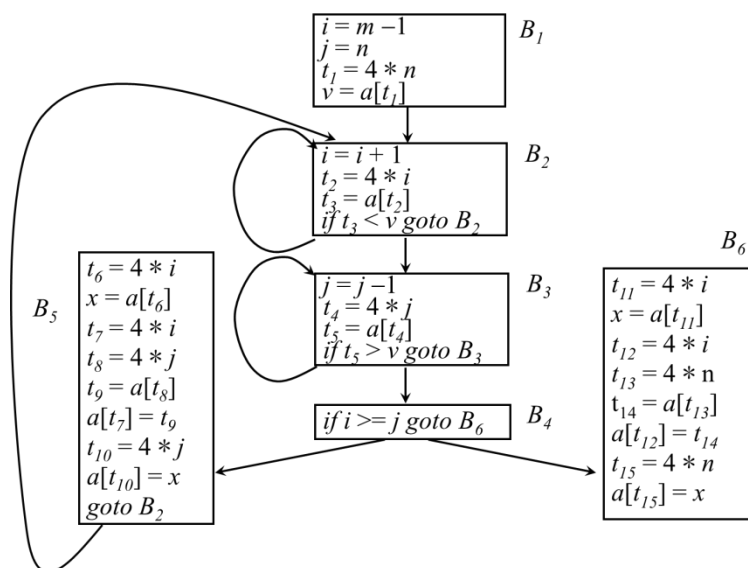


图 8-24 partition 函数中间代码流程图

8.2 优化的分类

从优化所依赖的代码层次来看，代码优化可分为两大类：

(1) 机器无关优化。这类优化在中间代码层面进行，不依赖于目标机器的具体硬件特性，主要利用程序本身的语法结构和静态分析结果，致力于提升代码的通用执行效率。

(2) 机器相关优化。这类优化在目标代码层面进行，与目标机器的硬件架构紧密相关，例如寄存器分配、指令调度、窥孔优化等。它旨在充分发挥特定硬件平台的性能优势，如寄存器数量、指令集等。

本章将重点讨论机器无关的优化技术，机器相关优化将在后续“代码生成”一章中展开。

从优化所作用的程序范围来看，机器无关优化又可进一步细分为：

(1) 局部优化。局限于单个基本块范围内的优化。由于范围受限，分析简单，但优化能力有限。

(2) 全局优化，或称过程内优化。跨越多个基本块，面向整个过程的优化。它通过分析程序的控制流和数据流，能够发现并消除更大范围内的冗余。

本章的核心内容是介绍过程内的局部优化和全局优化方法。本章将深入探讨一系列常用且高效的优化技术。这些技术旨在解决程序中普遍存在的性能

瓶颈。

8.2.1 删除公共子表达式

在程序执行过程中，同一段计算代码有可能被重复执行，尤其是在循环体或条件分支的交叉路径中。如果这些重复计算所依赖的输入值并未发生改变，那么这种计算无疑是冗余的。删除公共子表达式正是一种用于识别并消除此类冗余计算的核心优化技术。

如果表达式 $x \text{ op } y$ 先前已被计算过，并且从先前的计算到现在， $x \text{ op } y$ 中变量的值没有改变，那么 $x \text{ op } y$ 的这次出现就称为**公共子表达式**(common subexpression)。

一个基本块范围内的公共子表达式，称为**局部公共子表达式**，跨基本块的公共子表达式，称为**全局公共子表达式**。

例 8.3 分析图 8-4(a)中的基本块 B_5 。 B_5 中第一条指令计算了 $4*i$ 的值，并把它赋给变量 t_6 ，第三条指令再次计算了 $4*i$ 的值，并把它赋给变量 t_7 。由于在第 1 条与第 3 条指令之间， i 的值没有改变，即没有对 i 重新赋值，因此，第三条指令中的 $4*i$ 的出现是一个公共子表达式。因为上一次对该表达式的计算位于同一个基本块中，因此，它是一个局部公共子表达式。同理，第 4 条指令计算了 $4*j$ 的值，并把它赋给变量 t_8 ，第 7 条指令再次计算了 $4*j$ 的值，并把它赋给变量 t_9 。由于在这两条指令之间， j 的值没有改变，因此，第 7 条指令中 $4*j$ 的出现也是一个局部公共子表达式。于是，可以将这两个公共子表达式删除，并使用 t_6 来代替 t_7 ，使用 t_8 来代替 t_9 。因此，将 B_5 改写为如图 8-4(b)所示，完成了基本块 B_5 的局部公共子表达式的删除。

继续分析图 8-5(a)中的流图， B_5 中第一条指令计算了 $4*i$ 的值，并把它赋给变量 t_6 ，事实上， $4*i$ 在 B_2 中已被计算过，并且把计算结果赋给了 t_2 ，控制从 B_2 中计算 $4*i$ 的点到 B_5 中计算 $4*i$ 的点之间， i 的值没有改变，因此， B_5 中的 $4*i$ 是一个公共子表达式，因为上一次对该表达式的计算位于另一个基本块中，因此，它是一个全局公共子表达式。控制从 B_2 中计算 $4*i$ 的点到 B_5 中计算 $4*i$ 的点之间， t_2 的值也没有改变，因此，可以将公共子表达式 $4*i$ 删除，并用 t_2 来替代 t_6 。同理， B_5 中 $4*j$ 也是一个全局公共子表达式，它的上次计算在基本块 B_3 中的 $t_4=4*j$ ，可以将它删除，并用 t_4 来替代 t_8 。于是，将 B_5 改写为图 8-5(b)所示。

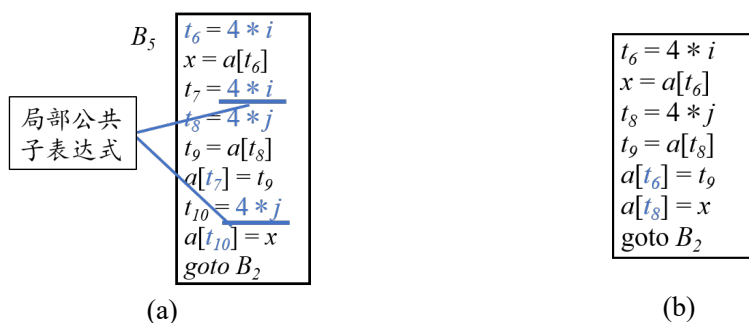


图 8-25 删除局部公共子表达式

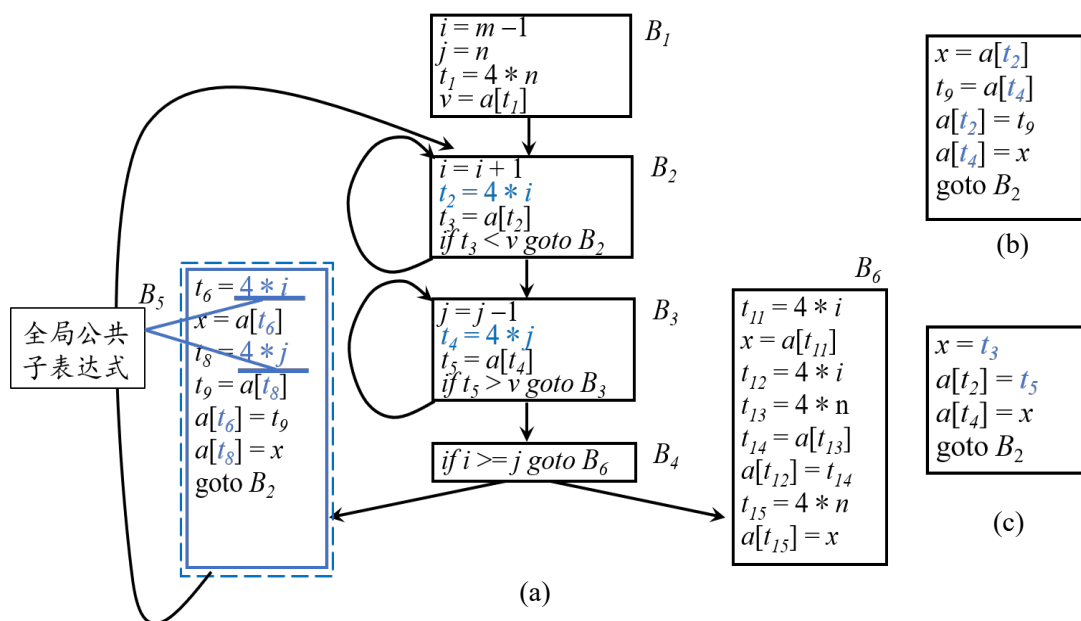


图 8-26 删除全局公共子表达式示例

继续分析图 8-5(b)，可见 B_5 中 $a[t_2]$ 在 B_2 中已被计算过，并且把计算结果赋给了 t_3 ，控制从 B_2 中计算 $a[t_2]$ 的点到 B_5 中计算 $a[t_2]$ 的点之间， t_2 的值没有改变，也没有对数组 a 中的元素赋值，因此 $a[t_2]$ 的值没有发生变化，是一个全局公共子表达式。控制从 B_2 中计算 $a[t_2]$ 的点到 B_5 中计算 $a[t_2]$ 的点之间， t_3 的值也没有改变，因此，可以用 t_3 来替代 $a[t_2]$ 。同理， $a[t_4]$ 也是一个公共子表达式，可以用 t_5 来替代 $a[t_4]$ ，于是，将 B_5 改写为如图 8-5(c) 所示。

需要强调的是优化转换要保证语义等价，必须是保守的，如果不能百分百

地确定某种转换不会改变程序的原有行为，那么就必须放弃可能有风险的优化。下面通过一个例子进行说明。

例 8.4 如图 8-6 是在例 8.3 的基础上，在基本块 B_6 中删除了公共子表达式 $4*i$ 和 $4*n$ 得到的。我们来重点分析 B_6 中的数组引用 $a[t_1]$ 是否能作为公共子表达式。 B_1 中有对 $a[t_1]$ 的引用，并赋值给了 v ，接下来检查从此赋值点到 B_6 中 $a[t_1]$ 的引用点的所有路径是不是都没有改变 $a[t_1]$ 的值。然而，经过分析会发现，从 B_1 中的 $a[t_1]$ 赋值点到 B_6 中 $a[t_1]$ 的引用点有路径会经过 B_5 ，而 B_5 中有对 $a[t_2]$ 和 $a[t_4]$ 的赋值，而 t_2 或 t_4 都有可能等于 t_1 ，因此，基于优化的保守原则，把 $a[t_1]$ 作为公共子表达式是不稳妥的。

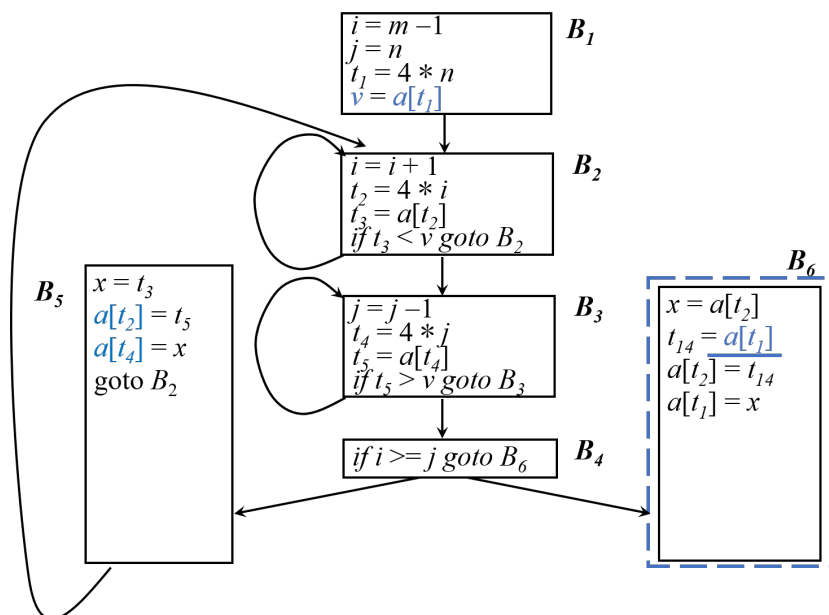


图 8-27 数组引用是非公共子表达式的情况

删除公共子表达式的优化思想简单直观，然而想将其转化为编译器中可靠、高效的自动化算法，关键在于如何精准、可靠地“识别”出程序中所有真正的公共子表达式。局部公共子表达式和全局公共子表达式有不同的识别策略，将在后面的章节中详细介绍。

8.2.2 基本块中的强度削弱

强度削弱（Strength Reduction）的核心思想是用执行速度更快、计算资源消耗更低的等效操作，替换掉速度较慢、消耗较高的操作。这种优化不改变程

序的语义，但能直接降低指令的执行开销，提升代码的运行效率。在基本块内，编译器可以安全实施多种类型的强度削弱操作。

（1）数学运算的替换与简化

这是强度削弱中最常见的一类，主要针对基本的算术和逻辑运算。首先，可以用移位操作替换乘除运算。当乘数或除数是 2 的整数次幂时，乘法与除法可以分别转换为代价更低的左移和右移操作。

例 8.5 表达式 $x = y * 8$ 可以被优化为 $x = y \ll 3$ 。因为 8 等于 2^3 ，左移 3 位等价于乘以 8。对于无符号整数或正数，表达式 $x = y / 4$ 可以被优化为 $x = y \gg 2$ 。

值得注意的是，对于有符号数的算术右移，其行为与除法在舍入方式上可能存在细微差异，例如，对负数的舍入方向，编译器需要根据语言标准和具体上下文进行保守且正确的转换。

其次，可以将代价高的运算转换为代价低的运算。

将乘法转换为加法。如表达式 $2*x$ 或 $2.0*x$ 可以被直接替换为 $x+x$ 。用乘法替换除法运算。如 $x/2$ 可以被直接替换为 $x*0.5$ 。用乘法替换指数运算。如表达式 $x = y ** 2$ （表示 y 的平方）可以被直接替换为 $x = y * y$ 。再如，表达式 $a_n x_n + a_{n-1} x_{n-1} + \dots + a_1 x + a_0$ 可以被替换为 $((\dots(a_n x + a_{n-1})x + a_{n-2})\dots)x + a_1)x + a_0$ 。

最后，常量合并（或称为常量折叠）与常量传播的结合。在强度削弱前，通常先进行常量折叠与传播，这本身可能创造出新的强度削弱机会，或直接简化表达式。

例如，在计算 $x = 3 * 2$ 时，可直接折叠为 $x = 6$ 。又如，对于 $a = 4$ ， $b = a * 5$ ，可先传播为 $b = 4 * 5$ ，再折叠为 $b = 20$ 。

（2）特殊函数与内置操作的优化

编译器可以识别一些特定模式的表达式，并用更高效的内置函数或指令序列来替代。

例如，乘方运算的简化。除了平方转换为乘法，对于小的整数常数次幂，编译器可以展开为一系列乘法，而非调用通用的指数函数。

再如，标准库函数的识别与替换。某些对标准库函数的调用，在特定条件下可以被替换。例如，对于 $\text{pow}(x, 2.0)$ ，优化器可以将其替换为 $x * x$ 。

基本块内的强度削弱主要通过基于模式匹配的代码转换来实现，其核心是维护一个预定义的“昂贵操作-廉价操作”规则库，编译器通过遍历基本块内的指令并与规则库进行匹配来识别优化机会；对于每个匹配到的模式，编译器必须进行保守的安全性检查以确保语义等价，包括验证操作数类型、溢出行为

及精度要求等关键因素；在验证通过后，编译器将用高效操作替换原有昂贵操作，这一过程常与常数折叠和常量传播协同进行，通过先简化表达式来创造更多优化机会，最终实现在保证程序语义不变的前提下，系统性地将高代价操作替换为等效低代价操作，从而有效降低计算强度、提升执行效率。

后续我们将看到，在循环等全局上下文中，强度削弱还能与归纳变量分析等技术结合，发挥出更强大的优化效能。

8.2.3 删除无用代码

无用代码，也称为死代码（Dead-Code），指其计算结果永远不会被使用的语句。程序员不大可能有意引入无用代码，无用代码通常是因为前面执行过的某些转换而造成的，如复制传播就会给删除无用代码带来机会。复制传播是指在复制语句 $x=y$ 之后的 x 的引用点尽可能地用 y 代替 x 。

常用的公共子表达式消除算法和其它一些优化算法会引入一些复制语句（形如 $x=y$ 的赋值语句）。例如，在前面分析中， B_5 中使用的 $a[t_2]$ 在 B_2 中已被计算过，是一个公共子表达式，可以用 t_3 来替代 $a[t_2]$ ，这样，就引入了一条复制语句 $x=t_3$ 。这些引入的复制语句为复制传播和删除无用代码创造了机会。

例 8.6 如图 8-7(a) 中在 $x=t_3$ 之后的 x 引用点 $a[t_4]=x$ ，使用 t_3 代替 x ，实现复制语句的 $x=t_3$ 传播。基本块 B_5 经过复制传播后，块中所有对 x 的使用都替换为对 t_3 的使用，这样第一条语句的计算结果 x 在块中永远不会被使用，如果 x 在其它块中也不被使用，那么这条语句就是无用代码，可以将其删除，删除后的代码变成如图 8-7(c) 的形式。

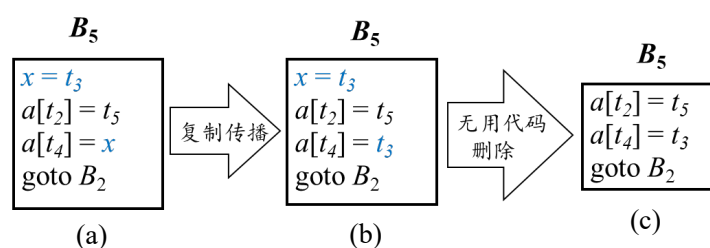


图 8-28 复制传播与无用代码删除

删除无用代码的关键在于如何实现对无用代码的自动识别。这就要求编译器具备精准的代码分析能力，通过追踪变量的引用关系、判断语句的可达性等方式，定位出真正的无用代码。具体方法将在后续小节中介绍。

经过以上的一系列优化，如图 8-8 所示，B5 中的指令由 9 条减少到 3 条，B6 中的指令由 8 条减少到 3 条，可见这两种代码优化方法都是很有有效的。

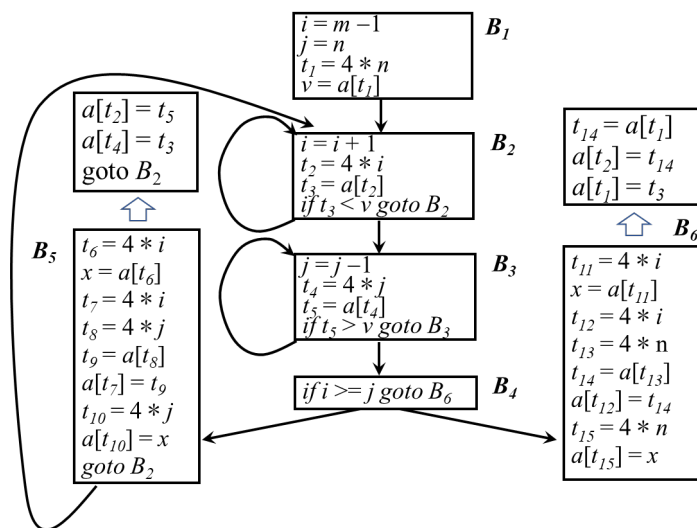


图 8-29 优化后的基本块

8.2.4 循环不变表达式外提

在代码优化中，循环(尤其是内部循环)是优化的重点对象。因为程序的大部分运行时间往往消耗在循环上，如果能够减少循环内部指令的数量，即使因此增加了循环外的代码，程序的整体运行时间也可能显著降低。循环不变表达式外提 (Loop-Invariant Code Motion) 正是这样一种优化技术，它通过将循环中不变的计算移动到循环外部，来减少循环体内的计算开销。

循环不变表达式是指在循环执行过程中，其值不会发生改变的表达式。这类表达式的结果仅依赖于在循环开始前就已确定的变量或常量，与循环的迭代次数无关。因此，可以在进入循环之前就对这些表达式进行求值，并将结果存储在临时变量中，从而避免在每次迭代中重复计算。

例 8.7 为了直观理解循环不变表达式外提，考虑一个简单的例子：计算半径为 r 的从 10 度到 360 度的扇形面积。源程序代码片段如下：

```

for ( n = 10; n < 360; n++ ) {
    s = 1/360 * PI * r * r * n;
    printf( "Area is %f", s );
}
  
```

对应的中间代码如下：

(1) $n = 1$	(8) $t_5 = t_4 * n$
(2) if $n > 360$ goto (21)	(9) $s = t_5$
(3) goto (4)	
(4) $t_1 = 1 / 360$	(18) $t_9 = n + 1$
(5) $t_2 = t_1 * \text{PI}$	(19) $n = t_9$
(6) $t_3 = t_2 * r$	(20) goto (4)
(7) $t_4 = t_3 * r$	(21)

在这个代码片段中，指令 (4) 到 (20) 构成了一个循环体。分析表达式 $1/360 * \text{PI} * r * r$ ，可以发现其中的符号常量 PI 和 r 在循环过程中值不变，因此这部分计算是循环不变的。指令 (4) 到 (7) 对应的表达式都属于循环不变计算。通过外提优化，可以将这些指令移动到循环外部，在进入循环前一次性计算完毕。优化后的中间代码如下：

(1) $t_1 = 1 / 360$	(8) $t_5 = t_4 * n$
(2) $t_2 = t_1 * \text{PI}$	(9) $s = t_5$
(3) $t_3 = t_2 * r$	
(4) $n = 1$	(18) $t_9 = n + 1$
(5) if $n > 360$ goto (21)	(19) $n = t_9$
(6) goto (4)	(20) goto (4)
(7) $t_4 = t_3 * r$	(21)

需要注意的是，外提操作必须保证程序的语义不变，即外提的表达式在循环外部求值的结果与在循环内部每次迭代求值的结果一致。

对于多重嵌套的循环结构，需要特别注意循环不变计算的相对性。一个表达式是否属于循环不变计算，是相对于其所处的特定循环层次而言的。在某一层循环中保持不变的表达式，在更外层的循环中可能就不再具有不变性。

例 8.8 考虑以下双重循环程序片段，任务是计算扇形面积。

```
for( i = 1; i < 10; i++ )
    for( n = 1; n < 360 / ( 5 * i ); n++ )
        { s = ( 5 * i ) / 360 * PI * r * r * n; ... }
```

在这个例子中，表达式 $(5 * i) / 360 * \text{PI} * r * r$ 相对于内层循环 (n 循环) 是循环不变计算，因为其中的变量 i 、 PI 和 r 在内层循环执行过程中保持不变。然而，对于外层循环 (i 循环)，由于变量 i 在每次外层迭代中都会改变，该表达式就不再具有不变性。

这一特性决定了多层循环优化的基本策略：通常采用自内而外的顺序逐层进行优化。首先识别并外提最内层循环中的不变表达式，然后将其视为内层循环的组成部分，再继续向外层循环推进。这种优化顺序能够确保在每一层都最大限度地减少重复计算，从而获得最优的性能提升效果。

循环不变表达式外提作为一种经典且高效的优化手段，其成功实施高度依赖于对中间代码中循环结构的精确识别以及对循环不变计算的可靠判定。在现代编译器的实现中，此项优化通常构建在数据流分析的基础之上，通过系统的程序分析为相关识别与验证提供理论支撑和实现保障。更具体的实现技术将在本章后续小节中进行讨论。

8.2.5 循环中的强度削弱

与基本块内的强度削弱思路一致，循环中的强度削弱核心是通过将高代价运算替换为低代价运算，提升程序执行效率。区别于基本块优化，循环内计算的执行频度极高，因此循环强度削弱的优化收益通常更为显著。

归纳变量计算的强度削弱通常是循环强度削弱的重点。归纳变量（Induction Variable）是指在循环中每次迭代都按固定量递增或递减的变量。具体来说，若变量 x 在每次被赋值时，其值总是增加或减少一个固定常数 c ，则称 x 为该循环中的归纳变量。

例 8.9 考虑图 8-9 中的代码片段，包含两个基本块组成的循环结构，在由基本块 B_2 构成的循环中，变量 i 每次迭代增加 1，显然是归纳变量。变量 t_2 由表达式 $t_2 = 4 * i$ 计算得到，由于 i 每次递增 1， t_2 会同步递增 4，因此， t_2 也是归纳变量。同理 j 和 t_4 则是以 B_3 为核心的循环中的归纳变量。

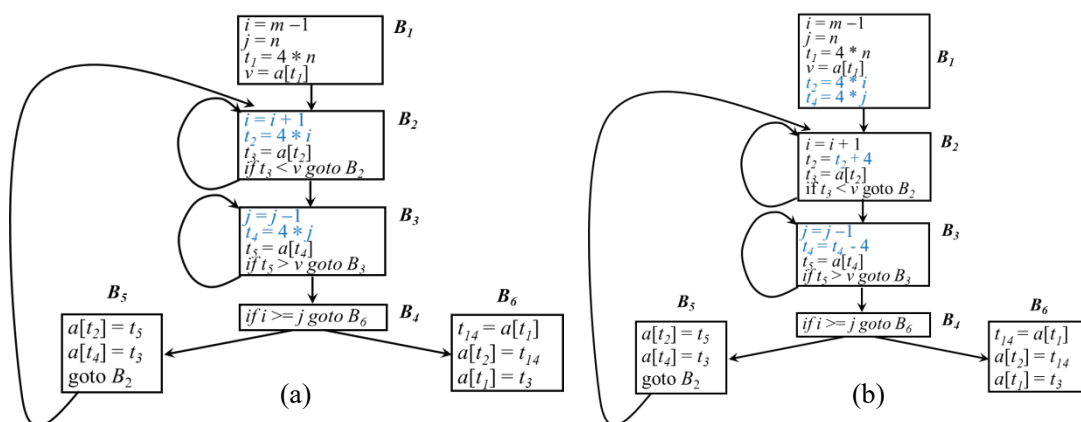


图 8-30 循环中的强度削弱

强度削弱的核心逻辑正是利用归纳变量的变化规律实现高代价运算向低代价运算的转换。仍以归纳变量 t_2 为例，其原计算依赖乘法运算 $t_2 = 4 * i$ ，结合 i 的递增规律（每次+1），可将该乘法运算替换为更高效的加法运算 $t_2 = t_2 + 4$ ，从而大幅降低循环迭代过程中的运算开销。

需要特别说明的是，强度削弱的关键前提是保证优化前后程序语义完全等价，具体需重点关注两点：第一，确保初始值正确。循环开始前，需要为经过强度削弱后的变量设置符合语义的初始值。在上例中，为保证进入 B_2 时 $t_2 = 4 * i$ 的关系始终成立，需在 B_1 块尾部添加 $t_2 = 4 * i$ 的初始化语句。尽管新增了一条指令，但该指令仅在 B_1 中执行一次，对整体性能的影响可忽略不计。第二，维持正确的数据依赖关系。优化后变量的更新顺序必须与原程序一致，确保每次使用变量时都能获取符合预期的正确值。

基于上述逻辑，可对另一归纳变量 t_4 进行类似优化：将原乘法运算 $t_4 = 4 * j$ 替换为加法运算 $t_4 = t_4 + 4$ （结合 j 的变化规律），同时在 B_1 块尾部添加 t_4 的初始化语句 $t_4 = 4 * j$ ，以保障优化前后程序语义的一致性。

8.2.6 删除归纳变量

在完成强度削弱优化后，循环中常常存在多个相关的归纳变量，它们之间的值保持固定的线性关系。此时可以进一步实施归纳变量删除优化，消除冗余的归纳变量，减少计算量和寄存器使用。

在循环执行过程中，如果一组归纳变量的值变化保持同步，即它们之间存在固定的线性关系，则称这些变量构成一个归纳变量组。对于这样的变量组，通常可以删除其中的部分变量，只保留必要的变量。

例 8.10 考虑图 8-9 中的示例代码。在 B_2 循环中，变量 i 和 t_2 构成一个归纳变量组，满足关系 $t_2 = 4 \times i$ ；在 B_3 循环中，变量 j 和 t_4 构成另一个归纳变量组，满足关系 $t_4 = 4 \times j$ 。

当需要从归纳变量组中删除部分变量时，选择保留哪些变量、删除哪些变量是一个关键决策。选择策略主要基于变量的使用情况，通常情况下，优先保留在循环内部被频繁使用的变量；优先保留在关键路径（如数组访问）中使用的变量；考虑后续优化的便利性。

例 8.11 考虑图 8-9 中的示例代码，归纳变量使用情况分析如下：

t_2 在 B_2 循环中被用于数组访问 $a[t_2]$ ， i 在 B_4 基本块中被用于条件测试

$i \geq j$; t_4 在 B_3 循环中被用于数组访问 $a[t_4]$, j 在 B_4 基本块中被用于条件测试 $i \geq j$ 。可见, i 和 j 的唯一用途是计算 B_4 中的测试结果。基于上述分析, 合理的策略是在 B_2 循环中保留 t_2 , 删除 i ; 在 B_3 循环中保留 t_4 , 删除 j 。因此, 测试 $i \geq j$ 可以被替换为 $t_2 \geq t_4$ 。一旦进行这个替换, 这两条语句 $i = i + 1$ 和 $j = j - 1$ 就变成了可以删除的无用代码。删除归纳变量后的流图如图 8-10 所示。

为了系统性地实施归纳变量删除优化, 首先需要准确识别出循环中的所有归纳变量。这一识别过程通常需要结合数据流分析技术。

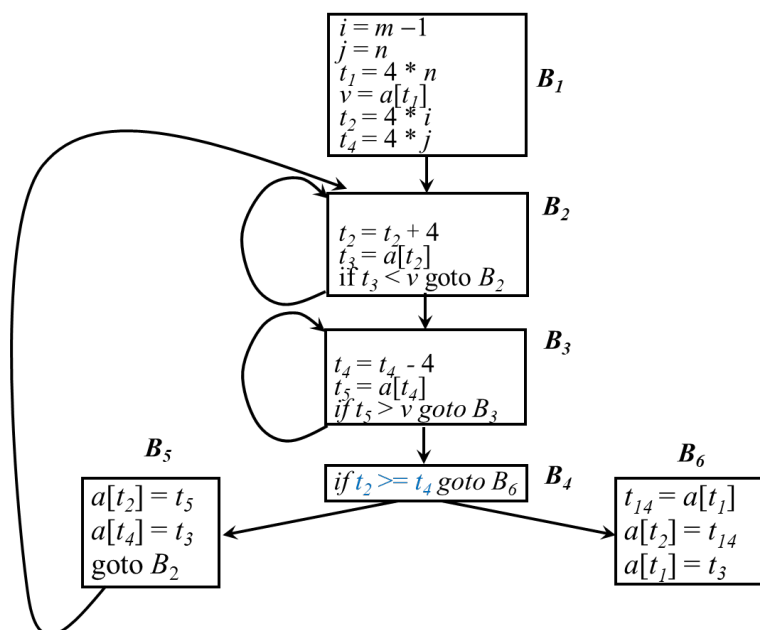


图 8-31 归纳变量的删除示例

8.3 基本块的局部优化

在中间代码优化中, 基本块内的局部优化是最基础且重要的优化层次。由于基本块内部不存在分支结构, 因此可以在不进行复杂数据流分析的情况下实施有效的优化。

许多重要的局部优化技术都基于无环有向图 (Directed Acyclic Graph, DAG) 来实现。基于 DAG 图的基本块优化的基本流程包括:

- (1) 将基本块转换为 DAG 表示;
- (2) 在 DAG 上进行各种优化变换;

(3) 将优化后的 DAG 重组为改进的语句序列。

8.3.1 基本块的 DAG 表示

首先来看单个语句的 DAG 表示。中间代码语句 s 的一般形式为 $a = b \text{ op } c$ ，对于此类语句，我们称其为对变量 a 的“定值”操作，同时是对变量 b 和 c 的“引用”操作。图 8-11 给出了主要三地址语句对应的 DAG 子图。

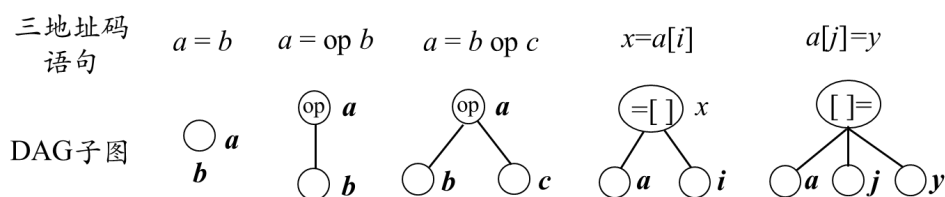


图 8-32 单个语句的 DAG 表示

每个语句 s 对应基本块 DAG 的一个内部节点 N ，其构建规则如下：

(1) 节点 N 的标号是 s 中的运算符；同时还有一个定值变量表被关联到 N ，表示 s 是在此基本块内最晚对表中变量进行定值的语句；

(2) N 的子节点是基本块中在 s 之前且最后一个对 s 所使用的运算分量进行定值的语句对应的节点。如果 s 的某个运算分量在基本块内没有在 s 之前被定值，则这个运算分量对应的子节点就是代表该运算分量初始值的叶节点。为区别起见，叶节点的定值变量表中的变量加上下脚标 0；

将单个语句的构建规则扩展至包含一组语句的基本块，按基本块内语句的执行顺序遍历每条语句 s ，按以下流程构建 DAG：

(1) 对于语句 s ，先检查 DAG 中是否已存在与 s 对应的节点，即节点运算符与 s 的运算符相同，且子节点完全一致（若运算符具备可交换性，子节点左右位置可不同）

- ① 若存在对应节点，直接将 s 的定值变量添加至该节点的定值变量表；
- ② 若不存在对应节点，则依据上述单语句构建规则新建节点 N ，明确其标号、子节点及初始定值变量表。

(2) 完成节点创建或变量添加后，若 s 的定值变量 x 已存在于 DAG 中某节点 M 的定值变量表内，需从 M 的定值变量表中删除 x 。这是因为新构建的节点 N 是本基本块内最晚对 x 定值的语句，此举可确保定值变量表的准确性。

例 8.12 考虑图 8-12(a)中给出的基本块，包含五条指令。结合上述流程详

细说明其 DAG 的构建过程：

首先，处理第一条语句 $a = b + c$ 。检查当前空 DAG，不存在运算符为 “+” 且子节点匹配的节点，因此新建标记为 “+” 的内部节点。将定值变量 a 加入该节点的定值变量表，其子节点为代表 b 初始值的叶节点 b_0 和代表 c 初始值的叶节点 c_0 （因 b 、 c 在此语句前未在本块内定值）。

处理第二条语句 $b = a - d$ 。检查 DAG，无运算符为 “-” 且子节点匹配的节点，新建标记为 “-” 的内部节点。将定值变量 b 加入该节点的定值变量表，其子节点为第一条语句对应的 “+” 节点（ a 的最晚定值节点）和代表 d 初始值的叶节点 d_0 。由于 b 此前的定值记录在 b_0 叶节点中，需要从该叶节点的定值变量表中删除 b （即 “注销” 原 b_0 的 b 定值记录）。此时 DAG 结构如图 8-12(b) 所示。

处理第三条语句 $c = b + c$ 。首先确定运算分量的最晚定值节点： b 的最晚定值节点为第二条语句对应的 “-” 节点， c 的最晚定值节点为初始叶节点 c_0 。检查 DAG，不存在以这两个节点为子节点且标记为 “+” 的节点，因此新建标记为 “+” 的内部节点，将定值变量 c 加入其定值变量表。同时，从 c_0 叶节点的定值变量表中删除 c ，完成旧定值记录的注销。

处理第四条语句 $d = a - d$ 。确定运算分量的最晚定值节点： a 的最晚定值节点为第一条语句对应的 “+” 节点， d 的最晚定值节点为初始叶节点 d_0 。检查 DAG 后发现，第二条语句对应的 “-” 节点恰好满足 “运算符为 ‘-’、子节点为 ‘+’ 节点与 d_0 ”，与当前语句的节点匹配条件一致，因此识别出这是局部公共子表达式，无需新建节点。直接将定值变量 d 加入该 “-” 节点的定值变量表，并从 d_0 叶节点的定值变量表中删除 d 。此时 DAG 结构如图 8-12(c) 所示。

处理第五条语句 $e = a$ 。此语句本质是对 a 的直接引用（无运算符，可理解为特殊的赋值语句），核心是找到 a 的最晚定值节点，即第一条语句对应的 “+” 节点。检查该节点后，直接将定值变量 e 加入其定值变量表，无需新建节点。最终构建的 DAG 如图 8-12(d) 所示。

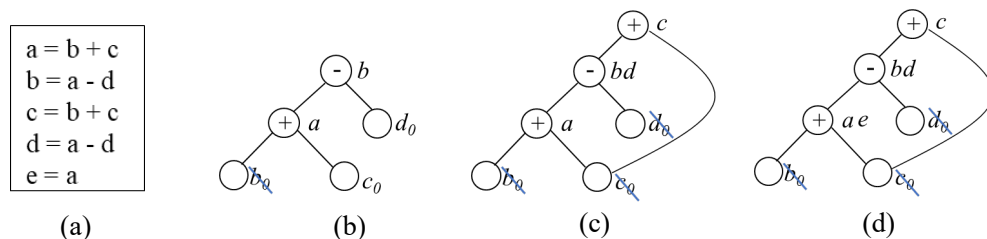


图 8-33 DAG 构建过程

8.3.2 基本块 DAG 的优化

分析小节 8.3.1 中的基本块 DAG 构建流程可知，在构建 DAG 过程中，为语句构建节点时，对语句进行一些检查和变换，即可同时完成多种类型的局部优化。

(1) 公共子表达式的识别

公共子表达式的识别是 DAG 构建过程中自然完成的一项优化。如果同一个表达式在基本块内多次出现，DAG 结构能够将其映射到同一个节点上，从而避免重复计算。

(2) 代数恒等式变换

在构造 DAG 时，可实施多种代数恒等式变换，对表达式计算进行简化。例如，对于形如 $x+0$ 、 $x-0$ 、 $x*1$ 或 $x/1$ 的运算，可直接优化为 x 。此外，还可以运用交换律，例如利用 $x*y$ 与 $y*x$ 的等价性，在构建节点时检查是否已存在相同运算分量的节点，从而识别公共子表达式。

(3) 局部强度削弱

例如将乘方运算 $x**2$ 替换为乘法 $x*x$ ，将乘法 $2*x$ 替换为加法 $x+x$ ，或者将除法 $x/2$ 替换为乘法 $x*0.5$ 。

(4) 常量合并

常量合并是指在编译期间完成常量表达式的计算，并将其替换为计算结果。例如将表达式 $2*3.14$ 直接替换为 6.28 。

(5) 无用代码的删除

DAG 构建完成后，按以下流程删除无用代码。

- ① 定位 DAG 中所有根节点，即无父节点的节点。
- ② 判断根节点是否为“无用节点”。核心标准是其定值变量表中的所有变量，既不会在本基本块后续语句中被引用，也不会在本基本块之外被使用（在本基本块的出口处是非活跃的）；
- ③ 删除所有符合“无用节点”条件的根节点；
- ④ 重复执行以上三步，直至无任何根节点可被删除。

对于第 2 步的判断条件，在 DAG 构建完成后，根节点一定是不被基本块内其它语句引用的。另一个条件在本基本块之外是否被使用，可以通过使用活跃变量分析获得有用信息。活跃变量是指其值可能会在以后被使用的变量，活跃

变量分析详见小节 8.4.4。

例 8.13 如图 8-13(a)中的 DAG，假设只有 a 和 b 在基本块出口处是活跃的。定值表为 e 的节点是根节点，且 e 在出口处不活跃，可以删除该节点，也就等价于删除了该节点对应的运算。此时的 DAG 如图 8-13(b)，可见定值表为 c 的节点成为根节点，且 c 在出口处不活跃，可以删除该节点。此时的 DAG 如图 8-13(c)。剩余的两个根节点分别为 a 和 b 定值，它们都是活跃变量，因此，这些节点不能删除。

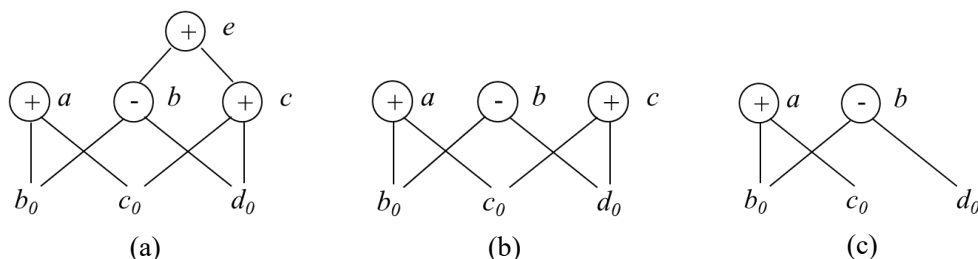


图 8-34 DAG 的无用代码删除

特别需要注意的是，在处理数组赋值语句时，如形如 $a[j] = y$ 的三地址代码，应创建一个运算符为“ $[\] =$ ”的节点，该节点包含三个子节点，分别表示数组 a 、索引 j 和赋值源 y ，且该节点没有定值变量表。该节点的生成将注销（或称“杀死”）所有值依赖于 a 的已有节点。被注销的节点将不能再获得新的定值变量，因而能够避免将后续语句中不稳妥的 $a[i]$ 引用识别为公共子表达式，从而保证优化的语义正确性。

例 8.14 有如下的三条语句，在构造 DAG 时必须要避免将第三条语句的 $a[i]$ 引用误判为公共子表达式。

$x = a[i]$

$a[j] = y$

$z = a[i]$

下面来看看 DAG 的构建过程。首先构建第一条语句对应的节点标记为“ $[\] =$ ”，定值变量表为 x ，子节点为叶节点 a_0 和 i_0 ；然后构建第二条语句 $a[j] = y$ 的节点标记为“ $[\] =$ ”，没有定值变量表，其子节点是 a_0 ， j_0 和 y_0 。此运算注销了所有依赖数组 a 的运算对应的节点，即依赖子节点 a_0 且标记为“ $[\] =$ ”的节点。此时的 DAG 如图 8-13(a)所示。对于第三条语句，是对数组 a 的引用，因 DAG 上已存在的数组引用节点因被注销而不能再获得新的定值变量，因此，需要新建数组引用节点“ $[\] =$ ”，其定值变量表为 z ，子节点为叶节点 a_0 和 i_0 ，

如图 8-14(b)所示。这样一来就避免了将本次的数组引用误判为公共子表达式。

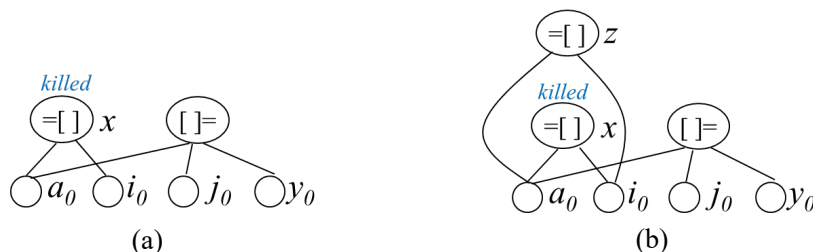


图 8-35 数组引用与赋值的 DAG 构建

除了上述优化手段之外，基本块内还可实施其他局部优化，例如常量传播和复写传播等。常量传播是将已知常量值替换到表达式中使用该常量的位置；复写传播则是在不改变语义的前提下，用同一变量替换另一变量的副本，为进一步删除冗余赋值创造条件。这些方法共同提升了中间代码的质量与执行效率。

8.3.3 从 DAG 到基本块的重组

在完成基本块 DAG 的构建与优化后，需要将优化后的 DAG 重新转换为三地址代码序列，这一过程称为从 DAG 到基本块的重组。通过重组能够将 DAG 所蕴含的优化效果具体落实到最终生成的代码上。

基本块的重组需遵循以下的原则：

(1) 指令的生成顺序必须严格遵循 DAG 节点间的依赖关系，即一个节点的计算指令必须在其所有子节点的计算指令之后出现，从而保证计算语义的正确性；

(2) 对每个具有若干定值变量的节点，只构造一个运算语句来计算其中某个定值变量的值；

(3) 倾向于把计算结果赋值给一个在基本块出口处活跃的变量，如果没有全局活跃变量的信息作为依据，就要假设所有变量都在基本块出口处活跃，除了编译器生成的临时变量；

(4) 如果节点有多个附加的活跃变量，就必须引入复制语句，以便给每一个变量都赋予正确的值；

(5) 对于那些在基本块出口处既不活跃且其值在后续计算中也未被引用的节点，其所对应的运算不必生成任何代码，这直接实现了无用代码的消除。

在生成代码时，一个关键优化体现在对公共子表达式的处理上。由于

DAG 中每个计算节点是唯一的，这意味着在重组过程中，每个计算节点至多只生成一条实现其运算的语句，从而保证了公共子表达式只被计算一次。

例 8.15 给定一个基本块，构建其对应的 DAG 如图 8-15(a)所示。假设仅变量 L 在基本块出口之后活跃。

- | | | |
|---------------|---------------|---------------|
| ① $B = 3$ | ⑤ $G = B * F$ | ⑨ $K = B * 5$ |
| ② $D = A + C$ | ⑥ $H = A + C$ | ⑩ $L = K + J$ |
| ③ $E = A * C$ | ⑦ $I = A * C$ | ⑪ $M = L$ |
| ④ $F = E + D$ | ⑧ $J = H + I$ | |

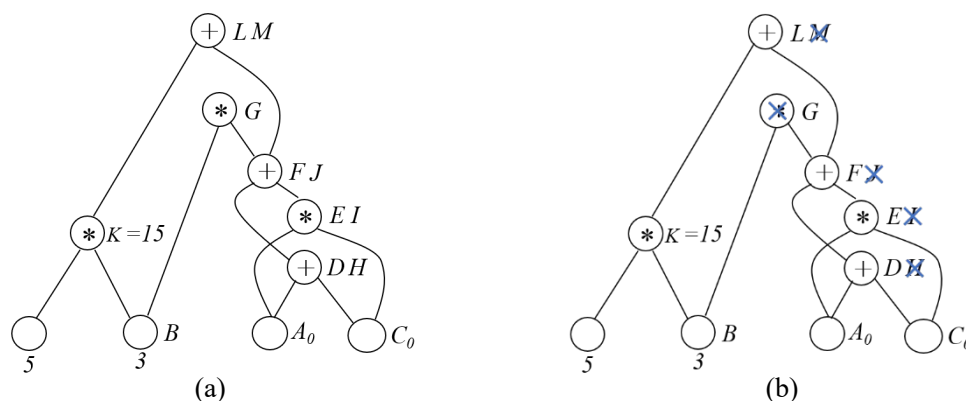


图 8-36 DAG 重组

由 $B=3$ 常量传播可知 $K=15$ ；进行无用代码删除定值变量表为 G 的节点。对于其它节点，从定值变量表中选择一个变量进行定值运算。因变量 L 活跃，因此定值变量表为 L 和 M 的节点运算结果赋值给 L；而其它节点可以依次选择最左边的节点 F、E、D，因变量 J、I 和 H 均不活跃，无需生成计算代码。最后按照计算的依赖关系生成代码如下：

```

D=A+C
E=A*C
F=E+D
L=15+F

```

综上所述，通过这种有规则的代码重组过程，能够将优化后的 DAG 高效且正确地映射回线性的三地址指令序列，从而圆满完成基本块内的局部优化。

8.4 数据流分析

在完成了基于基本块的局部优化之后，面临的挑战是如何将优化的范围扩展到整个程序的控制流图。然而，要对跨基本块的代码进行安全且有效的变换，编译器必须获得关于程序执行路径的全局信息。例如，为了删除一个全局公共子表达式，需要知道该表达式在程序的所有路径上，在到达当前点之前是否都已被计算过；为了将循环中的不变表达式外提，需要确定该表达式在循环体内是否确实保持不变。这些问题的答案无法从一个基本块内部获得，必须通过分析程序执行路径上的数据流动规律来求解。这种系统化的分析方法，就是数据流分析。

数据流分析为几乎所有类型的全局优化提供了至关重要的决策依据。其核心思想是通过在流图上建立并求解数据流动的方程，来推断程序在任意执行点上的属性。这些属性是进行全局优化的前提和保障。可以说，如果没有成熟的数据流分析理论和技术，现代编译器的优化能力将大打折扣。

数据流分析的思想和方法论，早已超越了传统编译器的范畴，在计算机科学的众多前沿和关键领域展现出强大的生命力和广泛的应用价值，主要包括软件安全与漏洞检测、大数据与并行计算、智能化程序分析与开发以及软件的形式化验证与静态分析工具等。

8.4.1 数据流基本概念

在深入数据流分析的模式和算法之前，首先需要建立一套概念体系。这些概念是描述程序行为、定义分析问题的基础。

在程序执行过程中，我们重点关注与程序变量值变化相关的关键位置。这些关键位置称为程序点。程序点（**program point**）是指令（或中间代码语句）之间、之前或之后的逻辑位置。具体来说，在基本块内部，每个语句之前和之后都是一个程序点；对于一个基本块，其开始位置称为入口点，结束位置称为出口点。程序点是我们观察和推理程序状态的“观察窗”。数据流分析的目标，就是计算出关键的程序点上所持有的某种程序状态信息。

程序在某个时刻的“全部情况”由其状态来描述。在进行数据流分析时，通常对状态进行抽象。程序状态（**program state**）是指在某个程序点，由程序中所有变量（包括临时变量）的当前值，以及运行时栈（存储函数调用信息、局部变量等）共同构成的一个快照。程序的执行，本质上就是程序状态的一系

列转换。每执行一条语句，都会读取其输入程序状态，经过计算，产生一个输出程序状态，这个输出状态将成为下一条语句的输入状态。

程序通常存在分支、循环等控制结构，因此，从程序点 p_1 到程序点 p_n 可能存在多条可能的执行路线。执行路径（execution path）是程序点的一个序列 $p_1, p_2, p_3, \dots, p_n$ ，对于每个 $i = 1, 2, 3, \dots, n-1$ 满足下列两个条件之一：

- (1) 要么从 p_i 到 p_{i+1} 的转换对应于某一条语句的执行；即它们是紧挨着某条语句之前和之后的点。
- (2) 要么 p_i 是某个基本块的出口点，而 p_{i+1} 是该基本块的某个后继基本块的入口点。

在流图中，一条执行路径对应于从入口节点到出口节点的一条路径。

数据流值是在每个程序点上，与正在进行的分析相关的所有信息的集合。它是对具体程序状态在该分析维度上的一个抽象。每一个数据流值都是某个数据流域（一个数学集合，如所有表达式的集合）的一个元素。程序状态是具体的、细粒度的；数据流值是抽象的、粗粒度的。例如，对于“可用表达式分析”，程序状态记录了所有变量的具体值，而数据流值仅记录“哪些表达式的结果已经被计算并保存”。数据流值在程序点之间流动，并被语句的语义所改变，因此它也被称为分析状态。数据流分析关心的是抽象的数据流值，这些数据流值沿着流图中可能的执行路径进行传播和交汇。

例 8.16 如图 8-16 中的流图。每条语句或基本块之前和之后都标注了程序点，程序点(1)即是语句 d_1 之前的程序点，也是基本块 B_j 的入口点；程序点(2)即是语句 d_1 之后的程序点，也是语句 d_2 之前的程序点；程序点(3)即是语句 d_2 之后的程序点，也是基本块 B_j 的出口点；可见对于相邻的两条语句 d_i 和 d_{i+1} ， d_i 之后的程序点与 d_{i+1} 之前的程序点相同；若 d_i 是某个基本块 B_j 的第一条语句，那么 d_i 之前的程序点与 B_j 的入口点相同；同理，若 d_i 是某个基本块 B_j 的最后一条语句，那么 d_i 之后的程序点与 B_j 的出口点相同。

再来观察流图的执行路径。图 8-16 中的流图很简单，然而执行路径却是无限多个。最短的执行路径是由程序点序列（1, 2, 3, 4, 5, 6, 7, 12, 13）组成，它不进入循环。次短的执行路径执行一次循环，它由程序点序列（1, 2, 3, 4, 5, 6, 7, 8, 9, 10, 11, 4, 5, 6, 7, 12, 13）组成。在本次执行中，第一次执行到程序点（4）时，因为定值 d_1 ， i 的值必然为 1。此时称定值 d_1 到达了程序点（4）。但在后面的迭代中，第二次执行到程序点（4）时， d_3 到达了程序点（4）， i 的值是 2。在本次执行过程中，我们重点关注了能够到达某个程序点的定值的集合，这对应了到达定值数据流分析应用，它将某个程序点

的数据流值与某个定值的集合关联起来。

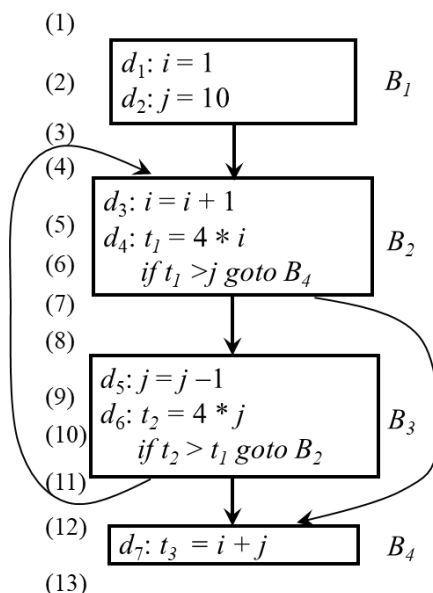


图 8-37 数据流抽象示例

正如所见，在具体的数据流分析应用中，并不跟踪程序的整体状态，而是根据应用抽象出所需要的数据。再比如在活跃变量分析中，只关心某个程序点上哪些变量的值在该点之后的某条路径中会被使用，是将某些变量的集合与程序点关联起来的一种分析。

8.4.2 数据流分析模式

数据流分析的实现途径是建立和求解数据流方程。数据流方程是用于描述流入和流出某个程序单元的数据流信息之间的联系。程序单元可以是单条语句、基本块或循环等。数据流方程的构建依赖程序单元之间语义约束和控制流约束。

语义约束指一个程序单元之前和之后的数据流值的关系，由程序单元包含的语句语义推导出来，表示为程序单元对它之前和之后的数据流值进行转换的函数。控制流约束指由控制流推导出的相邻程序单元之间的数据流值的关系。

8.4.2.1 语句的数据流模式

首先考虑语句的语义约束对数据流值的影响。一个语句 s 之前和之后的数据流值分别记为 $IN[s]$ 和 $OUT[s]$ 。如图 8-17 一个语句之前和之后的数据流值受

该语句的语义的约束。一个语句之前和之后的数据流值的关系被称为传递函数，通常被记为 f_s 。

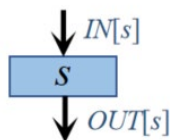


图 8-38 语句 s 之前和之后的数据流

根据数据流在执行路径中的流动方向，传递函数对应两种风格，一种是数据流信息沿执行路径前向传播称为前向数据流问题。在这种情况下，一个语句 s 的传递函数 f_s 以语句前的数据流值作为输入，以语句后的数据流值作为输出，称为前向数据流方程，即：

$$OUT[s] = f_s(IN[s]) \quad (8.1)$$

另一种是风格是数据流信息沿执行路径逆向传播称为逆向数据流问题。在这种情况下，一个语句 s 的传递函数 f_s 以语句后的数据流值作为输入，以语句前的数据流值作为输出，称为逆向数据流方程，即：

$$IN[s] = f_s(OUT[s]) \quad (8.2)$$

接下来考虑语句之间的控制流约束对数据流值的影响。基本块内相邻两个语句之间的控制总是满足顺序执行的关系，因此数据流值的关系简单，而基本块之间相邻两个语句之间的数据流值的关系就比较复杂了。首先考虑基本块内简单的情况，设基本块 B 由语句 $s_1, s_2, \dots, s_n$ 顺序组成，由于基本块内任意语句对 s_{i+1} 与 s_i 之间总是顺序执行的关系，那么 s_i 输出（或输入）的数据流值与 s_{i+1} 输入（或输出）的数据流值相同，也就是有

$$IN[s_{i+1}] = OUT[s_i] \quad i = 1, 2, \dots, n-1 \quad (8.3)$$

8.4.2.2 基本块上的数据流模式

对于每个基本块 B ，把紧靠基本块 B 之前和紧随基本块 B 之后的数据流值分别记为 $IN[B]$ 和 $OUT[B]$ ，基本块 B 的传递函数记为 f_B 。

设基本块 B 由语句 $s_1, s_2, \dots, s_n$ 顺序组成，则

$$IN[B] = IN[s_1] \quad (8.4)$$

$$\text{OUT}[B] = \text{OUT}[s_n] \quad (8.5)$$

首先考虑基本块 B 的传递函数。对于前向数据流问题，可知 $\text{OUT}[B] = f_B(\text{IN}[B])$ 。对于第 n 条语句 s_n 由基本块内语句之间的传递函数公式 (8.1) 可知 $\text{OUT}[s_n] = f_{s_n}(\text{IN}[s_n])$ ，再由基本块内语句之间的控制流约束公式 (8.3) 可知 $\text{IN}[s_{n-1}] = \text{OUT}[s_n]$ ，以此类推， f_B 可以通过将基本块中各语句的传递函数组合起来得到，即 $f_B = f_{s_n} \cdot \dots \cdot f_{s_2} \cdot f_{s_1}$ 。具体推导过程如下：

$$\begin{aligned} \text{OUT}[B] &= \text{OUT}[s_n] \\ &= f_{s_n}(\text{IN}[s_n]) \\ &= f_{s_n}(\text{OUT}[s_{n-1}]) \\ &= f_{s_n} \cdot f_{s_{n-1}}(\text{IN}[s_{n-1}]) \\ &= f_{s_n} \cdot f_{s_{n-1}}(\text{OUT}[s_{n-2}]) \\ &\dots\dots \\ &= f_{s_n} \cdot f_{s_{n-1}} \cdot \dots \cdot f_{s_2}(\text{OUT}[s_1]) \\ &= f_{s_n} \cdot f_{s_{n-1}} \cdot \dots \cdot f_{s_2} \cdot f_{s_1}(\text{IN}[s_1]) \\ &= f_{s_n} \cdot f_{s_{n-1}} \cdot \dots \cdot f_{s_2} \cdot f_{s_1}(\text{IN}[B]) \end{aligned}$$

同理，对于逆向数据流问题，其语句约束方程为： $\text{IN}[B] = f_B(\text{OUT}[B])$ ，其中 $f_B = f_{s_1} \cdot f_{s_2} \cdot \dots \cdot f_{s_n}$ 。

一个基本块可能会有多个前驱基本块或多个后继基本块，因而，前后相邻的基本块之间的数据流值的约束关系依情况而定。例如，在前向数据流问题中，流入基本块 B 的数据流值 $\text{IN}[B]$ 与它的前驱基本块 P 的 $\text{OUT}[P]$ 相关，而具体关系与具体的数据流问题相关。如在可用表达式分析中，一个表达式在 B 的入口处可用的前提是该表达式在 B 的所有前驱块的出口处都是可用的，因此，其 $\text{IN}[B]$ 值是它的所有前驱基本块 P 的 $\text{OUT}[P]$ 的交集，即

$$\text{IN}[B] = \bigcap_{P \text{ 是 } B \text{ 的前驱}} \text{OUT}[P] \quad (8.6)$$

而对于同是前向流问题的到达-定值分析问题，其 $\text{IN}[B]$ 值则是它的所有前驱基本块 P 的 $\text{OUT}[P]$ 的并集。

同理，在逆向流问题中，流入基本块 B 的数据流值 $\text{OUT}[B]$ 与它的后继基本块 S 的 $\text{IN}[S]$ 值相关，而具体关系与同样与具体的数据流问题相关。

综上所述，给定的具体的数据流分析应用，可根据语义约束即传递函数和控制流约束为所有基本块建立一组数据流方程。数据流方程通常没有唯一解，我们的目标是找到能够支持有效地代码改进，但是又不会导致不安全的转换的解。

正如下文中将阐述的，我们将采用迭代算法求一个精确而稳定的解。

8.4.3 到达-定值数据流方程

变量 x 的**定值 (Definition)** 是 (可能) 将一个值赋给 x 的语句。如果存在一条从紧跟在 x 的定值 d 后面的点到达某一程序点 p 的路径，而且在此路径上没有对 x 的其它定值，则称定值 d 到达程序点 p 。如果在此路径上有对变量 x 的其它定值 d' ，则称定值 d 被定值 d' “注销 (杀死)” 了，即定值 d 不能到达程序点 p 。

直观地讲，如果某个变量 x 的一个定值 d 到达程序点 p ，在点 p 处使用的 x 的值可能就是由 d 最后赋予的。

例 8.17 来看一个例子，根据上文中的定义来计算基本块入口和出口处的到达定值。假设有如图 8-18 的流图。为了便于计算，假设每个控制流图都有两个空基本块，包括代表图的开始点的 ENTRY 节点和结束点的 EXIT 节点。

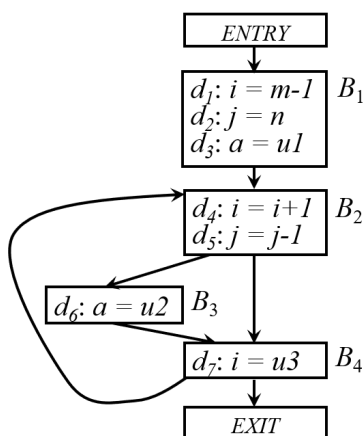


图 8-39 数据流分析流图示例

在这个流图中有 7 个定值，分别是 d_1 - d_7 ，先来看一下可以到达各基本块的入口处的定值。很显然没有任何定值到达 B_1 的入出处，即 $IN[B_1] = \emptyset$ 。 B_1 中的三个定值都可以到达 B_2 的入口， B_2 中的 d_4 却不能到达 B_2 的入口，因为控制在回到 B_2 之前，必须要经过 B_4 ，而 B_4 中的 d_7 对 i 重新定值了，因此， d_4 被 d_7 注销了，不能到达 B_2 的入口。而 d_5 可以到达 B_2 的入口，因为在回到 B_2 入口的路径上没有对 j 的新定值。同理， d_6 和 d_7 也能到达 B_2 的入口。因此， $IN[B_2] = \{d_1, d_2, d_3, d_5, d_6, d_7\}$ 。由流图中控制流可知， B_1 中的 d_1 和 d_2 在到达 B_3 的入口

处的路径上必经过 B_2 ，而 B_2 中的 d_4 和 d_5 分别注销了 d_1 和 d_2 ，因此 d_1 和 d_2 不能到达 B_3 的入口。由于 B_4 中的 d_7 到达 B_3 入口必经过 B_2 ，而 B_2 中的 d_4 注销了 d_7 ，因此 d_7 也无法到达 B_3 的入口。因此 $IN[B_3] = \{d_3, d_4, d_5, d_6\}$ 。同理， $IN[B_4] = \{d_3, d_4, d_5, d_6\}$ 。

到达定值是最常用的数据流应用之一。例如，如果知道某个变量 x 的某个引用点 p 的到达定值信息，编译器就能够知道 x 在 p 点上的值是否为常量，以决定是否可以使用常量替换 x ；再如，在流图的入口处为某个变量 x 引入一个哑定值，如果 x 的这个哑定值到达了一个可能使用 x 的程序点 p ，那么 x 就可能在程序点 p 未定值之前就使用。到达定值信息还可用于循环不变计算的检测。如果循环中含有赋值 $x = y + z$ ，而 y 和 z 所有可能的定值都在循环外面（包括 y 或 z 是常数的特殊情况），那么 $y + z$ 就是循环不变计算，就有机会被移到循环之外，以提升循环效率。

下面考虑如何构建数据流方程实现到达定值的求解。很显然，到达定值是前向流问题。数据流值求解顺序通常是先求解到达各个基本块入口处和出口处的数据流值，再根据基本块内各个语句的传递函数即可求出某个程序点处的数据流值。根据上文所述，基本块的传递函数是基本块内所有语句的传递函数的组合函数，因此，首先分析单个语句的传递函数。

设有定值语句 $d: u = v + w$ ，这里“+”号代表一个一般性的二元运算符。我们说该语句“生成”了一个对变量 u 的定值 d ，并“注销（或杀死）”了程序中所有其它对 u 的定值，维持其它变量的定值没有变化。因此，定值 d 的传递函数

$$f_d(x) = gen_d \cup (x - kill_d) \quad (8.7)$$

其中， x 为定值 d 之前的到达定值集合； $f_d(x)$ 为定值 d 之后的到达定值集合； gen_d 为由语句 d 生成的定值的集合，此处， $gen_d = \{d\}$ ； $kill_d$ 为由语句 d 注销（或杀死）的定值集合，即程序中所有其它对 u 的定值。可见到达定值 x 经过定值 d 之后， x 的一部分在流经 d 时被注销，同时还会生成新的定值 d ，因此，我们把传递函数的这种形式称为“生成-注销（杀死）形式”。

此处要说明的是，从语义上看， $kill_d$ 的精确定义应为 d 所注销的能够到达语句 d 的对 u 的定值集合。然而从计算的角度来看，对于这两种不同的定义， $x - kill_d$ 的结果都是一样的，而采用前一种定义时， $kill_d$ 的集合的计算代价更低，因此，本书采用前一种定义以加速计算。

接下来考虑基本块的传递函数。设基本块 B 的传递函数为 f_B 。假设基本块

B 包含 2 条语句，对应的传递函数分别为 $f_1(x) = gen_1 \cup (x - kill_1)$ 和 $f_2(x) = gen_2 \cup (x - kill_2)$ 。那么可知

$$\begin{aligned} f_2(f_1(x)) &= gen_2 \cup (gen_1 \cup (x - kill_1) - kill_2) \\ &= (gen_2 \cup (gen_1 - kill_2)) \cup (x - (kill_1 \cup kill_2)) \end{aligned}$$

那么，将以上规则扩展到基本块 B 中包含任意条语句的情况。假设基本块 B 中包含 n 条语句，而第 i 条语句的传递函数为 $f_i(x) = gen_i \cup (x - kill_i)$ ， $i = 1, 2, \dots, n$ 。定义 gen_B 为基本块 B 生成的定值的集合，而 $kill_B$ 表示由基本块 B 注销的定值的集合，那么有

$$f_B(x) = gen_B \cup (x - kill_B) \quad (8.8)$$

其中， x 为基本块 B 入口处的到达定值集合，而

$$\begin{aligned} gen_B &= gen_n \cup (gen_{n-1} - kill_n) \cup (gen_{n-2} - kill_{n-1} - kill_n) \cup \dots \cup \\ &\quad (gen_1 - kill_2 - kill_3 - \dots - kill_n) \end{aligned} \quad (8.9)$$

且

$$kill_B = kill_1 \cup kill_2 \cup \dots \cup kill_n \quad (8.10)$$

从语义上理解， gen_B 是基本块 B 中各个语句生成且未被 B 中位于生成该定值的语句后面的语句注销的所有定值的集合；而 $kill_B$ 是所有被 B 中各个语句注销的定值的集合。若给定了流图各个基本块的生成定值集合和注销定值集合是可以通过遍历流图直接计算出来的。

例 8.18 给定如图 8-19 的流图，求基本块 B_1 的 gen_{B_1} 和 $kill_{B_1}$ 。

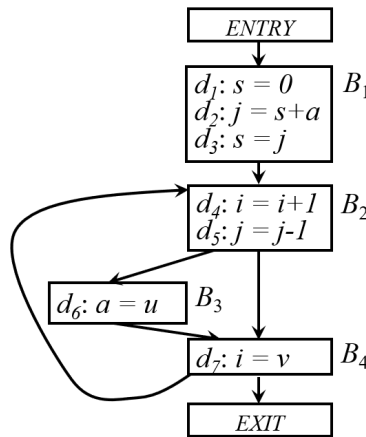


图 8-40 求基本块生成和注销定值集合流图示例

对于基本块 B_1 中的定值 d_1 生成对 s 的定值，但此定值被 B 中位于 d_1 之后的定值 d_3 注销了；而定值 d_2 和 d_3 没有被注销，因此 $gen_{B_1} = \{d_2, d_3\}$ 。由流图可知， $kill_{d_1} = \{d_3\}$ ， $kill_{d_2} = \{d_5\}$ ，而 $kill_{d_3} = \{d_1\}$ ，因此， $kill_{B_1} = kill_{d_1} \cup kill_{d_2} \cup kill_{d_3} = \{d_1, d_3, d_5\}$ 。

接下来考虑到达定值数据流应用中的控制流约束方程。

设 $IN[B]$ 为到达流图中基本块 B 的入口处的定值的集合； $OUT[B]$ 为到达流图中基本块 B 的出口处的定值的集合。根据到达定值的定义， $IN[B]$ 与基本块 B 的前驱 P 的 $OUT[P]$ 相关，且 $IN[B]$ 应为其所有前驱 P 的 $OUT[P]$ 的并集，即

$$IN[B] = \bigcup_{P \text{ 是 } B \text{ 的一个前驱}} OUT[P] \quad (B \neq ENTRY) \quad (8.11)$$

由于在控制流约束中，总是所有前驱块出口处的定值集合汇总得到一个后继块的入口处的定值集合，故此处的并运算也称为交汇运算。在其它数据流模式中，也使用交汇运算汇总各条路径会合点上的不同路径上的数据流值。

综上所述，可得到到达定值的数据流方程如下：

$$\begin{aligned} OUT[ENTRY] &= \Phi \\ OUT[B] &= gen_B \cup (IN[B] - kill_B) \quad (B \neq ENTRY) \\ IN[B] &= \bigcup_{P \text{ 是 } B \text{ 的一个前驱}} OUT[P] \quad (B \neq ENTRY) \end{aligned} \quad (8.12)$$

对于特定的流图，各个基本块的 gen 集和 $kill$ 集是可以直接从流图中计算出来的，因此在方程中作为已知量。假设流图有 n 个基本块（除了 $ENTRY$ 和 $EXIT$ ），可以构建 $2n$ 个与变量 $IN[B]$ 和 $OUT[B]$ 相关的线性方程组。我们的目标是求解所有的 $IN[B]$ 和 $OUT[B]$ 值。可以使用如下的迭代算法求解这个方程组。

算法 8.2 到达定值数据流方程求解迭代算法

输入：流图 G 及每个基本块的 gen 和 $kill$

输出：每个基本块 B 的 $IN[B]$ 和 $OUT[B]$

方法：

- 1) $OUT[ENTRY] = \Phi$;
- 2) for (除 $ENTRY$ 之外的每个基本块 B) $OUT[B] = \Phi$;
- 3) while (某个 OUT 值发生了改变)
- 4) for (除 $ENTRY$ 之外的每个基本块 B) {
- 5) $IN[B] = \bigcup_{P \text{ 是 } B \text{ 的一个前驱}} OUT[P]$;
- 6) $OUT[B] = gen_B \cup (IN[B] - kill_B)$;
- 7) }

首先，将各个基本块 B 的 $OUT[B]$ 初始化为空集。然后，按照数据流动的

方向自上而下地，分别根据公式（8.11）和公式（8.12）计算各个基本块 B 的 $IN[B]$ 和 $OUT[B]$ 值。只要某个基本块的 OUT 值发生了改变，就不停地自上而下迭代计算，直到各个 IN 值收敛，因此各个 OUT 值也收敛。

可以证明，算法 8.2 必然会终止，因为由公式（8.12）可知，对于各个基本块 B ， $OUT[B]$ 中的定值数量只会增加不会减少，而流图中定值的数量是有限的，最终必然会有一轮迭代未向任何 OUT 加入任何定值，此时算法就满足了迭代终止条件。

例 8.19 如图 8-18 的流图，求到达各个基本块 B 入口和出口处的定值集合。为了实现高效计算，使用 7 位的位向量表示定值的集合。位向量的第 i 位的值表示定值 d_i 是否在集合中，当 $d_i = 0$ 表示 d_i 不在集合中，而当 $d_i = 1$ 表示 d_i 在集合中。集合的并运算通过位向量的逻辑“或”运算实现；两个集合 S 和 T 的差运算 $S-T$ ，可先求 T 的位向量 a 的“补（按位取反）” $\bar{a}$ ，再将 $\bar{a}$ 与 S 的位向量进行逻辑与运算。

首先计算各个基本块 B 的 gen_B 和 $kill_B$ ，结果如下：

$gen_{B_1} = \{d_1, d_2, d_3\}$ ，位向量为 111 0000

$kill_{B_1} = \{d_4, d_5, d_6, d_7\}$ ，位向量为 000 1111

$gen_{B_2} = \{d_4, d_5\}$ ，位向量为 000 1100

$kill_{B_2} = \{d_1, d_2, d_7\}$ ，位向量为 110 0001

$gen_{B_3} = \{d_6\}$ ，位向量为 000 0010

$kill_{B_3} = \{d_3\}$ ，位向量为 001 0000

$gen_{B_4} = \{d_7\}$ ，位向量为 000 0001

$kill_{B_4} = \{d_1, d_4\}$ ，位向量为 100 1000

接下来根据算法 8.2 迭代计算各个基本块 B 的 IN 值和 OUT 值，计算过程表示为如图 8-20 所示。各个基本块的 OUT 初始值记为 $OUT[B]^0$ ，由算法 8.2 中第 2 行的初始化代码赋值。它们都是空集，因此对应的位向量都是 000 0000。算法迭代第 i 轮得到的 IN 值和 OUT 值通过上标 i 来区分。

B	$OUT[B]^0$	$IN[B]^1$	$OUT[B]^1$	$IN[B]^2$	$OUT[B]^2$	$IN[B]^3$	$OUT[B]^3$
B_1	000 0000	000 0000	111 0000	000 0000	111 0000	000 0000	111 0000
B_2	000 0000	111 0000	001 1100	111 0111	001 1110	111 0111	001 1110
B_3	000 0000	001 1100	000 1110	001 1110	000 1110	001 1110	000 1110
B_4	000 0000	001 1110	001 0111	001 1110	001 0111	001 1110	001 0111
$EXIT$	000 0000	001 0111	001 0111	001 0111	001 0111	001 0111	001 0111

图 8-41 迭代法求解到达定值的过程

假设算法 8.2 第 4 行对基本块的处理顺序是 B₁, B₂, B₃, B₄, EXIT。首先, 进行第一轮迭代, 令 $B = B_1$, 计算 $IN[B_1]^1$, 根据算法 8.2 第 5 行代码, $IN[B_1]$ 等于 B₁ 各个前驱的 OUT 值的并集。因 B₁ 只有一个前驱 ENTRY, 因此 $IN[B_1]^1$ 等于 $OUT[ENTRY]$, 即空集, 位向量为 000 0000。根据算法 8.2 第 6 行代码, 因 $IN[B_1]^1$ 为空集, $OUT[B_1]^1$ 等于 $gen(B_1)$, 即 111 0000。

接下来令 $B = B_2$, 计算 $IN[B_2]^1$, $IN[B_2]^1$ 等于 B₂ 各个前驱的 OUT 值的并集, 已知 B₂ 有两个前驱块 B₁ 和 B₄, 因此

$$\begin{aligned} IN[B_2]^1 &= OUT[B_1]^1 \cup OUT[B_4]^0 \\ &= 111\ 000 + 000\ 0000 \\ &= 111\ 0000. \end{aligned}$$

根据算法 8.2 第 6 行代码计算

$$\begin{aligned} OUT[B_2]^1 &= gen_{B_2} \cup (IN[B_2]^1 - kill_{B_2}) \\ &= 000\ 1100 + (111\ 000 - 110\ 0001) \\ &= 001\ 1100 \end{aligned}$$

依此类推, 继续计算出基本块 B₃ 和 B₄ 的 IN 值和 OUT 值, 第一轮迭代结束。由图 8-20 可见所有块的 OUT 值都发生了改变, 因此继续第二轮迭代, 用上角标 2 表示第 2 轮迭代计算出的各个基本块的 $IN[B]$ 和 $OUT[B]$ 的值。由图 8-20 可见, 第 2 轮迭代结束后, $OUT[B_2]^2$ 与 $OUT[B_2]^1$ 不同, 因此继续第三轮迭代。第三轮迭代结束, 所有基本块的 OUT 值都不再改变, 因此迭代结束。最终, 各基本块 IN 值和 OUT 值如图 8-20 最后两列所示。

当分析出各基本块入口处的到达定值信息后, 可以进一步确定基本块中每一个变量引用点的引用-定值链, 这对于进一步进行常量合并、循环不变量检测等优化处理是非常有用的。引用-定值链(Use-Definition Chains), 简称 ud 链, 是一个定值的列表, 对于变量的每一次引用都关联一个引用-定值链, 列表中包含到达该引用的所有定值。

在已知基本块 B 入口处到达定值集合的情况下, 块内各个引用的引用-定值链的计算分以下两种情况。如果块 B 中变量 a 的引用之前有 a 的定值, 如图 8-21(a)所示, 那么只有 a 的引用之前的最后一次对它的定值在该引用的 ud 链中; 如果块 B 中变量 a 的引用之前没有 a 的任何定值, 如图 8-21(b)所示, 那么 a 的这次引用的 ud 链就是 $IN[B]$ 中 a 的定值的集合。



(a) a 引用之前有对 a 的定值 (b) a 的引用之前没有对 a 的定值

图 8-42 引用-定值链的计算

例 8.20, 已知有基本块 B 如图 8-22, 假设其 $IN[B] = \{d_1, d_2\}$ 。对于定值 d_2 中 n 的引用, B 内在此引用之前未对 n 定值, 因此其引用-定值链为 $IN[B]$ 中对 n 的定值即 $[d_2]$ 。由于只有 n 唯一的定值 $n=1$ 到达 $j=n$, 因此, 此处的 n 可替换为 1。

再考虑 d_3 中 i 的引用, 由于在块内该引用之前有定值 d_1 对 i 的定值, 且是在该引用之前在块内的最后一次对 i 的定值, 因此, d_3 中 i 的引用-定值链为 $\{d_1\}$ 。由于 $i=1$ 是唯一能够到达 d_3 中 i 的定值, 因此, 此处的 i 也可替换为 1。这些替换也为将来 j 或 a 在后续代码中的引用创造了常量传播的潜在机会。

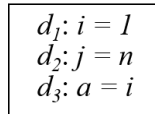


图 8-43 引用-定值链计算示例

8.4.4 活跃变量数据流方程

活跃变量分析是最常用的数据流应用之一, 它是典型的逆向数据流问题。对于变量 x 和程序点 p , 如果在流图中沿着从 p 开始的某条路径会引用变量 x 在 p 点的值, 则称变量 x 在点 p 是活跃(live)的, 否则称变量 x 在点 p 不活跃(dead)。从活跃变量的定义可见, 活跃变量信息是沿着执行路径逆向流动的。在具体应用中, 我们更关心在某个基本块 B 出口处哪些变量是活跃的, 以便依靠这些信息做出块 B 中代码优化的相关决策, 例如, 无用代码删除、DAG 重组时赋值变量的选择以及寄存器分配等。

例 8.21 如图 8-18 中的流图, 根据定义判断各个变量在各个基本块 B 出口处是否活跃。首先看变量 a , 由于在各个基本块中都没有引用变量 a 的值, 因此, a 在各个基本块出口处都不活跃。

接下来考虑变量 i , 控制离开 B_1 后会直接进入 B_2 , 而 B_2 中会引用变量

i ，而且在引用之前没有对 i 重新定值，因此， i 在 B_1 出口处活跃。变量 i 在基本块 B_4 中被定值，控制从 B_2 离开后不管走哪条路径都必须经过 B_4 ，但是从控制离开 B_2 到进入 B_4 之前的任何一条路径上没有再引用 i ，因此， i 在 B_2 出口处不活跃。同理，从控制离开 B_3 到进入 B_4 之前的任何一条路径上都没有再引用 i ，因此， i 在 B_3 出口处也不活跃。控制离开 B_4 后可能会直接进入 B_2 ，而 B_2 中会引用变量 i ，且在引用之前没有再次定值，因此， i 在 B_4 出口处活跃。

再考虑变量 j ，控制离开 B_1 至 B_4 后都可以进入到 B_2 中引用 j ，而且总能找到一条从各个基本块进入 B_2 的路径且该路径上没有对变量 j 的新定值，因此， j 在各基本块的出口处都活跃。

接下来考虑 m 、 n 和 u_1 ，除 B_1 以外，程序中任何地方都没有引用 m 、 n 和 u_1 ，因此，这三个变量在各基本块的出口处都不活跃。

最后考虑变量 u_2 和 u_3 ，控制离开 B_1 至 B_4 中的任何一个块后都可能进入到 B_3 中引用 u_2 ，而在整个程序中没有任何地方对 u_2 和 u_3 定值，因此， u_2 和 u_3 在各基本块的出口处都活跃。

活跃变量信息的用途主要包括删除无用赋值、DAG 到基本块重组以及为基本块分配寄存器等。如果 x 在点 p 的定值在基本块内所有后继点都不被引用，且 x 在基本块出口之后又是不活跃的，那么 x 在点 p 的定值就是无用的，可以被删除。DAG 到基本块重组时，若某个节点的定值变量表中包含多个变量，会优先选择在块的出口处活跃的变量进行运算。

在生成目标代码时进行寄存器分配时，如果所有寄存器都被占用，并且还需要申请一个寄存器，则应该考虑使用已经存放了不活跃变量值的寄存器，因为这个值不需要保存到内存。另外，如果一个值在基本块结尾处是不活跃的就无需在结尾处生成保存这个值的代码。

活跃变量分析是一个逆向数据流问题。也就是说，活跃变量信息是按照程序控制流的反方向进行计算的。因为根据定义，只有当变量的值在程序点 p 之后被引用了，才能确定变量在 p 点是活跃的。

对于基本块 B ，设 $IN[B]$ 为在块 B 的入口处的活跃变量集合； $OUT[B]$ 为在块 B 的出口处的活跃变量集合。下面来定义基本块的传递函数

$$f_B(x) = use_B \cup (x - def_B) \quad (8.13)$$

其中， x 表示在块 B 出口处的活跃变量的集合； def_B 为在块 B 中的首次出现是被定值的变量的集合。这些定值注销了变量在入口处的值，使得这些变量在入口处的值不再被使用，即这些变量在入口处不再活跃，因此它们被从 x 集合移

除。 use_B 为在块 B 中的首次出现是以被引用的变量的集合。这些引用使用的是变量在入口处的值，使得这些变量在入口处活跃，因此它被并入 x 集合。基本块的 use 集和 def 集是可以通过遍历块中的所有语句直接计算出来的。

例 8.22 对于图 8-18 中的流图，计算各个基本块的 use 集和 def 集。首先来看 B_1 ，变量 m 、 n 和 u_1 在块中的第一次出现都是被引用，因此都在 use_{B_1} 中；而变量 i 、 j 和 a 的第一次出现都是被定值，因此都在 def_{B_1} 中。再考虑 B_2 ，由于 $i = i + 1$ 是先引用 i ，再对 i 赋值，因此，变量 i 在 use_{B_2} 中，同理， j 也在 use_{B_2} 中；而 def_{B_2} 为空集。类似地，很容易计算出来， $use_{B_3} = \{u_2\}$ ， $def_{B_3} = \{a\}$ ； $use_{B_4} = \{u_3\}$ ， $def_{B_4} = \{i\}$ 。

接下来，给出活跃变量的数据流方程如下：

$$IN[EXIT] = \Phi \quad (8.14)$$

$$IN[B] = f_B(OUT[B]) = use_B \cup (OUT[B] - def_B) \quad (B \neq EXIT) \quad (8.15)$$

$$OUT[B] = \bigcup_{S \text{ 是 } B \text{ 的一个后继}} IN[S] \quad (B \neq EXIT) \quad (8.16)$$

首先，因为结束节点 $EXIT$ 是一个不包含任何语句的空块，因此，其入口处不包含任何活跃变量，即 $IN[EXIT] = \Phi$ 。公式 (8.15) 说明一个变量在进入基本块 B 时活跃（在 $IN[B]$ 中），要么它是该基本块中被重新定值之前就被引用（在 use_B 中），要么它在离开该基本块时活跃（在 $OUT[B]$ 中）且在该基本块中没有对它重新定值（不在 def_B 中）。最后，公式 (8.16) 说明一个变量在离开一个基本块 B 时活跃当且仅当它在进入该基本块的某个后继时活跃。各个基本块的 use 集和 def 集在方程中作为已知量。

与到达定值数据流方程求解方法类似，可以采用如算法 8.3 的迭代法求解活跃变量的数据流方程。

算法 8.3 活跃变量数据流方程求解迭代算法

输入：流图 G 及每个基本块的 use 和 def

输出：每个基本块 B 的 $IN[B]$ 和 $OUT[B]$

方法：

- 1) $IN[EXIT] = \Phi$;
- 2) for (除 $EXIT$ 之外的每个基本块 B) $IN[B] = \Phi$;
- 3) while (某个 IN 值发生了改变)
- 4) for (除 $EXIT$ 之外的每个基本块 B) {
- 5) $OUT[B] = \bigcup_{S \text{ 是 } B \text{ 的一个后继}} IN[S]$;
- 6) $IN[B] = use_B \cup (OUT[B] - def_B)$;

7) }

活跃变量分析的核心目标是判断“某变量在程序某点之后是否被使用”，这为寄存器分配、无用代码删除等优化提供了关键依据。而定值-引用链（DU-Chain）是数据流分析技术的另一核心工具，它直接承接活跃变量分析的思想，聚焦于变量的定值与引用之间的关联关系，是实现常量传播、复制传播以及部分冗余删除等优化的基础。设变量 x 有一个定值 d ，该定值能够到达的所有引用 u 的集合称为 x 在 d 处的定值-引用链，简称 du 链。

如果在求解活跃变量数据流方程中的 $OUT[B]$ 时，将 $OUT[B]$ 表示为从 B 的末尾处能够到达的引用的集合，那么，可以直接利用这些信息计算基本块 B 中每个变量 x 在其定值处的 du 链。假设基本块 B 的出口处的 du 链 $OUT[B]$ 已求出，那么 B 中定值 d 的 du 链的计算分为以下两种情况。如果 B 中 x 的定值 d 之后有 x 的另一个定值 d' ，如图 8-23(a)，则 d 和 d' 之间 x 的所有引用构成 d 的 du 链；如果 B 中 x 的定值 d 之后没有 x 的新的定值，如图 8-23(b)，则 B 中 d 之后 x 的所有引用以及 $OUT[B]$ 中 x 的所有引用构成 d 的 du 链。

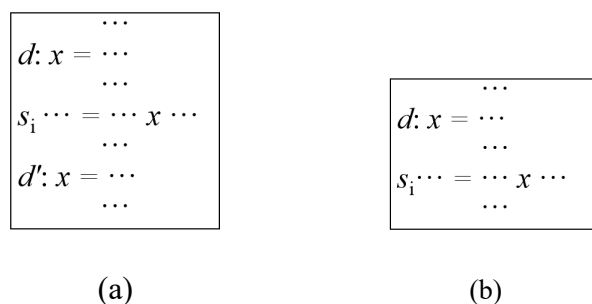


图 8-44 定值-引用链的计算

8.4.5 可用表达式数据流方程

可用表达式分析是数据流分析技术中的另一类核心方法，它属于前向数据流问题，其核心目标是判断在程序的某个程序点上，一个表达式的值是否已经被计算过并且可以被重用。

如果从流图的首节点到达程序点 p 的每条路径都对表达式 $x \text{ op } y$ 进行了计算，并且从最后一个这样的计算到点 p 之间没有再次对 x 或 y 定值，那么表达式 $x \text{ op } y$ 在点 p 是可用的(available)。表达式可用的直观意义是如果在点 p 上， $x \text{ op } y$ 是可用的，那么就不必重新计算了。

可用表达式分析的直接应用场景是全局公共子表达式删除：如果一个表达

式在多个程序点都是可用的，说明其值无需重复计算，只需在第一次计算时保存结果，后续直接复用即可，这能大幅减少程序的运算量。

例 8.23 如图 8-24 中的三种情况流图，判断在进入 B_3 时，表达式 $4*i$ 是否可用。首先看 (a) 的情况，可见从流图的首节点到 B_3 的入口处的所有路径都要经过 B_1 ，而在 B_1 中对 $4*i$ 进行了计算，并且从计算的点开始到 B_3 入口点的任何路径上都未对 i 定值，因此表达式 $4*i$ 在 B_3 的入口处是可用的， B_3 块中的 $4*i$ 是公共子表达式。再来看 (b) 的情况， B_1 中计算了 $4*i$ ，但存在一条从 B_1 经 B_2 到达 B_3 的路径，在 B_2 中有对 i 的定值，并且在该定值后未重新计算 $4*i$ 的值，因此， $4*i$ 在 B_3 的入口点不可用，即 B_3 中的 $4*i$ 值可能与 B_1 中 $4*i$ 的值不同。最后看 (c) 的情况，从首节点 B_1 到 B_3 有两条路径，而且都对 $4*i$ 进行了计算，在每次计算之后，到达 B_3 入口之前均未对 i 定值，因此， $4*i$ 在 B_3 的入口处是可用的，可以不必在 B_3 中重复计算。

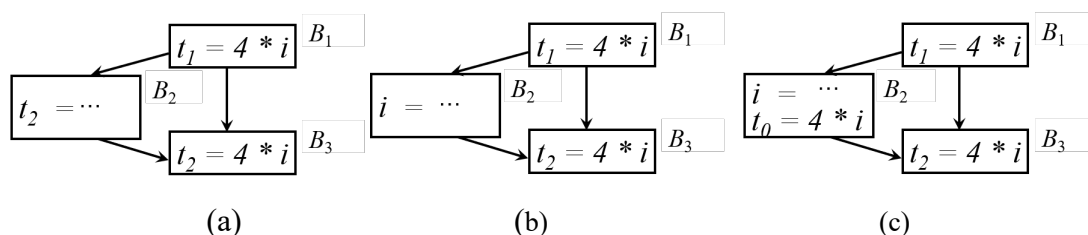


图 8-45 可用表达式判断示例

如果将类似 $x=y$ 的复制运算也看作表达式，复制运算表达式的可用性信息可用于复制传播优化。从流图的首节点到达 u 的每条路径都存在复制语句 $x=y$ ，并且从最后一条复制语句 $x=y$ 到点 u 之间没有再次对 x 或 y 定值，我们称复制语句 $x=y$ 在引用点 u 处可用。在 x 的引用点 u 可以用 y 代替 x 的条件是复制语句 $x=y$ 在引用点 u 处可用。

例 8.24 如图 8-25 所示，因从流图的首节点只有一条路径到达语句 $t_1 = x * 2$ ，在此路径中有对 $x=y$ 的计算，并且计算点之后该路径上未有对 x 或 y 的定值，因此， $x=y$ 在 x 的该引用点可用，可以使用 y 代替 x 。再考虑 x 的另一个引用点。由图可见，从流图的首节点有两条路径可以到达 $t_2 = x * 3$ ，但只有一条路径计算了 $x=y$ ，因此， $x=y$ 在该 x 的引用点不可用，不能进行复制传播。

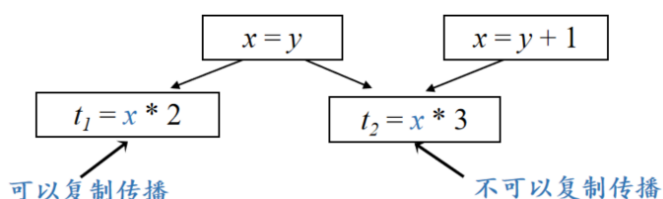


图 8-46 复制传播的条件示例

下面考虑基本块上的可用表达式的传递函数。如果基本块 B 对 $x \text{ op } y$ 进行了计算，并且之后没有重新定值 x 或 y ，则称 B 生成表达式 $x \text{ op } y$ ；如果基本块 B 对 x 或者 y 进行了(或可能进行)定值，且之后没有重新计算 $x \text{ op } y$ ，则称 B 注销(杀死)表达式 $x \text{ op } y$ 。定义 e_gen_B 为基本块 B 所生成的可用表达式的集合； e_kill_B 为基本块 B 所注销(杀死)的 U 中的表达式的集合，其中 U 为流图中的所有表达式的集合。那么基本块 B 上的传递函数为

$$f_B(x) = e_gen_B \cup (x - e_kill_B) \quad (8.17)$$

对于给定的基本块 B ，其生成的可用表达式集合 e_gen 和注销的表达式集合 e_kill 可以通过扫描基本块中的所有语句计算出来。计算基本块生成的可用表达式集合的核心思路是顺序扫描基本块内的每条语句 $z = x \text{ op } y$ ，并按顺序执行以下两步操作：

(1) 生成操作：将当前语句定义的表达式 $x \text{ op } y$ 加入集合 e_gen_B 。这是因为经过该语句后， $x \text{ op } y$ 已被计算，成为可被后续语句复用的可用表达式。

(2) 注销操作：从集合 e_gen_B 中删除所有包含变量 z 的表达式。这是因为当前语句对 z 进行了定值，若 e_gen_B 中存在涉及 z 的表达式，其计算结果会因 z 的定值而失效，不再具备可用性。

需要特别说明的是， z 可能与 x 或 y 相同，如语句为 $x = x \text{ op } y$ ，因此必须先执行生成操作，再执行注销操作。

类似地，计算基本块注销的表达式的集合只需顺序扫描基本块内的每条语句 $z = x \text{ op } y$ ，并按顺序执行以下两步操作：

(1) 复活操作：从 e_kill_B 中删除表达式 $x \text{ op } y$ 。这是因为 $x \text{ op } y$ 可能在之前的语句中因 x 或 y 被定值而被注销加入 e_kill_B 中，而此时 $x \text{ op } y$ 再次被计算，便成为可被后续语句复用的可用表达式，因而复活。

(2) 注销操作：把所有包含变量 z 的表达式加入到 e_kill_B 中。因为包含变量 z 的表达式计算结果会因 z 的定值而失效，不再具备可用性。

例 8.25 求基本块 B 生成的可用表达式集合 e_gen 和注销的表达式集合 e_kill 。

块B语句	e_gen_B	操作	块B语句	e_kill_B	操作
$a = b + c$	$\emptyset$	加入 $b + c$	$a = b + c$	$\emptyset$	加入 $a - d$
$b = a - d$	$\{b + c\}$	加入 $a - d$, 删除 $b + c$	$b = a - d$	$\{a - d\}$	删除 $a - d$, 加入 $b + c$
$c = b + c$	$\{a - d\}$	加入 $b + c$, 删除 $b + c$	$c = b + c$	$\{b + c\}$	删除 $b + c$, 加入 $b + c$
$d = a - d$	$\{a - d\}$	加入 $a - d$, 删除 $a - d$	$d = a - d$	$\{b + c\}$	加入 $a - d$
	$\emptyset$			$\{b + c, a - d\}$	

(a) e_gen_B 计算过程(b) e_kill_B 计算过程图 8-47 e_gen_B 和 e_kill_B 计算示例

由图 8-26(a)中的基本块可知, e_gen_B 初始为空集。按顺序扫描第一条语句 $a = b + c$, 该语句生成 $b + c$ 加入 e_gen_B , 同时该语句对 a 定值注销了之前包含 a 的可用表达式, 因此再移除 e_gen_B 中包含 a 的表达式, 因集合为空, 没有任何移除。此时 $e_gen_B = \{b + c\}$ 。扫描第二条语句 $b = a - d$, 该语句生成 $a - d$ 加入 e_gen_B , 再移除 e_gen_B 中包含 b 的表达式, 移除 $b + c$, 此时 $e_gen_B = \{a - d\}$; 扫描第三条语句 $c = b + c$, 该语句生成 $b + c$ 加入 e_gen_B , 再移除 e_gen_B 中包含 c 的表达式, 移除 $b + c$, 此时 $e_gen_B = \{a - d\}$; 扫描最后一条语句 $d = a - d$, 该语句生成 $a - d$ 加入 e_gen_B , 再移除 e_gen_B 中包含 d 的表达式, 移除 $a - d$, 此时 $e_gen_B = \emptyset$ 。至此处理结束, 基本块 B 未生成任何可用表达式。

接下来计算 e_kill_B 。首先遍历基本块将所有表达式加入集合 U 并将 e_kill_B 初始化为空集。按顺序扫描第一条语句 $a = b + c$, 该语句生成 $b + c$, 若它在 e_kill_B 中, 应当被移除, 当前 e_kill_B 不包含 $b + c$, 不需任何移除; 该语句对 a 定值, 注销了所有包含 a 的表达式, 因此, 将 U 中包含 a 的表达式 $a - d$ 加入 e_kill_B , 此时 $e_kill_B = \{a - d\}$; 扫描第二条语句 $b = a - d$, 该语句生成 $a - d$, 将它从 e_kill_B 移除, 再将 U 中包含 b 的表达式 $b + c$ 加入 e_kill_B , 此时 $e_kill_B = \{b + c\}$; 扫描第三条语句 $c = b + c$, 该语句生成 $b + c$, 将它从 e_kill_B 中移除, 该语句对 c 定值, 将 U 中与 c 有关的表达式 $b + c$ 加入 e_kill_B , 此时 $e_kill_B = \{b + c\}$; 扫描最后一条语句 $d = a - d$, 该语句生成 $a - d$, 它不在 e_kill_B , 将 U 中与 d 相关的表达式 $a - d$ 加入 e_kill_B , 此时 $e_kill_B = \{b + c, a - d\}$ 。至此处理结束, 基本块 B 注销的表达式集合为 $\{b + c, a - d\}$ 。

下面给出可用表达式的数据流方程。设 $IN[B]$ 为在 B 的入口处可用的 U 中的表达式集合； $OUT[B]$ 为在 B 的出口处可用的 U 中的表达式集合。 U 为流图中的所有表达式的集合。可用表达式的数据流方程如下：

$$OUT[ENTRY] = \Phi$$

$$OUT[B] = f_B(IN[B]) = e_genB \cup (IN[B] - e_killB) \quad (B \neq ENTRY)$$

$$IN[B] = \bigcap_{P \text{ 是 } B \text{ 的一个前驱}} OUT[P] \quad (B \neq ENTRY)$$

因为开始节点 $ENTRY$ 是一个不含任何语句的空基本块，因此， $ENTRY$ 的出口处没有任何可用表达式，即 $OUT[ENTRY] = \Phi$ 。对于所有不等于 $ENTRY$ 的基本块 B ，其入口处的可用表达式集合根据 B 的传递函数来计算。而每个基本块 B 入口处的可用表达式集合根据 B 的前驱基本块 P 来计算，根据可用表达式的定义，只有到达基本块 B 的入口处的每条路径上表达式 $x \text{ op } y$ 都可用，它在 B 的入口处才可用。因此，一个基本块 B 的 IN 值等于它的所有前驱基本块 P 的 OUT 值的交集。对于特定的流图，各个基本块的 e_gen 集和 e_kill 集也是可以直接从流图中计算出来的，因此在方程中作为已知量。

算法 8.4 可用表达式数据流方程求解迭代算法

输入：流图 G 及每个基本块的 e_gen 和 e_kill

输出：每个基本块 B 的 $IN[B]$ 和 $OUT[B]$

方法：

- 1) $OUT[ENTRY] = \Phi$;
- 2) for (除 $ENTRY$ 之外的每个基本块 B) $OUT[B] = \Phi$;
- 3) while (某个 OUT 值发生了改变)
- 4) for (除 $ENTRY$ 之外的每个基本块 B) {
- 5) $IN[B] = \bigcap_{P \text{ 是 } B \text{ 的一个前驱}} OUT[P]$;
- 6) $OUT[B] = e_genB \cup (IN[B] - e_killB)$;
- }

8.5 控制流分析

在代码优化中，循环尤其是内部循环是一个重要的地方，因为程序往往会将大部分运行时间花费在循环上。要想对循环进行优化，首先需要识别出流图中的循环。而要精准识别流图中的循环，需要先掌握控制流分析的核心概念，包括支配节点、回边和自然循环，这些概念是循环识别与后续优化的基础。

8.5.1 支配节点

支配节点（Dominators）是控制流分析中最基础的概念之一，其定义如下：如果从流图的入口节点（通常记为 ENTRY）到节点 n 的每条路径都经过节点 d ，则称节点 d 支配（dominate）节点 n ，记为 $d \text{ dom } n$ 。其中， d 称为支配节点， n 称为支配对象。根据该定义，一个显而易见的结论是每个节点都支配它自己。

例 8.26 给定图 8-27(a)中的流图。由图可知，入口节点是 1，从入口到节点 1 的“唯一路径”就是它自身，因此 1 支配 1；同时，流图中所有节点（1~10）的路径都必须从入口 1 出发，所以 1 支配所有节点（1~10）。从入口节点 1 到达节点 2 必经过它自己，而从入口节点到达 3~10 节点都存在不经过节点 2 的路径，因此，节点 2 只支配自己。从节点 1 到自身或 2 的路径均不经过 3，因此 1 和 2 不被 3 支配；从节点 1 到 3~10 的任意节点 n 的所有路径必须先到达 3，不存在不经过 3 的路径；因此，节点 3 支配 3~10。从 1 到 1、2 或 3 的路径均不经过 4，因此 1、2 和 3 不被 4 支配；从 1 到 5~10 的任意节点 n 的所有路径必须先到达 4，不存在不经过 4 的路径，因此节点 4 的支配对象是 4~10。对于节点 5，只有从 1 到 5 的路径必须经过 5，因此 5 仅支配它自身。类似地，可知 6 支配 6；7 支配 7~10；8 支配 8~10；9 和 10 均仅支配它自身。

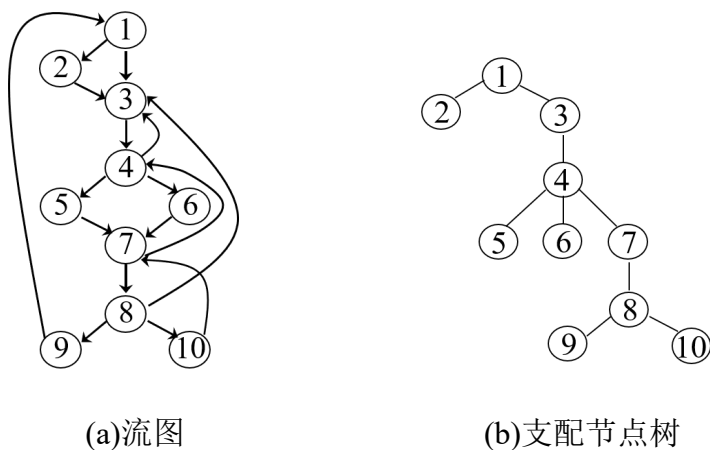


图 8-48 节点间支配关系及支配节点树

如图 8-27(b)，为了更清晰地展示流图中各节点的支配关系，通常会使用

“支配节点树”（也称为支配树）来组织这些信息。支配节点树的构造规则为：流图的入口节点是支配树的根节点，树的每个节点仅支配它在该树的所有后代节点。通过支配节点树，可以快速定位任意节点的所有支配节点，这对后续循环识别至关重要。

支配关系的本质是“所有路径的交集”即一个节点的支配节点是所有到达该节点的路径都共有的节点。因此，可以通过数据流方程定量计算每个节点的支配节点集合。

首先定义两个数据流集合， $IN[B]$ 为基本块 B 入口处的支配节点集合，即到达 B 入口前，所有必须经过的节点； $OUT[B]$ 为基本块 B 出口处的支配节点集合，即经过 B 后，所有仍对后续节点起支配作用的节点。

结合支配关系的定义，可建立如下数据流方程：

$$OUT[ENTRY] = \{ENTRY\} \quad (8.18)$$

$$OUT[B] = f(IN[B]) = IN[B] \cup \{B\} \quad (8.19)$$

$$IN[B] = \bigcap_{P \text{ 是 } B \text{ 的前驱}} OUT[P] \quad (8.20)$$

入口节点 $ENTRY$ 的出口处，仅支配自身，因为它是流图的起点，没有前置节点；公式（8.19）为支配节点分析的传递函数，对于非入口节点 B ，其出口处的支配节点，既包括入口处的支配节点（这些节点在到达 B 前已被支配），也包括 B 自身（ B 支配自身及后续节点）；公式（8.20）为支配关系产生的控制流约束，节点 B 入口处的支配节点，是其所有前驱节点出口处支配节点的交集，即只有所有前驱的出口都包含的节点，才是到达 B 的所有路径都必须经过的节点。

根据支配节点的定义， $OUT[B]$ 是节点 B 的所有支配节点的集合。

算法 8.5 计算支配节点的迭代算法

输入：流图 G ， G 的节点集是 N ，边集是 E ，入口节点是 $ENTRY$

输出：对于 N 中的各个节点 n ，给出 $D(n)$ ， $D(n) = OUT[n]$

方法：

- 1) $OUT[ENTRY] = ENTRY$;
- 2) for (除 $ENTRY$ 之外的每个基本块 B) $OUT[B] = N$;
- 3) while (某个 OUT 值发生了改变)
- 4) for (除 $ENTRY$ 之外的每个基本块 B) {
- 5) $IN[B] = \bigcap_{P \text{ 是 } B \text{ 的一个前驱}} OUT[P]$;
- 6) $OUT[B] = IN[B] \cup \{B\}$;
- 7) }

例 8.27 对于图 8-27(a)中的流图，采用算法 8.5 求解出各个节点的入口处和出口处的支配节点集合 $IN[B]$ 和 $OUT[B]$ ，求解过程如图 8-28。支配节点问题相对简单，经过两轮迭代后 OUT 值未发生变化，便收敛了。求得流图中各个节点的支配节点集合为 $OUT^2[B]$ 列对应的值。

	$OUT^0[B]$	$IN^1[B]$	$OUT^1[B]$	$IN^2[B]$	$OUT^2[B]$
E	$\{E\}$				
①	N	$\{E\}$	$\{E\}$	$\{E\}$	$\{E\}$
②	N	$\{E\}$	$\{E\}$	$\{E\}$	$\{E\}$
③	N	$\{E\}$	$\{E\}$	$\{E\}$	$\{E\}$
④	N	$\{E\}$	$\{E\}$	$\{E\}$	$\{E\}$
⑤	N	$\{E\}$	$\{E\}$	$\{E\}$	$\{E\}$
⑥	N	$\{E\}$	$\{E\}$	$\{E\}$	$\{E\}$
⑦	N	$\{E\}$	$\{E\}$	$\{E\}$	$\{E\}$
⑧	N	$\{E\}$	$\{E\}$	$\{E\}$	$\{E\}$
⑨	N	$\{E\}$	$\{E\}$	$\{E\}$	$\{E\}$
⑩	N	$\{E\}$	$\{E\}$	$\{E\}$	$\{E\}$

图 8-49 图 8-27(a)所示流图各个节点的入口处和出口处的支配节点集合

8.5.2 回边

回边是流图中构成循环的“核心链路”，循环的本质是控制流能够“回头”重新进入之前经过的节点，而回边正是这种“回头”关系的直接体现。

对于流图中的一条有向边 $n \rightarrow d$ （从节点 n 指向节点 d ），如果满足 d 支配 n ，则称这条边为**回边**。回边的核心特征是“终点支配起点”。这意味着，从流图入口节点 ENTRY 到达 n 的所有路径都必须经过 d ，而边 $n \rightarrow d$ 又将控制流从 n 拉回 d ，形成闭合路径，这正是循环的核心特征。

例 8.28 已知如图 8-27(a)中所示的流图各个节点的支配节点如图 8-28 最后一列所示。通过支配节点分析可知，3 支配 4，且有边 $4 \rightarrow 3$ ，因此边 $4 \rightarrow 3$ 是回边；4 支配 7 且有边 $7 \rightarrow 4$ ，因此边 $7 \rightarrow 4$ 也是回边；同理，边 $8 \rightarrow 3$ ， $9 \rightarrow 1$ ， $10 \rightarrow 7$ 都是回边。

回边是构成循环的核心链路，识别回边是后续识别自然循环的关键步骤，每一条回边都对应一个潜在的循环结构。

8.5.3 自然循环（natural loop）

通过回边我们找到了循环的核心链路，但一个完整的循环还包括所有参与闭合路径的节点。从程序优化的角度来看，循环的形式并不重要，重要的是它是否具有“易于分析和优化”的性质，自然循环正是一种满足该要求的循环结构。

自然循环是一种适合优化的循环结构，满足以下两个核心性质：

(1) 具有唯一的入口节点，称为首节点（也叫循环头节点，Header）。这意味着，到达循环内任意节点的所有路径都必须经过首节点，因此首节点支配循环中的所有节点。

(2) 循环中至少有一条返回首节点的回边。这条回边是循环闭合的关键，确保控制流能从循环内回到首节点，形成迭代。

这两个性质共同保证了自然循环的“易优化性”：唯一的首节点让我们可以将循环不变计算（如常量表达式）统一外提到首节点前；首节点支配所有循环节点，确保了优化后的控制流不会出现异常。

下面考虑自然循环的识别方法。自然循环与回边一一对应，每一条回边都对应一个自然循环。给定一条回边 $n \rightarrow d$ (d 是终点，且 d 支配 n)，识别其对应自然循环的步骤如下：

(1) 确定循环的首节点：回边的终点 d 即为该自然循环的首节点。因为 d 支配 n ，满足首节点支配循环内节点的性质。

(2) 收集循环节点：循环节点集合包括首节点 d 、回边的起点 n ，以及所有“不经过 d 就能到达 n ”的节点。简单来说，就是所有能通过某条路径到达 n ，且该路径不经过 d 的节点。这些节点都属于由回边 $n \rightarrow d$ 构成的闭合路径。

自然循环的识别如算法 8.6 所示。为了精准收集循环节点，可通过栈实现以下算法。

算法 8.6 识别一条回边的自然循环

输入：流图 G 和回边 $n \rightarrow d$

输出：由回边 $n \rightarrow d$ 的自然循环中的所有节点组成的集合

方法：

```

1)  $stack = \Phi$  ;  $loop = \{ n, d \}$ ;  $push(stack, n)$ ;
2) while  $stack$  不空 do {
3)    $m = top(stack)$ ;  $pop(stack)$ ;
4)   for  $m$  的每个前驱  $p$  {
5)     if  $p$  不在  $loop$  中 then {
6)        $loop = loop \cup \{ p \}$ ;

```

```

7)          Push ( stack, p );
8)          }
9)      }
10) }
```

以上算法的核心思想是逐个检查 n 的前驱节点 p ，只要 p 不是 d 就加入循环，并递归地检查 p 的前驱节点 p' ，只要 p' 不是 d ，那么它就能不经过 d 而到达 n ，就把它加入循环。

例 8.29 对于图 8-27(a)中所示的流图，求回边 $4 \rightarrow 3$ ($d = 3$, $n = 4$) 对应的自然循环。初始 $\text{loop} = \{ 3, 4 \}$ ，将 4 压入栈；接下来检查栈 stack 非空，弹出 4，遍历其前驱 3 和 7，3 已在 loop 中，不做处理，将 7 加入 loop 并压入栈 stack ，此时 $\text{loop} = \{ 3, 4, 7 \}$ ， $\text{stack} = [7]$ ；再检查栈 stack ，弹出 7，遍历其前驱 5、6 和 10，均不在 loop 中，都加入 loop 并压入栈 stack ，此时 $\text{loop} = \{ 3, 4, 5, 6, 7, 10 \}$ ， $\text{stack} = [5, 6, 10]$ ；检查栈 stack ，弹出 5，遍历其前驱 4，已在 loop 中，不做处理，此时 $\text{stack} = [6, 10]$ ；检查栈 stack ，弹出 6，遍历其前驱 4，已在 loop 中，不做处理，此时 $\text{stack} = [10]$ ；检查栈 stack ，弹出 10，遍历其前驱 8，不在 loop 中，加入 loop 并压入栈 stack ，此时 $\text{loop} = \{ 3, 4, 5, 6, 7, 8, 10 \}$ ， $\text{stack} = [8]$ ；检查栈 stack ，弹出 8，遍历其前驱 7，已在 loop 中，不做处理，此时 stack 为空，循环结束，此时 $\text{loop} = \{ 3, 4, 5, 6, 7, 8, 10 \}$ 。

自然循环的一个关键性质的是“嵌套或不相交”：如果两个自然循环的首节点不同，那么这两个循环要么完全不相交（没有共同节点），要么一个完全包含在另一个里面。这个性质让循环优化可以“由内向外”进行——先优化最内层循环（不包含其他循环的循环），再优化外层循环，避免优化过程中相互干扰。

如果两条回边对应的自然循环具有相同的首节点，则难以区分其内层外层，此时通常将这两个循环合并为一个循环，统一进行优化。

8.6 全局优化

全局优化是指作用于整个程序流图的优化技术，与局部优化（仅作用于单个基本块）相比，全局优化需要结合数据流分析结果，精准判断跨基本块的变量依赖和表达式计算情况。

常用的全局优化技术包括：全局公共子表达式消除、复制语句消除、代码

移动、递归变量的强度削弱以及递归变量删除等。本节将重点讲解这几项优化技术的核心原理、实现步骤及具体示例。

8.6.1 全局公共子表达式消除

全局公共子表达式消除致力于消除跨基本块的冗余计算。可用表达式的数据流问题可以帮助确定位于流图中 p 点的表达式是否为全局公共子表达式。根据可用表达式的定义，如果在某一程序点 p 对表达式 $x \text{ op } y$ 进行计算且表达式 $x \text{ op } y$ 在 p 点可用，那么 $x \text{ op } y$ 在 p 点的出现是一个公共子表达式。需要注意的是，仅通过基本块入口处的可用表达式集合，无法确保表达式在引用点可用。因为从基本块入口到表达式引用点之间，可能存在对表达式运算分量（ x 或 y ）的赋值操作，导致表达式的计算结果发生变化。因此，表达式程序点 p 可用需同时满足两个条件：第一，表达式在基本块入口处可用；第二，从基本块入口到引用点 p 之间，表达式的所有运算分量均未被重新定值。

例 8.30 如图 8-29 所示的左侧的流图，从流图的首节点到这个基本块 B_3 入口处的每条路径都对表达式 $x * y$ 进行了计算，在这个基本块入口处，表达式 $x * y$ 可用，假设从 B_3 入口处到 $x * y$ 引用点之前未对 x 或 y 赋值，因此它是一个公共子表达式。接下来考虑如何删除全局公共子表达式。控制流可能经过对 a 的赋值语句达到语句 $c = x * y$ 处，也可能经过对 b 的赋值语句达到语句 $c = x * y$ 处，因此，把 $c = x * y$ 替换成 $c = a$ 或 $c = b$ 都是不安全的。需要使用新的变量 u 来存放 $x * y$ 的值，如图所示，使用 $u = x * y$ 和 $a = u$ 替换原赋值语句 $a = x * y$ ；使用 $u = x * y$ 和 $b = u$ 替换原赋值语句 $b = x * y$ 。这样，就可以安全地将 $c = x * y$ 替换成 $c = u$ 了。

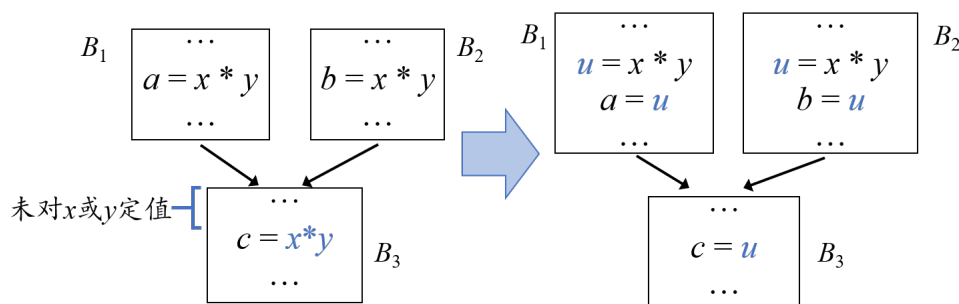


图 8-50 删除全局公共子表达式示例

扩展到一般情况，如果有语句 $z = x \text{ op } y$ ，且在该语句前，表达式 $x \text{ op } y$ 可用，那么可以用下面的方法优化：在该语句之前不同的计算 $x \text{ op } y$ 的位置，均把 $x \text{ op } y$ 的值赋值给同一个临时变量 u 。然后把语句 $z = x \text{ op } y$ 替换为复制语句 $z = u$ 。

算法 8.7 全局公共子表达式删除算法

输入：带有可用表达式信息的流图

输出：修正后的流图

方法：

对于语句 $s: z = x \text{ op } y$ ，如果 $x \text{ op } y$ 在 s 之前可用，那么执行如下步骤：

- ① 从 s 开始逆向搜索，但不穿过任何计算了 $x \text{ op } y$ 的块，找到所有离 s 最近的计算了 $x \text{ op } y$ 的语句；
- ② 建立新的临时变量 u ；
- ③ 把步骤①中找到的语句 $w = x \text{ op } y$ 用下列语句代替：

$u = x \text{ op } y;$

$w = u;$

- ④ 用 $z = u$ 替代 s 。

全局公共子表达式消除以可用表达式分析为基础，通过“统一存储结果、复用替换冗余计算”的思路，消除跨基本块的冗余表达式计算。核心是引入唯一临时变量避免路径冲突，同时严格检查表达式的可用性条件，确保优化后的程序逻辑正确。

8.6.2 复制语句的删除

复制语句是指形式为 $x = y$ 的语句。在程序编译过程中，复制语句可能因基本块重组、公共子表达式删除或归纳变量强度削弱等步骤被引入，例如前文在删除全局公共子表达式时将 $z = x \text{ op } y$ 替换为 $z = u$ ，就引入了复制语句。若复制语句的定值 x 未被有效引用，或所有对 x 的引用都可直接用 y 替代，则该复制语句是无用的，可被删除。

第一种情况， x 是否被有效引用信息可以通过定值-引用链（du 链）信息获得，若 $x = y$ 的定值点的 du 链为空，那么 $x = y$ 可删除。

第二种情况，所有对 x 的引用是否都可直接用 y 替代，这依赖可用复制语句分析获得的信息。将复制语句 $x = y$ 看作是一个特殊的表达式，其运算符为赋值号“=”，运算分量为 x 和 y ，分析其在各程序点的可用性。若在 x 的引用点

p , 复制语句 $x=y$ 是可用的, 此时对 x 的引用可直接用 y 替代。若 $x=y$ 可到达的所有 x 引用点都能进行这种替代, 该复制语句就成为无用赋值, 可被删除。

例 8.31 给定图 8-30(a) 中的简略流图, 假设 $x=y$ 的定值引用点只有 B_2 和 B_3 中的两处。由图可见在 B_2 中 x 的引用点 $x=y$ 是可用的; 在 B_3 中 x 的引用点 $x=y$ 也是可用的。因此这两处引用都可以将 x 替换为 y , 从而使得复制语句 $x=y$ 成为无用赋值, 可被删除。再考虑图 8-30(b) 中的情况。此时, 由于从流图的首节点到达 B_3 中 x 引用点有两条路径, 而路径 $B_4 \rightarrow B_3$ 的路径上, 并没有定值 $x=y$, 因此在 B_3 中 x 引用点 $x=y$ 不可用。因此, B_1 中的 $x=y$ 不能被删除。

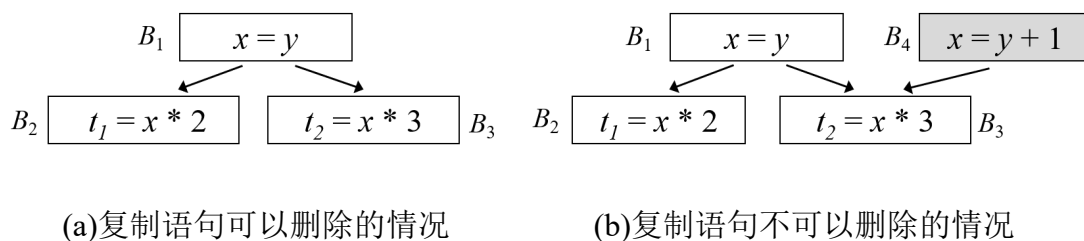


图 8-51 复制语句删除的示例

算法 8.8 删除复制语句的算法

输入: 流图 G 、 du 链、各基本块 B 入口处的可用复制语句集合

输出: 修正后的流图

方法:

对于每个复制语句 $x=y$, 执行下列步骤

- ① 根据 du 链找出该定值所能够到达的那些对 x 的引用;
- ② 确定是否对于每个这样的引用, $x=y$ 都在 $IN[B]$ 中 (B 是包含这个引用的基本块), 并且块 B 内该引用的前面没有 x 或者 y 的定值, 即对于每个引用点, $x=y$ 都是可用的;
- ③ 如果 $x=y$ 满足第②步的条件, 删除 $x=y$, 且把步骤①中找到的对 x 的引用用 y 代替。

8.6.3 代码外提

代码外提的目标是将循环体内的循环不变计算移动到循环体外执行, 从而避免重复计算, 提升程序运行效率。为了安全地外提循环不变计算, 首先需要

为循环构建一个“前置首节点”，可以外提的循环不变计算会被移至这个“前置首节点”。前置首节点的构建方法如图 8-31，具体构建规则如下：

(1) 创建一个新的基本块作为前置首节点，其唯一后继节点是循环的首节点（Header）；

(2) 修改流图中的控制流边：将所有从循环 L 外部指向首节点的边，全部改为指向前置首节点；

(3) 循环内部指向首节点的边（如回边）保持不变。

前置首节点的核心作用是“统一循环入口”，即所有进入循环的控制流都必须经过前置首节点，且前置首节点是循环内所有节点的支配节点，确保外提到此处的语句在每次循环执行前都仅执行一次，不会因控制流分支导致重复或遗漏。

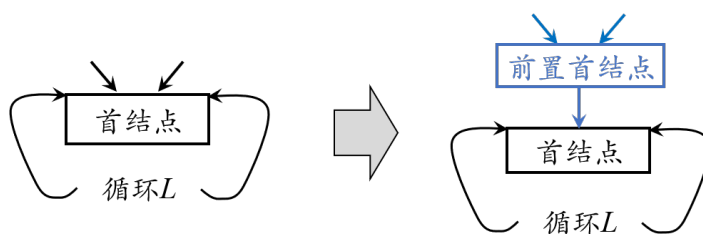


图 8-52 循环前置首节点的构建方法示意图

接下来需要找到循环不变计算。一条语句是循环不变计算，当且仅当其分量都是循环不变的。主要有三种情况：分量是一个常量，或者分量的定值点在循环外部，或者分量的定值点本身是一条循环不变语句。具体的循环不变计算检测算法如算法 8.9。

算法 8.9 循环不变计算检测算法

输入：循环 L，每个三地址指令的 ud 链

输出：L 的循环不变计算语句

方法：

- 1) 将下面这样的语句标记为“不变”：语句的各运算分量或者是常数，或者其所有定值点都在循环 L 外部；
- 2) 重复执行步骤(3)，直到某次没有新的语句可标记为“不变”为止；
- 3) 将下面这样的语句标记为“不变”：先前没有被标记过，且各运算分量或者是常数，或者其所有定值点都在循环 L 外部，或者只有一个到

达定值，该定值是循环中已经被标记为“不变”的语句。

为确保优化后程序逻辑正确，并非所有的循环不变计算都可以外提，循环不变计算语句 s 可以外提至前置首节点需同时满足以下三个条件：

(1) 执行必然性条件。 s 所在的基本块是循环所有出口节点的支配节点。循环出口节点是指有后继节点在循环外的节点。该条件确保每次循环迭代时， s 都会被执行。若 s 所在块不支配所有出口节点，则存在某条循环路径不执行 s ，而外提后语句 s 一定会被执行，那么这样会导致该路径下 s 的计算结果缺失或错误。

(2) 定值唯一性条件。循环内没有其他语句对 s 的左值变量（被赋值变量）进行定值。若循环内存在其他对该变量的定值，外提后会导致变量值被提前固定，无法反映循环内后续定值的影响，破坏程序逻辑。

(3) 引用可达性唯一条件。循环内所有对 s 左值变量的引用，其到达定值均唯一来自 s 。该条件可通过 **ud** 链（引用-定值链）分析验证。若存在某引用的到达定值来自循环外的语句，外提 s 后会覆盖该引用的原始定值来源，导致计算结果错误。

例 8.32 看一个循环不变计算不符合执行必然性条件的例子。如图 8-32(a) 可知， $i = 2$ 是循环不变计算，但所在节点 B_3 并不是出口节点 B_5 的支配节点，因为从首节点 B_1 可不经过 B_3 而到达 B_5 。因此，并不是循环的所有路径上都会执行 $i = 2$ 。此时，若将该语句外提至前置首节点，如图 8-32(b)，因为前置首节点是循环中所有节点的支配节点，进行循环之前一定执行，因此，外提后 $i = 2$ 从可能执行变成一定执行，显然逻辑是不对的。因此，不是循环每次都执行的循环不变计算语句不能外移到前置首节点。

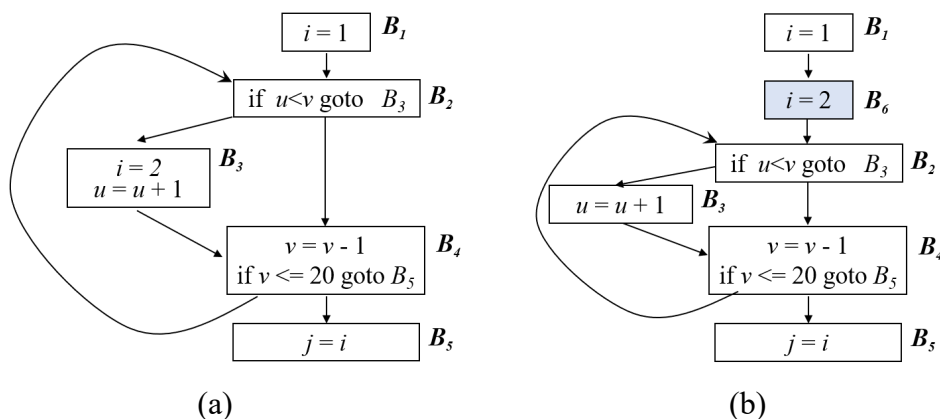


图 8-53 不符合执行必然性条件的情况示例

例 8.33 一个循环不变计算不符合定值唯一性条件的例子。如图 8-33(a)所示的流图中语句 $i = 3$ 是循环不变计算，并且其所在块 B_2 是出口节点 B_5 的支配节点，符合执行必然性条件。然而若将该语句外提，如图 8-33(b)所示， B_5 中对 i 的引用值在外提之前和之后可能会不一致。由于循环内块 B_3 中还有一处 $i = 2$ 的定值，在外提前，块 B_5 中引用的 i 值仅取决于控制进入 B_5 之前的最后一次循环时是否进入了 B_3 ，若进入了 B_3 ， i 为 2，否则 i 为 3。而将 $i = 3$ 外提后， B_5 中引用的 i 值取决于循环执行过程中控制是否曾经进入过 B_3 ，若从未进入过， i 为初始值 3；若曾经进入过哪怕一次， i 值就为 2，之后不会再改变。由此可见，如果循环中还有其他语句对 x 赋值，就不能将循环不变语句外提。

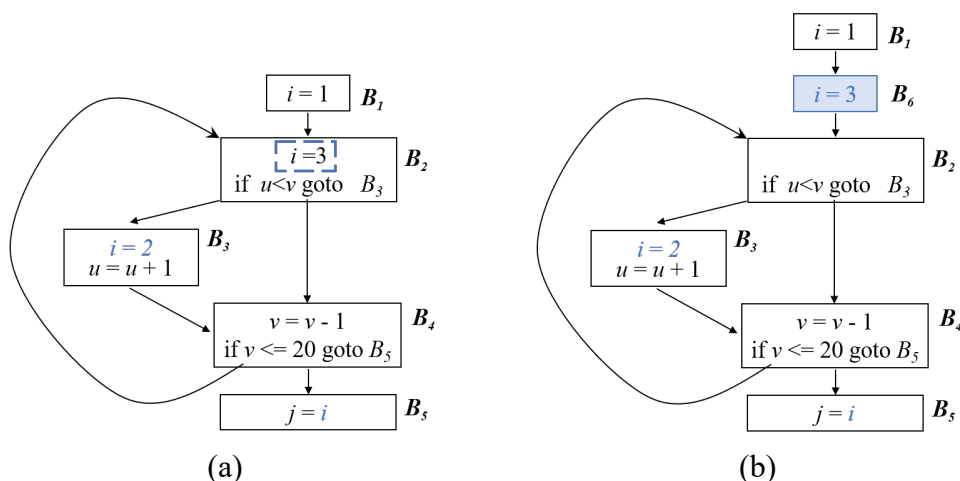


图 8-54 不符合定值唯一性的情况示例

例 8.34 一个循环不变计算不符合引用可达性唯一条件的例子。如图 8-34(a)所示的流图中语句 $i = 2$ 是循环不变计算，且其所在块 B_3 是出口节点 B_5 的支配节点，也是循环内 i 的唯一定值。然而，根据流图可见， B_1 中定值 $i = 1$ 和 B_4 中的定值 $i = 2$ 都能到达循环中 B_3 中 i 的引用。这样一来，在外提前， k 的值有可能等于 1，也可能等于 2，但在外提后，如图 8-34(b)所示， k 的值一定等于 2。因此，若循环内不仅仅 $i = 2$ 可到达 i 的引用， $i = 2$ 不能外提。

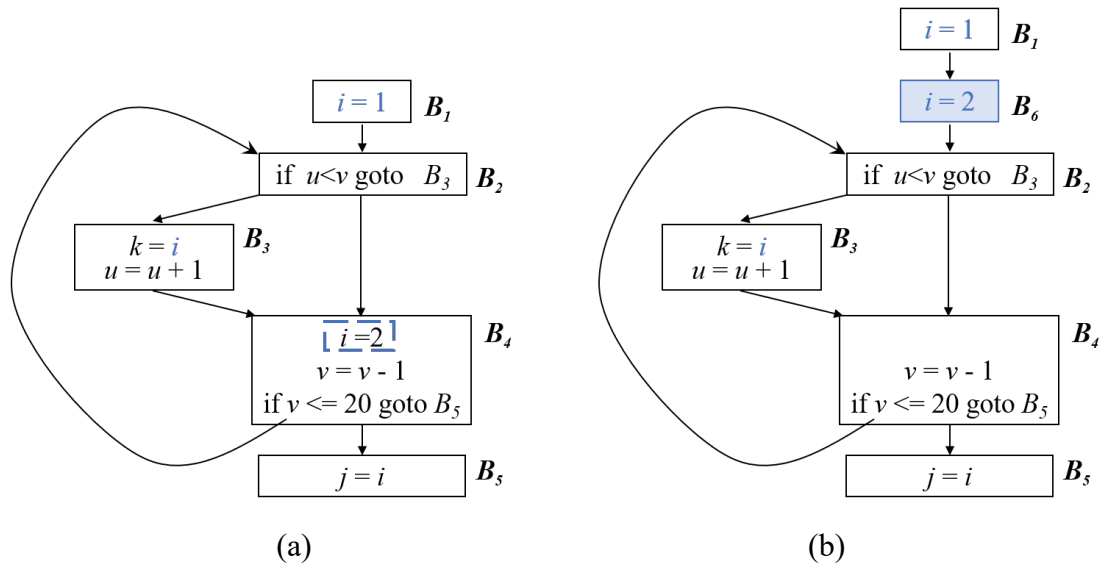


图 8-55 引用可达性唯一条件的情况示例

算法 8.10 代码移动算法

输入：循环 L、ud 链、支配节点信息

输出：修改后的循环

方法：

- 1) 寻找循环不变计算；
- 2) 对于步骤(1)中找到的每个循环不变计算，检查是否满足上文中的三个条件；
- 3) 按照循环不变计算找出的次序，把所找到的满足上述条件的循环不变计算外提到前置首节点中。如果循环不变计算有分量在循环中定值，只有将定值点外提后，该循环不变计算才可以外提。

8.6.4 强度削弱

强度削弱是针对归纳变量的优化技术，其核心目标是将循环内代价高昂的运算替换为代价低廉的运算。归纳变量是指在循环迭代过程中，其值按固定规律递增或递减的变量，如 $i = i + 1$ 、 $j = j + 4$ ，这类变量广泛存在于循环中，是强度削弱的主要优化对象。

如果循环 L 中的变量 i 只有形如 $i = i + n$ 的定值， n 为常量（或循环不变计算），则称 i 为循环 L 的基本归纳变量（basic induction variable）。例如， i

$= i + 1$ 中的 i 即为基本归纳变量。如果循环 L 中的变量 j 只有形如 $j = c \times i + d$ 的定值，其中 i 是基本归纳变量， c 和 d 是常量（或循环不变计算），则 j 也是一个归纳变量，称为派生归纳变量（derived induction variable），或导出归纳变量，并称 j 属于 i 族。基本归纳变量 i 属于它自己的族。

每个归纳变量都关联一个三元组。如果 $j = c \times i + d$ ，其中 i 是基本归纳变量， c 和 d 是常量，则与 j 相关联的三元组是 (i, c, d) 。

强度削弱的本质是利用归纳变量族的线性关系，将归纳变量的乘法运算转化为加法运算。例如，假设基本归纳变量 $i = i + 1$ ，派生归纳变量 $j = 4 * i$ ，当 i 每次递增 1 时， j 每次递增 4，因此可将循环内的 $j = 4 * i$ 替换为循环外初始化 $j = 4 * i$ ，循环内紧跟 $i = i + 1$ 添加 $j = j + 4$ ，从而用加法替换乘法。

算法 8.11 归纳变量检测算法

输入：带有循环不变计算信息和到达定值信息的循环 L

输出：一组归纳变量

方法：

1) 扫描 L 的语句，找出所有基本归纳变量。在此要用到循环不变计算信息。

与每个基本归纳变量 i 相关联的三元组是 $(i, 1, 0)$ ；

2) 寻找 L 中只有一次定值的变量 k ，且具有形式 $k = c' \times j + d'$ ，其中 c' 和 d' 是常量（或循环不变计算）， j 是基本的或派生的归纳变量；

① 如果 j 是基本归纳变量，那么 k 属于 j 族。 k 对应的三元组可以通过其定值语句确定，即 (j, c', d') ；

② 如果 j 是派生归纳变量，假设其属于 i 族，那么 k 也属于 i 族，且 k 的三元组可以通过 j 的三元组 (i, c, d) 和 k 的定值语句来计算，即 k 的三元组为 $(i, c' \times c, c' \times d + d')$ ，此时还要求：

a) 循环 L 中对 j 的唯一定值和对 k 的定值之间没有对 i 的定值；

b) 循环 L 外没有 j 的定值可以到达 k 。

算法的最后两个条件是为了保证 k 与 i 的函数关系是同步调变化，即线性的。

按照算法 8.11 找出所有的归纳变量，就可以进行归纳变量的强度削弱了，具体算法如算法 8.12。

算法 8.12 归纳变量的强度削弱算法

输入：带有到达定值信息和已计算出的归纳变量族的循环 L

输出：修改后的循环

方法：

对于每个基本归纳变量 i ，对 i 族中的每个派生归纳变量 j : (i, c, d)

执行下列步骤:

- 1) 建立新的临时变量 t 。如果变量 j_1 和 j_2 具有相同的三元组，则只为它们建立一个新变量；
- 2) 在前置节点的末尾，添加语句 $t = c * i$ 和 $t = t + d$ ，使得在循环开始的时候 $t = c * i + d$ ；
- 3) 在 L 中紧跟定值 $i = i + n$ 之后，添加 $t = t + c * n$ 。将 t 放入 i 族，其三元组为 (i, c, d) ；
- 4) 用 $j = t$ 代替对 j 的赋值。

例 8.35 给定图 8-35(a)简略流程图， $i = i + 1$ 和 $j = j - 1$ 是基本归纳变量。 $t_2 = 4 * i$ 是 i 族归纳变量，其三元组为 $(i, 4, 0)$ ，即 i 增加 1， t_2 增加 4； $t_4 = 4 * j$ 是 j 族归纳变量，其三元组为 $(j, 4, 0)$ ， j 增加-1， t_2 增加-4。根据强度削弱算法 8.12，首先对于归纳变量 t_2 ，在前置节点末尾，添加语句 $s_2 = 4 * i$ ，在 B_2 定值 $i = i + 1$ 之后，添加语句 $s_2 = s_2 + 4$ ， s_2 是 i 族归纳变量，其三元组为 $(i, 4, 0)$ ，最后用 $t_2 = s_2$ 代替 $t_2 = 4 * i$ 。类似地，对 t_4 做同样的处理，修改后的循环如图 8-35(b)所示。

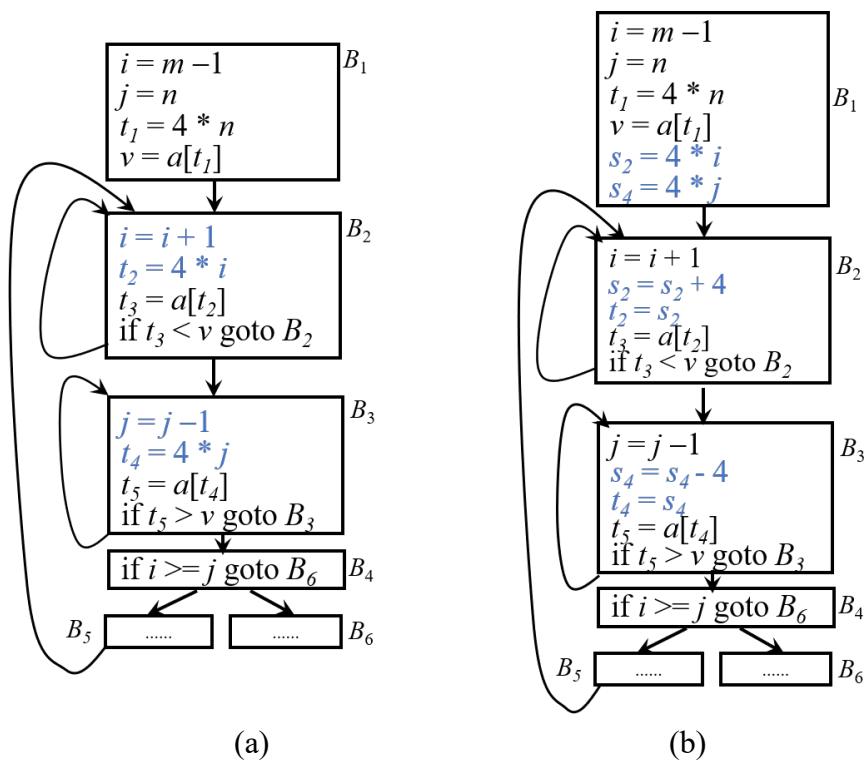


图 8-56 归纳变量强度削弱示例

强度削弱的核心是利用归纳变量族的线性关系，将高价运算转化为低价运算。其关键是准确识别基本归纳变量和派生归纳变量，通过到达定值分析验证线性关系的稳定性，并借助前置首节点完成临时变量的初始化，确保优化正确且高效。

8.6.5 归纳变量的删除

经过强度削弱后，循环内可能会出现“无用的基本归纳变量”，即其唯一用途是计算派生归纳变量，或用于循环终止条件的判断，且这些用途可被派生归纳变量替代。归纳变量的删除就是删除这类无用的基本归纳变量，进一步精简代码，减少变量的存储和操作开销。

基本归纳变量 i 可被删除的核心前提是其所有引用点都可被对应的派生归纳变量替代，且替代后不改变程序的逻辑和计算结果。若 j 是 i 族派生归纳变量，由于 j 与 i 存在线性关系 $j = c * i + d$ ($c \neq 0$)，二者的大小关系具有等价性，即当 $c > 0$ 时， i 增大则 j 增大；当 $c < 0$ 时， i 增大则 j 减小，因此循环终止条件中对 i 的判断可转化为对 j 的判断。

归纳变量删除的两种情况：

(1) 基本归纳变量仅用于派生归纳变量的计算。若基本归纳变量 i 的所有引用都来自派生归纳变量的定值（如 $j = c * i + d$ ），且经过强度削弱后，派生归纳变量的计算已通过临时变量 t 完成（ $j = t$ ），则 i 成为无用变量，可直接删除其所有定值语句和初始化语句。

(2) 基本归纳变量用于循环终止条件的判断。若基本归纳变量 i 既用于派生归纳变量计算，又用于循环终止条件，则需先将终止条件转化为对派生归纳变量 j 的判断，再删除 i 。具体转化规则如下：

转化的核心是利用线性关系的等价性，确保条件判断结果与原始逻辑一致。

若 $j = c * i + d$ ($c > 0$)，则原条件 $i < x$ 可转化为 $j < c * x + d$ ；

若 $j = c * i + d$ ($c < 0$)，则原条件 $i < x$ 可转化为 $j > c * x + d$ 。

若循环内存在两个基本归纳变量 i_1 和 i_2 ，且各自的派生归纳变量 j_1 ($j_1 = c * i_1 + d$) 和 j_2 ($j_2 = c * i_2 + d$) 具有相同的线性系数，则循环终止条件中对 i_1 和 i_2 的比较，如 `if (  $i_1 > i_2$  ) goto ...`，可转化为对 j_1 和 j_2 的比较 `if (  $j_1 > j_2$  ) goto ...`。此时， i_1 和 i_2 的唯一用途是条件判断，转化后可删除这两个基本归纳变量及其相关定值语句。若 j_1 和 j_2 的线性系数不同，则转化需引入额外的运算（如调整系

数)，优化收益可能抵消成本，此时不建议删除。

结合强度削弱后的代码，归纳变量删除的具体步骤如下：

(1) 分析基本归纳变量 i 的所有引用点。明确其用途是派生归纳变量计算、循环终止条件判断，还是其他用途；

(2) 替代引用点。若用于派生归纳变量计算，强度削弱后已通过临时变量 t 替代，无需额外操作；若用于循环终止条件，根据线性关系将条件转化为对派生归纳变量 j 的判断；

(3) 验证替代的正确性。确保所有引用点替代后，循环的迭代次数、变量的计算结果与原始代码一致；

(4) 删除无用基本归纳变量。删除 i 的初始化语句、循环内的定值语句（如 $i = i + n$ ），以及所有未被替代的无用引用。

例 8.36 对于图 8-35(b)中的归纳变量强调削弱后的简略流程图，假设 i 与 j 在 B_4 之后不活跃。由流程图可知，基本归纳变量 i 和 j 仅用于循环终止条件 $i > = j$ 的判断，而且它们具有相同线性关系的派生归纳变量 $s_2(i, 4, 0)$ 和 $s_4(j, 4, 0)$ 。因此，可以将终止条件替换为 $s_2 > = s_4$ 。这样一来， i 和 j 在循环中成为不活跃变量，与它相关的定值均可删除。修改后的流程图如图 8-36 所示。

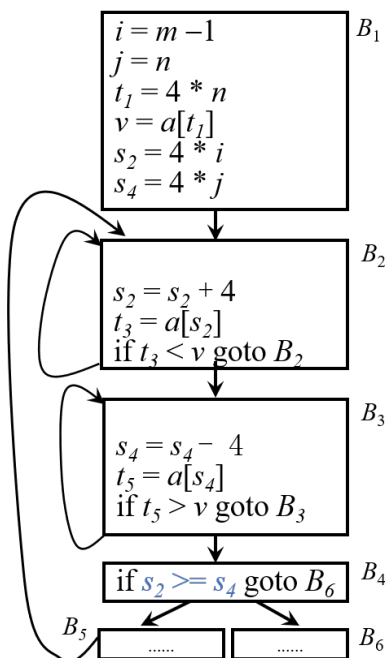


图 8-57 归纳变量删除示例

归纳变量删除是强度削弱的后续优化，核心是利用派生归纳变量替代基本归纳变量的所有引用，尤其是循环终止条件的判断。其关键是确保线性关系的等价性，通过替代引用点、验证正确性，最终删除无用的基本归纳变量，实现代码精简和效率提升。

8.7 本章小节

本章系统阐述了编译器中间代码优化的核心原理与技术体系。首先引入了流图作为程序控制流的抽象表示，为全局分析奠定了基础。局部优化聚焦于基本块内部，重点介绍了基于 DAG 的表示与优化方法，能够高效实现公共子表达式删除、强度削弱和无用代码消除等转换。全局优化则依赖于系统的数据流分析，本章深入讲解了到达定值、活跃变量和可用表达式三种经典分析模式，它们为删除全局公共子表达式、复制传播等优化提供了关键依据。控制流分析通过支配节点与回边识别出程序中的自然循环。基于此，本章探讨了针对循环的关键优化技术，包括循环不变表达式外提，以及归纳变量的强度削弱与删除，这些是提升程序执行效率的核心手段。通过本章学习，读者将建立起代码优化的系统认知，掌握数据流分析与控制流分析的基本方法，理解各类优化技术的原理与作用，为后续的目标代码生成及深入学习编译器后端技术打下坚实基础。

第9章 目标代码生成

在前面的章节，编译器已经完成了词法分析、语法分析、语义分析以及中间代码优化，确保了源程序在逻辑上的正确性与高效性。目标代码生成是编译过程的最后一个阶段，它的任务是将优化后的中间表示映射到具体目标机器的指令集和存储体系上，生成可执行的目标代码。代码生成器的质量直接决定了目标代码的执行速度和空间效率，其设计目标是在保证语义等价的前提下，充分利用目标机器的硬件特性，生成尽可能高效的代码。

然而，为任意给定的源程序生成绝对最优的目标程序，在理论上是一个不可判定问题。这意味着，不存在一个算法能够对所有程序都找到最优解。不仅如此，代码生成中的许多关键子问题，例如寄存器分配和指令排序，都被证明是 NP 难问题，具有难以处理的计算复杂度。这意味着在大型程序上寻求这些问题的最优解，需要消耗无法承受的计算时间。

面对这些理论上的局限，经过数十年的研究与发展，业界已经形成了非常成熟且高效的启发性技术与近似算法。这些方法虽然在最坏情况下无法保证找到最优解，但在绝大多数实际应用场景中能够生成质量非常高、接近最优的代码。

代码生成器的输入是前端产生的中间表示，其形式多样，包括但不限于线性表示，如三地址代码、字节码；树形结构，如抽象语法树；图结构，如表示基本块的有向无环图。其输出是目标程序，具体形式可以是能够在目标硬件上直接执行的绝对机器代码，或者是需要汇编器进一步处理的汇编程序。后者因为更易于人类阅读和调试，在教学和开发环境中尤为常见。本章以三地址码作为输入，以生成汇编程序为输出目标。

9.1 代码生成器的主要任务

代码生成器的目标是将语义正确的中间表示高效地映射到目标机器的指令集和存储体系上。这一过程并非简单的逐句翻译，而是一个涉及多重决策的复杂优化过程。其主要任务可以概括为三个关键方面，指令选择、寄存器分配与指派对以及指令排序。

9.1.1 指令选择

指令选择的任务是为中间表示中的每个操作或操作序列，选择适当的目标机器指令或指令序列来实现其功能。

指令选择过程的首要准则是严格保证语义一致性，即所选指令序列的执行效果必须与中间表示所定义的语义完全等价。在此前提下，目标机器指令集本身的特性对指令选择的难易程度与生成代码的质量有着决定性影响。其中，指令集的完备性与规整性是两个关键因素。

若目标机器指令集具备较高的完备性，意味着其为常用计算模式提供了丰富且直接的支持。这大大简化了指令选择的过程，编译器能够为绝大多数中间表示语句方便地找到一条或多条语义完全匹配的指令序列。反之，若指令集不完备，编译器则不得不通过多条基础指令来模拟实现某些复杂操作，这不仅增加了代码长度，也影响了执行效率。

此外，指令集的规整性也至关重要。规整的指令集要求性质相似的操作其指令格式与寻址模式也保持一致。这种一致性使得指令选择算法可以遵循统一的模式进行处理，降低了选择的复杂性，并有利于生成更优化、更紧凑的代码。反之，不规整的指令集会引入大量特殊情况和复杂规则，极大地增加了生成高质量代码的难度。

若目标机器指令集具备较高的完备性，而且不考虑目标代码的效率，那么指令选择是简单的，对于每一种三地址语句，可以设计一个代码框架，代码生成器只要按照预定义的框架进行指令选择即可。

例 9.1 对于一般的三地址运算语句 $x = y \text{ op } z$ ，给出它对应的目标代码框架为：

```
LD    R0, y          /* 把 y 的值加载到寄存器 R0 中 */
ADD   R0, R0, z       /* z 加到 R0 上 */
ST    x, R0          /* 把 R0 的值保存到 x 中 */
```

然而，必须考虑所生成目标代码的质量。目标代码的质量，体现为更快的执行速度与更紧凑的代码长度。对于具有丰富指令集的目标机器而言，实现同一中间表示操作往往存在多种候选指令序列，而这些不同实现方式在时钟周期消耗、寄存器使用以及指令长度上的开销可能差异显著。因此，若仅对中间代码进行简单、机械的逐句翻译，而缺乏深入的分析与优化，极有可能错失更高效的实现方案，从而导致最终生成的目标代码效率低下。

例 9.2 对于如下的三地址语句序列：

$$a = b + c$$

$$d = a + e$$

按照预定义的代码框架生成目标代码如下：

```
(1) LD    R0, b           // R0 = b
(2) ADD R0, R0, c         // R0 = R0 + c
(3) ST  a, R0            // a = R0
(4) LD  R0, a            // R0 = a
(5) ADD R0, R0, e         // R0 = R0 + e
(6) ST  d, R0 // d = R0
```

经过简单的分析可以发现，以上的代码序列中存在一个典型的“存储/加载”冗余对，在第(3)条指令 `ST a, R0` 中，将寄存器 `R0` 中计算得到的 $b + c$ 的结果存入内存变量 `a` 的地址。紧接着在第(4)条指令 `LD R0, a` 中，又立刻将刚刚存入内存的 `a` 的值重新加载回同一个寄存器 `R0`。这两条指令（存与取）执行了全量的内存读写操作，不仅增加了代码长度，还引入了两次本可完全避免的高延迟内存访问，严重拖慢了程序的执行速度。可见，只使用简单的代码框架生成的代码效率很低。

另外，为了提升目标代码效率，在指令选择时，还应该考虑特殊机器指令的使用和指令速度等因素。例如，如果目标机器有加 1 指令，则 $a = a + 1$ 的最有效实现是：`INC a`，而不是直接套用代码模板。

9.1.2 寄存器分配

代码生成需要解决的另一个关键问题，是如何高效地使用目标机器的寄存器资源。寄存器是目标机中运行速度最快的存储单元，其访问延迟远低于内存。因此，使用寄存器作为运算分量的指令通常比涉及内存操作数的指令更短、更快，能够显著提升代码的执行效率。然而，由于硬件成本限制，寄存器的数量通常非常有限，无法容纳程序中所有的值。那些未被分配至寄存器的值必须存放于内存中，从而在访问时带来额外的开销。因此，在代码生成阶段，必须对寄存器进行有效管理。

这项工作通常可进一步划分为两个子任务：

寄存器分配：确定在程序的各个执行点上，哪些变量或临时值应当驻留在寄存器中。其核心是在有限的寄存器数目下，尽可能让使用频率高、生命周期重叠少的变量占用寄存器，从而减少对内存的访问。

寄存器指派：在确定哪些值需要放入寄存器之后，进一步为它们分配合适的寄存器。指派过程需考虑目标机体系结构的约束，例如某些指令只能使

用特定类别的寄存器，或是操作系统及调用约定对某些寄存器的保护要求。

从计算复杂性角度看，确定最优的寄存器指派方案是一个 NP 完全问题。如果再叠加实际硬件架构和操作系统对寄存器使用的各类约束，例如专用寄存器、寄存器配对限制、调用者/被调用者保存约定等，这一问题的求解将更为复杂。因此，实际的编译器中往往采用启发式策略或近似算法，在合理时间内获得较优的寄存器使用方案。有关寄存器的分配策略将在小节 9.4 进行更详细讨论。

9.1.3 指令排序

代码生成的第三项关键任务是指令排序，即确定目标指令的执行顺序。指令的排列顺序会直接影响目标代码的执行效率，其原因主要在于两个方面：首先，合理的顺序可以更好地适配现代处理器的流水线与多发射等特性，减少指令间的相互等待，提高指令级并行度；其次，不同的计算顺序会导致中间结果的生命周期与依赖关系发生变化，某些顺序能够显著减少对存放中间结果的寄存器的需求，从而降低因寄存器不足而引发的访存开销。

然而，在优化指令顺序时，我们面临着严峻的计算复杂性挑战。与指令选择和寄存器分配问题类似，从所有可能的顺序中寻找最优解，通常是一个 NP 完全问题。鉴于这一复杂性，实际的编译器往往采用各种启发式算法来寻找在可接受时间内的高质量解。

为了聚焦于代码生成的核心概念与基础流程，本教材在后续的讨论中将对指令排序问题予以简化，主要介绍如何按照三地址码语句既有的顺序来生成目标代码。这种顺序生成方式算法简单直观，有助于读者清晰地理解从中间表示到目标代码的映射关系，为后续学习更复杂的优化方法奠定基础。

9.2 一个简单的目标机模型

设计一个高质量的代码生成器，必须建立在对目标机器的体系结构和指令集特性的深刻理解之上。然而，实际的处理器架构千差万别，若在一般性讨论中陷入具体机器的细节，将不利于揭示代码生成的核心原理。为此，本章将定义一个简化的目标机模型作为讨论的基础。该模型以现代的精简指令集计算机（RISC）为核心范式，同时吸纳了少量复杂指令集计算机（CISC）风格的寻址方式，以兼顾概念的纯粹性与实际问题的覆盖面。我们将使用汇编代码作为此模型的目标代码表示形式。

我们所采用的目标机模型具有以下主要特征：

指令集架构：机器提供加载（Load）、存储（Store）、算术运算（如 ADD, SUB 等）以及跳转（Jump）和条件跳转等基本操作。我们假设所有运算分量均为整数类型。为简化讨论，我们仅使用一个有限的指令集合，但其足以表达完整的计算过程。

内存系统：内存按字节进行编址，允许通过地址直接访问数据。这是现代计算机系统普遍采用的内存组织方式。

寄存器组织：机器配备有 N 个通用寄存器，记为 $R0, R1, \dots, RN-1$ 。这些寄存器可供所有指令使用，是进行算术运算和数据暂存的高速存储单元。

指令格式：模型的指令格式力求简洁。每条指令由一个运算符和紧随其后的地址字段构成。通常，一条指令最多包含一个目标地址和两个源运算分量地址。这种格式与三地址码有着直接的对应关系，便于代码生成算法的设计与理解。

标号：指令之前可以附加一个标号，用于标记该指令在代码段中的位置，作为控制流转移指令（如跳转）的目标地址。

此模型抽象掉了目标机高级硬件特性，使我们能够聚焦于代码生成中的三个核心任务。后续各节将基于此模型展开详细的算法讨论。

9.2.1.1 目标机器的主要指令

基于前述的目标机模型，本小节将详细说明其支持的核心指令集。这些指令是代码生成器输出的基本单位，理解其格式与功能是设计代码生成算法的前提。

我们采用统一的指令基本格式：

运算符 目标地址, 运算分量 1, 运算分量 2

多数运算指令遵循此三地址格式，但部分指令（如存储、跳转）因语义特殊而有所变化。

（1）加载指令 LD

此类指令负责在寄存器与内存之间，或寄存器与寄存器之间移动数据。格式如下：

LD dst, addr

加载指令从源位置 `addr` 加载数据到目标位置 `dst`。最常见的用法如下：

“LD r, x ”这是加载指令的典型用法，将内存变量 x 的值加载到寄存器 r 中；“LD r_1, r_2 ”此指令将寄存器 r_2 中的值传送到寄存器 r_1 中。

(2) 保存指令 ST

“ST x, r ”将寄存器 r 中的值保存到内存地址 x 中。这是唯一以内存地址作为目标的操作。

(3) 运算指令

此类指令对操作数执行算术运算。形式有如下两种：

$$\text{OP } dst, src_1, src_2$$

$$\text{OP } dst, src$$

其中，OP 执行特定的算术运算（如 ADD, SUB, MUL 等），将源位置 src_1 和 src_2 进行运算，结果存入目标位置 dst 。单目运算指令只包含一个源位置。为了计算效率，我们会将运算分量加载到寄存器，再进行 OP 运算。

例如：指令“SUB R1, R1, R2”表示运算 $R1 = R1 - R2$ 。

(4) 控制流指令

此类指令用于改变程序的执行顺序。

BR L：无条件地跳转至标号为 L 的指令处继续执行。

Bcond r, L ：根据寄存器 r 的值与条件 cond 是否满足，决定是否跳转到标号 L。

例如：指令“BLTZ r, L ”表示 “if ($r < 0$) jump to L”。其他常见条件包括 BGTZ（大于零）、BEQZ（等于零）等。

9.2.1.2 目标机器的寻址模式

寻址模式定义了指令中操作数的解释与定位方式。熟悉多样的寻址模式对于生成高效代码至关重要，因为它直接影响着指令的数量以及内存访问的开销。我们的目标机模型支持以下几种核心寻址模式：

(1) 直接寻址

操作数在指令中直接以变量名（即其内存地址）给出。

格式为： x ，其中 x 为一个变量名表示内存地址。

例：LD R1, x

表示将内存地址 x 处存放的值加载到寄存器 R1 中。即 $R1 = \text{Contents}(x)$ 。

(2) 基址+偏移寻址

有效地址由一个变量（基地址）与一个寄存器中的值（偏移量）相加得到。此模式对数组和结构体访问极为高效。

格式为： $a(r)$ ，其中 a 是变量名， r 是寄存器。表示操作数的有效地址为 $a + \text{Contents}(r)$ 。

例：LD R1, $a(R2)$

若变量 a 是数组的基地址，寄存器 $R2$ 存放元素偏移量，则该指令将地址 $a + \text{Contents}(R2)$ 处的值加载到 $R1$ 。即 $R1 = \text{Contents}(a + \text{Contents}(R2))$ 。

(3) 寄存器间接偏移寻址

有效地址由一个寄存器中的值（作为基地址）与一个整数常量（偏移量）相加得到。此模式主要用于通过指针访问局部变量。

格式为： $c(r)$ ，其中 c 为整数常量， r 为寄存器。表示操作数的有效地址为 $\text{Contents}(r) + c$ 。

例：LD $R1, 100(R2)$

假设 $R2$ 存放一个指针，则该指令将该指针向后偏移 100 个字节的内存地址处的值加载到 $R1$ 。即 $R1 = \text{Contents}(\text{Contents}(R2) + 100)$ 。

(4) 间接寻址

寄存器中存放的值不是一个直接的操作数，而是另一个操作数的地址。这实现了一级指针解引用。

格式为： $*r$ ，其中 r 为寄存器。表示操作数的有效地址为 $\text{Contents}(\text{Contents}(r))$ 。

例：LD $R1, *R2$

表示将 $R2$ 中的值视为一个内存地址，并将该地址处存放的值加载到 $R1$ 。即 $R1 = \text{Contents}(\text{Contents}(\text{Contents}(R2)))$ 。

(5) 带偏移的间接寻址

此模式是寄存器间接偏移寻址与间接寻址的结合，用于处理指针数组。

格式为： $*c(r)$ ，其中 c 为整数常量， r 为寄存器，表示首先计算中间地址 $\text{Addr_inter} = \text{Contents}(r) + c$ ，然后操作数的有效地址为 $\text{Contents}(\text{Addr_inter})$ 。

例：LD $R1, *100(R2)$

表示首先计算 $R2$ 中的地址加 100，得到一个中间地址，然后将该中间地址处存放的值（一个地址）所指向的内存内容加载到 $R1$ 。即 $R1 = \text{Contents}(\text{Contents}(\text{Contents}(R2) + 100))$ 。

(6) 立即数寻址

操作数本身就是一个常量，直接包含在指令中，无需内存访问。

格式为： $\#c$ ，其中 c 为整数常量。表示操作数即为常量 c 。

例：LD $R1, \#100$

9.2.1.3 程序和指令的代价

一个程序编译及运行代价是评价程序质量的重要指标，常用的度量包括编译时间、以及目标程序的大小、运行时间和能耗。然而，准确地度量一个程序

编译和运行的实际代价是不容易的问题。为了在算法设计和比较中量化“优良”的程度，我们需要一个可计算的代价模型。一个广泛使用的简化模型是为每条目标机器指令赋予一个固定的代价。为简单起见，我们把一个指令的代价设定为 1 再加上与运算分量寻址模式相关的附加代价 N 。这样指令代价大致对应于指令字的长度。寻址模式附加代价 N 的计算规则为使用寄存器作为运算分量，附加代价为 0。使用内存地址或立即数作为运算分量，附加代价为 1。

根据上述规则，我们对示例指令进行代价分析：

指令 LD R1, R2

该指令的两个运算分量 R1 和 R2 均为寄存器寻址模式，附加代价均为 0。因此，总代价为 1 (指令本身) + 0 (目标地址) + 0 (源运算分量) = 1。

指令 LD R1, x

目标地址 R1 为寄存器（附加代价 0），源运算分量 x 为内存地址（附加代价 1）。因此，总代价为 1 + 0 + 1 = 2。

指令 LD R1, *100(R2)

目标地址 R1 为寄存器（附加代价 0）。源运算分量 *100(R2) 是涉及内存（常数偏移 100 和间接寻址*）的复合寻址模式，根据规则，包含常量 100，其附加代价为 1。因此，总代价为 1 + 0 + 1 = 2。

最后，我们将指令的代价扩展到整个程序。一个目标语言程序 P 在某个特定输入上的运行代价，定义为程序 P 在该输入下所执行的所有指令的代价之和。优秀的代码生成算法的最终目标，正是在于通过精心的指令选择、寄存器分配与指令排序，使得程序在典型输入集合上的期望运行代价最小化。

9.3 指令选择

9.3.1 简单三地址语句的代码框架

若不考虑目标代码的效率，也不考虑目标机寄存器选择问题，指令选择最简单的策略就是为每个中间代码指令设计固定的目标代码框架。下面给出常用的三地址语句对应的目标代码框架。

(1) 运算语句的目标代码

三地址语句： $x = y - z$

目标代码：

LD R1, y // R1 = y

```
LD    R2 , z           // R2 = z
SUB   R1 , R1 , R2      // R1 = R1 - R2
ST    x , R1           // x = R1
```

(2) 数组元素寻址语句的目标代码

三地址语句: $b = a[i]$

注意, 此处 i 为数组元素下标。中间代码可以是不同层次的, 当 i 为数组元素下标时, 对应的中间代码的粒度更大, 生成目标代码的代价更大一些, 即需要生成更多的目标代码来完成元素地址的计算。此处假设 a 是一个实数数组, 每个实数占 8 个字节。

目标代码:

```
LD    R1 , i           // R1 = i
MUL   R1 , R1 , 8       // R1 = R1 * 8
LD    R2 , a(R1)        // R2 = contents ( a + contents(R1) )
ST    b , R2           // b = R2
```

三地址语句: $a[j] = c$

目标代码:

```
LD    R1 , c           // R1 = c
LD    R2 , j           // R2 = j
MUL   R2 , R2 , 8       // R2 = R2 * 8
ST    a(R2) , R1        // contents( a + contents(R2) ) = R1
```

(3) 指针存取语句的目标代码

三地址语句: $x = *p$

目标代码:

```
LD    R1 , p           // R1 = p
LD    R2 , 0(R1)        // R2 = contents ( 0 + contents (R1) )
ST    x , R2           // x = R2
```

三地址语句: $*p = y$

目标代码:

```
LD    R1 , p           // R1 = p
LD    R2 , y           // R2 = y
ST    0(R1) , R2        // contents ( 0 + contents ( R1 ) ) = R2
```

(4) 条件跳转语句的目标代码

三地址语句: $\text{if } x < y \text{ goto L}$

目标代码:

```
LD      R1 , x          // R1 = x
LD      R2 , y          // R2 = y
SUB     R1 , R1 , R2     // R1=R1 - R2
BLTZ    R1 , M          // if R1 < 0 jump to M
```

其中, M 是标号为 L 的三地址指令所产生的目标代码中的第一个指令的标号。

9.3.2 过程调用和返回的目标代码

过程调用和返回是程序执行中的核心操作, 涉及参数传递、控制转移、返回地址管理以及局部存储分配。目标代码生成需要为这些操作设计高效的指令序列, 以确保程序正确执行并优化性能。本节将分别讨论在静态内存分配和栈式内存分配方式下, 过程调用和返回的目标代码生成, 重点阐述指令选择策略和代码框架。

为了关注重点并简化讨论, 本节做如下的假设:

- (1) 按照 9.2.1.3 计算指令代价, 即指令的长度/代价= 1+指令中包含的常数数量+指令中包含的内存地址的数量(字);
- (2) 每个字是 4 字节;
- (3) 栈区从低地址向高地址增长;
- (4) SP 指向活动记录的起始位置;
- (5) 返回地址保存在活动记录的开始处。

9.3.2.1 使用静态内存分配

在静态内存分配方案中, 编译器在编译期间即为每个过程分配了固定不变的内存区域(静态区)作为其活动记录。这种方案实现简单。

(1) 调用者任务

当调用者需要调用一个过程时, 它主要承担两项任务: 保存好返回地址, 以便被调用过程执行完毕后能正确返回; 然后将控制权转移给被调用过程的代码。

使用如下的三地址中间代码来表示过程调用:

三地址语句: call callee

目标代码:

```
ST callee.staticArea, #here + 20
```

BR callee.codeArea

下面对这两条目标指令进行详细阐述：

第一条指令“ST callee.staticArea, #here + 20”是一条存储指令，核心作用是将返回地址 #here+20 保存在被调用者活动记录的开始位置 callee.staticArea。操作数 callee.staticArea 是一个在编译时确定的符号地址，代表了被调用者 callee 的活动记录在静态区中的起始地址。根据我们的假设，返回地址就存储在此处。操作数 #here + 20 是一个立即数，计算得到的是调用者的返回地址。#here 是一个伪代码，指代当前 ST 指令本身的存储地址。根据本章的指令代价模型，ST 指令（包含一个操作码、一个常数 #here+20、一个内存地址 callee.staticArea）的长度为 3 个字。BR 指令（包含一个操作码、一个代码地址 callee.codeArea）的长度为 2 个字。因此，从 ST 指令开始，到调用后需要执行的下一条指令之间，总共间隔了 $3 + 2 = 5$ 个字，即 $5 * 4 = 20$ 字节。所以，返回地址就是 #here + 20。

第二条指令“BR callee.codeArea”是一条无条件跳转指令。操作数 callee.codeArea 是一个在编译时确定的符号地址，代表了被调用者 callee 的目标代码在代码区中的起始地址。在代码生成阶段，这个符号地址会被替换为一个具体的常数地址。该指令执行后，处理器的控制流便从调用者跳转至被调用者 callee 的代码开始执行。

（2）被调用者任务

被调用过程执行完毕后的任务非常明确：将控制权交还给调用者。

使用如下的三地址中间代码来表示过程返回：

三地址语句：return

目标代码：

BR *callee.staticArea

这是一条间接无条件跳转指令。* 表示其操作数 callee.staticArea 是一个内存地址，该地址中存储的内容才是真正的跳转目标。操作数 callee.staticArea 正是调用者先前保存返回地址的那个内存单元——被调用者活动记录的起始地址。该指令的执行过程是处理器首先从内存地址 callee.staticArea 中取出之前保存的返回地址值（即 #here+20），然后跳转到该地址继续执行。通过这条指令，控制流成功地、准确地从被调用者 callee 返回到了调用者 call 语句之后下一条指令。

在静态内存分配下，过程调用与返回的协作依赖于一个固定的内存位置（callee.staticArea）来传递返回地址。调用者负责“写”入返回地址，被调用

者负责“读”出并跳回。整个机制清晰、高效，但受限于活动记录的静态性。

例 9.3 为了更直观地理解静态内存分配方式下过程调用与返回的目标代码生成机制，通过一个具体的实例进行分析。假设在目标机模型中，调用者过程 *c* 的代码起始地址为 100；被调用者过程 *p* 的代码起始地址为 200；过程 *p* 的活动记录在静态区中的起始地址为 364；每个 ACTION 指令序列占用 20 字节（即 5 个字）。

给定的三地址代码如下：

```
// 调用者 c (代码起始地址: 100)
action1    // 第一个动作序列
call p     // 调用过程 p
action2    // 调用返回后的动作序列
halt      // 程序结束
```

// 被调用者 p (代码起始地址: 200)

```
action3    // 过程 p 的动作序列
return     // 返回到调用者
```

根据上述三地址代码和假设条件，生成的目标代码如下：

调用者 *c* 的代码段：

```
100: ACTION1           // 动作序列 1，占用 20 字节（100-119）
120: ST 364, #140      // 保存返回地址 140 到 p 的活动记录
132: BR 200            // 跳转到 p 的代码入口地址 200
140: ACTION2           // 动作序列 2，占用 20 字节（140-159）
160: HALT              // 程序结束
```

被调用者 *p* 的代码段：

```
200: ACTION3           // 动作序列 3，占用 20 字节（200-219）
220: BR *364           // 间接跳转，返回到调用者
```

静态数据区：

```
364: 140              // 存储返回地址
```

9.3.2.2 使用栈式内存分配方式

在静态分配策略中，过程调用和返回的目标代码相对简单，因为所有活动记录的地址在编译时即可确定。然而，栈式分配需要运行时维护一个控制栈来管理过程的活动记录，从而实现过程的嵌套调用和返回。本节将详细阐述栈式分配策略下过程调用和返回的目标代码生成，重点介绍调用序列和返回序列的

设计，以及栈指针（SP）的维护机制。

过程调用的目标代码：调用序列

当调用者（Caller）需要调用一个过程（Callee）时，它主要承担以下任务：

- （1）保存返回地址：确保被调用过程执行完毕后能正确返回到调用点。
- （2）转移控制权：跳转到被调用过程的代码入口。
- （3）维护栈指针：分配新的活动记录空间，并更新 SP。

这些任务通过调用序列（Call Sequence）实现。假设调用者的活动记录大小为 `caller.recordsize`，被调用过程的代码入口为 `callee.codeArea`，则对应的三地址语句 `call callee` 的目标代码如下：

```
ADD SP, SP, #caller.recordsize    //分配新活动记录：SP 增加，
                                   //指向被调用者活动记录的起始位置
ST 0(SP), #here + 16             //保存返回地址：将返回地址
                                   //存储在新活动记录的首单元
BR callee.codeArea               //跳转到被调用过程的代码入口
```

这里，ADD 指令通过增加 SP 的值来分配空间，相当于在栈顶开辟新活动记录。ST 指令将返回地址保存到新活动记录的首单元（地址 `0(SP)`），其中 `#here + 16` 表示返回地址为当前指令地址加上偏移量（例如，16 字节，以覆盖后续指令）。最后，BR 指令实现控制权转移。

值得注意的是，调用序列中 SP 的调整由调用者完成，因为调用者知道自身活动记录的大小（`caller.recordsize`），从而能准确计算新活动记录的起始位置。

过程返回的目标代码：返回序列

当被调用过程执行完毕时，通过返回序列（Return Sequence）实现控制权交还和栈空间回收。返回序列包括两个部分：

- （1）被调用过程的任务：跳转到返回地址，恢复控制权。
- （2）调用过程的任务：回收活动记录空间，恢复栈指针 SP。

对应的三地址语句 `return` 的目标代码如下：

```
(1) 被调用过程部分
BR *0(SP) //间接跳转到返回地址：从当前活动记录首单元读取返回地址

(2) 调用过程部分
SUB SP, SP, #caller.recordsize //恢复 SP：减少 SP 值，
                                //回收被调用者的活动记录
```

在返回序列中，被调用过程通过 `BR *0(SP)` 指令从当前活动记录中取出返

回地址并跳转，这确保了控制流正确返回到调用点。随后，调用过程执行 SUB 指令，将 SP 减少 caller.recordsize，从而回收被调用者的活动记录空间，使 SP 指向调用者活动记录的起始位置。为什么 SP 的恢复由调用者完成？这是因为活动记录的大小信息（如 caller.recordsize）仅在调用者端已知。

例 9.4 为了更直观地理解上述代码序列，考虑以下示例。假设调用者过程 m 和被调用者过程 q 的代码分别从地址 100 和 300 开始，m 的活动记录大小为常量 MSIZE，控制栈起始地址为 600。ACTION 指令代表一般操作，占用 20 字节。三地址代码结构如下：

```
//调用者 m
action1
call q
action2
halt
//被调用者 q
action3
return
```

采用栈式存储分配时，对应的目标代码结构如下：

调用者 m 的目标代码：

```
100: LD SP, #600           //初始化 SP 指向栈起始地址
108: ACTION1               //执行操作 1
128: ADD SP, SP, #MSIZE    //调用序列：分配新活动记录
136: ST 0(SP), #152        //保存返回地址（152）
144: BR 300               //跳转到 q 的代码入口（300）
152: SUB SP, SP, #MSIZE    //返回序列：恢复 SP
160: ACTION2              //执行操作 2
180: HALT                 //停机
```

被调用者 q 的目标代码：

```
300: ACTION3              //执行操作 3
320: BR *0(SP)            //返回序列：跳转到返回地址
```

代码执行过程如下：

- （1）过程 m 初始化 SP 为 600，执行 ACTION1。
- （2）调用过程 q 时，ADD 指令将 SP 增加 MSIZE，指向新活动记录起始位置（例如， $600 + MSIZE$ ）。

- (3) ST 指令将返回地址 152 保存到新活动记录的首单元（地址 0(SP)）。
- (4) BR 指令跳转到地址 300，执行 q 的代码（ACTION3）。
- (5) 过程 q 返回时，BR *0(SP)从活动记录中取出地址 152 并跳转。
- (6) 控制返回到 m 后，SUB 指令将 SP 减少 MSIZE，恢复为 600，然后执行 ACTION2 和 HALT。

这个示例清晰地展示了栈式分配下调用和返回的动态过程：SP 的移动实现了活动记录的分配和回收，而返回地址的保存和跳转确保了控制流的正确性。

栈式内存分配策略通过运行时维护控制栈，支持了过程的递归和嵌套调用。调用序列和返回序列的目标代码需要精心设计，以处理返回地址的保存、控制权的转移以及栈指针的维护。关键点在于调用者负责分配新活动记录 and 保存返回地址，因为它知道自身记录大小；被调用者负责通过返回地址恢复控制权，而调用者负责回收空间。这种分工提高了代码的可维护性和效率，是编译原理中代码生成的重要基础。

9.4 寄存器选择

在目标代码生成的实践中，基本块作为程序执行流程中的线性代码序列，其代码生成具有特殊的重要性。基本块内部不含分支指令，这为高效的寄存器分配提供了有利条件。

9.4.1 基本块代码生成算法

在大多数现代处理器体系结构中，算术和逻辑运算指令要求操作数必须存放在寄存器中，运算结果也通常存放在寄存器中。这就引出了代码生成过程中的两个关键问题：

操作数加载：如何将运算分量从内存加载到合适的寄存器？

结果存储：何时将寄存器中的计算结果保存回内存？

简单的策略是在每次需要时加载操作数，在每次计算后立即存储结果。然而，这种策略会产生大量冗余的内存访问指令，严重降低代码执行效率。为了生成高效的目标代码，我们需要更智能的策略，下面给出考虑寄存器分配的基本块代码生成算法。

算法 9.1 基本块代码生成算法 1

输入：基本块三地址语句序列

输出：目标代码序列

1. for 基本块中的每条三地址语句 $I: x = y \text{ op } z$, 执行如下动作
 2. 根据 I 的类型选择目标代码模板
 3. 调用函数 `getReg(I)` 为 x 、 y 、 z 选择寄存器,
分别记为 R_x 、 R_y 、 R_z
 4. if R_y 中存放的不是 y 的当前值, 生成加载指令 `LD  $R_y, y'$` ,
其中 y' 是存放 y 的内存位置之一
 5. if R_z 中存放的不是 z , 生成加载指令 `LD  $R_z, z'$`
 6. 生成目标指令 `OP  $R_x, R_y, R_z$`
 7. end for
 8. for 每个在基本块的出口处可能活跃的变量 x , //基本块的收尾处理
 9. if 它的最新值没有存放在 x 的内存位置上,
 10. 生成指令 “`ST  $x, R$` ” // R 是存放 x 值的寄存器
 11. end for
- 其中, 函数 `getReg(I)` 接收一条三地址指令 I (例如 $x = y + z$), 并返回一个 (或一组) 用于执行该指令的寄存器。

例 9.5 考虑以下基本块的三地址代码:

$t_1 = a + b$

$t_2 = t_1 * c$

$d = t_2 + a$

假设初始状态下所有变量值都在内存中, 寄存器 R_1 、 R_2 可用。

代码生成过程:

// 处理 $t_1 = a + b$

`LD  $R_1, a$` // 加载 a 到 R_1

`LD  $R_2, b$` // 加载 b 到 R_2

`ADD  $R_1, R_1, R_2$` // $t_1 = a + b$, 结果在 R_1

// 处理 $t_2 = t_1 * c$

// t_1 已在 R_1 中, 只需加载 c

`LD  $R_2, c$` // 加载 c 到 R_2

`MUL  $R_1, R_1, R_2$` // $t_2 = t_1 * c$, 结果在 R_1

// 处理 $d = t_2 + a$

// t_2 已在 R_1 中, a 可能在内存中

```
LD R2, a      // 重新加载  $a$  到 R2 (假设  $a$  的值已被覆盖)
ADD R1, R1, R2 //  $d = t_2 + a$ , 结果在 R1

// 基本块收尾处理
// 假设  $d$  在出口活跃且需要保存回内存
ST d, R1      // 将  $d$  的值保存回内存
```

9.4.2 寄存器描述符和地址描述符

上述的基本块代码生成算法需通过管理和维护两个关键的数据结构即寄存器描述符和地址描述符，以便动态跟踪变量的存储位置，实现高效的寄存器分配与管理。

(1) 寄存器描述符 (Register Descriptor)

寄存器描述符记录每个寄存器当前存放的变量值。初始状态下，所有寄存器均为空，一个寄存器可以存放多个变量的值，此时这些变量具有相同值。

(2) 地址描述符 (Address Descriptor)

地址描述符记录每个变量的当前值所在的一个或多个位置。位置可能包括：寄存器、内存地址、栈单元或其组合。通常保存在变量名对应的符号表条目中。

在代码生成过程中，每生成一条目标指令，变量的值与其存储位置之间的映射关系就可能发生改变。为了确保后续指令能够正确、高效地访问操作数，代码生成器必须动态地维护寄存器描述符和地址描述符，使其始终准确反映最新状态。考虑了寄存器描述符和地址描述符维护的基本块代码生成算法如算法 9.2。

算法 9.2 基本块代码生成算法 2

输入：基本块的三地址语句序列

输出：目标机代码序列

1. 初始化寄存器描述符和地址描述符
2. for 基本块中的每条三地址语句 $I : x = y \text{ op } z$, do
3. 根据 I 的类型选择目标代码模板
4. 调用函数 `getReg(I)` 为 x 、 y 、 z 选择寄存器，
 分别记为 R_x 、 R_y 、 R_z
5. 生成加载指令（如需要）
6. 更新寄存器描述符和地址描述符（如需要）

6. 生成运算指令
7. 更新寄存器描述符和地址描述符
8. end for
9. 在基本块末尾生成必要的保存指令（如需要）
10. 更新寄存器描述符和地址描述符（如需要）

9.4.3 描述符的维护

维护寄存器描述符和地址描述符遵循着一套严谨的规则。下面我们将详细阐述针对加载指令和运算指令的描述符维护规则，并通过示例展示其具体操作过程。

（1）加载指令的维护

对于加载指令“LD R, x ”，需对描述符做如下更新。

- ① 修改 R 的描述符，使其只包含 x
- ② 修改 x 的地址描述符，把 R 作为新增位置加入到 x 的位置集合中
- ③ 从所有不同于 x 的其他变量的地址描述符中删除 R

例 9.6：假设执行指令前描述符状态为：

寄存器描述符：R: { y, z }

地址描述符： x : { 内存 x }, y : { 内存 y, R }, z : { 内存 z, R }

执行“LD R, x ”后，描述符更新为：

寄存器描述符：R: { x } // R 现在只存放 x 的值

地址描述符：

x : { 内存 x, R } // x 新增寄存器位置

y : { 内存 y } // y 移除寄存器位置 R

z : { 内存 z } // z 移除寄存器位置 R

（2）运算指令的维护

对于运算指令“OP R_x, R_y, R_z ”，需对描述符做如下更新。

- ① 修改 R_x 的描述符，使其只包含结果变量 x
- ② 从所有其他寄存器的描述符中删除 x
- ③ 修改 x 的地址描述符，使其只包含位置 R_x
- ④ 从所有其他变量的地址描述符中删除 R_x

例 9.7 假设执行指令“OP R_x, R_y, R_z ”前描述符状态为：

寄存器描述符： R_x : { i }, R_y : { b }, R_z : { c }, R_u : { a, m, n }

地址描述符: $a: \{ \text{内存 } a \}, b: \{ \text{内存 } b, R_y \}, c: \{ \text{内存 } c, R_z \}, i: \{ \text{内存 } i, R_x \}, m: \{ \text{内存 } m, R_u \}, n: \{ \text{内存 } n, R_u \}$

执行运算指令后, 描述符更新为:

寄存器描述符:

$R_x: \{ a \}$ // R_x 现在只存放运算结果 a

$R_y: \{ b \}, R_z: \{ c \},$ // 寄存器不变

$R_u: \{ m, n \}$ // 移除 a 的值, a 值已发生变化内存值失

效

地址描述符:

$a: \{ R_x \}$ // x 现在只存在于 R_x 中, 其内存位置的值已失效

$i: \{ \text{内存 } i \}$ // i 从 R_x 中移除

b, c, m, n 的描述符保持不变

(3) 存储指令的维护

对于存储指令 “ST x, R ” 需对描述符做如下更新:

修改 x 的地址描述符, 使其必定包含自己的内存位置。

例 9.8 假设执行指令前描述符状态为:

寄存器描述符: $R: \{ x \}$

地址描述符: $x: \{ R \}$ // x 的值仅在寄存器 R 中, 内存中的值已过期

执行 “ST x, R ” 后描述符状态更新为:

寄存器描述符: $R: \{ x \}$ // 保持不变, R 中的值仍然是 x 的最新值

地址描述符: $x: \{ \text{内存 } x, R \}$ // x 的地址描述符包含了其内存位置,
// 表明内存中已保存了最新值。

(4) 复制语句 “ $x = y$ ” 的维护

对于复制语句 “ $x = y$ ”, 其目标代码的生成策略取决于变量 y 的当前值所在的位置。一个高效的实现是尽可能不生成实际的复制指令, 而是通过共享寄存器来完成赋值。此过程可能涉及一步或两步描述符更新来实现赋值。

当 y 的值不在任何寄存器中时, 需要先将其加载到寄存器 R_y , 生成加载指令 “LD R_y, y ”, 此时根据加载指令的维护规则, 描述符需做如下更新:

- ① 修改 R_y 的寄存器描述符, 使之只包含 y 。
- ② 修改 y 的地址描述符, 把 R_y 作为新增位置加入到 y 的位置集合中。
- ③ 从任何不同于 y 的变量的地址描述符中删除 R_y 。

当 y 的值已在寄存器 R_y 中时, 则通过共享寄存器来完成 $x = y$ 的赋值, 直接更新描述符:

① 修改 R_y 的寄存器描述符，使之同时包含 y 和 x 。这表示寄存器 R_y 现在存放着这两个变量的值。

② 修改 x 的地址描述符，使之只包含 R_y 。

③ 从任何不同于 R_y 的寄存器描述符中删除 x 。这确保了 x 的位置信息是唯一的和一致的。

例 9.9 假设执行复制语句 $x=y$ 前， y 已经在寄存器 R_y 中，当前描述符状态如下：

寄存器描述符： $R_y: \{y\}$

地址描述符： $x: \{\text{内存 } x\}, y: \{\text{内存 } y, R_y\}$

不生成赋值指令，而通过更新描述符建立 x 与 R_y 的关联，完成复制，更新后的描述符为：

寄存器描述符： $R_y: \{x, y\}$ // R_y 现在同时代表 x 和 y 的值。

地址描述符：

$x: \{R_y\}$ // x 的值现在只在 R_y 中，内存位置 x 值已失效

$y: \{\text{内存 } y, R_y\}$ // y 的描述符没有变化

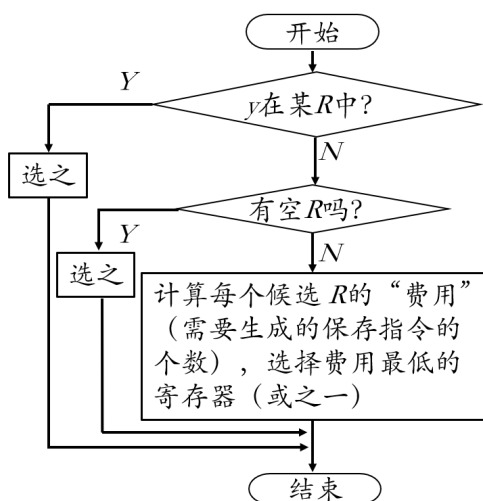
通过这种方式，我们无需生成 $x = y$ 的运算指令，仅通过更新描述符就完成了赋值，实现了高效的寄存器重用。后续对 x 的读写操作，只要其值仍在 R_y 中，就可以直接使用 R_y 。

9.4.4 寄存器选择函数

最后，来看看如何针对一个三地址指令 I 实现函数 $\text{getReg}(I)$ 。 $\text{getReg}(I)$ 函数是基本块代码生成算法的优化决策核心。它的职责不是简单地分配一个空闲寄存器，而是基于当前描述符的状态和变量的未来使用情况，为三地址指令中的每个操作数选择一个能够最小化目标代码中加载和保存指令数量的寄存器。以最典型的三地址指令 $x = y \text{ op } z$ 为例，详细拆解 getReg 如何为源操作数 y 和目标操作数 x 选择寄存器。

(1) 为源操作数 y 选择寄存器 R_y

为源操作数选择寄存器的目标是以最小的代价让变量 y 的值在一个寄存器中可用。其选择算法是一个基于优先级的决策过程，如图 9-1 和算法 9.3。

图 9-58 寄存器 R_y 选择算法**算法 9.3** 为源操作数 y 选择寄存器 R_y

1. 查询地址描述符，检查变量 y 的值是否已经存在于某个寄存器 R 中。
2. 若是，则直接选择 R 作为 R_y 。这是最优情况，无需生成加载指令。
3. 否则，寻找一个空闲寄存器（即其寄存器描述符为空）。
4. 若找到，则选择该空闲寄存器作为 R_y 。此情况需要生成一条“LD R_y, y ”指令。
5. 若未找到，必须选择一个寄存器进行溢出。此时，需要计算每个候选寄存器的溢出费用，并选择费用最低者，溢出费用计算如图 9-2 及算法 9.4。

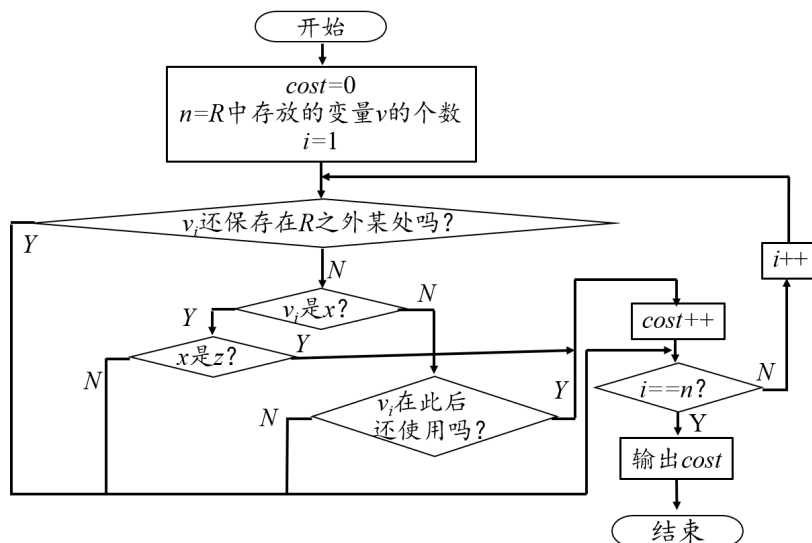


图 9-59 寄存器溢出费用计算

算法 9.4 计算寄存器 R 的溢出费用

1. 对于寄存器 R，其费用 $cost=0$ 。
2. for R 的寄存器描述符中记录的每一个变量 v ：
3. 如果 v 的地址描述符表明，其值除了在 R 中，还保存在其他地方（如内存或其他寄存器），则对 v 的溢出无需生成保存指令， $cost$ 不变。
4. 否则，如果 v 就是当前指令的 x 而不是 z （其值即将更新，无需保存代价）， $cost$ 不变，继续检查下一个 v 。
5. 否则，如果 v 是 x 也同时是 z ，那么， v 是操作数之一，则需生成一条保存指令， $cost++$ 。
6. 否则， v 不是当前指令的操作数，且其值仅存在于 R 中，则需要通过活跃变量分析判断 v 在基本块内后续是否还会被使用；
7. 如果是，则必须生成一条保存指令，因此 $cost++$ 。
8. 如果否，则 v 已成为死变量，可直接丢弃，无需保存， $cost$ 不变。
9. end for

例 9.10 费用计算。假设当前指令为 $x = y \text{ op } z$ ，寄存器状态为：R1: { a }, R2: { b, c }, R3: { d }。且已知： a 在内存有副本， b 仅在 R2 中且后续活跃， c 仅在 R2 中但后续不活跃， d 仅在 R3 中且后续活跃。那么选择 R_y 时，各寄存器的溢出费用如下：

$cost[R1] = 0$ ，因为 a 在内存有副本，溢出无需保存。

$cost[R2] = 1$ ，因为 b 需要保存， c 是死变量无需保存。

$cost[R3] = 1$ ，因为 d 需要保存。

此时，R1 是代价最低的选择，因此令 $R_y = R1$ 。

(2) 为目标操作数 x 选择寄存器 R_x

为目标操作数 x 选择寄存器的策略更为积极，其核心思想是尽可能地将计算结果保留在寄存器中，并为后续指令所重用。除了优先选择一个仅包含 x 的寄存器，另一个关键的优化机会是如果存放源操作数的寄存器在本次使用后不再活跃，那么它将是存放结果的最佳位置，最后就是使用与选择 R_y 相同的策略来选择 R_x 。其选择算法如算法 9.3 所示。

算法 9.5 为目标操作数 x 选择寄存器 R_x

1. 查询地址描述符，检查是否存在某个寄存器 R，其寄存器描述符仅包含变量 x 。

2. 若是，则选择 R 作为 R_x 。//即使 x 是 y 或 z （例如指令 $x = x + 1$ ），只要该寄存器只存放 x ，它就是可接受的。

3. 否则，检查为 y 选择的寄存器 R_y ，如果 y 在指令 I 之后不再被使用（通过活跃变量分析得知），并且 R_y 的寄存器描述符仅包含 y （即 R_y 不包含其他变量），那么 R_y 就是一个极佳的 R_x 候选。

4. 否则，类似地，检查为 z 选择的寄存器 R_z （如果 z 与 y 不同），条件同上。

5. 否则，采用与选择 R_y 相同的策略，调用算法 9.3（为源操作数选择寄存器的算法）来为一个全新的 R_x 分配寄存器。这可能选择一个空闲寄存器，或通过计算费用溢出一个已占用的寄存器。

（3）特殊情况：复制指令 $x = y$ 的优化处理

对于复制指令 $x = y$ ，`getReg` 会进行特殊优化，尽可能不生成实际的目标代码。首先，调用算法 9.2 为源操作数 y 选择一个寄存器 R_y 。然后，直接令 $R_x = R_y$ 。在后续的描述符更新中（如 9.4.3 节所述），将 R_y 的寄存器描述符增加 x ，并将 x 的地址描述符设置为 R_y 。这样，仅通过更新描述符就完成了赋值，实现了零指令的复制操作。

9.5 窥孔优化

9.5.1 窥孔与窥孔优化

在之前的小节中，我们阐述了如何通过精巧的指令选择与寄存器分配来生成高质量的目标代码。然而，编译器生成的初始代码中，仍然可能存在一些局部的、小范围的低效模式。这些低效模式可能源于中间代码生成的固定模板，也可能源于代码生成算法本身的局限性。为了消除这些低效性，一种名为窥孔优化的简单、有效且通用的技术被广泛应用。

窥孔(peephole)是程序上的一个小的滑动窗口。窥孔优化是指在优化的时候，检查目标指令的一个滑动窗口(即窥孔)，并且只要有可能就在窥孔内用更快或更短的指令来替换窗口中的指令序列。

窥孔优化具有以下显著优点：

（1）简单有效：优化器只需识别和匹配局部的小模式，实现简单，且效果立竿见影。

（2）广泛适用：它既可以应用于最终的目标代码，也可以在中间代码生

成后立即使用，以优化中间表示。

(3) 机器相关与无关：许多优化（如冗余删除）是机器无关的；同时，它也能方便地利用特定目标机器的高效指令。

9.5.2 典型的窥孔优化技术

窥孔优化的变换种类繁多，我们可以将其归纳为以下几个主要类别：

(1) 冗余指令删除

此类优化的目标是消除不必要的指令，减少指令数量和内存访问次数。

① 消除冗余的加载和保存指令

当一条加载指令从一个刚刚被存储指令写入的位置读取值时，该加载指令通常是冗余的。

例 9.11 考虑以下由三地址代码生成的目标代码序列

```
LD  R0, b      // (1) R0 = b
ADD R0, R0, c   // (2) R0 = R0 + c
ST  a, R0       // (3) a = R0
LD  R0, a       // (4) R0 = a (冗余!)
ADD R0, R0, e   // (5) R0 = R0 + e
ST  d, R0       // (6) d = R0
```

第(4)条指令“LD R0, a”紧跟在第(3)条指令“ST a, R0”之后，此时 *a* 的值已经在寄存器 R0 中。因此，第(4)条指令是冗余的，可以被安全删除。

值得注意的是，如果第(4)条指令前有一个标号（即它是某个跳转指令的目标），则通常不能删除，因为其他指令可能会跳转至该加载指令。

② 消除不可达代码

永远不会被执行到的代码是程序的“死代码”，应该被删除。

例 9.12 紧跟在无条件的跳转指令之后，且没有被任何标号标记的指令，是不可达的。如下的代码：

```
goto L2
print debugging information ; 该指令永远不会被执行
L2: ...
```

优化器可以删除 print 指令及其相关的代码。

(2) 控制流优化

这类优化旨在简化程序的控制流图，减少不必要的跳转指令。

例如，跳转至跳转的优化：当一条跳转指令的目标是另一条跳转指令时，在某些条件下可以进行简化。

```
    if a < b goto L1
    ...
    goto L3
L1: goto L2
可以优化为：
    if a < b goto L2    //直接跳转到最终目标
    ...
    goto L3
L1: goto L2            //该指令暂时保留
```

如果标号 L1 不再被其他指令引用（即它是“死标号”），并且“L1: goto L2”之前是一个无条件跳转，那么这条指令本身也变成了不可达代码，可以被删除。

（3）代数优化与强度削弱

这类优化利用代数定律或替换为代价更低的操作来简化计算。

① 代数恒等式

消除显而易见的恒等运算。例如， $x = x + 0$ 或 $x = x * 1$ 等指令可以直接被删除而不影响结果。

② 强度削弱

用速度更快或代价更低的指令替代昂贵的指令。对于整数运算，用移位操作替代乘数/除数为 2 的幂的乘法/除法。例如， $x = x * 8$ 可以替换为 $x = x \ll 3$ 。对于浮点数运算，在某些情况下，用乘一个常数的倒数来替代除以该常数（ $x = x / 2.0 \rightarrow x = x * 0.5$ ），因为乘法通常比除法更快。

（4）机器特有指令的使用

为了最大化性能，窥孔优化器应充分了解目标机器的指令集，并用其特有的高效指令替换低效的通用指令序列。例如，用自增指令 `INC Rx` 替代“`ADD Rx, Rx, #1`”，前者通常更短、更快。用块清零指令（如 `CLR`）来快速初始化一大段内存，而不是使用循环。利用复合寻址模式，将“加载-运算-保存”序列合并为一条指令。在某些架构上，还可以将“`LD R0, x; ADD R0, R0, #1; ST x, R0`”替换为一条 `INC x` 指令。

9.5.3 窥孔优化的实现方式

窥孔优化器通常使用一个模式匹配-替换的循环来实现。一个典型的窥孔优化器实现包含以下组件：

指令缓冲区 (buffer)：一个固定大小的队列或数组，用作“窥孔”，通常能容纳 N 条指令 (N 一般为 2 到 10)。这个缓冲区是优化的核心工作区。

优化规则集 (rules)：一个由 (pattern, replacement) 对组成的预定义列表。pattern 定义了要匹配的低效指令序列，replacement 定义了用于替换的高效指令序列。规则是机器相关与无关规则的混合。

输入/输出指令流：优化器从输入流读取原始指令，处理后的指令写入输出流。

窥孔优化器维护一个指令缓冲区（窥孔），并反复执行以下步骤：

(1) 若指令缓冲区指令条数小于 N ，且输入流非空，则从输入指令流中读取下一条指令，放入窥孔，直至缓冲区满。

(2) 将窥孔内的指令序列与预定义的优化规则一一进行匹配。

(3) 如果匹配成功，则用替换序列覆盖窥孔中的原始模式，并可能调整窥孔内容，若不足 N 条，从输入流中补充，或多于 N 条，溢出到输入流中。

(4) 直到如果没有任何规则匹配，则将窥孔中的第一条指令输出，并继续执行 (1)。

上述算法框架中有个关键点需要进一步阐明：

窗口大小 N 的选择至关重要。它必须至少与最长优化规则的模式长度相等。例如，要优化 `goto L1; ...; L1: goto L2` 这样的序列， N 必须足够大以容纳第一条 `goto` 和 `L1` 处的跳转指令。

规则集通常需要按特定顺序组织。更特殊、更具体的规则应放在更通用规则的前面。

例如：规则 “`LD R, m; ST m, R →` ”（空，即删除）应该比 “`ST m, R; LD R, m → ST m, R`” 具有更高优先级，因为前者是完全冗余。

模式匹配的复杂性：

通配符：模式应支持通配符，以匹配任意操作码或操作数。例如，模式 `(ST, m, R); (LD, R, m)` 中的 m 和 R 可以是通配的，只要它们两次匹配到同一个内存地址和寄存器。

标号处理：匹配时必须考虑标号。一条带有标号的指令通常不能被简单地删除或作为冗余指令的一部分，除非该标号再无其他引用。优化器需要维护基

本的标号引用信息。

通过这种方式，窥孔优化器能够以一种轻量级的方法，显著地提升最终生成代码的质量，是编译过程中一个非常经济且高效的收尾步骤。

9.6 本章小节

本章阐述了目标代码生成的核心技术。其任务是将优化的中间表示高效映射到目标机器指令集，质量直接决定程序性能。生成器主要完成三项关键工作：指令选择（选取语义等价且高效的机器指令）、寄存器分配与指派（在有限寄存器中优化存放高频值以减少内存访问）以及指令排序（调整指令顺序以适配硬件流水线）。本章引入一个简化的目标机模型作为讨论基础，并概述了常见三地址语句及过程调用的代码生成方法。重点在于基本块内的代码生成与寄存器分配，通过动态维护寄存器与地址描述符，并利用 `getReg` 函数智能选择寄存器，能够有效重用寄存器、消除冗余访存。最后，介绍了窥孔优化技术，它作为轻量的后处理步骤，通过识别和替换目标代码中的局部低效模式，进一步优化代码。本章内容为理解与实现高效代码生成器的核心流程与关键技术奠定了基础。

参考文献

- 1 AHO A, ect. 编译原理[M]. 李建中, 姜守旭, 译. 北京: 机械工业出版社, 2003.
- 2 AHO A, ect. 编译原理[M]. 2版. 赵建华等译. 北京: 机械工业出版社, 2009
- 3 AHO A, ect. Compilers: Principles, Techniques and Tools [M]. 英文版, 北京: 人民邮电出版社, 2002.
- 4 AHO A, ect. Compilers: Principles, Techniques and Tools [M] Second Edition. 英文版, 北京: 人民邮电出版社, 2008.
- 5 肖军模. 程序设计语言编译方法[M]. 第3版. 大连: 大连理工大学出版社, 2000.
- 6 陈意云,张昱.编译原理[M]. 第3版. 北京: 高等教育出版社, 2014.
- 7 陈意云,张昱.编译原理习题精选与解析[M]. 第3版. 北京:高等教育出版社, 2014
- 8 LESK M E, SCHMIDT E E. Lex: a Lexical Analyzer Generator[J]. Computing Science Technical Report 39, AT&T Bell Laboratories, Murray Hill, New Jersey, 1975.
- 9 JOHNSON S C. Yacc: Yet Another Compiler - Compiler[J]. Computing Science Technical Report 32, AT&T Bell Laboratories, Murray Hill, New Jersey,1975.
- 10 HOPCROFT J E, ect. Introduction to Automata Theory, Languages, and Computation[M].2nd ed. 英文版. 北京:清华大学出版社, 2001
- 11 现代编译原理: C语言描述. 修订版, 作者: [美] Andrew W.Appel/ [美] MaiaGinsburg, 译者: 赵克佳、黄春、沈志宇, 出版社: 人民邮电出版社, 2018-4
- 12 蒋宗礼,姜守旭, 编译原理, 高等教育出版社(第二版), 2017
- 13 许畅, 冯洋, 郑艳伟, 陈鄞. 计算机领域101计划核心教材 编译方法技术与实践, 机械工业出版社. 2024